Aplikace mikroprocesorů

Učební text — ČVUT FEL

Stanislav Vítek

2026

Obsah

Od klopného obvodu k procesoru **1**
1.1 Číslicové obvody, hodiny a časování 1
1.2 Od automatu k datové cestě 7
1.3 Mikroprogramový řadič a vykonávání instrukcí 18
1.4 ARM Cortex-M: konkrétní procesor 26
1.5 Kontrolní otázky 32

Čas, hodiny a periferie **34**
2.1 Zdroje času a hodinový systém 34
2.2 Periferie a GPIO 40
2.3 Přerušovací systém 44
2.4 Časovače a PWM 50
2.5 Syntéza: čas drží systém pohromadě 55
2.6 Kontrolní otázky 56

Od C k běžícímu obrazu **58**
3.1 Programování vestavných zařízení 58
3.2 Preprocesor: od konstant po složité konstrukce 60
3.3 Kompilátor: překlad a optimalizace 63
3.4 Volací konvence (AAPCS) 70
3.5 Členění kódu: moduly, knihovny a ovladače 72
3.6 Limity vestavného prostředí 76
3.7 MISRA-C: pravidla, vynucování a důsledky 79
3.8 Ladění a vhled do běžícího čipu 81
3.9 Závěr — od zdroje k běžícímu obrazu a zpět 83
3.10 Kontrolní otázky 83

Paměť a systémové sběrnice **86**
4.1 Paměťová mapa: jeden adresový prostor pro všechno 86
4.2 Linker a umístění v paměti 89
4.3 Memory-mapped I/O: periferie jako adresy 91
4.4 AMBA a bus matrix: jak je čip propojen 93
4.5 Arbitráž, wait states, paměťová zeď a cache 95
4.6 Dynamická alokace 96
4.7 Závěr — základ pro datový tok 97
4.8 Kontrolní otázky 98

Datový tok bez procesoru: DMA, převodníky a buffery **100**
5.1 Procesor jako úzké hrdlo přenosu dat 100
5.2 DMA: autonomní přesun dat 102
5.3 Analogové rozhraní: DAC a generování signálu 106
5.4 Analogové rozhraní: ADC 108
5.5 Správa bufferů a vlastnictví 112
5.6 Syntéza: streamovací ovladač 115
5.7 Kontrolní otázky 117

Přístrojové sběrnice **119**
6.1 Kritéria výběru sériové sběrnice . . . 119
6.2 On-board synchronní sběrnice: SPI a I2C . . . 120
6.3 On-board minimalistické a moderní: 1-Wire, I3C, I2S . . . 123
6.4 Field a síťové: diferenciál, vzdálenost, robustnost . . . 123
6.5 Zprávově orientované a aplikační: CAN, LIN, DALI . . . 125
6.6 Návrh sériového protokolu . . . 127
6.7 Syntéza: výběr sběrnice a její vlastnictví . . . 129
6.8 Kontrolní otázky . . . 130

Architektury procesorů a assembler **132**
7.1 ISA a mikroarchitektura . . . 132
7.2 RISC versus CISC: válka, která skončila smírem . . . 137
7.3 Architektura ARM: od skromných začátků k vašemu Macu . . . 140
7.4 Programování v ARM assembleru . . . 144
7.5 RISC-V: jiná sada kompromisů . . . 149
7.6 Syntéza: proč architektury koexistují . . . 150
7.7 Kontrolní otázky . . . 151

Multicore a specializovaný hardware **153**
8.1 Konec škálování jednoho jádra . . . 153
8.2 Paralelismus: principy a meze . . . 154
8.3 Multicore v embedded . . . 156
8.4 Specializovaný hardware . . . 158
8.5 AI akcelerátory (NPU) . . . 160
8.6 Syntéza: heterogenní SoC . . . 162
8.7 Kontrolní otázky . . . 164

RTOS: souběžnost, plánování a vlastnictví zdrojů **166**
9.1 Proč RTOS: souběžnost, determinismus, včasnost . . . 166
9.2 Úlohy a čas . . . 167
9.3 Plánovatelnost . . . 169
9.4 Architektury: od superloopu k preemptivnímu multitaskingu . . . 170
9.5 Synchronizace a komunikace . . . 172
9.6 Postavme si RTOS na Cortex-M . . . 175
9.7 Syntéza . . . 178
9.8 Kontrolní otázky . . . 180

Nízká spotřeba a spolehlivost **182**
10.1 Odkud se bere spotřeba . . . 182
10.2 Napájecí architektura STM32 . . . 184
10.3 Úsporné režimy . . . 186
10.4 Probouzení a energetický kompromis . . . 188
10.5 Návrhové vzory nízké spotřeby . . . 190
10.6 Spolehlivost: supervize a watchdog . . . 192
10.7 RTOS v low-power . . . 193
10.8 Syntéza . . . 195
10.9 Kontrolní otázky . . . 196

Číslicové zpracování signálů na MCU **198**
11.1 Proč DSP na MCU . . . 198
11.2 Číselné formáty . . . 199
11.3 FPU a DSP instrukce . . . 201
11.4 Číslicová filtrace . . . 203
11.5 FFT a spektrální analýza . . . 204

11.6 Matematické akcelerátory (STM32G4) . . . 207
11.7 Číslicový PID regulátor . . . 209
11.8 Syntéza . . . 211
11.9 Kontrolní otázky . . . 212

Strojové učení na embedded (TinyML) 214
12.1 Proč ML na embedded . . . 214
12.2 Inference versus trénink . . . 215
12.3 Workflow TinyML . . . 216
12.4 Kvantizace . . . 217
12.5 Inference na MCU . . . 219
12.6 Případové studie . . . 221
12.7 Syntéza . . . 224
12.8 Kontrolní otázky . . . 225

Bezpečnost embedded zařízení 227
13.1 Bezpečnostní model a attack surface . . . 227
13.2 Anatomie útoku . . . 228
13.3 Paměťová izolace: MPU . . . 229
13.4 ROP a obrana v kódu . . . 231
13.5 Chain of trust a izolace . . . 233
13.6 Hardwarové útoky . . . 234
13.7 Syntéza . . . 235
13.8 Kontrolní otázky . . . 236

Příloha A — Vlastnictví zdrojů a moderní C++ v embedded 238
A.1 Od object-based C k C++ . . . 238
A.2 RAII: životnost objektu je životnost zdroje . . . 238
A.3 Unikátní vlastnictví a přesun . . . 239
A.4 Výpůjčka bez vlastnictví . . . 239
A.5 Sdílené vlastnictví . . . 240
A.6 Pluggable alokátory: std::pmr . . . 240
A.7 Rust jako kontrast: vlastnictví vynucené překladačem . . . 241
A.8 Cena C++ v embedded: co ano a co s rozmyslem . . . 241
A.9 Syntéza: uzavření ownership-spirály . . . 241

Příloha B — Základy strojového učení 242
B.1 Klasifikační úloha . . . 242
B.2 Příznaky: co dáme modelu na vstup . . . 243
B.3 Perceptron: lineární klasifikátor . . . 243
B.4 Vícevrstvá síť a nelinearita . . . 245
B.5 Jak se síť učí: ztráta a gradient . . . 245
B.6 Backpropagation . . . 245
B.7 Optimalizátory: od SGD k Adamovi . . . 246
B.8 Trénink, validace a přeučení . . . 246
B.9 Od tréninku k nasazení . . . 246

Příloha C — Návrh sériového protokolu a marshalling 248
C.1 Vrstvy: od sběrnice k aplikaci . . . 248
C.2 Rámcování a synchronizace . . . 249
C.3 Integrita: od checksumu ke CRC . . . 250
C.4 Spolehlivost: potvrzování a opakování . . . 250
C.5 Marshalling: serializace dat . . . 251
C.6 Bezpečná deserializace . . . 252
C.7 Robustní protokoly v praxi . . . 253

C.8 Návrhový checklist 254

Příloha D — Ladění a implementace PID regulátoru **255**
D.1 Co dělají členy P, I a D 255
D.2 Diskretizace: poziční a přírůstkový tvar 255
D.3 Anti-windup 256
D.4 Derivace a žádaná hodnota 257
D.5 Implementace v pevné řádové čárce 257
D.6 Ladění regulátoru 258
D.7 Praktická rozšíření a checklist 258

Příloha E — AI nástroje pro embedded vývoj **260**
E.1 Dvě roviny AI v embedded vývoji 260
E.2 LLM asistenti pro psaní firmwaru 261
E.3 Datasheety, registry a konfigurace 262
E.4 Kontrola kódu, testy a statická analýza 262
E.5 Ladění: analýza logů, pádů a trace 262
E.6 Meze a rizika v bezpečnostně kritickém kódu 263
E.7 Nasazení modelů na MCU: toolchainy 264
E.8 Od dat k čipu: typický pipeline 264
E.9 Doporučení a checklist 265

1

Od klopného obvodu k procesoru

> Tato kapitola ukazuje, že mezi **číslicovým obvodem** a **mikroprocesorem** není propast: procesor je jen velký synchronní sekvenční obvod. Složíme ho od klopného obvodu přes konečný automat a řadič — a přes mikroprogram a instrukční sadu — až po konkrétní jádro **ARM Cortex-M**.

Obsah kapitoly

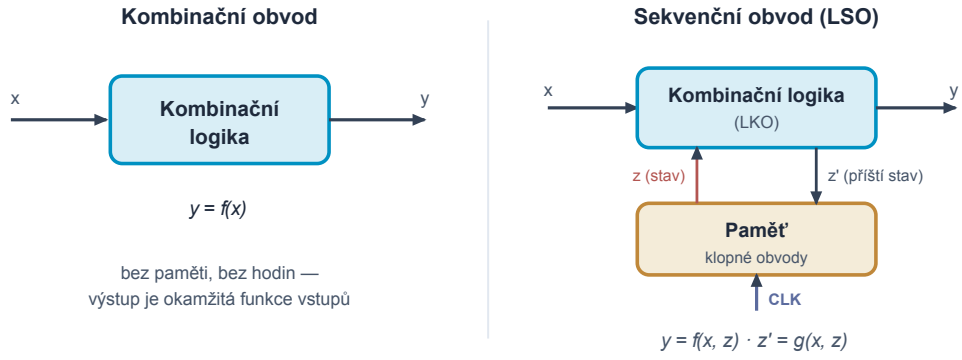
1.1 Číslicové obvody, hodiny a časování
1.2 Od automatu k datové cestě
1.3 Mikroprogramový řadič a vykonávání instrukcí
1.4 ARM Cortex-M: konkrétní procesor
1.5 Kontrolní otázky

1.1 Číslicové obvody, hodiny a časování

V úvodní části první kapitoly rozdělíme digitální obvody na kombinační a sekvenční, seznámíme se s principem synchronního návrhu a jeho realizací klopným obvodem, a nakonec i s důsledky časování (maximální frekvence) a jeho možného selhání (metastabilita). Tyto stavební kameny pak v následující části složíme do automatu, řadiče a nakonec procesoru.

Kombinační a sekvenční obvody

Číslicové obvody dělíme podle přítomnosti **paměti** na dvě třídy, kombinační a sekvenční. Zatímco **logický kombinační obvod (LKO)** je samostatný blok logiky, **logický sekvenční obvod (LSO)** je blok logiky doplněný o **paměť ve zpětné vazbě**. Obě třídy jsou znázorněny na obr. 1.1.



Obrázek 1.1. Kombinační obvod vs. sekvenční obvod

Kombinační obvod má výstup y, který je funkcí *výhradně okamžitých hodnot vstupů* x_i:

Kombinační obvod
Výstup je funkcí pouze okamžitých vstupů; obvod nemá paměť.

$$y = f(x_1, x_2, \dots, x_n).$$

Na předchozích hodnotách vstupů nezáleží - po ustálení vstupů se po průchozím zpoždění hradel ustálí i výstup. Do této třídy patří hradla, multiplexory, dekodéry, sčítačky i celá **aritmeticko-logická jednotka (ALU)**. Obvod je asynchronní a sám o sobě neuchovává žádnou informaci.

> **Příklad LKO.** Jednobitová sčítačka určí ze dvou operandů a vstupního přenosu součet a výstupní přenos; multiplexor 4→1 vybere podle dvoubitové adresy jeden ze čtyř vstupů. V obou případech jde o determinismus bez paměti — ani sčítačka, ani multiplexor nemá vnitřní stav, který by se dal využít při následujícím výpočtu.

U **sekvenčního obvodu** je **výstup** y funkcí vstupu x **i vnitřního stavu** z; ten se s aktivní hranou hodin aktualizuje na *příští stav* z':

Sekvenční obvod
Výstup je funkcí vstupu i vnitřního stavu uloženého v paměti.

$$y = f(x, z), \qquad z' = g(x, z).$$

Příští stav z' se uloží do paměti a na následující aktivní hraně se stává aktuálním stavem z — odtud **zpětná vazba**. Hodiny `CLK` přitom určují jen *okamžik* aktualizace, nikoli její hodnotu.

> **Příklad LSO. Čítač modulo** n (nebo také čítač s dělením frekvence n) počítá příchozí hodinové pulzy a po dosažení n stavů se cyklicky vrací na nulu. K tomu musí uchovávat **vnitřní stav** — kolik pulzů už napočítal. Týž hodinový pulz tak vede pokaždé do jiného stavu; závislost na stavu, nikoli pouze na okamžitém vstupu, je podstatou sekvenčního chování.

Hodiny a synchronní návrh

Už víme, že příští stav se v LSO ukládá do paměti na aktivní hraně hodin. Proč se ale stav mění jen na hraně, a ne **kdykoli**? Uvažme obvod **bez hodin** (asynchronní): jakákoli změna vstupu se začne okamžitě šířit logikou dál. Jenže různé cesty mají různá zpoždění — signál projde jedním hradlem za 1 ns, jiným řetězcem za 10 ns. Dočasně tak vznikají nesmyslné mezistavy (**hazardy**) a pokud na sebe dvě cesty „závodí" o to, která dorazí dřív (**race condition**), může obvod skončit v nesprávném stavu. Takový návrh se extrémně obtížně ladí, protože chování závisí na přesných zpožděních konkrétního kusu křemíku a na teplotě.

Synchronní návrh tento problém řeší jediným pravidlem:

Synchronní návrh
Stav se mění výhradně na aktivní hraně společného hodinového signálu.

> Stav se mění **pouze v okamžiku aktivní hrany společného hodinového signálu**.

Mezi dvěma akivními hranami hodinového signálu se nechají všechny kombinační signály v klidu ustálit; teprve následující hrana je zachytí do paměťových prvků. Hodiny tak tvoří **společnou časovou referenci**, podle níž se celý obvod řídí. Důsledky jsou dva, oba zásadní:

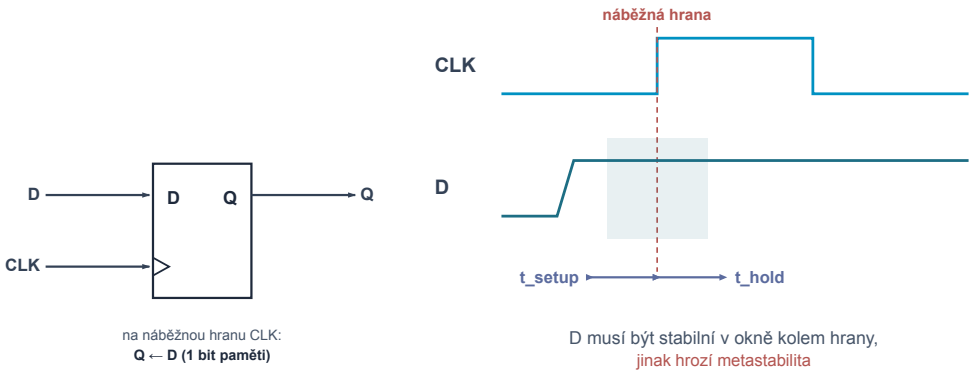
- **Determinismus.** Nezáleží na tom, *jak rychle* se která kombinační cesta ustálí, pokud se ustálí *včas před další hranou*. Výsledek je pak vždy stejný a předvídatelný.
- **Skládatelnost.** Bloky navržené samostatně lze spojit, sdílejí-li hodiny a dodržují časování. Právě díky tomu lze vůbec navrhnout něco tak složitého jako procesor — po částech, které se chovají jako „černé skříňky" reagující na hodinovou hranu.

Prakticky každý dnešní procesor je synchronní. Odtud plynou dvě otázky: **čím se „zachycení na hraně" realizuje** a **jakou frekvenci smí mít hodinový signál**.

Klopný obvod D, časování a maximální frekvence

Pravidlo, že se stav mění jen na aktivní hraně hodin, realizuje například **klopný obvod typu D** (D flip-flop). Jeho chování je prosté: na aktivní (typicky náběžnou) hranu hodin `CLK` přepíše hodnotu vstupu `D` na výstup `Q` a tam ji drží až do další aktivní hrany hodin (viz obr. 1.2).

Klopný obvod typu D
Paměťová buňka: na hraně hodin zachytí vstup D na výstup Q a tam jej drží.



(a) Klopný obvod typu D (b) Časování: okno setup/hold kolem hodinové hrany

Obrázek 1.2. Klopný obvod D a jeho časování.

Jeden klopný obvod uchová jeden bit informace. Registry, se kterými pracujete v procesoru, jsou skupiny takových buněk: 32bitový registr je 32 paralelních paměťových buněk sdílejících společný hodinový signál. Registrový soubor ARM (16 32bitových registrů) je tedy $16 \times 32 = 512$ paměťových buněk plus logika pro výběr aktivního registru.

Setup a hold

Klopný obvod není ideální — aby spolehlivě zachytil správnou hodnotu, musí být vstup `D` **stabilní v úzkém okně kolem hodinové hrany** (**setup/hold**). Toto okno popisují dva parametry:

Setup/hold
Okno kolem hodinové hrany, v němž musí být vstup ‘D‘ ustálený (setup před hranou, hold po ní).

- **doba předstihu `t_setup`** — jak dlouho *před* hranou musí být `D` ustálené,
- **doba přesahu `t_hold`** — jak dlouho *po* hraně musí `D` ještě vydržet. Jsou-li `t_setup` a `t_hold` dodrženy, klopný obvod po čase `t_cq` (clock-to-Q — zpoždění od hodinové hrany k ustálení výstupu `Q`) vydá platnou hodnotu. Tyto tři parametry plně charakterizují časování klopného obvodu a vedou na dvě různá omezení: `t_cq` a `t_setup` přes nejdelší cestu určují **maximální hodinovou frekvenci**, `t_hold` přes nejkratší cestu klade **spodní mez na zpoždění logiky** (nezávisle na frekvenci). Poruší-li se okno naopak, hrozí metastabilita.

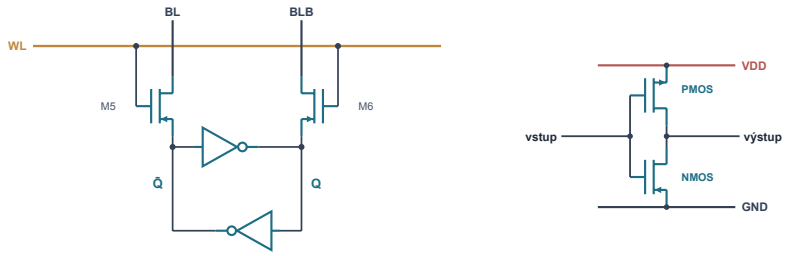
Jak vypadá jeden bit v křemíku

6T buňka
Statická paměťová buňka ze šesti tranzistorů: dva křížově vázané invertory (čtyři tranzistory) a dva přístupové tranzistory.

Vodič výběru buňky
Vodič výběru buňky (angl. word line, WL): aktivuje ho dekodér z adresy řádku a přes přístupové tranzistory připojí buňku na bitové vodiče. Log. 1 buňku vybere pro čtení či zápis, log. 0 ji ponechá držet svůj stav.

Rovnice „registr = skupina klopných obvodů” je správná jako *idea*, ale v **register file** (banka mnoha registrů s víceportovým přístupem) se jeden bit obvykle realizuje úsporněji než plným klopným obvodem. Staví na kompaktní **buňce podobné SRAM**: na dvou křížově vázaných invertorech, které se navzájem drží v jednom ze dvou stabilních stavů, a dvou **přístupových tranzistorech**, jimiž se buňka připojí na bitové vodiče (obr. 1.3). Odtud název **6T buňka** — šest tranzistorů na jeden bit.

Řídicím signálem buňky je **vodič výběru buňky** `WL` (angl. *word line*): dokud je v log. 0, přístupové tranzistory jsou rozepnuté a buňka pouze drží svůj stav; log. 1 je sepne a propojí vnitřní uzly `Q`/$\bar{\text{Q}}$ s **bitovými vodiči** `BL`/`BLB`, po nichž se hodnota přečte nebo přepíše. `WL` tedy



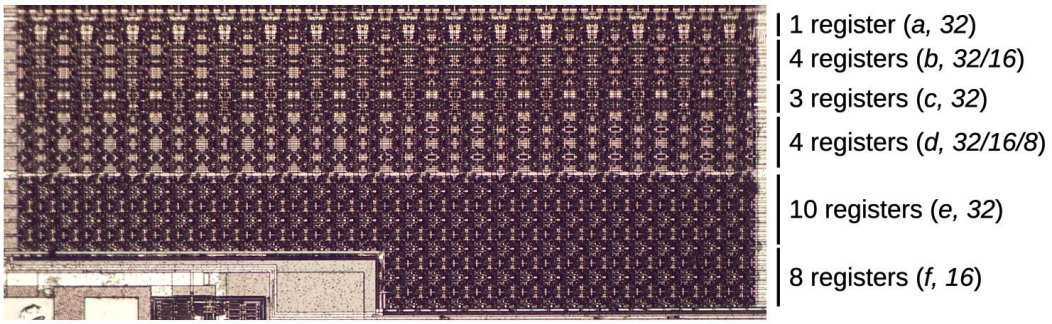
(a) 6T buňka (b) Zapojení invertoru

Obrázek 1.3. Statická paměťová buňka 6T a její stavební invertor.

buňku *vybírá* (řídicí signál), `BL`/`BLB` nesou *data* — totéž rozdělení na řídicí a datové vodiče uvidíme ve velkém u sběrnic v oddílu 1.4.

Kolik tranzistorů je tedy potřeba pro realizaci 32bitového registru v register file? Na samotné uložení dat je při šesti tranzistorech na bit potřeba zhruba **192 tranzistorů** a k nim ještě dekodéry adres, multiplexery, budiče bitových vodičů a čtecí zesilovače. Ve schématu sice registr vystupuje jako jediný blok, ve skutečnosti je to samostatný malý obvod. Uvedený rozbor platí pro **register file** s víceportovým přístupem; jednotlivé pipeline a řídicí registry procesoru se realizují běžnými klopnými obvody popsanými výše.

Že jde o fyzicky pravidelnou strukturu, je nejlépe vidět na skutečném křemíku. Na snímku register file procesoru Intel 386 (obr. 1.4) tvoří paměťové buňky pravidelnou **matici bit-slice**: každý svislý sloupec je jedna bitová buňka, vodorovné pásy jsou jednotlivé registry a 32bitový registr je doslova 32 stejných buněk vedle sebe.



Obrázek 1.4. Register file procesoru Intel 386 pod mikroskopem — pravidelná matice bitových buněk (svislé sloupce = bity, vodorovné pásy = jednotlivé registry). Foto: Ken Shirriff, righto.com.

Kritická cesta a maximální frekvence

Mezi dvěma registry v synchronním obvodu leží kombinační logika. Aby data dorazila na vstup cílového registru *včas* — tedy s předstihem `t_setup` před příští aktivní hranou hodin — musí perioda hodin pokrýt celý řetězec zpoždění na cestě:

$$T_{clk} \geq t_{cq} + t_{logika} + t_{setup},$$

kde `t_cq` a `t_setup` jsou parametry klopného obvodu (zpoždění zdrojového registru, resp. rezerva na spolehlivé zachycení v cílovém) a `t_logika` je doba průchodu dat kombinační logikou. Nejpomalejší taková cesta v celém obvodu se nazývá **kritická cesta** a určuje maximální hodinovou frekvenci

Kritická cesta
Nejpomalejší kombinační cesta mezi registry; určuje maximální hodinovou frekvenci.

$$f_{max} = \frac{1}{T_{clk}}.$$

Zrychlit obvod tedy znamená buď **zkrátit kritickou cestu** — vložit registry doprostřed dlouhé logiky a rozdělit práci na více kratších taktů (to je *pipelining*, viz přednáška o architekturách) — nebo **použít rychlejší technologii** s menším zpožděním hradla.

Hold: druhé omezení, tentokrát na nejkratší cestě

Perioda hodin ošetřuje **nejdelší** cestu. Existuje ale ještě druhé, nezávislé omezení — tentokrát na **nejkratší** cestě. Jakmile hodinová hrana přepne zdrojový registr, začnou se nová data hrnout na vstup `D` cílového registru už po čase `t_cq`. Dorazí-li příliš *brzy*, přepíšou vstup dřív, než u téže hrany uplyne doba přesahu `t_hold` — a cílový registr místo staré hodnoty zachytí rovnou tu novou. Aby k tomu nedošlo, musí platit

$$t_{cq} + t_{logika}^{\min} \geq t_{hold},$$

kde $t_{logika}^{\min}$ je zpoždění **nejrychlejší** cesty logikou. Zásadní rozdíl oproti setupu: **hold se nedá spravit zpomalením hodin** — perioda hodin v nerovnosti vůbec nevystupuje. Porušený hold se řeší naopak *zpomalením* nejkratší cesty (vložením zpoždění), nebo se mu předejde už návrhem klopného obvodu s $t_{hold} \leq t_{cq}$; pak lze dva klopné obvody zapojit přímo za sebe bez logiky mezi nimi — čehož využívá i synchronizér.

Technologie určuje strop

Zpoždění jednoho hradla, a tím i dosažitelná frekvence, je dáno zejmnéa použitou technologií. Typické hodnoty zpoždění digitálních technologií a z toho vyplývající dosažitelné hodinové frekvence jsou shrnuty v tab. 1.1.

Tabulka 1.1. Limity digitálních technologií.

Technologie	zpoždění hradla	max. hodiny
TTL 74LS	~10 ns	~35 MHz
CMOS 74HC	~8 ns	~50 MHz
CMOS 74LVC/AC	~2–3 ns	~150 MHz
FPGA (fabric)	stovky ps	~0,3–0,7 GHz
ASIC 28 nm	desítky ps	~1–2 GHz
ASIC ≤ 5 nm	jednotky ps	~3–5 GHz

Mikrokontroléry, se kterými budeme na cvičeních pracovat, leží uprostřed spektra: STM32L4 běží až na 80 MHz, STM32U5 na 160 MHz, výkonné STM32H7 až na 550 MHz. Vyšší frekvence by znamenala kratší kritickou cestu (a tedy složitější návrh), vyšší spotřebu a větší potíže s rozvodem hodin po čipu — proto se u mikrokontrolérů frekvence obvykle dál nezvyšuje.

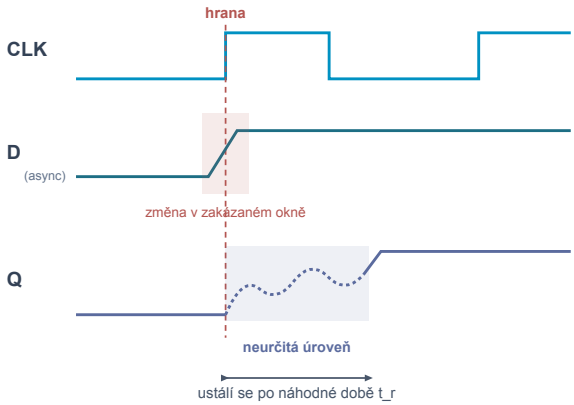
Metastabilita

Setup/hold okno lze v rámci jedné hodinové domény dodržet návrhem (krátká kritická cesta). Problém nastává u **asynchronních vstupů** — signálů, které se mění *bez vztahu k našim hodinám*: výstup čidla nebo data přicházející z **jiné hodinové domény** (jiný oscilátor, jiná frekvence). Takový signál může hodinovou hranu zastihnout přesně uprostřed přechodu a setup/hold okno porušit.

Co se stane

Klopný obvod je v jádru bistabilní prvek — má dva stabilní stavy (0 a 1) a mezi nimi **labilní rovnováhu**. Porušíme-li časování, může výstup uvíznout v této labilní poloze — vzniká **metastabilita**: na napětí někde mezi log. 0 a log. 1, nebo zakmitávat, a teprve po **náhodně dlouhé** době se „překlopí" do některého z platných stavů (obr. 1.5). Názorně je to jako kulička na vrcholku kopce mezi dvěma údolími: teoreticky tam může zůstat, ale sebemenší porucha ji nakonec skulí do jednoho z nich — a čím přesněji na vrcholku balancuje, tím déle se to zdrží.

Metastabilita
Dočasně neurčitý stav klopného obvodu po porušení setup/hold okna; ustálí se po náhodně dlouhé době.



Obrázek 1.5. Metastabilita: výstup uvízne v neurčité úrovni

Důsledky jsou závažné:

- po dobu metastability je výstup **neurčitý** — různé navazující obvody jej mohou interpretovat různě (jeden jako 0, druhý jako 1), což rozbije konzistenci stavu;
- doba ustálení je **náhodná**, takže porucha se projeví nepravidelně a vzácně — špatně se hledá a reprodukuje.

Důležité: metastabilitu **nelze fyzikálně zcela vyloučit**. Plyne z podstaty bistabilního prvku. Lze ji jen učinit *dostatečně nepravděpodobnou*.

Jak nepravděpodobnou? — MTBF

Pravděpodobnost selhání se kvantifikuje pomocí **střední doby mezi poruchami** (MTBF, Mean Time Between Failures). Pro klopný obvod, kterému na ustálení dáme čas t_r (resolution time), platí přibližně:

Střední doba mezi poruchami
MTBF: u metastability roste exponenciálně s časem vyhrazeným na ustálení.

$$\mathrm{MTBF}(t_r) = \frac{e^{\,t_r/\tau}}{T_0 \cdot f_{clk} \cdot f_{data}},$$

kde

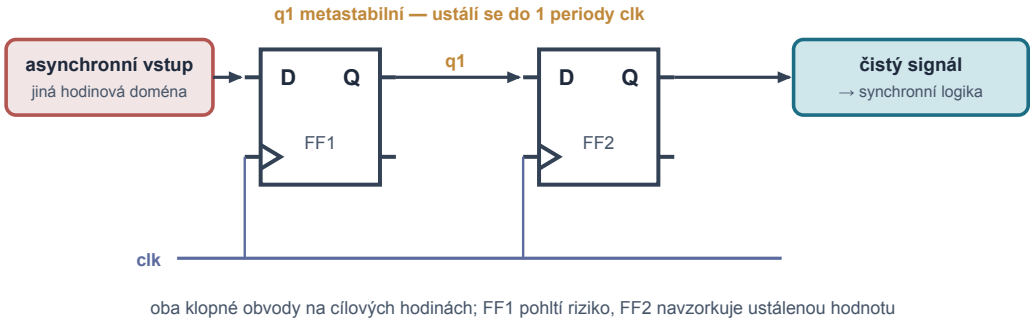
- τ je **časová konstanta metastability** dané technologie klopného obvodu (jak rychle se labilní stav rozpadá; u moderních procesů jednotky až desítky ps),
- T_0 je konstanta charakterizující šířku „nebezpečného okna",
- f_{clk} je frekvence vzorkujících hodin,
- f_{data} je četnost změn asynchronního vstupu.

Podstatné je, že MTBF roste **exponenciálně** s časem t_r vyhrazeným na ustálení — v tom spočívá celé řešení.

Řešení: synchronizér

Praktickým řešením je **dvoustupňový synchronizér**, naznačený na obr. 1.6. Synchronizér tvoří dva klopné obvody zapojené za sebou, připojené na stejný hodinový signál.

Dvoustupňový synchronizér
Dva klopné obvody za sebou, které omezují metastabilitu asynchronního vstupu.



Obrázek 1.6. Dvoustupňový synchronizér

První klopný obvod (FF1) navzorkuje asynchronní signál a má **celou jednu periodu hodin** na to, aby se ustálil, než jeho výstup navzorkuje druhý klopný obvod (FF2). Dosadíme-li $t_r \approx T_{clk}$ do vzorce výše, MTBF vzroste o mnoho řádů — z „jednou za vteřinu" až na „déle než stáří vesmíru". Proto v drtivé většině případů **dva stupně stačí**; pro extrémně rychlý hodinový signál nebo vysokou spolehlivost se přidává třetí.

Na co si dát pozor

- **Synchronizujte každý asynchronní vstup** dřív, než ho použijete v logice.
- Pro **více bitů najednou (sběrnice)** dvoustupňový synchronizér *nestačí* — jednotlivé bity se mohou ustálit v různých taktech a vznikne nekonzistentní slovo. Tam je potřeba **handshake** (potvrzovací protokol) nebo **FIFO** s oddělenými hodinami pro zápis a čtení.
- Mezi FF1 a FF2 nedávejte žádnou kombinační logiku — ukrojila by z času na ustálení.

Na STM32 mají vstupy GPIO synchronizéry zabudované (proto čtení pinu trvá několik hodinových cyklů, než se hodnota objeví ve vstupním registru), a přechody mezi hodinovými doménami (USB, RTC, …) řeší příslušné periferie interně. Jako programátoři se s metastabilitou setkáte přímo zřídka — podstatné je rozumět, *proč* tam synchronizéry jsou.

> **Shrnutí:** Procesor je synchronní sekvenční obvod. Hodiny zavádějí pravidlo, že se stav mění pouze na aktivní hraně, a tím činí návrh deterministickým a skládatelným. Toto pravidlo fyzicky realizuje klopný obvod typu D, jehož časování (setup/hold/t_cq) přes kritickou cestu určuje maximální frekvenci. Na rozhraní s nesynchronizovaným světem hrozí metastabilita, kterou omezují synchronizéry. V dalších oddílech přejdeme od jednotlivých registrů ke **konečným automatům** a jejich realizaci jako **řadiče** procesoru.

1.2 Od automatu k datové cestě

Z registrů a hradel teď složíme obvod, který se *rozhoduje*: **konečný automat**. Ukážeme jeho dvě podoby — Mooreův a Mealyho automat — i systematický návrh jeho realizace — řadiče. Spojením řadiče s **datovou cestou** (registry, ALU, paměť) pak dojdeme až ke vzniku **procesoru** a k otázce, zda vůbec potřebuje hodiny.

Řadiče
Automat generující řídicí signály datové cesty.

Konečný automat (Mooreův a Mealyho)

Registr z k klopných obvodů může nabývat 2^k různých hodnot, a obvod postavený kolem něj se tak vždy nachází v jednom z **konečně mnoha stavů** — právě odtud pochází název **konečný automat** (finite-state machine, FSM). Doplníme-li stavový registr o logiku, která ze současného stavu a vstupů určuje *příští stav* a *výstupy*, dostaneme obvod, jenž na každou aktivní hranu hodin přejde do stavu následujícího. Je to základní stavební prvek každého řadiče.

Konečný automat
Stavový registr + logika příštího stavu + výstupní logika (FSM).

Formálně je konečný automat popsán vztahem

$$M = (S,\ I,\ O,\ \delta,\ \lambda,\ s_0),$$

kde S je konečná množina **stavů**, I množina vstupních symbolů, O množina výstupních symbolů, $\delta : S \times I \to S$ **přechodová funkce** (next-state), λ **výstupní funkce** a $s_0 \in S$ počáteční stav. V hardwaru:

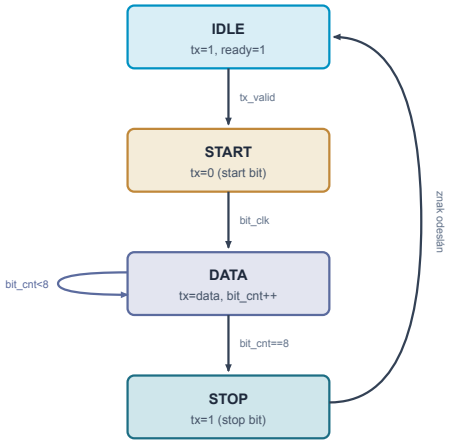
- stav S drží **stavový registr** (klopné obvody),
- přechodovou funkci δ realizuje **kombinační logika následujícího stavu**,
- výstupní funkci λ realizuje **kombinační výstupní logika**,
- přechod ze stavu do stavu nastává na **hraně hodin**.

Ve značení blokového schématu je přechodová funkce δ totéž co g (příští stav $z' = g(x, z)$) a výstupní funkce λ totéž co f (výstup $y = f(x, z)$). Dále v kapitole se držíme tohoto značení: **výstup** f**, příští stav** g**, stav** z**, vstup** x.

Příklad: řadič vysílače asynchronní sériové linky (UART)

Klasickým příkladem řadiče je vysílač **UART**. Jeho úkolem je přenos serializovaného datového proudu jediným datovým vodičem. Na obr. 1.7 jsou zakresleny čtyři stavy stavového automatu, který tento řadič popisuje.

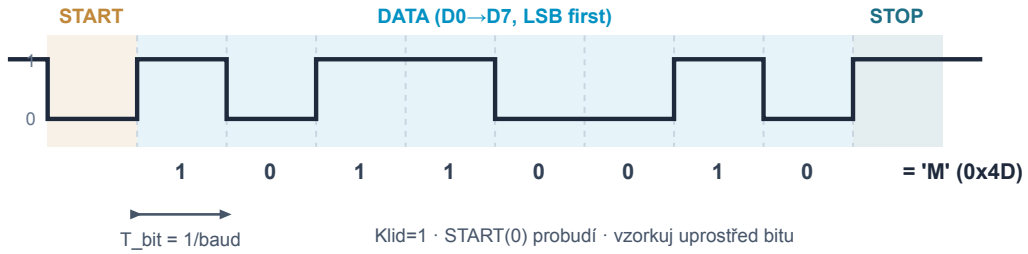
- **IDLE** — klid, linka drží log. 1, vysílač čeká na povel k vysílání;
- **START** — vyšle se *start bit* (log. 0), kterým se ohlásí začátek rámce;
- **DATA** — postupně se vysílá 8 datových bitů; ve stavu se setrvává (smyčka), dokud čítač bitů nedosáhne 8 — stav tedy může mít **podmíněný přechod sám na sebe**;
- **STOP** — vyšle se *stop bit* (log. 1), který oddělí rámce, a automat se vrací do IDLE.



Obrázek 1.7. Stavový automat vysílače UART.

Časový průběh signálu na lince ukazuje, co automat produkuje (obr. 1.8). UART je *asynchronní*, obě strany se musí předem dohodnout na **přenosové rychlosti**. Užitečná data jsou ohraničena dohodnutými značkami - **start** a **stop** bitem.

Linka je v klidu v log. 1. Sestupná hrana *start bitu* spustí a časově synchronizuje přijímač; ten pak vzorkuje **uprostřed** každého bitu, aby byl co nejodolnější vůči nepřesnosti hodin (tolerance bývá kolem ±2–3 %). Následuje 8 datových bitů (LSB první) a *stop bit*. Jednotlivé části rámce přesně odpovídají stavům automatu:



Obrázek 1.8. Rámec UART v čase

Část rámce	Stav	Linka
klid	IDLE	trvale 1
start bit	START	1 → 0 (spustí příjem)
8 datových bitů	DATA	data, LSB první
stop bit	STOP	1 (oddělí rámce)

Smyčka ve stavu DATA (`bit_cnt < 8`) je vestavěný **čítač bitů**.

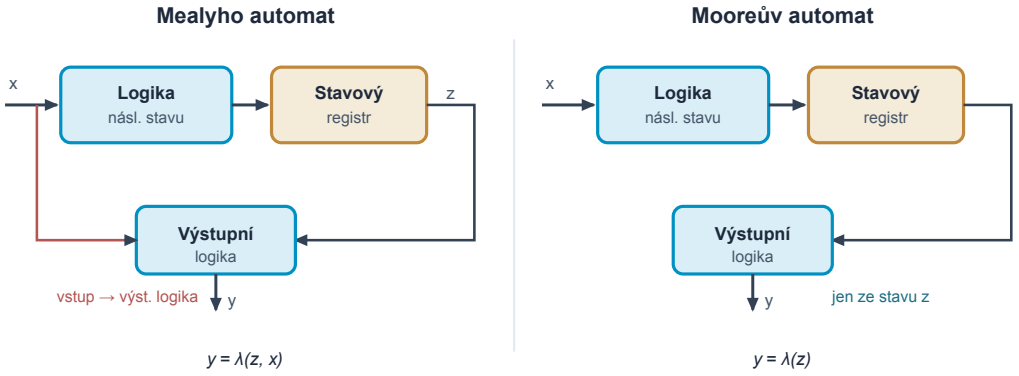
Mooreův versus Mealyho automat

Podle toho, na čem závisí výstup, rozlišujeme dva typy automatů:

- **Mooreův automat** — výstup závisí **pouze na stavu**: $y = f(z)$.
- **Mealyho automat** — výstup závisí **na stavu i na vstupu**: $y = f(z, x)$.

Mooreův automat
Automat, jehož výstup závisí pouze na stavu (Mealyho automat: na stavu i vstupu).

Rozdíl je dobře vidět na blokovém schématu (obr. 1.9). Obě varianty mají stejnou kostru — logiku následujícího stavu, stavový registr a výstupní logiku. Liší se jediným spojem: u **Mealyho** automatu vstupuje signál `x` **i do výstupní logiky**, u **Mooreova** automatu výstupní logika čte **jen stav** `z`:



Obrázek 1.9. Bloková struktura Mooreova a Mealyho automatu

Tento spoj způsobuje odlišné časování: protože výstup Mealyho automatu závisí přímo na vstupu, reaguje na jeho změnu **okamžitě** (ve stejném taktu), ale tím i kopíruje jeho případné zákmity. Výstup Mooreova automatu čte jen stav, takže se objeví **až po hraně hodin** — o takt později, zato **stabilní** po celý takt. Rozdíl v počtu stavů má jinou příčinu: Mealyho automat

může výstup vázat na přechody, ne jen na stavy, takže jeden stav zastane práci, kterou by Mooreův automat rozdělil do několika.

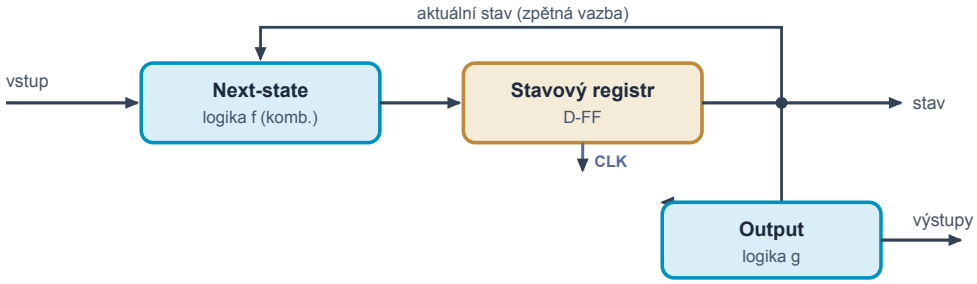
Tabulka 1.2. Praktické rozdíly mezi automaty.

Vlastnost	Mooreův	Mealyho
Výstup	$f(z)$	$f(z, x)$
Reakce na vstup	až v dalším taktu	v témže taktu (rychlejší)
Výstup v čase	stabilní (registrovaný)	může krátce zakmitat
Počet stavů	obvykle více	obvykle méně

Pro **řadič procesoru** zpravidla volíme **Mooreův automat**: řídicí signály (např. „zapiš do registru", „povol ALU") musí být **stabilní po celý takt**, jinak by se datová cesta chovala nepředvídatelně. Stabilita výstupu je tu důležitější než úspora jednoho taktu. Proto i v následujícím příkladu navrhneme Mooreův automat.

Syntéza a návrh obvodového řadiče

Převést konečný automat na hardware znamená naplnit obecné schéma konkrétní logikou (obr. 1.10):



Obrázek 1.10. Syntéza: stavový registr + logika následujícího stavu a výstupní logika

Tok je uzavřená smyčka: stavový registr drží aktuální stav; logika následujícího stavu z něj (a ze vstupů) spočítá příští stav, který se na hraně hodin zapíše zpět do registru; výstupní logika ze stavu odvodí řídicí signály. Celý postup syntézy se dá shrnout do několika kroků:

1. nakreslit **stavový diagram** (stavy a přechody),
2. sestavit **přechodovou tabulku** (úplný popis chování),
3. **zakódovat stavy** do bitů a odvodit **logické rovnice** pro příští stav a výstupy,
4. rovnice realizovat **hardwarem** (hradla, nebo paměť).

Kódování stavů

Stavy je nejprve nutno přiřadit konkrétní bitové kombinace. Volba kódování rozhoduje, kolik logiky pak bude potřeba — a **jednoduchý vzorec na to nejlepší neexistuje**: buď se volí *citem* (aby příbuzné stavy a výstupy sdílely bity), nebo možnosti prohledá návrhový nástroj (CAD). Tři běžné přístupy (ukázané na čtyřech stavech UART z předchozího oddílu):

Stav	Binární	One-hot	Grayův
IDLE	`00`	`0001`	`00`
START	`01`	`0010`	`01`
DATA	`10`	`0100`	`11`
STOP	`11`	`1000`	`10`
počet klopných obvodů	**2**	**4**	**2**

- **Binární** — minimum klopných obvodů ($\lceil \log_2 n \rceil$), ale složitější logika příštího stavu i dekódování stavu;
- **One-hot** — jeden klopný obvod na stav; **vymění klopné obvody za jednodušší logiku** — protože stav je „právě jeden aktivní bit", je logika příštího stavu i výstupů triviální a rychlá. Proto se hodí do **FPGA**, kde klopných obvodů bývá dost;
- **Grayův** — sousední stavy se liší v jediném bitu, což omezuje hazardy při přechodu.

Nejlepší volba tak závisí na konkrétním automatu — a na tom, čeho je v dané technologii škoda víc, zda hradel, nebo klopných obvodů. V následujícím příkladu zvolíme binární kódování, které vede na nejmenší stavový registr.

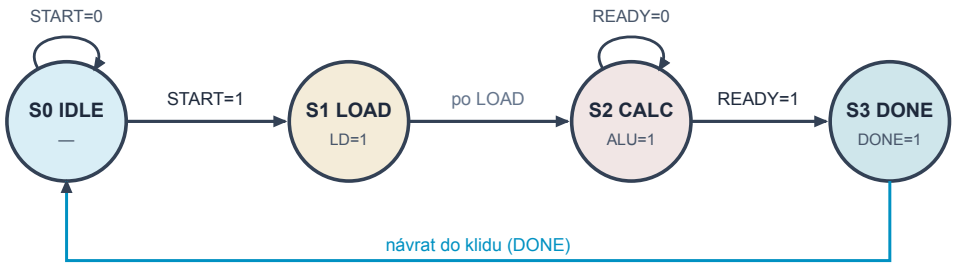
Návrh řadiče krok za krokem

Nyní celý postup projdeme na konkrétním zadání.

> **Zadání.** Navrhněte řadič, který na povel `START` provede jednu operaci datové cesty: počká v klidu na `START`, **načte** operand (signál `LD`), spustí **výpočet** v ALU (signál `ALU`) a počká na `READY`, a nakonec ohlásí **hotovo** (signál `DONE`) a vrátí se do klidu.
>
> Vstupy automatu: `START`, `READY`. Výstupy: `LD`, `ALU`, `DONE` (Mooreovy — závisejí jen na stavu).

Stavový diagram

Z popisu plynou čtyři stavy a dvě místa, kde automat **čeká na vstup** (obr. 1.11):



Obrázek 1.11. Stavový diagram řadiče

- **S0 IDLE** — dokud `START = 0`, automat zůstává (přechod sám na sebe); při `START = 1` přejde do S1;

- **S1 LOAD** — nastaví `LD = 1` a bezpodmínečně přejde do S2;
- **S2 CALC** — nastaví `ALU = 1` a **čeká**: dokud `READY = 0`, zůstává (čekací smyčka), při `READY = 1` přejde do S3;
- **S3 DONE** — nastaví `DONE = 1` a vrátí se do S0.

V diagramu nekreslíme hodiny — v synchronním obvodu jsou všudypřítomné a určují jen *kdy* přechod nastane; *který* přechod se stane, říká diagram podle vstupů na hranách. Obě čekací smyčky (na `START` v S0 a na `READY` v S2) jsou podmíněné přechody sám na sebe — přesně ten vzor, který jsme viděli u stavu DATA vysílače UART. Po zapnutí a při signálu reset musí automat začít v definovaném stavu — zde v **S0**; v hardwaru to zajistí klopné obvody s asynchronním resetem, které se do S0 nastaví bez ohledu na předchozí obsah.

Přechodová tabulka

Stavy zakódujeme binárně: `S0 = 00`, `S1 = 01`, `S2 = 10`, `S3 = 11`; stav je tedy dvojice bitů $Q_1 Q_0$. Tabulka je vlastně **pravdivostní tabulka** logiky příštího stavu i výstupů — pro každou kombinaci stavu a vstupů udává příští stav a výstupy. Právě proto z ní v dalším oddílu logické rovnice jen *odečteme*:

Stav	$Q_1 Q_0$	START	READY	Příští $Q_1' Q_0'$	LD	ALU	DONE
S0	00	0	–	00	0	0	0
S0	00	1	–	01	0	0	0
S1	01	–	–	10	1	0	0
S2	10	–	0	10	0	1	0
S2	10	–	1	11	0	1	0
S3	11	–	–	00	0	0	1

Pomlčka – značí *na hodnotě nezáleží* (don't care) — např. ve stavu S1 přejdeme do S2 bez ohledu na vstupy.

Odvození logických rovnic

Z tabulky odvodíme logické funkce pro každý bit příštího stavu a pro každý výstup. Postup je vždy stejný: vyhledáme řádky, kde má daný bit hodnotu 1, a výraz zjednodušíme.

Bit Q_1'. Hodnotu 1 nabývá v řádcích, kde je stav `01` (S1) nebo `10` (S2) — tedy právě tehdy, když se Q_1 a Q_0 **liší**. To je definice logické nonekvivalence:

$$\boxed{Q_1' = Q_1 \oplus Q_0}$$

Ověření: `00` → 0, `01` → 1, `10` → 1, `11` → 0 — souhlasí s tabulkou.

Bit Q_0'. Jedničku dává jen řádek `00` s `START = 1` a řádek `10` s `READY = 1`:

$$Q_0' = (\overline{Q_1}\,\overline{Q_0} \cdot START) + (Q_1\,\overline{Q_0} \cdot READY).$$

Oba členy obsahují $\overline{Q_0}$, který lze vytknout:

$$\boxed{Q_0' = \overline{Q_0}\,(\overline{Q_1} \cdot START + Q_1 \cdot READY)}$$

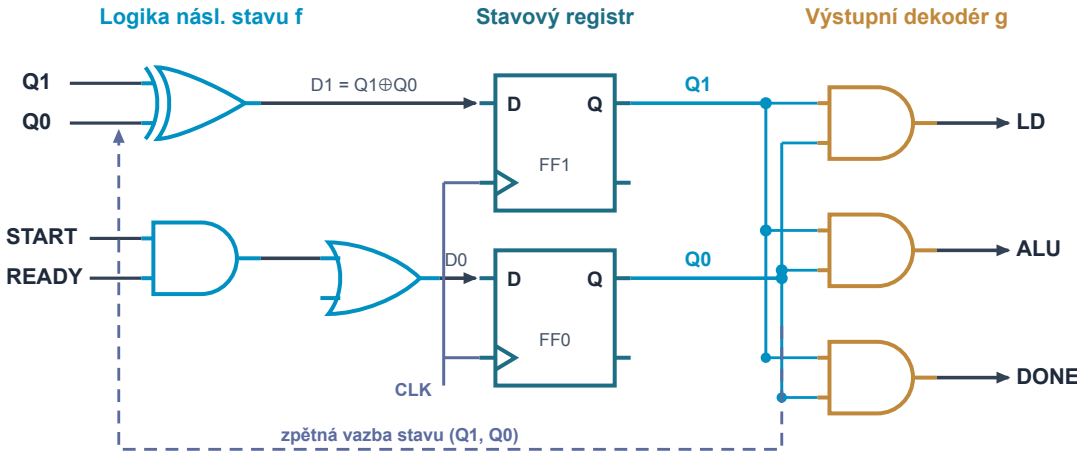
Výstupy. Mooreovy výstupy závisejí jen na stavu — každý je jednoduchý dekodér jednoho stavu:

$$LD = \overline{Q_1}\,Q_0\ (\text{S1}), \qquad ALU = Q_1\,\overline{Q_0}\ (\text{S2}), \qquad DONE = Q_1\,Q_0\ (\text{S3}).$$

Tím je návrh hotov: máme rovnice pro logiku následujícího stavu i pro výstupní logiku. Zbývá je realizovat hardwarem — a zde existují **dvě cesty**.

Obvodová (hradlová) realizace

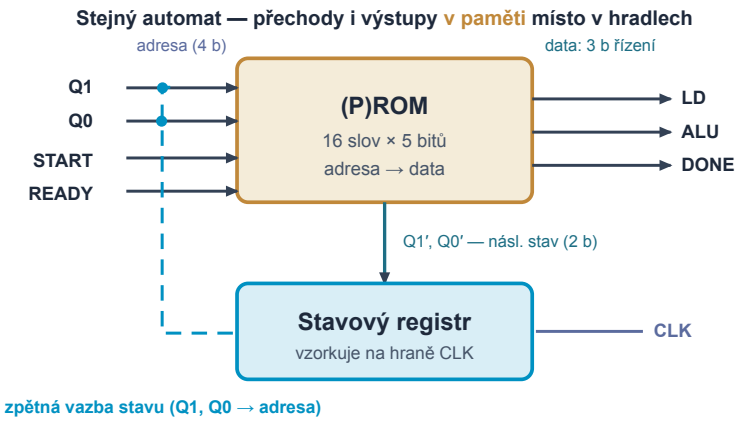
První možností je rovnice přímo „nakreslit" jako hradla kolem dvou klopných obvodů (`FF1` pro Q_1, `FF0` pro Q_0), jak ukazuje obr. 1.12:



Obrázek 1.12. Hradlová realizace řadiče

Logika následujícího stavu je vlevo: hradlo XOR počítá $Q_1' = Q_1 \oplus Q_0$, kaskáda hradel AND/OR realizuje výraz pro Q_0'. Vpravo je výstupní dekodér — tři hradla AND generují `LD`, `ALU`, `DONE`. Oba klopné obvody sdílejí hodinový signál `CLK`.

Taková realizace je **rychlá** (zpoždění = jen průchod několika hradly) a **úsporná** (málo tranzistorů), ale je **pevně zadrátovaná** — jakákoli změna chování znamená přepracovat logiku, tj. nový čip. Přesně tudy jde **RISC** (ARM): instrukce jsou tak jednoduché, že jim stačí rychlý obvodový řadič.



Obrázek 1.13. Realizace řadiče pomocí ROM

Realizace pomocí (P)ROM

Druhou možností je nepoužít hradla vůbec a celou přechodovou tabulku **uložit do paměti** (obr. 1.13). Adresou jsou bity stavu a vstupy, daty jsou bity příštího stavu a výstupy:

Pro náš automat má adresa 4 bity ($Q_1, Q_0, START, READY$) a data 5 bitů ($Q_1', Q_0', LD, ALU, DONE$) — paměť tedy stačí o velikosti 16×5 bitů. Obsah paměti je doslova přepsaná tabulka:

Adresa `Q1 Q0 · S R`	Data `Q1' Q0' · LD ALU DONE`
`00 · 0 –`	`00 · 0 0 0`
`00 · 1 –`	`01 · 0 0 0`
`01 · – –`	`10 · 1 0 0`
`10 · – 0`	`10 · 0 1 0`
`10 · – 1`	`11 · 0 1 0`
`11 · – –`	`00 · 0 0 1`

Výhoda je obrovská **flexibilita**: změnit chování automatu znamená jen **přepsat paměť**, ne předělat čip. To je krok k **mikroprogramovému řadiči**, kterému se budeme věnovat v další části — tam se z „tabulky v paměti" stane plnohodnotný program řídící celý procesor. Nevýhodou je nižší rychlost (čtení paměti) a větší plocha; proto se v jednoduchých rychlých jádrech (RISC) dává přednost hradlové realizaci.

> **Shrnutí:** Konečný automat = stavový registr + logika následujícího stavu + výstupní logika. Řadič navrhujeme postupem stavový diagram → tabulka → rovnice → hardware. Rovnice lze realizovat buď **hradly** (rychlé, pevné — cesta RISC), nebo **pamětí** (pomalejší, ale přepsatelné — cesta k mikroprogramu). Tím máme hotový obvodový řadič. Co ale řadič *řídí* — a jak z něj vznikne procesor — ukáže.

Datová cesta a vznik procesoru

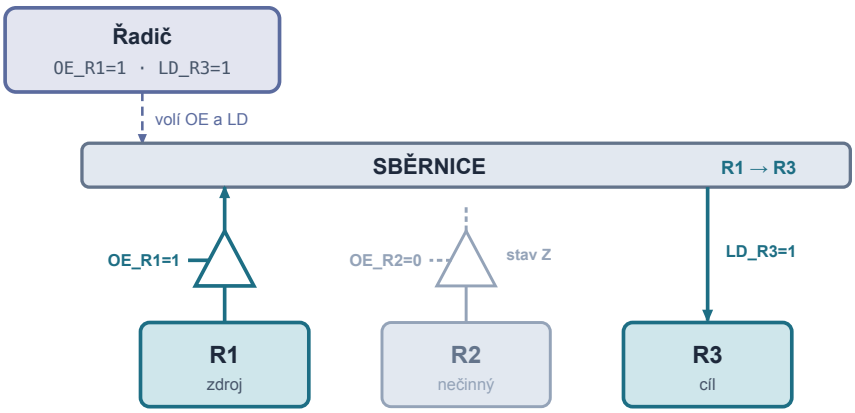
Řadič sám o sobě nic nepočítá — jen **generuje řídicí signály**. Aby byl k něčemu, musí řídit **datovou cestu** (datapath): registry, sběrnice a aritmeticko-logickou jednotku, kde se data skutečně přesouvají a zpracovávají. V tomto oddíle ukážeme, jak z řadiče a datové cesty postupně vznikne procesor — touž cestou, jakou se procesory historicky zrodily.

Datová cesta
Registry, ALU a sběrnice, kde se zpracovávají data; řídí je řadič.

Přenos mezi registry po sběrnici

Začněme nejjednodušší operací: přesunout obsah jednoho registru do druhého. Registry sdílejí společnou **sběrnici** (bus). Kdyby ji budilo víc registrů současně, dostaly by se jejich výstupy do konfliktu — jeden by táhl vodič k logické 1, druhý k 0, což je fakticky zkrat. Proto mají výstupy **třístavové** (tri-state): kromě 0 a 1 umí i stav „vysoká impedance" (Z), kdy jsou od sběrnice fakticky odpojeny. V daném okamžiku tak smí sběrnici budit **jediný** registr.

Třístavový výstup
Tri-state výstup s třetím stavem vysoká impedance, který umožní sdílení sběrnice.

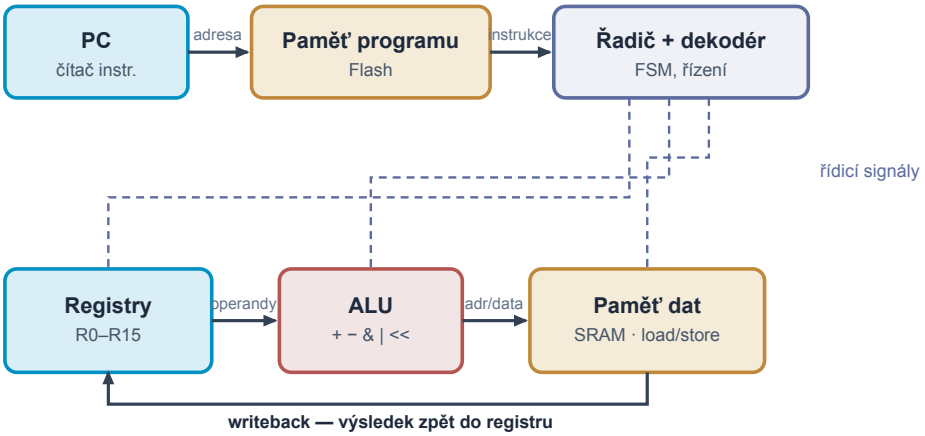


Obrázek 1.14. Přenos mezi registry po sběrnici s třístavovými výstupy.

Na obr. 1.14 je zachycen přenos `R1 → R3`: řadič ho provede dvěma řídicími signály — povolí **výstup zdroje** zdrojového registru (`OE_R1`, output enable) a **zápis do cíle** (`LD_R3`, load). Ostatní registry drží výstup ve stavu Z, takže sběrnici nebudí. Je to přesně to, co dělá řadič z předchozích oddílů — generuje signály `OE` a `LD` ve správném pořadí. Posloupnost takových přenosů (mikro-operací) už dokáže cokoli: zkopírovat, prohodit, posunout data mezi registry.

Datová cesta: registry + ALU

Přidáme-li ke sběrnici **ALU**, získáme výpočetní jádro. ALU vezme operandy z registrů, provede operaci (součet, rozdíl, logickou funkci, posun) a výsledek pošle zpět do registru. Operaci i výběr registrů opět určuje **řadič** (obr. obr. 1.15):



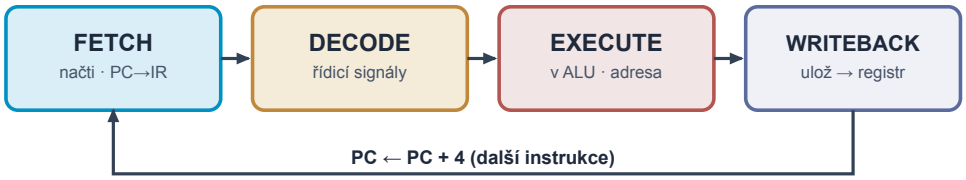
Obrázek 1.15. Datová cesta — registry, ALU a řízení

Toto uskupení — **banka registrů + ALU + sběrnice řízené řadičem** — je jádrem každého procesoru (v μP se mu říká *zápisníková paměť a ALU*). Operace typu `R3 ← R1 + R2` se rozpadne na posloupnost mikro-operací: přečti `R1` jako první operand, přečti `R2` jako druhý, nastav ALU na součet, zapiš výsledek do `R3` — a každý krok řídí řadič svými signály. Historicky přesně takhle fungovalo výpočetní jádro stolních kalkulaček konce 60. let.

Od pevné sekvence k programu — a je z toho procesor

Kalkulačka má posloupnost operací **pevně danou** — je zadrátovaná v řadiči a změnit ji znamená předělat hardware. Zásadní krok vpřed je tuto posloupnost **uložit do paměti jako instrukce** a nechat řadič, aby je postupně **četl a vykonával**. Tím se oddělí *co* se počítá (program v paměti) od toho, *čím* se to počítá (pevný obvod): tentýž hardware teď zvládne libovolný úkol — od kalkulačky po přehrávač videa — jen podle toho, jaký program mu do paměti nahrajeme. To je **koncept uloženého programu** a je to důvod, proč je počítač tak univerzální. Řadič se přitom zjednoduší na stále se opakující cyklus (obr. obr. 1.16):

Koncept uloženého programu
Program (instrukce) i data leží ve společné paměti; procesor instrukce postupně čte a vykonává. Umožňuje univerzální počítač — chování se mění přepsáním paměti, ne hardwaru.



Řadič = konečný automat s pevnou sekvencí stavů, který řídí datovou cestu

Obrázek 1.16. Cyklus instrukce: fetch — decode — execute — writeback

1. **Fetch** — načti instrukci z paměti (podle programového čítače `PC`),
2. **Decode** — dekóduj, *jakou* mikro-operaci datové cesty představuje,
3. **Execute** — proveď ji v ALU / na sběrnici,
4. **Writeback** — ulož výsledek; posuň `PC` na další instrukci.

A to **je procesor**: datová cesta (registry + ALU + paměť) řízená řadičem, který v nekonečné smyčce čte a vykonává instrukce z paměti. Řadič přitom může být obvodový (rychlý, pevný — RISC), nebo mikroprogramový (přepsatelný — viz oddíl 1.3). Tatáž struktura, kterou jsme právě složili z hradel a registrů, stojí — jen v mnohem větším měřítku — i v jádru **ARM Cortex-M**, se kterým budeme pracovat na cvičeních.

> **Shrnutí:** Řadič řídí **datovou cestu**. Registry sdílejí sběrnici a komunikují přes ni díky **třístavovým výstupům**; řadič vybírá zdroj (`OE`) a cíl (`LD`). Doplněním **ALU** vznikne výpočetní jádro (registry + ALU). Uložíme-li posloupnost operací do paměti jako **instrukce**, řadič je v cyklu *fetch–decode–execute* čte a vykonává — a tím se z datové cesty a řadiče stává **procesor**.

Synchronní, nebo asynchronní?

Celý dosavadní výklad stál na jednom nevysloveném předpokladu: **existují globální hodiny**, které udávají tempo všem prvkům současně. Byla to ale nutnost, nebo jen jedna z možných voleb? Podíváme se teď na *druhou cestu* — a zároveň dovysvětlíme, co se skrývá za forward-pointerem („pro sběrnici je potřeba **handshake**").

Proč zvítězily hodiny — a co to stojí

Globální takt stojí na jediném pravidle: **stav se mění jen na aktivní hraně hodin**. Časová správnost celého obvodu se tím scvrkne na jedinou podmínku — nejdelší kombinační cesta se musí stihnout do periody. To dělá návrh **skládatelným** (bloky spojím, když každý stíhá takt), **analyzovatelným** (statická časová analýza) a **testovatelným** (obvod lze krokovat po hranách). Právě proto na hodinách stojí prakticky všechny procesory i nástroje pro jejich návrh.

Má to ale svou cenu. Hodinový signál se musí rozvést ke **každému** klopnému obvodu se zanedbatelným rozdílem zpoždění (**clock skew**), což vyžaduje pečlivě vyvážený *hodinový strom* — a ten spotřebuje značnou část příkonu čipu. Celý obvod se navíc řídí **nejhorším případem**: takt je nastaven na nejpomalejší cestu, i když konkrétní data byla hotová dřív. A protože ve stejný okamžik spíná mnoho hradel, vzniká na frekvenci hodin výrazná **špička rušení** (EMI). Otázka tedy zní, zda jde tempo řídit *lokálně* — podle toho, kdy je která operace skutečně hotová.

Asynchronní obvody a Mullerův C-element

Asynchronní obvod nemá globální hodiny. Každý stupeň místo toho **signalizuje, že dokončil práci** (completion detection), a další stupeň se rozjede teprve tehdy. Obvod tak běží **průměrnou** rychlostí danou daty, ne nejhorším případem; odpadá hodinový strom i jeho spotřeba a rušení se rozprostře v čase.

Roli, jakou má v synchronním světě klopný obvod, hraje v tom asynchronním **Mullerův C-element**: jeho výstup se změní, **až když se oba vstupy shodnou** (00 → 0, 11 → 1), jinak drží starou hodnotu. Tím detekuje, že se dvě větve výpočtu „setkaly", a je proto přirozeným prvkem pro synchronizaci bez hodin.

Asynchronní obvod
Číslicový obvod bez globálního hodinového signálu; tempo si prvky řídí lokálně signalizací dokončení operace. Pracuje průměrnou, ne nejhorší rychlostí, nemá hodinový strom a méně ruší, ale hůř se navrhuje a testuje standardními nástroji.

Mullerův C-element
Základní paměťový prvek asynchronní logiky. Výstup nastaví na společnou hodnotu, teprve když se oba vstupy shodnou (00 dá 0, 11 dá 1); při neshodě drží starou hodnotu. Slouží k detekci, že se dvě větve výpočtu setkaly (rendez-vous).

Handshake: request a acknowledge

Dva asynchronní bloky se na tempu domluví **potvrzovacím protokolem** — **handshake**: zdroj signálem `req` (request) ohlásí, že má platná data, a příjemce je potvrdí signálem `ack` (acknowledge). Nejběžnější je **čtyřfázová** varianta (return-to-zero), jak ukazuje obr. 1.17: zdroj zvedne `req` (žádost), příjemce odpoví zvednutím `ack` (potvrzení), zdroj `req` zase stáhne (konec žádosti) a příjemce vrátí `ack` do klidu — čtyři přechody na jeden přenos. Alternativní **dvoufázová** varianta bere každou *změnu* úrovně jako jednu událost, takže na přenos stačí dva přechody místo čtyř (rychlejší, ale logika je složitější).

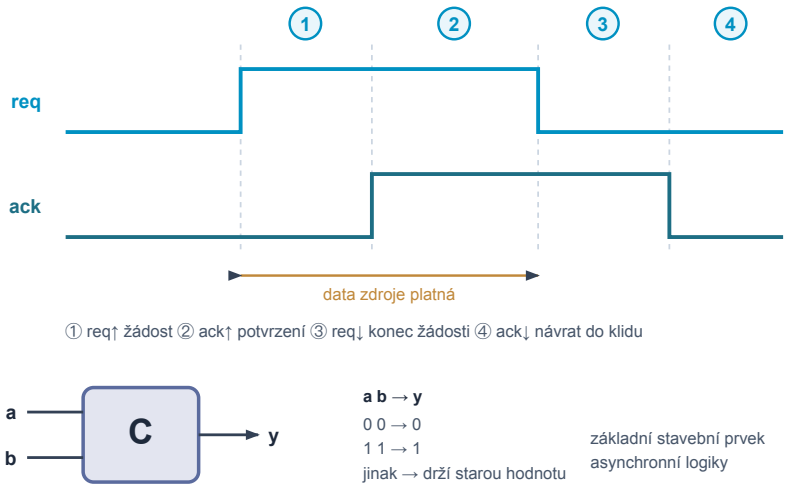
Zbývá otázka, jak příjemce pozná, že jsou data *platná*. **Bundled-data** vede data souběžně s `req` a spoléhá, že zpožďovací linka udrží `req` o něco pozadu za nejpomalejším datovým bitem (asynchronní obdoba předpokladu o kritické cestě). **Dual-rail** kóduje každý bit dvěma vodiči (jeden pro 0, druhý pro 1), takže samotná data nesou informaci „už jsem platné" — obvod je pak *necitlivý na zpoždění*, ale za cenu dvojnásobku vodičů.

Handshake
Dvouvodičový potvrzovací protokol mezi zdrojem a příjemcem (request a acknowledge). Zdroj signálem request ohlásí platná data, příjemce je odbavením potvrdí signálem acknowledge. Nahrazuje globální hodiny lokální dohodou o tempu a je základem přenosu mezi hodinovými doménami i sběrnicových protokolů.

Kde se s tím reálně potkáš

Čistě asynchronní procesory zůstaly kuriozitou — návrhové nástroje, časová analýza i testování jsou postavené kolem hodin a asynchronní logika se jim vzpírá. Přesto handshake a asynchronní myšlení **nezmizely**, jen se schovaly:

- **Přechod mezi hodinovými doménami** (clock-domain crossing). Reálný čip má hodin víc (jádro, USB, RTC…) a data mezi nimi musí projít bezpečně — přesně ten problém. Řeší ho **asynchronní FIFO**, jehož ukazatele se přenášejí v **Grayově kódu** (mezi



Obrázek 1.17. Čtyřfázový handshake `req`/`ack` a Mullerův C-element jako základní prvek asynchronní logiky

sousedními hodnotami se mění jen jeden bit, takže případná metastabilita zkazí nejvýš tento bit).

- **Sběrnicové protokoly.** Signály `VALID`/`READY` u sběrnic AXI a AHB jsou doslova synchronní handshake `req`/`ack`: master ohlásí `VALID`, slave odbaví `READY`. Myšlenka potvrzování tak přežívá **uvnitř** synchronních sběrnic (přednášky 4 a 6).
- **GALS** (globally asynchronous, locally synchronous). Kompromis, který dnes vede: ostrovy synchronní logiky s vlastními hodinami, propojené asynchronním handshakem. Takto jsou stavěné vícejádrové systémy i sítě na čipu (NoC).

Plně asynchronní návrh si drží niky, kde jeho přednosti převáží: **ultranízká spotřeba** (spíná jen aktivní logika) a **odolnost proti odběrové analýze** v bezpečnostních čipech (nepravidelný odběr hůř prozradí, co obvod počítá). Pro nás je ale hlavní ponaučení, že **hodiny jsou inženýrská volba, ne přírodní zákon** — a že handshake, na který narazíme u sběrnic i DMA, je tatáž myšlenka jako potvrzování mezi asynchronními bloky.

1.3 Mikroprogramový řadič a vykonávání instrukcí

V oddílu 1.2 jsme navrhli řadič pro *jednu* pevně danou operaci. Skutečný procesor ale musí zvládnout **mnoho různých instrukcí**, a navíc s měnitelnou sadou. Ukážeme, jak se z obvodového řadiče stane **mikroprogramový řadič** — malý počítač uvnitř počítače, který pro každou instrukci projde posloupnost elementárních kroků.

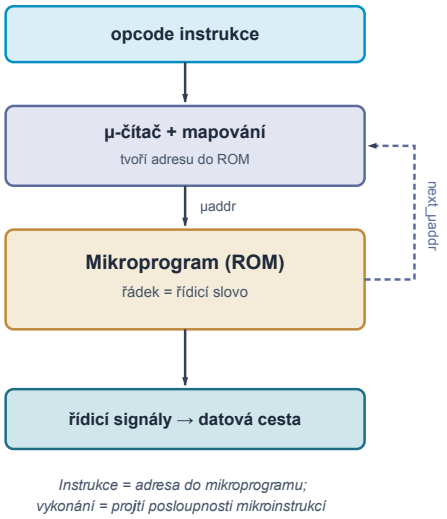
Mikroprogramový řadič
Řadič, jehož řídicí signály jsou uloženy jako mikroinstrukce v paměti (control store).

Od obvodového k mikroprogramovému řadiči

V jsme viděli, že přechodovou tabulku automatu lze místo hradel uložit do **paměti**. Tato myšlenka se dá dotáhnout mnohem dál. Obvodový řadič má totiž zásadní nevýhodu: pro každou instrukci procesoru je třeba navrhnout vlastní logiku, a celá sada instrukcí je **napevno zadrátovaná do křemíku**. U procesoru se stovkami instrukcí by stavový automat narostl do nezvládnutelné velikosti a jakákoli změna instrukční sady by znamenala nový čip.

Mikroprogramový řadič to řeší tak, že řídicí signály neukládá do hradel, ale do **paměti** (control store, typicky ROM). Každá instrukce procesoru se rozloží na posloupnost elementárních kroků — **mikroinstrukcí** — a tato posloupnost je uložena jako *program* v paměti. Řadič pak jen postupně čte mikroinstrukce a jejich bity přímo ovládají datovou

cestu (obr. obr. 1.18).



Obrázek 1.18. Mikroprogramový řadič: μPC adresuje control store, jehož výstup řídí datovou cestu

Srdcem je **mikroprogramový čítač** `μPC`, který adresuje paměť mikroprogramu. Na dané adrese leží mikroinstrukce, jejíž jedna část přímo budí řídicí signály datové cesty a druhá část říká, **odkud vzít další mikroinstrukci**. Vykonání jedné strojové instrukce = projití příslušné posloupnosti mikroinstrukcí.

> **Historická poznámka.** Princip mikroprogramování navrhl **Maurice Wilkes** v roce 1951 — s motivací učinit návrh řadiče *systematickým a pravidelným* místo změti hradel navrhované ad hoc. První významnou realizací byl počítač EDSAC 2 (1958). Mikroprogramování se pak stalo základem velkých sálových počítačů a — jak uvidíme — v transformované podobě žije v procesorech dodnes.

Mikroinstrukce a formát řídicího slova

Jedna mikroinstrukce je **řídicí slovo**: sada bitů, kde každý bit (nebo skupina bitů) ovládá v daném taktu nějakou část datové cesty (obr. obr. 1.19).

Jedna mikroinstrukce — každé pole ovládá část datové cesty:

ALU_op	RegWrite	MemRead	MemWrite	PC_src	next_μaddr
ADD	1	0	0	+4	0x07

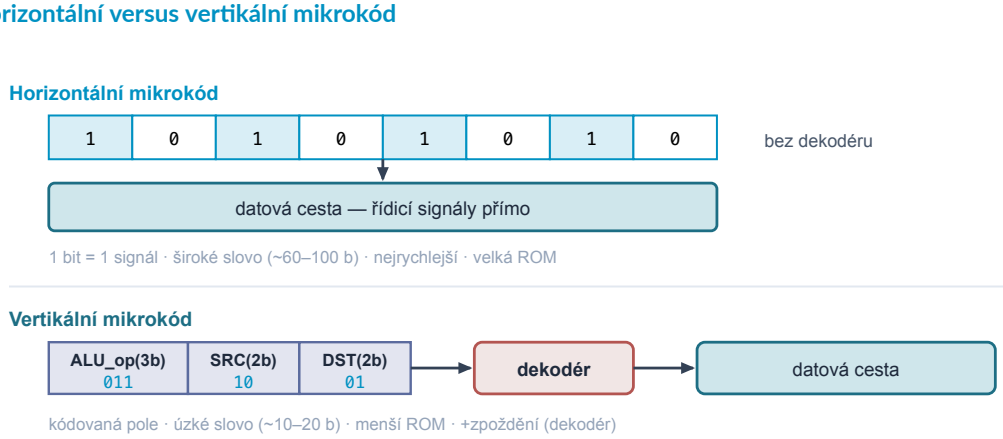
řídicí signály + jak pokračovat (next_μaddr)

Obrázek 1.19. Formát mikroinstrukce — řídicí slovo

Typická pole takového slova:

- `ALU_op` — jakou operaci má provést ALU,
- `RegWrite` — má se výsledek zapsat do registru,
- `MemRead` / `MemWrite` — přístup do paměti,
- `PC_src` — odkud vzít další hodnotu programového čítače,
- `next_μaddr` — adresa další mikroinstrukce (k tomu se vrátíme u sekvencování).

Otázka návrhu zní: má mít každý řídicí signál **vlastní bit**, nebo je výhodnější signály **zakódovat do polí**? To vede ke dvěma filozofiím mikrokódu.



Obrázek 1.20. Horizontální a vertikální mikrokód

Obě filozofie staví proti sobě obr. obr. 1.20. **Horizontální mikrokód** dává každému řídicímu signálu vlastní bit. Slovo je tím **široké** (klidně 60–100 bitů), ale výhody jsou zásadní: signály jdou do datové cesty **přímo, bez dekodéru** (nejrychlejší možné), a v jednom taktu lze aktivovat **libovolnou kombinaci** signálů (plný paralelismus). Cenou je velká paměť mikroprogramu.

Vertikální mikrokód naopak signály **zakóduje do polí** — např. 3 bity vyberou jednu z 8 operací ALU. Slovo je **úzké** (10–20 bitů), paměť malá, ale je nutný **dekodér**, který pole rozbalí na signály (a tím přidá zpoždění), a v jednom poli lze zvolit jen **jednu akci** (omezený paralelismus).

Je to tentýž kompromis *šířka za dekódování*, jaký jsme viděli u kódování stavů: horizontální mikrokód je obdobou **one-hot** (jeden bit na signál — široké, ale bez dekodéru a rychlé), vertikální odpovídá **binárnímu** kódování (úzké, zato s dekodérem). Stejná úvaha, jiná úroveň.

V praxi se používá kompromis (tzv. **diagonální** mikrokód). Procesory CISC (x86) historicky stojí na vertikálním/smíšeném mikrokódu; RISC (ARM) je naopak tak jednoduchý, že mikrokód nepotřebuje — vystačí s rychlým obvodovým řadičem.

Sekvencování, větvení a podprogramy

Klíčová otázka mikroprogramového řadiče zní: jak se určí **adresa další** mikroinstrukce (`µPC`)? Nejjednodušší pravidlo je **`µPC ← µPC + 1`** — mikroinstrukce se vykonávají po řadě, přesně jako řádky programu. To stačí na **přímou posloupnost**, například na společnou úvodní fázi *fetch* (načtení instrukce). V zápisu mikrokódu vystupují tři pomocné registry datové cesty, které si zde zavedeme: **MAR** (Memory Address Register) drží adresu právě prováděného přístupu do paměti, **MDR** (Memory Data Register) drží slovo čtené z paměti nebo do ní zapisované a **IR** (Instruction Register) drží právě načtenou instrukci. Zápis `X ← Y` znamená „přenes obsah `Y` do `X`“, `Mem[MAR]` je slovo v paměti na adrese uložené v `MAR`:

`µaddr`	Mikroinstrukce
`0x00`	`MAR ← PC`
`0x01`	`MDR ← Mem[MAR]`
`0x02`	`IR ← MDR ; PC ← PC+1`
`0x03`	dekóduj podle `opcode` …

Řadič ale potřebuje i **rozhodování** (čekací smyčky, různé instrukce) a **opakování**. K tomu

MAR
Memory Address Register — adresní registr paměti: drží adresu, na kterou právě probíhá přístup do paměti.

MDR
Memory Data Register — datový registr paměti: drží slovo právě čtené z paměti nebo do ní zapisované.

IR
Instruction Register — instrukční registr: drží právě načtenou instrukci, jejíž operační kód se pak dekóduje.

slouží dvě rozšíření: větvení a podprogramy.

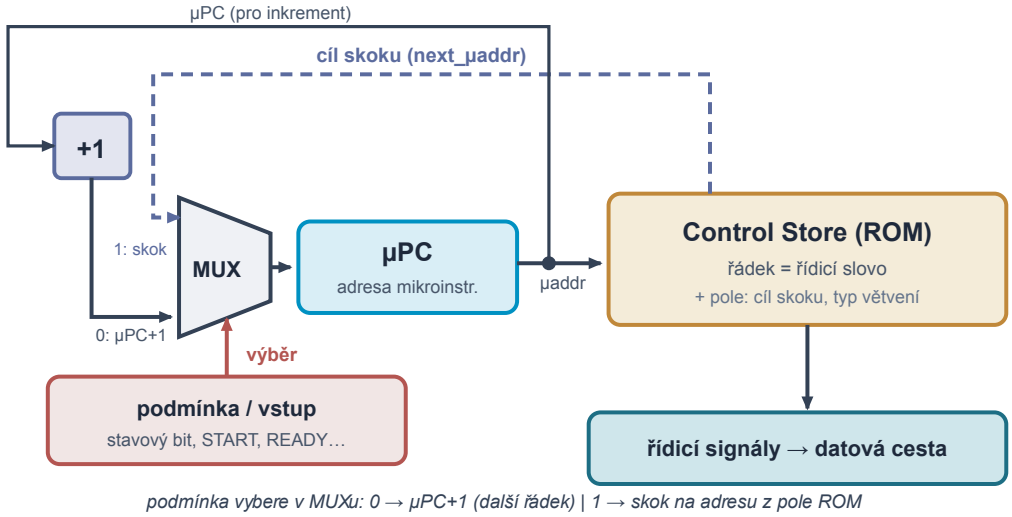
Větvení v mikroprogramu

Aby mikroprogram mohl reagovat na podmínky, nese mikroinstrukce kromě řídicích bitů i **pole adresy skoku** a informaci o **typu větvení**. Multiplexor pak podle nějaké **podmínky** (stavový bit, vstup jako `START` nebo `READY`) vybere, zda bude další adresou `µPC + 1`, nebo cíl skoku z pole (obr. obr. 1.21):

Jako konkrétní příklad vezměme **čekací smyčku** — tutéž logiku, jakou měl stav `S2` (čekání na `READY`) z obvodového řadiče, teď ale v mikrokódu:

µaddr	Mikroinstrukce	Větvení
`0x10`	`MAR ← adresa ; spusť čtení`	`µPC+1`
`0x11`	(čekej)	`READY=0` → skok `0x11` · `READY=1` → `µPC+1`
`0x12`	`data ← MDR`	`µPC+1`

Řádek `0x11` **skáče sám na sebe**, dokud `READY = 0` — to je čekací smyčka. Jediný vstup (`READY`) tak řídí, zda se adresa **zvětší**, nebo **zůstane**. Je to přímá mikroprogramová obdoba podmíněného přechodu, který jsme kreslili ve stavovém diagramu.



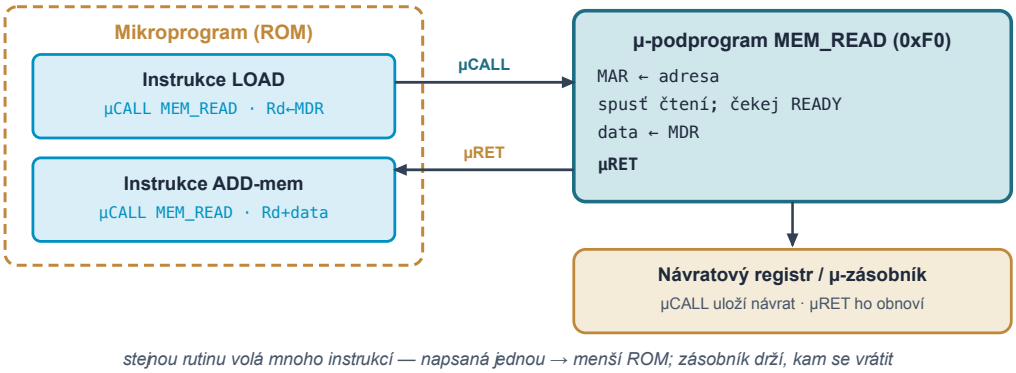
Obrázek 1.21. Větvení v mikroprogramu: podmínka řídí adresu

Podprogram v mikroprogramu

Mnoho strojových instrukcí potřebuje **stejnou** dílčí mikro-sekvenci — typicky čtení operandu z paměti. Psát ji znovu v každé instrukci by plýtvalo pamětí mikroprogramu. Řešením je **mikro-podprogram** (obr. obr. 1.22):

- `µCALL adr` — uloží návratovou mikroadresu (`µPC + 1`) a skočí na `adr`,
- `µRET` — obnoví `µPC` z uložené návratové adresy.

Aby to fungovalo, potřebuje řadič navíc **návratový registr** (pro jednu úroveň), nebo **mikro-zásobník** (pro vnořená volání). Je to přesná obdoba instrukce `BL` (Branch-with-Link) a návratu přes registr `LR` v ARM — jen o úroveň níž, uvnitř řadiče. Tatáž hierarchie se opakuje na třech úrovních: podprogram v mikrokódu ↔ funkce v assembleru ↔ funkce v jazyce C.

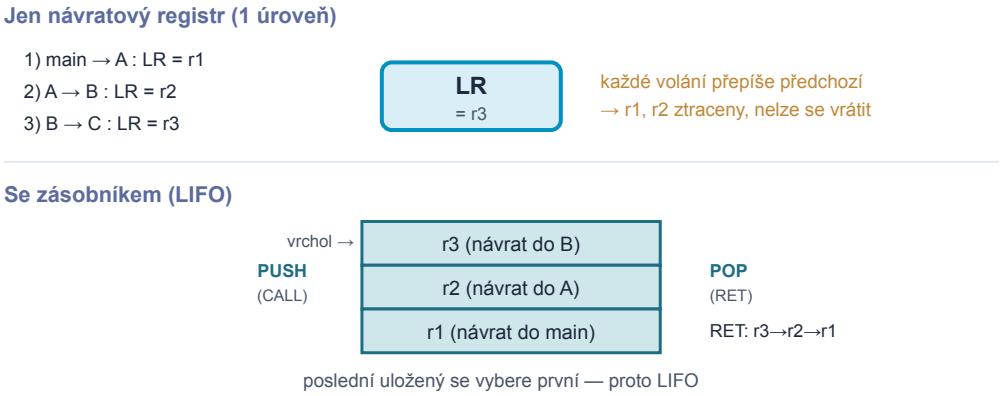


Obrázek 1.22. Podprogram v mikroprogramu: μCALL/μRET a návratový zásobník

A právě potřeba uložit návratovou adresu — a zejména **více návratových adres při vnořování** — nás přivádí k jednomu z nejdůležitějších konceptů: zásobníku.

Zásobník

Podprogram musí uložit návratovou adresu. Jeden registr (LR, resp. mikro-návratový registr) na to stačí — ale **jen na jednu úroveň**. Jakmile zavolá podprogram další podprogram, druhé volání návratovou adresu v registru **přepíše** a původní návrat je nenávratně ztracen (obr. obr. 1.23):



Obrázek 1.23. Proč zásobník: jeden registr vnořené volání přepíše

Potřebujeme tedy uchovat **více návratových adres** a vybírat je v **opačném pořadí**, než byly uloženy (z volání `main` → `A` → `B` → `C` se vracíme `C` → `B` → `A` → `main`). Datová struktura s tímto chováním „poslední dovnitř, první ven" se nazývá **zásobník** (LIFO, Last-In First-Out). Totéž platí pro **vnořená přerušení** — i ta se musí vracet v opačném pořadí, než přišla.

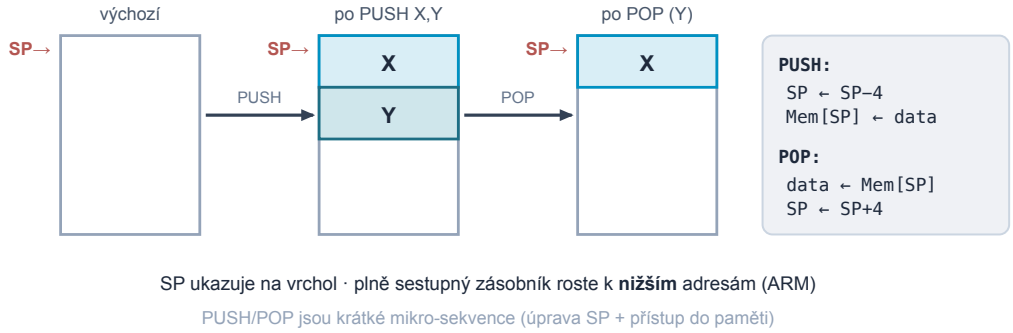
Zásobník
LIFO struktura pro návratové adresy a lokální data; na ARM roste dolů.

Jak zásobník funguje

Zásobník je oblast paměti, jejíž vrchol ukazuje registr **ukazatel zásobníku** (Stack Pointer, SP) — obr. obr. 1.24. Dvě operace:

- **PUSH** (ulož) — posuň SP a zapiš hodnotu,
- **POP** (vyzvedni) — přečti hodnotu a posuň SP zpět.

Zásobník bývá **plně sestupný** — roste směrem k **nižším** adresám, takže PUSH SP zmenší a POP

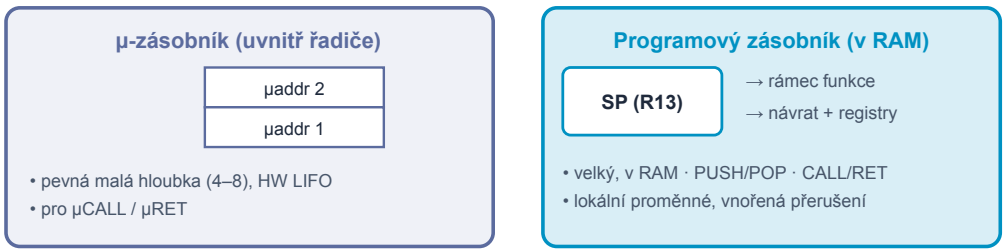


Obrázek 1.24. Operace zásobníku: SP, PUSH a POP

zvětší (to je konvence procesorů ARM). V hardwaru je celá věc triviální: `SP` je obyčejný registr a `PUSH`/`POP` jsou krátké mikro-sekvence (úprava `SP` plus přístup do paměti). Na ARM je `SP = R13` a existují přímo instrukce `PUSH {…}` / `POP {…}`.

Dvě úrovně zásobníku

Tentýž princip se objevuje **dvakrát** — jednou uvnitř řadiče, podruhé v programu (obr. obr. 1.25):



Obrázek 1.25. Dvě úrovně zásobníku: mikro-zásobník vs. programový zásobník

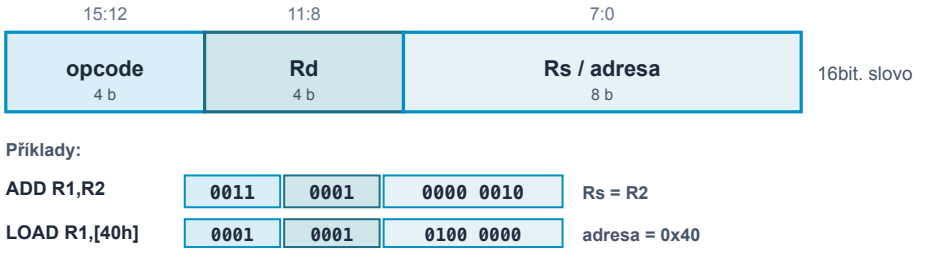
Mikro-zásobník je malý hardwarový LIFO s pevnou hloubkou přímo v řadiči; slouží vnořeným `μCALL`/`μRET` a ukládá jen mikroadresy. **Programový zásobník** je velký, leží v hlavní paměti (RAM), spravuje ho `SP` a slouží instrukcím `CALL`/`RET`, lokálním proměnným, uloženým registrům a vnořeným přerušením. Jeden koncept, dvě měřítka. Detaily programového zásobníku (konvence volání AAPCS, přepínání kontextu) probereme v přednáškách 4 a 9.

Instrukční sada a obvodový vs. mikroprogramový řadič

Abychom viděli celý smysl mikroprogramu, definujme **malou instrukční sadu** (ISA) procesoru s registry `R0`–`R15`, ALU a pamětí:

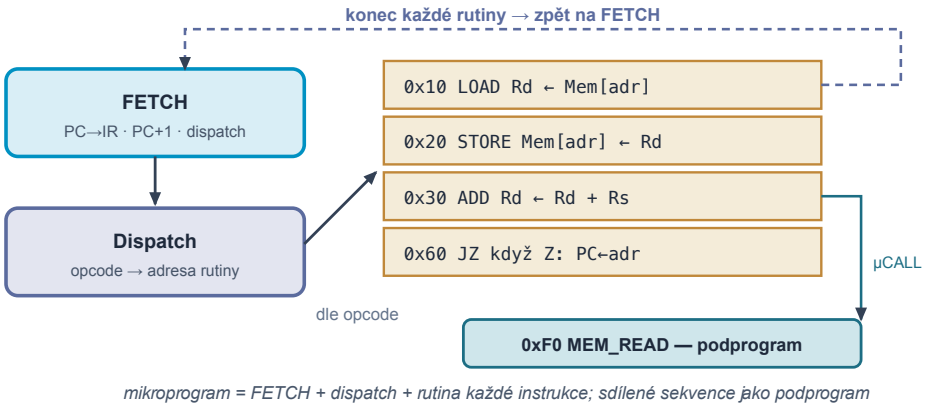
Opcode	Instrukce	Operace
`0000`	`NOP`	nic
`0001`	`LOAD Rd,[adr]`	Rd ← Mem[adr]
`0010`	`STORE Rd,[adr]`	Mem[adr] ← Rd
`0011`	`ADD Rd,Rs`	Rd ← Rd + Rs
`0100`	`SUB Rd,Rs`	Rd ← Rd − Rs
`0101`	`JMP adr`	PC ← adr
`0110`	`JZ adr`	když Z=1: PC ← adr

Každá instrukce má **formát** určující, kde jsou v 16bitovém slově opcode a operandy (obr. obr. 1.26):



Obrázek 1.26. Formát instrukce

Instrukční sada (ISA) je „smlouva" mezi programátorem a procesorem — *co* procesor umí. *Jak* to splní, je věc implementace, kterou zde řeší mikroprogram. Konkrétně: **každá instrukce ISA odpovídá jedné rutině v mikroprogramu.** Po společné fázi *fetch* se podle `opcode` skočí na začátek příslušné rutiny — tomuto kroku se říká **dispatch** (obr. obr. 1.27):



Obrázek 1.27. Od instrukce k mikroprogramu: fetch → dispatch → rutina → zpět na fetch

Zde se konečně spojí všechno, co jsme zavedli — sekvencování, větvení i podprogram:

Rutina	Mikrokroky (`;` = paralelně v jednom taktu, `·` = další takt)	Poznámka
`FETCH 0x00`	`MAR ← PC ; MDR ← Mem[MAR]` · `IR ← MDR ; PC ← PC+1` · `dispatch dle opcode`	společná fáze
`LOAD 0x10`	`MAR ← IR.adr` · `μCALL MEM_READ` · `Rd ← MDR` · `μJMP FETCH`	`Rd ← Mem[adr]`; volá podprogram
`ADD 0x30`	`ALU: Rd ← Rd + Rs ; nastav Z` · `μJMP FETCH`	`Rd ← Rd + Rs`
`JZ 0x60`	`když Z=0 → μJMP FETCH` · `PC ← IR.adr` · `μJMP FETCH`	podmíněný skok (větvení dle `Z`)
`MEM_READ 0xF0`	`spusť čtení ; čekej na READY` · `MDR ← data` · `μRET`	sdílený podprogram (čekací smyčka)

Procesor je tedy **nekonečná smyčka FETCH**, která pro každou načtenou instrukci spustí její mikroprogram a vrátí se pro další. Přidat novou instrukci znamená jen **přidat rutinu do paměti mikroprogramu** — žádná změna hradel.

Obvodový versus mikroprogramový řadič

Oba přístupy k realizaci řadiče, které jsme viděli, mají své místo (shrnuje tab. 1.3):

Tabulka 1.3. Srovnání obvodového a mikroprogramového řadiče

Vlastnost	Obvodový (hardwired)	Mikroprogramový
Realizace	pevná logika (hradla)	mikroprogram v ROM
Rychlost	vyšší	nižší (čtení ROM)
Změna / oprava	nový křemík	přepsat ROM
Složité instrukce	obří automat	snadno (sekvence)
Spotřeba plochy	malá	větší
Typické pro	**RISC** (ARM, RISC-V)	**CISC** (x86), historické CPU

Mikroprogram lze chápat jako **„software běžící uvnitř procesoru"**, který interpretuje instrukce ISA. Pokud je ale instrukční sada dostatečně jednoduchá a pravidelná (typicky **RISC**), lze tytéž řídicí signály generovat přímo **kombinační logikou**. Takový obvodový řadič je rychlejší a nevyžaduje čtení mikroinstrukcí z řídicí paměti. Proto klasické RISC procesory (např. **ARM Cortex-M**) používají převážně obvodový řadič, zatímco historické CISC procesory (např. x86) byly založené na mikroprogramování. Moderní vysoce výkonné procesory často **kombinují oba přístupy**. Například x86 dekóduje každou strojovou instrukci na jednu či více **mikrooperací** (`μop`): jednoduché instrukce přeloží přímo hradly na jedinou mikrooperaci, kdežto složité spustí **mikrokódový engine**, který vygeneruje celou jejich posloupnost. Uvnitř tak i „CISC" procesor nakonec vykonává jednoduché, RISC-ové mikrooperace.

Mikrooperace
Elementární vnitřní operace (micro-op), na kterou procesor rozloží strojovou instrukci; jádro pak vykonává právě tyto mikrooperace.

Tím se uzavírá cesta od jednoho klopného obvodu až k procesoru vykonávajícímu instrukce. Co je *instrukční sada* zač, jak se liší architektury (ARM, RISC-V, x86) a jak souvisí ISA s implementací — to je téma přednášky 7 o architekturách.

> **Shrnutí:** Mikroprogramový řadič nahrazuje pevnou logiku „programem" v paměti. Mikroinstrukce (řídicí slovo) řídí datovou cestu; sekvencování, větvení a podprogramy z něj dělají plnohodnotný řídicí program. Potřeba uchovat návratové adresy při vnořování vede na zásobník (LIFO, `SP`, PUSH/POP), který se objevuje jak v řadiči, tak v programu. Každá instrukce ISA je rutina v mikroprogramu a procesor je smyčka fetch–dispatch–vykonej. RISC (ARM) volí pro rychlost

a úspornost obvodový řadič; mikrokód patří spíše do světa CISC.

1.4 ARM Cortex-M: konkrétní procesor

Dosud jsme procesor stavěli „zdola" jako obecnou strukturu; teď se na něj podíváme „shora" — jako na konkrétní součástku ze cvičení, jádro **ARM Cortex-M**. Pod povrchem je to přesně ta datová cesta řízená řadičem z oddílu 1.2, jen ve větším měřítku a obalená periferiemi.

Tato část je **praktická orientace** v procesoru. Hlubší srovnání architektur (RISC vs. CISC, ARM vs. RISC-V, zřetězení) je tématem následující přednášky; zde jde o to, abyste rozuměli tomu, *s čím* budete na cvičeních pracovat.

Rodina Cortex-M a bloková architektura

Firma **ARM** je zvláštní tím, že **sama čipy nevyrábí** — pouze licencuje návrh jádra a instrukční sadu výrobcům (ST, NXP, Nordic, TI…). Díky tomu existují tisíce různých mikrokontrolérů sdílejících totéž jádro (tab. 1.4) a tytéž základní principy. ARM je architektura typu **RISC** (jednoduché instrukce pevné délky, architektura load/store), což jí dává nízkou spotřebu a předvídatelné chování. **Load/store** znamená, že aritmetika a logika pracují *výhradně s registry* a do paměti sahají jen dvě instrukce — `LDR` (načti) a `STR` (ulož). Je to přímý důsledek datové cesty z oddílu 1.2: ALU bere operandy z banky registrů a výsledek do ní vrací, takže data musí být nejdřív v registrech.

Tabulka 1.4. Profily jader ARM Cortex

Profil	Použití	Příklady
Cortex-M	mikrokontroléry, real-time	STM32, nRF52
Cortex-R	systémy s tvrdým real-time	řízení disků, modemy
Cortex-A	aplikační procesory s OS	mobil, Raspberry Pi

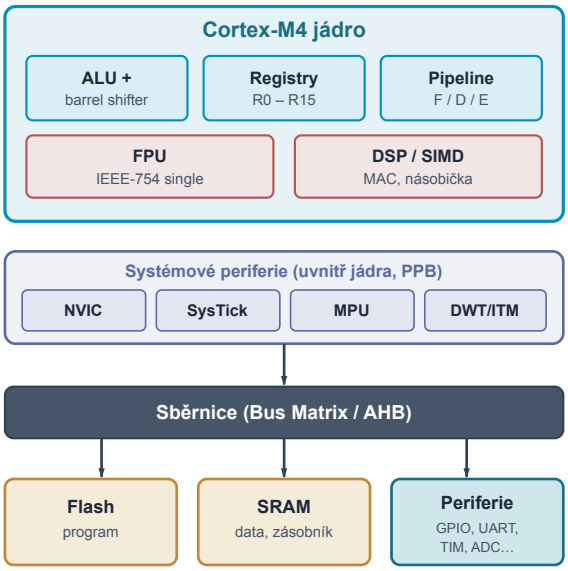
Nás zajímá profil **M**. V rámci něj existuje řada jader od nejmenšího M0+ (jednotky μW, jednoduché senzory) po M55 (strojové učení na MCU). Na cvičeních potkáte **Cortex-M4** (STM32L4/F4) a **Cortex-M33** (STM32U5). Důležité je, že všechna jádra M sdílejí stejný programátorský model i paměťovou mapu — co se naučíte na M4, platí napříč rodinou.

Cortex-M4 — bloková architektura

Uvnitř jádra (obr. 1.28) najdeme přesně to, co známe z datové cesty:

- **ALU + barrel shifter** — aritmetika, logika, posuvy (kombinační část),
- **registrový soubor** `R0`–`R15` — rychlá pracovní paměť (sekvenční část),
- **3stupňové zřetězení** (fetch / decode / execute) — překrývá vykonávání instrukcí,
- **FPU** — hardwarové operace v plovoucí řádové čárce (IEEE-754, jednoduchá přesnost),
- **DSP instrukce** — násobení s akumulací (MAC) a SIMD pro zpracování signálů.

Kolem jádra jsou **systémové periferie** mapované přímo do adresního prostoru: **NVIC** (řadič přerušení), **SysTick** (systémový časovač, časová základna pro RTOS), **MPU** (ochrana paměti) a ladicí jednotky **DWT/ITM**. Vše komunikuje přes **sběrnici** s pamětí Flash, SRAM a s periferiemi — sběrnice je v podstatě tatáž sdílená cesta jako, jen výkonnější.



Obrázek 1.28. Bloková architektura Cortex-M4

Registry a programátorský model

Cortex-M má šestnáct 32bitových jádrových registrů; většina je k volnému použití, několik má zvláštní roli. Kromě nich existuje hrstka **speciálních registrů** pro maskování přerušení a řízení režimu (obr. obr. 1.29).

Obecné registry `R0`–`R12` drží data a mezivýsledky. Konvence volání (AAPCS, přednáška 3) je dělí na **caller-saved** (`R0`–`R3` — argumenty a návratová hodnota, volně k použití) a **callee-saved** (`R4`–`R11` — lokální proměnné, které funkce musí zachovat); `R12` je scratch. **Speciální jádrové registry**: `R13` = **`SP`** (ukazatel zásobníku — *banked*, viz níže), `R14` = **`LR`** (návratová adresa z `BL`, resp. `EXC_RETURN` v handleru), `R15` = **`PC`** (adresa prováděné instrukce).

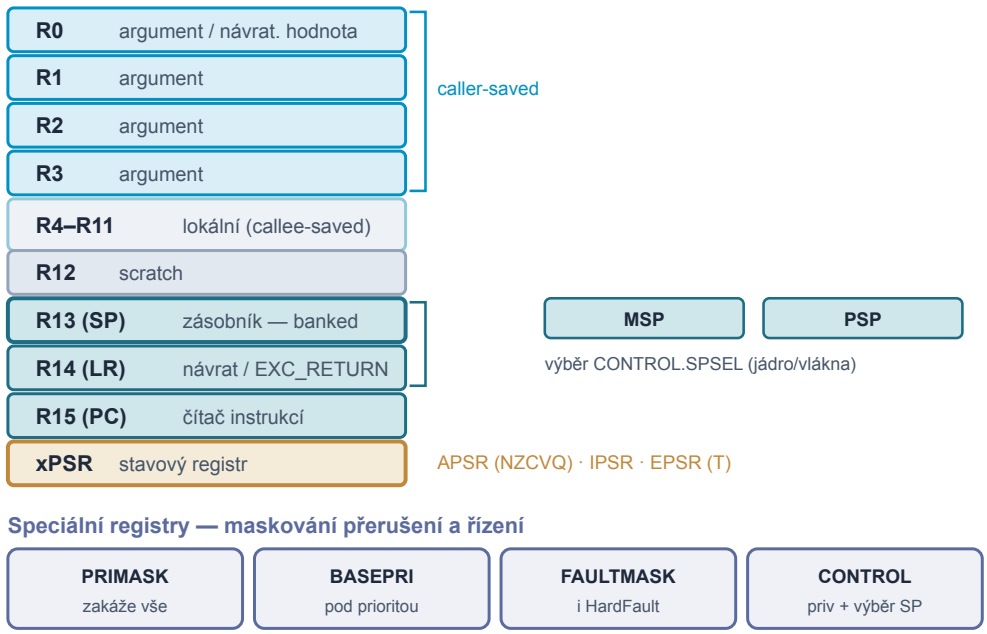
`xPSR` je stavový registr složený ze tří částí: **APSR** (příznaky `N` negative, `Z` zero, `C` carry, `V` overflow, `Q` saturace), **IPSR** (číslo právě obsluhované výjimky) a **EPSR** (bit `T` — Thumb). Příznaky nastavují aritmetické instrukce s příponou `S` (`ADDS`, `CMP`) a čte je podmíněné větvení — je to přesně ten „příznak `Z`" od instrukce `JZ` v mikroprogramu.

Samostatnou skupinou jsou **maskovací a řídicí registry**: `PRIMASK` (zakáže všechna přerušení), `BASEPRI` (zakáže jen přerušení pod zvolenou prioritou), `FAULTMASK` (zakáže i `HardFault`) a `CONTROL` (úroveň oprávnění + výběr `SP`). K maskování se vrátíme u kritických sekcí v přednášce 2.

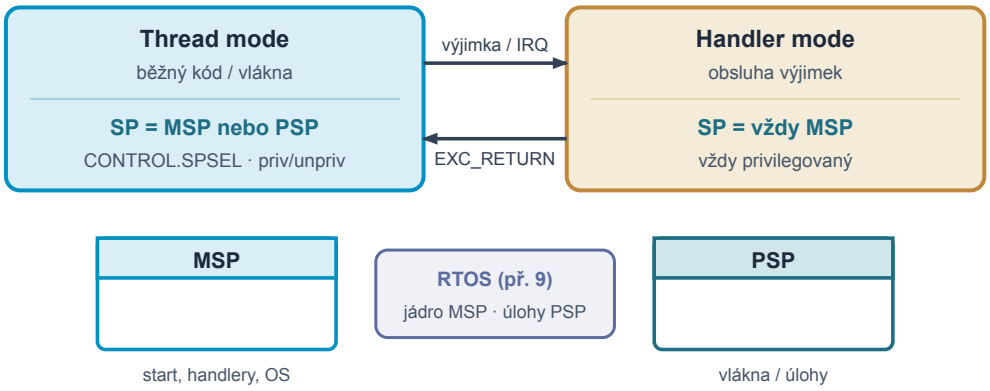
Režimy procesoru a dva zásobníky

Programátorský model stojí na třech dvojicích. Procesor běží buď v **Thread mode** (běžný kód aplikace), nebo v **Handler mode** (obsluha výjimek a přerušení — ten je vždy privilegovaný). Nezávisle na tom rozlišuje **privilegovanou** a **neprivilegovanou** úroveň oprávnění; omezení neprivilegovaného kódu chrání systémové prostředky. A konečně má **dva ukazatele zásobníku**: **`MSP`** (Main Stack Pointer) pro start, jádro a obsluhu přerušení, a **`PSP`** (Process Stack Pointer) pro vlákna aplikace (obr. obr. 1.30).

Výběr `SP` i úroveň oprávnění řídí registr `CONTROL`; po resetu běží procesor v Thread mode privilegovaně na `MSP`. Při výjimce hardware automaticky přepne do Handler mode na `MSP`



Obrázek 1.29. Registrový soubor Cortex-M

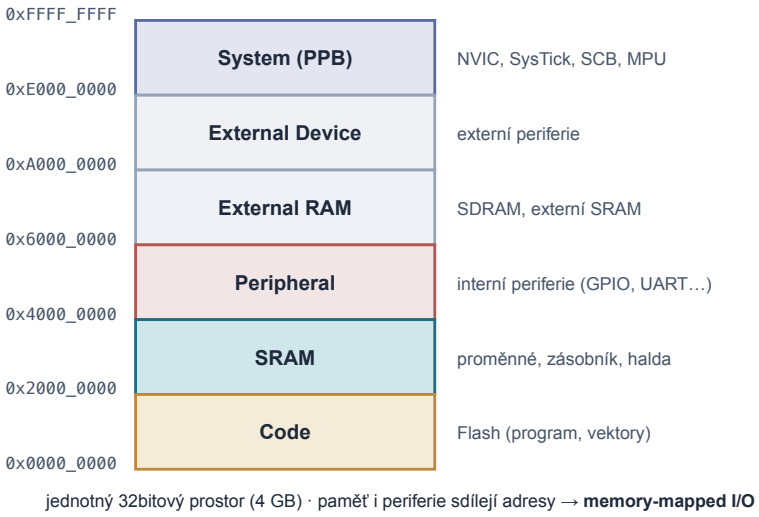


Obrázek 1.30. Režimy procesoru a dva zásobníky MSP/PSP

a uloží na zásobník rámec (`R0`–`R3`, `R12`, `LR`, `PC`, `xPSR`); návratová hodnota `EXC_RETURN` v `LR` pak řízení vrátí zpět (podrobně v přednášce 2). Tohle oddělení je základ, na kterém staví **RTOS** (přednáška 9): operační systém běží na `MSP` v Handler mode, jednotlivé úlohy na `PSP` v Thread mode — a právě zde se zúročí dva zásobníky.

Paměťová mapa, periferie a sběrnice

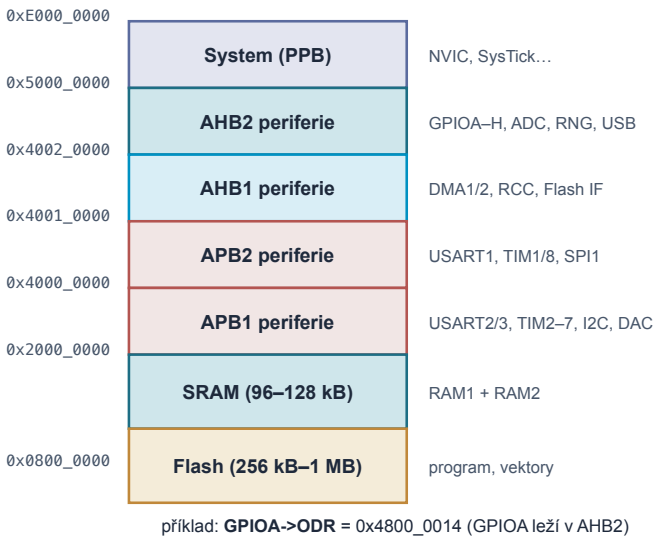
Cortex-M má **jednotný 32bitový adresní prostor** (4 GB), který je architekturou **pevně** rozdělen na oblasti (obr. obr. 1.31):



Obrázek 1.31. Paměťová mapa Cortex-M (ARMv7-M)

Zásadní rys: **paměť i periferie sdílejí týž adresní prostor**. Procesor nemá zvláštní instrukce pro vstup/výstup (na rozdíl od x86 s `IN`/`OUT`) — s periferií se komunikuje **čtením a zápisem na adresu**. Tomu se říká **memory-mapped I/O**. Konkrétní rozložení u STM32L4 ukazuje obr. obr. 1.32:

Memory-mapped I/O
Registry periferií jsou přístupné jako adresy v paměťovém prostoru.



Obrázek 1.32. Paměťová mapa STM32L4

Jak funguje `GPIOA->ODR`

Periferie je sada **registrů** na pevných adresách. V jazyce C ji popíšeme strukturou „přiloženou" na základní adresu:

```
typedef struct {
  volatile uint32_t MODER;  // +0x00
  volatile uint32_t OTYPER;  // +0x04
  volatile uint32_t OSPEEDR; // +0x08
  volatile uint32_t PUPDR;  // +0x0C
  volatile uint32_t IDR;   // +0x10 (vstupní data)
  volatile uint32_t ODR;   // +0x14 (výstupní data)
  // …
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef*) 0x48000000UL)
```

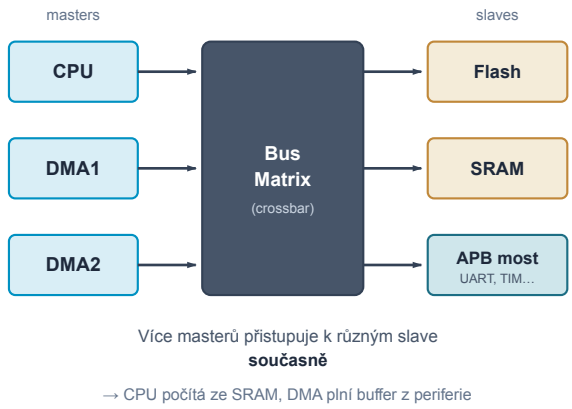
Proto `GPIOA->ODR` ukazuje na adresu `0x48000000 + 0x14 = 0x48000014`. Klíčové je slovo **`volatile`**: říká překladači, že hodnotu může kdykoli změnit hardware, takže přístup **nesmí** optimalizovat pryč ani přeházet. Bez něj by překladač mohl číst stále starou hodnotu, přestože ji hardware mezitím změnil. `volatile` je most mezi jazykem C a křemíkem.

Sběrnice AHB/APB a bit-banding

ARM definuje standard sběrnic **AMBA**. V mikrokontroléru najdeme dvě úrovně:

- **AHB** (Advanced High-performance Bus) — rychlá, pro CPU, DMA, Flash, SRAM a rychlé periferie,
- **APB** (Advanced Peripheral Bus) — jednodušší a pomalejší, pro běžné periferie (UART, I2C, časovače), připojená přes **most** AHB→APB.

Důležitou součástí je **bus matrix** (křížový přepínač), který umožní více „masterům" přistupovat k různým „slave" **současně** (obr. obr. 1.33):



Obrázek 1.33. Bus matrix — paralelní přístup více masterů

Díky tomu může CPU počítat z SRAM, zatímco DMA zároveň plní buffer z periferie — to je klíč k výkonu, ke kterému se vrátíme u DMA (přednáška 5).

Bit-banding je užitečná vlastnost jader M3/M4: každý bit ve vyhrazené oblasti má vlastní **slovo-adresu** v *alias* oblasti, takže zápisem celého slova lze **atomicky** nastavit nebo smazat jediný bit — bez sekvence read-modify-write, do které by mohlo zasáhnout přerušení. Novější

jádra (M33) bit-banding nemají; atomicitu tam zajistí buď registr `BSRR` (nastaví či vynuluje bit GPIO jediným zápisem, bez read-modify-write), nebo obecně instrukce `LDREX`/`STREX` (přednáška 8).

Syntéza: blikání LED (cvičení 1)

Spojme vše do prvního programu cvičení 1 — rozblikání uživatelské LED na `PA5`:

```
// 1) zapni hodiny pro port GPIOA (bez hodinového signálu port nepracuje)
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

// 2) nastav PA5 jako digitální výstup (MODER, 2 bity na pin)
GPIOA->MODER &= ~(3u << (5 * 2));  // vymaž oba bity pinu 5
GPIOA->MODER |= (1u << (5 * 2));  // 01 = výstup

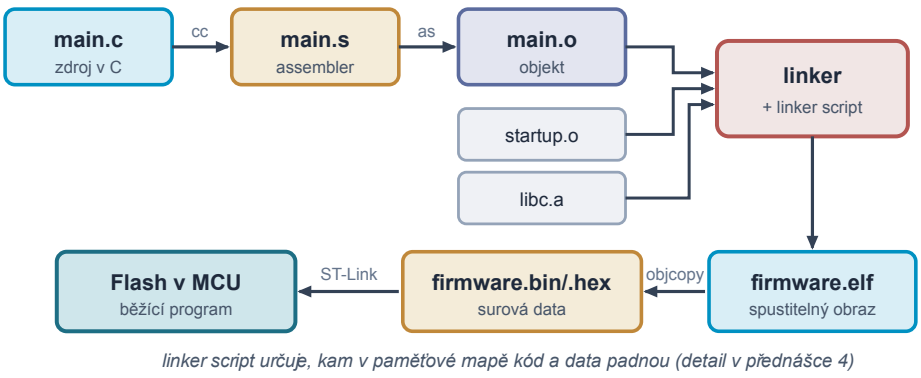
// 3) blikej
while (1) {
  GPIOA->ODR ^= (1u << 5);         // přepni pin PA5
  for (volatile int i = 0; i < 200000; ++i); // hrubé zpoždění
}
```

`PA5` je uživatelská LED na deskách **Nucleo**. Těch několik řádků v sobě nese celou tuto přednášku:

- bez zápisu do `RCC->AHB2ENR` zůstává port bez hodin nečinný — princip synchronního návrhu,
- `GPIOA->ODR` je **memory-mapped registr** na adrese `0x48000014`,
- GPIO je připojené k sběrnici **AHB2**,
- LED drží svůj stav → v periferii GPIO je **klopný obvod**, který výstup uchová.

Od jednoho klopného obvodu jsme dospěli k běžícímu programu na reálném procesoru; všechny mezikroky do sebe zapadají.

Aby ovšem tento `main.c` skutečně **běžel v čipu**, projde ještě **překladovým řetězcem**: překladač C → assembler → objektové soubory → **linker**, který podle *linker scriptu* rozhodne, kam v paměťové mapě kód a data padnou → výsledný binární obraz se **nahraje do Flash** (obr. obr. 1.34). Tím se uzavírá cesta od zdrojového textu k běžícímu hardwaru; podrobně (C → asm → strojový kód, MISRA) v přednášce 3.



Obrázek 1.34. Překladový řetězec: z main.c do běžícího čipu

Shrnutí: ARM Cortex-M je konkrétní RISC procesor, jehož jádro je tatáž datová cesta + řadič jako v oddílu 1.2 (ALU, registry `R0`–`R15`, zřetězení), obalená systémovými periferiemi (NVIC, SysTick, MPU). Programátorský model zahrnuje speciální registry (`SP`, `LR`, `PC`, `xPSR`) a režimy Thread/Handler s `MSP`/`PSP`. Paměť i periferie sdílejí jednotný adresní prostor (**memory-mapped I/O**), periferie jsou připojené ke sběrnicím AHB/APB. Tím je uzavřena cesta od NANDu k běžícímu programu na Cortex-M. (Instrukční sada a srovnání architektur patří do přednášky 7.)

1.5 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 1.1 — Číslicové obvody, hodiny a časování

1. Podle čeho rozlišujeme **kombinační** a **sekvenční** obvod a proč mezi sekvenční nutně patří i jednoduchý čítač? Čím se liší výstupní funkce obou tříd?
2. Proč se v synchronním návrhu smí stav měnit **jen na hraně** společného hodinového signálu? Co v obvodu bez hodin způsobí, že se různě dlouhé cesty ustálí v různý čas (hazardy), a jak z pravidla „změna jen na hraně" plyne **determinismus** a **skládatelnost**?
3. Které **tři parametry** charakterizují časování klopného obvodu D a co každý znamená? Jak z nich a z **kritické cesty** plyne vztah pro maximální hodinovou frekvenci a která cesta — nejdelší, nebo nejkratší — rozhoduje o **setup** a která o **hold**?
4. U časování registru v synchronním obvodu se rozlišují podmínky **setup** a **hold**. Proč lze porušený **setup** napravit zpomalením hodin, ale porušený **hold** nikoli? Jak se porušený hold projeví a jak se odstraňuje?
5. Kdy u klopného obvodu vzniká **metastabilita** a proč ji nelze fyzikálně zcela vyloučit? Jak ji omezuje **dvoustupňový synchronizér**, proč roste MTBF s časem vyhrazeným na ustálení **exponenciálně** a proč tentýž synchronizér **nestačí** pro vícebitovou sběrnici?

K oddílu 1.2 — Od automatu k datové cestě

1. Jaký je strukturální rozdíl mezi Mooreovým a Mealyho automatem a jak se projeví na výstupu (glitche, zpoždění)?
2. Popište čtyři kroky syntézy obvodového řadiče — od stavového diagramu k hardwaru.
3. Co tvoří datovou cestu a co řadič? Jak z nich vznikne procesor (cyklus fetch–decode–execute)?
4. Jaké výhody a jakou cenu má globální hodinový signál? Popište čtyřfázový handshake `req`/`ack` a uveďte, kde se s handshakem v procesoru reálně potkáte.

K oddílu 1.3 — Mikroprogramový řadič a vykonávání instrukcí

1. Srovnejte obvodový a mikroprogramový řadič (rychlost, flexibilita) a přiřaďte je k filozofiím RISC a CISC.
2. Co je řídicí (mikro)slovo a jaký je rozdíl mezi horizontálním a vertikálním mikrokódem?
3. Proč jediný návratový registr (`LR`) nestačí pro vnořená volání a jak to řeší zásobník (LIFO, `SP`, PUSH/POP)?
4. Vysvětlete „dvě úrovně zásobníku" — co řeší μ-zásobník a co programový zásobník a co mají společného?
5. V jakém smyslu je každá instrukce ISA rutinou v mikroprogramu a procesor smyčkou fetch–dispatch–vykonej?

K oddílu 1.4 — ARM Cortex-M: konkrétní procesor

1. V jakém smyslu je jádro ARM Cortex-M tatáž datová cesta řízená řadičem, kterou jsme složili z hradel a registrů? Co v něm poznáte z registrů, ALU a sběrnice?
2. Jak umožňuje memory-mapped I/O ovládat periferie bez zvláštních I/O instrukcí? Co se fyzicky stane při zápisu do registru periferie a proč je nutné `volatile`?
3. Jak se sdílená sběrnice promítá do dvojice **AHB/APB** a k čemu slouží bus matrix?
4. Jak registry se zvláštní rolí (`SP`, `LR`, `PC`) navazují na zásobník a návrat z podprogramu?
5. Na příkladu blikání LED (cvičení 1) vysvětlete, jak se v několika řádcích potká celá kapitola — povolení hodin, memory-mapped registr a klopný obvod ve výstupu pinu.

2

Čas, hodiny a periferie

> Organizujícím tématem této kapitoly je **čas**. Procesor je synchronní obvod — **bez hodinového signálu se v něm neděje vůbec nic**, takže hodiny jsou nutnou podmínkou činnosti *každé* periferie, od jejího spuštění po měření a generování signálů v čase. Zároveň budou v kapitole představeny základní perieferie pro komunikaci procesoru s vnějším světem.

Obsah kapitoly

2.1 Zdroje času a hodinový systém
2.2 Periferie a GPIO
2.3 Přerušovací systém
2.4 Časovače a PWM
2.5 Syntéza: čas drží systém pohromadě
2.6 Kontrolní otázky

2.1 Zdroje času a hodinový systém

Synchronní návrh zaručuje, že se logika mezi dvěma aktivními hodinovými hranami ustálí a teprve další hrana ji „zachytí" do registrů. To dělá chování deterministickým a skládatelným — ale také to znamená, že **hodinový signál řídí veškerou činnost**. Vypneme-li hodiny periferie, její klopné obvody přestanou pracovat a periferie se zastaví; žádný zápis do jejích registrů se neprojeví. Této vlastnosti později využijeme i k úspoře energie (kapitola 11), zde je podstatná jako vysvětlení, proč musí jako úplně první krok při práci s jakoukoli periferií zaznít *„zapni jí hodiny"*.

Čas navíc není jen vnitřní záležitost procesoru — je to **klíčová veličina řídicích a měřicích aplikací**:

Úloha	Realizace v procesoru
řízení výkonu (motor, topení, podsvícení)	**PWM** — poměr zapnuto/vypnuto
měření otáček, doby letu signálu (ToF)	**input capture**
generování přesných impulzů, synchronizace	**output compare**
časová základna OS, plánování úloh	**RTOS tick**, watchdog

Všechny tyto úlohy řeší časovače (oddíl 2.4). Předtím je nutné vědět, *odkud* hodiny pocházejí a jak přesné jsou — protože přesnost času přímo určuje přesnost měření i generování signálů.

Časovače
Čítač, který odměřuje čas a události a generuje nebo měří signály.

Zdroje času — oscilátory

Každý mikrokontrolér STM32 má několik nezávislých zdrojů hodin. Liší se třemi parametry, mezi kterými volíme kompromis: **frekvencí, přesností a spotřebou** (a cenou — externí krystal je součástka navíc).

Zdroj	Frekvence	Přesnost	Typické využití
HSI – High Speed Internal	8–16 MHz	±1–2 %	výchozí po resetu, bez krystalu
HSE – High Speed External	4–26 MHz	±30 ppm	přesné aplikace, USB, měření času
LSI – Low Speed Internal	~32 kHz	±5 %	watchdog, záložní RTC
LSE – Low Speed External	32,768 kHz	±20 ppm	RTC, nízkopříkonové časování
PLL – Phase-Locked Loop	násobí vstup	—	vytváří systémový takt (SYSCLK)

Rozdíl mezi **interním RC oscilátorem** (HSI, LSI) a **externím krystalem** (HSE, LSE) je zásadní. RC oscilátor je integrovaný přímo v čipu, takže nepotřebuje žádnou externí součástku a **startuje velmi rychle** (desítky μs — MCU může běžet téměř okamžitě po resetu). Daní je nepřesnost a teplotní závislost. Krystal je naopak přesný (jednotky až desítky ppm), ale potřebuje **dobu náběhu** (HSE 0,5–5 ms, LSE až 500 ms, protože 32 kHz krystal kmitá pomalu) a zabírá místo na desce.

Interním RC oscilátorem
HSI/LSI: interní RC oscilátory — rychle naběhnou, ale přesnost jen jednotky procent.

Externím krystalem
HSE/LSE: externí krystaly — přesné (desítky ppm) za cenu delší doby náběhu.

Přesnost, kalibrace a stabilita HSI

Protože HSI je po resetu výchozím zdrojem, stojí za to vědět, jak je přesný. Jeho frekvence se mění s teplotou a napětím, a výrobce ho proto **kalibruje** ve výrobě. Kalibrační hodnoty jsou v registru `RCC_ICSCR`:

- `HSICAL[7:0]` — **tovární** kalibrační hodnota (změřená při 25 °C a 3,3 V),
- `HSITRIM[4:0]` — **uživatelské** jemné doladění (rozsah cca ±1 %).

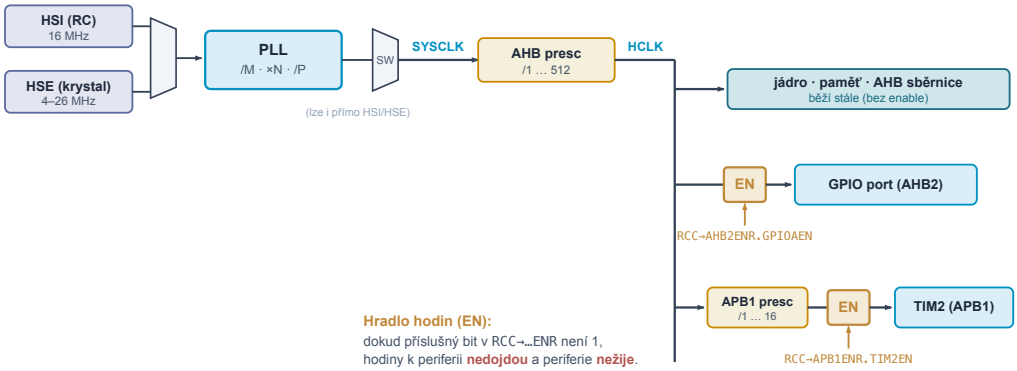
Parametr	Typická hodnota	Komentář
přesnost po kalibraci	±1 %	tovární, při 25 °C
teplotní drift	±0,1 % / 10 °C	při výrazném zahřátí až ±2 %
napěťová závislost	±0,5 %	v rozsahu 1,8–3,6 V
doba náběhu	~100 μs	velmi rychlá oproti krystalu

Pro běžné účely (UART na 115 200 Bd, běžná časování) HSI bohatě stačí. Pro **metrologii** nebo USB (které vyžaduje ±0,25 %) je nutná přesnější reference. Zajímavá technika je *software trimming*: máme-li externí přesný zdroj (např. pulz z GPS nebo LSE krystal), můžeme jeho periodu **změřit přes input capture časovače** a podle odchylky dolaďovat `HSITRIM` za běhu — časovač tu slouží jako měřidlo vlastních hodin.

Input capture
Měření času vnější události: časovač zaznamená hodnotu CNT v okamžiku hrany na pinu.

Hodinový strom a PLL

Jednotlivé zdroje samy o sobě nestačí: jádro Cortex-M4 běží na desítkách až stovkách MHz, což žádný levný oscilátor přímo nedodá. Proto se hodiny vedou přes **hodinový strom** — soustavu multiplexorů, násobičky (PLL) a děliček (prescalerů), které z jednoho zdroje generuje všechny potřebné frekvence:



Obrázek 2.1. Hodinový strom STM32 se zvýrazněnými hradly clock-enable

Tok je následující: jeden ze zdrojů (HSI/HSE) jde do **PLL**, která frekvenci znásobí na plnou rychlost; výstup PLL (nebo přímo HSI/HSE) vybere multiplexor jako **SYSCLK** — hlavní systémový takt. `SYSCLK` se pak přes **AHB prescaler** stane `HCLK` (jádro, paměť, DMA) a dále přes **APB prescalery** napájí periferní sběrnice.

PLL
Phase-Locked Loop: násobička frekvence; z nízké vstupní vytvoří plnou rychlost jádra (SYSCLK).

Klíčový detail, na kterém celá přednáška stojí: **ke každé periferii vede hradlo „clock enable".** Dokud příslušný bit v registru `RCC_…ENR` není nastaven na 1, zůstává hodinový signál k periferii uzavřen a periferie nečinná. Naopak hodiny k jádru, pamětem a sběrnici běží vždy.

> **Nejdříve hodiny, poté konfigurace.** Dokud periferii nepovolíte hodiny v `RCC_…ENR`, žádný zápis do jejích registrů se neprojeví. Když „periferie nefunguje", je nejpravděpodobnější příčinou chybějící clock-enable.

PLL — násobení frekvence

PLL (Phase-Locked Loop, fázový závěs) je obvod, který vstupní frekvenci **vydělí, ve VCO znásobí a znovu vydělí** pro jednotlivé výstupy. Konfiguruje se třemi (resp. čtyřmi) děliteli/násobiteli:

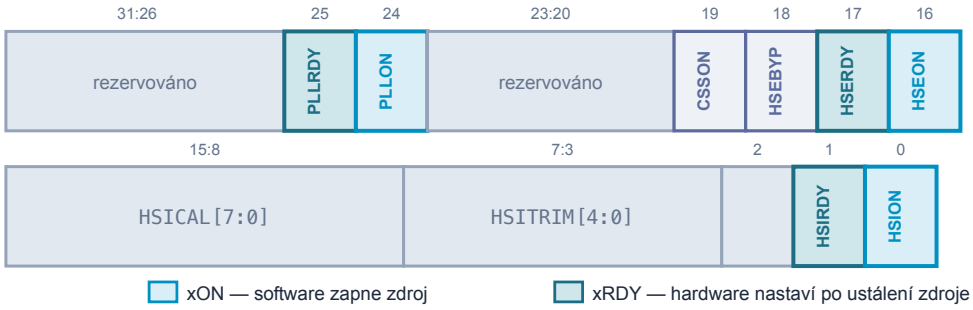
$$f_{VCO} = \frac{f_{IN}}{PLL_M} \times PLL_N, \qquad f_{SYSCLK} = \frac{f_{VCO}}{PLL_P}, \qquad f_{USB} = \frac{f_{VCO}}{PLL_Q}.$$

Význam jednotlivých členů: `PLL_M` sníží vstupní frekvenci na doporučené pásmo VCO (typicky 1–2 MHz, kvůli stabilitě fázového závěsu), `PLL_N` ji ve VCO znásobí na vysokou hodnotu a `PLL_P`/`PLL_Q` ji rozdělí pro jádro a pro USB. Příklad konfigurace registru `RCC_PLLCFGR` pro `HSE = 8 MHz` a cíl `SYSCLK = 168 MHz`:

Parametr	Hodnota	Výpočet
`PLL_M`	8	8 MHz / 8 = 1 MHz (vstup VCO)
`PLL_N`	336	1 MHz × 336 = 336 MHz (VCO)
`PLL_P`	2	336 / 2 = **168 MHz** (SYSCLK)
`PLL_Q`	7	336 / 7 = 48 MHz (USB)

RCC: inicializace, děličky a povolení hodin

Zapnutí každého zdroje hodin má **dvě fáze**, které jsou odděleny dvěma různými bity v registru `RCC_CR`. Bit **`xON`** zdroj *zapne*; bit **`xRDY`** nastaví **hardware sám**, teprve **až je zdroj stabilní**. Na `RDY` se musí čekat — krystal ani PLL nenaběhnou okamžitě:



Obrázek 2.2. Registr RCC_CR — vzor zapni (ON) a počkej (RDY)

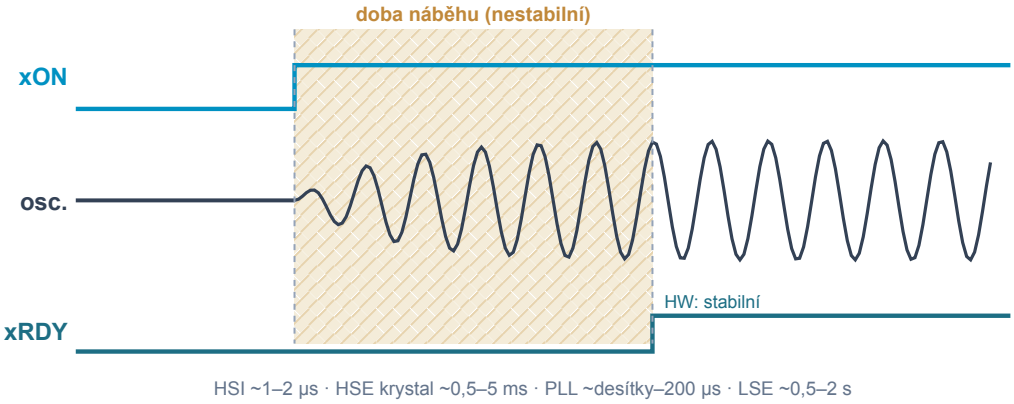
Tento vzor „zapni a počkej" je v práci s hodinami všudypřítomný:

```
RCC->CR |= RCC_CR_HSEON;                 // zapni HSE
while (!(RCC->CR & RCC_CR_HSERDY)) ;     // čekej, až je stabilní
```

Vynechání čekání na `RDY` je častá a obtížně odhalitelná chyba: kdyby se PLL zamkla na ještě neustálený krystal, generovala by nestabilní `SYSCLK` a procesor by se choval nepředvídatelně.

Jak hardware pozná, že je zdroj „ustálený"

Oscilátor po zapnutí nenaběhne skokem — jeho kmity se teprve **rozkmitávají** na plnou amplitudu. Hardware proto po nastavení `xON` čeká: u krystalových zdrojů odpočítá **stabilizační čítač** pevný počet ustálených period, PLL má **detektor zámku** (lock). Teprve když je výstup stabilní, nastaví `xRDY`.



Obrázek 2.3. Náběh oscilátoru — zapnout (ON) a počkat na ustálení (RDY)

Doba náběhu se mezi zdroji liší o několik řádů (tab. 2.1):

Tabulka 2.1. Typická doba náběhu zdrojů hodin

Zdroj	Typická doba náběhu
HSI (interní RC)	~1–2 μs
HSE (krystal)	~0,5–5 ms
PLL (zámek smyčky)	~desítky μs až 200 μs
LSE (32 kHz krystal)	~0,5–2 s

Proto se na `RDY` skutečně **musí** čekat a délka čekání odpovídá zdroji — u HSE/PLL jde o zlomky milisekundy, u LSE až o sekundy (tam se nečeká aktivní smyčkou). HSE má navíc **CSS** (Clock Security System) a startup-timeout: pokud krystal nenaběhne nebo později selže, hardware automaticky přepne systém zpět na HSI a vyvolá `NMI`, takže se procesor nezastaví bez hodin.

Celý postup inicializace

Přechod z výchozího HSI na 168 MHz z PLL vyžaduje **správné pořadí** kroků. Pozor zejména na krok 4 — *wait states* paměti Flash:

```
RCC->CR |= RCC_CR_HSEON;                          // 1) HSE on
while (!(RCC->CR & RCC_CR_HSERDY));

RCC->PLLCFGR = (8 << RCC_PLLCFGR_PLLM_Pos) |      // 2) PLL: M, N, P, zdroj
               (336 << RCC_PLLCFGR_PLLN_Pos) |
               (0 << RCC_PLLCFGR_PLLP_Pos) |      //    00 = /2
               RCC_PLLCFGR_PLLSRC_HSE;
RCC->CR |= RCC_CR_PLLON;                          // 3) PLL on
while (!(RCC->CR & RCC_CR_PLLRDY));

FLASH->ACR = FLASH_ACR_ICEN | FLASH_ACR_DCEN |    // 4) Flash latency
             FLASH_ACR_LATENCY_5WS;

RCC->CFGR |= RCC_CFGR_SW_PLL;                      // 5) SYSCLK = PLL
```

Proč **wait states**: paměť Flash je **pomalejší než jádro**. Na 168 MHz trvá čtení z Flash několik hodinových cyklů, takže procesor musíme nakonfigurovat tak, aby po požadavku na instrukci chvíli počkal (zde 5 *wait states*). Bez toho by jádro četlo instrukci dřív, než ji Flash stačí dodat, a program by havaroval. Toto je obecný motiv, který uvidíme znovu u cache a přístupů do paměti (přednáška 5).

Wait states
vložené čekací cykly, po které jádro čeká na pomalou paměť Flash, než dostane platná data

Děličky sběrnic

Jakmile hodiny jednou běží a `SYSCLK` je stabilní, zbývá je rozvést ke sběrnicím a periferiím. `SYSCLK` se pro jednotlivé sběrnice dělí samostatnými prescalery — periferie tak mohou běžet pomaleji než jádro (a šetřit energii), zatímco jádro běží naplno:

Větev	Registr	Dělička	Napájí
AHB	`RCC_CFGR.HPRE`	1–512	jádro, paměť, DMA, GPIO
APB1	`RCC_CFGR.PPRE1`	1, 2, 4, 8, 16	`TIM2–7`, `USART2–5`, `I2C`
APB2	`RCC_CFGR.PPRE2`	1, 2, 4, 8, 16	`TIM1`, `TIM8`, `ADC`, `USART1`

Zvláštnost frekvence časovačů U časovačů platí jedno překvapivé pravidlo: **je-li dělička APB $\neq$ 1, hodiny časovačů se oproti sběrnici násobí $\times$2.** Důvod je praktický — časovače potřebují co nejvyšší rozlišení pro PWM a měření, takže ST je nechá běžet rychleji, než by odpovídalo dělené sběrnici:

- APB1 dělička 4 → `f_APB1 = 42 MHz`, ale `f_TIMxCLK = 84 MHz`,
- APB2 dělička 2 → `f_APB2 = 84 MHz`, ale `f_TIMxCLK = 168 MHz`.

Na tuto zdvojenou frekvenci je třeba myslet při výpočtu `PSC`/`ARR` (oddíl 2.4) — jinak budou všechny časy dvakrát kratší, než čekáme.

Povolení hodin periferie Tím se vracíme k ústřední myšlence. Než s periferií uděláme cokoli, **povolíme jí hodiny** nastavením bitu v příslušném `RCC_…ENR` — podle toho, ke které sběrnici je připojena:

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;   // GPIO port A (na AHB2)
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;    // časovač TIM2 (na APB1)
RCC->APB2ENR |= RCC_APB2ENR_USART1EN;  // USART1 (na APB2)
```

Tři praktické důsledky:

- bez enable jsou registry periferie „mrtvé" — zápis nemá efekt a čtení často skončí chybou sběrnice (HardFault),
- vypnuté hodiny periferie znamenají **nulovou dynamickou spotřebu** té periferie — tomuto uzavření hodinového hradla se říká **clock gating** a je to hlavní páka řízení spotřeby (přednáška 10),
- po zápisu do `ENR` bývá nutná **krátká prodleva** (několik cyklů), než jsou registry periferie skutečně dostupné.

Stejná páka platí i pro samotné jádro: instrukce `WFI` (Wait For Interrupt) zastaví hodiny procesoru, dokud nepřijde přerušení (například z **EXTI** nebo **RTC**), takže jádro mezi událostmi „spí" a nespotřebovává dynamický proud. Zapínání a vypínání hodin tedy není jen podmínkou funkce periferie, ale zároveň základem řízení spotřeby.

Toto je most do zbytku přednášky: každá periferie, kterou dále potkáme — **GPIO** (oddíl 2.2), **EXTI/NVIC** (oddíl 2.3) i **časovače** (oddíl 2.4) — začíná stejně: povolením hodin v `RCC`.

GPIO
General-Purpose I/O: brána procesoru do světa; každý pin lze nezávisle nakonfigurovat.

Monitorování a diagnostika hodin

Externí krystal (HSE) je přesný, ale jako mechanická součástka může **selhat** — prasklý spoj nebo poškozený rezonátor a hodiny zmizí. Protože na nich závisí celý systém, nabízí STM32 hardwarovou pojistku **Clock Security System** (CSS). Zapne se jediným bitem (`RCC->CR |= RCC_CR_CSSON`) a od té chvíle trvale monitoruje HSE; jakmile oscilace ustane, hardware sám přepne `SYSCLK` zpět na vždy dostupné HSI — systém běží dál, jen s nižší přesností — a vyvolá **NMI**. Obsluha nemaskovatelného přerušení pak může chybu zaznamenat, přejít do bezpečného režimu nebo řízeně resetovat. Je to pojistka pro aplikace, kde výpadek hodin nesmí znamenat tiché zamrznutí, například řízení motoru nebo komunikaci.

Clock Security System
Hardwarový monitor externího oscilátoru HSE. Zmizí-li oscilace, systém automaticky přepne hodiny na interní HSI a vyvolá nemaskovatelné přerušení (NMI), takže výpadek krystalu se stane řízenou událostí místo tichého zamrznutí.

Druhým užitečným nástrojem je **MCO** (Microcontroller Clock Output), který vyvede zvolený vnitřní hodinový signál — `SYSCLK`, `HSE`, `HSI` či výstup PLL — přes nastavitelnou děličku na pin. Osciloskopem na něm se dá přímo změřit, zda hodiny běží na očekávané frekvenci; když se rozběh hodin nedaří, je MCO obvykle prvním diagnostickým krokem, protože rovnou ukáže, jestli PLL naběhla a na jaké frekvenci. Týmž signálem lze i taktovat externí obvod ze společného zdroje.

MCO
Microcontroller Clock Output: vyvedení zvoleného a poděleného vnitřního hodinového signálu na pin. Slouží k ověření frekvence osciloskopem nebo jako zdroj hodin pro jiný obvod.

Shrnutí oddílu 2.1: Procesor je synchronní, takže klíčovou veličinou je čas a nutnou podmínkou činnosti jsou hodiny. Zdroje (HSI/HSE/LSI/LSE) se liší přesností a dobou náběhu; PLL je znásobí na `SYSCLK`, prescalery rozvedou ke sběrnicím. Každý zdroj se zapíná vzorem „nastav `ON` → počkej na `RDY`" a **každá periferie potřebuje povolit hodiny v `RCC_…ENR`**, jinak zůstává nečinná.

2.2 Periferie a GPIO

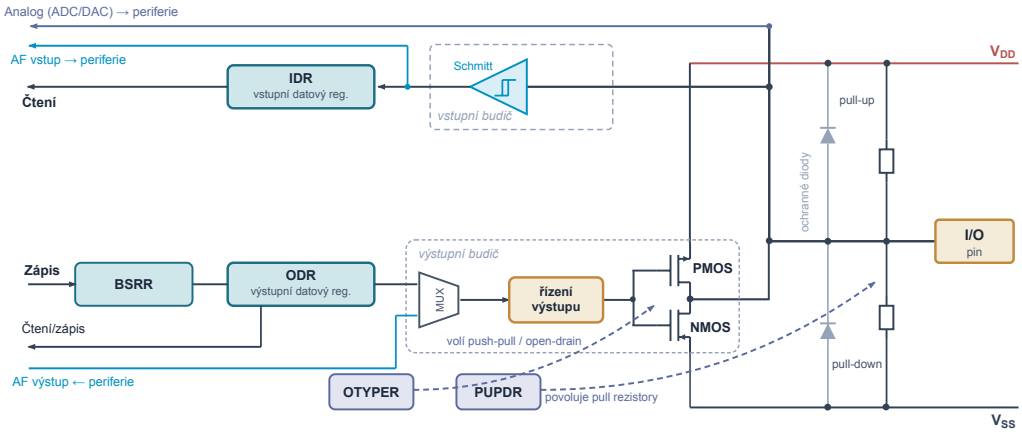
Aby procesor ovlivnil okolní svět a četl z něj, potřebuje vstupně-výstupní piny. Nejuniverzálnější periferií jsou brány GPIO, jejichž každý pin lze nezávisle nakonfigurovat jako vstup, výstup nebo připojit k jiné periferii. Tato část projde vnitřní strukturu pinu, registry, kterými se nastavuje, a bezpečný atomický zápis dat.

GPIO — brána procesoru do světa

GPIO (General-Purpose Input/Output) je nejjednodušší a nejuniverzálnější periferie: každý pin lze nezávisle nastavit jako digitální **vstup** (čtení tlačítka, čidla), digitální **výstup** (řízení, signalizace), připojit k jiné periferii (**alternativní funkce** — UART, časovač, SPI…), nebo použít **analogově** (vstup ADC, výstup DAC).

Alternativní funkce
Režim pinu, kdy ho ovládá vnitřní periferie (UART, časovač, SPI) místo GPIO.

Pod „jedním pinem" se přitom skrývá netriviální obvod. Stojí za to ho znát, protože vysvětluje, proč pin nastavujeme tolika registry:



Obrázek 2.4. Struktura jednoho GPIO pinu

Pin prochází několika cestami a každá má svůj registr:

- **vstupní cesta** — pin → Schmittův klopný obvod (tvaruje pomalou hranu na čistou digitální úroveň) → **synchronizér** (dva klopné obvody z přednášky 1, brání metastabilitě, proto čtení pinu „stojí" několik cyklů) → registr `IDR`,
- **výstupní cesta** — registr `ODR`/`BSRR` → výstupní budič (dva tranzistory) → pin,
- **alternativní funkce** — výstup/vstup vnitřní periferie (TIM, UART…) je multiplexorem připojen na pin,
- **analogová cesta** — pin je veden přímo do ADC/DAC, mimo digitální budiče.

Kromě toho jsou na pinu **ochranné diody** (svedou přepětí do napájecích railů) a volitelné **pull-up/pull-down rezistory**. Která z cest je aktivní, určuje registr `MODER`. A jako vždy: než s GPIO uděláme cokoli, musíme mu povolit hodiny (`RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;`, oddíl 2.1).

Konfigurace pinu: režim, typ, rychlost a pull

Čtyři registry postupně určí, co pin dělá a jak: `MODER` vybere cestu, `OTYPER` typ výstupního budiče, `OSPEEDR` rychlost hrany a `PUPDR` vnitřní rezistory. Projdeme je v tomto pořadí.

Režim pinu — registr `MODER`

Registr `MODER` určuje režim každého pinu **dvěma bity** — 16 pinů tedy zabere 32 bitů:



Obrázek 2.5. Registr GPIOx_MODER — 2 bity na pin

`MODER[2n+1:2n]`	Režim	Použití
`00`	**vstup**	čtení tlačítka, čidla
`01`	**výstup**	řízení, signalizace
`10`	**alternativní funkce**	pin patří periferii (UART, TIM, SPI…)
`11`	**analogový**	vstup ADC, výstup DAC

Protože jsou bity dva na pin, je obvyklý idiom „vymaž oba bity, pak nastav nový režim”:

```
GPIOA->MODER &= ~(3u << (5 * 2));   // vymaž oba bity pinu PA5
GPIOA->MODER |=  (1u << (5 * 2));   // 01 = výstup
```

Nejdůležitější je režim **alternativní funkce** (`10`): jím se pin „odevzdá” vnitřní periferii. Bez něj by se signál časovače nebo UARTu nikdy nedostal na fyzický pin. *Který* z až šestnácti alternativních významů to bude, určuje samostatný registr `AFR` (dále v této části).

Stav po resetu. Většina pinů má po resetu `MODER = 00`, tedy je **vstupem** s vysokou impedancí. To je důležité bezpečnostní chování: výstupní budič je vypnutý, pin nedrží žádnou úroveň a **neprotéká jím žádný proud** (kromě nepatrných svodů) — po zapnutí napájení tak žádný pin nechtěně nebudí připojený obvod. Nejnižší spotřebu *nepoužitého* pinu dá **analogový** režim (`11`), protože odpojí i vstupní Schmittův klopný obvod, takže nemůže vznikat ani příčný proud při „plovoucí” úrovni blízko prahu. Jedinou výjimkou z resetového vstupu jsou **ladicí piny** SWD/JTAG (PA13/14/15, PB3/4), které startují rovnou v alternativní funkci, aby byl čip po resetu programovatelný.

Typ výstupu — `OTYPER`: push-pull vs. open-drain

Push-pull
Výstup aktivně budí obě úrovně (0 i 1); nízká impedance v obou stavech.

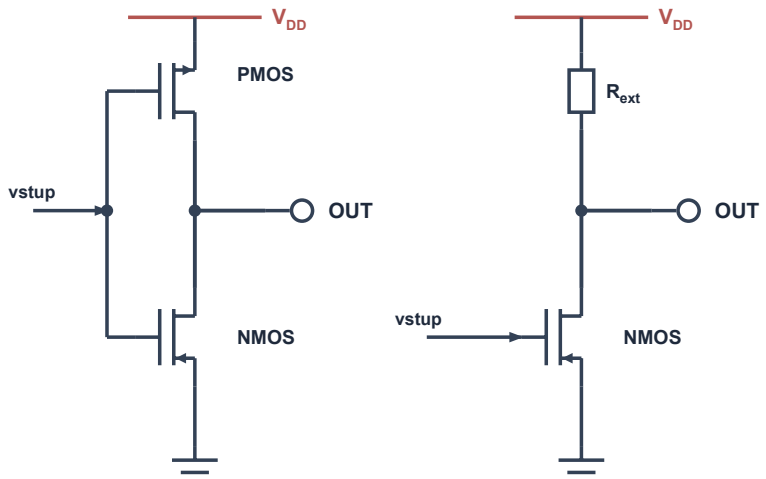
Open-drain
Výstup táhne jen k 0, úroveň 1 zajistí pull-up; umožní sdílenou sběrnici (I2C).

Je-li pin výstup, registr `OTYPER` volí jedním bitem na pin typ výstupního budiče (viz obr. 2.6). Rozdíl je v tom, **na jaké úrovně pin aktivně táhne**:

Push-pull (`0`, výchozí) má dva tranzistory — horní táhne pin na log. 1, dolní na log. 0. Pin tedy aktivně budí **obě** úrovně, je rychlý a hodí se pro většinu digitálních výstupů.

Open-drain (`1`) má jen dolní tranzistor — pin umí aktivně stáhnout na **0**, ale log. 1 musí zajistit **externí pull-up** rezistor. Vypadá to jako omezení, ale je to nutné tam, kde více zařízení

sdílí jeden vodič: každé může vodič stáhnout na 0, a když nikdo netáhne, pull-up ho vrátí na 1. Tak funguje sběrnice **I2C** a používá se to i pro **převody napěťových úrovní**.



(a) Push-pull (b) Open-drain

Obrázek 2.6. Výstupní budiče GPIO.

Rychlost a pull rezistory — `OSPEEDR`, `PUPDR`

Registr `OSPEEDR` nastavuje **rychlost přeběhu** (slew rate) výstupu. Je to kompromis: rychlejší hrana umožní vyšší frekvence, ale produkuje **více elektromagnetického rušení (EMC)** a vyšší spotřebu. Pravidlo je volit nejnižší rychlost, která ještě stačí — pro pomalé výstupy je „low speed" víc než dost, rychlá hrana se hodí jen pro vysokofrekvenční signály (SPI, hodinové výstupy).

Registr `PUPDR` (opět 2 bity na pin) zapíná vnitřní **pull-up** (`01`) nebo **pull-down** (`10`) rezistor. Nejčastěji se používá u **vstupů**, aby měl pin definovanou klidovou úroveň: tlačítko spojené proti zemi potřebuje pull-up, jinak by nepřipojený vstup „plaval" a četl náhodný šum. Vnitřní pull (typicky desítky kΩ) přitom ušetří externí součástku:

```
// tlačítko na PC13: vstup s pull-up → klid = 1, stisk = 0
GPIOC->MODER &= ~(3u << (13*2));   // 00 vstup
GPIOC->PUPDR &= ~(3u << (13*2));
GPIOC->PUPDR |=  (1u << (13*2));   // 01 pull-up
```

Data a atomický zápis — `IDR`, `ODR`, `BSRR`

Za běhu se s pinem pracuje přes datové registry:

Registr	Přístup	Význam
`IDR` (Input Data)	**jen čtení**	okamžitý stav vstupních pinů
`ODR` (Output Data)	čtení i zápis	hodnota na výstupních pinech

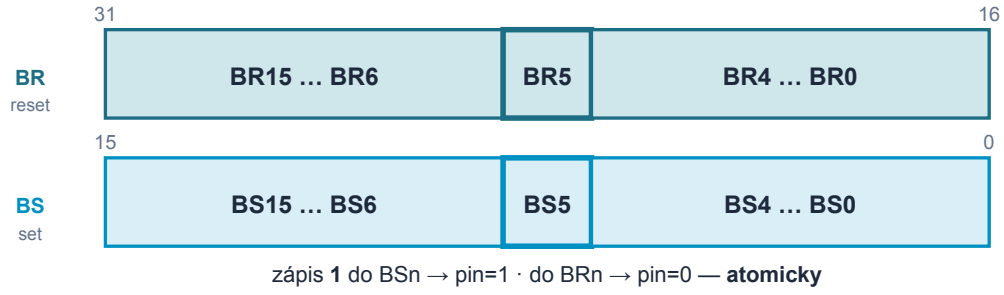
```
uint8_t stisk = !(GPIOC->IDR & (1u << 13));   // čti pin PC13
GPIOA->ODR ^= (1u << 5);                       // přepni PA5
```

Čtení vstupu přes `IDR` má přitom jednu jemnost, kterou hardware řeší za nás. Pin je vůči hodinám jádra **asynchronní** — signál (třeba z tlačítka) může přijít v libovolném okamžiku.

Kdyby ho `IDR` vzorkoval přímo, hrozila by metastabilita (přednáška 1). Proto je mezi pinem a `IDR` vestavěný **dvoustupňový synchronizér** — tytéž dva klopné obvody za sebou, které jsme kreslili u metastability, jádro je má na každém vstupu. Změna pinu se proto v `IDR` objeví až po **jednom až dvou taktech** jádra; čtení GPIO tedy není „okamžité", ale právě díky tomu se s metastabilitou v aplikaci setkáte jen zřídka. Pro **sběrnici (více bitů najednou)** ovšem dvoustupňový synchronizér nestačí — jednotlivé bity se mohou ustálit v různých taktech a vznikne nekonzistentní slovo; tam je nutný **handshake** nebo **FIFO** (přednáška 1).

Zápis do `ODR` má ale skrytý problém: je to **`read-modify-write`** — procesor přečte celé slovo, změní jeden bit a zapíše zpět. Mezi čtením a zápisem může přijít **přerušení**, které změní jiný bit téhož portu; náš následný zápis pak tuto změnu **přepíše** a ztratíme ji. Je to klasický souběh (race condition).

Řešením je registr **`BSRR`** (Bit Set/Reset Register), který mění piny **bez read-modify-write**: jeho horních 16 bitů (`BR`) bity **nuluje**, dolních 16 (`BS`) **nastavuje** — a to **jediným zápisem, atomicky**:



Obrázek 2.7. Registr GPIOx_BSRR — atomický set/reset

```
GPIOA->BSRR = (1u << 5);         // nastav PA5 = 1   (atomicky)
GPIOA->BSRR = (1u << (5 + 16));  // nuluj  PA5 = 0   (atomicky)
```

Protože jde o jediný zápis celého slova, přerušení jej nemůže rozdělit a nelze jím pokazit sousední piny. (Je-li současně `BS` i `BR` téhož pinu, má přednost set.) Souvisí to s bit-bandingem z přednášky 1 — obojí řeší **atomický přístup k jednomu bitu**.

Alternativní funkce — AFR

Když je pin v režimu `MODER = 10` (alternativní funkce), registr **`AFR`** (4 bity na pin, tedy hodnoty AF0–AF15) vybere, **která** vnitřní periferie pin ovládá:

```
GPIOA->MODER &= ~(3u << (5*2));
GPIOA->MODER |=  (2u << (5*2));    // 10 = alternativní funkce
GPIOA->AFR[0] &= ~(0xFu << (5*4));
GPIOA->AFR[0] |=  (1u   << (5*4)); // AF1 = TIM2_CH1
```

Tím se výstup kanálu časovače `TIM2_CH1` (tj. PWM) dostane na fyzický pin PA5. Mapování kódů AF na konkrétní periferie najdeme v **datasheetu** dané součástky (ne v reference manuálu) — liší se pin od pinu, protože každý pin má jinou sadu možných funkcí. Přesně takto v oddílu 2.4 přivedeme PWM z časovače na pin.

Konfigurace GPIO — kompletní příklad

Spojme vše do programu, který rozsvěcí LED na PA5 při stisku tlačítka na PC13 — od hodin až po atomický zápis:

```
// 1) hodiny obou portů
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOCEN;

// 2) PA5 = výstup push-pull
GPIOA->MODER = (GPIOA->MODER & ~(3u<<(5*2))) | (1u<<(5*2));

// 3) PC13 = vstup s pull-up
GPIOC->MODER &= ~(3u<<(13*2));
GPIOC->PUPDR = (GPIOC->PUPDR & ~(3u<<(13*2))) | (1u<<(13*2));

// 4) tlačítko řídí LED (atomicky přes BSRR)
while (1) {
    if (!(GPIOC->IDR & (1u<<13)))  GPIOA->BSRR = (1u<<5);        // stisk → LED on
    else                           GPIOA->BSRR = (1u<<(5+16));   // klid  → LED off
}
```

Tato smyčka funguje, ale je to **polling** — procesor opakovaně čte stav tlačítka a nevykonává jinou užitečnou práci. Lepší je nechat se vyrušit, až se něco stane. Tím se dostáváme k přerušovacímu systému.

> **Shrnutí oddílu 2.2:** GPIO je sada cest (vstup/výstup/AF/analog), které vybírá `MODER`. Výstup je push-pull nebo open-drain (`OTYPER`), vstup mívá vnitřní pull (`PUPDR`). Data se čtou z `IDR`, zapisují do `ODR`, ale pro bezpečnou změnu bitů slouží **atomický `BSRR`**. Alternativní funkce (`AFR`) propojí pin s vnitřní periferií — tak se k pinu dostane i PWM časovače.

2.3 Přerušovací systém

Obsluha tlačítka pollingem z předchozí části plýtvá výkonem i energií. Efektivnější je nechat se hardwarem vyrušit, teprve když nastane událost. Tato část vysvětlí, co se při přerušení odehraje, jak řadič NVIC vybírá a řadí obsluhy a jak se signál z pinu dostane přes EXTI až k jádru.

Od pollingu k přerušení

Na konci oddílu 2.2 jsme tlačítko obsluhovali **pollingem** — procesor v nekonečné smyčce opakovaně zjišťuje, zda událost nastala. To má dvě nevýhody: **plýtvá výkonem i energií** (CPU nedělá nic užitečného, jen kontroluje) a **reaguje pozdě** (událost se zpracuje až při příští kontrole). U jediného tlačítka to nevadí, ale u systému s mnoha periferiemi je to neúnosné.

Přerušení (interrupt) obrací princip řízení: procesor vykonává svou úlohu a událost aktivně nesleduje; teprve když nastane, ho hardware **vyruší**, nechá vykonat krátkou obsluhu a vrátí se zpět. Program je tak **řízen událostmi**, ne neustálým dotazováním.

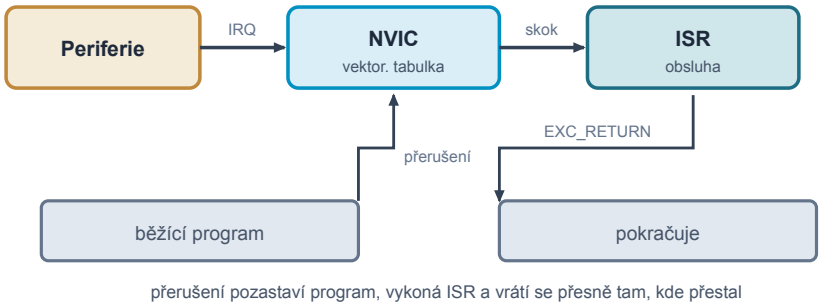
Co se při přerušení stane

Přerušení je hardwarový signál, který pozastaví běžící program, nechá procesor vykonat **obslužnou rutinu** (ISR, Interrupt Service Routine) a pak se vrátí přesně tam, kde přestal — jako by se nic nestalo.

Na Cortex-M se přitom odehraje pozoruhodně mnoho **hardwarově**, bez jediné instrukce programu. Při příchodu žádosti procesor:



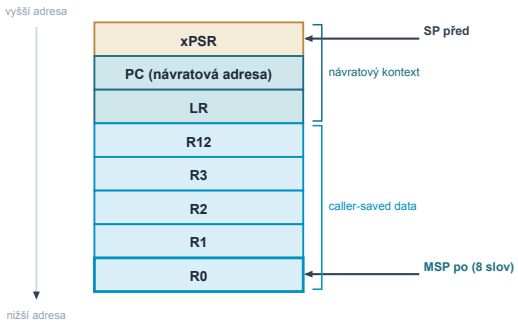
Obrázek 2.8. Polling vs. přerušení



Obrázek 2.9. Tok přerušení: periferie → NVIC → vektor → ISR → návrat

1. **automaticky uloží na zásobník 8 registrů** — `R0`–`R3`, `R12`, `LR`, `PC`, `xPSR` (~12 cyklů); to jsou právě caller-saved registry podle konvence AAPCS, takže ISR může bez obav volat běžné funkce,
2. přepne se z **Thread mode** do **Handler mode** (a používá `MSP`),
3. nahraje do `LR` speciální hodnotu **`EXC_RETURN`** (`0xFFFFFFFx`) a skočí na adresu ISR z vektorové tabulky.

Při návratu (`BX LR`) hardware tuto hodnotu `EXC_RETURN` dekóduje, **obnoví uložených 8 registrů** a vrátí se do Thread mode. Proto se ISR píše jako úplně obyčejná funkce `void f(void)` — žádný speciální atribut není potřeba.



Obrázek 2.10. Rámec přerušení — 8 registrů uložených hardwarem na zásobník

Na rámci stojí za povšimnutí jeho **složení**: jsou to právě registry, které smí podle AAPCS přepsat volaná funkce (`R0`–`R3`, `R12` jako caller-saved data) plus návratový kontext (`LR`, `PC`,

xPSR). Hardware tedy uloží přesně to, co je potřeba, aby ISR mohla běžet jako běžný kód — a callee-saved registry (R4–R11) si případně uloží sama obsluha, stejně jako každá funkce. Rámec leží na MSP (v Handler mode se vždy používá hlavní zásobník).

Rozdíl oproti běžnému volání funkce je podstatný:

	Funkce (BL/BLX)	ISR (přerušení)
Spuštění	**synchronní** — program ji zavolá	**asynchronní** — hardware při události
Uložení kontextu	ručně (AAPCS: caller/callee)	**hardware** uloží 8 registrů (~12 cyklů)
Režim	zůstává Thread/Handler	přepne **Thread** → **Handler**
Zásobník	MSP nebo PSP	MSP (Handler mode)
Návrat	BX LR	BX LR s **EXC_RETURN**, dekóduje HW

V terminologii ARM je nadřazený pojem **exception (výjimka)** — sem patří interní události (Reset, NMI, HardFault, SVC, PendSV, **SysTick**) i externí **přerušení (IRQ)** od periferií. „Přerušení" je tedy podmnožina výjimek; v praxi se ale oba pojmy zaměňují, protože obojí spustí ISR.

Vektorová tabulka a NVIC

Jak procesor pozná, **kterou** ISR spustit? Na začátku paměti leží **vektorová tabulka** — pole adres obslužných rutin, jedna položka na každý zdroj výjimky:

Vektorová tabulka
Tabulka adres obsluh; jádro podle čísla výjimky skočí na příslušnou ISR.

```
adresa     obsah
0x0000     počáteční hodnota SP
0x0004     Reset_Handler          ← start programu
0x0008     NMI_Handler
0x000C     HardFault_Handler
  …        SVC, PendSV, SysTick …
0x0058     EXTI15_10_IRQHandler   ← přerušení od tlačítka
  …        TIM2_IRQHandler, USART2_IRQHandler, …
```

Při události najde řadič přerušení v tabulce adresu příslušné ISR a skočí na ni. Tabulku lze přemístit registrem SCB->VTOR — toho využívá bootloader (přednáška 1).

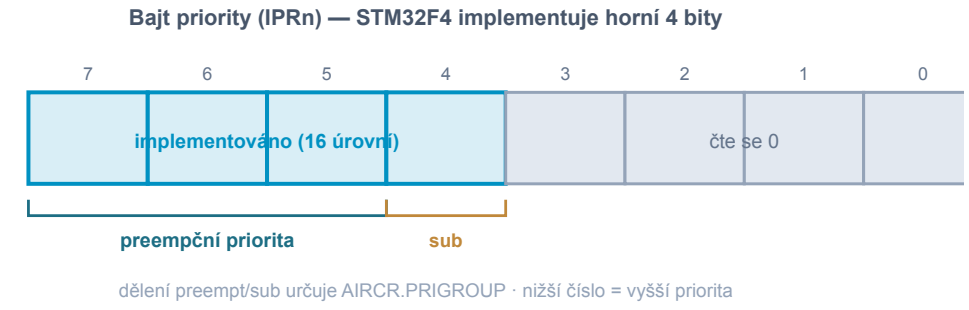
Řadičem je **NVIC** (Nested Vectored Interrupt Controller) — hardwarová jednotka přímo v jádře Cortex-M. Jeho vlastnosti dávají jméno: **Nested** (přerušení mohou přerušovat jiná podle priority), **Vectored** (každý zdroj má vlastní handler), **prioritní** (na STM32F4 16 úrovní), **deterministický** (fixní latence ~12 cyklů) a s optimalizacemi *tail-chaining* a *late-arrival* (níže). Ovládá se přes funkce CMSIS:

NVIC
Nested Vectored Interrupt Controller: řadí priority a vnořování přerušení.

```
NVIC_SetPriority(EXTI15_10_IRQn, 2);   // priorita (nižší číslo = vyšší)
NVIC_EnableIRQ(EXTI15_10_IRQn);        // povol zdroj
// příznak ve zdroji (EXTI->PR1) se maže až v ISR
```

Priorita, preempce a latence

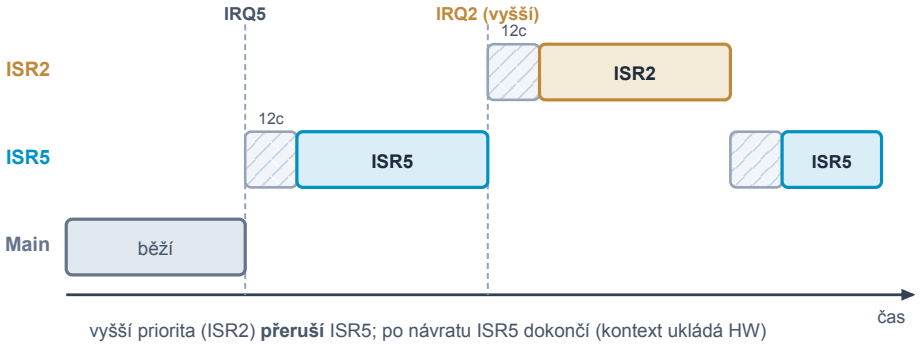
Nižší číslo priority znamená **vyšší** prioritu. Priorita se navíc dělí na **preempční** (rozhoduje o tom, zda jedno přerušení smí vnořit do druhého) a **sub-prioritu** (pořadí při shodě). Subtilní detail, který studenty mate: bajt priority má sice 8 bitů, ale na STM32F4 jsou **implementované jen horní 4** — spodní se čtou jako 0:



Obrázek 2.11. Bajt priority NVIC — implementované horní bity, dělení preempt/sub

Dělení na preempční a sub-prioritu určuje pole `AIRCR.PRIGROUP`. Přijde-li během obsluhy přerušení jiné s **vyšší preempční prioritou**, NVIC tu probíhající **přeruší** (**preempce**) a vrátí se k ní později:

Preempce
pozastavení probíhající ISR příchodem přerušení s vyšší preempční prioritou; po jeho obsloužení se původní ISR dokončí

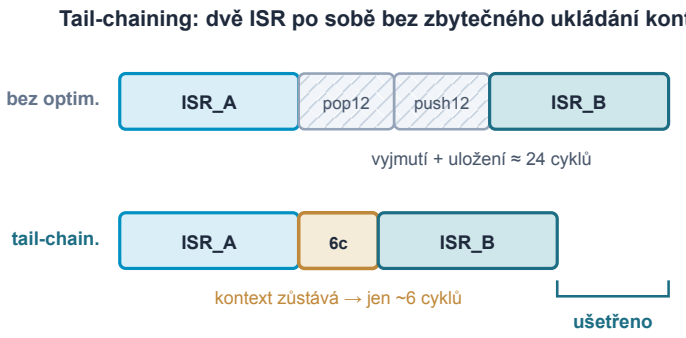


Obrázek 2.12. Preempce v čase

Hlavní program běží, IRQ5 ho přeruší; během obsluhy ISR5 přijde IRQ2 s vyšší prioritou, takže ISR5 je preemptována, vykoná se ISR2 a teprve pak se dokončí ISR5 a vrátí řízení do hlavního programu. Hloubka vnoření je omezena jen počtem úrovní a velikostí zásobníku.

NVIC navíc latenci snižuje dvěma optimalizacemi:

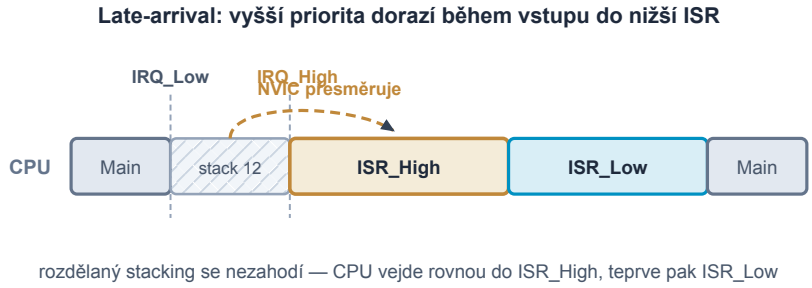
- **tail-chaining** — čekají-li dvě přerušení po sobě, neukládá a neobnovuje kontext dvakrát zbytečně; mezi ISR přejde rovnou (~6 cyklů místo plného unstacking + stacking),



Obrázek 2.13. Tail-chaining

- **late-arrival** — dorazí-li během *vstupu* do nižší ISR vyšší priorita, NVIC obslouží napřed

ji (uložení kontextu už beztak probíhá, jen se změní cíl skoku).



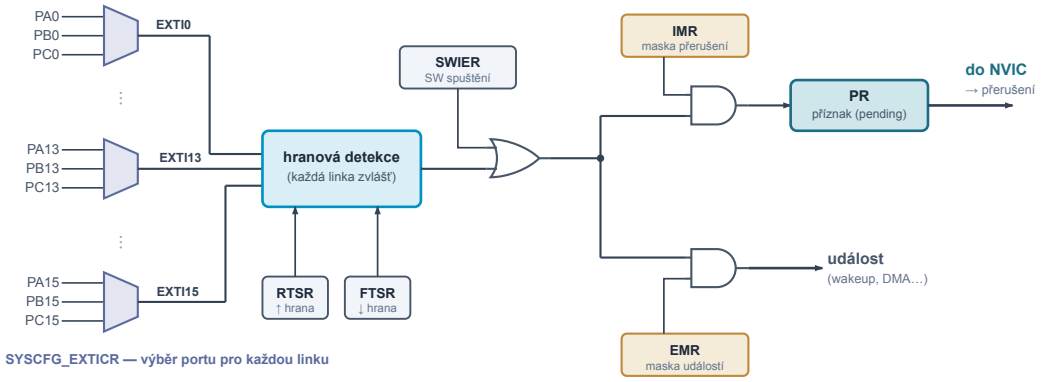
Obrázek 2.14. Late-arrival

Obě optimalizace drží **latenci přerušení nízkou a deterministickou** — což je přesně to, co potřebují systémy reálného času (přednáška 9).

EXTI — přerušení z GPIO pinu

Pin GPIO sám o sobě přerušení negeneruje. Vede přes samostatný řadič **EXTI** (External Interrupt/Event Controller), který hlídá hranu signálu a teprve on žádá NVIC:

EXTI
External Interrupt: spojí hranu na GPIO pinu s přerušením jádra.



Obrázek 2.15. Cesta přerušení z pinu přes EXTI

Konfigurace má několik kroků, protože více pinů sdílí jednu „linku”: linky 0–15 odpovídají **číslu pinu** (PA13, PB13, PC13 sdílejí linku 13), a multiplexor `SYSCFG->EXTICR` vybírá, **ze kterého portu** se linka odebírá:

```
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;          // hodiny SYSCFG (!)
SYSCFG->EXTICR[3] |= SYSCFG_EXTICR4_EXTI13_PC; // linka 13 ← port C
EXTI->IMR1  |= (1u<<13);                        // odmaskuj linku 13
EXTI->FTSR1 |= (1u<<13);                        // reaguj na sestupnou hranu (stisk)
NVIC_EnableIRQ(EXTI15_10_IRQn);
```

Psaní ISR, `volatile` a kritické sekce

ISR běží „nad" hlavním programem a po dobu svého běhu blokuje přerušení nižší priority. Proto musí být **krátká a rychlá**: udělat jen nejnutnější (nastavit příznak, uložit vzorek), těžkou práci přenechat hlavní smyčce a **smazat příznak** ve zdroji — jinak se ISR po návratu hned spustí znovu. Naopak do ISR nepatří dlouhé výpočty, čekací smyčky ani `printf`.

```
volatile uint8_t flag;
void EXTI15_10_IRQHandler(void){
  EXTI->PR1 = (1u<<13);   // smaž příznak (zápis 1)
  flag = 1;               // signál hlavní smyčce
}
```

Proměnnou sdílenou mezi ISR a hlavním programem je nutné označit **`volatile`**. Bez něj by si překladač mohl hodnotu „zapamatovat" v registru a změna z ISR by se v hlavní smyčce nikdy neprojevila — typicky by se z `while (!flag)` stala nekonečná smyčka:

```
volatile uint8_t flag = 0;            // mění ISR, čte main
while (1) { if (flag) { flag = 0; obsluha(); } }
```

Pro **víceslovní** data `volatile` nestačí — hrozí, že ISR zasáhne uprostřed úpravy a hlavní program přečte napůl změněnou hodnotu. Pak je nutná **kritická sekce**, po jejíž dobu přerušení dočasně zakážeme:

```
__disable_irq();      // PRIMASK = 1 – zakáže všechna přerušení
// ... krátká operace nad sdílenými daty ...
__enable_irq();       // PRIMASK = 0
```

Kritickou sekci je třeba držet **co nejkratší**, protože po její dobu rostou latence všech přerušení. Jemnějším nástrojem je registr `BASEPRI`, který zakáže jen přerušení **pod** zvolenou prioritou — kritická (vysoce prioritní) přerušení tak projdou i během sekce. Tytéž problémy souběhu uvidíme ve velkém u vláken a RTOS (přednášky 8–9).

SysTick — systémový časovač jádra

Kromě periferních časovačů (`TIMx`, oddíl 2.4) má **každé** jádro Cortex-M vestavěný vlastní **SysTick** — jednoduchý **24bitový sestupný čítač**. Po dočítání do nuly vyvolá výjimku `SysTick` a automaticky se znovu nahraje z registru `LOAD`:

```
SysTick->LOAD = (SystemCoreClock / 1000) - 1;   // tik po 1 ms
SysTick->VAL  = 0;
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk       // hodiny jádra
              | SysTick_CTRL_TICKINT_Msk         // povol výjimku
              | SysTick_CTRL_ENABLE_Msk;         // spusť
```

SysTick
System Tick Timer: jednoduchý 24bitový sestupný čítač, který je součástí jádra Cortex-M (ne periferie na sběrnici). Po dočítání do nuly vyvolá výjimku SysTick a znovu se nahraje; typicky tvoří systémový tik pro odměřování času a plánování úloh.

Dvě vlastnosti ho odlišují od periferních časovačů. Za prvé je **součástí jádra**, ne periferie na sběrnici, takže funguje na každém Cortex-M stejně bez ohledu na výrobce (a nepotřebuje povolení hodin v `RCC`). Za druhé je záměrně **minimalistický** — jen čítač, práh a přerušení. Typicky z něj vzniká **systémový tik**, například po 1 ms: v obsluze `SysTick_Handler()` se inkrementuje čítač milisekund, který pak slouží k odměřování zpoždění, časovým značkám nebo plánování. Právě tento periodický tik je **srdcem plánovače RTOS** (přednáška 9): tam z něj bude časová základna přepínající úlohy, zde je to zatím jen pravidelné přerušení jádra.

> **Shrnutí oddílu 2.3:** Přerušení nahrazují plýtvavý polling reakcí na událost. Na Cortex-M ukládá kontext (8 registrů) hardware a přepíná do Handler mode; `NVIC` podle vektorové tabulky skočí do správné ISR. Priority řídí preempci (vnořování), tail-chaining a late-arrival drží latenci nízkou. GPIO přerušení vede přes `EXTI`. Sdílená data vyžadují `volatile` a u víceslovních hodnot kritickou

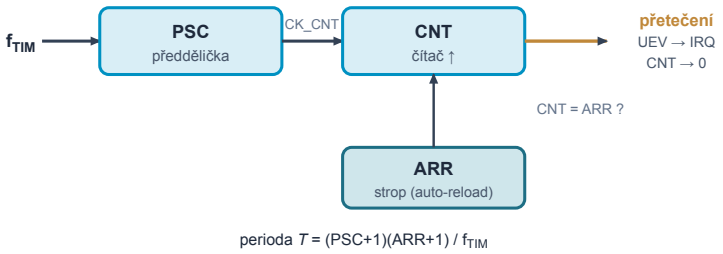
sekci.

2.4 Časovače a PWM

Vrátíme se k ústřední veličině přednášky — času — a k periferii, která ho měří i generuje. Časovač je čítač hodin, z něhož vzniká periodické přerušení, signál PWM i měření vnějších událostí, a to bez zatížení jádra. Tato část projde jeho jádro (PSC, CNT, ARR), řídicí registry a komparační a záchytný systém.

Časovač je čítač s periodou

Vrátíme se k myšlence z oddílu 2.1: procesor je synchronní a vše měří hodinami. **Časovač (timer)** je periferie, která tyto hodiny **počítá** a v pravidelných okamžicích generuje událost — bez zatížení CPU. Je to tedy nejpřímější způsob, jak v synchronním systému „udělat čas viditelným".



Obrázek 2.16. Bloková struktura časovače: PSC → CNT → ARR

Tři registry tvoří jádro:

- `PSC` (Prescaler) — předdělička vstupní frekvence; čítač tiká rychlostí `f_TIM / (PSC+1)`,
- `CNT` (Counter) — vlastní čítač, inkrementuje se každým (před)tikem,
- `ARR` (Auto-Reload Register) — strop; při dosažení `ARR` čítač **přeteče**, vynuluje se a vznikne **update event** (UEV).

Perioda přetečení je tedy

$$T = \frac{(PSC + 1)\,(ARR + 1)}{f_{TIM}}.$$

Příklad: `f_TIM = 84 MHz`, chceme `1 ms`. Zvolíme `PSC = 8399` (čítač tiká 10 kHz) a `ARR = 9` → `T = 8400·10 / 84\,000\,000 = 1 ms`. Dvojice `(PSC, ARR)` dává obrovskou volnost — tutéž periodu lze složit mnoha způsoby; jemné rozlišení PWM chce malou `PSC` a velkou `ARR`.

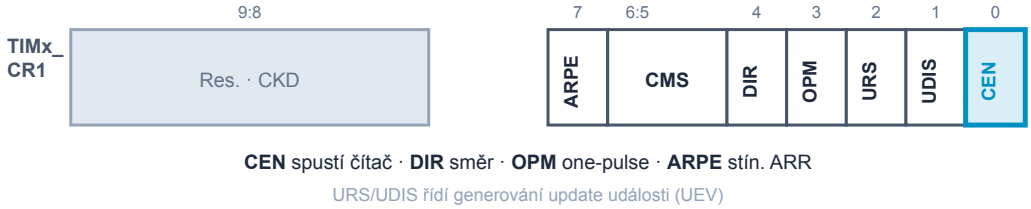
> **Pozor na ×2:** je-li sběrnice APB dělená (`PPRE > 1`), násobí se hodiny časovačů na této sběrnici dvěma — `TIMxCLK = 2 × PCLK`. Je to častý zdroj „dvojnásobně rychlých" časovačů (oddíl 2.1).

Řízení čítače: `CR1`, `SR` a stínové registry

Jádro časovače řídí tři registry: `CR1` spouští a nastavuje čítač, `SR` hlásí události a stínové (preload) registry zajišťují bezzákmitovou změnu za běhu.

Řízení čítače — registr CR1

Chování čítače řídí registr `CR1`:



Obrázek 2.17. Registr TIMx_CR1

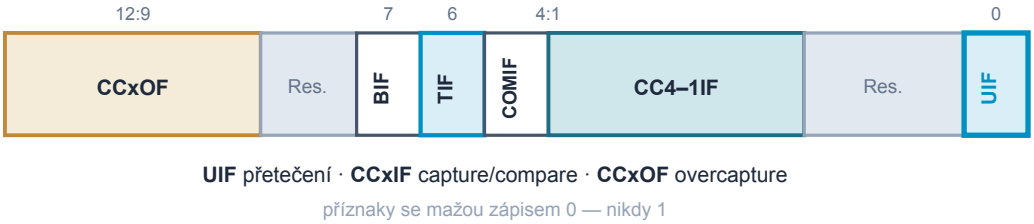
Bit	Význam
`CEN`	**Counter Enable** — spustí/zastaví čítač
`DIR`	směr (0 = nahoru, 1 = dolů)
`OPM`	**One-Pulse Mode** — po jednom přetečení se sám zastaví
`URS`	zdroj update události (jen přetečení vs. i softwarový `UG`)
`UDIS`	dočasně zakáže generování UEV
`ARPE`	**Auto-Reload Preload** — zapne stínový registr `ARR` (viz níže)

Nejjednodušší použití — periodické přerušení po 1 ms — je tedy „nastav PSC/ARR, zapni přerušení, spusť `CEN`“:

```
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
TIM2->PSC = 8399;  TIM2->ARR = 9;     // 1 ms při 84 MHz
TIM2->DIER |= TIM_DIER_UIE;           // povol update přerušení
NVIC_EnableIRQ(TIM2_IRQn);
TIM2->CR1 |= TIM_CR1_CEN;             // spusť
```

Příznaky a přerušení — registr SR

Když nastane událost, časovač ji oznámí příznakem v registru `SR` (Status Register):



Obrázek 2.18. Registr TIMx_SR — příznaky událostí

Klíčový je **`UIF`** (Update Interrupt Flag) — přetečení. Příznaky `CCxIF` patří komparačnímu/záchytnému systému (dále v této části). Pravidlo, na které studenti často zapomenou: **příznak se v ISR**

musí smazat, jinak se přerušení po návratu spustí znovu — a maže se **zápisem 0**, ne 1 (zápis 1 nemá efekt):

```
void TIM2_IRQHandler(void){
    if (TIM2->SR & TIM_SR_UIF) {
        TIM2->SR &= ~TIM_SR_UIF;     // smaž příznak (zápis 0!)
        // ... obsluha každou 1 ms ...
    }
}
```

Tím spojujeme oddíl 2.3 a 4: časovač je nejčastější zdroj pravidelných přerušení (časová základna, SysTick, plánovač RTOS).

Stínové (shadow) registry a bezpečná aktualizace

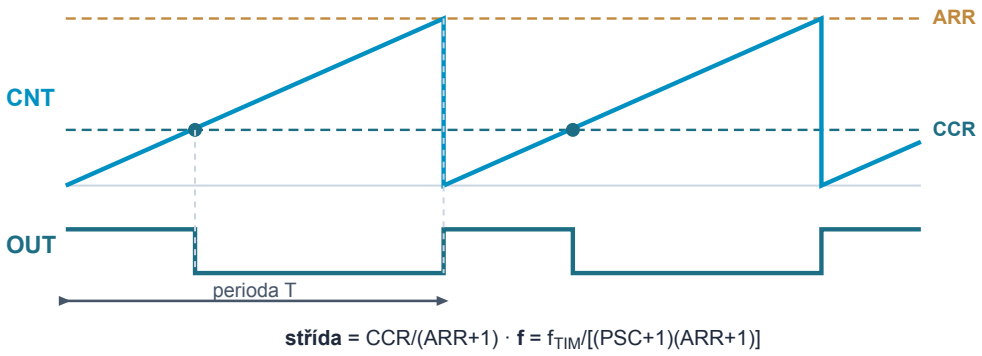
Kdybychom za běhu PWM změnili `ARR` nebo `CCR` přímo, mohla by se změna projevit **uprostřed periody** a způsobit zákmit (glitch) — pin by skončil v nečekané úrovni. Časovač to řeší **stínovými (preload) registry**: do `ARR`/`CCR` zapisujeme do „nárazníku", a hardware ho do skutečného registru překlopí teprve při **update události**, tedy na hranici periody. Změna je tak **atomická vůči periodě** a beze zákmitů.

Preload `ARR` se zapíná bitem `CR1.ARPE`, preload kanálu bitem `CCMRx.OCxPE`. Je to přesně ta vlastnost, díky níž lze plynule měnit střídu PWM za běhu, aniž by na výstupu vznikaly zákmity.

Komparační systém, PWM a výstup na pin

Časovač umí kromě přetečení i **porovnávat** `CNT` s hodnotou v záchytně-srovnávacím registru `CCRx` a podle výsledku **přepínat výstupní pin**. Režim určuje pole `OCxM` v `CCMRx`. Nejdůležitější jsou **PWM mód 1** a **mód 2** (vzájemně invertované):

V PWM módu 1 je výstup v log. 1, dokud `CNT < CCR`, pak v log. 0:



Obrázek 2.19. Časování PWM: CNT, ARR, CCR a výstup

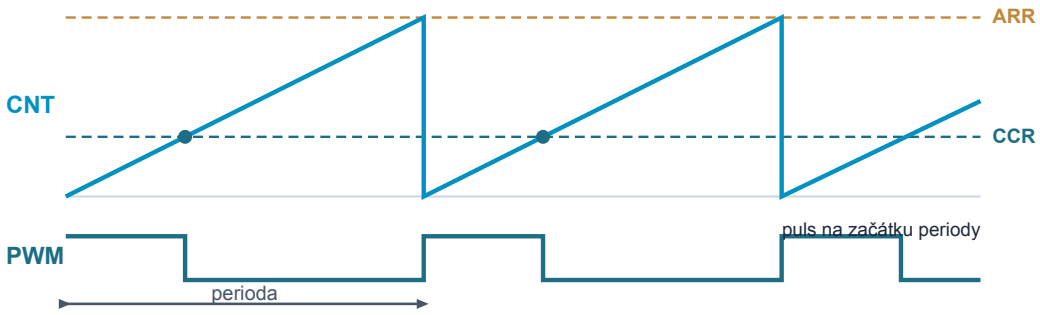
Frekvence PWM je dána periodou přetečení, **střída** poměrem `CCR/(ARR+1)`:

Střída
Poměr doby zapnuto k periodě PWM (duty cycle); řídí střední hodnotu výstupu.

$$f_{PWM} = \frac{f_{TIM}}{(PSC+1)(ARR+1)}, \qquad \text{střída} = \frac{CCR}{ARR+1}.$$

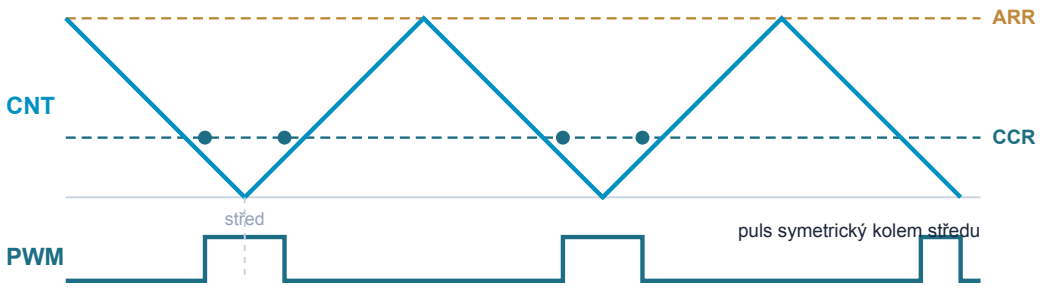
Změnou `CCR` tedy za běhu plynule řídíme střídu (jas podsvícení, otáčky motoru, polohu serva), zatímco frekvence zůstává konstantní. Existují dvě zarovnání:

Hranově zarovnané (edge-aligned) — čítač počítá jen nahoru, hrana je „na začátku" periody:



Obrázek 2.20. Edge-aligned PWM

Středově zarovnané (center-aligned) — čítač počítá nahoru i dolů (`CMS`), puls je symetrický kolem středu. To je výhodné pro řízení motorů, protože sousední kanály spínají souměrně a sníží se rušení:



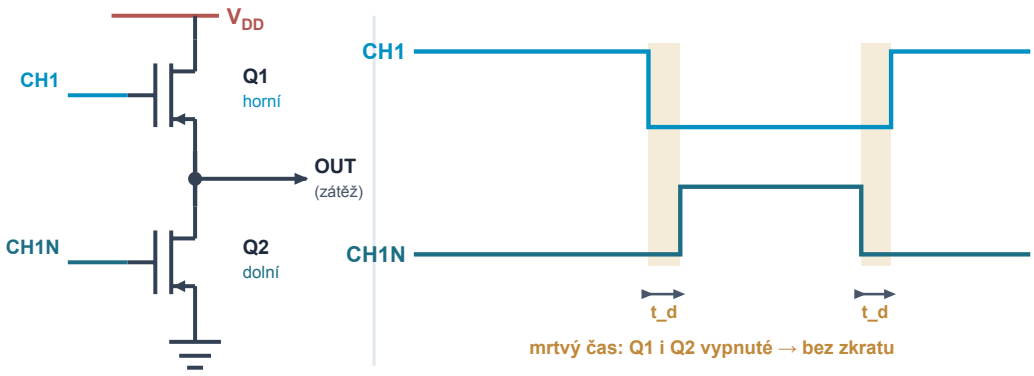
Obrázek 2.21. Center-aligned PWM

PWM na pin a mrtvý čas

Aby se signál komparátoru dostal ven, musí být pin v režimu **alternativní funkce** a `AFR` nastaven na příslušný kanál časovače (část 2.2). Kompletní řetězec „od hodin po pin" pro PWM na PA5 (TIM2_CH1):

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
GPIOA->MODER = (GPIOA->MODER & ~(3u<<(5*2))) | (2u<<(5*2)); // AF
GPIOA->AFR[0] |= (1u<<(5*4));                                // AF1 = TIM2_CH1
TIM2->PSC = 0;  TIM2->ARR = 999;                            // f = f_TIM/1000
TIM2->CCR1 = 250;                                            // střída 25 %
TIM2->CCMR1 |= (6u<<4) | TIM_CCMR1_OC1PE;                   // PWM mód 1 + preload
TIM2->CCER  |= TIM_CCER_CC1E;                                // povol výstup kanálu
TIM2->CR1   |= TIM_CR1_ARPE | TIM_CR1_CEN;                   // preload ARR + start
```

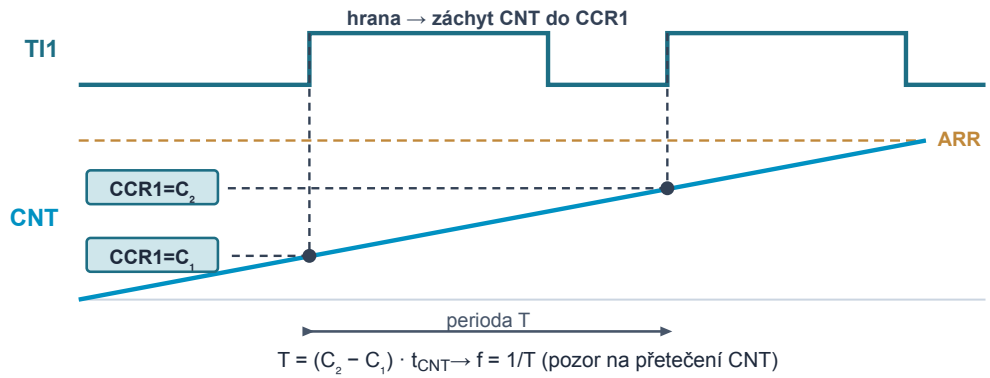
U výkonových aplikací (řízení můstku motoru) potřebujeme dva **komplementární** výstupy a mezi jejich přepnutím **mrtvý čas (dead-time)** — krátkou prodlevu, kdy jsou oba tranzistory vypnuté, aby nedošlo ke zkratu napájení. To umí pokročilé časovače (TIM1/TIM8) hardwarově:



Obrázek 2.22. PWM s mrtvým časem

Záchytný systém — měření času na vstupu

Doteď časovač čas **generoval**. Opačně umí čas i **měřit**: v režimu **input capture** zachytí na hraně vstupního signálu okamžitou hodnotu `CNT` do `CCRx` a nastaví `CCxIF`. Rozdíl dvou po sobě zachycených hodnot dává **periodu** (a převrácená hodnota frekvenci) nebo **šířku pulsu** — to vše hardwarově, bez závislosti na latenci softwaru:



Obrázek 2.23. Časování záchytného systému

```
// změř periodu signálu na TIM2_CH1
TIM2->CCMR1 |= (1u<<0);              // CC1 = vstup, mapováno na TI1
TIM2->CCER  |= TIM_CCER_CC1E;        // povol záchyt (vzestupná hrana)
TIM2->DIER  |= TIM_DIER_CC1IE;       // přerušení při záchytu
// v ISR:  perioda = CCR1 - predchozi;  (pozor na přetečení CNT)
```

Přijde-li další hrana dřív, než stihneme přečíst `CCRx`, nastaví se příznak **přetečení záchytu** `CCxOF` (overcapture) — signál, že vstup je rychlejší, než stíháme obsloužit. Tytéž tři režimy — **čítání, srovnávání, záchyt** — pokrývají drtivou většinu časových úloh ve vestavných systémech.

Řetězení časovačů: master/slave a TRGO

Časovače nemusí běžet izolovaně — jeden může řídit druhý vnitřním signálem **TRGO** (Trigger Output), aniž by se plýtvalo piny nebo přerušeními. **Master** časovač vysílá TRGO odvozený od zvolené události (typicky update), **slave** se jím nechá spustit, zastavit, resetovat nebo přímo hodinovat:

TRGO
Trigger Output: vnitřní signál časovače odvozený od zvolené události (update, srovnání, spuštění). Slouží k řízení jiného časovače (master/slave) nebo k periodickému spouštění dalších periferií, například ADC či DAC, bez zásahu jádra.

```
TIM2->CR2  |= (0b010 << TIM_CR2_MMS_Pos);    // master: TRGO = Update
TIM3->SMCR |= (0b001 << TIM_SMCR_TS_Pos)     // slave: trigger = ITR1 (TIM2)
           |  (0b111 << TIM_SMCR_SMS_Pos);   // režim: external clock
```

Řetězením se ze dvou 16bitových čítačů složí efektivně jeden delší (32/48bit). Ještě důležitější je, že TRGO vede i **napříč periferiemi**: časovač tak dokáže periodicky spouštět převodník ADC nebo DAC bez jakéhokoli zásahu procesoru. Právě to je most do přednášky 5 — řetěz **časovač → TRGO → DMA → ADC/DAC** dává deterministické vzorkování a generování signálu úplně bez jádra.

Časovač jako kvadraturní enkodér

Rotační enkodér vydává dva signály `A` a `B` posunuté o 90°; směr otáčení určuje, který z nich předbíhá. Časovač to umí dekódovat **v hardwaru**, v **režimu enkodéru**:

Režimu enkodéru
Režim časovače, ve kterém se čítač řídí dvojicí kvadraturních signálů A a B z rotačního enkodéru. Podle jejich fáze sám inkrementuje nebo dekrementuje, takže v registru CNT je průběžně poloha; čítáním obou hran obou kanálů dává čtyřnásobné rozlišení.

```
TIM3->SMCR  |= (0b011 << TIM_SMCR_SMS_Pos);  // encoder mode 3 (obě hrany A i B)
TIM3->CCMR1 |= (1u << 0) | (1u << 8);        // CC1S, CC2S = vstupy TI1, TI2
TIM3->CR1   |= TIM_CR1_CEN;
```

Čítač pak sám inkrementuje nebo dekrementuje podle fáze `A`/`B`, takže v `CNT` je průběžně **poloha** hřídele a v bitu `TIM_CR1_DIR` směr; počítáním obou hran obou kanálů se dosáhne čtyřnásobného rozlišení. Nepotřebuje k tomu žádná přerušení ani softwarovou logiku — jádro jen občas přečte `CNT`. Časovač zde tedy není generátor času, ale **dekodér pohybu**; typicky slouží k odečítání polohy v řízení motorů.

> **Shrnutí oddílu 2.4:** Časovač je čítač (`PSC`/`CNT`/`ARR`) s periodou `T=(PSC+1)(ARR+1)/f`, řízený `CR1` a hlásící události v `SR`. Stínové registry zajistí bezzákmitovou změnu za běhu. Komparační systém generuje **PWM** (střída `CCR/(ARR+1)`), alternativní funkce přivede signál na pin, dead-time chrání výkonové stupně. Záchytný systém naopak **měří** periodu a šířku pulsu na vstupu.

2.5 Syntéza: čas drží systém pohromadě

Zbývá spojit hodiny, GPIO, přerušení a časovače do jednoho obrazu. Tato závěrečná část ukáže, že všechny stojí na jediné myšlence — synchronní procesor bez hodin nefunguje — a shrne opakující se vzory, které poneseme do dalších přednášek.

Jedna myšlenka a opakující se vzory

Celá přednáška vyšla z jediného pozorování z oddílu 2.1: **procesor je synchronní obvod, a tudíž bez hodin nefunguje nic.** Z toho plyne celý řetězec, kterým jsme prošli:

1. **Hodiny (oddíl 2.1).** Oscilátor → PLL → `SYSCLK` → děličky sběrnic. Periferie je po resetu „mrtvá", dokud jí v `RCC` nepovolíme hodiny. *Spuštění periferie = povolení hodin.*
2. **GPIO (oddíl 2.2).** Když periferie tiká, musíme ji propojit se světem. Pin se konfiguruje registry (`MODER`, `OTYPER`, `PUPDR`, `AFR`) a ovládá daty (`IDR`/`ODR`/`BSRR`).
3. **Přerušení (oddíl 2.3).** Místo plýtvavého pollingu necháme hardware, ať nás vyruší. `NVIC` přepne kontext a podle vektorové tabulky skočí do ISR — to celé v desítkách cyklů.
4. **Časovače (oddíl 2.4).** Nejčistší zdroj času: čítač generuje pravidelná přerušení, PWM a měří vstupní signály — a uzavírá kruh zpět k myšlence, že vše je odvozené od hodin.

Mezi částmi vedou **mosty**, ne švy: hodiny rozběhnou GPIO i časovač; přerušení obsluhují události z GPIO (`EXTI`) i z časovače (`UIF`); časovač přivede PWM na pin přes alternativní funkci GPIO. Tytéž tři operace časovače — **čítat, srovnávat, zachytávat** — a tentýž vzor **„povol hodiny → nastav registry → spusť → obslož příznak"** se opakují u každé další periferie.

Opakující se vzory, které poneseme dál

Z této přednášky si stojí za to odnést několik vzorů, které platí obecně:

- **`ON`/`RDY`** — zapni zdroj a počkej, až je připraven (oscilátory, PLL).
- **Atomický přístup k bitu** — `BSRR`, bit-banding; nikdy slepě `read-modify-write` nad registrem, který může změnit přerušení.
- **Příznak se musí smazat** — v ISR i u časovače (`SR`), jinak se událost „zacyklí".
- **Stínové registry** — měň kritické hodnoty na hranici periody, ne uprostřed.
- **`volatile` a kritická sekce** — vše sdílené mezi ISR a hlavním programem.

Atomický přístup k bitu
Registr BSRR: nastaví nebo vynuluje vybrané bity ODR bez čtení, atomicky.

Co bude dál

Přednáška postavila časový a událostní základ, na kterém stojí zbytek kurzu:

- **sériové komunikace (UART, SPI, I2C)** — periferie řízené týmiž vzory; jejich časování (baud rate, hodiny sběrnice) vychází přímo z hodinového stromu této přednášky,
- **DMA** — přenos dat mezi periferií a pamětí bez CPU, často spouštěný *událostí časovače*,
- **nízká spotřeba** — vypínání nepoužitých hodinových větví (přesně těch `…ENR` gateů),
- **systémy reálného času a RTOS (přednášky 8–9)** — plánovač staví na periodickém přerušení časovače (SysTick) a na kritických sekcích z oddílu 2.3.

> **Závěr:** Naučili jsme se rozběhnout hodiny, propojit čip se světem přes GPIO, reagovat na události přerušeními a měřit i generovat čas časovači. To jsou čtyři kameny, na kterých stojí každá další periferie i celý vestavný systém. Klíčová věta, kterou si odneseme: **v synchronním procesoru je čas to první, co musí začít fungovat — a teprve pak může fungovat cokoli dalšího.**

2.6 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 2.1 — Zdroje času a hodinový systém

1. Proč musí být úplně prvním krokem při práci s libovolnou periferií povolení jejích hodin? Co se stane, když to vynecháte?
2. Srovnejte interní RC oscilátor a krystal z hlediska přesnosti, doby náběhu a ceny. Pro USB byste zvolili který a proč?
3. Vysvětlete vzor „ON → RDY". Proč nelze hned po nastavení `xON` spustit PLL a jak hardware pozná, že je zdroj ustálený?
4. Jak z nízké vstupní frekvence vznikne systémový takt — proč PLL nejdřív dělí (`PLL_M`), pak násobí (`PLL_N`) a zase dělí (`PLL_P`)? Ověřte na `HSE = 8 MHz`, `M = 8`, `N = 336`, `P = 2`.
5. Proč musí rychlé jádro čekat na pomalejší paměť Flash (*wait states*) a co by se stalo bez nich?
6. Jak STM32 chrání systém proti výpadku krystalu HSE (Clock Security System) a jaký princip návrhu spolehlivých systémů tím ztělesňuje?

K oddílu 2.2 — Periferie a GPIO

1. Proč se jeden GPIO pin konfiguruje hned několika registry? Co určuje každý z registrů `MODER`, `OTYPER`, `OSPEEDR` a `PUPDR`?
2. Vysvětlete rozdíl push-pull vs. open-drain. Proč sběrnice I2C vyžaduje open-drain?
3. Proč je přímý zápis do `ODR` (read-modify-write) náchylný na souběh s přerušením a jak to řeší `BSRR`?
4. V jakém režimu jsou piny po resetu a proč je to bezpečné? Který režim má nejnižší spotřebu a proč?
5. Proč se pin „odevzdá" vnitřní periferii přes alternativní funkci a co by bez toho nefungovalo (např. výstup časovače na pinu)?

K oddílu 2.3 — Přerušovací systém

1. Čím se liší obsluha přerušení od běžného volání funkce (spuštění, kontext, režim, návrat)?
2. Které registry uloží hardware při vstupu do přerušení a proč právě tuto sadu? Co umožní, že se ISR píše jako obyčejná C-funkce?
3. Vysvětlete preempci, tail-chaining a late-arrival. Jak každý z nich přispívá k nízké a deterministické latenci?
4. Proč musí být proměnná sdílená mezi ISR a hlavní smyčkou `volatile`? Kdy `volatile` nestačí a je nutná kritická sekce?
5. Proč GPIO pin sám přerušení negeneruje a jakou roli hraje řadič EXTI mezi pinem a NVIC? Co se musí stát s příznakem zdroje na konci obsluhy a proč?
6. Čím se SysTick liší od periferních časovačů `TIMx` a proč je právě díky těmto vlastnostem vhodný jako přenositelná systémová časová základna?

K oddílu 2.4 — Časovače a PWM

1. Odvoďte vztah pro periodu přetečení časovače a navrhněte `PSC`/`ARR` pro 1 ms při `f_TIM = 84 MHz`.
2. Jak se počítá střída PWM? Jak ji za běhu plynule změníte, aniž by výstup zakmital, a jaký hardwarový mechanismus to zajišťuje?
3. Vysvětlete rozdíl mezi hranově a středově zarovnanou PWM. Proč se center-aligned hodí pro řízení motorů?
4. K čemu slouží mrtvý čas u komplementárních výstupů a jaký problém řeší?
5. Jak režim input capture změří periodu vnějšího signálu? Na co si dát pozor u běžícího čítače?
6. Jak umožní signál TRGO (propojení master–slave), aby jeden časovač spouštěl druhý nebo ADC/DAC **bez zásahu procesoru**? Jaký obecný princip za tím stojí?
7. Jak časovač v režimu enkodéru dekóduje polohu z kvadraturních signálů A/B a proč nepotřebuje přerušení?

3

Od C k běžícímu obrazu

> Tato kapitola sleduje, **jak se z jazyka C stane firmware**: co se s programem děje mezi zdrojovým textem a bajty ve Flash a co z toho má programátor pod kontrolou (**překladač není černá skříňka**). Druhá půle je inženýrská — členění kódu do modulů, ovladače periferií, pravidla MISRA-C a ladění z disassembly.

Obsah kapitoly

3.1 Programování vestavných zařízení
3.2 Preprocesor: od konstant po složité konstrukce
3.3 Kompilátor: překlad a optimalizace
3.4 Volací konvence (AAPCS)
3.5 Členění kódu: moduly, knihovny a ovladače
3.6 Limity vestavného prostředí
3.7 MISRA-C: pravidla, vynucování a důsledky
3.8 Ladění a vhled do běžícího čipu
3.9 Závěr — od zdroje k běžícímu obrazu a zpět
3.10 Kontrolní otázky

3.1 Programování vestavných zařízení

Na PC běží program nad operačním systémem: alokaci paměti, vstup/výstup i plánování zajišťuje jádro a standardní knihovna. Vestavný program běží zpravidla **bare-metal** — přímo nad hardwarem, bez operačního systému pod sebou. Cílem je **plná kontrola** nad periferiemi, výkonem a velikostí kódu:

Bare-metal
Program běžící přímo nad hardwarem, bez operačního systému pod sebou.

- bez operačního systému (žádné FreeRTOS pod programem),
- typicky bez **HAL** (Hardware Abstraction Layer),
- přístup přímo k registrům periferií,
- minimální startup kód a linker script,
- běh přímo po resetu z adresy `0x0800 0000` (Flash).

Veškerou inicializaci — rozběh hodin, nastavení zásobníku, kopii inicializovaných dat — musí program zařídit sám, případně s tenkou vrstvou runtime knihovny. Tím se mění význam standardní knihovny: `printf`, `malloc` či `fopen` na PC fungují bez dalšího, protože se opírají o systémová volání jádra. Na bare-metal žádné jádro není — funkce knihovny se buď neimplementují vůbec, nebo se napojí na **stub** (záslepku), kterou dodá programátor.

Stub
Záslepka: prázdná náhrada funkce (např. syscallu), aby linker prošel.

Mezi zápisem binárních čísel přímo do registrů a voláním hotových HAL funkcí je několik úrovní abstrakce, mezi nimiž volíme podle požadavku na výkon, čitelnost a přenositelnost:

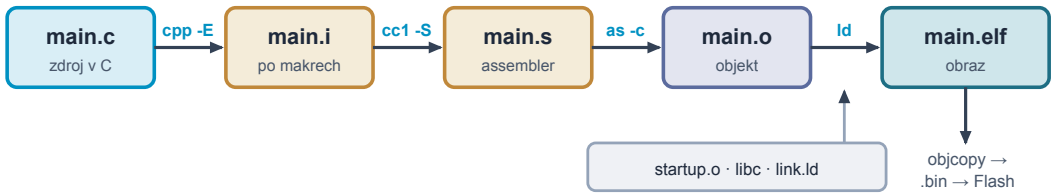
1. **Přímý přístup do registrů** (register-level). Používáme registry přímo z Reference Manualu, např. `GPIOA->MODER = 0x1;`. Maximální výkon, minimální kód, přesná kontrola — ale kód je svázaný s konkrétním čipem a vyžaduje znalost datasheetu.
2. **CMSIS** (Cortex Microcontroller Software Interface Standard). Standardizované hlavičky od ARM (`stm32f4xx.h`, `core_cm4.h`) definují struktury registrů, bitové masky a intrinsics. Kód je čitelnější, stále blízko železu a přenositelný mezi výrobci.
3. **LL (Low Layer)** knihovny od ST. Součást STM32Cube, tenké inline funkce nad registry: `LL_GPIO_SetOutputPin(GPIOA, LL_GPIO_PIN_5);`. Kompromis mezi HAL a bare-metal, velmi efektivní.

V tomto kurzu pracujeme na úrovni **registrů a CMSIS** — abstrakce, kde je stále vidět, co procesor dělá, a kde se dá vše dohledat v reference manuálu.

Proces kompilace

Sestavení obrazu z jednoho či více `*.c` souborů není jeden monolitický „překladač", ale **řetězec samostatných nástrojů**, z nichž každý lze vyvolat i zvlášť:

Fáze	Co dělá	Nástroj
Preprocessing	rozbalí makra, vloží hlavičky, odstraní komentáře	`cpp`
Kompilace	převede C → assembler	`cc1`
Assembling	assembler → objektový kód (`.o`)	`as`
Linkování	spojí `.o` + knihovny → `.elf`	`ld`
Konverze	`.elf` → `.bin`/`.hex`	`objcopy`
Nahrání	zápis do FLASH	`st-flash`



Obrázek 3.1. Překladový řetězec: od zdroje k obrazu ve Flash

Každý nástroj nese prefix `arm-none-eabi-*` (GNU toolchain pro ARM bez operačního systému, *embedded ABI*). Schopnost vyvolat každou fázi samostatně je při ladění neocenitelná — například `gcc -S -O2 main.c` ukáže přesně, jaký assembler překladač generuje.

Jako konkrétní příklad, který nese celé předchozí přednášky, poslouží minimální program ovládající pin:

```c
#include "stm32f4xx.h"

int main(void)
{
    RCC->AHB1ENR |= (1 << 3);     // zapnout hodiny GPIOD
    GPIOD->MODER |= (1 << 24);    // PD12 = výstup
    while (1)
        GPIOD->ODR ^= (1 << 12);  // přepni výstup
}
```

Tři operace odpovídají třem pojmům z přednášky 2: **clock-enable** (RCC), **konfigurační registr** (MODER) a **datový registr** (ODR).

Preprocesor samostatně

Preprocesor není samostatný program; jeho výstup získáme přepínačem kompilátoru:

```
arm-none-eabi-gcc -E main.c -o main.i
```

Výsledek `main.i` je čistý C kód: `#include` je nahrazen obsahem souboru, `#define` odstraněno a makra expandována, komentáře smazány. Například `RCC->AHB1ENR` se rozvine na `(*(volatile uint32_t *)0x40023830)` — adresu a kvalifikátor `volatile` dodává hlavička CMSIS.

Kompilace do assembleru

Přepínač kompilátoru určuje cílovou platformu a instrukční sadu:

```
arm-none-eabi-gcc -S -mcpu=cortex-m4 -mthumb main.i -o main.s
```

```
ldr     r3, =0x40023830
ldr     r2, [r3]
orr     r2, r2, #8
str     r2, [r3]
```

Úlohou kompilátoru je převod na instrukce **a správa ABI**: jak se předávají argumenty (`r0`–`r3`), kam se ukládá návratová hodnota (`r0`), co se ukládá na zásobník a jak vzniká rámec funkce (`push {lr}`, `pop {pc}`). Přepínače `-mcpu` a `-mthumb` jsou povinné — bez nich generuje překladač kód pro klasický 32bitový ARM profil, který se na Cortex-M **nespustí**, protože jádra řady M umějí jen Thumb-2. Pro jádra s FPU se přidává `-mfpu=fpv4-sp-d16 -mfloat-abi=hard`.

Linkování a konverze

Více modulů a knihoven linker propojí, vyřeší symboly a umístí je do reálné paměti podle linker scriptu:

```
arm-none-eabi-gcc main.o startup_stm32f4xx.o \
    -T stm32f4.ld -o main.elf -Wl,-Map=main.map
```

Vznikne `main.elf` — plnohodnotný spustitelný soubor s debug informacemi a (díky `-Wl,-Map`) soubor `main.map` s přehledem sekcí a symbolů. Pro nahrání do čipu se z `.elf` odstraní debug data:

```
arm-none-eabi-objcopy -O binary main.elf main.bin     # nebo -O ihex … main.hex
st-flash write main.bin 0x08000000                     # nebo openocd … program main.elf
```

Linkeru a paměťové mapě se věnuje přednáška 4; nejdříve ale projdeme preprocesor a kompilátor.

3.2 Preprocesor: od konstant po složité konstrukce

Preprocesor pracuje **čistě textově**: nezná typy ani rozsah platnosti, jen nahrazuje posloupnosti znaků. Tato jednoduchost je jeho síla i jeho nebezpečí. Ve vestavném kódu se makry definují adresy registrů, masky bitů i celé generované konstrukce — a právě zde vzniká velká část záludných chyb.

Makra: od konstant ke generování kódu

Nejjednodušší použití je pojmenovaná konstanta a přístup do registru přes adresu:

```
#define LED_PIN   (1 << 5)
#define REG32(addr)   (*(volatile uint32_t *)(addr))
#define GPIOA_ODR     REG32(0x40020014)
```

Parametrická makra zapouzdřují opakované operace s bity a registry:

```
#define BIT(n) (1U << (n))
#define SET_BIT(REG, N)    ((REG) |= BIT(N))
#define CLEAR_BIT(REG, N)  ((REG) &= ~BIT(N))
#define TOGGLE_BIT(REG, N) ((REG) ^= BIT(N))

#define PERIPH_REG(BASE, OFFSET) (*(volatile uint32_t *)((BASE) + (OFFSET)))
#define RCC_BASE     0x40023800U
#define RCC_AHB1ENR  PERIPH_REG(RCC_BASE, 0x30U)
```

Závorky kolem každého parametru a celého výrazu nejsou kosmetika. Bez nich by `BIT(a+1)` expandovalo na `1U << a+1`, což se kvůli prioritě operátorů vyhodnotí jako `(1U<<a)+1`. **Každý argument makra i celý výsledek se proto obalují závorkami.**

Operátory # a

Preprocesor nabízí dva operátory, na kterých stojí generování kódu:

- **stringizace** `#` převede argument na řetězcový literál:

```
#define TO_STRING(x) #x
TO_STRING(hello)   // "hello"
```

- **slepování** `##` spojí dva tokeny v jeden:

```
#define CONCAT(a, b) a##b
CONCAT(GPIO, A)    // GPIOA
```

Kombinací obojího lze generovat celé skupiny symbolů z jediného vzoru. To je užitečné všude, kde jsou registry a obsluhy pojmenovány **systematicky**:

```
#define MAKE_REG(port, reg)  GPIO##port##_##reg
MAKE_REG(A, ODR) = 1;   // -> GPIOA_ODR = 1;
```

Generování opakujícího se kódu

Vícestupňová makra generují celé funkce. Typickým příkladem je hromadná definice obsluh přerušení:

```
#define DEFINE_ISR(name)           \
    void name##_IRQHandler(void) { \
        handle_interrupt(#name);   \
    }

DEFINE_ISR(TIM2)        // -> void TIM2_IRQHandler(void) { handle_interrupt("TIM2"); }
DEFINE_ISR(USART1)
```

Nejmocnější vzor kombinuje makro s **vlastní linker sekcí** a řeší jím častý problém: jak **automaticky zaregistrovat** každý ovladač, aniž bychom museli ručně udržovat centrální seznam, do něhož se při přidání ovladače nesmí zapomenout dopsat. Makro `REGISTER_DRIVER`

umístí ke každému ovladači jeden **záznam** (`driver_entry` se jménem a inicializační funkcí) do vyhrazené sekce `.drivers`:

```
#define REGISTER_DRIVER(NAME, INITFN)              \
    __attribute__((section(".drivers")))           \
    static const struct driver_entry NAME##_driver = { #NAME, INITFN }

REGISTER_DRIVER(spi1, spi1_init);
REGISTER_DRIVER(uart2, uart2_init);
```

Atribut `section(".drivers")` určuje překladači, aby záznam neuložil mezi ostatní data, ale do pojmenované sekce `.drivers`. **Linker** pak všechny takto rozeseté záznamy — z libovolného počtu zdrojových souborů — sesbírá do jednoho **souvislého pole** mezi hraniční symboly `_drivers_start` a `_drivers_end`; runtime jimi při startu jen projde jako polem a každý ovladač inicializuje:

```
extern const struct driver_entry _drivers_start[], _drivers_end[];
for (const struct driver_entry *d = _drivers_start; d < _drivers_end; d++)
    d->init();
```

Přidání nového ovladače je tak otázkou **jediného řádku v jeho vlastním souboru** — bez zásahu do jádra systému. Tento vzor (linker-set / init array) používají Linux, Zephyr i řada RTOS. Jak přesně linker sekci sesbírá a odkud se berou hraniční symboly, rozebíráme u linkeru v **přednášce 4**.

Výrobci SDK takto generují i celá přístupová rozhraní k pinům:

```
#define GPIO_PIN_DEFINE(PORT, PIN)                          \
    static inline void GPIO##PORT##_PIN##PIN##_Set(void)    \
    { GPIO##PORT->BSRR = (1 << PIN); }                      \
    static inline void GPIO##PORT##_PIN##PIN##_Reset(void) \
    { GPIO##PORT->BSRR = (1 << (PIN + 16)); }

GPIO_PIN_DEFINE(A, 5)   // vygeneruje GPIOA_PIN5_Set() a _Reset()
```

Rizika maker a alternativy

Makra jsou výkonná, ale mají závažná omezení: **žádnou typovou kontrolu** (vše je textová náhrada), **složitou laditelnost** (debugger ukazuje kód až po expanzi), těžkou čitelnost při vnoření a **neočekávané chování** s vedlejšími efekty v argumentech:

```
#define SQR(x) ((x)*(x))
int a = 2;
int b = SQR(a++);   // expanduje na ((a++)*(a++)) — nedefinované chování
```

Proto moderní vestavný kód makra omezuje ve prospěch jazykových prostředků s typovou kontrolou:

- `static inline` **funkce** místo parametrického makra — kompilátor je inlinuje stejně efektivně, ale s kontrolou typů a bez pastí s vedlejšími efekty;
- `enum` nebo `const` místo `#define` konstanty;
- **generátory kódu** (Python, CMake, CubeMX) místo extrémních maker.

MISRA-C tento přístup vyžaduje (část 3.7).

Obsluha přerušení a její členění

Obsluha přerušení (**ISR**, Interrupt Service Routine) je funkce, kterou procesor zavolá asynchronně při události (timer, UART RX, DMA done, EXTI). Má vyšší prioritu než běžný kód, přerušuje jeho běh a po dokončení se vrací přes `bx lr`. Z toho plynou tvrdé požadavky: ISR musí být **co nejkratší, deterministická, bez rekurze** a **bez blokujících operací** (`while` čekající na hardware, `printf`, `malloc`, `sleep`).

Antivzor je obsluha, která dělá veškerou práci přímo:

```
void USART1_IRQHandler(void) {
    while (!(USART1->SR & USART_SR_RXNE));   // blokující čekání v ISR
    uint8_t c = USART1->DR;
    printf("RX: %c\n", c);                   // printf může dlouho čekat
}
```

Správný vzor je **odložení práce**: ISR jen přečte data a nastaví příznak, zatímco skutečné zpracování proběhne v hlavní smyčce nebo v úloze:

```
volatile uint8_t rx_byte;
volatile bool    rx_ready = false;

void USART1_IRQHandler(void) {
    if (USART1->SR & USART_SR_RXNE) {
        rx_byte = USART1->DR;     // přečti a signalizuj
        rx_ready = true;
    }
}

void main(void) {
    for (;;)
        if (rx_ready) { rx_ready = false; handle_rx_byte(rx_byte); }
}
```

Sdílené proměnné musí být `volatile` (část 3.3); samotné `volatile` ale neřeší atomicitu. Toto rozdělení „rychlá ISR + odložené zpracování" je základ robustního členění obsluhy událostí; v části 3.5 ho povýšíme na ovladač s frontou a callbackem.

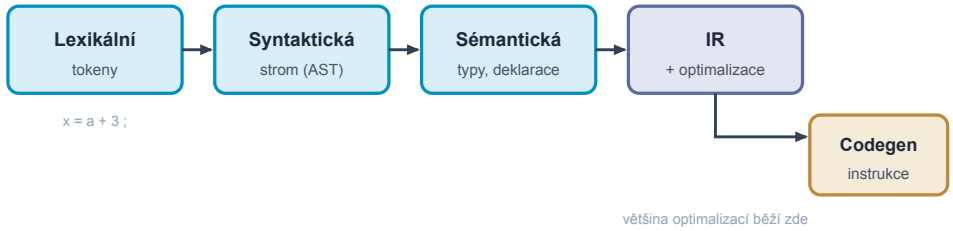
3.3 Kompilátor: překlad a optimalizace

Preprocesor a linker text jen přesouvají a spojují. Skutečný překlad — z myšlenky v C na posloupnost instrukcí — dělá **kompilátor**. Tato část otevře jeho výstup: jak funkce projde fázemi překladu, jak vypadá přeložená funkce, jak ji mění optimalizace a proč některý zdánlivě správný C kód po optimalizaci přestane fungovat.

Fáze kompilace na příkladu

Kompilátor je posloupnost analýz a transformací. U GCC i Clangu rozlišujeme:

1. **Lexikální analýza** — rozdělení zdroje na **tokeny** (`if`, `x`, `=`, `42`, `;`).
2. **Syntaktická analýza** (parser) — z tokenů vzniká **strom abstraktní syntaxe** (AST).
3. **Sémantická analýza** — kontrola typů, rozsahů platnosti, volání funkcí.
4. **Mezikód (IR)** — AST se převede na architekturně nezávislý mezikód (GIMPLE/SSA v GCC), nad nímž běží většina optimalizací.
5. **Optimalizace** podle úrovně `-O0` až `-O3`.
6. **Generování assembleru** cílové architektury.



Obrázek 3.2. Fáze překladače: od znaků zdroje k instrukcím

Pro programátora je podstatný výstup poslední fáze: assembler, který lze přečíst a porovnat s očekáváním. Schopnost číst disassembly je jedna z mála spolehlivých cest, jak zjistit, co procesor *skutečně* dělá — zvlášť při ladění optimalizovaného kódu.

Příklad přes všechny fáze

Celý postup projdeme na funkci, která sečte pole:

```
int sum(int *arr, int n) {
    int s = 0;
    for (int i = 0; i < n; i++) {
        s += arr[i];
    }
    return s;
}
```

Lexikální analýza rozdělí kód na tokeny; každý má typ a hodnotu (`int` → klíčové slovo, `s` → identifikátor, `=` → operátor). Lexer odstraní mezery a komentáře, ale význam nezachycuje:

```
[int] [sum] [(] [int] [*] [arr] [,] [int] [n] [)] [{]
[int] [s] [=] [0] [;]
[for] [(] [int] [i] [=] [0] [;] [i] [<] [n] [;] [i] [++] [)]
[s] [+=] [arr] [[] [i] []] [;]
[return] [s] [;] [}]
```

Syntaktická analýza sestaví AST, kde každý příkaz, výraz a operátor má svůj uzel; zde se detekují chyby jako špatné závorky či chybějící středník:

```
FunctionDecl "sum"
 ├── Param arr : int*, Param n : int
 └── CompoundStmt
     ├── VarDecl s : int = 0
     ├── ForStmt (i=0; i<n; i++)
     │     └── BinaryOp '+=', s, ArraySubscript(arr, i)
     └── ReturnStmt(s)
```

Sémantická analýza ověří význam: odpovídají typy (`arr[i]` je `int`)? jsou proměnné deklarovány? odpovídá návratová hodnota typu `int`? Pokud vše sedí, kód se přepíše do **mezikódu** — bez složitostí C (typy, scoping), ale ještě ne pro konkrétní procesor:

```
s_1 = 0; i_2 = 0;
L1: if (i_2 >= n_0) goto L2;
    s_1 = s_1 + arr_0[i_2];
    i_2 = i_2 + 1;
    goto L1;
L2: return s_1;
```

Nad tímto mezikódem běží optimalizace: rozbalování smyček, eliminace mrtvého kódu, propagace konstant. Teprve pak se generuje assembler.

Anatomie přeložené funkce (`-O0`)

Bez optimalizace zůstává zachována struktura C — vhodné pro ladění, ale neefektivní:

```
sum:
    push    {r4, r5, r6, lr}     ; prolog
    movs    r3, #0               ; s = 0
    movs    r4, #0               ; i = 0
.L2:
    cmp     r4, r1               ; if (i >= n)
    bge     .L3                  ;   break
    ldr     r5, [r0, r4, lsl #2] ; arr[i]   (32b int -> i*4)
    add     r3, r3, r5           ; s += arr[i]
    adds    r4, r4, #1           ; i++
    b       .L2
.L3:
    mov     r0, r3               ; návratová hodnota
    pop     {r4, r5, r6, pc}     ; epilog: obnova + návrat
```

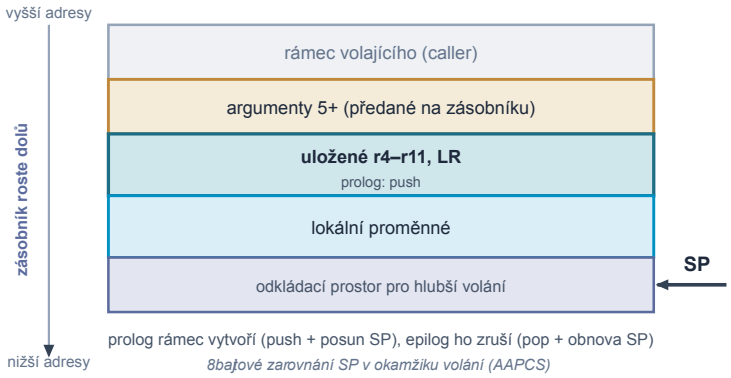
Volání funkce není elementární operace procesoru — je to **konvence** (AAPCS, část 3.4) realizovaná posloupností instrukcí. Každá netriviální funkce má tři části:

- **Prolog** vytvoří rámec zásobníku: `push {r4, lr}` uloží návratovou adresu (`lr`) a **callee-saved** registry. AAPCS žádá, aby funkce, která použije `r4`–`r11`, je po sobě obnovila; GCC to na `-O0` dělá vždy, když registr použije.
- **Tělo** vykoná vlastní práci. Na `-O0` drží GCC každou proměnnou ve vlastním registru (`s` v `r3`, `i` v `r4`), aby je debugger viděl přesně podle zdroje — i za cenu horšího kódu.
- **Epilog** rámec zruší: `pop {r4, pc}` obnoví `r4` a zápisem návratové adresy přímo do `pc` **provede návrat** (idiom „restore and return”).

Prolog
Začátek funkce: uloží návratovou adresu a callee-saved registry, vyhradí lokální proměnné.

Callee-saved
Registry r4–r11, které volaná funkce musí obnovit; caller-saved r0–r3 smí přepsat.

Epilog
Konec funkce: obnoví registry, uvolní rámec a vrátí řízení volajícímu.



Obrázek 3.3. Rámec zásobníku funkce: prolog, lokální proměnné, epilog

Registrová konvence Cortex-M: `r0`–`r3` jsou argumenty a dočasné (caller-saved), `r4`–`r11` zachovávané mezi voláními (callee-saved). Argument `arr` je předán v `r0`, `n` v `r1`, výsledek se vrací v `r0`.

Optimalizační úrovně

Tatáž funkce vypadá po překladu zcela jinak podle úrovně optimalizace. Při **`-O1`** GCC odstraní zbytečný rámec (žádný `push`/`pop`), vrací přes `bx lr` a používá méně registrů. Při **`-O2`** se objeví zásadní transformace smyčky:

```
sum:                    ; -O2
    cmp     r1, #0
    movs    r2, #0
    beq     .L3
.L2:
    ldr     r3, [r0], #4     ; post-inkrement ukazatele
    subs    r1, r1, #1       ; počítadlo dolů (while n--)
    adds    r2, r2, r3
    bne     .L2
.L3:
    mov     r0, r2
    bx      lr
```

Počítadlo jde **dolů** (test proti nule je kratší), ukazatel se posouvá **post-inkrementem** (`ldr [r0], #4`) a index zmizel úplně. Při **`-O3`** je u krátké smyčky výsledek podobný `-O2` — není co rozbalovat. Jakmile je ale počet iterací **známý konstantně** (`sum(a, 4)`), GCC smyčku **rozbalí** (loop unrolling):

```
ldr r2, [r0]      ; sum(a, 4) rozbaleno
ldr r3, [r0, #4]
ldr r12, [r0, #8]
ldr r1, [r0, #12]
adds r0, r2, r3
adds r0, r0, r12
adds r0, r0, r1
bx lr
```

Unrolling zrychluje běh **na úkor velikosti** tím, že odstraní instrukce porovnání a manipulace s řídicí proměnnou. Pro čipy s malou Flash je naopak důležitá úroveň **`-Os`**, která minimalizuje velikost (kratší instrukce, např. `cbz` místo `cmp`+`beq`). Úroveň **`-Og`** zachovává strukturu blízkou `-O0`, ale je rychlejší a drží proměnné tam, kde je debugger čeká.

Úroveň	Kdy použít
`-O0`	vývoj, plné možnosti ladění
`-Og`	vývoj s laděním, ale potřeba vyššího výkonu
`-O1`/`-O2`	produkční build, vyvážený výkon/velikost
`-O3`	maximální výkon, větší kód
`-Os`	minimální velikost (embedded s omezenou Flash)

> Optimalizace **nemění význam korektního programu** — mění jen způsob, jakým ho procesor provede. Liší-li se chování mezi `-O0` a `-O2`, je v kódu skoro vždy **nedefinované chování** (UB) nebo chybějící `volatile`. Optimalizátor jen zviditelní chybu, kterou kód už obsahoval.

Optimalizace se dramaticky projeví i jinde: podmíněné větvení s konstantou se přeloží na porovnání + podmíněný přesun (bez skoku), malé funkce se **vkládají inline** (žádný `bl`), a krátký `memcpy` GCC nahradí vlastním kódem místo volání knihovny.

Inline assembler

Tam, kde C nestačí — speciální instrukce (`WFI`, `BKPT`, `REV16`), přístup ke speciálním registrům jádra (`PSR`, `CONTROL`, `PRIMASK`) nebo přesně řízená smyčka — umožňuje GCC vložit assembler přímo do C. Rozšířená syntaxe propojuje instrukce s C proměnnými přes operandy, omezení (*constraints*) a seznam přepsaných registrů (*clobbers*):

```
asm [volatile] ("assembly code"
                : output operands      // např. "=r"(x)
                : input operands       // např. "r"(y)
                : clobbered registers);// např. "cc", "memory"
```

Omezení operandů: `r` obecný registr, `m` operand v paměti, `i` konstanta při překladu, `I` malá konstanta 0–255, `=r` výstupní registr, `+r` vstupně-výstupní registr.

Několik reprezentativních příkladů:

```
__asm__("wfi");                                  // uspí jádro do přerušení

uint32_t get_psr(void) {                         // čtení speciálního registru
    uint32_t r;
    __asm__ volatile ("mrs %0, psr" : "=r"(r));
    return r;
}

int x = 5, y;
__asm__ ("adds %0, %1, #1" : "=r"(y) : "r"(x) : "cc");   // mění příznaky
```

Praktická čekací smyčka s přesným počtem cyklů — bez overheadu C volání (typicky o 15–30 % rychlejší):

```
static inline void delay_cycles(uint32_t n) {
    __asm__ volatile (
        "1: subs %[n], %[n], #1 \n"
        "   bne 1b              \n"
        : [n] "+r"(n));
}
```

> Inline assembler je krajní prostředek: obchází typovou kontrolu i optimalizátor a snadno vede k chybám, které se projeví jen při určité úrovni optimalizace. Pro běžné potřeby existují **CMSIS intrinsics** — `__WFI()`, `__disable_irq()`, `__DMB()` — které dělají totéž čitelně a přenositelně. Ruční `asm` si nechte pro případy, kde intrinsic neexistuje.

`volatile`: přístup k hardwaru, výkon a jeho meze

Optimalizátor předpokládá, že paměť se mění **jen** v důsledku instrukcí programu. U hardwarových registrů a proměnných sdílených s přerušením to **neplatí**: stavový registr periferie se změní sám od sebe, příznak nastaví ISR. Pokud o tom překladač neví, čtení „zoptimalizuje” pryč. Uvažme čekání na bit ve stavovém registru:

```
#define STATUS_REG (*(uint32_t*)0x40000000)
void wait_ready(void) {
    while ((STATUS_REG & 0x1) == 0) { }
}
```

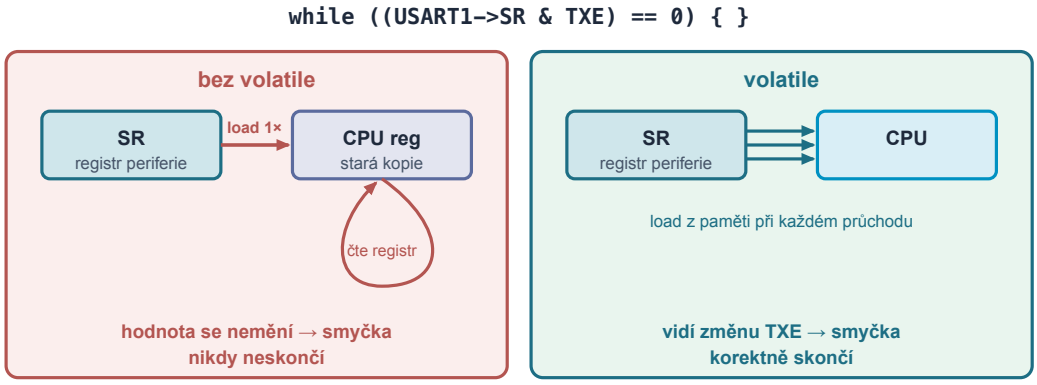
Kompilátor vidí „obyčejnou proměnnou”, načte ji **jednou** a smyčku zahodí jako nekonečnou:

```
wait_ready:
    ldr  r3, =0x40000000
    ldr  r2, [r3]
    tst  r2, #1
    beq  .L2
.L2:
    bx   lr            ; smyčka odstraněna
```

Kvalifikátor `volatile` to zakazuje — vynutí skutečné čtení při každém průchodu:

```
#define STATUS_REG (*(volatile uint32_t*)0x40000000)
```

```
wait_ready:
    ldr  r3, =0x40000000
.L1:
    ldr  r2, [r3]      ; znovu čte z paměti
    tst  r2, #1
    beq  .L1
    bx   lr
```



Obrázek 3.4. Vliv volatile: opakované čtení vs. načtení do registru

Proto ST všechny periferní registry v CMSIS definuje jako `volatile uint32_t`. Stejný problém nastane u příznaku nastaveného v přerušení:

```
volatile int flag = 0;
void EXTI0_IRQHandler(void) { flag = 1; }
void main(void) { while (!flag) { } }   // bez volatile by smyčka nikdy neskončila
```

Deklarace `volatile uint32_t x;` říká kompilátoru: *tato proměnná se může změnit kdykoli mimo kontrolu kompilátoru (hardwarem, přerušením), proto musí být při každém čtení znovu načtena z paměti a při každém zápisu zapsána přímo do paměti.* Kompilátor se nesmí spoléhat na žádné mezivýsledky, cache ani registry této hodnoty.

`volatile` a výkon

`volatile` je obousečné. Uvažme přepínání pinu ve smyčce:

```
volatile uint32_t *GPIOA_ODR = (uint32_t*)0x40020014;
void toggle(void) { *GPIOA_ODR ^= (1 << 5); }
for (int i = 0; i < 1000; i++) toggle();
```

Každý průchod udělá plné čtení–XOR–zápis přes sběrnici AHB, CPU čeká na periferii (bus stall) a **žádné** slučování, unrolling ani přemístění instrukcí není povoleno. Bez `volatile` by kompilátor sekvenci sloučil a smyčku rozbalil — což je u skutečné periferie nepřípustné, ale ukazuje cenu kvalifikátoru.

`volatile` je tedy **bezpečnostní brzda, ne optimalizační nástroj**. Pravidlo: hardwarové registry vždy `volatile` (to je správné); pro krátké kritické úseky raději inline asm než označovat celou proměnnou a přijít o veškerou optimalizaci v jejím okolí.

Meze `volatile`: bariéry a atomicita

`volatile` řeší **viditelnost** přístupu, ale **nezaručuje** dvě jiné věci:

Pořadí vůči neoznačeným přístupům. Kompilátor (a sběrnice) smí přístupy do paměti přeskupit, pokud to z pohledu jazyka C „nic nezmění". Tam, kde na pořadí závisí korektnost, je třeba **paměťová bariéra**. Nejlehčí je compiler memory barrier:

Paměťová bariéra
Idiom/instrukce bránící přeuspořádání paměťových přístupů přes ni (DMB/DSB/ISB nebo compiler barrier).

```
REG_CTRL = ENABLE;
asm volatile ("" ::: "memory");   // všechny dřívější zápisy hotové před bariérou
REG_START = 1;                     // a žádné přeuspořádání přes ni
```

Bez bariéry by kompilátor mohl zápisy přehodit, protože mu z pohledu C připadají záměnné — přitom na pořadí (nejdřív konfigurace, pak start) záleží. Stejně se zajistí, že CPU dokončí zápis do bufferu před spuštěním DMA:

```
memcpy(buffer, data, len);
asm volatile ("" ::: "memory");
DMA->START = (uint32_t)buffer;
```

Na úrovni hardwaru existují silnější bariéry — CMSIS `__DSB()` (data synchronization), `__ISB()` (instruction synchronization) a `__DMB()` (data memory barrier) — které vynutí pořadí i vůči sběrnici a pipeline.

Atomicitu. Operace `tick++` je *čtení–úprava–zápis*, tedy tři instrukce. Nastane-li přerušení uprostřed, hodnota se může ztratit. Atomicitu zajistí buď dočasné zakázání přerušení (kritická sekce z přednášky 2), nebo atomické instrukce `LDREX`/`STREX`.

> Tři různé mechanismy, které se nesmí zaměňovat: `volatile` = „čti a zapisuj skutečně a vždy"; **bariéra** vynutí pořadí přístupů; **kritická sekce** / **`LDREX`/`STREX`** zajistí atomicitu. Pro proměnnou sdílenou s přerušením potřebujete `volatile` **a** atomicitu; záměna těchto tří je jednou z nejčastějších příčin nereprodukovatelných chyb.

Globální a statické proměnné

V C rozlišujeme dva druhy statických proměnných. **Globální statická** (`static int c;` na úrovni souboru) má interní linkage — je viditelná jen v daném souboru. **Lokální statická** (`static int c;` ve funkci) si zachovává hodnotu mezi voláními:

```
void foo(void) {
    static int counter = 0;   // drží hodnotu mezi voláními
    counter++;
}
```

Obě žijí v datové sekci, **ne na zásobníku**: inicializovaná `.data` se kopíruje z Flash do RAM při startu, neinicializovaná `.bss` se vynuluje (přednáška 4). Statické proměnné drží RAM po celou dobu běhu, takže jejich počet je součástí rozpočtu paměti. Statická proměnná přístupná z ISR musí být `volatile` nebo chráněná kritickou sekcí.

MISRA-C k statickým proměnným váže několik pravidel: musí být inicializovány před použitím (**8.5**), lokální statické konstantou (**8.12**), nepoužité globální se odstraní nebo zdůvodní (**8.10**) a proměnné přístupné z ISR musí být správně označeny (**9.1/9.3**).

3.4 Volací konvence (AAPCS)

Aby spolu mohly spolupracovat funkce přeložené samostatně — a aby k sobě pasovaly objektové soubory od různých překladačů a knihovny dodané jako binárky — musí všechny dodržovat společnou dohodu: **ABI** (*Application Binary Interface*). Pro ARM ji definuje standard **AAPCS** (*Procedure Call Standard for the Arm Architecture*). ABI je pro binární kód totéž, co API pro zdrojový: kontrakt, který umožní skládat díly dohromady.

ABI
Application Binary Interface: binární kontrakt — předávání argumentů, zachování registrů, zarovnání.

AAPCS
Volací konvence ARM: r0–r3 argumenty, r4–r11 zachovat, návrat v r0, SP zarovnaný na 8 B.

Registry a předávání argumentů

AAPCS určuje, jak se předávají argumenty, kde je návratová hodnota, které registry smí funkce volně přepsat a které musí zachovat:



Obrázek 3.5. Rozdělení registrů podle AAPCS: caller-saved, callee-saved, speciální

Registr	Role	Kdo zachovává
`r0`–`r3`	argumenty a návratová hodnota	**caller** (caller-saved)
`r4`–`r11`	proměnné s delší životností	**callee** (callee-saved)
`r12` (IP)	dočasný (scratch)	caller-saved
`r13` (SP)	ukazatel zásobníku	speciální
`r14` (LR)	návratová adresa	speciální
`r15` (PC)	programový čítač	speciální

Rozdělení **callerc–** versus **callee-saved** je kompromis minimalizující zbytečné přesuny na zásobník: volaná funkce smí `r0`–`r3` libovolně přepsat (potřebuje-li je volající i po návratu, uloží si je sám), zatímco `r4`–`r11` musí před návratem obnovit (uloží je v prologu, obnoví v epilogu).

Předávání argumentů a zarovnání

Pravidla pro argumenty:

1. první **4** celočíselné argumenty / ukazatele → `r0`–`r3`,
2. **další** argumenty → na zásobník (v pořadí),
3. **návratová hodnota** → `r0` (32 b), `r0:r1` (64 b),
4. **struct return** → `r0` obsahuje adresu výsledku,
5. **zarovnání zásobníku** → `SP` musí být v okamžiku volání zarovnaný na **8 bajtů**.

Zarovnání zásobníku
AAPCS žádá ukazatel zásobníku zarovnaný na 8 bajtů v okamžiku volání funkce.

> Požadavek na 8bajtové zarovnání plyne z výkonu i korektnosti (64bitové přístupy, některé instrukce). Prolog proto často vyhradí o slovo víc. Kompilátor zarovnání zajišťuje automaticky — **při ručním psaní assembleru je třeba dávat pozor** (`push {r4, lr}` zarovná na 8 B, samotné

> `push {r4}` ne). Hardware Cortex-M navíc zarovná zásobník při vstupu do přerušení (bit `STKALIGN`).

Rámec zásobníku do hloubky

U vnořených volání se rámce skládají na sebe; návratové adresy uložené v jednotlivých rámcích tvoří **řetěz volání**, který lze rekonstruovat při ladění (*stack unwinding*). Pořadí položek v rámci (zásobník roste **dolů**, *full descending*):

```
vyšší adresy
+------------------+
|   Saved LR       |  <- předchozí rámec
+------------------+
|   Saved R4-R11   |  <- callee-saved registry
+------------------+
|   Lokální vars   |  <- aktuální rámec
+------------------+
|   Spilled temps  |
+------------------+  <- SP
|   Outgoing args  |  <- argumenty 5+ pro hlubší volání
+------------------+
nižší adresy
```

Přetečení zásobníku do oblasti `.bss`/`.data` je ve vestavných systémech reálné riziko — hlídá se buď strážním vzorem (*stack canary*), nebo hardwarově jednotkou MPU.

Příklady volání

Funkce se šesti argumenty ukazuje pravidla 1–2: čtyři jdou v registrech, dva na zásobník; volající po návratu zásobník uklidí:

```
main:
    push {lr}
    mov r0,#1 ; mov r1,#2 ; mov r2,#3 ; mov r3,#4   ; a..d v registrech
    mov r4,#6 ; push {r4}        ; f na zásobník
    mov r4,#5 ; push {r4}        ; e na zásobník
    bl  complex_func
    add sp, sp, #8               ; úklid zásobníku (2 args)
    pop {pc}

complex_func:                    ; R0=a R1=b R2=c R3=d, [SP+0]=e [SP+4]=f
    ldr r4, [sp, #0]             ; e
    ldr r5, [sp, #4]             ; f
    add r0, r0, r1               ; a+b
    add r0, r0, r2 ; add r0, r0, r3 ; add r0, r0, r4 ; add r0, r0, r5
    bx  lr                       ; výsledek v R0
```

Obecný prolog/epilog a funkce s lokálními proměnnými na zásobníku (`factorial`):

```
factorial:
    push {r4, r5, lr}       ; prolog: callee-saved + návrat
    sub  sp, sp, #8         ; lokální (result, i)
    mov  r1, #1 ; str r1, [sp, #4]   ; result = 1
    mov  r2, #1 ; str r2, [sp, #0]   ; i = 1
loop:
    ldr r2, [sp, #0] ; cmp r2, r0 ; bgt end_loop   ; while (i <= n)
    ldr r1, [sp, #4] ; mul r1, r1, r2 ; str r1, [sp, #4]  ; result *= i
```

```
    add r2, r2, #1 ; str r2, [sp, #0]                    ; i++
    b   loop
end_loop:
    ldr r0, [sp, #4]        ; návratová hodnota
    add sp, sp, #8          ; dealokace
    pop {r4, r5, pc}        ; epilog: restore and return
```

3.5 Členění kódu: moduly, knihovny a ovladače

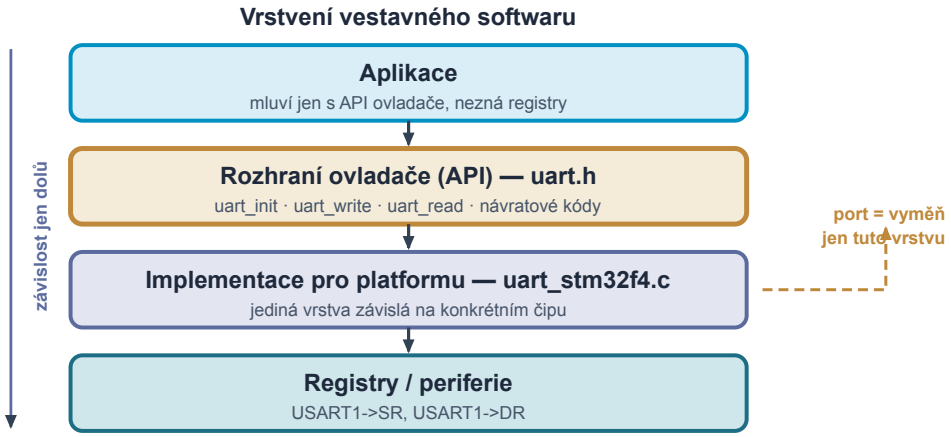
Předchozí části ukázaly, *jak* se z C stane obraz ve Flash. Tato část se ptá na něco jiného, ale stejně zásadního: **jak ten C kód uspořádat**, aby vznikl systém, který se dá rozšiřovat, testovat a udržovat. Vestavné projekty rostou z „jednoho `main.c`" do desítek souborů; bez disciplíny v členění se stanou nečitelnými a křehkými. Materiál této části nepochází z reference manuálu — je to inženýrská praxe, kterou student magisterského studia potřebuje stejně jako znalost registrů.

Kompilační jednotky, hlavička a zdroj

Kompilační jednotku
Translation unit: jeden soubor .c přeložený samostatně do .o; jednotky komunikují přes deklarace.

Překladač zpracovává každý `*.c` soubor samostatně jako **kompilační jednotku** (translation unit). Z každé vznikne jeden `*.o`; teprve linker je spojí. Z toho plynou dvě věci: jednotky o sobě navzájem vědí jen přes **deklarace** (hlavičky), a každou lze přeložit i otestovat zvlášť. Dobré členění tuto vlastnost využívá — rozdělí systém na **moduly**, z nichž každý má jednu odpovědnost a úzké, dobře definované rozhraní.

Vedoucí princip je **oddělení rozhraní od implementace** (*information hiding*). Uživatel modulu má vidět *co* modul dělá (rozhraní v `.h`), ne *jak* to dělá (implementace v `.c`). Změna implementace pak nevyžaduje změnu uživatelů; modul lze nahradit jiným se stejným rozhraním (např. ovladač pro jiný čip).



Obrázek 3.6. Vrstvení vestavného softwaru: aplikace, ovladač s API, HW vrstva, registry

Hlavička a zdroj: co kam patří

Modul se typicky skládá z dvojice `modul.h` (rozhraní) a `modul.c` (implementace).

Do hlavičky `modul.h` patří jen to, co uživatel potřebuje k použití modulu: deklarace veřejných funkcí, veřejné typy, konstanty a makra rozhraní. Hlavička se chrání proti vícenásobnému vložení **include guardem**:

```
#ifndef UART_H
#define UART_H
#include <stdint.h>
#include <stddef.h>

typedef enum { UART_OK = 0, UART_ERR_BUSY, UART_ERR_PARAM } uart_status_t;

uart_status_t uart_init(uint32_t baud);
uart_status_t uart_write(const uint8_t *data, size_t len);
int           uart_read_byte(void);   /* -1 = prázdno */
#endif /* UART_H */
```

Do zdroje `modul.c` patří implementace a vše, co je vnitřní. Pomocné funkce a stavové proměnné, které nemají být vidět zvenčí, se označí `static` — získají *interní linkage*, takže nezahltí globální jmenný prostor a linker je nevidí z jiných jednotek:

```
#include "uart.h"
#include "stm32f4xx.h"

static volatile uint8_t  rx_buf[64];   /* viditelné jen v tomto souboru */
static volatile uint16_t rx_head, rx_tail;

static void rx_push(uint8_t b) { /* … */ }   /* privátní pomocná funkce */

uart_status_t uart_init(uint32_t baud) { /* … */ }
```

Pravidlo „co nemusí být veřejné, je `static`" je jednou z nejlevnějších cest k udržitelnému kódu: zmenšuje rozhraní, brání nechtěným závislostem a umožňuje překladači agresivnější optimalizaci (vidí všechna použití symbolu).

Neprůhledné typy (opaque handle)

Chce-li modul před uživatelem skrýt i *strukturu* svých dat, použije **neprůhledný typ**: v hlavičce je jen dopředná deklarace, plná definice zůstává v `.c`. Uživatel drží jen ukazatel (*handle*) a strukturu nevidí:

Neprůhledný typ
Opaque handle: v hlavičce jen dopředná deklarace, definice skrytá v .c; uživatel drží jen ukazatel.

```
/* sensor.h — uživatel vidí jen jméno typu */
typedef struct sensor sensor_t;
sensor_t *sensor_create(uint8_t i2c_addr);
int       sensor_read(sensor_t *s, float *out);
void      sensor_destroy(sensor_t *s);
```

```
/* sensor.c — plná definice skrytá */
struct sensor {
    uint8_t addr;
    float   calib;
    uint32_t last_raw;
};
```

Výhody: uživatel se nemůže spolehnout na vnitřní pole (a tím si svázat ruce), velikost

a rozložení struktury lze měnit bez rekompilace uživatelů, a invarianty si hlídá výhradně modul. Daní je nutnost přidělit paměť handlu — ve vestavných systémech zpravidla **staticky** (pool objektů), ne přes `malloc`.

Návrh ovladače: API, fronty a callbacky

Ovladač (driver) je modul, který zapouzdřuje **periferii** a poskytuje aplikaci rozumné rozhraní místo přímé práce s registry. Dobré API ovladače má několik ustálených rysů:

- **životní cyklus**: `init`/`config` (jednorázové nastavení), `read`/`write` (provoz), případně `deinit`;
- **návratové kódy** místo tichého selhání — funkce vrací stavový `enum` (`UART_OK`, `UART_ERR_*`), aby volající mohl chybu ošetřit;
- **žádný veřejně přístupný globální stav**: stav je uvnitř (`static`, nebo handle), ne ve veřejných proměnných;
- **jednotná chybová a parametrová konvence** napříč ovladači (stejné názvy, stejný styl návratů) — usnadní psaní vyšších vrstev.

Rozhraní jednoduchého blokujícího UART ovladače (rozdělení hlavička/zdroj výše v této části) takový tvar má. Implementace `uart_write` skryje práci s registry a čekání na příznak:

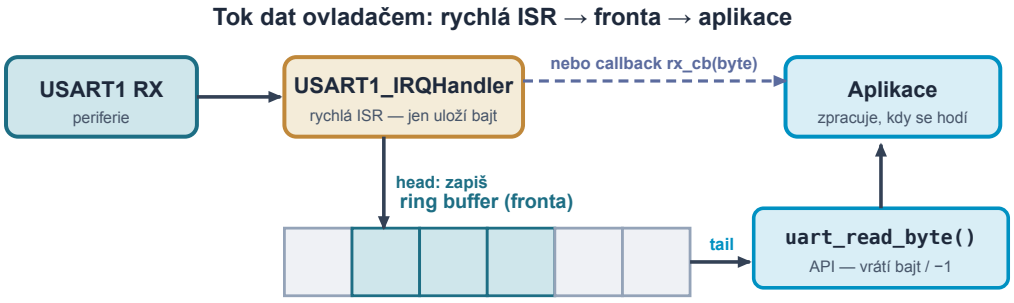
```
uart_status_t uart_write(const uint8_t *data, size_t len) {
    if (data == NULL) return UART_ERR_PARAM;
    for (size_t i = 0; i < len; i++) {
        while (!(USART1->SR & USART_SR_TXE)) { }   /* čekej na volný vysílač */
        USART1->DR = data[i];
    }
    return UART_OK;
}
```

Aplikace pak nepracuje s jediným registrem — používá jen API:

```
uart_init(115200);
uart_write((const uint8_t*)"ready\n", 6);
```

Asynchronní rozhraní: fronty a callbacky

Blokující `uart_write` je v pořádku pro krátké výpisy, ne pro proudová data. Výkonný ovladač přenese práci do **přerušení** a aplikaci nabídne **frontu** (ring buffer) nebo **callback**. Tím se uplatní vzor „rychlá ISR + odložené zpracování" z části 3.2, ale zabalený do čistého rozhraní.



ISR jen uloží bajt do fronty (head), aplikace si jej vyzvedne přes API (tail) — nebo ji ovladač zavolá callbackem

fronta odděluje rychlou obsluhu od pomalejšího zpracování; callback dědí omezení ISR (krátký, neblokující)

Obrázek 3.7. Tok dat ovladačem: RX přerušení → ring buffer → aplikace; volitelný callback

Příjem: ISR jen uloží bajt do kruhové fronty, aplikace si jej vyzvedne, kdy se jí hodí:

```
/* uart.c */
static volatile uint8_t  rx_buf[64];
static volatile uint16_t head, tail;

void USART1_IRQHandler(void) {                /* rychlá ISR */
    if (USART1->SR & USART_SR_RXNE) {
        uint16_t next = (head + 1u) % sizeof rx_buf;
        if (next != tail) { rx_buf[head] = (uint8_t)USART1->DR; head = next; }
    }
}

int uart_read_byte(void) {                    /* API: -1 = prázdno */
    if (tail == head) return -1;
    uint8_t b = rx_buf[tail];
    tail = (tail + 1u) % sizeof rx_buf;
    return b;
}
```

Alternativou k frontě je **callback** — ovladač zavolá funkci dodanou aplikací při události. To je užitečné pro reaktivní styl a pro **oddělení závislostí**: ovladač nezná aplikaci, jen typ funkce, kterou má zavolat:

```
typedef void (*uart_rx_cb_t)(uint8_t byte);
static uart_rx_cb_t rx_cb;

void uart_on_rx(uart_rx_cb_t cb) { rx_cb = cb; }   /* registrace */

void USART1_IRQHandler(void) {
    if (USART1->SR & USART_SR_RXNE) {
        uint8_t b = (uint8_t)USART1->DR;
        if (rx_cb) rx_cb(b);                       /* zpětné volání aplikace */
    }
}
```

Předání funkce **ukazatelem na funkci** (callback) je zároveň základní forma *dependency injection*: vyšší vrstva si „vstříkne" chování do nižší, aniž by mezi nimi vznikla pevná závislost. Callback volaný z ISR ovšem dědí všechna její omezení — musí být krátký a neblokující.

Přenositelnost a testovatelnost

Oddělení rozhraní od implementace přináší dvě praktické výhody. **Přenositelnost**: oddělíme-li čistě HW-závislou vrstvu (přístup k registrům) od logiky, je port na jiný čip otázkou výměny jedné vrstvy. Proto bývá ovladač rozdělen na obecné API (`uart.h`) a platformní implementaci (`uart_stm32f4.c`, `uart_stm32u5.c`) — aplikace nad API se nemění.

Testovatelnost: logiku oddělenou od hardwaru lze testovat na PC (host) s falešnou (*mock*) implementací HW vrstvy. Závislost na hardwaru se vstříkne ukazatelem na funkce (injekce rozhraní), takže v testu se místo skutečné periferie podstrčí simulace:

```
typedef struct {
    void    (*write_reg)(uint32_t addr, uint32_t val);
    uint32_t(*read_reg )(uint32_t addr);
} hw_ops_t;
```

```c
/* v testu se dosadí mock, na cíli skutečný přístup k registrům */
void protocol_init(const hw_ops_t *hw);
```

Tyto vzory — moduly s úzkým rozhraním, neprůhledné typy, ovladač s návratovými kódy a callbacky, oddělená HW vrstva — jsou tím, co dělá z vestavného kódu **knihovnu**, kterou lze sdílet mezi projekty, a ne jednorázový skript. Vlastní deterministická správa paměti (pool, ring, slab alokátory), která tyto knihovny často doplňuje, je tématem pozdější přednášky.

3.6 Limity vestavného prostředí

Jazyk C dovoluje napsat tentýž program pro PC i pro mikrokontrolér. Co se ale na PC ztratí v šumu, je na vestavném cíli zásadní: paměť se počítá na bajty, čas na takty a selhání nemá kdo zachytit. Tato část pojmenovává **mantinely**, uvnitř kterých se vestavný kód musí pohybovat — a teprve z nich plyne smysl pravidel MISRA v části 3.7. Mnohé z těchto omezení přímo navazují na to, co jsme viděli u překladu: rozložení sekcí, zásobník, `volatile`, nedefinované chování.



Obrázek 3.8. Mantinely vestavného prostředí: paměť, čas, runtime, robustnost

Omezené zdroje

Mikrokontrolér řady STM32L4 má jednotky stovek kB Flash a desítky až stovky kB SRAM — o tři až čtyři řády méně než PC. Není virtuální paměť, není stránkování, není odkládací prostor. Z toho plyne:

- **velikost kódu** (`.text`) je rozpočtová položka — proto `-Os`, proto odlehčená libc, proto se zvažuje každá přilinkovaná knihovna;
- **RAM footprint** je součtem `.data`, `.bss`, zásobníku a haldy; přetečení nehlásí žádný „out of memory" dialog — projeví se přepsáním sousední paměti a pádem;
- **statické rozvržení**: kolik paměti program zabere, musí být **známé při překladu**. Nástroj `size` a `.map` soubor (přednáška 4) jsou proto běžnou součástí buildu.

Důsledek pro styl kódu: paměť se alokuje předem a staticky; datové struktury mají pevnou, ohraničenou velikost; velké tabulky se umísťují do Flash (`const`), ne do RAM.

WCET
Worst-Case Execution Time: nejhorší možná doba vykonání úseku; musí být ohraničená.

Determinismus a čas reálného času

Vestavný systém často **řídí fyzický proces** — motor, převodník, komunikační linku — kde nezáleží jen na tom, že se výpočet provede *správně*, ale i *včas*. Klíčovou veličinou je **WCET** (Worst-Case Execution Time), nejhorší možná doba vykonání úseku. Aby byla ohraničená, musí být ohraničené i všechno, na čem závisí:

- **žádné neohraničené smyčky** — počet iterací musí mít horní mez;
- **žádná operace s nepředvídatelnou dobou** — typicky `malloc` (dále v této části) nebo blokující I/O;
- **omezený jitter** (rozptyl latence) — proto se kritická obsluha drží v krátkých ISR s vysokou prioritou (přednáška 2) a těžká práce se odkládá.

Determinismus je důvod, proč se ve vestavných systémech nepoužívají konstrukce, jejichž doba běhu závisí na datech nepředvídatelně. „Průměrně rychlé" zde neznamená „dost rychlé" — rozhoduje nejhorší případ.

Dynamická alokace, zásobník a rekurze

`malloc`/`free` jsou na vestavném cíli problematické hned z několika důvodů, které se sčítají:

- **nedeterministická doba** — alokátor prochází seznam volných bloků; doba závisí na stavu haldy, takže porušuje WCET;
- **fragmentace** — střídavé alokace a uvolňování rozdrobí haldu na malé kousky; časem selže i alokace, na kterou by *celkově* paměti bylo dost;
- **selhání za běhu bez záchrany** — `malloc` vrátí `NULL`, ale vestavný program často nemá rozumnou cestu, jak se z nedostatku paměti zotavit;
- **obtížná analyzovatelnost** — statická analýza nedokáže ohraničit spotřebu paměti programu, který alokuje dynamicky.

Proto MISRA dynamickou alokaci **zakazuje** (pravidlo 21.3) a vestavné systémy ji nahrazují **statickou alokací** (vše předem), případně **deterministickými alokátory** s pevnou strukturou — fixní *pool* objektů stejné velikosti, kruhový buffer, zásobníkový alokátor. Ty mají ohraničenou dobu i spotřebu a nefragmentují. Návrhu takových alokátorů je věnována samostatná pozdější přednáška.

Zásobník a zákaz rekurze

Zásobník (část 3.4) má ve vestavném systému **pevně vyhrazenou velikost** z linker scriptu — typicky jednotky kB. Každé volání na něm zabere rámec; hloubka volání proto musí být ohraničená. To přímo vylučuje **rekurzi**: rekurzivní funkce má hloubku závislou na datech, takže její spotřebu zásobníku nelze staticky ohraničit a hrozí přetečení do sousedních sekcí.

Přetečení zásobníku je obtížně odhalitelné — neprojeví se výjimkou, ale tichým přepsáním `.bss`/`.data` nebo jiného rámce. Hlídá se proto aktivně: strážním vzorem (*stack canary* — známá hodnota na dně zásobníku, jejíž změna signalizuje přetečení), nebo hardwarově jednotkou **MPU**, která přístup za hranici zásobníku zachytí jako chybu.

Bez operačního systému a plné knihovny

Bare-metal prostředí postrádá služby, které jsou na PC samozřejmé, a i to, co je k dispozici, má svou cenu:

- **`printf` je drahý** — formátování čísel zvětšuje kód a u plné varianty vyžaduje i podporu `float`; do výkonové cesty nepatří (a do ISR tím spíše);

- **plovoucí řádová čárka** bez FPU znamená **soft-float** — knihovní emulaci v řádu stovek instrukcí na operaci; tam, kde stačí, se volí **pevná řádová čárka** (fixed-point);
- **žádný souborový systém, žádné `stdin`/`stdout`** — `printf` se musí ručně napojit přes `_write` (přednáška 4);
- **žádné výjimky a RTTI** — v C neexistují vůbec; v C++ se pro vestavné systémy zpravidla vypínají (`-fno-exceptions -fno-rtti`), protože nesou nedeterministickou režii a velikost.

Chybí tedy záchranná síť: na PC neošetřená výjimka program „jen" ukončí, na MCU se chyba projeví nedefinovaným chováním běžícího zařízení.

Aritmetika a nedefinované chování

Velikosti typů a chování při přetečení jsou v C zčásti **implementačně závislé** — nebezpečí, které se na různých cílech projeví různě. Proto vestavný kód:

- používá **typy s pevnou šířkou** (`uint32_t`, `int16_t` z `<stdint.h>`) místo `int`, jehož šířka není standardem pevně dána;
- hlídá **přetečení**: přetečení **znaménkového** celého čísla je v C **nedefinované chování** (kompilátor s ním smí počítat jako s nemožným a podle toho optimalizovat), zatímco u **bezznaménkového** je definováno modulo;
- vyhýbá se **míchání znaménkovosti** v jednom výrazu (implicitní konverze mohou změnit význam porovnání) a **implicitním zúžením** (přiřazení širšího typu do užšího).

To je přesně třída chyb, kterou cílí MISRA svým **modelem esenciálních typů** (část 3.7): omezit implicitní konverze tak, aby výsledek nezávisel na šířce `int` konkrétního překladače.

Souběh s přerušeními

Hlavní smyčka a obsluhy přerušení sdílejí data a běží „současně" v tom smyslu, že ISR může přerušit hlavní kód kdykoli. To zavádí **souběh** i na jednojádrovém procesoru:

- proměnná sdílená s ISR musí být `volatile` (viditelnost, část 3.3) **a** přístup k ní atomický;
- víceslovní nebo *čtení–úprava–zápis* operace (`tick++`, 64bitová proměnná) nejsou atomické a vyžadují **kritickou sekci** (krátké zakázání přerušení) nebo `LDREX`/`STREX`;
- funkce volaná z hlavní smyčky i z ISR musí být **reentrantní** — nesmí se spoléhat na sdílený stav, který by druhé volání porušilo.

Souběh je zdrojem chyb, které se projevují vzácně a nereprodukovatelně (část 3.3) — proto se sdílení dat mezi kontexty drží na minimu a explicitně se chrání.

Chyby bez výjimek: obrana do hloubky

Bez výjimek se chyby ošetřují **návratovými kódy** (část 3.5) a kód musí počítat s tím, že hardware může selhat. Vestavná robustnost stojí na několika vrstvách:

- **kontrola vstupů a návratových kódů** na každé hranici (žádné tiché ignorování chyby);
- **watchdog** — hardwarový časovač, který restartuje systém, pokud ho program přestane pravidelně „krmit"; pojistka proti zaseknutí v nečekaném stavu;
- **fault handlery** — Cortex-M má vektory `HardFault`, `MemManage`, `BusFault`, `UsageFault`, které zachytí neplatný přístup či instrukci; smysluplná obsluha (log, řízený restart) je lepší než nekonečná smyčka;
- **odolnost vůči poruše bitu** (single-event upset) v náročných aplikacích — kontrolní součty, redundance, ECC paměti.

Watchdog
Hardwarový časovač, který restartuje systém, pokud ho program přestane pravidelně obsluhovat.

Vestavný systém často nemá obsluhu u sebe; musí se **zotavit sám**. To formuje celý styl

programování — od kontroly každého návratového kódu po watchdog jako poslední záchranu.

3.7 MISRA-C: pravidla, vynucování a důsledky

Předchozí části pojmenovaly meze vestavného prostředí; z nich plyne smysl pravidel, která tato část kodifikuje. MISRA-C je oborový soubor pravidel, jenž z dobré praxe psaní C dělá **vynutitelný standard**. Projdeme, co MISRA je, kde se normativně vyžaduje, jaká pravidla nese, jak se strojově vynucuje a jaké reálné havárie stály za jejím vznikem.

Co je MISRA a kde se vyžaduje

MISRA-C
Soubor pravidel pro C v bezpečnostně kritických systémech; vyhýbá se nedefinovanému chování.

MISRA-C (Motor Industry Software Reliability Association) je soubor pravidel pro psaní C v bezpečnostně kritických systémech. Vznikl v automobilovém průmyslu (první vydání 1998, zásadní revize MISRA C:2012 a pozdější doplňky), ale dnes se používá napříč odvětvími, kde selhání softwaru ohrožuje životy nebo majetek. Často je **smluvně nebo normativně vyžadován**:

- **ISO 26262** — funkční bezpečnost silničních vozidel,
- **DO-178C** — software v civilním letectví,
- **IEC 61508** — funkční bezpečnost elektronických systémů obecně,
- **IEC 62304** — software zdravotnických prostředků.

Smyslem není „lepší styl", ale **odstranění celých tříd chyb** ještě před spuštěním. Cílem je vyhnout se konstrukcím vedoucím k **nedefinovanému** (*undefined*), **nespecifikovanému** (*unspecified*) a **implementačně závislému** (*implementation-defined*) chování — tedy programům, jejichž běh závisí na konkrétním překladači, úrovni optimalizace nebo platformě (přesně to, co při přechodu z `-O0` na `-O2` selže, část 3.3). Motivace je trojí: **determinismus**, **přenositelnost** a **auditovatelnost**.

Vybraná pravidla

Pravidla MISRA pokrývají celý jazyk; následující výběr ukazuje jejich povahu — u každého je podstatné nejen *co* zakazuje, ale *jakou chybu tím předchází*.

Oblast	Pravidlo (smysl)	Jaké chybě brání
Paměť	žádná dynamická alokace (21.3)	fragmentace, nedeterminismus, OOM bez záchrany
Tok řízení	žádná rekurze (17.2)	neohraničený zásobník, přetečení
Typy	pevná šířka, model esenciálních typů	implicitní konverze, závislost na šířce `int`
Typy	žádné míchání znaménkovosti	chybné porovnání signed/unsigned
Aritmetika	hlídat přetečení, žádné UB	nedefinované chování po optimalizaci
Tok řízení	jediný bod návratu, žádné `goto`	nečitelný a neanalyzovatelný tok
Tok řízení	vždy složené závorky u `if`/`for`	„visící else", chybně přiřazené větve
Ukazatele	aritmetika jen v rámci pole	přístup mimo paměť objektu
Preprocesor	omezit makra, závorkovat	pasti expanze, přednost `static inline`/`const`
Inicializace	proměnné inicializovat před použitím	čtení neurčité hodnoty
ISR	sdílené proměnné `volatile` (9.x)	optimalizace odstraní čtení/zápis

Mnohá z těchto pravidel jsme potkali už dříve jako dobrou praxi — pevné typy a `volatile` v části 3.3, zákaz `malloc` a rekurze v části 3.6, závorkování maker v části 3.2. MISRA z nich dělá **vynutitelný standard**.

Kategorie pravidel, odchylky a vynucování

Ne všechna pravidla mají stejnou závaznost. MISRA C:2012 je dělí do tří kategorií:

- **povinná** (*mandatory*) — nesmí být porušena za žádných okolností;
- **vyžadovaná** (*required*) — smí být porušena jen formálně zdůvodněnou **odchylkou** (*deviation*);
- **doporučená** (*advisory*) — porušení se eviduje, ale nevyžaduje formální zdůvodnění.

Pravidla se dále dělí na **rozhodnutelná** (*decidable* — nástroj o nich rozhodne ze zdrojového kódu) a **nerozhodnutelná** (*undecidable* — vyžadují posouzení, např. hodnotu za běhu). Když je dodržení pravidla v konkrétním místě neproveditelné nebo by uškodilo, použije se **proces odchylky**: porušení se zdokumentuje, zdůvodní a schválí. Výsledkem auditu je **matice shody** (*compliance matrix*), která dokládá, že každé pravidlo je buď dodrženo, nebo má schválenou odchylku — to je podstata certifikace.

Vynucování statickou analýzou

MISRA se nedodržuje „dobrou vůlí", ale **strojovou kontrolou**. Statické analyzátory (cppcheck, PC-lint/Flexelint, Polyspace, Coverity, Helix QAC) procházejí zdrojový kód bez jeho spuštění a hlásí porušení pravidel. Zařazují se přímo do buildu — typicky tak, že porušení **povinného** pravidla zastaví překlad stejně jako chyba kompilátoru. Tím se chyba odhalí v okamžiku psaní, ne při testování na cíli (kde je řádově dražší ji najít) nebo až u zákazníka.

Statická analýza zachytí i to, co překladač mlčky přejde: implicitní konverze, čtení neinicializované proměnné, nedosažitelný kód, možné přetečení, porušení pořadí. Je proto součástí stejné disciplíny jako zapnutá varování překladače (`-Wall -Wextra -Werror`) — stroj

kontroluje to, co člověk přehlédne.

Důsledky nedodržení

Pravidla MISRA nejsou akademická — vznikla jako reakce na **reálné havárie**, kde chyba v C kódu měla katastrofální následky. Tři poučné příklady:

- **Ariane 5 (1996)** — převod 64bitového čísla na 16bitové **přeteklo** (neošetřená konverze), naváděcí počítač havaroval a raketa se po 37 sekundách letu sama zničila. Přesně třída chyby, kterou cílí pravidla o konverzích a přetečení.
- **Therac-25** — dávkovací chyby ozařovače stály životy; v pozadí byly **souběhy** (race conditions) mezi vlákny a chybějící kontrola stavů. Připomínka, proč MISRA tlačí na ošetření souběhu a determinismus.
- **Toyota „unintended acceleration"** — soudní analýza kódu odhalila mimo jiné hluboké volání, rozsáhlé globální stavy a nedostatečnou ochranu zásobníku; argumentace stála na tom, že kód **nesplňoval** principy, které MISRA kodifikuje (ohraničený zásobník, žádná rekurze, kontrola integrity).

Poučení je společné: ve vestavném systému nemá kdo chybu zachytit za běhu. Disciplína **posune odhalení chyby co nejblíže k jejímu vzniku** — do statické analýzy a recenze, místo do provozu zařízení.

> MISRA pravidla nejsou byrokracie — každé pojmenovává konkrétní třídu chyb, které ve vestavných systémech opakovaně způsobily reálné škody. Dodržování se kontroluje **staticky** jako součást překladu, takže porušení se odhalí dřív, než kód doběhne na cíl.

> **Shrnutí oddílu 3.7.** MISRA-C je oborový soubor pravidel, který z dobré praxe psaní C dělá **vynutitelný standard**; v automobilovém, leteckém a zdravotnickém kódu je normativně vyžadován. Pravidla se dělí na **mandatory / required / advisory** a od required se lze odchýlit jen s doložením (deviation). Zaměřují se na **nedefinované a implementačně závislé chování**, implicitní konverze a konstrukce náchylné k chybám. Vynucuje je **statická analýza** jako součást překladu — levněji a dřív než ladění na cíli.

3.8 Ladění a vhled do běžícího čipu

Statická analýza hlídá zdroj **před** během. Chyby, které jí proklouznou — špatná hodnota z čidla, závada časování, přetečení zásobníku — se ovšem projeví **až za běhu**, a na mikrokontroléru není ani obrazovka, ani konzole pro ladicí výpis. Potřebujeme se proto dostat **dovnitř běžícího čipu**: zastavit ho, přečíst paměť a registry, krokovat instrukce. Vestavné ladění není „printf na konzoli", ale **hardwarové** nahlédnutí do jádra zvenčí.

Ladicí rozhraní: JTAG a SWD

Čip má vyvedené **ladicí rozhraní**, kterým ho ovládá externí sonda (ST-Link, J-Link). Historicky se používá **JTAG** — pět vodičů, řetězení více zařízení a navíc boundary scan (test propojů na desce). Dnešním standardem na Cortex-M je **SWD** (Serial Wire Debug) — jen **dva vodiče** (SWDIO, SWCLK). Přes ně sonda jádro **zastaví a krokuje**, **čte a zapisuje** paměť i registry a **programuje** Flash; na hostiteli běží `OpenOCD` nebo `pyOCD` a nad nimi `GDB`.

Ladicí rozhraní
Vyhrazené piny, jimiž externí sonda ovládá jádro: zastaví a krokuje ho, čte a zapisuje paměť i registry a programuje Flash.

JTAG
Pětivodičové ladicí a testovací rozhraní, umožňuje řetězit více zařízení a testovat propoje (boundary scan).

SWD
Serial Wire Debug: dvouvodičové ladicí rozhraní ARM (SWDIO, SWCLK), funkčně rovnocenné JTAGu pro ladění, dnešní standard na Cortex-M.

CoreSight
Sada hardwarových ladicích a trasovacích bloků integrovaných v jádře Cortex-M; hlídá adresy a sbírá trasu bez instrumentování kódu a bez zpomalení.

FPB
Flash Patch and Breakpoint: blok, který umístí breakpoint na adresu instrukce i v ROM nebo Flash a zastaví jádro, jakmile k té instrukci dojde.

DWT
Data Watchpoint and Trace: blok, který sleduje přístupy k datům (watchpoint na adresu nebo hodnotu) a obsahuje čítač taktů jádra pro profilování.

On-chip debug: CoreSight a trace

Ladění není softwarová emulace — Cortex-M má **vestavěné hardwarové bloky** souhrnně zvané **CoreSight**. **FPB** (Flash Patch and Breakpoint) umístí **breakpoint** na adresu instrukce, i v paměti jen pro čtení, a zastaví jádro, jakmile k té instrukci dojde. **DWT** (Data Watchpoint and Trace) hlídá **data** — watchpoint zastaví při čtení nebo zápisu zadané adresy či hodnoty (odpověď na otázku „kdo mi přepisuje proměnnou?“) — a obsahuje **čítač taktů** jádra pro přesné profilování úseků kódu. Klíčové je, že hardware sleduje adresy sám: žádné instrumentování kódu, žádné zpomalení běhu.

Trace: vhled bez zastavení

Breakpoint program zastaví; někdy ale potřebujeme vidět chování **za běhu**, aniž bychom ho narušili. **ITM** spolu s výstupem **SWO** (Single Wire Output) pošle textovou trasu ven **jediným pinem**, s časovými značkami a bez zdržení jádra; **SWV** (Serial Wire Viewer) nad tím živě sleduje proměnné, profiluje a trasuje výjimky; **ETM** rekonstruuje instrukční trasu (tok programu a pokrytí kódu). Výstup je tedy jen otázka jediného zápisu:

```
ITM_SendChar('A');          // znak ven přes SWO; hostitel ho zobrazí v reálném čase
```

ITM
Instrumentation Trace Macrocell: blok, který posílá ven textovou trasu a události jediným pinem (SWO), s časovými značkami a bez zdržení jádra.

printf bez UARTu: semihosting a RTT

Když nemáme UART ani displej, lze textový výstup získat dvěma způsoby. **Semihosting** nechá `printf`, `scanf` i práci se soubory vyřídit **debugger na hostiteli**; je pohodlný, ale **každé volání zastaví jádro**, takže patří jen do ladění, nikdy do provozu. **RTT** (Real-Time Transfer) místo toho píše do rychlého **kruhového bufferu v RAM**, který sonda průběžně vyčítá — `printf` tak nezastavuje jádro a je použitelný i za provozu.

Semihosting
Mechanismus, kdy volání jako printf, scanf nebo práci se soubory vyřídí debugger na hostiteli; každé volání ale zastaví jádro, takže je vhodný jen při ladění.

RTT
Real-Time Transfer: rychlý kruhový buffer v RAM, který sonda průběžně vyčítá, takže textový výstup nezastavuje jádro a je použitelný i za běhu.

GDB, profilování a fault analýza

Celé ladění řídíme z hostitele přes `GDB` připojený k sondě:

```
(gdb) target extended-remote :3333   # připoj se přes OpenOCD k sondě
(gdb) load                           # nahraj .elf do Flash
(gdb) break main                     # breakpoint
(gdb) monitor reset halt             # reset a zastav
(gdb) stepi                          # krok po instrukci
(gdb) info registers                 # R0–R15, xPSR
(gdb) x/16xw 0x20000000              # vypiš 16 slov SRAM
(gdb) watch sensor_value             # zastav při změně proměnné
```

On-chip debug a GDB dohromady znamenají: zastav, prohlédni **registry, paměť i zásobník**, krokuj a hledej příčinu.

Profilování, pokrytí a fault analýza

Tytéž bloky dávají i **inženýrské míry**, ne jen breakpointy. Čítač taktů DWT a SWV ukážou, **kde program spotřebuje nejvíce taktů** (profilování). ETM trasa doloží, **co se skutečně vykonalo** (pokrytí kódu — požadavek bezpečnostní certifikace). A po pádu do **HardFault** lze z odloženého rámce a stavového registru určit **příčinu**:

```
void HardFault_Handler(void) {
    uint32_t *sp = (uint32_t*)__get_PSP();   // odložené registry (R0..R3, R12, LR, PC, xPSR)
    uint32_t pc  = sp[6];                     // adresa chybující instrukce
    uint32_t cfsr = SCB->CFSR;                // proč: konkrétní příznak chyby
```

HardFault
Výjimka vyvolaná závažnou chybou (neplatný přístup, nezarovnání, špatná instrukce); z odloženého rámce a stavového registru CFSR lze určit adresu a příčinu.

```
    log_fault(pc, cfsr);                    // PC pak mapuj přes .map na řádek zdroje
}
```

Adresu `pc` stačí dohledat v `.map` souboru a lze ji mapovat na konkrétní řádek zdroje.

> **Shrnutí oddílu 3.8.** Ladicí rozhraní **SWD/JTAG** dovolí zastavit jádro, číst paměť i registry a programovat Flash (na hostiteli OpenOCD a GDB). **CoreSight** přidává v hardwaru **FPB** breakpointy, **DWT** watchpointy a čítač taktů — bez instrumentování a bez zpomalení. **Trace** (ITM/SWO, SWV, ETM) a **semihosting/RTT** dají vhled i `printf` bez UARTu. Tytéž bloky slouží k **profilování, pokrytí kódu a fault analýze**: statická analýza hlídá u zdroje, debugger za běhu.

3.9 Závěr — od zdroje k běžícímu obrazu a zpět

Spojme celou přednášku do jediného toku. Zdrojový soubor v C projde **preprocesorem** (makra, `#include`), který vydá čistý C kód; **kompilátor** jej přes fáze analýzy a optimalizace přeloží do assembleru, přičemž každou funkci obalí prologem a epilogem podle **AAPCS** a u hardwarových přístupů respektuje `volatile`; **assembler** vytvoří objektový soubor. Zde končí *překlad* — téma této přednášky. Slinkování a **umístění** sekcí do Flash a SRAM, připojení libc a **startup kód** (kopie `.data`, nulování `.bss`, předání řízení do `main`) už patří paměti a sběrnicím v **přednášce 4**, kam jsme proto odkázali i linker.

Souběžně s tímto řetězem stojí **inženýrská vrstva**: kód členěný do modulů s úzkým rozhraním, periferie zabalené do ovladačů s API, vše uvnitř mantinelů vestavného prostředí a držené disciplínou **MISRA** a statickou analýzou. A když chyba přece projde až do běžícího obrazu, máme **ladicí řetěz** z oddílu 3.8 — SWD, CoreSight a GDB —, kterým se do čipu podíváme a příčinu najdeme. Každý článek, od přepínačů překladače přes členění do knihoven a statickou analýzu po hardwarové ladění, je věc, které programátor rozumí a může ji ovlivnit.

> **Závěr.** Překlad ani běh nejsou černá skříňka. Mezi C zdrojem a obrazem ve Flash leží řetěz srozumitelných transformací — preprocesor, kompilátor (s optimalizací a ABI), assembler — a za ním paměť, startup a běh, do něhož vidíme ladicím rozhraním. Vestavné prostředí klade tvrdé mantinely (omezenou paměť, determinismus, žádnou záchrannou síť) a MISRA z dobré praxe dělá vynutitelný standard. **Spolehlivý vestavný systém vzniká tam, kde programátor rozumí každému kroku mezi zdrojem a křemíkem — a respektuje meze, v nichž se pohybuje.**

3.10 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 3.2 — Preprocesor: od konstant po složité konstrukce

1. Vyjmenujte úrovně abstrakce přístupu k periferiím a uveďte kompromis, který každá řeší.
2. V čem spočívá síla i nebezpečí toho, že preprocesor pracuje jako čistá textová substituce před překladem? (Ilustrujte na spojování jmen operátorem `##`.)
3. Proč se každý argument makra i celý výraz obalují závorkami? Uveďte chybu, kterou to brání.
4. Jaké jsou požadavky na ISR a proč? Vysvětlete vzor „rychlá ISR + odložené zpracování”.
5. Proč se ve vestavném kódu dává přednost `static inline` funkci před parametrickým makrem?

K oddílu 3.3 — Kompilátor: překlad a optimalizace

1. Proč překlad probíhá v oddělených fázích (preprocesor → překladač → assembler → linker) a co která fáze do výsledku přidává?
2. Popište prolog a epilog funkce na `-O0`. Co se ukládá na zásobník a proč?
3. Jaké transformace provede `-O2` s jednoduchou smyčkou sčítající pole celých čísel (v textu funkce `sum()`) a proč je výsledek rychlejší? Uveďte alespoň dvě konkrétní transformace.
4. Vysvětlete na čekací smyčce, proč je u hardwarového registru nutné `volatile`.
5. Čím se liší `volatile`, paměťová bariéra a kritická sekce? Pro jaký problém je každá z nich?

K oddílu 3.4 — Volací konvence (AAPCS)

1. Co je ABI a proč ho musí dodržovat všechny objektové soubory a knihovny?
2. Vysvětlete rozdíl mezi caller-saved a callee-saved registry. Které to jsou v AAPCS?
3. Proč musí být předání argumentů a návratové hodnoty pevně dané konvencí, a jak to AAPCS řeší pro celočíselné argumenty?
4. Co znamená 8bajtové zarovnání zásobníku a proč je `push {r4}` samotné porušení AAPCS?
5. Popište pořadí položek v rámci zásobníku a jak se hlídá jeho přetečení.

K oddílu 3.5 — Členění kódu: moduly, knihovny a ovladače

1. Co je kompilační jednotka a jak spolu jednotky komunikují? Proč je to základ modularity.
2. Co patří do hlavičky a co do zdroje? K čemu slouží include guard a `static` u funkcí.
3. Vysvětlete neprůhledný typ (opaque handle) a uveďte, co tím modul získá.
4. Proč dobré API ovladače skrývá vnitřní stav a chyby hlásí návratovými kódy? Co tím získá volající kód?
5. Jak se realizuje asynchronní ovladač přes frontu a přes callback? Jak souvisí s vzorem „rychlá ISR".
6. Jak oddělení HW vrstvy přispívá k přenositelnosti a testovatelnosti kódu?

K oddílu 3.6 — Limity vestavného prostředí

1. Proč je ve vestavném systému velikost kódu i RAM rozpočtová položka a jak se sleduje?
2. Co je WCET a které konstrukce ho porušují? Proč „průměrně rychlé" nestačí.
3. Vyjmenujte důvody, proč se na vestavném cíli vyhýbáme `malloc`. Čím ho nahradíme?
4. Proč je zakázána rekurze a jak se hlídá přetečení zásobníku?
5. Proč je přetečení znaménkového celého čísla nebezpečnější než bezznaménkového?
6. Ve vestavném systému chybí výjimky a záchranná síť. Jakými vrstvami se proto buduje robustnost proti chybám za běhu? Uveďte alespoň tři.

K oddílu 3.7 — MISRA-C: pravidla, vynucování a důsledky

1. Co je MISRA-C, kde se normativně vyžaduje a jakým třem kategoriím chování se vyhýbá?
2. Vysvětlete kategorie pravidel (mandatory/required/advisory) a proces odchylky.
3. Jak se MISRA vynucuje a proč je statická analýza levnější než ladění na cíli?
4. Uveďte na příkladu Ariane 5, jakou třídu chyb řeší pravidla o konverzích.
5. Proč je MISRA spíš disciplína návrhu než seznam zákazů?

K oddílu 3.8 — Ladění a vhled do běžícího čipu

1. Čím se liší JTAG a SWD a proč je SWD na Cortex-M dnešním standardem?
2. Vysvětlete rozdíl mezi breakpointem (FPB) a watchpointem (DWT) a kdy který použijete.
3. Co umožní trace (ITM/SWO, ETM), čeho se breakpointem nedosáhne?
4. Porovnejte semihosting a RTT z hlediska vlivu na běh jádra.
5. Jak po pádu do HardFault zjistíte adresu a příčinu chyby?

4

Paměť a systémové sběrnice

> Tato kapitola nahlíží na **celý čip jako na síť pamětí a periferií spojených sběrnicemi**. Řeší, jak je na mikrokontroléru uspořádaná paměť a jakými cestami se k ní i k periferiím dostane procesor a další masteři — a proč to rozhoduje o výkonu i o správnosti.

Obsah kapitoly

4.1 Paměťová mapa: jeden adresový prostor pro všechno
4.2 Linker a umístění v paměti
4.3 Memory-mapped I/O: periferie jako adresy
4.4 AMBA a bus matrix: jak je čip propojen
4.5 Arbitráž, wait states, paměťová zeď a cache
4.6 Dynamická alokace
4.7 Závěr — základ pro datový tok
4.8 Kontrolní otázky

4.1 Paměťová mapa: jeden adresový prostor pro všechno

Architektury Cortex-M stojí na jednom zásadním rozhodnutí: **veškerý hardware — paměti i periferie — leží v jediném 32bitovém adresovém prostoru** o velikosti 4 GB. Procesor nemá zvláštní instrukce pro čtení z periferie a jiné pro čtení z paměti; používá tytéž `LDR`/`STR` a jen míří na jinou adresu. Rozsvícení LED je zápis na adresu, čtení tlačítka je čtení z adresy, přečtení vzorku z ADC je čtení z adresy. Tomuto principu se říká **memory-mapped I/O** a vrátíme se k němu v oddílu 4.3.

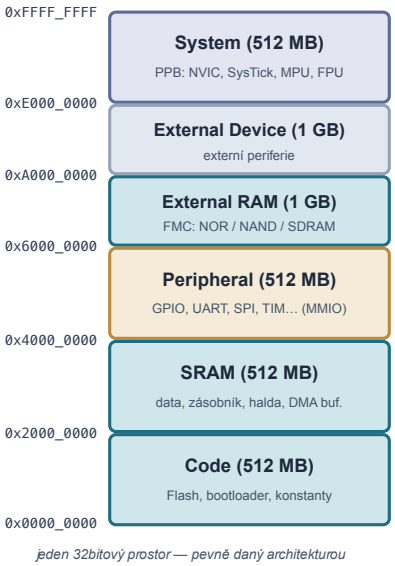
Memory-mapped I/O
Periferie jsou dostupné jako adresy v paměťovém prostoru; přistupuje se k nim týmiž instrukcemi load/store jako k paměti.

Druhé zásadní rozhodnutí je, že **rozložení tohoto prostoru je pevně dané architekturou** — ne výrobcem čipu. Každý Cortex-M má Flash u nuly, SRAM od `0x2000_0000`, periferie od `0x4000_0000` a systémové bloky (NVIC, SysTick, MPU) od `0xE000_0000`. Díky tomu je kód **přenositelný** mezi čipy a výrobci a překladač i ladicí nástroje vědí, kde co hledat. Výrobce (STMicroelectronics u STM32) pak do vyhrazených oken doplní konkrétní velikosti a periferie.

Standardní mapa ARMv7-M

Architektura dělí 4 GB na osm oblastí po 512 MB; pro nás jsou důležité čtyři. **Code** (od `0x0000_0000`) drží program a konstanty — typicky Flash. **SRAM** (od `0x2000_0000`) drží data, zásobník a haldu. **Peripheral** (od `0x4000_0000`) je okno memory-mapped I/O — GPIO, UART, SPI, časovače. **System** (od `0xE000_0000`) obsahuje řídicí bloky jádra na **PPB** (Private Peripheral Bus). Mezi nimi leží okna pro **External RAM** (přes paměťový řadič) a externí zařízení.

PPB
Private Peripheral Bus: vyhrazená sběrnice pro řídicí bloky jádra (NVIC, SysTick, MPU, FPU, ladění) v oblasti System.



Obrázek 4.1. Standardní paměťová mapa ARMv7-M: jeden 32bitový prostor, do něhož je namapováno všechno — Flash (Code), SRAM, periferie i systémové bloky.

Rozdělení není svévolné. Adresy rostou od **kódu** (čte se, nepřepisuje) přes **data** (čte i píše) k **periferiím** (každý přístup má vedlejší účinek) a **systému**. Vysoký bit adresy tak hrubě určuje *typ* paměti a tím i to, jak se s ní smí zacházet — což využije jak překladač, tak jednotka ochrany paměti (MPU).

Konkrétní mapa STM32F4

Architektura vymezí okna; výrobce je naplní. Na STM32F407 vypadá obsazení takto (tab. 4.1):

Tabulka 4.1. Paměťová mapa STM32F407

oblast	rozsah	obsah
Flash	`0x0800_0000`	512 KB programu a konstant
System memory	`0x1FFF_0000`	tovární bootloader (DFU)
CCM RAM	`0x1000_0000`	64 KB rychlé RAM **jen pro jádro** (DMA k ní nemá přístup)
SRAM	`0x2000_0000`	128 KB dat / zásobníku / haldy
APB1	`0x4000_0000`	pomalé periferie @ 42 MHz (TIM2–7, I2C, SPI2/3, USART2–5)
APB2	`0x4001_0000`	rychlejší periferie @ 84 MHz (TIM1, USART1, ADC, SPI1)
AHB1	`0x4002_0000`	GPIO, **DMA1/2**, RCC @ 168 MHz
FMC	`0x6000_0000`	externí paměti (NOR/NAND/SDRAM)
PPB	`0xE000_0000`	NVIC, SysTick, MPU, FPU, ladění

Dvě věci si zaslouží pozornost už teď, protože se vrátí v přednášce 5. Za prvé **CCM RAM**: je rychlá, ale **těsně svázaná s jádrem** — DMA řadič k ní **nemá přístup**, takže DMA buffer v ní

nikdy nenaplní. Za druhé **rozdělení periferií podle sběrnic** (APB1/APB2/AHB1): periferie na pomalejší APB jsou „dál" a přístup k nim stojí víc cyklů než k AHB. Obojí jsou důsledky toho, jak je čip fyzicky propojen — a tomu je věnována oddíl 4.4.

Oblasti Code a SRAM: kód, data, zásobník a halda

Oblast Code drží to, co se nemění za běhu: **program a konstanty** ve Flash (`0x0800_0000`), tovární **bootloader** v system memory (umožní naprogramovat čip přes USB/UART bez programátoru) a u některých čipů rychlou instrukční RAM (ITCM). Oblast je **spustitelná** (bit *execute-never* je nula), pro Flash **jen pro čtení** a — má-li čip cache — **cacheovatelná**. Jádro z ní načítá instrukce po vyhrazené cestě (I-Code bus), takže fetch instrukcí může běžet **souběžně** s přístupem k datům v SRAM po jiné cestě; to je první náznak paralelismu, který doplní bus matrix.

Oblast SRAM: data, zásobník, halda

SRAM je pracovní paměť: **inicializované globální proměnné** (`.data`), **nulované globální proměnné** (`.bss`), **halda** (rostoucí nahoru) a **zásobník** (rostoucí dolů od horního konce). Je čtecí i zápisová, typicky **nespustitelná** a běží na plné rychlosti jádra (*zero wait state*) — proto časově kritický kód někdy kopírujeme do RAM, aby nedoplácel na pomalost Flash. Sem také patří **DMA buffery** (přednáška 5).

Rozvržení SRAM přímo navazuje na obraz programu z přednášky 3. Linker definuje hraniční symboly (`_sdata`, `_edata`, `_sbss`, `_ebss`, `_estack`) a **startup kód** (Reset Handler) je při zapnutí použije: zkopíruje `.data` z Flash do RAM (ve Flash leží jejich počáteční hodnoty, protože RAM se po resetu nedefinuje), vynuluje `.bss`, inicializuje systém a skočí do `main()`.

```
void Reset_Handler(void) {
    uint32_t *src = &_sidata, *dst = &_sdata;   // .data: Flash -> RAM
    while (dst < &_edata) *dst++ = *src++;
    for (dst = &_sbss; dst < &_ebss; ) *dst++ = 0;   // .bss: vynulovat
    SystemInit();                                     // hodiny, FPU…
    main();
}
```

Proč to rozdělení? **Flash je jen pro čtení**, takže proměnná `int x = 42;` musí mít své „42" uloženo ve Flash (load address), ale za běhu žít v RAM (run address), kam ji startup zkopíruje. Toto rozdvojení adresy je přesně to, co řeší linker skript z přednášky 3.

Přemístění vektorové tabulky (VTOR)

Po resetu jádro čte tabulku vektorů přerušení od adresy nula (alias Flash). Jakmile ale na čipu běží **bootloader**, který teprve spustí aplikaci uloženou jinde, musí aplikace uplatnit vlastní tabulku vektorů. K tomu slouží registr **VTOR** (Vector Table Offset Register):

VTOR
Vector Table Offset Register: říká jádru, od které adresy číst tabulku vektorů přerušení; umožní bootloaderu předat řízení aplikaci s vlastní tabulkou.

```
#define SCB_VTOR (*(volatile uint32_t*)0xE000ED08)
SCB_VTOR = 0x08008000 & 0xFFFFFE00;   // aplikace za bootloaderem; zarovnání na 512 B
```

Od zápisu NVIC načítá vektory z nové adresy. Bez tohoto přesměrování by NVIC četl vektory ze staré adresy — tabulky bootloaderu — a přerušení aplikace by končila ve špatných obslužných rutinách. VTOR je drobnost, ale je to základ každého aktualizovatelného zařízení (bootloader → aplikace) a vrací se i v zabezpečených systémech.

Externí paměť: FMC

Vnitřní SRAM má kapacitu v desítkách až stovkách kilobajtů; když to nestačí (grafický buffer, velký záznam dat), připojí se **externí paměť** přes **FMC** (Flexible Memory Controller). Ten namapuje NOR/PSRAM, NAND Flash nebo SDRAM do oken od `0x6000_0000` a obstará jejich časování (u SDRAM osvěžování, CAS latency, burst). Z pohledu programu je pak externí SDRAM **jen další adresa** — opět síla memory-mapped přístupu — jen s vyšší latencí a jinou propustností než vnitřní SRAM.

FMC
Flexible Memory Controller: řadič, který mapuje externí paměti (NOR, NAND, SDRAM) do adresového prostoru a obsluhuje jejich časování.

Oblast System: jádro samo

Nejvyšší oblast (`0xE000_0000`) hostí **řídicí bloky jádra** na PPB: **NVIC** (řadič přerušení z přednášky 2), SysTick (systémový časovač), **MPU** (jednotka ochrany paměti), FPU a ladicí komponenty (ITM, DWT, FPB). Že je i samo jádro řízeno přes paměťově mapované registry, je elegantní důsledek principu „vše je adresa": konfigurace přerušení nebo MPU je zápis na pevnou adresu. MPU navíc dovolí jednotlivým oblastem **přiřadit typ paměti** (cacheovatelná / necacheovatelná / device) a práva — což využijeme proti nekoherenci cache v přednášce 5.

NVIC
Nested Vectored Interrupt Controller: řadič přerušení integrovaný v jádře Cortex-M (z přednášky 2).

MPU
Memory Protection Unit: jednotka, která vynucuje přístupová práva a typ paměti pro definované regiony — ochrana a řízení cache.

> **Shrnutí oddílu 4.1.** Cortex-M má jediný 32bitový adresový prostor, do něhož je namapováno vše (paměti i periferie) a jehož **rozložení je pevně dané architekturou** (Code/SRAM/Peripheral/System). Výrobce okna naplní konkrétními pamětmi a periferiemi. SRAM nese data, zásobník a haldu (startup kopíruje `.data`, nuluje `.bss`); Flash drží program; periferie jsou adresy; FMC připojí externí paměť. Dva detaily míří do přednášky 5: **CCM RAM je pro DMA nedostupná** a **periferie na APB jsou „dál".**

4.2 Linker a umístění v paměti

Kompilátor a assembler z přednášky 3 vyrobí objektové soubory, ale **nevědí, kam do paměti** kód a data patří — generují je s relativními či symbolickými odkazy. Poslední článek překladu, **linker**, objektové soubory a knihovny spojí, vyřeší odkazy mezi moduly, podle **linker scriptu** rozmístí sekce do paměťové mapy z oddílu 4.1, dosadí konečné adresy (relokace) a vytvoří výstupní formát (ELF, BIN, HEX). Ve vestavném kontextu musí navíc respektovat hardwarové adresy periferií a tabulku vektorů — správné rozmístění rozhoduje o spolehlivosti i velikosti obrazu a umožňuje bootloadery i vlastní paměťové regiony.

Linker
Nástroj, který spojí objektové soubory a knihovny, vyřeší odkazy mezi moduly, podle linker scriptu rozmístí sekce do paměti a dosadí konečné adresy.

Linker scriptu
Textový předpis rozmístění sekcí do paměťových oblastí (Flash, RAM) a definice hraničních symbolů, které využije startup kód.

Sekce a linker script

Program se dělí do standardních sekcí podle obsahu a umístění za běhu (tab. 4.2):

Tabulka 4.2. Standardní sekce programu a jejich umístění

Sekce	Obsah	Umístění za běhu
`.text`	strojový kód, konstanty `const`	Flash
`.rodata`	řetězcové literály, tabulky	Flash
`.data`	inicializované globální proměnné	SRAM (kopie z Flash)
`.bss`	neinicializované globální proměnné	SRAM (vynulováno)
`.stack` / `.heap`	zásobník a halda	SRAM

Zásadní je `.data`. Inicializované globální proměnné musí mít svou počáteční hodnotu uloženou v **trvalé** paměti (Flash), ale za běhu k nim program přistupuje v **RAM**. Hodnoty proto leží v obrazu ve Flash a **startup kód je při zapnutí zkopíruje do SRAM** (sekci `.bss` tentýž kód vynuluje) — přesně ten Reset Handler, který jsme rozebrali v části 4.1. Tato kopie je důvod, proč má `.data` odlišnou „load adresu" (ve Flash) a „run adresu" (v RAM).

Linker script

Linker script popisuje paměťové oblasti čipu a přiřazuje do nich sekce:

```
MEMORY {
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 512K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
SECTIONS {
  .text : {
    KEEP(*(.isr_vector))   /* tabulka vektorů na začátek */
    *(.text*)              /* kód */
    *(.rodata*)            /* konstanty */
  } > FLASH
  .data : {
    _sdata = .; *(.data*) _edata = .;
  } > RAM AT> FLASH        /* běží v RAM, load z FLASH */
  .bss : {
    _sbss = .; *(.bss*) _ebss = .;
  } > RAM
}
```

Zápis `> RAM AT> FLASH` říká přesně předchozí: proměnné běží v RAM, ale jejich inicializační obraz je uložen ve Flash. Symboly `_sdata`, `_edata`, `_sbss`, `_ebss` (hranice sekcí) používá startup kód ke kopii a nulování. Na začátku `.text` leží `.isr_vector` — tabulka vektorů přerušení z přednášky 2, kterou jádro po resetu čeká na adrese `0x08000000`; `KEEP` brání linkeru ji odstranit jako „nepoužitou".

Vlastní sekce a automatická registrace

Linker dokáže sesbírat rozeseté položky do souvislého pole — což je základ elegantního vzoru **automatické registrace** ovladačů. Definuje-li linker script sekci `.drivers` s vyznačenými hranicemi:

```
.drivers : {
    _drivers_start = .;
    KEEP(*(.drivers))
    _drivers_end = .;
} > FLASH
```

pak makro, které každý ovladač umístí do této sekce (vzor X-maker z přednášky 3), umožní runtime projít je jako polem, aniž by existoval centrální seznam:

```
extern const struct driver_entry _drivers_start[], _drivers_end[];

void drivers_init_all(void) {
    for (const struct driver_entry *d = _drivers_start; d < _drivers_end; d++)
        d->init();
}
```

Přidání nového ovladače je tak otázkou jediného záznamu v jeho souboru — bez zásahu do jádra systému. Tento vzor (linker-set / init array) používají Linux, Zephyr i mnoho RTOS.

libc na bare-metal a syscaly

`libc` je implementace základních funkcí jazyka C (`memcpy`, `printf`, `malloc`, `strlen`). Na PC je to `glibc`, která je pro mikrokontroléry **nepoužitelná** — potřebuje operační systém. Pro ARM bez OS existuje několik minimalistických variant:

Varianta	Velikost	Vlastnosti
newlib	100–200 kB	úplná libc (stdio, malloc), vyžaduje syscaly
newlib-nano	~30 kB	odlehčená, `printf` bez float, výchozí pro MCU
picolibc	velmi malá	moderní (newlib + avr-libc), aktivní vývoj
microlib	desítky kB	ARM/Keil, minimální, bez plné kompatibility

Společné mají, že nevyžadují souborový systém ani OS, umožňují vlastní implementaci systémových volání a jsou kompatibilní s `arm-none-eabi-gcc`.

Syscaly: jak se libc propojuje s hardwarem

Funkce libc, které by na PC potřebovaly operační systém, očekávají **systémová volání** — například `_write` (výstup), `_sbrk` (posun haldy) nebo `_read` (vstup). Na bare-metal žádné jádro neexistuje, takže je třeba je dodat. Nejjednodušší cestou je přepínač `-lnosys`, který přilinkuje knihovnu s **prázdnými** **stuby** vracejícími chybové kódy:

Stuby
Záslepka: prázdná náhrada syscallu vracející chybu, aby linker prošel; teprve vlastní implementace ji napojí na hardware.

```
arm-none-eabi-gcc main.c -o main.elf -lc -lnosys                    # newlib
arm-none-eabi-gcc main.c -o main.elf -lc -lnosys --specs=nano.specs  # newlib-nano
```

Bez stubů linker neuspěje — `printf` totiž přes `_write` a `_sbrk` zavádí nedefinované symboly (`undefined reference to '_write'`). S `-lnosys` obraz vznikne, ale `printf` **nikam nezapíše**, dokud `_write` nepřesměrujeme na periferii (typicky UART):

```c
int _write(int file, char *ptr, int len) {
    for (int i = 0; i < len; i++)
        uart_send_char(ptr[i]);   // napojení printf na sériový výstup
    return len;
}
```

Funkce `_sbrk` posouvá hranici haldy, kterou ohraničuje linker script (symbol `_end`); je to spojnice mezi linkerem a dynamickou alokací, kterou rozebereme v oddílu 4.6.

> **Shrnutí oddílu 4.2.** Linker spojí objektové soubory a podle **linker scriptu** rozmístí sekce do Flash a RAM, dosadí adresy a vytvoří obraz. Klíčový je rozdíl load/run adresy u `.data` (`> RAM AT> FLASH`), který obsluhuje **startup kód** (část 4.1). Vlastní sekce umožní **automatickou registraci** ovladačů. Na bare-metal se připojuje minimalistická **libc** (newlib-nano), jejíž **syscaly** (`_write`, `_sbrk`) si programátor napojí na hardware — tím se `printf` propojí s UARTem a `malloc` s haldou.

4.3 Memory-mapped I/O: periferie jako adresy

Z principu jediného adresového prostoru plyne, že se periferie ovládají jako adresy. Na rozdíl od pasivní paměti mají ale registry periferií vedlejší účinky a záleží u nich na pořadí přístupů. Tato část ukáže, co z toho plyne pro `volatile`, pro vyloučení periferií z cache a pro atomický zápis jednotlivých bitů.

Registr je adresa

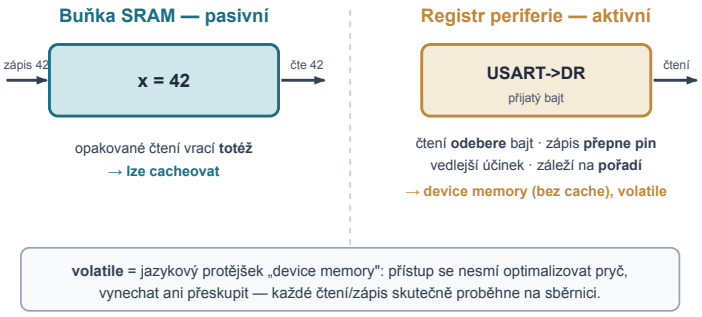
Z principu „vše je adresa” plyne, že **ovládání periferie = čtení a zápis na pevné adresy** jejích registrů. GPIO výstup je zápis do `GPIOx->ODR`, stav ADC je čtení `ADC->SR`, vyslání bajtu je zápis do `USART->DR`. Žádné zvláštní I/O instrukce (jako `IN`/`OUT` na x86) tu nejsou; veškerá komunikace s periferiemi probíhá přes `LDR`/`STR`. To zjednodušuje jádro i překladač — ovladač je „jen” práce s ukazateli na struktury registrů (jak jsme viděli u ovladačů s API v přednášce 3).

Dvě úskalí: `volatile` a atomicita

Paměť a periferie se ale chovají jinak, a to je jádro této části. Buňka SRAM je **pasivní**: co do ní zapíšeme, to v ní zůstane, a opakované čtení vrátí totéž. Registr periferie je **aktivní**: čtení `USART->DR` *odebere* přijatý bajt (a uvolní místo pro další), zápis do `GPIO->ODR` *přepne* fyzický pin. Každý přístup má **vedlejší účinek** a **na pořadí přístupů záleží**. Proto se periferní okno mapuje jako **device memory** — **bez cache** a se **striktním pořadím**: každý přístup musí skutečně proběhnout na sběrnici, nesmí se vynechat, sloučit ani přeskupit.

Device memory
Typ paměti bez cache a se striktním pořadím přístupů; každé čtení/zápis musí skutečně proběhnout na sběrnici (periferie mají vedlejší účinky).

Na úrovni jazyka C tuto „nepasivnost” vyjadřuje `volatile` (z přednášky 3): řekne překladači, že přístup k proměnné **nesmí optimalizovat pryč** ani přesunout, protože hodnota se mění „zvenčí” a samotný přístup něco působí. Bez `volatile` by překladač směl dvojí zápis do `GPIO->ODR` sloučit do jednoho nebo čtení stavového registru ve smyčce nahradit jediným přečtením — a ovladač by selhal. `volatile` je tedy přesně ten jazykový protějšek hardwarové vlastnosti „device memory”.



Obrázek 4.2. SRAM versus periferní registr: buňka SRAM je pasivní (lze cacheovat), registr periferie má vedlejší účinky a striktní pořadí (device memory, volatile).

Read-modify-write a atomické set/reset

Druhá past memory-mapped I/O je **čtení-úprava-zápis**. Zápis jednoho bitu typu `GPIOA->ODR |= (1 << 5);` se přeloží na tři kroky: přečti celý registr, nastav bit, zapiš celý registr. Když mezi čtení a zápis vstoupí přerušení, které zapíše **jiný** bit téhož registru, o jeho změnu přijdeme — to je **read-modify-write hazard**. Periferie proto často nabízejí **atomické set/reset** registry, kde *zápis* jedničky na pozici bit nastaví (a nuly nechá beze změny), takže není co číst:

Read-modify-write hazard
Souběh při neatomické sekvenci čtení-úprava-zápis registru: zásah jiného kontextu mezi čtení a zápis se ztratí.

```
GPIOA->ODR |= (1 << 5);   // NEbezpečné: read-modify-write, lze přerušit
GPIOA->BSRR = (1 << 5);   // bezpečné: čistý zápis, set bitu 5 řeší hardware
```

Tatáž logika (atomická operace místo neatomické sekvence) se vrátí u sdílených zdrojů v RTOS. Memory-mapped I/O je pohodlné, ale má svou cenu: jeho pravidla — `volatile`, pořadí, atomicita — jsou součástí korektního ovladače stejně jako logika samotná.

Shrnutí oddílu 4.3. Periferie se ovládají jako adresy (load/store), bez zvláštních I/O instrukcí. Na rozdíl od pasivní SRAM mají registry **vedlejší účinky a záleží na pořadí**, proto se mapují jako **device memory** (bez cache, striktní pořadí) a v C se k nim přistupuje přes `volatile`. Pozor na **read-modify-write hazard** — pro jednotlivé bity použijte atomické set/reset registry (`BSRR`) místo `|=`.

4.4 AMBA a bus matrix: jak je čip propojen

Paměti, jádro, DMA a periferie musí být uvnitř čipu nějak propojeny. Rodina sběrnic AMBA a křížový přepínač bus matrix to řeší tak, aby rychlé i pomalé bloky dostaly, co potřebují, a mohly pracovat souběžně. Tato část projde protokoly AHB, APB a AXI a mechanismus, který rozhoduje o skutečném souběhu jádra a DMA.

Proč víc sběrnicových protokolů

Uvnitř čipu propojuje jádro, paměti, DMA a periferie rodina otevřených standardů ARM zvaná **AMBA** (Advanced Microcontroller Bus Architecture). A hned první otázka je v duchu celé přednášky o trade-offech: **proč jich je víc?** Protože paměť a rychlá periferie potřebují propustnost, kdežto pomalá periferie potřebuje hlavně jednoduchost a nízkou spotřebu — a jeden protokol nemůže být optimální v obojím. AMBA proto nabízí tři:

AMBA
Advanced Microcontroller Bus Architecture: rodina otevřených sběrnicových protokolů ARM (AHB, APB, AXI) pro propojení bloků uvnitř SoC.

- **AHB** (Advanced High-performance Bus) — výkonná, zřetězená sběrnice pro paměť, DMA a rychlé bloky;
- **APB** (Advanced Peripheral Bus) — jednoduchá, nezřetězená, nízkopříkonová pro pomalé periferie;
- **AXI** (Advanced eXtensible Interface) — vícekanálová sběrnice s vysokou propustností pro výkonná jádra (Cortex-M7).

Cortex-M4 typicky používá **AHB-Lite** (zjednodušené AHB pro jednoho mastera na vrstvu) a APB, propojené přes most. Toto rozdělení je přesně to, co jsme v mapě STM32F4 viděli jako APB1/APB2 (pomalé) versus AHB1/AHB2 (rychlé).

Bus matrix: víc masterů, víc slave

Páteří propojení je **bus matrix** — křížový přepínač (crossbar), který spojuje **mastery** (jádro přes I-Code a D-Code rozhraní, jeden či dva DMA řadiče, ethernet, grafika) se **slave** (Flash, banky SRAM, mosty na periferie). Jeho klíčová schopnost: dokud různí masteři míří na **různé** slave, obsluhuje je **souběžně**. Jádro může načítat instrukce z Flash, zároveň číst data z jedné SRAM banky a DMA přitom plnit buffer v **jiné** bance — všechno v jednom taktu, bez vzájemného čekání.

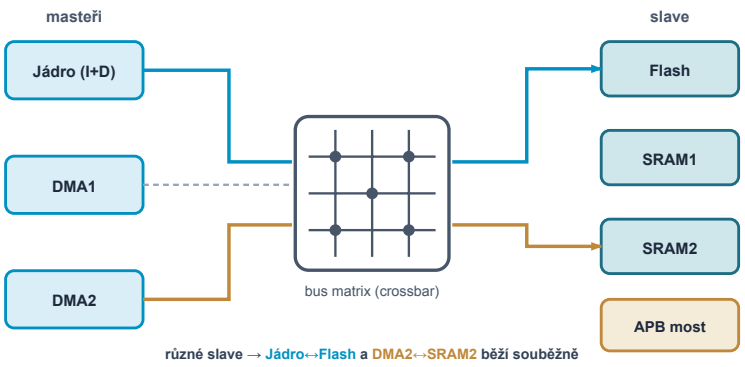
Bus matrix
Křížový přepínač (crossbar) propojující více sběrnicových masterů s více slave; nepřekrývající se dvojice master-slave běží souběžně.

Tato vlastnost je důvod, proč v přednášce 5 zdůrazňujeme **rozvržení bufferů do bank**: souběh DMA a jádra je skutečný jen tehdy, když se nepotkají na témže slave. Bus matrix tedy není detail — je to mechanismus, který rozhoduje, jestli DMA poběží opravdu „na pozadí", nebo bude odebírat jádru takty.

AHB, APB a most mezi nimi

AHB dosahuje vysoké propustnosti dvěma prostředky. Za prvé je **zřetězená**: přenos má **adresní** a **datovou** fázi a ty se překrývají — zatímco po sběrnici tečou data prvního přenosu, na adresní vodiče už master klade adresu druhého. Sběrnice tak může předávat slovo prakticky **každý takt**. Za druhé umí **burst** — souvislý blok 4/8/16 slov, kde se adresy samy odvozují (signál `HBURST`), takže se ušetří opakované adresní fáze. Typ každého přenosu nese `HTRANS` (IDLE / BUSY / NONSEQ / první nebo SEQ / následující), velikost `HSIZE` (bajt/půlslovo/slovo),

Burst
Souvislý blok přenosů na jednu adresní fázi (4, 8, 16 slov); efektivnější než jednotlivé přístupy díky předvídatelnosti adres.



Obrázek 4.3. Bus matrix (crossbar): masteři vlevo (jádro I/D, DMA), slave vpravo (Flash, banky SRAM, mosty); nepřekrývající se dvojice běží paralelně, kolize řeší arbitr.

připravenost slave `HREADY`. Burst a zřetězení jsou přesně ty vlastnosti, které DMA využívá k efektivnímu plnění bufferů (FIFO/burst z přednášky 5).

APB: jednoduchá a pomalá

Most na druhé straně spektra je APB. Je **nezřetězená** (adresa i data v jednom taktu), **bez burstů**, s minimem logiky a tím i nízkou spotřebou — a obvykle běží na **zlomku frekvence** AHB (na STM32F4 je APB1 na 42 MHz, APB2 na 84 MHz proti 168 MHz AHB). Přenos má dvě fáze: *setup* (vybere se slave signálem `PSEL`, ustálí se adresa a data) a *access* (`PENABLE`, čeká se na `PREADY`). Pro pomalé periferie — UART, I2C, většina časovačů — je to ideální: nepotřebují propustnost, potřebují levné a úsporné připojení.

Most AHB→APB: cena za „dál"

Jádro i DMA jsou na AHB; aby sáhly na periferii na APB, musí přes **AHB-APB most**, který přeloží protokol a překlene rozdíl frekvencí. To stojí **několik taktů latence** navíc a **burst se rozpadne** na jednotlivé přenosy. Odtud plyne to, co jsme v oddílu 4.1 nazvali „periferie na APB jsou dál": přístup k registru na APB je dráž než k AHB bloku, a **DMA k APB periferiím je pomalejší** než k AHB. Při návrhu datového toku se to projeví — proto rychlé proudy (vysokorychlostní SPI, paměť) sedí na AHB, kdežto pomalé (UART, I2C) klidně na APB.

AHB-APB most
Převodník mezi rychlou AHB a pomalou APB; překládá protokol a synchronizuje frekvence za cenu několika taktů latence.

AXI: vícekanálová sběrnice Cortexu-M7

Výkonná jádra (Cortex-M7) potřebují ještě víc než AHB, a proto sahají po **AXI**. Jeho hlavní myšlenka je **rozdělit přenos na nezávislé kanály**: zvlášť adresa zápisu (AW), data zápisu (W), odpověď (B), adresa čtení (AR) a data čtení (R). Díky tomu může běžet **víc transakcí současně** a dokončovat se **mimo pořadí** (out-of-order), což skryje latenci pomalé paměti a zvýší propustnost. AXI je sběrnicovým protějškem out-of-order zpracování, které u jader potkáme v přednášce 7 (architektury). Pro nás je podstatné, že **vyšší výkon = víc paralelismu na sběrnici** — a víc paralelismu znamená i víc příležitostí k nekoherenci cache (oddíl 4.5).

> **Shrnutí oddílu 4.4.** Uvnitř čipu propojuje bloky rodina AMBA: **AHB** (zřetězená, burstová, rychlá), **APB** (jednoduchá, pomalá, úsporná) a **AXI** (vícekanálová, out-of-order, M7). Páteří je **bus matrix** — crossbar, který nepřekrývající se dvojice master–slave obsluhuje **souběžně** (odtud rozvržení bufferů do bank). Periferie na APB jsou za **mostem**, který přidává latenci a rozbíjí burst — proto jsou „dál" a DMA k nim je pomalejší.

4.5 Arbitráž, wait states, paměťová zeď a cache

Sdílené sběrnice a různě rychlé paměti vedou ke konfliktům a čekání. Tato část vysvětlí, jak arbitr rozhodne konflikt o tentýž slave, proč pomalá Flash vynutí wait states a jak na obecnější **paměťovou zeď** odpovídá cache — se zádrhelem koherence vůči DMA, který míří do další přednášky.

Arbitráž: kdo vyhraje konflikt

Když dva masteři přece jen zacílí na **týž** slave, musí **arbitr** rozhodnout, kdo projde a kdo počká (**bus stall**). Používají se tři strategie. **Fixní priorita**: každý master má pevné pořadí, vyšší vždy vyhraje — jednoduché, ale nižší priorita může „hladovět". **Round-robin**: masteři se střídají spravedlivě — nikdo nehladoví, ale latence je méně předvídatelná. **Vážený round-robin**: kombinace, kde vyšší priorita dostane víc slotů. STM32 používá fixní priority s tím, že prioritu kanálu DMA lze nastavit (`PL` v `DMA_SxCR`). Praktický důsledek: když dva DMA proudy přistupují současně k SRAM, přednost dostane ten s vyšší prioritou — a návrhář tím řídí, který datový tok má přednost (např. zvuk před logováním).

Arbitr
Logika bus matrix, která při konfliktu o tentýž slave rozhodne, který master dostane přístup; druhý čeká (bus stall).

Wait states a paměťová zeď

Ne každá paměť stíhá rychlost jádra. **Flash** je výrazně pomalejší než jádro na plné frekvenci (řádově 25–30 MHz proti 168 MHz), takže při čtení musí jádro **čekat** — vloží se **wait states**. Na STM32F4 při 168 MHz to je **pět wait states** na přístup do Flash:

```
FLASH->ACR = FLASH_ACR_LATENCY_5WS    // 5 wait states @ 168 MHz
           | FLASH_ACR_ICEN           // I-cache
           | FLASH_ACR_DCEN           // D-cache (Flash)
           | FLASH_ACR_PRFTEN;        // prefetch
```

Wait states
Vložené čekací takty, když paměť nestíhá rychlost jádra; Flash na 168 MHz potřebuje na STM32F4 pět wait states.

Bez protiopatření by jádro při každém fetchi instrukce čekalo pět taktů — propustnost by spadla na šestinu. Proto i „malé" Cortexy mají u Flash **prefetch** (načítá instrukce dopředu) a malou **instrukční/datovou cache** přímo v paměťovém rozhraní Flash. **SRAM** naproti tomu běží s **nulovým wait state** (stejná rychlost jako jádro) — proto časově nejcitlivější kód někdy přesouváme do RAM.

Paměťová zeď

Wait states jsou jen lokální projev obecnějšího jevu. Výkon jader rostl desítky let mnohem rychleji než rychlost (latence) hlavní paměti, takže se mezi nimi otevřela propast, které se říká **paměťová zeď** (memory wall). Bez protiopatření by procesor většinu času **čekal na data**, místo aby počítal. Tato propast je důvod, proč moderní paměťová hierarchie vůbec existuje — a proč i drobný mikrokontrolér řeší totéž v malém (Flash vs. jádro).

Paměťová zeď
Rostoucí propast mezi rychlostí procesoru a latencí hlavní paměti; bez protiopatření by jádro většinu času čekalo na data.

Cache a koherence

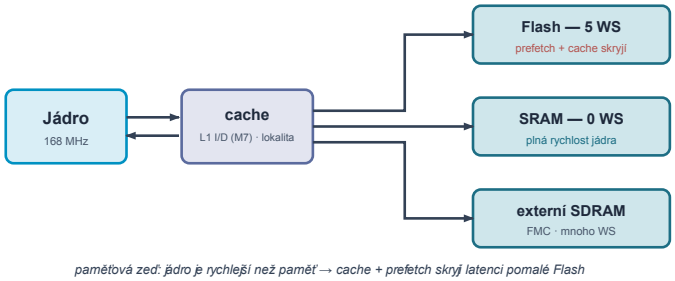
Odpovědí na paměťovou zeď je **cache** — malá rychlá paměť, která drží **kopie nedávno a okolně použitých dat**. Funguje díky **lokalitě**: programy přistupují opakovaně k týmž místům (časová lokalita) a k sousedním adresám (prostorová lokalita), takže malá cache zachytí velkou část přístupů a pomalou paměť skryje. Velká jádra mají několik úrovní (L1/L2/L3), **TLB** (cache překladu adres) a prefetcher; mikrokontrolér si vystačí s jednoduchou cache u Flash.

Cache
Malá rychlá paměť mezi jádrem a hlavní pamětí, která drží kopie nedávno a okolně použitých dat (princip lokality); zrychluje opakovaný přístup.

TLB
Translation Lookaside Buffer: cache překladu virtuálních adres na fyzické; zrychluje stránkování.

Embedded svět tu má důležitý rozdíl. **Cortex-M0/M3/M4 nemají datovou cache** — jejich determinismus stojí na tom, že přístup do SRAM trvá vždy stejně, a Flash řeší prefetch a wait states. **Cortex-M7/M55 mají oddělenou instrukční a datovou cache** (harvardsky), čímž

získají výkon, ale **platí za to koherencí**: cache drží kopie, které se mohou rozejít s hlavní pamětí, jakmile do ní zapíše jiný master. A přesně to dělá DMA.



Obrázek 4.4. Paměťová hierarchie a wait states: jádro × cache × Flash (5 WS) × SRAM (0 WS) × externí SDRAM; cache a prefetch skryjí pomalou Flash.

Koherence: kde se to láme

Dokud je jediným masterem jádro, cache funguje neviditelně správně. Jakmile ale do hlavní paměti zapíše **DMA** (které cache obchází), nastane **nekoherence**: jádro má v cache starou kopii a novější data v SRAM nezaznamená (po DMA příjmu), nebo jádro zapsalo do cache a DMA z paměti odešle zastaralá data (před DMA vysíláním). Řešení — **invalidovat** po příjmu, **vyčistit (clean)** před vysíláním, zarovnat buffery na řádek cache nebo region označit jako necacheovatelný přes MPU — je téma přednášky 5. Zde jen zasaďme příčinu: je to **přímý důsledek toho, že cache je abstrakce postavená na jednom masteru, a DMA je master druhý.**

Nekoherence
Stav, kdy cache jádra a hlavní paměť obsahují různá data téže adresy, protože jiný master (DMA) zapsal do paměti mimo cache.

> **Shrnutí oddílu 4.5.** Při konfliktu o slave rozhoduje **arbitr** (fixní/round-robin/vážený) a poražený čeká (**bus stall**). Pomalá paměť si vynutí **wait states** (Flash 5 WS @ 168 MHz; SRAM 0 WS) — projev obecné **paměťové zdi**, na niž je odpovědí **cache** (lokalita, L1/L2, TLB, prefetch). Embedded rozdíl: M0–M4 bez datové cache (determinismus), **M7/M55 s cache** platí **koherencí** vůči DMA — most do přednášky 5.

4.6 Dynamická alokace

Paměťová mapa (oddíl 4.1) vyhradila v SRAM **haldu**; linker (oddíl 4.2) jí dal hranice přes `_sbrk`. Teď se podíváme, co se na ní děje, když program za běhu žádá a vrací paměť — a proč se tomu ve vestavných systémech většinou vyhýbáme.

Halda
Oblast SRAM pro paměť alokovanou za běhu; roste od konce statických dat nahoru, proti zásobníku, který roste shora dolů.

Halda, fragmentace a proč se malloc vyhýbáme

Halda je poslední oblast SRAM — leží za statickými daty (`.data`, `.bss`) a roste **nahoru**, proti zásobníku, který roste shora dolů. **malloc** na ní najde a vyhradí souvislý volný blok požadované velikosti a vrátí na něj ukazatel; `free` blok uvolní zpět. Alokátor přitom udržuje **seznam volných bloků** a každý alokovaný blok nese skrytou hlavičku s velikostí, takže `free` ví, kolik paměti vrací. Už z tohoto popisu plynou dva problémy: alokace musí prohledat seznam (**proměnlivá doba běhu**) a opakované alokace a uvolnění různě velkých bloků haldu **rozdrobí**.

Malloc
Funkce, která na haldě najde a vyhradí souvislý volný blok požadované velikosti; free jej vrátí. Spravuje seznam volných bloků.

Fragmentace
Stav, kdy je na haldě dost volné paměti celkem, ale ne v jednom souvislém bloku; velká alokace pak selže, ač by se součtem vešla.

Fragmentace

Fragmentace je hlavní důvod nedůvěry k haldě. Po sérii alokací a uvolnění zůstanou mezi obsazenými bloky **díry**; volné paměti je dost, ale je rozdrobená do malých kousků, takže větší souvislá alokace **selže**, přestože by se součtem volného místa vešla. Na dlouho běžícím zařízení (měsíce bez restartu) fragmentace postupně narůstá a selhání přijde v nepředvídatelný okamžik — což je v embedded nepřijatelné.

Proč se malloc na MCU vyhýbáme

Shrňme námitky, které jsme už naznačili v přednášce 3 (limity prostředí). Dynamická alokace je **nedeterministická** (doba běhu `malloc` závisí na stavu haldy), **selhává za běhu** (a v prostředí bez výjimek je obtížné se zotavit), **fragmentuje** a **ztěžuje analýzu paměti** (kolik RAM program zabere, nelze určit při překladu). MISRA ji proto v kritickém kódu zakazuje. Náhradou nejsou „lepší alokátory zdarma", ale **disciplinované strategie**, kde je chování předvídatelné.

Náhrady malloc: statická, pool a arena

Nejjednodušší a nejbezpečnější je **žádná halda**: paměť se vyhradí **staticky při překladu** (globální pole, `static` buffery) nebo se vezme **ze zásobníku** jako lokální proměnná s životností bloku. Velikost je pak známá při překladu, alokace je zdarma a fragmentace neexistuje. Cenou je, že paměť drží celou dobu běhu (statická) nebo má omezenou životnost (zásobníková) — a že zásobník je malý, takže velké buffery na něj nepatří.

Pool (fixed-block) alokátor

Když opravdu potřebujeme alokovat a uvolňovat za běhu, ale objekty mají **stejnou velikost** (rámce paketů, struktury úloh), poslouží **pool alokátor**. Předem vyhradíme pole bloků pevné velikosti a vedeme seznam volných; alokace i uvolnění je pak jen **převzetí nebo vrácení** bloku v konstantním čase. Protože jsou všechny bloky stejné, **fragmentace nevzniká** a doba běhu je deterministická — přesně to, co `malloc` nezaručí.

Pool alokátor
Předpřipravené pole bloků pevné velikosti se seznamem volných; alokace i uvolnění je převzetí nebo vrácení bloku v konstantním čase, bez fragmentace.

Arena (bump) alokátor

Pro paměť, která se alokuje po fázích a uvolňuje **najednou** (zpracování jednoho snímku, jedné zprávy), se hodí **arena (bump) alokátor**. Z velkého souvislého bloku alokujeme prostým **posunem ukazatele**; jednotlivé objekty se neuvolňují, místo toho se na konci fáze **celá arena zahodí** (ukazatel se vrátí na začátek). Alokace je tak nejrychlejší možná a fragmentace opět nevzniká — za cenu hrubozrnného uvolňování.

Arena (bump) alokátor
Souvislý blok paměti, z něhož se alokuje pouhým posunem ukazatele; neuvolňuje se po jednotlivých objektech, ale celá arena se zahodí naráz (reset ukazatele).

> **Shrnutí oddílu 4.6.** Halda roste proti zásobníku; `malloc` na ní hledá volné bloky, čímž je **nedeterministický** a náchylný k **fragmentaci** (volná paměť je, ale ne souvislá). V embedded se mu proto vyhýbáme a volíme předvídatelné strategie: **statickou/zásobníkovou** alokaci (žádná halda), **pool** pro stejně velké objekty (konstantní čas, bez fragmentace) a **arenu** pro paměť uvolňovanou po fázích naráz. Pokročilejší alokátory a alokaci v RTOS rozebírají pozdější přednášky.

4.7 Závěr — základ pro datový tok

Tato přednáška vytvořila základ, na nějž přednáška 5 naváže popisem datového toku. Stojí za to spojit, co z paměti a sběrnic plyne pro datový tok. Z **paměťové mapy** (oddíl 4.1) víme, že ne každá paměť je pro DMA použitelná — banky těsně svázané s jádrem (CCM/TCM) jsou pro DMA nedostupné, takže buffer musí ležet v „obyčejné" SRAM. Z **bus matrix** (oddíl 4.4)

víme, že crossbar obsluhuje nepřekrývající se dvojice master–slave souběžně, takže DMA buffer a zásobník patří do **různých bank**, jinak si DMA a jádro navzájem odebírají takty (a arbitráž z oddílu 4.5 jen určí, kdo počká). A z **cache** (oddíl 4.5) víme, že na M7 drží jádro data v cache, kterou DMA obchází — proto bude přednáška 5 po příjmu **invalidovat** a před vysíláním **čistit** řádky bufferu.

Jinými slovy: **paměťová mapa řekne, kam buffer smí; bus matrix, jak ho umístit pro souběh; cache, jak ho udržet koherentní.** Procesor jsme v přednáškách 1–3 naučili počítat a běžet; zde jsme pochopili **prostředí**, v němž pracuje; v přednášce 5 pak obsluhu datového toku přenecháme DMA.

> **Závěr.** Mikrokontrolér je malý systém na čipu: jeden adresový prostor, do něhož linker umístí kód i data, periferie dostupné jako `volatile` adresy, bloky propojené sběrnicemi AMBA a bus matrix a paměťová hierarchie s cache a wait states. Kdo zná **kam** smí buffer, **jak** ho umístit pro souběh a **jak** ho udržet koherentní, dokáže navrhnout datový tok, který využije sběrnici naplno — a to je téma přednášky 5.

4.8 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 4.1 — Paměťová mapa: jeden adresový prostor pro všechno

1. Proč je výhodné, že paměti i periferie leží v jednom adresovém prostoru a přistupuje se k nim týmiž instrukcemi?
2. Co dělá startup kód s `.data` a `.bss` a proč musí počáteční hodnoty `.data` ležet ve Flash?
3. Proč musí aplikace spuštěná za bootloaderem přesměrovat tabulku vektorů (VTOR) a co by se stalo bez toho?
4. Proč nelze umístit DMA buffer do CCM RAM?

K oddílu 4.2 — Linker a umístění v paměti

1. Proč je při překladu potřeba linker (co dělá se sekcemi z více objektů) a jak linker script řídí jejich umístění v paměti?
2. Proč musí mít `.data` odlišnou load a run adresu a kdo kopii provede?
3. Jak linker realizuje automatickou registraci ovladačů přes vlastní sekci?
4. Jaký kompromis (velikost kódu vs. funkčnost) řeší volba varianty libc na embedded cíli?
5. Co je syscall na bare-metal a jak se `printf` napojí na UART?

K oddílu 4.3 — Memory-mapped I/O: periferie jako adresy

1. Čím se liší přístup k buňce SRAM a k registru periferie a proč to vyžaduje `volatile`?
2. Co znamená „device memory" a proč se periferie nesmí cacheovat?
3. Vysvětlete read-modify-write hazard na `GPIOA->ODR |= (1<<5)` a jak ho `BSRR` odstraní.

K oddílu 4.4 — AMBA a bus matrix: jak je čip propojen

1. Proč AMBA nabízí víc protokolů místo jednoho univerzálního?
2. Jak bus matrix umožní souběžný běh jádra a DMA a kdy souběh zmizí?
3. Co znamená, že je AHB „zřetězená", a jak to zvyšuje propustnost?
4. Proč je přístup k APB periferii dražší než k AHB a co z toho plyne pro umístění rychlých datových proudů?

K oddílu 4.5 — Arbitráž, wait states, paměťová zeď a cache

1. Porovnejte fixní prioritu a round-robin arbitráž — výhody a rizika každé.
2. Proč Flash potřebuje wait states a SRAM ne? Jak je prefetch/cache zmírní?
3. Co je paměťová zeď a jak na ni cache odpovídá (princip lokality)?
4. Proč M0–M4 obvykle nemají datovou cache a co M7 za cache „platí"?

K oddílu 4.6 — Dynamická alokace

1. Jak pracuje `malloc`/`free` a proč má proměnlivou dobu běhu?
2. Co je fragmentace a proč je v dlouho běžícím zařízení nebezpečná?
3. Proč je `malloc` v kritickém embedded kódu riziko? (Uveďte důvody z hlediska determinismu, paměti a selhání.)
4. Kdy zvolíte pool a kdy arenu a co každý z nich zaručuje oproti `malloc`?

5

Datový tok bez procesoru: DMA, převodníky a buffery

> Tato kapitola řeší, **jak v reálném čase přesouvat data mezi periferiemi a pamětí** rychle, deterministicky a úsporně — aniž by procesor jen čekal a kopíroval slovo po slově — a jak takový proud bezpečně **sdílet** mezi hardwarem, přerušením a aplikací bez zámku. Stojí na třech pilířích: **DMA**, **analogové převodníky** (DAC/ADC) a **správa bufferů a vlastnictví**.

Obsah kapitoly

5.1 Procesor jako úzké hrdlo přenosu dat
5.2 DMA: autonomní přesun dat
5.3 Analogové rozhraní: DAC a generování signálu
5.4 Analogové rozhraní: ADC
5.5 Správa bufferů a vlastnictví
5.6 Syntéza: streamovací ovladač
5.7 Kontrolní otázky

5.1 Procesor jako úzké hrdlo přenosu dat

Než sáhneme po DMA, je třeba pochopit, proč se procesor na přesouvání dat nehodí. Tato část ukáže, kolik stojí každé zkopírované slovo, proč se pevně tikající periferie a nárazově pracující software míjejí a jaké tři způsoby přenosu dat existují — a proč u trvalých proudů obstojí jen jeden.

Cena kopírování a míjející se rytmy

Procesor umí přesouvat data. Smyčka `while (n--) *dst++ = *src++;` zkopíruje pole — a na první pohled je to triviální. Problém není v tom, *že* to procesor umí, ale **kolik ho to stojí a co přitom nedělá**. Každé přesunuté slovo znamená nejméně načtení (`LDR`), uložení (`STR`) a údržbu smyčky (dekrement, podmíněný skok). Na Cortexu-M4 to jsou řádově **2–4 takty na slovo** i při dokonalém zřetězení a bez čekání na paměť. Po dobu kopírování procesor **nedělá nic jiného** — neobsluhuje regulační smyčku, nepočítá FFT, nereaguje na jinou událost.

To samo o sobě nemusí vadit u jednorázové kopie při startu. Stává se to však kritickým ve chvíli, kdy data **přicházejí nebo odcházejí nepřetržitě a pevnou rychlostí** — typicky přes periferii. Tehdy se z kopírování stává **trvalá daň** odváděná v každém okamžiku běhu, a tato daň přímo ukrajuje z výpočetního rozpočtu i z energetické bilance.

Producent a konzument, kteří se míjejí

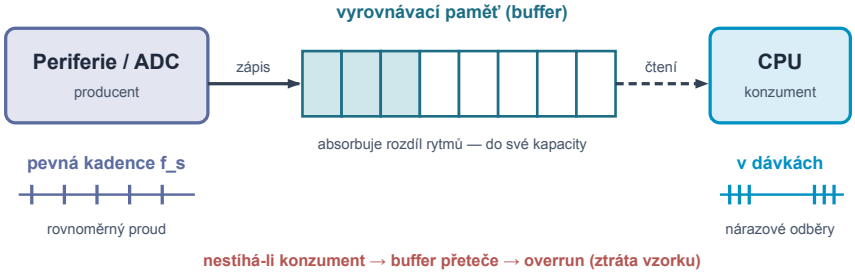
Uvažme zvukový vstup: **ADC** vzorkuje mikrofon **48 000× za sekundu**. To je producent s **neúprosně pevnou** kadencí — každých ≈20,8 μs leží v registru periferie nový vzorek a další ho po další periodě přepíše, ať se děje co se děje. Proti němu stojí **konzument** — aplikace, která vzorky zpracovává, ale ne rovnoměrně: chvíli počítá blok FFT, chvíli obsluhuje displej, chvíli čeká na sběrnici. Producent tlačí konstantní proud, konzument odebírá v **dávkách**.

ADC
Analog-Digital Converter: převodník spojité analogové veličiny na číslicový vzorek.

Tento nesoulad je jádro problému. Když konzument nestihne odebrat vzorek dřív, než ho producent přepíše, vzniká **overrun** — ztráta dat. V opačném směru (procesor plní výstupní periferii) hrozí **underrun**: periferie chce další vzorek, ale žádný není připraven, a do proudu se vloží slyšitelná mezera nebo zákmit. Mezi pevně tikajícím hardwarem a nárazově pracujícím softwarem proto **musí stát vyrovnávací paměť** — buffer — a někdo musí spolehlivě zajistit její plnění a vyprazdňování. Tím „někým" bude DMA.

Overrun
Přepsání dosud nepřečteného vzorku novým — ztráta dat, protože konzument nestíhal.

Underrun
Periferie potřebuje další vzorek, ale producent ho nedodal včas — mezera ve výstupním proudu.



Obrázek 5.1. Producent (periferie, pevná kadence) a konzument (CPU, dávky) se míjejí; vyrovnávací paměť absorbuje rozdíl rytmů, ale jen do své kapacity.

Tři způsoby, jak data dostat dovnitř

Existují tři kvalitativně odlišné způsoby, jak přesunout datový proud z periferie do paměti. Liší se tím, **kdo a jak často** musí zasáhnout procesor.

Dotazování (polling). Procesor v těsné smyčce čte stavový příznak periferie a po jeho nastavení odebere vzorek. Je to jednoduché a má nejnižší latenci na jednotlivý vzorek, ale procesor je **plně vytížen čekáním** — nedělá nic užitečného a spotřebovává energii naprázdno. Použitelné jen krátkodobě nebo na pomalé periferie.

Přerušení na vzorek (interrupt-driven). Periferie po každém vzorku vyvolá přerušení; obsluha vzorek odebere a uloží. Procesor mezi vzorky volně počítá. Háček je v **režii kontextu**: každý vstup do přerušení a návrat z něj stojí na Cortexu-M nejméně ∼12 + ∼12 taktů na uložení a obnovu rámce (viz přednáška 2), plus práci samotné obsluhy. Spočítejme: ADC s **1 Msps** by generoval **milion přerušení za sekundu**; jen jejich vstup/výstup by při ≈30 taktech režie spotřeboval **30 milionů taktů** — tedy téměř **pětinu** výkonu jádra na 168 MHz, a to ještě nic neudělal s daty. Per-sample přerušení tedy škáluje špatně: funguje pro kilohertzové kadence, ale u megahertzových proudů procesor zahltí.

DMA. Periferie nepošle žádost procesoru, ale **přímo řadiči DMA**, který vzorek odebere a uloží do paměti **bez jediné instrukce procesoru**. Procesor je vyrušen až po **celém bloku** (nebo jeho polovině) — místo milionu přerušení za sekundu jich vidí třeba sto, jedno na každý naplněný blok. Tím se režie zmenší o tři až čtyři řády a procesor je volný pro skutečnou práci.

způsob	kdo přesouvá vzorek	režie procesoru	vhodné pro
polling	procesor v aktivní smyčce	**100 %** (čeká)	pomalé / krátké přenosy
přerušení/vzorek	procesor v obsluze	režie kontextu × kadence	jednotky až stovky kHz
DMA	**řadič DMA**	jen 1 IRQ na blok	**vysoké, trvalé proudy**

Oddělit pohyb dat od jejich zpracování

Z předchozího plyne návrhová zásada, která tvoří páteř celé přednášky: **pohyb dat a jejich zpracování jsou dvě různé úlohy a patří dvěma různým strojům.** Zpracování — rozhodování, výpočet, řízení — je práce pro procesor, protože vyžaduje logiku a stav. Přesun bloku dat z pevné adresy periferie na rostoucí adresu v paměti je naopak **mechanická, předvídatelná** činnost bez rozhodování, a právě takovou činnost lze svěřit jednoduchému specializovanému automatu. Tím automatem je DMA řadič.

Tato dělba má tři důsledky, které se potáhnou celou přednáškou. Za prvé **výkon**: procesor přestane být úzkým hrdlem datového toku. Za druhé **determinismus**: DMA běží nezávisle na tom, co zrovna dělá software, takže časování přenosu je pevné a nezávislé na zatížení jádra — což je pro real-time systémy zásadní. Za třetí **nový problém vlastnictví**: jakmile data přesouvá někdo jiný než procesor, vzniká otázka, **kdo a kdy** smí na buffer sahat — a té je věnována oddíl 5.5.

> **Shrnutí oddílu 5.1.** Procesor je drahý a špatně škálující kopírovací stroj: každé přesunuté slovo ho stojí takty, které pak chybí jinde. Mezi pevně tikající periferií (producent) a nárazově pracujícím softwarem (konzument) musí stát vyrovnávací paměť. Ze tří způsobů přenosu — polling, přerušení na vzorek, DMA — jen DMA zvládá vysoké trvalé proudy s minimální režií, protože **odděluje pohyb dat od jejich zpracování**.

5.2 DMA: autonomní přesun dat

Řešením úzkého hrdla je specializovaný obvod, který přesune data bez jediné instrukce jádra. Tato část představí řadič DMA — jeho deskriptor, normální a cirkulární režim, soupeření o sběrnici na bus matrix a konkrétní podobu na STM32 — a ukáže jeho konfiguraci i hlášení dokončení a chyb.

Co je řadič DMA

DMA řadič je samostatný **master na sběrnici** — tedy obvod, který stejně jako procesor umí iniciovat čtení a zápis do paměti a periferií, ale na rozdíl od procesoru **nevykonává program**. Místo instrukcí se řídí **deskriptorem**: sadou registrů, do nichž software předem zapíše, odkud, kam, co a kolik se má přesunout. Po spuštění řadič tuto úlohu provede sám a procesoru se ozve teprve po jejím dokončení.

Řadič je rozdělen na **kanály** (u STM32 řady F4 nazývané *streamy*). Každý kanál má vlastní sadu registrů a obsluhuje **jeden logický přenos** — například „ADC1 → buffer". Kanálů je typicky 8–16, takže současně může běžet více nezávislých proudů: jeden plní buffer z ADC, druhý vyprazdňuje buffer do DAC, třetí obsluhuje SPI. Fyzicky řadič sběrnici sdílí, takže přenosy se *prokládají*, ale logicky jsou oddělené.

Periferie signalizuje požadavek **žádostní linkou** (*DMA request*): vodičem mimo datovou sběrnici oznámí řadiči, že je připraven další prvek k přesunu (u výstupu naopak že je prázdná

DMA řadič
Specializovaný sběrnicový master, který podle předem nastaveného deskriptoru přesune blok dat mezi adresami bez účasti procesoru.

Kanály
Nezávislá přenosová cesta DMA s vlastním deskriptorem; kanálů je více, takže běží paralelně logicky nezávislé přenosy.

Žádostní linkou
Hardwarový signál (DMA request) z periferie do řadiče DMA: oznamuje, že je připraven další prvek k přesunu.

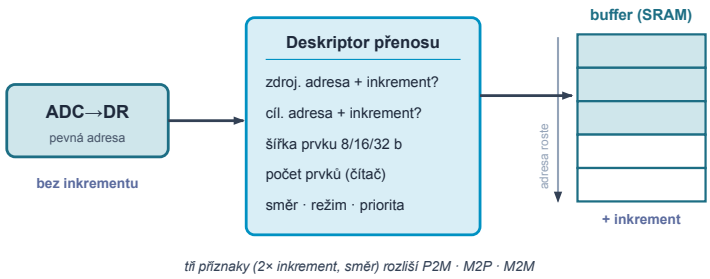
a čeká na další). Která periferie je svázána s kterým kanálem, určuje na moderních čipech **DMAMUX** — programovatelný multiplexor žádostí, který nahradil starší pevné mapování periferie↔kanál a dává návrháři volnost.

DMAMUX
Programovatelný multiplexor žádostí: libovolnou periferní DMA žádost nasměruje na libovolný kanál řadiče.

Deskriptor přenosu a režimy

Celý přenos je popsán několika málo údaji; v nich je síla i jednoduchost DMA. Deskriptor obsahuje:

- **zdrojovou adresu** a příznak, zda se má po každém prvku **inkrementovat** (paměť: ano; pevný registr periferie: ne),
- **cílovou adresu** a obdobný příznak inkrementu,
- **šířku prvku** (8/16/32 bitů) zvlášť pro zdroj a cíl — DMA umí i konverzi šířky,
- **počet prvků** k přenosu (čítač, který se dekrementuje),
- **směr** přenosu a **režim** (normální / cirkulární, viz dále),
- **prioritu** kanálu pro arbitráž.



Obrázek 5.2. Deskriptor přenosu: pevná adresa periferie (bez inkrementu) na jedné straně, rostoucí adresa v paměti (s inkrementem) na druhé; čítač prvků určuje konec.

Z kombinace „inkrementovat / neinkrementovat" plynou tři základní směry. **Periferie → paměť** (P2M): zdroj je pevný registr (např. `ADC->DR`), cíl je rostoucí adresa v bufferu — tak se streamuje vstup. **Paměť → periferie** (M2P): zrcadlově, rostoucí zdroj v paměti, pevný cíl (`DAC->DHR`) — tak se streamuje výstup. **Paměť → paměť** (M2M): obě adresy rostou — rychlá bloková kopie, kterou DMA zvládne bez procesoru. Všimněte si, že *jediný* rozdíl mezi streamem zvuku ven a kopií pole v paměti je nastavení dvou příznaků inkrementu — tatáž mašinerie obslouží oboje.

Normální a cirkulární režim

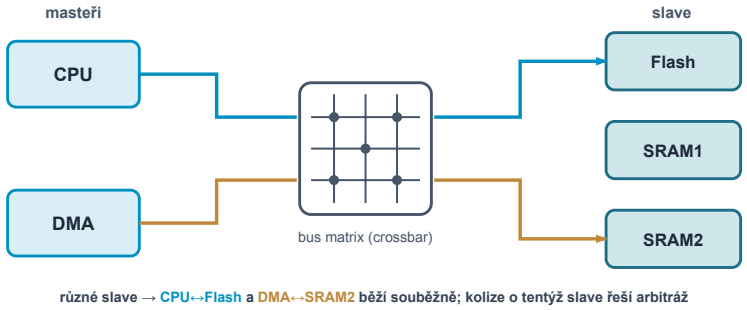
V **normálním** režimu řadič přenese zadaný počet prvků a zastaví; čítač doběhne na nulu, kanál se vypne a vyvolá přerušení. Hodí se na jednorázový blok — odeslání rámce, kopii.

V **cirkulárním** režimu se po dosažení konce **čítač automaticky znovu naplní** a adresa v paměti se vrátí na začátek bufferu. Přenos tak běží **donekonečna** bez zásahu procesoru: DMA stále dokola plní (nebo vyprazdňuje) tentýž buffer. To je přesně režim pro nepřetržité proudy — vzorkování ADC, generování signálu z tabulky — a je základem dvojitého bufferu, o němž bude řeč v oddílu 5.5. Cirkulární režim je důvod, proč může zvuk hrát hodiny bez jediného přerušení navíc oproti tomu, co si vyžádá samotné zpracování.

DMA na sběrnici: bus matrix, FIFO a burst

Ani DMA řadič nepracuje bez ceny. Aby přesunul slovo, musí — stejně jako procesor — **získat přístup na sběrnici** a provést čtení a zápis. Na čipu je přitom obvykle **více masterů** (jádro, jeden či dva DMA řadiče, někdy ethernetový nebo grafický kontrolér) a omezený počet **slave** paměťových a periferních bloků. Propojuje je **bus matrix** — křížový přepínač, který jsme potkali už v přednášce 4.

Bus matrix
Křížový přepínač (crossbar) spojující více sběrnicových masterů s více slave; umožňuje souběžné nepřekrývající se přenosy.



Obrázek 5.3. DMA i procesor jsou masteři na bus matrix; dokud míří na různé slave (DMA do SRAM2, jádro do Flash), běží souběžně. Kolize o tentýž slave řeší arbitráž.

Klíčový důsledek: dokud DMA a procesor přistupují k **různým** slave blokům — třeba jádro vykonává kód z Flash a počítá v jedné SRAM bance, zatímco DMA plní buffer v **jiné** bance — běží oba **skutečně souběžně** a navzájem se nezdrží. Jakmile však zacílí na **týž** slave, musí o něj **arbitr** rozhodnout a jeden počká. Tomuto čekání se říká **bus stall**; obvykle je odložen přístup DMA (má nižší prioritu), takže přenos se nepatrně zpozdí — ale data nezmizí. Praktický návrhový důsledek je důležitý: **DMA buffery a kód/zásobník je výhodné umístit do různých paměťových bank**, aby se masteři nepotkávali a souběh byl skutečný. Špatné rozvržení paměti dokáže zničit propustnost, kterou DMA nabízí.

Bus stall
Pozdržení sběrnicového masteru, protože arbitr přidělil přístup ke slave jinému masteru.

FIFO, burst a zarovnání

Jednotlivé přístupy na sběrnici mají režii (adresní fáze, arbitráž). Efektivnější je přenášet data **po dávkách** — *burst*. Proto má kanál DMA často malou frontu **FIFO**: nasbírá několik prvků, a pak je přenese jediným souvislým burstem, čímž lépe využije sběrnici a umožní i **konverzi šířky** (periferie dodává po bajtech, do paměti se zapisuje po 32bitových slovech). Burst i FIFO ale vyžadují **správné zarovnání** adres a velikostí; chybné nastavení vede k FIFO chybě nebo k přístupu na nezarovnanou adresu. Zarovnání se navíc znovu zásadně vrátí u cache (oddíl 5.5).

FIFO
Krátká vyrovnávací fronta uvnitř kanálu DMA; nasbírá několik prvků a přenese je na sběrnici jedním burstem.

DMA na STM32 konkrétně

Na řadě STM32F4 jsou **dva řadiče** DMA1 a DMA2, každý s **8 streamy**, a každý stream umí vybrat z několika **kanálů** (request od konkrétní periferie). Mapování periferie na stream a kanál je u této řady **pevně dané tabulkou** v referenčním manuálu — návrhář si nevybírá libovolně, ale musí najít stream, na kterém jeho periferie leží:

periferie (request)	řadič	stream	kanál
ADC1	DMA2	0 nebo 4	0
DAC1	DMA1	5	7
SPI1_TX	DMA2	3 nebo 5	3
USART2_RX	DMA1	5	4
TIM2_UP	DMA1	1 nebo 7	3

Z pevného mapování plyne reálné návrhové omezení: dva souběžné proudy **nesmí vyžadovat týž stream**, jinak je nelze provozovat zároveň, a člověk musí kombinaci streamů volit tak, aby ke konfliktu nedošlo. K tomu se přidává, že **paměť↔paměť zvládne jen DMA2**. Novější řady (L4, G4, H7) tato omezení odstranily zavedením **DMAMUX** — multiplexoru, který libovolnou žádost nasměruje na libovolný kanál, takže konflikt mapování zmizí. Rodiny se liší i dál: **H7** má kromě DMA ještě **MDMA** (rychlé přesuny velkých bloků mezi pamětmi) a **BDMA** (v doméně nízké spotřeby) a navíc **datovou cache** (oddíl 5.5); **L4/G4** mají **hardwarový oversampler** ADC. Principy této přednášky platí napříč — liší se detaily mapování a koherence.

Typický zdroj žádostí je sama periferie (ADC dokončil vzorek, SPI vyprázdnil vysílací registr), ale velmi často je to **časovač**: jeho update nebo capture/compare událost spustí DMA přenos. Tím se uzavírá oblouk z přednášky 2 — časovač, který tam pouze generoval PWM a přerušení, zde řídí časování datového toku: udává takt, na který DMA odebírá nebo dodává vzorky. Ekvidistantní vzorkování i přesné generování signálu stojí právě na této trojici **časovač → žádost → DMA**.

Dokončení, chyby a konfigurace

Aby procesor věděl, kdy s daty pracovat, hlásí DMA dvě události. **Transfer-complete** (TC) přijde po přenosu **celého** bloku — buffer je plný (nebo prázdný) a lze ho zpracovat. **Half-transfer** (HT) přijde **v polovině** bloku — to je drobnost s velkým dopadem: dovoluje zpracovat první polovinu bufferu, zatímco DMA souběžně plní druhou. Dvojice HT + TC je hardwarový základ dvojitého bufferu z oddílu 5.5.

Transfer-complete
Přerušení DMA po přenesení celého bloku: data jsou kompletní a připravená ke zpracování.

Half-transfer
Přerušení DMA v polovině bloku; umožňuje zpracovat první půlku, zatímco DMA plní druhou (dvojitý buffer).

Kromě toho řadič hlásí **chyby**: *transfer error* (přístup na neplatnou adresu, např. špatně nastavený ukazatel nebo periferie bez práva přístupu), *FIFO error* (špatné zarovnání nebo nedoběh fronty) a *direct-mode error*. Tyto stavy se nesmí ignorovat — v ladění odhalí chybný deskriptor, v provozu spustí bezpečné zotavení.

Konfigurace v praxi

Nastavení kanálu je posloupnost zápisů do registrů: adresy zdroje a cíle, počet prvků, šířky, příznaky inkrementu, směr, režim, priorita, povolení požadovaných přerušení (HT/TC/ chyby) a nakonec **povolení kanálu**. CMSIS/HAL to zabalí do struktury, ale význam polí odpovídá deskriptoru výše v této části. Pro představu kostra přenosu paměť→periferie (vysílání bufferu do DAC, cirkulárně):

```
DMA2_Stream5->PAR  = (uint32_t)&DAC->DHR12R1;   // cíl: pevný registr DAC
DMA2_Stream5->M0AR = (uint32_t)wave;            // zdroj: tabulka v SRAM
```

```
DMA2_Stream5->NDTR = WAVE_LEN;                  // počet prvků
DMA2_Stream5->CR =
      (CHAN << DMA_SxCR_CHSEL_Pos)              // výběr kanálu (request DAC)
    | DMA_SxCR_MINC                             // inkrementuj adresu v paměti
    | DMA_SxCR_DIR_0                            // směr: paměť -> periferie
    | DMA_SxCR_CIRC                             // cirkulární režim
    | (1 << DMA_SxCR_PL_Pos)                    // priorita kanálu
    | DMA_SxCR_HTIE | DMA_SxCR_TCIE             // přerušení HT a TC
    | DMA_SxCR_EN;                              // spuštění kanálu
```

Po tomto zápisu řadič na každou žádost časovače odebere jeden prvek tabulky a zapíše ho do DAC; po dosažení konce začne od začátku. Procesor mezitím nedělá **nic** — a přesto z výstupu plyne nepřetržitý, časově přesný signál.

> **Shrnutí oddílu 5.2.** DMA řadič je sběrnicový master řízený deskriptorem (zdroj/cíl + inkrementy, šířka, počet, směr, režim). Tři příznaky určují směr P2M/M2P/M2M; cirkulární režim dělá z přenosu nekonečný proud. DMA soupeří s procesorem na bus matrix, takže **rozvržení bufferů do bank** rozhoduje o skutečném souběhu; FIFO/burst zvyšují propustnost za cenu nároků na zarovnání. Události **HT a TC** ohlašují polovinu a konec bloku — hardwarový základ dvojitého bufferu.

5.3 Analogové rozhraní: DAC a generování signálu

Chceme-li z číslicového světa ven do spojitého napětí, potřebujeme převodník. Tato část vysvětlí, proč na přesný průběh nestačí PWM filtr, jak DAC tvoří napětí a jaké vady (glitch, nemonotonicita) s tím souvisejí, a spojí DAC s DMA a časovačem do generátoru signálu bez zatížení jádra.

Proč pravý převodník, a ne PWM

V přednášce 2 jsme „analogový" výstup vyrobili **PWM filtrem**: obdélník o proměnné střídě vyhladil RC článek na střední hodnotu. Pro řízení výkonu nebo pomalu se měnící úroveň to stačí. Jakmile však potřebujeme **rychle a přesně** definovaný průběh — referenci, audio, testovací signál — naráží PWM na limity: rozlišení je svázané s nosnou frekvencí, filtr zanáší zpoždění a zvlnění a o vyšších kmitočtech není řeč. Tehdy nastupuje **DAC** — obvod, který číslicový kód přímo převede na úměrné napětí.

Jak DAC tvoří napětí: žebříky, glitch a monotonicita

Vnitřně DAC sestavuje výstupní napětí z **referenčního napětí** váženého podle bitů kódu. Nejnázornější je **binárně vážený žebřík**: každý bit kódu sepne (nebo nesepne) referenci přes odpor s vahou 2^{-n} — MSB přes nejmenší odpor R (největší proud), LSB přes $8R$ — a operační zesilovač sečte proudy ve společném uzlu.

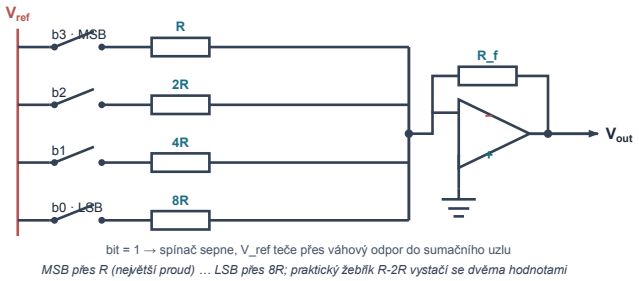
Tato topologie je intuitivní, ale **nepraktická**: rozsah odporů od R po $8R$ (u 12 bitů poměr 1 : 2048) nelze vyrobit dostatečně přesně. Proto se v praxi používá **žebřík R-2R**, který týž efekt složí jen ze **dvou hodnot** odporů (R a $2R$), nebo **odporový řetězec** (resistor string), kde napěťový dělič vytvoří všechny úrovně a multiplexor vybere tu, která odpovídá kódu. Na přesnosti a shodě odporů závisí **rozlišení**: STM32 má 12bitový DAC, tedy 4096 úrovní; jeden krok (**LSB**) při rozsahu 3,3 V činí ≈0,8 mV. Výstup DAC bývá veden přes interní operační zesilovač (output buffer), aby unesl zátěž bez poklesu napětí.

DAC
Digital-Analog Converter: převede číslicový kód na úměrné výstupní napětí.

Binárně vážený žebřík
Nejjednodušší DAC: každý bit spíná referenci přes odpor s vahou dvě na minus n do sumačního uzlu; součet proudů dá výstup.

Rozlišení
Počet bitů převodníku; n bitů dělí rozsah na dvě na n-tou úrovní. Velikost jednoho kroku je LSB = rozsah dělený počtem úrovní.

LSB
Least Significant Bit: nejmenší rozlišitelný krok převodníku, rovný rozsahu děleno počtem úrovní.

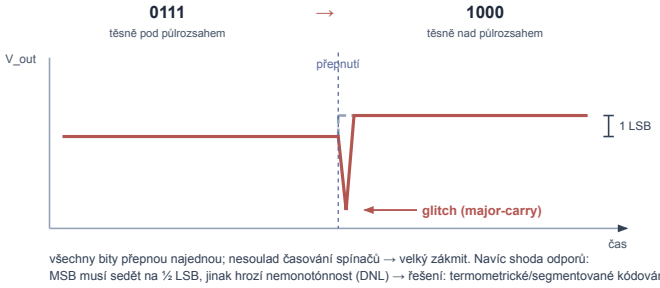


Obrázek 5.4. Binárně vážený odporový DAC: bit = 1 sepne V_ref přes váhový odpor do sumačního uzlu; op. zesilovač sečte proudy. MSB přes R, LSB přes 8R.

Problém binárního kódu: glitch a monotonicita

S binárně kódovaným DAC se pojí dvě vady, které názorně ukazují, proč „víc bitů" není bez ceny. První je **glitch**. Uvažme přechod přes půlrozsah, z kódu `0111` na `1000` — tzv. **major carry**, při němž **všechny bity přepnou najednou** (MSB se zapne, nižší tři se vypnou). Ideálně má výstup vzrůst o jediný LSB; protože se ale spínače jednotlivých bitů nepřepnou v přesně týž okamžik, výstup během přechodu krátce vyletí (k `1111`) nebo spadne (k `0000`) a vznikne **velký zákmit**, mnohonásobně větší než užitečný krok.

Glitch
Krátký velký zákmit výstupu DAC při přechodu kódu, kdy mnoho bitů přepne najednou a jejich spínače se nepřepnou v týž okamžik.



Obrázek 5.5. Major-carry glitch: ideální výstup vzroste o 1 LSB, ale při současném přepnutí všech bitů vznikne velký zákmit; navíc nepřesný MSB odpor ohrožuje monotonicitu (DNL).

Druhá vada souvisí s **monotonicitou**. Aby výstup s rostoucím kódem nikdy neklesl, musí odpor MSB sedět na ½ **LSB** vůči součtu všech nižších vah — což je u binárně vážené struktury nejtěžší právě v bodě major carry. Nepřesnost se projeví jako **DNL** (differential non-linearity); je-li DNL horší než -1 LSB, výstup u major carry **klesne, ač kód roste** — převodník je nemonotonní, což je v regulační smyčce závažná vada. Řešením je **termometrické** (unární) nebo **segmentované** kódování: horní bity se převedou na mnoho **stejných** jednotkových prvků, které se jen přidávají — žádný velký společný přechod, a tím ani velký glitch, ani riziko nemonotonicity. **Settling time** a glitch dohromady shora omezují, jak rychle smíme měnit vzorky; proto i kvalitní DAC má svou stropní vzorkovací frekvenci.

DNL
Differential Non-Linearity: odchylka skutečného kroku mezi sousedními kódy od ideálního LSB; DNL pod minus 1 LSB znamená nemonotonicitu.

Settling time
Doba, než se výstup DAC po změně kódu ustálí v pásmu kolem cílové hodnoty; spolu s glitchem shora omezuje vzorkovací frekvenci.

Rekonstrukce: vzorky zpět na spojitý signál

DAC nevydává spojitou křivku, ale **schodovitý** průběh — drží každý vzorek konstantní až do příchodu dalšího (**zero-order hold**). Tento schod obsahuje kromě užitečného signálu i jeho **spektrální obrazy** kolem násobků vzorkovací frekvence. Aby na výstupu zůstal jen hladký užitečný průběh, řadí se za DAC **rekonstrukční filtr** — dolní propust, která obrazy odřízne. Čím vyšší vzorkovací frekvence vůči užitečnému pásmu, tím dál jsou obrazy a tím snazší je filtr. Toto je zrcadlový obraz anti-aliasingu u ADC (část 5.4) — tatáž teorie, opačný směr.

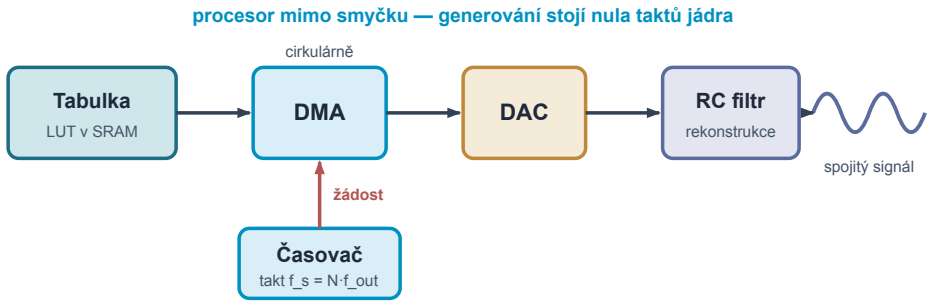
Zero-order hold
Podržení hodnoty vzorku konstantní po celou vzorkovací periodu; dává schodovitý výstup DAC.

Generování harmonického signálu přes DMA

Spojme dosavadní díly. Chceme na výstupu **sinus** o dané frekvenci, a to **bez zatížení procesoru**. Postup:

1. **Předpočítáme tabulku** (look-up table, LUT) jedné periody sinu — N vzorků 12bitových hodnot, jednou, klidně při překladu.
2. **Časovač** nastavíme tak, aby jeho update událost tikala vzorkovací frekvencí $f_s = N \times f_{out}$.
3. **DMA** v cirkulárním režimu propojí tabulku (zdroj, inkrement) s registrem DAC (cíl, pevný); žádost o přenos dává časovač.

Po spuštění běží vše samo: na každý tik časovače DMA odebere další vzorek tabulky a zapíše ho do DAC, po N vzorcích začne tabulku znovu. **Frekvenci** výstupu měníme změnou periody časovače, **amplitudu** a tvar změnou tabulky. Procesor do smyčky nezasahuje vůbec — generování stojí nula taktů jádra. Na stejném principu stojí přehrávání zvukového vzorku, generování testovacích průběhů i tvarování buzení.



Obrázek 5.6. Generování signálu: tabulka v SRAM → časovač udává takt → DMA (cirkulárně) → DAC → rekonstrukční filtr → spojitý sinus. Procesor mimo smyčku.

Rozšířením této myšlenky je DDS (Direct Digital Synthesis): místo abychom tabulkou procházeli po jednom prvku, **adresuje** se zlomkem — *fázovým akumulátorem*, k němuž se v každém kroku přičítá *fázový přírůstek*. Velikost přírůstku pak plynně ladí výstupní frekvenci s jemným rozlišením, **aniž** bychom měnili periodu časovače. DDS je princip, na němž stojí signálové generátory i rádiové syntezátory.

DDS
Direct Digital Synthesis: fázový akumulátor adresuje tabulku zlomkem; změnou fázového kroku se plynně ladí frekvence bez změny časovače.

> **Shrnutí oddílu 5.3.** DAC převádí kód na napětí; nejjednodušší je binárně vážený žebřík, prakticky se používá R-2R nebo odporový řetězec. Binární kód s sebou nese **major-carry glitch** a riziko **nemonotonicity (DNL)** — řeší termometrické/segmentované kódování. Schodovitý výstup (zero-order hold) je nutné vyhladit rekonstrukčním filtrem. Harmonický signál vznikne **bez procesoru** trojicí tabulka → časovač → DMA→DAC v cirkulárním režimu; DDS tutéž myšlenku zjemní fázovým akumulátorem.

5.4 Analogové rozhraní: ADC

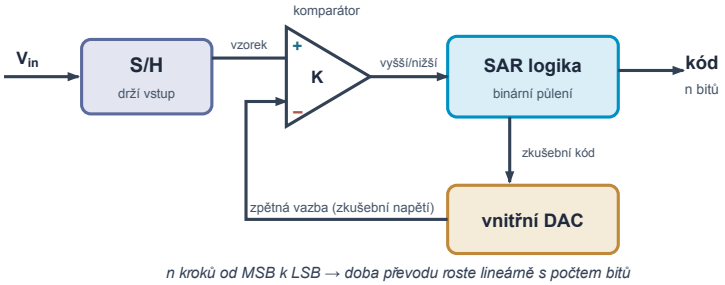
Opačný směr — spojité napětí na číslo — řeší analogově-číslicový převodník. Tato část projde princip postupné aproximace, meze kvantování a vzorkování (kvantizační šum, aliasing) a spojí ADC s časovačem a DMA do plně automatického řetězce sběru dat.

SAR ADC
Successive-Approximation-Register ADC: převodník, který kód najde binárním půlením rozsahu pomocí vnitřního DAC a komparátoru.

SAR ADC a kvantování

Opačný směr — spojité napětí na číslo — řeší ve většině mikrokontrolérů **SAR ADC** (převodník s registrem postupné aproximace). Funguje jako **binární vyhledávání** napětí. Na vstupu je **sample-and-hold** — obvod, který v okamžiku vzorkování zachytí vstupní napětí na kondenzátor a **drží** ho konstantní po celou dobu převodu. Pak SAR logika postupuje od nejvyššího bitu: nastaví ho na 1, **vnitřní DAC** vyrobí odpovídající napětí a **komparátor** porovná, zda je vstup vyšší či nižší. Podle výsledku bit ponechá nebo zruší a pokračuje k dalšímu. Po n krocích (jeden na bit) je kód hotov. Proto **doba konverze roste lineárně s počtem bitů** a SAR je výborný kompromis rychlosti, přesnosti a spotřeby pro jednotky Msps — typický rozsah mikrokontrolérů.

Sample-and-hold
Obvod, který v okamžiku vzorkování zachytí a po dobu konverze drží vstupní napětí konstantní, aby se během převodu neměnilo.



Obrázek 5.7. SAR ADC: sample-and-hold drží vstup, komparátor porovnává s napětím vnitřního DAC, SAR logika binárně půlí rozsah bit po bitu od MSB k LSB.

Kvantování a jeho meze

Převod na konečný počet úrovní je nevyhnutelně **ztrátový**: spojité napětí se zaokrouhlí na nejbližší úroveň, a tato chyba zaokrouhlení je **kvantizační šum**. Pro ideální převodník určuje teoretický odstup signálu od šumu vztah **SNR ≈ 6,02·n + 1,76 dB**, kde n je počet bitů — každý bit přidá zhruba 6 dB. Reálný převodník je horší kvůli šumu, nelinearitě a rušení; skutečnou kvalitu proto vyjadřuje **ENOB** — efektivní počet bitů, odvozený ze skutečného SNR. 12bitový ADC s ENOB ≈ 10,5 je v praxi běžný a je **věrnějším** údajem než nominální rozlišení. Část kvantizačního šumu lze „vyměnit za rychlost" **převzorkováním** (dále v této části).

Kvantizační šum
Chyba zaokrouhlení spojité hodnoty na nejbližší úroveň převodníku; jeví se jako šum o velikosti zhruba LSB/√12.

ENOB
Effective Number Of Bits: počet bitů, který by měl ideální převodník o stejném reálném odstupu signál/šum; nižší než nominální rozlišení.

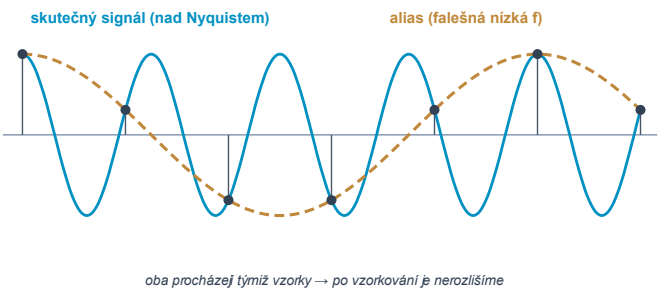
Vzorkovací teorém a aliasing

Vzorkováním v čase vzniká fundamentální omezení, které **nelze obejít softwarem**. **Nyquistův teorém** říká, že signál lze ze vzorků jednoznačně obnovit jen tehdy, vzorkujeme-li **rychleji než dvojnásobkem** jeho nejvyšší frekvence. Je-li ve vstupu složka nad polovinou vzorkovací frekvence (Nyquistovým kmitočtem), **přeloží se** (aliasuje) do užitečného pásma jako falešná nízká frekvence — a od pravého signálu ji už nikdo nerozezná.

Klíčový inženýrský důsledek: **aliasing se musí odstranit analogově, před vzorkováním.** Před ADC proto patří **anti-aliasingový filtr** — dolní propust, která složky nad Nyquistem utlumí dřív, než je vzorkování nevratně zamíchá. Je to hardwarová součást návrhu, ne volba v kódu; vynechat ji znamená nevratně znehodnotit data. Symetrický protějšek u DAC je rekonstrukční filtr z části 5.3.

Nyquistův teorém
Aby šel signál z vzorků jednoznačně rekonstruovat, musí být vzorkovací frekvence vyšší než dvojnásobek nejvyšší frekvence v signálu.

Anti-aliasingový filtr
Analogová dolní propust před ADC, která odstraní složky nad Nyquistovým kmitočtem dřív, než je vzorkování nevratně přeloží do pásma.



Obrázek 5.8. Aliasing: signál nad Nyquistovým kmitočtem dává tytéž vzorky jako nízkofrekvenční „dvojník“; po vzorkování je nerozlišíme.

Režimy, spouštění a STM32 ADC

ADC umí pracovat v několika režimech. **Single** provede jeden převod na pokyn. **Continuous** převádí stále dokola, jakmile dokončí, začíná znovu. **Scan** projde sekvenci více kanálů (více vstupů) v jednom cyklu. Pro kvalitní data je však zásadní **čím** se převod spouští.

Spouštět převod softwarem (z hlavní smyčky nebo časovaného přerušení) je téměř vždy chyba: okamžik vzorkování pak kolísá podle toho, co procesor zrovna dělá (**jitter**), a nepravidelné rozestupy vzorků vnesou do spektra šum a zkreslení. Správné řešení je **externí trigger z časovače**: časovač spustí převod v **přesně ekvidistantních** okamžicích nezávisle na zatížení jádra. Tím se opět vrací řídicí role časovače z přednášky 2 — týž obvod, který generoval PWM, teď udává **rytmus vzorkování**.

Jitter
Časová nejistota okamžiku vzorkování; nepravidelné rozestupy vzorků vnášejí do spektra šum a chyby.

STM32 ADC: skupiny, sekvence a spuštění

ADC na STM32 organizuje převody do dvou skupin. **Regular group** je hlavní sekvence až **16 kanálů**; její výsledek leží v `ADC_DR` a typicky ho odvádí **DMA** — to je náš nepřetržitý stream. **Injected group** je až **4 převody s vyšší prioritou**, které **přeruší** regular sekvenci — chovají se jako „přerušení“ vložené mezi běžné převody — a jejich výsledky padnou do registrů `JDRx` (bez DMA). Typické rozdělení rolí: regular skupina nese trvalý sběr dat přes DMA, injected skupina obslouží občasné **prioritní** měření (například proudovou ochranu ve výkonové elektronice, synchronizovanou s PWM), aniž by přerušila plynulý proud.

Regular group
Hlavní sekvence až 16 převodů ADC; výsledek do datového registru, typicky odváděn DMA — nepřetržitý stream.

Injected group
Až 4 prioritní převody ADC, které přeruší regular sekvenci (obdoba přerušení mezi převody); výsledky v JDRx, bez DMA.

Vícekanálové měření zařídí **scan režim** se sekvencí kanálů v registrech `SQR1`–`SQR3` (pole `SQ1`–`SQ16`): ADC projede zadané kanály v pořadí a každý výsledek DMA uloží na své místo v poli. Každému kanálu se přitom nastavuje **doba vzorkování** (`SMPR1/2`, pole `SMP`): vysokoimpedanční zdroj potřebuje delší dobu, aby se stačil nabít vzorkovací kondenzátor — příliš krátká doba vede k systematické chybě. Řada F4 navíc umí **dual/triple ADC** režim: více převodníků vzorkuje současně, buď *prokládaně* (interleaved → vyšší efektivní vzorkovací frekvence), nebo *simultánně* (fázová shoda mezi kanály, např. pro měření výkonu).

Sestavení celého řetězce časovač → ADC → DMA na úrovni registrů ukazuje, jak málo kódu vlastně stačí — a že v cestě dat není ani jedna instrukce procesoru:

```
// 1) TIM2 generuje TRGO na každý update (přetečení)
TIM2->ARR = period - 1;
TIM2->CR2 = (2 << TIM_CR2_MMS_Pos);     // MMS=010: update -> TRGO
TIM2->CR1 = TIM_CR1_CEN;

// 2) ADC1: externí trigger = TIM2_TRGO, výsledky přes DMA
ADC1->SQR3 = channel;                    // SQ1 = vstupní kanál
```

```
ADC1->SMPR2 = (smp << (3 * channel));     // doba vzorkování kanálu
ADC1->CR2 = ADC_CR2_EXTEN_0               // náběžná hrana ext. triggeru
          | (6 << ADC_CR2_EXTSEL_Pos)     // EXTSEL = TIM2_TRGO
          | ADC_CR2_DMA | ADC_CR2_DDS     // DMA + průběžně (nový request po každém)
          | ADC_CR2_ADON;                 // zapni ADC

// 3) DMA2 Stream0, kanál 0: ADC1->DR -> buf, cirkulárně (viz část 5.2)
```

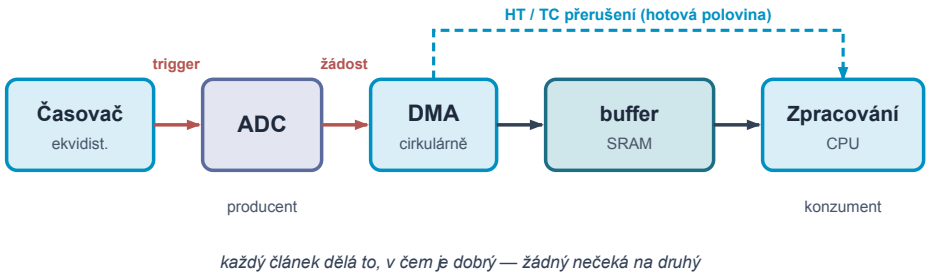
Časovač spouští převod, ADC po dokončení žádá DMA, DMA uloží vzorek do bufferu a procesor se ozve teprve na half/transfer-complete — přesně řetězec z následující sekce.

ADC + DMA: řetězec sběru dat

Spojení ADC s DMA je vyvrcholením celé přenosové části. ADC po každém převodu vydá **DMA žádost**; DMA odebere vzorek z datového registru ADC a uloží ho do bufferu v paměti, v cirkulárním režimu donekonečna. Procesor je vyrušen jen na HT/TC — po polovině a po konci bufferu. Vznikne tak plně automatický **řetězec sběru dat**:

$$\text{časovač} \rightarrow \text{ADC (převod)} \rightarrow \text{DMA (uložení)} \rightarrow \text{buffer} \rightarrow \text{zpracování}.$$

Časovač udává přesný takt, ADC vzorkuje, DMA plní paměť, procesor zpracovává hotové bloky — každý článek dělá to, v čem je dobrý, a žádný nečeká na druhý. Tatáž sestava nese vícekanálové měření (scan + DMA rozloží kanály do pole), softwarově definované rádio i regulační smyčky.



Obrázek 5.9. Řetězec sběru dat: časovač spouští ADC ekvidistantně, ADC žádá DMA, DMA cirkulárně plní buffer, procesor zpracovává poloviny na HT/TC.

Dvě běžná vylepšení. **Převzorkování (oversampling)**: vzorkujeme rychleji, než je nutné, a průměrováním $2\hat{}k$ vzorků získáme až $k/2$ efektivních bitů navíc — vyměníme rychlost za přesnost a zčásti potlačíme kvantizační šum. **Kalibrace**: ADC před měřením změří svou nulu a zesílení a koriguje systematickou chybu; u přesných měření je nutná.

Shrnutí oddílu 5.4. SAR ADC najde kód binárním půlením rozsahu (sample-and-hold, komparátor, vnitřní DAC) — doba převodu roste lineárně s počtem bitů. Kvantování vnáší šum (SNR $\approx$ 6·n dB; reálně ENOB). Nyquistův teorém a **aliasing** vynucují **analogový anti-aliasingový filtr před ADC** — softwarem to nelze napravit. Pro čistá data se ADC spouští **časovačem** (ekvidistantně, bez jitteru) a vzorky odvádí **DMA** do bufferu; dohromady tvoří automatický řetězec časovač→ADC→DMA→buffer.

5.5 Správa bufferů a vlastnictví

Dosud jsme řešili **propustnost**: jak data přesunout, aniž by procesor čekal. Tato část se ptá na něco jemnějšího a v praxi mnohem zákeřnějšího — na **správnost**. Jakmile data přesouvá DMA, sdílí tentýž buffer **tři strany**: hardware (DMA), přerušení (obsluha HT/TC) a aplikace (zpracování). A na rozdíl od dvou softwarových vláken, mezi nimiž lze postavit zámek, **mezi DMA hardwarem a softwarem žádný zámek neexistuje** — DMA bude do bufferu zapisovat, ať si software myslí cokoli. Korektnost se proto nedá vynutit zamykáním; musí ji zajistit **disciplína časování a vlastnictví**.

Tři vlastníci jednoho bufferu

Vlastnictví (ownership) bufferu je dohoda o tom, **kdo v daném okamžiku smí na paměť sahat**. Není to jazyková konstrukce ani zámek — je to **invariant**, který musí návrh dodržet. Pokud aplikace čte z téže oblasti, kterou právě přepisuje DMA, dostane směs starých a nových dat (**data race**); pokud zapíše do oblasti, kterou DMA právě vysílá, poškodí odchozí proud. Cílem návrhových vzorů níže je zajistit, aby se **okno zápisu DMA a okno přístupu softwaru nikdy nepřekryla**.

Vlastnictví
Pravidlo určující, která strana smí v daném okamžiku do bufferu zapisovat nebo z něj číst; bez něj vzniká souběh DMA a softwaru.

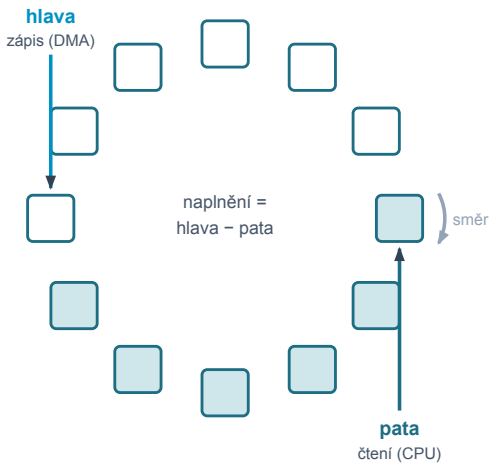
Data race
Souběžný přístup k téže paměti, kde alespoň jeden je zápis, bez uspořádání; výsledek je nedefinovaný — směs starých a nových dat.

Navíc je tu `volatile` z přednášky 3: DMA mění obsah bufferu i indexy „za zády" překladače. Proměnné, které sdílí ISR a hlavní kód (indexy, příznaky hotovo), musí být `volatile`, jinak si je překladač zoptimalizuje do registru a změnu z přerušení hlavní smyčka nikdy neuvidí. `volatile` ale řeší jen **viditelnost**, ne **uspořádání** — k tomu se vrátíme u bariér.

Kruhový a dvojitý buffer

Nejstarší a nejuniverzálnější nástroj proti nesouladu rytmů producenta a konzumenta je **kruhový buffer** (ring / circular buffer): pole, do něhož producent zapisuje na pozici **hlavy** a konzument čte z pozice **paty**; oba indexy po dosažení konce **přeskočí na začátek**, takže pole se chová jako nekonečný pás. Rozdíl hlavy a paty udává naplnění; buffer je prázdný, když se sejdou, a plný, když by hlava dohnala patu.

Kruhový buffer
Pole s ukazatelem zápisu (hlava) a čtení (pata), které po dosažení konce přeskočí na začátek; absorbuje rozdíl rytmů producenta a konzumenta.



Obrázek 5.10. Kruhový buffer: producent zapisuje na hlavu, konzument čte z paty; po konci pole oba indexy přeskočí na začátek. Rozdíl indexů = naplnění.

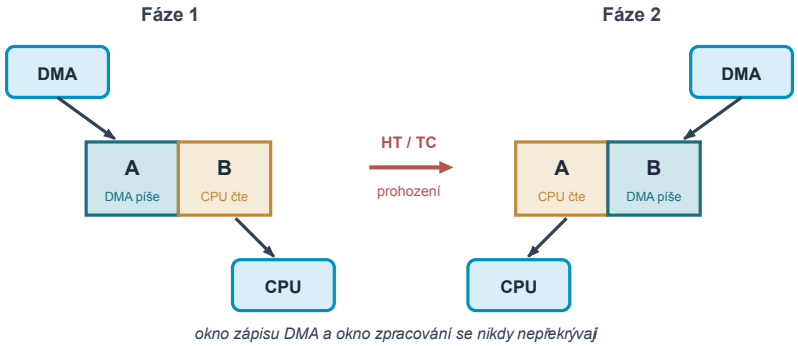
Klíčová vlastnost: pro **jednoho producenta a jednoho konzumenta** (SPSC) lze kruhový buffer udělat **bez zámku**. Stačí, aby každou stranu indexu měnil jen jeden účastník — producent píše jen hlavu, konzument jen patu — a aby oba indexy byly `volatile`

a aktualizovaly se **až po** zápisu/čtení dat (správné uspořádání). Tehdy se obě strany nikdy nedostanou do sporu o tutéž buňku. To je přesně situace **DMA (producent) ↔ aplikace (konzument)** a důvod, proč je kruhový buffer základním kamenem ovladačů. Při přetečení (konzument trvale nestíhá) je nutné definovat politiku: zahodit nejstarší, zahodit nový, nebo hlásit chybu — mlčky přepsat nepřečtená data je nejhorší volba.

Dvojitý buffer (ping-pong) a half-transfer

DMA v cirkulárním režimu spolu s dvojicí **HT + TC** dává obzvlášť elegantní podobu téhož principu — **dvojitý buffer** (double buffering, ping-pong). Buffer se logicky rozdělí na dvě poloviny. DMA plní jednu polovinu, zatímco software zpracovává druhou; jakmile DMA dosáhne poloviny (HT) nebo konce (TC), **role se prohodí**. Half-transfer přerušení zde není luxus, ale **podmínka funkčnosti**: bez něj by software nevěděl, že první polovina je hotová, a musel by čekat na celý blok — čímž by se latence zdvojnásobila a souběh by zmizel.

Dvojitý buffer
Dvě poloviny bufferu: zatímco DMA plní jednu, software zpracovává druhou; po každém HT/TC se role prohodí.



Obrázek 5.11. Ping-pong: DMA plní polovinu A, software zpracovává B; na HT/TC se prohodí. Okno zápisu DMA a okno zpracování se nikdy nepřekrývají.

Předností dvojitého bufferu je, že **vlastnictví je vždy jednoznačné**: v každém okamžiku patří jedna polovina výhradně DMA a druhá výhradně softwaru, a hranice se přehazují v přesně definovaných bodech (HT, TC). Tím je invariant vlastnictví z úvodu této části zaručen **konstrukcí**, ne hlídáním.

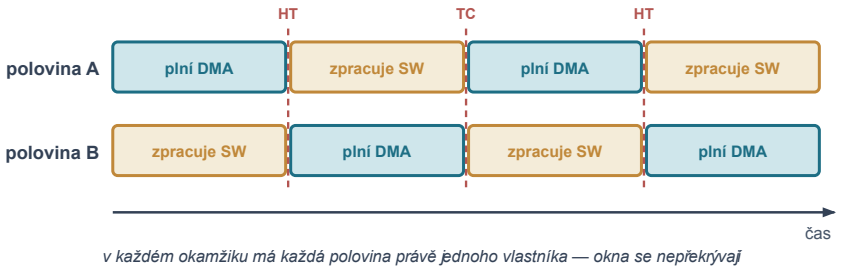
Vlastnictví v čase

Pohled, který sjednocuje předchozí: vykreslete si **časovou osu vlastnictví**. Buffer (nebo jeho polovina) přechází mezi třemi stavy — *patří DMA* (probíhá přenos), *předáván* (událost HT/TC), *patří softwaru* (zpracování) — a tyto stavy se **nesmí překrýt**. Návrh je správný právě tehdy, když pro každý bajt bufferu platí, že v každém okamžiku má **nejvýše jednoho vlastníka s právem zápisu**.

Tento obraz je důležitější než jakákoli konkrétní API, protože je **přenositelný**: tatáž úvaha „kdo teď vlastní zdroj" se vrátí u RAII a `move` v C++, u borrow-checkeru v Rustu a u předání vlastnictví zprávou v RTOS (viz oddíl 5.6). Streamovací buffer je první místo, kde student na vlastnictví **narazí z nutnosti**, ne z teorie.

Kam buffer umístit a koherence cache

Ne každá paměť je pro DMA použitelná a ne každá je stejně rychlá. Dvě pravidla. Za prvé, **DMA musí na danou paměť vidět**: na některých čipech existují banky (např. tightly-coupled DTCM/CCM SRAM přilepené přímo k jádru), na které **DMA řadič nemá přístup** — buffer v nich DMA nenaplní. Za druhé, kvůli souběhu na bus matrix (část 5.2) je výhodné dát **DMA**



Obrázek 5.12. Časová osa vlastnictví poloviny bufferu: DMA → (HT/TC) → software → (zpět) → DMA. Okna se nikdy nepřekrývají; to je invariant správnosti.

buffer a zásobník/kód do různých bank, aby masteři nesoupeřili. Umístění do konkrétní sekce (`.dma_buffer`) se řeší v **linker skriptu** — což přímo navazuje na členění obrazu paměti z přednášky 3.

Cache a koherence: past na Cortex-M7

Na jádrech s datovou cache (Cortex-M7, M55) vzniká problém, který na M4 neexistuje a který **tiše poškozuje data**. Jádro nečte SRAM přímo, ale přes **datovou cache**; DMA naproti tomu pracuje s **pravou pamětí**, cache ho nevidí. Vzniká **nekoherence**:

- **Příjem (periferie → paměť):** DMA zapíše nová data do SRAM, ale jádro má v cache **starou** kopii téhož řádku a čte ji — nová data nevidí. Řešení: po dokončení přenosu řádky bufferu **invalidovat** — zahodit kopie z cache, aby si je jádro znovu načetlo z paměti.
- **Vysílání (paměť → periferie):** jádro zapíše vzorky, ale ty zatím leží jen v cache (write-back), do SRAM ještě nedoputovaly; DMA z SRAM odešle **stará** data. Řešení: před spuštěním přenosu řádky **vyčistit** (clean) — vyprázdnit cache do paměti.

Datovou cache
Rychlá vyrovnávací paměť mezi jádrem a SRAM; jádro vidí kopie v cache, zatímco DMA pracuje s pravou pamětí — vzniká nesoulad.

Nekoherence
Stav, kdy cache jádra a hlavní paměť obsahují různá data téže adresy, protože DMA zapsal do paměti mimo cache.

Invalidovat
Zneplatnit řádky cache, aby si je jádro při dalším čtení znovu načetlo z paměti (po DMA příjmu).

Vyčistit
Zapsat „špinavé“ řádky cache zpět do paměti, aby DMA odeslal aktuální data (před DMA vysíláním).



Obrázek 5.13. Cache vs. DMA na M7: jádro vidí cache, DMA hlavní paměť. Po RX invalidovat, před TX vyčistit; jinak jádro čte stará data nebo DMA vyšle stará data.

Operace clean/invalidate pracují po **celých řádcích cache** (32 B na M7), takže DMA buffer musí být **zarovnaný na řádek** a nesmí v něm s ničím jiným řádek sdílet — jinak by invalidace sousedních dat zahodila i platné proměnné. Elegantnější alternativou je vyhradit DMA bufferům **necacheovatelný region** přes MPU (z přednášky 4); pak žádné ruční clean/invalidate netřeba, za cenu pomalejšího přístupu jádra k bufferu. Tato past je učebnicový příklad toho, jak se abstrakce (cache) rozbije, jakmile do hry vstoupí druhý master (DMA) — a proč inženýr musí rozumět vrstvě pod jazykem.

Shrnutí oddílu 5.5. Streamovaný buffer sdílí DMA, ISR a aplikace, a mezi hardwarem a softwarem **není zámek**; správnost zajišťuje disciplína vlastnictví — invariant „nejvýše jeden vlastník s právem zápisu". Kruhový buffer řeší SPSC bez zámku (`volatile` indexy, správné uspořádání); dvojitý buffer s HT+TC činí vlastnictví jednoznačným konstrukcí. Buffer musí ležet v paměti **viditelné pro DMA** a kvůli souběhu v jiné bance než zásobník. Na M7/M55 hrozí **nekoherence cache**: po příjmu invalidovat, před vysíláním vyčistit, zarovnat na řádek (nebo region označit jako necacheovatelný).

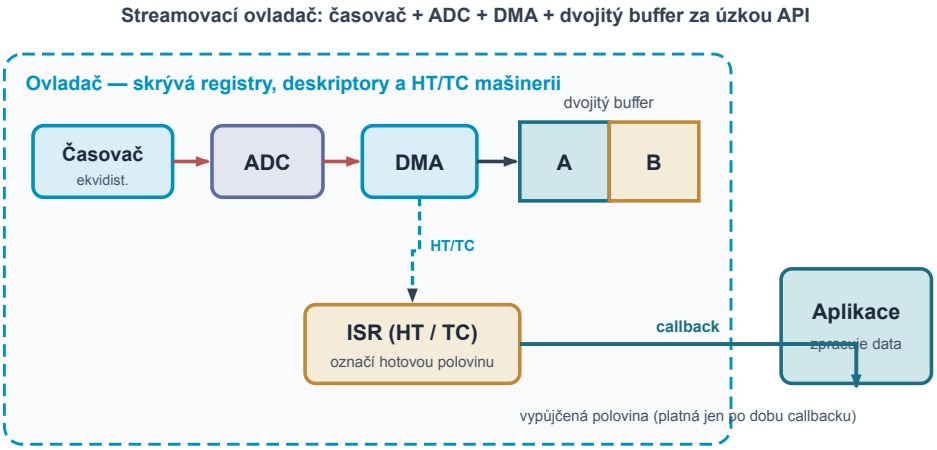
5.6 Syntéza: streamovací ovladač

Zbývá složit časovač, ADC, DMA a dvojitý buffer do jednoho návrhového vzoru. Tato závěrečná část ukáže streamovací ovladač, jehož API respektuje vlastnictví dat, a zasadí ho do širší spirály vlastnictví, která se táhne celým kurzem.

Streamovací ovladač a API vlastnictví

Spojme všechno do jednoho **návrhového vzoru**, který je v embedded systémech všudypřítomný: streamovacího ovladače pro sběr dat. Skládá se z dílů, které jsme postavili:

- **časovač** (přednáška 2) udává ekvidistantní vzorkovací takt,
- **ADC** převádí a na každý vzorek žádá DMA,
- **DMA** v cirkulárním režimu plní **dvojitý buffer** a hlásí HT/TC,
- **ISR** na HT/TC označí hotovou polovinu a předá ji aplikaci,
- **aplikace** hotovou polovinu zpracuje, **dokud ji DMA nezačne zase přepisovat**.



Obrázek 5.14. Streamovací ovladač: časovač→ADC→DMA plní dvojitý buffer; ISR na HT/TC předává hotovou polovinu callbackem; aplikace ji zpracuje v okně vlastnictví. Ovladač skrývá registry za API.

Tím se uzavírá oblouk k přednášce 3: ovladač **skryje registry, deskriptory a HT/TC mašinerii za úzkou API** a aplikaci nabídne jen smysluplné operace — `start`, `stop` a callback „je hotová polovina, zde je". Vnitřek (DMA, cache, zarovnání) zůstává skrytý; ven vystupuje jen čistý datový proud.

API, které respektuje vlastnictví

Návrh API je zde **rozhodnutí o vlastnictví**, ne kosmetika. Callback předá aplikaci **vypůjčený pohled** (borrow) na hotovou polovinu — ukazatel a délku — s jasným kontraktem: *data jsou platná jen po dobu callbacku; do příštího HT/TC je DMA zase přepíše.* Aplikace si je tedy musí buď zpracovat hned, nebo **zkopírovat** do vlastní paměti, pokud je potřebuje déle. To je přesně rozdíl mezi **vypůjčením** (borrow — dočasný přístup bez vlastnictví) a **vlastnictvím**

(ownership — právo držet a měnit), a embedded student ho zde potká poprvé **vynucený realitou hardwaru**, dávno než narazí na `std::span` nebo na borrow-checker.

```
// kontrakt: 'samples' platí JEN po dobu callbacku (vypůjčení, ne vlastnictví);
// po návratu polovinu opět vlastní DMA a přepíše ji.
typedef void (*adc_block_cb)(const uint16_t *samples, size_t n);

void adc_stream_start(adc_block_cb on_block);   // spustí časovač+ADC+DMA
void adc_stream_stop(void);

void DMA2_Stream0_IRQHandler(void) {
    if (DMA2->LISR & DMA_LISR_HTIF0) {          // hotová první polovina
        DMA2->LIFCR = DMA_LIFCR_CHTIF0;
        on_block(&buf[0], HALF);                 // vypůjčíme aplikaci
    }
    if (DMA2->LISR & DMA_LISR_TCIF0) {          // hotová druhá polovina
        DMA2->LIFCR = DMA_LIFCR_CTCIF0;
        on_block(&buf[HALF], HALF);
    }
}
```

Co dělat s přetečením

Robustní ovladač musí počítat s tím, že aplikace občas **nestihne** zpracovat polovinu dřív, než ji DMA dohoní (**backpressure**). Mlčet a ztrácet data je nepřijatelné. Definované reakce: detekovat zpoždění (např. další HT/TC přišel dřív, než aplikace dokončila předchozí), **započítat ztracený blok** a stav signalizovat výš; případně zvětšit buffer (víc rezervy), zpomalit vzorkování, nebo přidat mezistupeň kruhového bufferu mezi ISR a zpracování. Návrh musí ztrátu dat **vidět a hlásit**, ne ji zakrývat — tichá ztráta vzorků je v měřicím i regulačním systému těžko dohledatelná závada.

Backpressure
Situace, kdy konzument nestíhá tempo producenta; systém musí definovaně reagovat (zahodit, signalizovat, zpomalit), ne mlčky ztrácet data.

Spirála vlastnictví dopředu

Vlastnictví bufferu, na něž jsme zde narazili **z nutnosti**, je první závit širší spirály, která se v kurzu vrací na stále vyšší úrovni abstrakce:

- **zde (lec05)** — ownership bufferu vynucený hardwarem; hlídaný ručně, disciplínou a API kontraktem;
- **C++ / moderní jazyky** — totéž **formalizované jazykem**: RAII (životnost objektu = životnost zdroje), `move` (přenos vlastnictví), `span`/reference (vypůjčení bez vlastnictví); v Rustu navíc **vynucené překladačem** (borrow-checker), takže souběh, jaký hrozí u DMA bufferu, vůbec nezkompiluje;
- **RTOS** — ownership zdroje mezi vlákny: mutex, fronta zpráv, **předání vlastnictví bufferu zprávou** (producent ho po odeslání už nevlastní).

Pokaždé jde o tutéž otázku — *kdo teď zdroj vlastní a kdy ho předá* — jen s rostoucí mírou, v níž ji za nás hlídá nástroj místo disciplíny. Že tato otázka v embedded vůbec vyvstává, je přímý důsledek toho, co tato přednáška ukázala: jakmile data přesouvá někdo jiný než procesor, musí být jasné, čí jsou.

Závěr

Datový tok není vedlejší téma, ale **prvotřídní návrhový problém**. Procesor je špatný kopírovací stroj, a proto pohyb dat svěřujeme DMA; analogové převodníky DAC a ADC tvoří nejcitlivější datové proudy a s DMA i časovačem skládají automatické řetězce **bez zatížení jádra**. Cenou za autonomii dat je nový problém — **vlastnictví sdíleného bufferu** — a jeho řešení (kruhový a dvojitý buffer, časová osa vlastnictví, koherence cache) je disciplína, kterou dobrý embedded inženýr ovládá dřív, než ji potká formalizovanou v jazyce. Procesor jsme zbavili kopírování dat; nyní musí zajišťovat správu a kontrolu přenosu.

> **Shrnutí oddílu 5.6.** Streamovací ovladač skládá časovač+ADC+DMA+dvojitý buffer a skrývá je za API s callbackem, který aplikaci **vypůjčuje** hotovou polovinu (platnou jen po dobu callbacku). Robustní návrh definovaně řeší backpressure místo tiché ztráty dat. Vlastnictví bufferu je první závit spirály, jež pokračuje do C++ (RAII/move/span), Rustu (borrow-checker) a RTOS (předání vlastnictví zprávou).

5.7 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 5.1 — Procesor jako úzké hrdlo přenosu dat

1. Proč je per-sample přerušení nevhodné pro proud 1 Msps, ačkoli pro 10 ksps funguje dobře? Odhadněte zatížení jádra v obou případech.
2. Co je rozdíl mezi overrunem a underrunem a u kterého směru přenosu každý nastává?
3. Vysvětlete, proč determinismus DMA přenosu nezávisí na okamžitém zatížení procesoru — a proč je to pro regulační smyčku důležité.

K oddílu 5.2 — DMA: autonomní přesun dat

1. Čím se v konfiguraci DMA liší přenos periferie↔paměť od kopie paměť↔paměť (směr, inkrementace adres)?
2. Proč se vyplatí umístit DMA buffer a zásobník procesoru do různých SRAM bank?
3. K čemu slouží half-transfer přerušení a proč ho samotné transfer-complete nenahradí?
4. Proč cirkulární režim DMA umožní nekonečný datový proud bez zásahu procesoru — co se stane s čítačem a adresou na konci bloku?

K oddílu 5.3 — Analogové rozhraní: DAC a generování signálu

1. Proč PWM s RC filtrem nestačí na generování audio signálu a DAC ano?
2. Proč je binárně vážený žebřík nepraktický a co řeší R-2R?
3. Co je major-carry glitch a proč u něj hrozí i nemonotonicita? Jak ji odstraní termometrické kódování?
4. Co omezuje maximální vzorkovací frekvenci DAC?
5. K čemu slouží rekonstrukční filtr a jak souvisí s anti-aliasingem u ADC?
6. Jak v sestavě tabulka→časovač→DMA→DAC změníte výstupní frekvenci, aniž byste sáhli na tabulku?

K oddílu 5.4 — Analogové rozhraní: ADC

1. Proč doba převodu SAR ADC roste lineárně s počtem bitů?
2. Co je aliasing a proč ho nelze odstranit softwarem po vzorkování?
3. Proč je špatné spouštět ADC z časovaného přerušení a co použít místo toho?
4. Jaký je rozdíl mezi regular a injected skupinou a k čemu se každá hodí?
5. Proč potřebuje vysokoimpedanční zdroj delší dobu vzorkování?
6. Jak převzorkování vymění rychlost za rozlišení?
7. Rozepište řetězec časovač→ADC→DMA→buffer a uveďte, který článek je producent a který konzument.

K oddílu 5.5 — Správa bufferů a vlastnictví

1. Proč mezi DMA a aplikací nelze korektnost zajistit zámkem a co ji zajišťuje místo toho?
2. Proč musí být indexy kruhového bufferu `volatile` a proč to samo o sobě nestačí?
3. Jak dvojitý buffer s HT+TC zaručuje, že se okno zápisu DMA a okno zpracování nepřekryjí?
4. Na M7 přijímáte data přes DMA do cacheovaného bufferu a čtete stará data — co se stalo a jak to napravit?
5. Proč musí být DMA buffer zarovnaný na řádek cache?

6

Přístrojové sběrnice

> Tato kapitola představí sériové sběrnice pro připojování externích periferií. Žádná není nejlepší ve všem — každá je kompromis mezi rychlostí, dosahem, odolností, spotřebou a počtem vodičů. Projdeme proto aplikační požadavky a kritéria (návrhové osy), podle nichž se sběrnice vybírá.

Obsah kapitoly

6.1 Kritéria výběru sériové sběrnice
6.2 On-board synchronní sběrnice: SPI a I2C
6.3 On-board minimalistické a moderní: 1-Wire, I3C, I2S
6.4 Field a síťové: diferenciál, vzdálenost, robustnost
6.5 Zprávově orientované a aplikační: CAN, LIN, DALI
6.6 Návrh sériového protokolu
6.7 Syntéza: výběr sběrnice a její vlastnictví
6.8 Kontrolní otázky

6.1 Kritéria výběru sériové sběrnice

Sériová sběrnice řeší zdánlivě jednoduchý úkol — přenést data z jednoho obvodu do druhého — ale podmínky přenosu se případ od případu zásadně liší. Jednou jde o dva čipy vzdálené centimetr na téže desce, jindy o uzly roztažené po hale uprostřed elektromagnetického rušení. Jednou potřebujeme desítky megabitů za sekundu, jindy stačí kilobity, ale za každou cenu spolehlivě. Jednou je master je jediný a řídí přenos, jindy vysílá víc uzlů, které se musí koordinovat. **Tyto požadavky si často odporují**, a proto neexistuje jedna ideální sběrnice — existuje rodina kompromisů. Následujících sedm os je souřadný systém, v němž každá sběrnice zaujímá svůj bod.

Sedm návrhových os

1. Synchronní vs. asynchronní. Synchronní sběrnice vede zvláštní vodič hodin (SPI, I2C) — přijímač vzorkuje data hranou hodin, takže rychlost může být libovolná a měnit se za běhu. Asynchronní sběrnice (UART) hodiny nevede; obě strany se musí **předem dohodnout na rychlosti** (baud rate) a přijímač si takt obnoví z hran dat. Synchronní je rychlejší a flexibilnější, asynchronní ušetří vodič za cenu nutné shody rychlostí.

Synchronní
Sběrnice s vodičem hodin: vysílač i přijímač sdílejí takt, takže není potřeba předem dohodnutá rychlost.

Asynchronní
Sběrnice bez vodiče hodin: obě strany se musí předem dohodnout na přenosové rychlosti a přijímač si takt obnoví ze samotných dat.

2. Počet vodičů vs. propustnost vs. adresace. Více vodičů obvykle znamená vyšší rychlost nebo jednodušší protokol, méně vodičů šetří piny a místo, ale platí se složitostí. SPI má čtyři vodiče a žádné adresy (zařízení vybírá zvláštní signál), I2C dva vodiče a **adresy** v protokolu,

1-Wire jediný vodič a adresu zašitou do časování. Adresace je cena za sdílení vodičů více zařízeními.

3. Single-ended vs. diferenciální. Single-ended signál (SPI, I2C, RS-232) se měří proti společné zemi — jednoduché a levné, ale citlivé na rušení a krátké. Diferenciální signál (RS-485, CAN) se přenáší jako **rozdíl dvou vodičů**: rušení zasáhne oba stejně a v přijímači se odečte, takže projde stovky metrů i v hluku. Diferenciál je klíč k dosahu a odolnosti.

4. Master/slave vs. multi-master / zprávově orientovaná. U jednoduchých sběrnic **jeden master řídí přenos** a ostatní jen odpovídají (SPI, klasicky I2C). U sítí vysílá víc rovnocenných uzlů a musí se určit, který právě vysílá — buď' arbitráží, nebo časovým plánem. Vrcholem je **zprávově orientovaný** model CAN, kde se nevysílá „uzlu X", ale **zpráva s identifikátorem** a uzly přijmou zprávy, které jsou pro ně relevantní.

5. Topologie. Bod-bod (RS-232), sběrnice s odbočkami (multi-drop: RS-485, CAN), hvězda nebo řetěz. Topologie určuje, kolik uzlů a jak daleko, a souvisí s adresací i terminací vedení.

6. Determinismus a real-time. Pro řízení nestačí „rychle", musí to být **včas a zaručeně**. CAN garantuje, že nejprioritnější zpráva projde bez zpoždění, FlexRay přiděluje pevné časové sloty. Determinismus je důvod, proč v autě nenajdete USB na brzdách — a pojítko k plánování úloh v RTOS.

7. Napájení po sběrnici a izolace. Některé sběrnice nesou i napájení (1-Wire parazitně, PoE po ethernetu, DALI ve vedení), což ušetří vodiče. V průmyslu se navíc vyžaduje **galvanická izolace** — oddělení zemí, aby se rušení a poruchová napětí nešířila mezi uzly.

Single-ended
Signál vztažený k jedné společné zemi; jednoduché a levné, ale citlivé na rušení a omezené na krátké vzdálenosti.

Diferenciální
Signál přenášený jako rozdíl dvou vodičů; rušení postihne oba stejně a odečte se, což umožní velké vzdálenosti a odolnost.

Arbitráží
Mechanismus, který při současném vysílání více uzlů určí bez kolize, kdo pokračuje; u CAN podle priority zprávy (ID).

Dvě rodiny: on-board a field

Sedm os se přirozeně shlukuje do **dvou světů**. On-board sběrnice (SPI, I2C, I3C, 1-Wire, I2S) spojují čip se součástkami na **téže desce**: krátká vzdálenost, single-ended, jednoduchá topologie, MCU obvykle master. Zde jde o **pohodlí a propustnost** mezi blízkými obvody. Field sběrnice (RS-485, CAN, LIN, DALI) spojují **celá zařízení** na metry až stovky metrů: diferenciální, odolné, multi-drop, často deterministické a izolované. Zde jde o **robustnost a dosah**. Toto rozdělení použijeme jako osnovu zbytku přednášky.

> **Shrnutí oddílu 6.1.** Sběrnic je mnoho, protože jejich požadavky si odporují; každá je kompromis v sedmi osách: synchronní/asynchronní, vodiče/propustnost/adresace, single-ended/diferenciální, master-slave/multi-master, topologie, determinismus, napájení/izolace. Osy se shlukují do dvou rodin — **on-board** (chip↔chip, krátké, single-ended) a **field** (zařízení ↔zařízení, dlouhé, diferenciální, odolné).

On-board sběrnice
Sběrnice spojující čip s dalšími obvody na téže desce; krátká vzdálenost, single-ended, MCU bývá master.

Field sběrnice
Sběrnice spojující celá zařízení na vzdálenost; diferenciální, odolné, multi-drop, často s determinismem a izolací.

6.2 On-board synchronní sběrnice: SPI a I2C

Dvě nejrozšířenější on-board sběrnice jsou synchronní a leží na opačných koncích kompromisu mezi počtem vodičů a rychlostí. Tato část projde SPI, které upřednostňuje rychlost za cenu více vodičů, a I2C, které šetří vodiče za cenu rychlosti a protokolové režie — a nakonec ukáže, podle čeho mezi nimi volit.

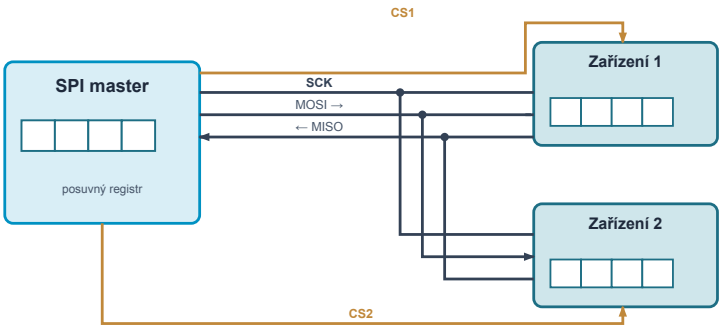
SPI: rychlost za cenu vodičů

SPI (Serial Peripheral Interface) je nejrychlejší a nejjednodušší on-board sběrnice. Má **čtyři vodiče**: hodiny `SCK`, data z mastera `MOSI`, data do mastera `MISO` a výběr zařízení `CS` (chip select). Je **synchronní** a **full-duplex** — v každém taktu se posílá bit jedním směrem a zároveň přijímá druhým, jako kdyby si master a zařízení vyměňovaly obsah dvou posuvných registrů. **Adresování řeší vodič `CS`**: každé zařízení má svůj, a master aktivuje právě to, s nímž

SPI
Serial Peripheral Interface: synchronní full-duplex sběrnice se 4 vodiči (MOSI, MISO, SCK, CS); master řídí přenos, zařízení vybírá CS, bez adres.

komunikuje. To je jednoduché a rychlé (desítky MHz), ale **neškáluje počtem zařízení** — každé další vyžaduje další pin `CS`.

Master volí **polaritu a fázi hodin** (`CPOL`/`CPHA`, čtyři „SPI módy") podle toho, na které hraně zařízení vzorkuje data; nesoulad módu je klasická příčina nefunkčního přenosu. SPI nemá potvrzování ani detekci chyb — spoléhá na krátký, čistý spoj na desce. Historickým předchůdcem je **Microwire** (třídrátová varianta od National pro EEPROM); rozšířením směrem k paměti je **QSPI/Octo-SPI**, které místo jednoho datového vodiče použije čtyři nebo osm a tím **několikanásobí propustnost** k sériové Flash — tak rychle, že z ní lze i přímo vykonávat kód (execute-in-place).



Obrázek 6.1. SPI: master a zařízení propojené SCK/MOSI/MISO, výběr zařízení samostatným CS; full-duplex výměna jako dva propojené posuvné registry.

I2C: vodiče za cenu rychlosti

I2C (Inter-Integrated Circuit) jde opačnou cestou: **dva vodiče** pro libovolný počet zařízení. Vede jen data `SDA` a hodiny `SCL`, obojí v zapojení **open-drain** se společnými pull-up rezistory — proto může vodič stáhnout kterékoli zařízení, aniž by vznikl konflikt. Místo výběrového vodiče nese protokol **adresu** (7 nebo 10 bitů): master vyšle start, adresu a směr, odpoví jen zařízení, jehož adresa se shoduje, a po každém bajtu potvrdí příjem bitem `ACK`; přenos uzavře **stop**. Cena za dva vodiče je nižší rychlost (standardně 100 kHz, 400 kHz, 1 MHz; rychlý režim 3,4 MHz) a větší protokolová režie.

Klíčem k pochopení I2C je, že `SDA` i `SCL` jsou **wired-AND**: log. 0 je **dominantní** a log. 1 **recesivní**. Pull-up vytáhne vodič do 1, jen když ho uvolní všechna zařízení; jakmile ho kdokoli stáhne k 0, je tam 0. Tato jediná elektrická vlastnost dává I2C dvě jeho typické schopnosti — bezkolizní arbitráž a clock stretching.

Arbitráž více masterů je přímým důsledkem wired-AND. Začnou-li dva masteři vysílat zároveň, každý sleduje, zda se `SDA` shoduje s tím, co sám vysílá. Dokud posílají totéž, oba pokračují; jakmile jeden pošle recesivní 1, ale na vodiči vidí dominantní 0 (kterou tam dal druhý), **ztrácí arbitráž** a okamžitě přestane vysílat. Master, který arbitráž vyhrál, pokračuje bez ztráty dat.

Clock stretching využívá totéž na hodinovém vodiči. Pomalé zařízení, které nestačí zpracovat bajt, **podrží `SCL` u země** i poté, co ho master pustil. Master po každém taktu kontroluje, že `SCL` opravdu vyletělo do 1; dokud čte 0, čeká. Zařízení tak získá potřebný čas, aniž by se ztratila data. Rubem je, že **zaseknuté zařízení držící `SCL` může celou sběrnici zablokovat** — robustní master proto má timeout.

Nad fyzickým I2C konečně stojí **aplikační profily** — přísnější pravidla a standardizovaná sada příkazů. **SMBus** je přísnější varianta pro správu (například stav baterie), **PMBus** řídí napájecí měniče. Sběrnice je stejná; profil přidává *co a jak* se po ní posílá — je to vyšší vrstva, podobně jako Modbus nad RS-485 (oddíl 6.4). I2C je proto typickou sběrnicí pro propojení mnoha

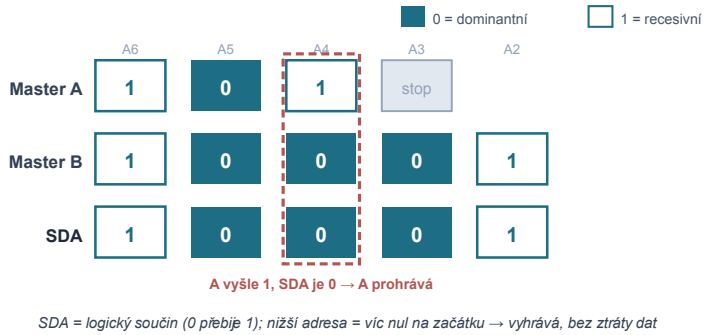
I2C
Inter-Integrated Circuit: synchronní 2vodičová sběrnice (SDA, SCL) s open-drain vedením, 7/10bitovými adresami a podporou více masterů; pomalejší, ale úsporná na piny.

Open-drain
Výstup, který umí jen stáhnout vodič k zemi; do log. 1 ho vytáhne společný rezistor. Umožní více zařízením sdílet vodič bez konfliktu.

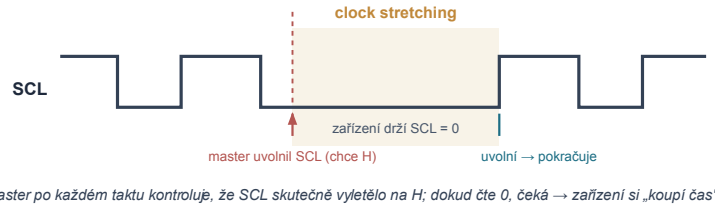
Wired-AND
Vodič v open-drain zapojení, kde log. 0 je dominantní a log. 1 recesivní: pull-up dá 1, jen když ho pustí všechna zařízení; kdokoli ho stáhne k 0, přebije ostatní.

Arbitráž
Řešení konfliktu, když dva masteři začnou vysílat zároveň: porovnávají vysílaný bit se skutečným stavem SDA a kdo vyšle recesivní 1, ale uvidí dominantní 0, ztrácí arbitráž a přestane vysílat — bez ztráty dat protistrany.

Clock stretching
Zařízení, které nestačí zpracovat data, podrží SCL u země; master po každém taktu kontroluje, že SCL skutečně vyletělo do 1, a dokud čte 0, čeká. Tím pomalé zařízení získá potřebný čas.



Obrázek 6.2. Arbitráž na I2C: dva masteři vysílají zároveň; ten, kdo pošle recesivní 1 a uvidí na SDA dominantní 0, ztrácí arbitráž a přestane vysílat. Druhý master pokračuje bez ztráty bajtu.



Obrázek 6.3. Clock stretching: master uvolnil SCL (chce 1), ale zařízení ho drží u 0; master čeká, dokud SCL nevyletí. Tím pomalé zařízení získá čas.

pomalých senzorů a konfiguračních pamětí dvěma vodiči — s vědomím, že clock stretching, sdílené adresy a dimenzování pull-upů jsou jeho typická úskalí.

SPI versus I2C: kdy co

Obě jsou synchronní on-board sběrnice, ale leží na opačných koncích osy 2 (vodiče × rychlost × adresace), jak shrnuje tab. 6.1:

Tabulka 6.1. Srovnání sběrnic SPI a I2C

	SPI	I2C
vodiče	4 (+ 1 CS na zařízení)	**2** (sdílené)
rychlost	**desítky MHz**	0,1–3,4 MHz
výběr zařízení	vodič CS	adresa v protokolu
full/half duplex	**full**	half
potvrzení / multi-master	ne / ne	ACK / **ano**
typické použití	rychlé převodníky, displeje, Flash	mnoho pomalých senzorů, EEPROM

Orientační pravidlo: **rychlost nebo full-duplex → SPI; úspora pinů u mnoha pomalých zařízení → I2C.** Obě se v přednášce 5 napojí na DMA — rychlý SPI proud (např. externí ADC) se streamuje přesně jako vnitřní převodník.

> **Shrnutí oddílu 6.2.** SPI a I2C jsou synchronní on-board sběrnice na opačných koncích osy vodiče×rychlost×adresace. **SPI** — 4 vodiče, full-duplex, rychlé, výběr přes CS, bez adres a potvrzení (+ Microwire, QSPI/OSPI pro Flash). **I2C** — 2 vodiče open-drain, adresy 7/10 b, ACK, clock stretching, multi-master, pomalejší (+ profily SMBus/PMBus). SPI pro rychlost, I2C pro úsporu pinů u mnoha pomalých zařízení.

6.3 On-board minimalistické a moderní: 1-Wire, I3C, I2S

Vedle SPI a I2C existují on-board sběrnice, které kompromis tlačí do krajnosti nebo ho modernizují. Tato část představí minimalistické 1-Wire (jediný vodič, který nese i napájení), moderní nástupce I2C zvaný I3C a audio sběrnici I2S, jež propojuje procesor s převodníky.

Jeden vodič: 1-Wire a UNI/O

Krajní bod osy „počet vodičů" obsadí 1-Wire: **jediný datový vodič a zem**. Bity se kódují **délkou impulzů** (krátký puls = jednička, dlouhý = nula), master udává takt stažením vodiče. Pozoruhodné je **parazitní napájení** — zařízení se napájí z téhož vodiče přes diodu a kondenzátor v době, kdy je vodič v klidu vysoko, takže opravdu stačí dva vodiče pro data i energii. Každé zařízení má **64bitovou ROM adresu**, takže na jednom vodiči může viset víc senzorů a master je adresuje. Je to pomalé (jednotky kbit/s), ale extrémně levné a oblíbené pro teploměry (DS18B20). Podobně **UNI/O** od Microchipu používá jediný vodič pro malé EEPROM.

1-Wire
Sběrnice s jediným datovým vodičem (+ zem), parazitním napájením a 64bitovou ROM adresou; data se kódují délkou impulzů. Pomalá, levná (DS18B20).

I3C: moderní nástupce I2C

I3C (MIPI I3C) je odpověď na to, že I2C stárne, ale jeho dvouvodičová elegance je cenná. Zůstává u **dvou vodičů a zpětné kompatibility** s I2C, ale přidává to, co moderní senzorové systémy potřebují: **vyšší rychlost** (desítky Mbit/s), **in-band přerušení** (zařízení signalizuje přerušení po datovém vodiči, bez zvláštního IRQ pinu), **dynamické přidělování adres** (řadič rozdá adresy za běhu, konec ručního nastavování) a **nižší spotřebu**. Cílí na **sdružování mnoha senzorů** (telefon s desítkami čidel) na několik vodičů — tam, kde by SPI vyžadovalo příliš pinů a I2C by bylo pomalé a bez úsporných režimů.

I3C
MIPI I3C: nástupce I2C zpětně kompatibilní na dvou vodičích; vyšší rychlost, in-band přerušení, dynamické přidělování adres a nižší spotřeba — pro sdružování senzorů.

I2S a PDM: zpět k převodníkům

Audio má vlastní on-board sběrnici, I2S (Inter-IC Sound), a je to přímý most zpět do přednášky 5. Vede **bitové hodiny**, **výběr slova/kanálu** (L/R) a **data**, a slouží k propojení procesoru s **audio kodekem** — tedy externím DAC a ADC. Proud vzorků po I2S je přesně ten druh datového toku, který se odvádí **DMA** v cirkulárním režimu a zpracovává po polovinách (ping-pong z přednášky 5). Příbuzné PDM (Pulse-Density Modulation) je jednobitový proud z **digitálního mikrofonu**: amplitudu kóduje hustota pulzů a procesor (nebo periferie) ho decimuje na vzorky. Tím se kruh uzavírá — sběrnice z této přednášky jsou vodiče, po nichž tečou data z přednášky 5.

I2S
Inter-IC Sound: synchronní sběrnice pro digitální zvuk; vede bitové hodiny, výběr kanálu (L/R) a data — propojuje MCU s audio kodekem (DAC/ADC).

PDM
Pulse-Density Modulation: jednobitový datový tok z digitálního mikrofonu; hustota pulzů kóduje amplitudu, decimace v procesoru dá vzorky.

> **Shrnutí oddílu 6.3.** Minimalistické **1-Wire** (a UNI/O) zvládnou data i napájení jediným vodičem s 64bitovou adresou — levné a pomalé. **I3C** modernizuje I2C na dvou vodičích: rychlost, in-band přerušení, dynamické adresy, nízká spotřeba pro sdružování senzorů. **I2S/ PDM** jsou audio sběrnice, které propojují MCU s externími převodníky a jejich proud se odvádí DMA — přímý most do přednášky 5.

6.4 Field a síťové: diferenciál, vzdálenost, robustnost

Vydáme se z desky na vzdálenost. Field sběrnice stojí na asynchronním primitivu UART; tato část ukáže, jak z něj vznikne fyzická linka (RS-232), proč diferenciální přenos (RS-485) vyhraje na dálku a co obnáší signálová integrita a galvanická izolace.

UART a RS-232: asynchronní primitiv

Než se vydáme na vzdálenost, potřebujeme **asynchronní primitiv**, na němž field sběrnice stojí: UART (Universal Asynchronous Receiver-Transmitter). UART nevede hodiny; každý bajt zabalí mezi **start bit** a **stop bit(y)** a obě strany se musí předem dohodnout na **přenosové rychlosti** (baud). Přijímač start bitem pozná začátek, navzorkuje bity podle dohodnuté rychlosti a stop bitem ověří rámec. UART sám o sobě je jen logický protokol uvnitř čipu (potkali jsme ho jako automat vysílače v přednášce 1); aby z něj byla **fyzická sběrnice na vzdálenost**, musí se doplnit budičem.

UART
Universal Asynchronous Receiver-Transmitter: obvod posílající bajty bez vodiče hodin; rámcuje je start/stop bity a obě strany se dohodnou na přenosové rychlosti (baud).

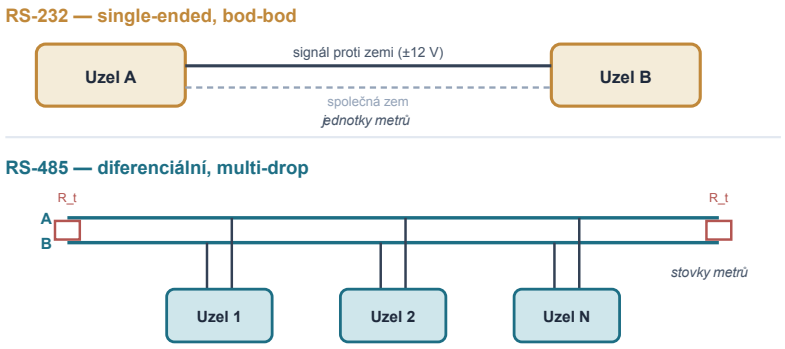
RS-232 přidává k UART **napěťovou fyzickou vrstvu**: logické úrovně se přenášejí jako **single-ended napětí ±5 až ±15 V** proti zemi (záporné = logická 1). Vyšší napětí dává proti TTL větší odstup od šumu, ale signál je stále vztažený k jedné zemi, takže **dosah je krátký** (jednotky metrů) a topologie **bod-bod**. RS-232 je klasické rozhraní starších PC, laboratorních přístrojů a servisních konzolí — pomalé, ale po desetiletí všudypřítomné.

RS-232
Fyzická vrstva pro asynchronní sériovou linku: single-ended napětí ±5 až ±15 V, bod-bod, krátké vzdálenosti; klasické PC a přístrojové rozhraní.

RS-485 a proč diferenciál vyhrává

RS-485 bere tentýž asynchronní rámec a vymění single-ended fyzickou vrstvu za **diferenciální** (osa 3). Signál se přenáší jako **rozdíl dvou vodičů (A, B)**; přijímač měří rozdíl, takže rušení, které zasáhne oba vodiče stejně (common-mode), se **odečte**. Důsledek je dramatický: RS-485 zvládne **stovky metrů**, vysoké rychlosti a hlavně **multi-drop** — na jedné dvojici vodičů visí desítky uzlů (typicky half-duplex, jeden vysílá, ostatní poslouchají). Aby se na dlouhém vedení neodrážely signály, **zakončuje se** odpory na koncích. Po RS-485 běží v průmyslu nejčastěji **Modbus** — jednoduchý protokol typu dotaz-odpověď master-slave.

RS-485
Diferenciální fyzická vrstva pro sériovou linku: multi-drop, stovky metrů, odolná vůči rušení; nese např. Modbus. Half-duplex po dvou vodičích.



Obrázek 6.4. RS-232 (single-ended, bod-bod, krátké) versus RS-485 (diferenciální A/B, multi-drop, stovky metrů, terminace na koncích).

Proč diferenciál vyhrává na dálku

Stojí za to chvíli u diferenciálu zůstat, protože je to klíčová myšlenka celé field rodiny. Single-ended signál nese informaci jako napětí proti zemi — jenže **zem dvou vzdálených uzlů není stejná** (zemní smyčky, úbytky) a každé rušení naindukované do vodiče se sečte se signálem. Diferenciální pár nese informaci jako **rozdíl** dvou blízko vedených vodičů: posun společné úrovně i naindukované rušení postihnou **oba vodiče stejně** a v rozdílu zmizí (potlačení common-mode). Proto prakticky všechny sběrnice určené na vzdálenost a do hluku — RS-485, CAN, USB, ethernet — používají diferenciální páry.

Signálová integrita a galvanická izolace

Na krátké desce je vodič prostý „drát", ale na metrech se z něj stává **vedení** s charakteristickou impedancí Z0 (u krouceného páru zhruba 120 Ω). Nekončí-li vedení právě touto impedancí, hrana signálu se na jeho konci **odrazí** zpět a zkreslí následující bity. Proto se sběrnice na dálku **zakončují**: na **oba konce** se dá odpor rovný Z0 (u RS-485 i CAN 120 Ω), který energii hrany pohltí. S tím souvisí i topologie — odbočky (stubs) od páteřního vedení musí být mnohem kratší než náběžná hrana, jinak samy odrážejí, a proto se sběrnice vede jako **daisy-chain**, ne do hvězdy. Čím rychlejší hrana a delší vedení, tím přísnější meze; právě proto se rychlost a dosah na ose 3 vzájemně vylučují a přestává platit zjednodušení „vodič je okamžitý zkrat".

Druhým problémem vzdálených uzlů je, že **nesdílejí stejnou zem**: rozdíl potenciálů nebo zemní smyčka může přesáhnout rozsah vysílače a zničit ho. Řešením je **galvanická izolace** — přerušení vodivého spojení, přes které se přenáší jen informace. Klasickým prvkem je **optočlen** (signál přeskočí světlem přes optickou bariéru), moderním **digitální izolátor** (kapacitní nebo magnetická vazba, rychlejší a integrovaný). Izolované transceivery (isolated CAN, RS-485) mají bariéru přímo v pouzdře a napájení oddělené strany dodá izolovaný měnič DC-DC. Izolace je povinná v medicíně, průmyslu a všude, kde se zemní potenciály liší o stovky voltů; je to tentýž princip jako level shifting u I2C, jen o řád přísnější — místo úrovní se odděluje celá galvanická reference.

Zakončují
Připojení odporu rovného charakteristické impedanci vedení na jeho konce, aby energie signálové hrany byla pohlcena a neodrazila se zpět. U RS-485 a CAN jsou to typicky odpory 120 Ω na obou koncích sběrnice.

Galvanická izolace
Přenos signálu mezi dvěma obvody bez vodivého spojení jejich zemí. Realizuje se optočlenem (přenos světlem) nebo digitálním izolátorem (kapacitní či magnetickou vazbou); izolované transceivery mají bariéru v pouzdře a napájení druhé strany dodá izolovaný měnič. Odděluje zemní domény, čímž chrání obvod i obsluhu.

> **Shrnutí oddílu 6.4.** Field sběrnice stojí na asynchronním primitivu **UART** (rámec start/stop, dohodnutý baud). **RS-232** mu dá single-ended fyzickou vrstvu ±12 V, bod-bod, krátkou. **RS-485** ji vymění za **diferenciální** (rozdíl A/B), čímž získá stovky metrů, odolnost a **multi-drop** (Modbus). Diferenciál vyhrává, protože **potlačí common-mode rušení**. Na dálku je vodič **vedení**: zakončuje se odporem Z0 (120 Ω) na obou koncích (jinak odrazy) a vede jako daisy-chain. **Galvanická izolace** (optočlen, digitální izolátor, izolovaný transceiver) odděluje zemní domény tam, kde se liší potenciály.

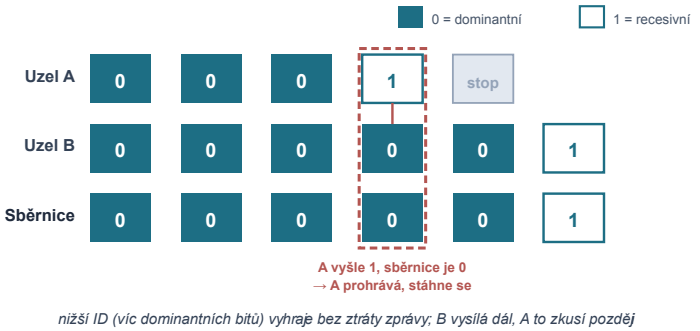
6.5 Zprávově orientované a aplikační: CAN, LIN, DALI

Na vrcholu field sběrnic stojí zprávově orientovaný model, kde se neadresují uzly, ale data. Tato část představí CAN s jeho nedestruktivní arbitráží, levnou pod-sběrnici LIN a doménovou sběrnici DALI pro osvětlení, a naznačí, kam krajina vede dál.

CAN: arbitráž bez kolize

CAN (Controller Area Network) je vrchol osy 4 (multi-master / zprávově orientovaný) a nejdůležitější field sběrnice vůbec. Tři myšlenky ji dělají výjimečnou. Za prvé je **zprávově orientovaná**: nevysílá se „uzlu číslo 5", ale **zpráva s identifikátorem** (např. „otáčky motoru"), a každý uzel přijme zprávy, které jsou pro něj relevantní — adresují se *data*, ne uzly. Za druhé řeší souběh **nedestruktivní arbitráží**: vodič je diferenciální s **dominantní (0) a recesivní (1) úrovní**; když dva uzly vysílají naráz, ten, který pošle dominantní bit proti recesivnímu, **přepíše** sběrnici a druhý uzel přestane vysílat — ale **bez ztráty zprávy**, vysílání jen odloží. Protože nižší ID = víc dominantních bitů na začátku, **nejprioritnější zpráva vždy projde bez zpoždění**. To je za třetí důvod jejího **determinismu**: na CAN lze garantovat, že kritická zpráva (brzdy) má přednost. CAN je extrémně **robustní** (CRC, potvrzování, automatické opakování) a jeho rychlejší varianta **CAN-FD** zvyšuje propustnost i délku rámce. Proto je páteří automobilů a průmyslové automatizace.

CAN
Controller Area Network: diferenciální multi-master sběrnice, kde se vysílají zprávy s identifikátorem (ne adresy uzlů); arbitráž podle priority ID probíhá bez ztráty zprávy. Páteř automobilů a průmyslu.



Obrázek 6.5. CAN arbitráž: dva uzly vysílají naráz, dominantní bit (0) přepíše recesivní (1); uzel s nižším ID získá sběrnici bez ztráty zprávy, druhý uzel vysílání odloží.

LIN a DALI: levná a doménová sběrnice

Ne všechno v autě potřebuje robustnost a cenu CAN. Pro pomalé, nekritické funkce (okénka, zrcátka, světla v kabině) slouží LIN (Local Interconnect Network) — **levná jednovodičová master-slave** sběrnice postavená na UART. Jeden master vysílá dotazy, podřízené uzly odpovídají v pevném plánu; žádná arbitráž, žádný diferenciál, minimální cena. LIN se typicky zapojuje jako **pod-sběrnice pod CAN**: brána (gateway) překládá mezi rychlým páteřním CAN a levnými LIN větvemi. Je to ukázka, že i uvnitř jednoho zařízení se kombinuje víc sběrnic podle toho, kde na osách leží daná funkce.

LIN
Local Interconnect Network: levná jednovodičová master-slave sběrnice postavená na UART; podřízená pod-sběrnice pod CAN pro nekritické funkce v automobilu.

DALI: aplikačně specifická

DALI (Digital Addressable Lighting Interface) ukazuje druhý extrém — sběrnici **šitou na jednu aplikaci**, řízení osvětlení. Dva vodiče, **Manchester kódování** (každý bit má hranu, takže nese i hodiny), pomalá (1200 bit/s), half-duplex, s adresami svítidel a skupin. Pomalost nevadí — světla se nestmívají megabitovou rychlostí — a za to nabízí odolnost, jednoduchou instalaci a standardizované příkazy (úroveň, scéna, skupina). DALI je připomínka, že vedle „univerzálních" sběrnic existují **doménové standardy**, které vyladí kompromis přesně pro svůj obor.

DALI
Digital Addressable Lighting Interface: pomalá dvouvodičová sběrnice pro adresovatelné řízení osvětlení; Manchester kódování, half-duplex.

Krajina, kterou jen pojmenujeme

Field svět je široký; pro úplnost zmiňme, kam vede dál, aniž bychom zabíhali do detailů. **FlexRay** přidává k determinismu CAN **časově řízené sloty** (time-triggered) pro nejkritičtější automobilové funkce. **Automotive Ethernet** (100/1000BASE-T1) přináší do auta vysokou propustnost po jediném páru pro kamery a páteřní sítě. **EtherCAT** a **PROFINET** jsou průmyslové real-time varianty ethernetu pro řízení strojů. **SENT** a **PSI5** jsou levná jednosměrná rozhraní automobilových senzorů. Všechny zapadají do týchž sedmi os — jen posunuté k vyšší propustnosti nebo přísnějšímu determinismu.

> **Shrnutí oddílu 6.5. CAN** je zprávově orientovaná diferenciální sběrnice s nedestruktivní arbitráží podle priority ID — robustní a deterministická, páteř aut a průmyslu (CAN-FD rychlejší). **LIN** je levná UART-based pod-sběrnice pod CAN pro nekritické funkce. **DALI** je doménová sběrnice pro osvětlení (Manchester, pomalá, odolná). Dál vedou FlexRay, automotive Ethernet, EtherCAT/PROFINET, SENT/PSI5 — tytéž osy, posunuté.

6.6 Návrh sériového protokolu

Sběrnice z předchozích částí přenášejí **data**, ale aplikace potřebuje **zprávy**: ohraničené, celistvé a ověřené jednotky s významem. Mezi „datový proud po UARTu" a „přišel platný příkaz nastav PWM na 40 %" leží **protokol** — dohodnutá pravidla, jak data seskládat do rámců, ověřit jejich neporušenost, adresovat je a bezpečně rozbalit. Tato část shrnuje, jak takový protokol navrhnout; hlubší rozbor (serializační schémata, robustní protokoly jako CSP, bezpečná deserializace) je v **Příloze C**.

Protokol se přirozeně dělí na vrstvy nad fyzickou sběrnicí: **linková** vrstva dělá z datového proudu rámce a ověřuje jejich integritu, **aplikační** vrstva dává rámcům význam (typy zpráv, příkazy, odpovědi). Tomuto dělení odpovídá i členění návrhu.

Rámec zprávy

Přijímač vidí jen nepřetržitý datový proud a musí v něm poznat, **kde zpráva začíná a končí** — to je **rámcování**. Používají se dvě základní techniky:

- **Oddělovací znak** (delimiter) ohraničí zprávu vyhrazeným bytem (například `0x7E`). Je to jednoduché, ale ten byte se nesmí objevit v datech; buď se **uniká** (byte stuffing), nebo se použije bezúnikové kódování **COBS** (Consistent Overhead Byte Stuffing), které z rámce odstraní všechny nuly s pevnou režií.
- **Délkový prefix** nese v hlavičce počet bytů datové části; přijímač jich pak přesně tolik načte. Je to efektivní, ale ztráta synchronizace (jediný ztracený byte) rozhodí všechny další rámce, dokud se přijímač neresynchronizuje.

Rámcování
Vymezení hranic zprávy v nepřetržitém datového proudu. Realizuje se oddělovacím znakem (a únikem/stuffingem uvnitř dat), nebo délkovým polem v hlavičce.

COBS
Consistent Overhead Byte Stuffing: kódování, které z rámce odstraní všechny výskyty oddělovacího bytu (nuly) s pevnou, malou režií (1 bajt na 254 dat). Umožní spolehlivou resynchronizaci na hranicích rámců.

Typický rámec skládá obojí do pevné struktury:

Pole	Význam
SOF	úvodní značka (start of frame) pro synchronizaci
LEN	délka datové části
TYPE / ID	typ zprávy nebo adresa uzlu
PAYLOAD	užitečná data (LEN bytů)
CRC	kontrolní součet přes hlavičku a data

Tuto strukturu poznáváme u CAN i u aplikačních protokolů nad UARTem; liší se detaily, ne principem.

Integrita a spolehlivost

Sériová linka není bezchybná — rušení překlopí bit, ztráta synchronizace zahodí byte. **Integritu** rámce ověří **CRC** (cyklický redundantní součet): odesílatel jej spočítá přes rámec a přiloží, přijímač jej přepočítá a porovná. CRC odhalí shluky bitových chyb mnohem spolehlivěji než prostý součet bytů (checksum). Chybu ovšem jen **detekuje**; opravu bez opakování by zajistil až samoopravný kód (FEC), který se na krátké linky obvykle nevyplatí.

CRC
Cyclic Redundancy Check: kontrolní součet počítaný polynomiálním dělením, který spolehlivě odhalí shluky bitových chyb. Je robustnější než prostý součet bytů a většina MCU má na jeho výpočet hardwarovou jednotku.

Detekovaná chyba se řeší **opakováním**. Spolehlivý protokol proto přidává:

- **potvrzení** (ACK/NACK) — přijímač správný rámec potvrdí, chybný odmítne,
- **sekvenční čísla** — očíslují rámce, takže se pozná ztráta i **duplikát** (po zbytečném opakování),
- **timeout a retransmise** — nepřijde-li ACK včas, odesílatel rámec pošle znovu (návaznost na timeouty z přednášky 9).

ACK
Kladné potvrzení příjmu rámce. Nepřijde-li do vypršení časovače (timeout), odesílatel rámec zopakuje. Spolu se sekvenčními čísly zajišťuje spolehlivé doručení.

Tuto trojici — potvrzení, pořadí, opakování — jsme v hardwaru viděli u CAN; v protokolu nad UARTem či RS-485 si ji musíme naimplementovat sami.

Marshalling: jak zabalit data

Datová část nese strukturovaná data — čísla, příznaky, pole. Jejich převodu na posloupnost bytů a zpět se říká **marshalling** (serializace). Podstatné je, že odesílatel a příjemce mohou být **různé architektury**, takže se nesmí spoléhat na to, jak si data ukládá zrovna jeden procesor:

Marshalling
Serializace: převod strukturovaných dat (čísel, struktur) na plochou posloupnost bytů pro přenos a zpět. Musí být nezávislý na architektuře odesílatele i příjemce — pevná šířka typů a dohodnuté pořadí bytů.

- **Endianita.** Vícebajtové číslo se musí přenášet v dohodnutém pořadí bytů; zvykem je **network byte order** (big-endian). Ledabylé `memcpy` struktury odešle byty v pořadí odesílatele a na jiné architektuře se přečtou špatně.
- **Pevná šířka a zarovnání.** V rámci se používají typy s pevnou šířkou (`uint16_t`, `int32_t` z přednášky 11), ne `int`, jehož šířka není standardem daná. Přímé posílání C struktury je křehké kvůli **zarovnání** (padding mezi členy); `__attribute__((packed))` sice padding zruší, ale zavádí nezarovnané přístupy, proto se pole raději skládají do bufferu explicitně, byte po bytu.
- **TLV a rozšiřitelnost.** Formát **TLV** (Type-Length-Value) zapíše každou položku jako *typ, délka, hodnota*; neznámou položku umí příjemce podle délky přeskočit, takže protokol lze rozšiřovat zpětně kompatibilně.

TLV
Type-Length-Value: kódování položky jako trojice typ, délka, hodnota. Neznámou položku umí příjemce podle délky přeskočit, takže formát lze rozšiřovat, aniž se rozbijí starší implementace.

Ruční skládání se vyplatí u malých pevných rámců; u složitějších zpráv se používají **serializační schémata** (Protocol Buffers/nanopb, CBOR), která z popisu vygenerují kód — rozebírá je Příloha C.

Bezpečná deserializace

Rozbalení přijatého rámce je práce s **nedůvěryhodným vstupem** — může přijít od kohokoli a být poškozený nebo záměrně škodlivý. Platí proto pravidla z přednášky 13:

- **nikdy nedůvěřovat délkovému poli** — než se podle `LEN` cokoli zkopíruje, je nutné ověřit, že se to vejde do cílového bufferu; jinak vznikne přetečení,
- **kontrolovat meze u každého pole** — chybějící kontrola je nejčastější bezpečnostní díra,
- **ověřit CRC dřív, než se data použijí** — poškozený rámec se nesmí interpretovat.

Deserializace bez kontroly mezí je učebnicová cesta k přetečení bufferu; robustní protokol validuje **před** zpracováním.

Parser stavovým automatem

Rámec se z datového proudu skládá nejlépe **konečným automatem** (přednáška 1) — každý přijatý byte posune automat o krok, aniž by se blokovalo čekáním. Přijímač tak zvládne i ztrátu synchronizace: při chybě se vrátí do výchozího stavu a čeká na další SOF.

Konečným automatem
Parser rámce řízený stavem: každý přijatý byte posune automat mezi stavy (čekání na SOF, čtení délky, čtení dat, kontrola CRC). Je robustní vůči ztrátě synchronizace a vhodný pro obsluhu přerušení.

```c
enum { WAIT_SOF, LEN, DATA, CHECK } st = WAIT_SOF;

void rx_byte(uint8_t b) {              // volá se z ISR na každý přijatý byte
    switch (st) {
    case WAIT_SOF: if (b == SOF) { idx = 0; st = LEN; }            break;
    case LEN:      len = b; st = (len <= MAX) ? DATA : WAIT_SOF;    break;  // hlídej mez!
    case DATA:     buf[idx++] = b; if (idx == len) st = CHECK;      break;
    case CHECK:    if (crc_ok(buf, len, b)) deliver(buf, len);
                   st = WAIT_SOF;                                   break;
    }
}
```

Tentýž automat, který jsme v přednášce 1 kreslili jako řadič, tu rozpoznává strukturu v proudu

dat — přijímač je řízen událostí (příchodem bytu), krátce a bez blokování, přesně jako obsluha přerušení z přednášky 2.

Shrnutí oddílu 6.6. Nad fyzickou sběrnicí stojí **protokol**, který z datového proudu dělá ověřené zprávy. **Rámcování** vymezí hranice (oddělovač s COBS, nebo délkový prefix); **CRC** ověří integritu; **ACK, sekvenční čísla a retransmise** zajistí spolehlivost (jako u CAN, timeouty z přednášky 9). **Marshalling** zabalí data nezávisle na architektuře (network byte order, pevná šířka, TLV); **bezpečná deserializace** validuje meze a CRC před použitím (přednáška 13). Rámec se rozpoznává **konečným automatem** (přednáška 1). Hloubka — serializační schémata a robustní protokoly (CSP) — je v Příloze C.

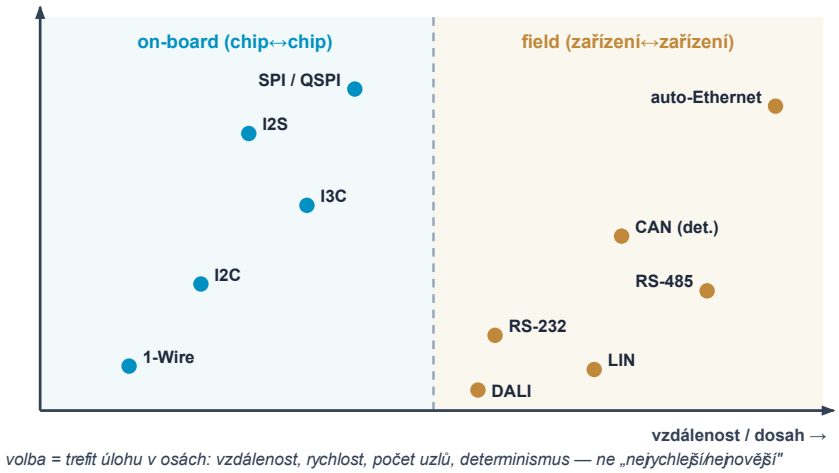
6.7 Syntéza: výběr sběrnice a její vlastnictví

Zbývá z probraných os udělat schopnost vybrat. Tato závěrečná část dá rozhodovací návod, ukáže, že sdílená sběrnice je sdílený zdroj vyžadující vlastnictví, a uzavře oblouk zpět k datovému toku a DMA.

Rozhodovací návod podle os

Smyslem přednášky nebyl katalog, ale schopnost **vybrat**. Volba sběrnice je posloupnost otázek podél sedmi os:

- **Jak daleko a do jakého prostředí?** Centimetry na desce → on-board (single-ended). Metry, rušení, izolace → field (diferenciál).
- **Kolik uzlů a kdo řídí přenos?** Jeden master, několik zařízení → SPI/I2C. Síť rovnocenných uzlů → CAN/RS-485.
- **Jak rychle?** Desítky Mbit/s, full-duplex → SPI/QSPI/I2S. Pomalu a úsporně → I2C/1-Wire/LIN.
- **Kolik vodičů a pinů lze obětovat?** Minimum → 1-Wire/I2C/I3C. Bez omezení → SPI.
- **Musí to být zaručeně včas?** Ano (řízení) → CAN/FlexRay (determinismus). Ne → cokoli.



Obrázek 6.6. Rozhodovací mapa sběrnic podle os: vzdálenost (on-board×field), rychlost, počet uzlů a determinismus rozdělují SPI, I2C, I3C, 1-Wire, RS-485, CAN, LIN, DALI.

Žádná z os není „lepší" — jsou to **kompromisy**, a dobrý návrh je ten, který trefí požadavky úlohy, ne ten, který použije nejrychlejší nebo nejmodernější sběrnici.

Sdílený zdroj a datový tok: vlastnictví a DMA

Jakmile na jedné sběrnici visí víc zařízení a přistupuje k ní víc úloh, vrací se téma **vlastnictví** z přednášky 5. Sběrnice je **sdílený zdroj**: dvě úlohy nesmí zahájit transakci na témže I2C zároveň, jinak si navzájem poškodí rámce. Proto se přístup k sběrnici **vlastní** — typicky zámkem (mutex), který si úloha vezme na dobu transakce a pak ho předá dál. Je to tentýž invariant jako u DMA bufferu („v daném okamžiku jeden vlastník"), jen na úrovni periferie místo paměti — a v plné podobě ho formalizuje **RTOS** (mutexy, fronty), kam tato linka míří. Zprávově orientovaný CAN jde ještě dál: vlastnictví sběrnice řeší **sám hardware** arbitráží, takže software souběh neřeší — jiný bod na téže ose návrhu.

Sdílený zdroj
Prostředek, který používá víc úloh a v daném okamžiku smí být v jedněch rukou; vyžaduje řízení vlastnictví, aby se transakce nepřekrývaly.

Sběrnice a datový tok: zpět k DMA

A poslední oblouk zpět do přednášky 5: rychlé sběrnicové proudy se **streamují přes DMA** úplně stejně jako vnitřní převodníky. Externí ADC po SPI, audio kodek po I2S, příjem po UART — všude periferie po každém slově **vystaví žádost o přenos DMA**, kanál plní cirkulární buffer a procesor zpracovává poloviny na HT/TC. Sběrnice z této přednášky jsou tedy **vstupní a výstupní brány** datového toku z přednášky 5; ring a ping-pong buffer, vlastnictví i koherence cache platí pro proud po SPI nebo I2S stejně jako pro vnitřní ADC.

Závěr

Sběrnic je mnoho ze stejného důvodu jako architektur procesorů: **požadavky si odporují a každá sběrnice je kompromis** umístěný do prostoru sedmi os. Kdo ty osy ovládá, nepotřebuje memorovat pinouty — pozná, proč SPI sedí na rychlý převodník a I2C na pole senzorů, proč na deseti metrech v hluku vyhraje RS-485 a v autě CAN, a proč i jedno zařízení uvnitř kombinuje víc sběrnic. Procesor jsme naučili počítat (1–3), pochopili jsme jeho paměťové prostředí (4) a datový tok (5); teď umí **komunikovat navenek** — a víme, jakými vodiči a proč právě jimi.

> **Shrnutí oddílu 6.7.** Volba sběrnice je posloupnost otázek podél sedmi os (vzdálenost, uzly, rychlost, vodiče, determinismus) — žádná není absolutně lepší, jde o trefit úlohu. Sdílená sběrnice je **sdílený zdroj** → vlastnictví (mutex; u CAN řeší arbitráž hardware) → most do RTOS. Rychlé sběrnicové proudy se **streamují přes DMA** stejně jako vnitřní převodníky — most zpět do přednášky 5.

6.8 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 6.1 — Kritéria výběru sériové sběrnice

1. Proč nemůže existovat jediná univerzální sběrnice?
2. Vysvětlete rozdíl synchronní/asynchronní a co každá platí.
3. Jak diferenciální signalizace získává odolnost a dosah?
4. Čím se liší svět on-board a field sběrnic?

K oddílu 6.2 — On-board synchronní sběrnice: SPI a I2C

1. Jak SPI vybírá, s kým mluví, a proč to neškáluje počtem zařízení?
2. Proč je SDA/SCL u I2C open-drain a co to umožní?
3. Jakými mechanismy I2C zajistí spolehlivý přenos i mezi různě rychlými zařízeními (role clock stretchingu a potvrzení ACK)?
4. Podle čeho se mezi SPI a I2C rozhodnete?

K oddílu 6.3 — On-board minimalistické a moderní: 1-Wire, I3C, I2S

1. Jak 1-Wire přenese po jediném vodiči data i napájení?
2. Co I3C přidává k I2C a proč je to cenné pro senzorové systémy?
3. Proč je I2S přirozeně spjato s DMA a převodníky z přednášky 5?

K oddílu 6.4 — Field a síťové: diferenciál, vzdálenost, robustnost

1. Co je asynchronní rámec UART a proč se obě strany musí dohodnout na baudu?
2. Čím se liší fyzická vrstva RS-232 a RS-485 a jaký to má důsledek pro dosah a topologii?
3. Proč diferenciální přenos potlačí rušení?
4. Proč se vedení zakončuje impedancí Z0 na obou koncích a jak s tím souvisí délka odboček a volba topologie?
5. Jaký problém řeší galvanická izolace a kdy je nutná? Jak souvisí s posunem napěťových úrovní (level shifting) u I2C?

K oddílu 6.5 — Zprávově orientované a aplikační: CAN, LIN, DALI

1. Co znamená, že je CAN „zprávově orientovaný", a jak se liší od adresování uzlů?
2. Jak funguje nedestruktivní arbitráž na sběrnici CAN a proč zaručí determinismus (přednost nejprioritnější zprávy)?
3. Proč se LIN zapojuje jako pod-sběrnice pod CAN?
4. Co je na DALI „doménového" a proč mu pomalost nevadí?

K oddílu 6.6 — Návrh sériového protokolu

1. Proč syrový přenos bytů po sběrnici nestačí a co přidává protokol nad ní?
2. Porovnejte rámcování oddělovačem a délkovým prefixem; k čemu slouží COBS?
3. Jakou roli hraje CRC a jak se spolu s ACK, sekvenčními čísly a retransmisí zajistí spolehlivé doručení?
4. Co je marshalling a proč se nesmí posílat C struktura přímo přes `memcpy`?
5. Proč je deserializace nedůvěryhodného vstupu bezpečnostní problém a jakými kontrolami (LEN, meze polí, CRC před použitím) se ošetří?
6. Jak konečný automat rozpozná rámec v datového proudu a proč je robustní vůči ztrátě synchronizace?

7

Architektury procesorů a assembler

> Tato kapitola odpovídá na otázku **proč dnes vedle sebe koexistují ARM, x86, RISC-V, GPU i AI akcelerátory** místo jedné univerzální architektury. Nejde o přehled instrukčních sad, ale o způsob uvažování: **jaké architektonické rozhodnutí vede k jakému kompromisu** ve výkonu, spotřebě, složitosti a programovatelnosti.

Obsah kapitoly

7.1 ISA a mikroarchitektura
7.2 RISC versus CISC: válka, která skončila smírem
7.3 Architektura ARM: od skromných začátků k vašemu Macu
7.4 Programování v ARM assembleru
7.5 RISC-V: jiná sada kompromisů
7.6 Syntéza: proč architektury koexistují
7.7 Kontrolní otázky

7.1 ISA a mikroarchitektura

Celá přednáška stojí na jednom rozlišení: instrukční sada určuje, *co* procesor dělá, mikroarchitektura *jak*. Tato část ho vysvětlí, ukáže překvapivě univerzální kostru moderních jader a projde zřetězení, hazardy, predikci větvení a to, jak se jde za hranici jedné instrukce za takt.

Instrukční sada jako kontrakt

Nejdůležitější rozlišení celé přednášky se vejde do jedné věty: **instrukční sada určuje *co* procesor dělá, mikroarchitektura *jak* to dělá.**

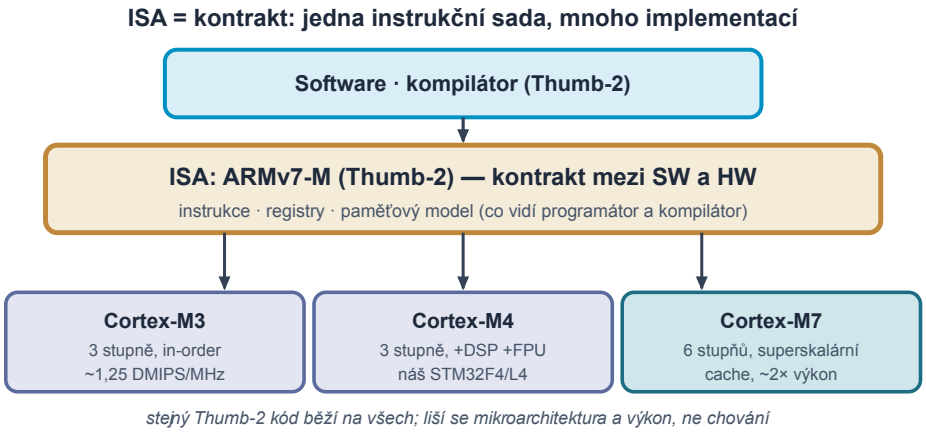
ISA (Instruction Set Architecture) je *kontrakt* mezi softwarem a hardwarem — přesně to, co vidí programátor a překladač. Definuje:

- **jaké instrukce** existují a co dělají (sémantika),
- **kolik registrů** je k dispozici a jak se jmenují,
- **jak funguje paměť** (adresování, zarovnání, endianita),
- **formát instrukcí** (jak jsou zakódované do bitů).

ISA
Instruction Set Architecture: kontrakt SW/HW — instrukce, registry, paměťový model, formát.

Mikroarchitektura je naproti tomu *konkrétní implementace* tohoto kontraktu v křemíku: kolik má pipeline stupňů, zda provádí instrukce mimo pořadí (out-of-order), jaké má cache, kolik má tranzistorů. Dobrá analogie je **API a jeho implementace**: API (ISA) určuje, co funkce dělají; implementací (mikroarchitektur) může být mnoho, a navzájem se liší výkonem, ne

Mikroarchitektura
Konkrétní implementace ISA v křemíku: pipeline, cache, počet jednotek — určuje výkon.

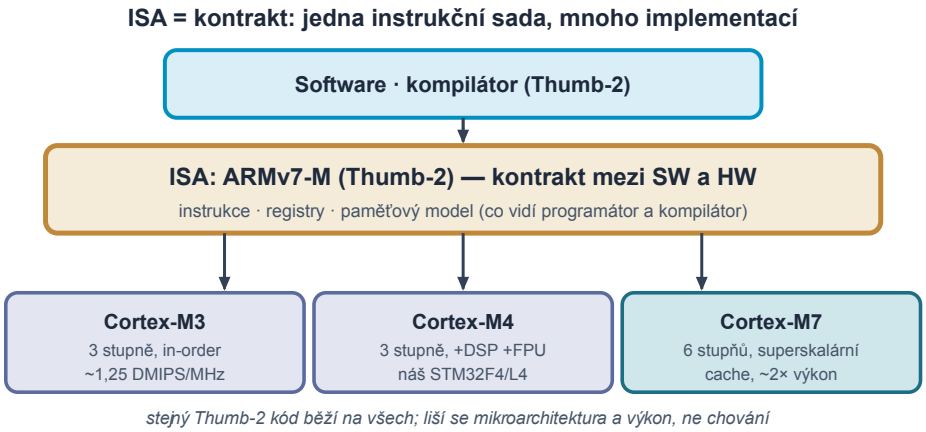


Obrázek 7.1. ISA jako kontrakt: jedna instrukční sada, mnoho implementací

chováním.

Jedna ISA, mnoho implementací

Sílu tohoto rozdělení nejlépe ukáže příklad z našeho světa. Cortex-M4 a Cortex-M7 sdílejí **tutéž instrukční sadu** ARMv7E-M (Thumb-2) — a přesto:



Obrázek 7.2. ISA jako kontrakt: táž ISA, tři implementace M3/M4/M7

	Cortex-M4	Cortex-M7
ISA	ARMv7E-M (Thumb-2)	**tatáž**
pipeline	3 stupně, in-order	6 stupnů, superskalární
cache	ne	I + D cache
výkon	~1,25 DMIPS/MHz	~2,14 DMIPS/MHz

Mezi nimi je zhruba dvojnásobek výkonu — daný čistě mikroarchitekturou (delší pipeline, superskalární vydávání, cache) — a **týž přeložený Thumb-2 kód běží na obou bez překompilování**. To je celá pointa ISA: je to stabilní rozhraní, za kterým se hardware může libovolně proměňovat. Právě proto může Intel každý rok vydat nový procesor, aniž by si svět musel přepsat programy.

Univerzální mikroarchitektura

Zajímavé je, že navzdory různým ISA mají všechny moderní výkonné procesory **překvapivě podobnou kostru**. Instrukce protékají toutéž posloupností fází:

1. **Instruction Fetch** — načtení instrukce z I-cache, prvotní dekódování, **predikce větvení**;
2. **Decode** — překlad instrukce na vnitřní mikrooperace, čtení registrů, případně přejmenování registrů a *reorder buffer* (pro běh mimo pořadí);
3. **Execute** — výpočetní jednotky: několik ALU, FPU (plovoucí čárka), LSU (load-store), větvení;
4. **Memory Access** — D-cache, TLB (překlad virtuálních adres), prefetcher;
5. **Write Back** — zápis výsledku zpět do registrů, *commit/retire*.

Tato kostra je výsledkem desetiletí konvergence: bez ohledu na to, zda je procesor „RISC" nebo „CISC", uvnitř vypadá podobně. Rozdíl je hlavně v tom, **co do této linky vstupuje** — tedy v ISA.

Tři instrukční sady dneška a proč na ISA záleží

Tři ISA dnes dominují — a dobře ilustrují různá filozofická rozhodnutí (tab. 7.1):

Tabulka 7.1. Tři dnešní instrukční sady

	ARM (ARMv7)	x86-64	RISC-V (RV32I)
formát instrukce	32 b pevný (16/32 Thumb-2)	1–15 B proměnný	32 b pevný (16/32 s RVC)
počet registrů	16 GP	16 GP	32 GP (+ `x0`=0)
počet instrukcí	~200 + rozšíření	3000+	47 base + moduly
typická sémantika	podmíněné vykonávání	operace paměť'–paměť'	striktní load-store, `x0`=0

RISC-V je zde nováčkem, který stojí za pozornost: je to **otevřená, modulární** ISA (bez licenčních poplatků), kterou si může implementovat kdokoli. Tam, kde ARM i x86 jsou proprietární, RISC-V sází na minimální základ (47 instrukcí) a volitelná rozšíření — a rychle se prosazuje od mikrokontrolérů po výzkumné superpočítače.

Proč na ISA záleží

ISA není akademický detail — určuje tři praktické věci:

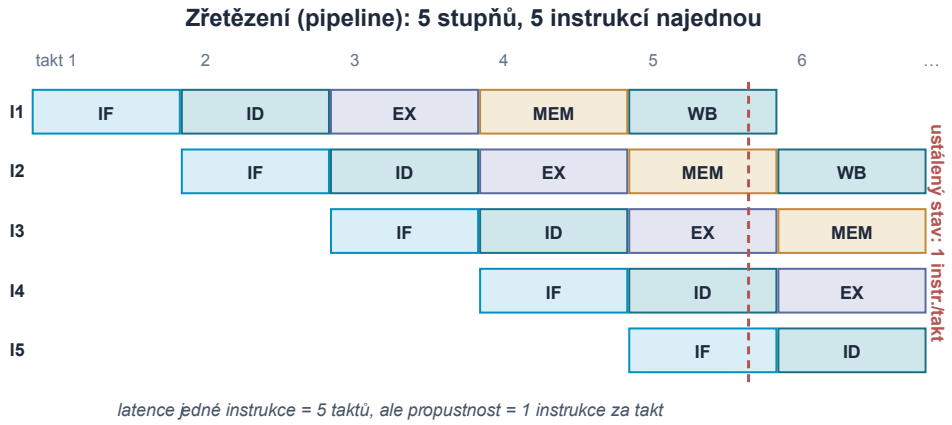
- **kompatibilita softwaru** — binárka pro ARM nepoběží na x86; je nutné překompilovat (nebo pomalu emulovat). Proto je přechod na novou ISA velká věc (a proto Apple u přechodu na ARM dodal překladovou vrstvu Rosetta);
- **práce kompilátoru** — překladač musí ISA znát do detailu; RISC nechává víc práce na kompilátoru, CISC na hardwaru (část 7.2);
- **výkon a spotřeba** — jednoduchá ISA → jednodušší hardware, nižší spotřeba, vyšší možná frekvence (ale více instrukcí na úlohu); složitá ISA → hustší kód, ale složitější a energeticky náročnější hardware.

Poslední bod je přesně to, co rozhodlo souboj v mobilech a vestavných systémech ve prospěch ARM — a k němu se ted' dostaneme přes filozofii, ze které ARM vzešel.

Zřetězení a hazardy

Hlavní technika, jak procesor zrychlit, je **zřetězení**: rozdělit zpracování instrukce na stupně a nechat je běžet **překrytě**, jako montážní linku. Klasická RISC pipeline má pět stupňů — Fetch, Decode, Execute, Memory, Write Back — a zatímco jedna instrukce počítá v EX, další se dekóduje a třetí se načítá:

Zřetězení
Pipeline: instrukce se zpracovávají překrytě po stupních; propustnost až 1 instrukce za takt.



Obrázek 7.3. Zřetězení: pět stupňů, pět instrukcí najednou

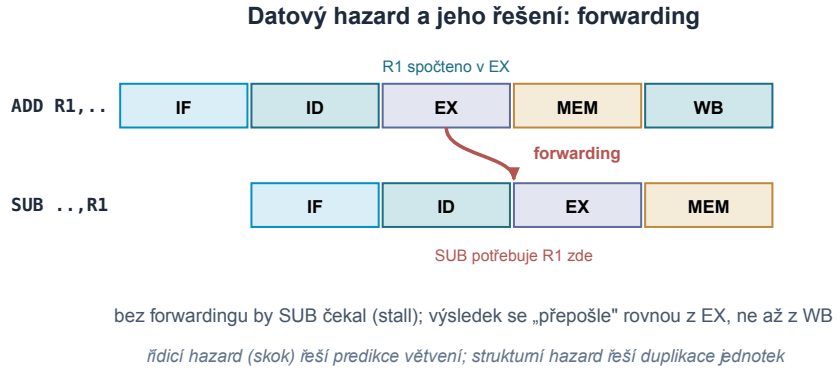
Latence *jedné* instrukce zůstává pět taktů, ale **propustnost** v ustáleném stavu je **jedna instrukce za takt** — pětinásobné zrychlení. Právě proto RISC těží z pevné délky a jednotných instrukcí: aby každý stupeň trval podobně dlouho a linka nikde nezadrhávala. Hloubka pipeline je návrhový kompromis: hlubší pipeline umožní vyšší frekvenci, ale dražší chybnou predikci. Cortex-M3 má 3 stupně, výkonné Cortex-A i 15 a více.

Hazardy a jejich řešení

Idealizovaná linka „jedna instrukce za takt" naráží na **hazardy** — situace, kdy instrukce nemůže hladce pokračovat. Jsou tři druhy:

Hazardy
Situace, kdy instrukce nemůže hladce projít pipeline (datový, řídicí, strukturní).

- **datový hazard** — instrukce potřebuje výsledek, který předchozí ještě nedopočítala;
- **řídicí hazard** — skok změní tok, ale pipeline už spekulativně načetla instrukce, které se neměly vykonat;
- **strukturní hazard** — dvě instrukce chtějí v témž taktu tutéž jednotku.



Obrázek 7.4. Datový hazard a forwarding

Forwardingem
Forwarding: výsledek se přepošle rovnou z EX další instrukci, bez čekání na zápis do registru.

Datový hazard se řeší **forwardingem**: výsledek se přepošle rovnou z výstupu EX zpět na vstup

další instrukce, aniž by se čekalo na zápis do registru ve fázi WB. Když ani forwarding nestačí (např. po `load`), vloží se **stall** (bublina) — nebo, lépe, kompilátor instrukce přeskupí tak, aby mezeru vyplnila nezávislá práce (instruction scheduling, část 7.2). Řídicí hazard řeší **predikce větvení** (níže), strukturní **duplikace jednotek** (víc ALU, oddělené I/D cache — harvardská architektura).

Predikce větvení
Procesor predikuje výsledek skoku a spekulativně pokračuje; chyba vyprázdní pipeline.

Predikce větvení a podmíněné vykonávání

Skoky tvoří kolem 20 % instrukcí; bez ošetření by každý vyprázdnil pipeline. Procesor proto **predikuje** výsledek skoku a spekulativně pokračuje. Predikce může být statická (jednoduché pravidlo — zpětný skok bývá smyčka, tedy „skočí se") nebo dynamická (tabulka historie skoků s dvoubitovými čítači), s úspěšností přes 95 %. Cena chyby roste s hloubkou pipeline.

A právě zde má smysl jedno z nejstarších rozhodnutí ARM: **podmíněné vykonávání**. Místo krátkého skoku přes několik instrukcí je ARM umí vykonat *podmíněně* (`ADDGT`, `MOVLE`) — pipeline pak nemůže chybně predikovat, protože žádný skok nenastal. U malých jader je to elegantní a levné; u širokých out-of-order jader naopak překáží, a proto je 64bitový ARMv8 zase odstranil.

Memory wall a cache

Mezi registry a hlavní pamětí je propast latence **100–1000**×, která se s léty jen prohlubuje („memory wall"). Bez protiopatření by procesor většinu času jen čekal. Řešením je **cache** — rychlá kopie nedávno a okolně použitých dat (princip lokality) v několika úrovních (L1/L2/L3) — doplněná o **TLB** (cache překladu adres) a **prefetcher** (přednačítání dat dopředu). Proto má RISC velký registrový soubor (méně přístupů do paměti) a oddělenou paměťovou fázi pipeline. Jádra Cortex-M0/M3 bez cache jsou závislá na rychlosti Flash (wait states, přednáška 2).

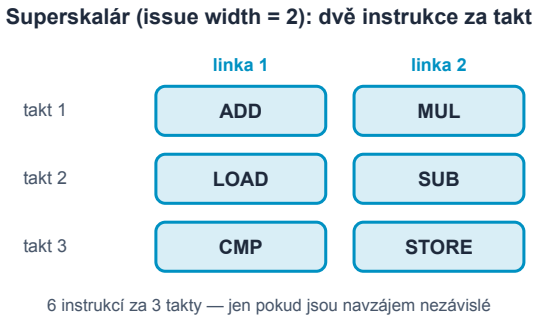
Za hranici jedné instrukce za takt: superskalár a spekulace

Zřetězení dovede pipeline k propustnosti *jedna instrukce za takt*. Výkonná jádra (Cortex-A, x86) jdou ještě dál — a stojí za to vědět jak, protože to vysvětluje, proč na moderním procesoru **záleží na pořadí instrukcí** a proč je mikroarchitektura i otázkou bezpečnosti.

Superskalární
Procesor s více paralelními linkami: vydá víc instrukcí za takt.

ILP
Instruction-level parallelism: kolik nezávislých instrukcí lze z jednoho proudu vykonat souběžně; dokáže ho využít hardware.

Superskalární procesor vydá **víc instrukcí za takt** (tzv. *issue width*). Potřebuje k tomu **víc výkonných jednotek** (několik ALU, FPU, load-store) a hlavně instrukce, které jsou na sobě **nezávislé** (obr. 7.5). Kolika nezávislými instrukcemi proud disponuje, tomu se říká **ILP** (paralelismus na úrovni instrukcí).



Obrázek 7.5. Superskalární vydávání: dvě nezávislé instrukce za takt

Háček jsou **závislosti**: dvě instrukce nelze spustit současně, potřebuje-li jedna výsledek druhé.

```
add r1, r2, r3    ; r1 = r2 + r3
mul r4, r1, r5    ; potřebuje r1, takže musí počkat – souběh nejde
```

Právě proto na pořadí a nezávislosti instrukcí záleží — a proto překladač instrukce přeskupuje, aby závislé kusy od sebe oddálil.

Out-of-order execution. Studenti obvykle čekají, že procesor zpracovává instrukce přesně v pořadí shora dolů. Moderní jádro si počíná sofistikovaněji: instrukci spustí, **jakmile jsou hotová její data**, ne v pořadí zápisu. Uvázne-li jedna na pomalé paměti, ostatní ji předběhnou:

```
ldr r1, [r0]      ; čeká na RAM (desítky až stovky taktů)
add r8, r9, r10   ; nezávislá, poběží hned
mul r4, r5, r6    ; nezávislá, poběží hned
```

Out-of-order jádro vede rozpracované výsledky v *reorder bufferu* a navenek je poskládá zpět do původního pořadí, takže program vidí správný výsledek. (Detaily — Tomasulův algoritmus — sem nepatří; stačí princip.)

Out-of-order
Vykonávání v pořadí dostupnosti dat, ne zápisu; výsledky se skládají zpět přes reorder buffer.

Přejmenování registrů. Uvažme dvakrát zapsaný týž registr:

```
add r1, r2, r3    ; první r1
; ... nesouvisející kód ...
add r1, r6, r7    ; druhý r1 – jiné, nesouvisející datum
```

Obě `r1` spolu logicky nesouvisejí, jen sdílejí *jméno*, což by je zbytečně serializovalo. Hardware proto jméno `r1` pokaždé namapuje na **jiný fyzický registr** (architektonických jmen je 16, fyzických registrů desítky). Falešná závislost tím zmizí a instrukce mohou běžet mimo pořadí.

Spekulativní vykonávání. Aby procesor nečekal na výsledek skoku (viz výše v této části), **predikuje** cestu a spekulativně počítá dopředu; při chybné předpovědi výsledky zahodí. To je motor výkonu — ale i **bezpečnostní slabina**: spekulativně načtená data zanechají stopu v cache, a útoky **Spectre** a **Meltdown** (2018) z těch stop dokázaly vyčíst i chráněnou paměť. Je to učebnicový příklad toho, že **mikroarchitektura ovlivňuje bezpečnost** — vrátíme se k tomu v přednášce 13.

A Cortex-M? Nic z toho nemá — je **in-order, bez spekulace, deterministický**. U systémů reálného času je *předvídatelnost* cennější než špičkový výkon (a odpadá i celá třída spekulativních útoků). Superskalár, out-of-order a spekulace jsou svět Cortex-A a x86.

Superskalár těží z *různých* nezávislých instrukcí (ILP). Souběžný druhý druh paralelismu — **SIMD** — naopak aplikuje *jednu* instrukci na *mnoho dat* najednou (data-level paralelismus); věnují se mu vektorové jednotky a DSP (přednáška 8 a 11).

7.2 RISC versus CISC: válka, která skončila smírem

V 80. letech to bylo předmětem ostrých sporů: měl by procesor umět **málo jednoduchých**, nebo **hodně složitých** instrukcí? Dnes víme, že odpověď je „obojí, a nakonec na tom tolik nezáleží" — ale cesta k tomu poznání tvaruje každý procesor, který dnes používáme.

Svět CISC a vzpoura RISC

CISC (Complex Instruction Set Computer) je starší filozofie 70. a 80. let, a vycházela z tehdejších reálií:

CISC
Complex Instruction Set: mocné, různě dlouhé instrukce; hustý kód, složitější hardware.

- **paměť byla extrémně drahá** → bylo nutné minimalizovat velikost programu, tedy mít hustý kód s mocnými instrukcemi;

- **kompilátory byly primitivní** → složité instrukce blízké vysokoúrovňovým jazykům usnadňovaly (ruční) programování. Tomuto úsilí zmenšit propast mezi strojovým kódem a jazykem se říká *semantic gap*;
- **mikroprogramování** (přednáška 1) umožnilo složité instrukce snadno implementovat — stačilo je rozepsat do mikrokódu, ne zadrátovat do hradel.

Výsledkem jsou procesory jako x86: stovky až tisíce instrukcí, z nichž některé umějí v jednom kroku načíst operand z paměti, něco s ním spočítat a výsledek zase uložit.

Vzpoura RISC: méně je více

V 80. letech výzkum na Berkeley (David Patterson) a Stanfordu (John Hennessy) zpochybnil celý přístup. Klíčová pozorování:

- **pravidlo 80/20** — analýza reálných programů ukázala, že ~80 % času procesor stráví v ~20 % instrukcí; složité instrukce se používaly zřídka a často byly **pomalejší** než ekvivalentní sekvence jednoduchých;
- **kompilátory dospěly** — dokázaly efektivně optimalizovat i jednoduchou instrukční sadu, takže lidská pohodlnost složitých instrukcí přestala být argumentem;
- **paměť zlevnila** — hustota kódu přestala být kritická.

Závěr byl provokativní: zjednodušme instrukční sadu na minimum rychlých, jednotných instrukcí a zbytek nechme na kompilátoru. Tak vznikl **RISC** (Reduced Instruction Set Computer) — a s ním procesory, které běžely rychleji s méně tranzistory.

RISC
Reduced Instruction Set: málo jednoduchých instrukcí pevné délky, load-store, hodně registrů.

Zásady RISC a CISC

RISC stojí na několika provázaných principech.

Jednoduché instrukce pevné délky. U většiny RISC má každá instrukce 32 bitů a pevný formát (opcode + čísla registrů + pole). To má dalekosáhlé důsledky pro hardware:

- **paralelní dekódování** — pole instrukce jsou na známých pozicích ještě před dekódováním;
- **triviální programový čítač** — `PC += 4` vždy, žádné hledání hranic instrukcí;
- **snadná predikce větvení** a **pipeline-friendly** chování (každá fáze trvá podobně dlouho).

Architektura load-store. S pamětí pracují **výhradně** instrukce `load`/`store`; veškerá aritmetika běží jen mezi registry. Tatáž operace pak v RISC a CISC vypadá jinak:

```
; RISC – tři jednoduché kroky:
LOAD  R1, [R2]      ; načti z paměti do registru
ADD   R3, R1, R4    ; aritmetika jen registr–registr
STORE R3, [R5]      ; ulož výsledek do paměti

; CISC – jedna mocná instrukce:
ADD [R5], [R2], R4  ; načti, sečti a ulož najednou
```

RISC potřebuje víc instrukcí, ale každá má **předvídatelnou latenci** (aritmetika 1 cyklus, load 2–3 cykly) a pipeline má jasně oddělenou fázi přístupu do paměti.

Velký registrový soubor. RISC architektury mají typicky 32 registrů (proti 8–16 u x86). Důvod je prostý: paměť je 100–1000× pomalejší než registry, takže čím víc proměnných udržíme v registrech, tím méně se přistupuje do paměti. Velký registrový soubor také usnadní předávání parametrů funkcím bez zásobníku.

Práci dělá kompilátor. Co CISC řešil hardwarem za běhu, řeší RISC při překladu: plánování instrukcí (*instruction scheduling*), alokaci registrů, rozbalování smyček. Krásný příklad je skrytí datového hazardu přeházením instrukcí:

```
; Před: SUB čeká na výsledek ADD (stall)
ADD R1, R2, R3
SUB R4, R1, R5    ; musí počkat na R1

; Po: kompilátor vloží nezávislou práci mezi ně
ADD R1, R2, R3
MUL R6, R7, R8    ; nezávislá instrukce vyplní mezeru
SUB R4, R1, R5    ; R1 už je hotové
```

Zásady CISC

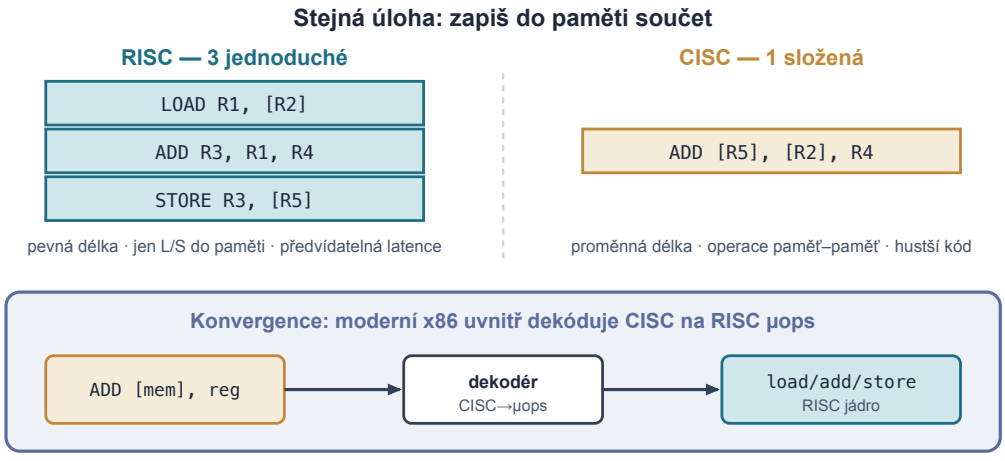
CISC jde opačným směrem. **Proměnná délka instrukcí** (x86: 1 až 15 bajtů) maximalizuje hustotu kódu — `NOP` zabere 1 bajt, složitá instrukce s adresou a konstantou klidně 10+:

```
NOP                     ; 1 B
MOV AL, 42              ; 2 B
ADD EAX, [EBX+4]        ; 3 B
MOV [ESI+disp32], imm32  ; 10+ B
```

Daň je vysoká: dekodér nejdřív musí zjistit, kde instrukce končí, než ji vůbec dekóduje — což komplikuje pipeline a paralelní zpracování. K tomu CISC přidává **bohaté adresovací režimy**, **operace přímo nad pamětí** a **mikrokód** pro nejsložitější instrukce.

Válka, která skončila smírem

Pointa je, že **dnes obě filozofie vyhrály — tím, že se sblížily**:



Obrázek 7.6. Stejná úloha v RISC a CISC; moderní x86 dekóduje CISC na RISC μops

```
1990:  RISC = jednoduché, in-order      CISC = složité, mikrokód
2025:  RISC = out-of-order, superskalární, složité
       CISC = dekóduje instrukce na RISC-like µops, jádro pak stejné jako RISC
```

Moderní x86 uvnitř **překládá své CISC instrukce na jednoduché mikrooperace** (μops) a ty vykonává RISC-like jádrem:

```
ADD [mem], reg      (CISC instrukce)
   ↓  dekodér
load  t1, [mem]     (RISC µops)
add   t1, t1, reg
store t1, [mem]
   ↓
RISC-style execution core
```

RISC zase časem přibral složitější instrukce (SIMD, DSP). **Mikroarchitektury konvergovaly; viditelným rozdílem zůstává ISA** — a s ní hustota kódu, energetická efektivita a ekosystém. Právě v energetické efektivitě má RISC stále navrch, což rozhodlo o jeho vítězství tam, kde záleží na každém miliwattu: v mobilech a vestavných systémech. A nejúspěšnějším RISC v dějinách je ARM.

Dekódování: pevná vs. proměnná délka

Důsledek pevné délky se nejlépe ukáže na dekódování. U **RISC** (pevných 32 bitů) jsou pole instrukce na známých pozicích — opcode, čísla registrů i konstanta se vyzvednou přímo a **paralelně**, ještě než se ví, o jakou instrukci jde:

```
0x00A60333  →  [6:0] opcode   [11:7] rd   [19:15] rs1   [24:20] rs2
```

U **CISC** (proměnných 1–15 bajtů) musí dekodér nejdřív zjistit, **kde instrukce končí**, teprve pak ji rozebrat — proces je v jádru sériový, drahý a špatně se paralelizuje. Moderní x86 proto věnuje značnou plochu a energii širokému dekodéru a převodu na μops; je to daň za zpětnou kompatibilitu sahající do roku 1978.

Iron law výkonu

Vztah mezi RISC a CISC shrnuje jedna rovnice — „železný zákon" výkonu procesoru:

$$T_{CPU} = IC \times CPI \times T_{takt},$$

kde **IC** je počet vykonaných instrukcí, **CPI** počet taktů na instrukci a **T_takt** perioda hodin. CISC tlačí dolů `IC` (mocné instrukce, méně jich), RISC zase `CPI` (jednoduché instrukce blízké jednomu taktu) i `T_takt` (jednodušší hardware → vyšší frekvence). RISC tedy ve `IC` „prohrává", ale vyhrává ve zbylých dvou činitelích — a rozhoduje **součin**, ne jeden faktor.

To je důležité poučení i pro optimalizaci obecně: zrychlit program znamená minimalizovat *součin* `IC` $\times$ `CPI` $\times$ `T`, ne slepě snižovat jeden z nich. Méně instrukcí nepomůže, pokud každá trvá dvakrát déle. V doméně embedded a mobilů navíc do hry vstupuje čtvrtý, nepsaný činitel — **spotřeba na instrukci** — a právě ten rozhodl ve prospěch RISC.

7.3 Architektura ARM: od skromných začátků k vašemu Macu

Nejúspěšnější RISC v dějinách si zaslouží vlastní část. Projdeme cestu ARM od úsporného čipu do domácího počítače přes éru Cortex a 64bitový přechod až po programátorský model — registry, instrukční sady, formát instrukce a barrel shifter.

Zrod a klasické rodiny ARM

ARM nezačal v žádném křemíkovém gigantu, ale v britské firmě **Acorn Computers**, která v roce 1983 potřebovala procesor do svých domácích počítačů a žádný existující jí nevyhovoval. Inspirována akademickým výzkumem RISC (Berkeley a Stanford) navrhla vlastní — **Acorn**

RISC Machine. První čip **ARM1 (1985)** měl pouhých **25 000 tranzistorů** (dnešní procesory jich mají desítky miliard) a cíl byl jasný: **jednoduchý a energeticky efektivní** procesor.

Jak úsporný ARM1 byl, dokládá historka, kterou rád vypráví jeho spolutvůrce Steve Furber: když první čip oživovali, fungoval — a teprve pak zjistili, že kvůli chybě na desce **neměl vůbec připojené napájení jádra**. Běžel z proudu, který „protékal" jeho signálovými vstupy. Ať už je verze přesná do detailu nebo ne, vystihuje, co měl ARM v DNA od začátku: **mimořádně nízkou spotřebu**. Právě ta o dvě desetiletí později rozhodla mobilní revoluci.

Klíčová designová rozhodnutí ARM1 byla na svou dobu odvážná:

- **32 bitů od začátku** (zatímco většina konkurence byla 8/16bitová),
- **podmíněné vykonávání každé instrukce** (elegantní způsob, jak se vyhnout krátkým skokům),
- **architektura load-store** s pouhými 27 instrukcemi,
- **žádná cache** — kvůli úspoře tranzistorů.

Architektura load-store
S pamětí pracují jen load/store; aritmetika běží výhradně mezi registry.

Klasické rodiny: ARM v každé kapse

V 90. letech a na začátku tisíciletí se ARM stal tichým motorem spotřební elektroniky — licencoval své jádro a nechával je vyrábět jiné, takže se dostal úplně všude:

Rodina	Pipeline	Charakteristika	Kde běžel
ARM7 (1994)	3 stupně (IF/ID/EX)	von Neumannova, varianta **ARM7TDMI**	Game Boy Advance, Nokia, iPod
ARM9 (1998)	5 stupňů (+MEM/WB)	harvardská, MMU → podpora OS	Nintendo DS, „feature phones"
ARM11 (2002)	8 stupňů, až 1 GHz	SIMD, L1+L2 cache	Raspberry Pi 1, iPhone 1/3G

Zkratka **ARM7TDMI** je sama o sobě poučná: T = Thumb (komprimovaná 16bitová sada), D = Debug, M = Multiplier (hardwarové násobení), I = ICE (ladicí rozhraní). Tento čip se vyrobil v miliardách kusů a stal se de facto standardem vestavných systémů své doby.

Éra Cortex a 64bitová éra

Od roku 2004 ARM přejmenoval jádra na **Cortex** a — což je zásadní — rozdělil je do **tří profilů** podle určení. To je nejdůležitější tabulka celé části, protože vysvětluje, proč „ARM" v telefonu a „ARM" v pračce jsou velmi odlišné věci:

Profil	Určení	Typické rysy	Příklady použití
Cortex-M (Microcontroller)	vestavné, real-time	ultra-nízká spotřeba, **jen Thumb-2**, NVIC, deterministické přerušení (~12 cyklů)	STM32, pračky, senzory, **náš čip**
Cortex-R (Real-time)	bezpečnostně kritické	deterministická cache, ECC paměti, **dual-core lockstep**	airbagy, ABS, řadiče disků
Cortex-A (Application)	výkonné, OS	out-of-order, multi-core, big.LITTLE	telefony, tablety, servery, Macy

V rámci řady **Cortex-M**, se kterou pracujeme, je vlastní jemné dělení: **M0/M0+** pro nejúspornější aplikace (0,9 μW/MHz), **M3** mainstream, **M4** s DSP instrukcemi a FPU (to je

náš **STM32F4/L4**), **M7** výkonný s dvojnásobnou přesností FPU, a **M33/M55** s bezpečnostním rozšířením **TrustZone** a vektorovým **Helium** (MVE). Charakteristický rys celé řady M je **determinismus**: pevná, předvídatelná latence přerušení, kterou jsme potřebovali v přednášce 2.

TrustZone
Hardwarové rozdělení čipu na secure a non-secure svět (Cortex-M33/M55).

64bitová éra a Apple Silicon

V roce 2011 přišel **ARMv8-A** s 64bitovým režimem **AArch64**: 31 univerzálních 64bitových registrů, standardní vektorové NEON — a zajímavé rozhodnutí **odstranit všeobecné podmíněné vykonávání** (ukázalo se, že u širokých out-of-order jader spíš překáží, než pomáhá). **ARMv9 (2021)** přidal škálovatelné vektory SVE2 a bezpečnostní Confidential Compute.

Symbolickým vrcholem byl rok **2020**, kdy Apple přešel z x86 na vlastní ARM procesory **Apple Silicon (M1–M4)** — a ukázal, že ARM už dávno není „jen pro telefony", ale dokáže porazit x86 i ve výkonných noteboocích, a to při zlomkové spotřebě. Architektura, která začala jako úsporný čip do domácího počítače, tak uzavřela kruh.

Instrukční sady ARM, Thumb a Thumb-2

ARM během vývoje nabídl tři varianty kódování, mezi nimiž je napínavý kompromis mezi **velikostí kódu** a **rychlostí** (tab. 7.2):

Tabulka 7.2. Kompromis velikosti kódu u sad ARM, Thumb a Thumb-2

Sada	Šířka	Velikost kódu	Poznámka
ARM	32 b pevná	100 % (základ)	nejrychlejší, ale největší
Thumb	16 b pevná	**65 %**	menší kód, ale méně schopné instrukce
Thumb-2	mix 16/32 b	**74 %**	**sweet spot** — hustota i výkon

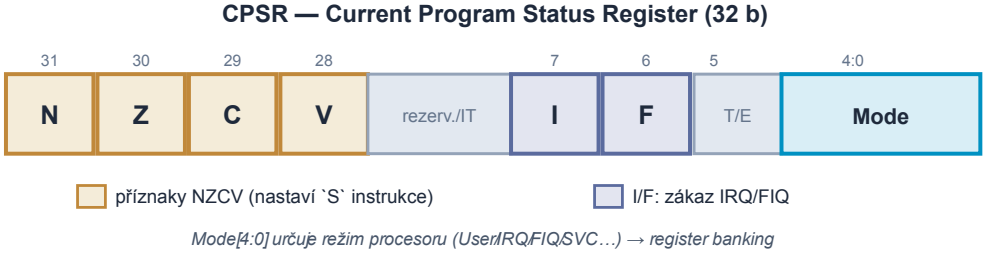
Thumb-2 je proto jediná sada jader Cortex-M: dává hustotu kódu blízkou čistému Thumbu, ale s plnou schopností 32bitových instrukcí tam, kde je potřeba. Praktickou prací s ní se zabývá oddíl 7.4.

Thumb-2
Smíšená 16/32bitová sada Cortex-M: hustota kódu i výkon. Cortex-M umí výhradně ji.

Registry, stavový registr a banking

Programátorský model ARMv7 má **16 viditelných registrů** `R0`–`R15`, z nichž tři mají speciální roli: `R13` = ukazatel zásobníku (**SP**), `R14` = návratová adresa (**LR**, Link Register — `BL` ji nastaví, `BX LR` se podle ní vrátí) a `R15` = programový čítač (**PC**). Stav procesoru drží **CPSR** (Current Program Status Register) s příznaky a řídicími bity:

CPSR
Current Program Status Register: příznaky N Z C V, řídicí bity (I/F), režim procesoru.



Obrázek 7.7. Stavový registr CPSR: příznaky N Z C V a řídicí bity

Příznaky **N** (záporný), **Z** (nula), **C** (přenos) a **V** (přetečení) nastaví aritmetická instrukce

s příponou `S` (`ADDS`, `SUBS`) a čtou je podmíněné skoky (část 7.4). Bity `I`/`F` zakazují přerušení, pole `Mode` určuje režim procesoru.

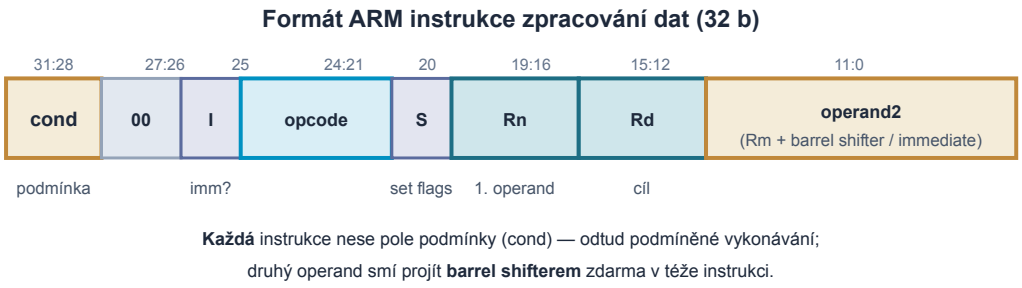
A právě režimy vedou k jednomu z nejelegantnějších řešení klasického ARM — **register bankingu**. ARM má sedm režimů (User, System, FIQ, IRQ, Supervisor, Abort, Undefined) a některé z nich mají **vlastní fyzické kopie** některých registrů pod stejným logickým jménem. Aktivní kopie závisí na režimu:

```
           User     FIQ       IRQ       SVC
R0–R7      sdílené  sdílené   sdílené   sdílené
R8–R12     sdílené  R8_fiq…   sdílené   sdílené
R13 (SP)   R13      R13_fiq   R13_irq   R13_svc
R14 (LR)   R14      R14_fiq   R14_irq   R14_svc
```

Smysl je výkonový: režim **FIQ** (Fast Interrupt) má vlastní kopie `R8`–`R14`, takže obsluha rychlého přerušení **nemusí tyto registry ukládat na zásobník** — jsou „její vlastní". To zkracuje vstup do přerušení, odtud „Fast". (Cortex-M tento mechanismus zjednodušil: místo bankingu používá automatické uložení části kontextu hardwarem, jak jsme viděli v přednášce 2.)

Formát instrukce, barrel shifter a konstanty

Pohled na bitový formát ARM instrukce zpracování dat odhalí dvě věci, které dělají tuto architekturu charakteristickou:



Obrázek 7.8. Formát ARM instrukce zpracování dat

Za prvé, **každá** instrukce nese ve čtyřech nejvyšších bitech pole podmínky `cond` — odtud všeobecné podmíněné vykonávání. Za druhé, druhý operand smí projít **barrel shifterem**: posun nebo rotace proběhne **v téže instrukci, bez taktu navíc**:

Barrel shifterem
Barrel shifter: druhý operand projde posunem/rotací zdarma v téže instrukci.

```
add r0, r1, r2, lsl #3   ; r0 = r1 + (r2 << 3) = r1 + r2*8
rsb r0, r1, r1, lsl #4   ; r0 = r1*16 - r1     = r1*15
mov r0, r1, asr #31      ; r0 = znaménko r1 (0 nebo -1)
```

Násobení mocninou dvojky, indexace polí i postupy práce se znaménkem — to vše **bez násobičky a bez skoku**. Barrel shifter je učebnicový příklad RISC elegance: jeden mechanismus, mnoho použití.

Jak to zvládne bez taktu navíc? Není to posuvný registr, který by posouval o jeden bit za takt (a na posun o k pozic potřeboval k taktů). Barrel shifter je čistě **kombinační obvod** — síť multiplexorů, která operand posune nebo zrotuje o **libovolný počet pozic v jediném průchodu**, během téhož taktu. Staví se z $\log_2 n$ vrstev, kde každá posouvá o mocninu dvojky (o 1, 2, 4, 8 … pozic); podle jedniček v binárním zápisu požadovaného posunu se příslušné vrstvy zapnou. Za cenu kusu kombinační logiky tak ARM dostane posun „zdarma" hned před ALU — a právě proto si ho může dovolit vložit do **každé** instrukce zpracování dat.

Kódování konstant

Pevná délka má i nepříjemný důsledek: do 32bitové instrukce se nevejde 32bitová konstanta. ARM to řeší kódováním **8bitová hodnota + sudá rotace**, což pokryje mnoho „kulatých" hodnot, ale ne libovolnou:

```
mov r0, #0xFF000000   ; 0xFF rotováno o 8 → lze zakódovat
mov r0, #0x12345678   ; nelze zakódovat přímo!
ldr r0, =0x12345678   ; pseudoinstrukce → načte z literal poolu
```

Pseudoinstrukce `ldr rX, =konstanta` libovolnou hodnotu vyřeší: assembler ji uloží do **literal poolu** (oblast konstant poblíž kódu) a nahradí ji obyčejným `ldr` z paměti. Je to přímý důsledek volby pevné délky instrukcí.

Výjimky a moderní rozšíření (SIMD, DSP, bezpečnost)

Přerušení i chybové stavy jsou pro procesor **výjimky**: přeruší běžící kód a skočí přes **vektorovou tabulku** na příslušnou obsluhu (přednáška 2). Klasický ARM k tomu používá sedm režimů a register banking (výše v této části) s vektory na pevných adresách. **Cortex-M** to zjednodušil: vektorová tabulka leží na začátku Flash, priority řadí **NVIC** a část kontextu se ukládá **automaticky hardwarem** — žádný banking. Výsledkem je determinismus, na který jsme spoléhali v přednášce 2: vstup do přerušení za pevných zhruba dvanácti cyklů a tail-chaining bez zbytečného obnovování kontextu.

SIMD, DSP a bezpečnost

Moderní ARM přidává rozšíření pro náročné domény. SIMD (Single Instruction, Multiple Data) provede jednu operaci nad více daty najednou — základ zpracování signálu, obrazu a strojového učení: **NEON** (128bitové vektory, standard na Cortex-A), **DSP rozšíření** Cortex-M4/M7 (saturující aritmetika, násobení-akumulace MAC, SIMD nad 16/8 bity) a **Helium/MVE** pro malá jádra Cortex-M55 (TinyML na mikrokontroléru). Právě jednotaktové MAC je důvod, proč náš Cortex-M4 zvládne filtraci a FFT v reálném čase — využijeme to na cvičení se syntezátorem.

SIMD
Single Instruction, Multiple Data: jedna operace nad více daty (signál, obraz, ML).

Bezpečnost dnes architektura prosazuje přímo v hardwaru: **MPU** (Memory Protection Unit) hlídá přístupy do paměti (přetečení zásobníku z přednášky 3, izolace úloh), zatímco **TrustZone** (Cortex-M33/M55) rozděluje čip na **secure** a **non-secure** svět, takže klíče a kryptografie zůstávají izolované od aplikace. Bezpečnost už není jen otázka softwaru — detailně se jí věnuje pozdější přednáška.

7.4 Programování v ARM assembleru

Číst assembler je nutné pro ladění; psát jej ručně je vzácné, ale v několika situacích nezbytné (startup kód, atomické sekvence, výkonová jádra). Tato část shrnuje instrukční sadu Thumb-2 jádra Cortex-M4 do té míry, aby šlo disassembly přečíst a rozumět, co každá instrukce dělá s daty a tokem řízení.

Kódování a instrukce zpracování dat

Klasická ARM instrukce je 32bitová s polem podmínky v horních bitech; Thumb je 16bitová komprimovaná sada; **Thumb-2** (Cortex-M) je mix 16/32 b — 16bitové instrukce pro běžné operace, 32bitové pro komplexní, což dává lepší hustotu kódu než čistý ARM:

```
32-bit ARM:   [31:28] Condition | [27:20] Type | [19:0] Operands
   0xE2800004 = ADD R0, R0, #4
16-bit Thumb: [15:10] Instruction | [9:0] Operands
   0x3004     = ADD R0, #4
```

Jádra Cortex-M umějí **výhradně Thumb-2** — proto je `-mthumb` při překladu povinný.

Instrukce zpracování dat

Operují výhradně nad registry (architektura load-store z přednášky 1):

```
add   r0, r1, r2        ; r0 = r1 + r2
sub   r0, r1, r2        ; r0 = r1 - r2
mul   r0, r1, r2        ; r0 = r1 * r2 (32b)
umull r0, r1, r2, r3    ; r1:r0 = r2 * r3 (64b)
adds  r0, r1, r2        ; jako add, navíc nastaví příznaky NZCV

and r0, r1, r2          ; r0 = r1 & r2
orr r0, r1, r2          ; r0 = r1 | r2
eor r0, r1, r2          ; r0 = r1 ^ r2  (XOR)
bic r0, r1, r2          ; r0 = r1 & ~r2 (bit clear)
mvn r0, r1              ; r0 = ~r1

lsl r0, r1, #2          ; r0 = r1 << 2  (logický posun vlevo)
lsr r0, r1, #2          ; r0 = r1 >> 2  (logický posun vpravo)
asr r0, r1, #2          ; r0 = r1 >> 2  (aritmetický, zachová znaménko)
ror r0, r1, #2          ; rotace vpravo
```

Síla ARM je v **kombinovaných instrukcích** — posun operandu proběhne v téže instrukci:

```
add r0, r1, r2, lsl #2  ; r0 = r1 + (r2 << 2)
```

Přípona `s` (`adds`, `subs`) nastaví příznakové bity, na nichž závisí podmíněné větvení.

Load/store a adresovací režimy

Load/store je jediná cesta mezi registry a pamětí. Existují varianty pro slovo (`ldr`/`str`), bajt (`ldrb`/`strb`) a půlslovo (`ldrh`/`strh`). Síla ARM je v **adresovacích režimech**, které do jedné instrukce skládají výpočet adresy i přístup:

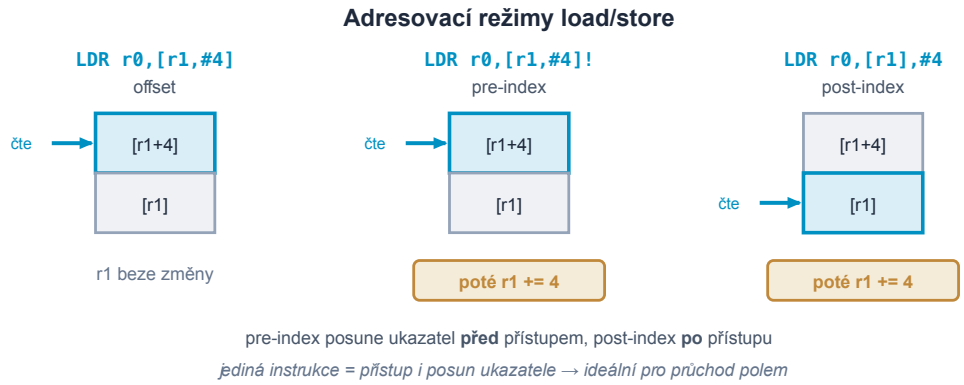
Režim	Zápis	Význam
immediate offset	`ldr r0, [r1, #4]`	adresa `r1+4`, `r1` beze změny
register offset	`ldr r0, [r1, r2, lsl #2]`	adresa `r1 + (r2<<2)`
pre-index	`ldr r0, [r1, #4]!`	`r1 += 4`, **poté** přístup
post-index	`ldr r0, [r1], #4`	přístup na `r1`, **poté** `r1 += 4`

Pre-a post-index jsou ideální pro průchod polem: jediná instrukce načte prvek a posune ukazatel. Registrový režim s posunem (`[r1, r2, lsl #2]`) přímo indexuje pole 32bitových slov.

Větvení a podmíněné vykonávání

Tok řízení čtou instrukce větvení z příznakových bitů registru `CPSR`:

```
N = Negative (31)   Z = Zero (30)   C = Carry (29)   V = Overflow (28)
```



Obrázek 7.9. Adresovací režimy load/store: offset, pre-index, post-index

```
b   label        ; nepodmíněný skok
bl  function     ; skok s uložením návratu do LR (volání funkce)
bx  lr           ; návrat (skok na adresu v registru)
beq / bne / blt / bgt label   ; podmíněné skoky podle příznaků
```

Mapování podmínek na příznaky:

Kód	Význam	Příznaky	Kód	Význam	Příznaky
EQ	rovno	Z=1	GT	větší (signed)	Z=0 ∧ N=V
NE	nerovno	Z=0	GE	větší/rovno	N=V
LT	menší (signed)	N≠V	HI	vyšší (unsigned)	C=1 ∧ Z=0
LE	menší/rovno	Z=1 ∨ N≠V	LS	nižší/rovno (uns.)	C=0 ∨ Z=1

Klasická 32bitová ARM sada uměla **podmíněně vykonat libovolnou instrukci** (`addgt`, `movle`), čímž se vyhýbala krátkým skokům — to je ono „podmíněné vykonávání", které měl ARM v DNA od ARM1 (část 7.3). V Thumb-2 je tato schopnost zachována jen přes blok `IT` (If-Then), který podmíní až čtyři následující instrukce:

```
    cmp   r0, #0
    it    lt
    neglt r0, r0      ; abs(r0) bez skoku
```

Eliminace krátkých skoků chrání pipeline před chybnou predikcí a vyprázdněním.

Praktický příklad, direktivy a pseudoinstrukce

Funkce, která přičte k globální proměnné, ukazuje celý vzor přístupu do paměti (načti adresu → načti hodnotu → uprav → ulož):

```
.data
global_var: .word 42
.text
.global add_to_global
add_to_global:
    ldr r1, =global_var     ; načti adresu
    ldr r2, [r1]            ; načti hodnotu
    add r2, r2, r0          ; přičti x (v r0)
    str r2, [r1]            ; ulož zpět
    bx lr                   ; návrat
```

Direktivy a pseudoinstrukce

Assemblerový zdroj není jen posloupnost instrukcí. **Direktivy** (začínají tečkou) říkají assembleru a linkeru, kam co umístit a jak to pojmenovat; **pseudoinstrukce** se rozvinou na jednu či více skutečných instrukcí:

```
.text                ; sekce kódu (do Flash)
.data                ; inicializovaná data (do RAM)
.global add_to_global ; export symbolu pro linker
.word 42             ; 32bitová konstanta v paměti
.align 2             ; zarovnání na 4 bajty

ldr r0, =0x12345678  ; pseudoinstrukce → ldr z literal poolu
nop                  ; rozvine se na mov r0, r0
```

Direktivy `.text`, `.data`, `.global` jsou přesně to, co spojuje assembler se sekcemi a linkerem z přednášky 3 — sám překladač je generuje do `*.s`.

Barrel shifter a součet pole v praxi

Barrel shifter z části 7.3 je v assembleru nenápadný, ale klíčový mechanismus. Indexace pole i násobení konstantou se vejdou do jediné instrukce:

```
ldr r0, [r3, r4, lsl #2]   ; načti a[index]: adresa = r3 + index*4
add r0, r1, r1, lsl #1     ; r0 = r1 * 3
rsb r0, r1, r1, lsl #5     ; r0 = r1 * 31
```

Příklad: součet pole ve smyčce

Spojme adresovací režimy, příznaky a větvení do celé funkce — součtu pole, který jsme v přednášce 3 viděli z druhé strany (jako výstup kompilátoru na `-O2`):

```
; int sum(int *a, int n)   — a = r0, n = r1, výsledek r0
sum:
    movs  r2, #0            ; s = 0
    cbz   r1, .done         ; n == 0 → konec
.loop:
    ldr   r3, [r0], #4      ; r3 = *a++   (post-index!)
    subs  r1, r1, #1        ; n-- a nastav příznaky
    add   r2, r2, r3        ; s += r3
    bne   .loop             ; dokud n != 0
.done:
    mov   r0, r2            ; návratová hodnota
    bx    lr
```

Post-index `[r0], #4` posune ukazatel po načtení, počítadlo jde **dolů** (`subs`/`bne` testuje proti nule zdarma) — přesně transformace, kterou provedl kompilátor na `-O2`. Číst tyto vzory je klíč k pochopení disassembly.

Kdy ruční assembler skutečně zrychlí kód

Předchozí oddíly mohou svádět k dojmu, že ruční assembler je cesta k rychlému kódu. Není — s `-O2` kompilátor **většinou vytvoří lepší kód**, protože zná plánování instrukcí, alokaci registrů i přidělení podmíněného vykonávání lépe, než je člověk ochoten ručně ladit. Ruční assembler (nebo **intrinsics**) zrychlí jen ve třech situacích: když kompilátor **nevygeneruje** speciální instrukci z přenositelného C (DSP/SIMD, saturace, bit-reverse, počítání nul); když potřebujeme **cyklově přesný** kód (časované smyčky, bit-banging protokolu); a ve velmi těsné

horké smyčce (jádro DSP), kde rozhoduje každý takt. Celé funkce v assembleru přitom píšeme zřídka — většinou stačí **intrinsics** (CMSIS `__CLZ`, `__SSAT` a podobně): jediná instrukce, ale čitelná a kompilátor kolem ní plánuje.

První rodina jsou **bitové operace bez přímého operátoru v C**. Obrácení pořadí bitů je v C smyčka přes 32 iterací, ale ARM to umí jedinou instrukcí:

```
// naivní C: ~32 iterací smyčky
uint32_t rbit(uint32_t x){ uint32_t r=0; for(int i=0;i<32;i++){ r=(r<<1)|(x&1); x>>=1; } return r; }

uint32_t r = __RBIT(x);        // 1 instrukce (RBIT) — bit-reverse pro FFT
int      l = 31 - __CLZ(x);    // 1 instrukce (CLZ) — floor(log2), normalizace, priorita
uint32_t n = __REV(x);         // 1 instrukce (REV) — prohození bajtů (endianness, htonl)
```

Protože `CLZ`, `RBIT` a `REV` nemají v C přímý operátor, naivní kód je smyčka, kdežto intrinsic je **jedna instrukce**.

Druhá rodina je **DSP/SIMD**: jádra Cortex-M4 a M7 mají rozšíření, kde jedna instrukce zpracuje dvě 16bitová data zabalená ve 32bitovém registru. Skalární součin dvou polí `int16` pak provede dvě násobení a součet na instrukci:

```
int32_t dot(const int16_t *a, const int16_t *b, int n){
    int32_t s = 0;
    const uint32_t *pa=(const uint32_t*)a, *pb=(const uint32_t*)b;
    for(int i=0;i<n/2;i++) s = __SMLAD(pa[i], pb[i], s);   // (a0*b0 + a1*b1) + s
    return s;
}
```

`SMLAD` a `SMUAD` provedou dva součiny 16×16 a akumulaci v **jedné instrukci**, takže FIR/IIR filtr běží zhruba dvakrát rychleji než skalární C smyčka — na těchto instrukcích stojí celá knihovna **CMSIS-DSP**.

Třetí rodina je **saturující aritmetika bez větvení**. V audiu a řízení musí hodnota zůstat v rozsahu (clamp); v C jsou to dvě podmínky, tedy dva skoky, které brzdí pipeline. ARM saturuje jednou instrukcí bez skoku:

```
int16_t clamp_c(int32_t x){           // dvě větve = dva skoky
    if (x >  32767) return  32767;
    if (x < -32768) return -32768;
    return (int16_t)x;
}
int32_t y = __SSAT(x, 16);            // 1 instrukce (SSAT), bez skoku
int32_t z = __QADD(a, b);             // sčítání se saturací (QADD) — bez přetečení
```

`SSAT`, `USAT` a `QADD` ořežou nebo sečtou se saturací bez podmíněného skoku — žádný pipeline stall a kratší kód.

Čtvrtá rodina je **blokový přesun** instrukcemi load/store multiple, které přenesou víc registrů najednou:

```
; kopíruj 8 slov (32 B) na iteraci: r0=cíl, r1=zdroj, r2=počet slov
.loop:
    ldmia r1!, {r4-r11}    ; načti 8 slov, r1 += 32
    stmia r0!, {r4-r11}    ; ulož 8 slov, r0 += 32
    subs  r2, r2, #8
    bne   .loop
```

Osm slov na dvě instrukce místo osmi dvojic `ldr/str`. Knihovní `memcpy` tohle už dělá za nás;

ručně to píšeme jen pro speciální zarovnané buffery.

Nad tím vším stojí **pořadí optimalizace** — assembler je až poslední krok. Nejprve **změřte** (profilerem nebo čítačem `DWT->CYCCNT` z přednášky 3): čas obvykle žije v několika horkých smyčkách. Pak hledejte **lepší algoritmus**, který přebije jakoukoli mikrooptimalizaci. Teprve potom zapněte **`-O2` nebo `-O3`** a podívejte se na výstup (`-S`). Následují **intrinsics** (CMSIS, CMSIS-DSP) — čitelné a přenositelné v rámci jádra. A úplně nakonec **ruční assembler**, jen v horké smyčce a tam, kde intrinsic nestačí. Assembler v horké smyčce dá násobky; jinde jen ztratíte čitelnost a přenositelnost — a `asm` je navíc v MISRA výjimka vyžadující zdůvodnění (přednáška 3).

> **Shrnutí oddílu 7.4.** Cortex-M běží v Thumb-2; assembler stojí na třech operandech, barrel shifteru, příznacích a podmíněném vykonávání. Ruční assembler ale s `-O2` **většinou nepředčí kompilátor** — vyplatí se jen na instrukce bez ekvivalentu v C (`CLZ`/`RBIT`/`REV`), na DSP/SIMD (`SMLAD`), saturaci (`SSAT`/`QADD`) a blokový přesun (`LDM`/`STM`), a to nejlépe přes **intrinsics**. Platí pořadí: nejdřív měř, pak lepší algoritmus, pak `-O2`, pak intrinsics a teprve nakonec ruční asm v horké smyčce.

7.5 RISC-V: jiná sada kompromisů

Architektur jsme dosud probrali dostatek; smysl této části ale není přidat další položku do katalogu, ale ukázat na **RISC-V** ještě jednu věc: že tatáž rodina (RISC) může dělat **jiná návrhová rozhodnutí** než ARM — a nést jiné důsledky. RISC-V je zde zajímavý ne proto, že je populární, ale protože svými rozhodnutími otevírá otázky, které u ARM ani x86 nevyvstávají.

Rozhodnutí, modularita a minimalizace

RISC-V vznikl na Berkeley v roce 2010 (opět u Davida Pattersona) a stojí na třech rozhodnutích, z nichž každé má jasný důsledek:

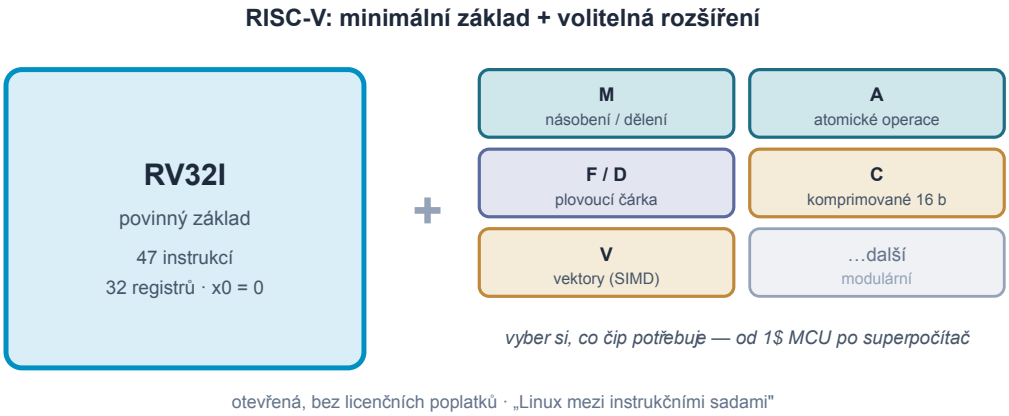
- **otevřenost** — ISA je volně dostupná bez licenčních poplatků. Důsledek: kdokoli si může postavit vlastní křemík, přidat **vlastní instrukce**, a není vázán na jednoho dodavatele;
- **modularita** — malý povinný základ plus volitelná rozšíření. Důsledek: tatáž ISA škáluje od čipu za dolar po superpočítač, a návrhář **platí jen za to, co použije**;
- **záměrná minimalizace** — základ má 47 instrukcí a registr `x0` natvrdo nulový. Důsledek: jednodušší dekodér a hardware, ale o něco víc instrukcí na úlohu (složitost se přesouvá do kompilátoru a rozšíření).

Modularita
RISC-V: malý povinný základ + volitelná rozšíření; platí se jen za to, co se použije.

Registr `x0 = 0` je učebnicový příklad, jak jedno rozhodnutí ušetří mnoho speciálních instrukcí: `mov` je `addi rd, rs, 0`, `nop` je `addi x0, x0, 0`, negace je `sub rd, x0, rs`.

Modularita jako trade-off

Mikrokontrolér si vezme `RV32IMC` (základ + násobení + komprimované instrukce), výkonné jádro `RV64GC` s plovoucí čárkou a vektory. Pro vestavné systémy je nejcennější právě možnost **doménově specifických rozšíření** — vlastních instrukcí pro DSP, kryptografii nebo edge-AI — bez zásahu do zbytku ISA a bez vyjednávání s držitelem licence. Daní za tuto čistotu a škálovatelnost je o něco vyšší počet instrukcí na úlohu.



Obrázek 7.10. RISC-V: minimální základ plus volitelná rozšíření

Záměrná minimalizace

Srovnání s ARM ukazuje, že nejde o „lepší" a „horší", ale o **jiný bod v prostoru kompromisů**:

	ARM (Cortex-M)	RISC-V (RV32I)
podmíněné vykonávání	ano (bloky `IT`)	**ne** (jen větvení)
barrel shifter	ano (zdarma v instrukci)	**ne** (posun zvlášť)
registry	16	32 (+ `x0` = 0)
licence	proprietární	**otevřená**

RISC-V vědomě vynechává ARM prvky navíc jako podmíněné vykonávání a barrel shifter. Důsledkem je jednodušší hardware a dekodér za cenu o něco delšího kódu — přesně ten typ rozhodnutí, o kterém je celá tato přednáška.

7.6 Syntéza: proč architektury koexistují

Celá přednáška mířila k jediné otázce: **proč dnes vedle sebe koexistují ARM, x86, RISC-V, GPU i AI akcelerátory, místo aby všechno používalo jednu univerzální architekturu?** Odpověď plyne ze všeho, co jsme viděli — a je to odpověď o kompromisech.

Žádná architektura nevyhrává všechno

Návrh procesoru se pohybuje v prostoru se **čtyřmi osami, které si navzájem odporují**:

- **latence** — udělat jednu věc co nejrychleji — versus **propustnost** — udělat mnoho věcí paralelně;
- **spotřeba a plocha** — vyřešit pevně danou úlohu co nejlevněji — versus **programovatelnost** — zvládnout cokoli.

Žádný jediný bod nemůže být optimální ve všech čtyřech směrech najednou. Procesor optimalizovaný na nízkou latenci a univerzálnost (CPU) bude vždy horší v propustnosti než křemík specializovaný na jednu úlohu (akcelerátor) — a naopak. Proto pro danou doménu **specializace předčí univerzálnost**.

Heterogenní embedded SoC

Koexistence architektur není jen otázka různých čipů na trhu — odehrává se **uvnitř jediného pouzdra**. Moderní mikrokontrolér nebo SoC je obvykle směs:

- **Cortex-M** pro řízení a rozhodování, kde rozhoduje nízká latence a determinismus;
- **DSP / SIMD jednotky** (Cortex-M4 s DSP, Helium) pro zpracování signálu, kde jde o propustnost;
- **kryptografické bloky a NPU** (neuronové akcelerátory) pro pevně dané úlohy, které vyplatí řešit specializovaným hardwarem za zlomek energie.

Každý blok je odpověď na jiný trade-off; dohromady tvoří heterogenní systém, kde univerzální jádro je doplněno specializovanými.

Odpověď a co dál

Odpověď na úvodní otázku tedy zní: **prostor kompromisů — výkon, spotřeba, složitost a programovatelnost — nemá jediné optimum.** Každá architektura je odpovědí na jinou otázku. Dobrý inženýr proto nehledá „nejlepší" architekturu; zná **trade-offy** a vybírá podle domény. Právě schopnost uvažovat o kompromisech — místo memorování parametrů jednotlivých procesorů — odlišuje inženýra od pozorovatele.

Tam, kde se specializace a paralelismus dotahují do extrému — **vícejádrové systémy a AI akcelerátory** (systolická pole, tensor jednotky) — pokračuje přednáška 8.

7.7 Kontrolní otázky

Otázky prověřují látku celé kapitoly; jsou seskupené podle oddílů, ze kterých vycházejí.

K oddílu 7.1 — ISA a mikroarchitektura

1. Vysvětlete rozdíl mezi ISA a mikroarchitekturou na analogii s API.
2. Jak je možné, že Core i3 (2010) a Core i9 (2023) spustí týž software bez překompilování?
3. Proč mají všechny moderní procesory navzdory různým ISA překvapivě podobnou kostru (fetch → decode → execute → memory → writeback)? Co se v jednotlivých fázích děje?
4. Čím se RISC-V liší od ARM a x86 a proč je to zajímavé?
5. Proč je přechod na novou ISA pro ekosystém softwaru velká událost?

K oddílu 7.2 — RISC versus CISC: válka, která skončila smírem

1. Z jakých dobových reálií vzešla filozofie CISC (paměť, kompilátory, semantic gap)?
2. Co je „pravidlo 80/20" a jak podpořilo myšlenku RISC?
3. Vyjmenujte zásady RISC a u každé řekněte, jaký hardwarový či výkonový přínos má.
4. Ukažte na příkladu, jak kompilátor RISC skryje datový hazard přeskupením instrukcí.
5. Vysvětlete, v jakém smyslu „obě architektury vyhrály" — jak konverguje moderní x86 a RISC.

K oddílu 7.3 — Architektura ARM: od skromných začátků k vašemu Macu

1. Z jaké firmy a filozofie ARM vzešel a jaká dvě rozhodnutí ARM1 měla na svou dobu výjimečná?
2. Proč ARM nabízí tři profily Cortex (M/R/A) — jaký kompromis (spotřeba vs. výkon vs. determinismus) každý z nich řeší?
3. Proč je Thumb-2 jedinou instrukční sadou jader Cortex-M? Co je na ní „sweet spot"?
4. Jak register banking zrychlí obsluhu přerušení (FIQ) — jaký princip (méně ukládání

kontextu) za tím stojí?
5. Jak podmíněné příznaky (v CPSR) umožňují podmíněné vykonávání — kdo je nastavuje a kdo čte?

K oddílu 7.4 — Programování v ARM assembleru

1. Proč je `-mthumb` u Cortex-M povinný? Čím se Thumb-2 liší od klasické ARM sady?
2. Vysvětlete rozdíl mezi pre-index a post-index adresováním na průchodu polem.
3. Jak instrukce `bl` realizuje volání funkce a k čemu při tom slouží registr `LR`?
4. K čemu slouží blok `IT` a jak souvisí s výkonem pipeline a s historií ARM?
5. Napište v assembleru vzor „načti–uprav–ulož” pro inkrement hodnoty na adrese v `r1`.
6. Ve kterých třech situacích se vyplatí ruční assembler a proč jsou intrinsics obvykle lepší volba než celá funkce v asm?

K oddílu 7.5 — RISC-V: jiná sada kompromisů

1. Jmenujte tři návrhová rozhodnutí RISC-V a u každého uveďte jeho důsledek.
2. Jak registr `x0` zjednodušuje instrukční sadu? Uveďte dva příklady.
3. Co znamená modularita RISC-V a proč je cenná zrovna pro vestavné systémy?
4. Které ARM mechanismy RISC-V vynechává a jaký to má důsledek pro hardware a velikost kódu?

8

Multicore a specializovaný hardware

> Tato kapitola odpovídá na otázku **proč už nestačí jedno rychlejší jádro** a proč vedle procesoru přibývají další jádra a specializovaný křemík (DSP, GPU, NPU). Od roku ~2005 výkon roste jen dvěma cestami — **víc jader** a **specializovaný hardware** — a obě jsou o jednom: **víc užitečné práce na joule** (operace na watt, ne hertze).

Obsah kapitoly

8.1 Konec škálování jednoho jádra
8.2 Paralelismus: principy a meze
8.3 Multicore v embedded
8.4 Specializovaný hardware
8.5 AI akcelerátory (NPU)
8.6 Syntéza: heterogenní SoC
8.7 Kontrolní otázky

8.1 Konec škálování jednoho jádra

Nejprve je třeba pochopit, proč jedno jádro přestalo zrychlovat samo. Tato část projde konec Dennardova škálování, výkonovou a paměťovou zeď i strop paralelismu instrukcí — a ukáže, že z toho plynou jen dvě cesty dál: víc jader a specializace.

Konec free lunch: Moore versus Dennard

Do zhruba roku 2005 stačilo spustit existující kód na novějším čipu a běžel rychleji **bez zásahu programátora**. Výkon zvyšovaly současně tři faktory: rostoucí **takt** (z megahertzů na gigahertze), rostoucí **paralelismus na úrovni instrukcí** (pipeline, superskalární a out-of-order provádění z přednášky 7) a rostoucí **počet tranzistorů** (větší cache, víc jednotek). Tomuto období se říká „the free lunch". Skončilo — a je důležité přesně pochopit proč: ne proto, že by došly tranzistory, ale proto, že došla **energie**.

Mooreův zákon versus Dennardovo škálování

Dvě tvrzení, která se často zaměňují. **Mooreův zákon** říká, že počet tranzistorů na čipu se zhruba zdvojnásobí každé dva roky; **stále zhruba platí**, byť tempo zpomaluje. **Dennardovo škálování** říká něco jiného: když tranzistor zmenšíme, klesne i jeho napájecí napětí, takže **hustota výkonu (příkon na plochu) zůstane konstantní** a takt lze zvyšovat zdarma. Právě tohle pravidlo **skončilo kolem roku 2005** — napětí už dál klesat nemůže, protože tranzistor by přestal spolehlivě spínat. Tranzistorů tedy přibývá dál, ale **nemůžeme je všechny napájet**

Mooreův zákon
Empirické pozorování, že počet tranzistorů na čipu se přibližně zdvojnásobí každé dva roky.

Dennardovo škálování
Pravidlo, že při zmenšení tranzistoru klesá i jeho napětí, takže hustota výkonu zůstává konstantní a takt lze zvyšovat bez růstu spotřeby na plochu.

a uchladit současně. Konec Dennardova škálování je technickou příčinou konce „free lunch".

Výkonová zeď a strop ILP

Dynamickou spotřebu hradla lze přiblížit vztahem $P \approx C \cdot V^2 \cdot f$, kde C je přepínaná kapacita, V napájecí napětí a f takt. Zvýšení taktu f zvedá spotřebu lineárně, ale vyšší takt obvykle žádá i vyšší napětí V — a to vstupuje do vztahu **druhou mocninou**. Spotřeba se navíc celá mění na **teplo**, které je nutné odvést. Proto se takty zastavily kolem **3–4 GHz** u desktopů a na **stovkách MHz až jednotkách GHz** v embedded; dál už by se čip buď přehřál, nebo by vyčerpal baterii. Tomuto stropu se říká **výkonová zeď**.

Výkonová zeď
Mez, za níž už nelze zvyšovat takt, protože rostoucí spotřeba (úměrná taktu a druhé mocnině napětí) vytvoří víc tepla, než lze odvést.

Strop paralelismu instrukcí a dark silicon

Druhá historická cesta — získat víc **ILP** (paralelismu na úrovni instrukcí) z jednoho proudu instrukcí — také narazila. Superskalární a out-of-order jádra (přednáška 7) hledají v programu nezávislé instrukce, jenže běžný kód má **omezené množství** nezávislosti. Přidávat víc spekulace a přeskupovací logiky znamená **rychle rostoucí složitost a spotřebu** za stále menší zisk. K tomu se přidává jev zvaný **dark silicon**: tranzistorů je dost, ale výkonový rozpočet dovolí napájet jen jejich část, takže zbytek musí zůstat vypnutý. „Chytřejší" jediné jádro se přestalo vyplácet.

ILP
Instruction-level parallelism: souběžné provádění nezávislých instrukcí z jednoho proudu; zajišťuje hardware (pipeline, superskalár, out-of-order).

Dark silicon
Část čipu, kterou nelze napájet současně se zbytkem, protože by se překročil výkonový rozpočet; tranzistory existují, ale musí zůstat vypnuté.

Paměťová zeď a embedded rozpočet

K tomu se připojuje **paměťová zeď** z přednášky 4: jádro je mnohem rychlejší než hlavní paměť, takže rychlejší jádro samo o sobě jen **rychleji čeká na data**. Přístup do DRAM stojí stovky taktů a — což je pro zbytek přednášky klíčové — **energeticky** je čtení slova z DRAM řádově **stokrát dražší** než jedna aritmetická operace. Z toho plyne dvojí důsledek, který nás bude provázet: výkon i spotřebu dnes určuje hlavně **pohyb dat**, ne počítání samo.

Paměťová zeď
Rostoucí propast mezi rychlostí jádra a latencí hlavní paměti; rychlejší jádro pak jen rychleji čeká na data. Z přednášky 4.

Embedded rozpočet a dvě cesty ven

V embedded jsou meze ještě ostřejší než na desktopu. **Baterie** činí z každého joule cennou veličinu — výkon se měří v **operacích na watt**, ne v gigahertzích. **Pasivní chlazení** (žádný ventilátor) omezuje, kolik tepla smí pouzdro odvést. **Cena a velikost** tlačí na malý čip a jednoduché napájení. Embedded inženýr proto neřeší „jak rychlé", ale **„kolik užitečné práce za daný výkonový rozpočet"** — a to je přesně otázka paralelismu a specializace.

Když tedy nejde zrychlit jedno jádro, zbývají dvě odpovědi a moderní čipy je používají **obě zároveň**: **víc jader** (provádět víc činností najednou — paralelismus, oddíl 8.2 a 3) a **specializovaný hardware** (provádět méně, ale to pravé efektivně — oddíl 8.4 a 5). oddíl 8.6 obě cesty spojí do heterogenního SoC.

> **Shrnutí oddílu 8.1.** Mooreův zákon (víc tranzistorů) zhruba platí, ale **Dennardovo škálování** (takt zdarma) skončilo kolem 2005. **Výkonová zeď** ($P \approx CV^2f$) zastavila takty; **ILP** má strop a roste neúměrně složitost (dark silicon). **Paměťová zeď** činí z pohybu dat hlavní omezení výkonu i energie. Embedded rozpočet (baterie, pasivní chlazení) měří výkon v **operacích na watt**, což tlačí na paralelismus a specializaci — dvě cesty, jak růst dál.

8.2 Paralelismus: principy a meze

Než přidáme jádra, je dobré znát meze paralelismu. Tato část rozliší jeho tři úrovně, zařadí stroje Flynnovou taxonomií a ukáže, proč sériová část zastropuje zrychlení (Amdahlův zákon) a co zisk dále snižuje.

Tři úrovně paralelismu a Flynnova taxonomie

Paralelismus není jedna věc; existuje na třech úrovních, každá jinak náročná na získání. **ILP** (instruction-level) je souběh nezávislých instrukcí v jednom proudu — řeší ho **hardware sám** (pipeline, out-of-order), ale má strop (oddíl 8.1). **DLP** (data-level) je táž operace nad mnoha daty současně — ideální pro vektorové jednotky, SIMD a GPU. **TLP** (thread-level) je souběh nezávislých vláken či úloh na více jádrech. Zásadní rozdíl: ILP dostane programátor zdarma od hardwaru, kdežto **DLP a TLP musí vyjádřit v programu** — a tam začíná skutečná práce.

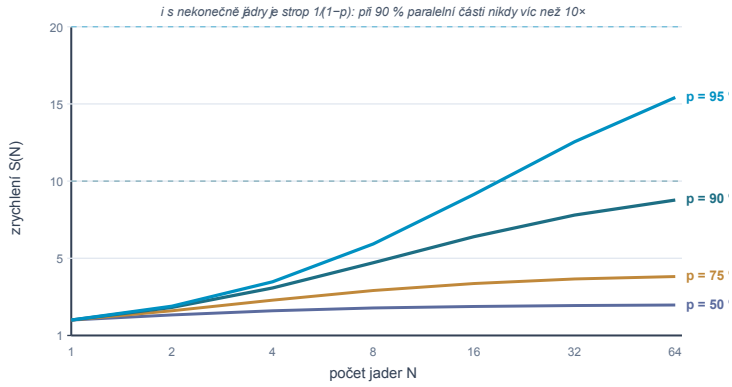
Flynnova taxonomie

Klasické rozdělení strojů podle počtu proudů instrukcí a dat. **SISD** je jeden proud instrukcí nad jedním proudem dat (klasické jednoduché jádro). **SIMD** (single instruction, multiple data) je jeden proud instrukcí nad mnoha daty (vektorová jednotka, DSP). **MIMD** je mnoho proudů instrukcí nad mnoha daty (multicore). GPU používá variantu zvanou **SIMT** (single instruction, multiple threads) — mnoho vláken vykonává tutéž instrukci, každé nad jinými daty. Taxonomie není samoúčelná: říká, jaký druh paralelismu která jednotka umí, a tím i jaké úlohy na ni patří.

Amdahlův a Gustafsonův zákon

Klíčové a limitující pravidlo paralelizace: **zrychlení omezuje sériová část programu**. Je-li podíl p programu paralelizovatelný a běží na N jádrech, je zrychlení dáno **Amdahlovým zákonem**:

$$S(N) = \frac{1}{(1-p) + \dfrac{p}{N}}$$



Obrázek 8.1. Amdahlův zákon: zrychlení podle počtu jader pro různý paralelní podíl. I s nekonečně mnoha jádry je strop dán sériovou částí — při 90 % paralelní práce nikdy víc než desetinásobek.

I s **nekonečně mnoha** jádry je strop $1/(1-p)$: při 90 % paralelní části nikdy víc než desetinásobek, při 95 % nejvýš dvacetinásobek. Malá sériová část (inicializace, vstup/výstup, kritická sekce pod zámkem) tedy brzy zastropuje zisk a další jádra už nepomohou.

Gustafsonův pohled a co paralelizaci brzdí

Amdahlův zákon předpokládá **pevnou** úlohu. **Gustafsonův zákon** ten pohled změkčuje: s víc jádry obvykle řešíme **větší** úlohu (víc dat), kde sériová část zůstává relativně malá, takže zisk roste lépe, než Amdahlův zákon pro pevnou úlohu předpovídá. Proto se vyplatí

DLP
Data-level parallelism: táž operace aplikovaná současně na mnoho datových prvků; ideální pro vektorové jednotky, SIMD a GPU.

TLP
Thread-level parallelism: nezávislá vlákna nebo úlohy běžící současně na více jádrech.

SIMT
Single instruction, multiple threads: mnoho vláken vykonává tutéž instrukci nad různými daty; programovací model grafických procesorů.

Amdahlovým zákonem
Zrychlení paralelního výpočtu je omezeno sériovou částí: i s nekonečně mnoha jádry nepřekročí převrácenou hodnotu sériového podílu.

Gustafsonův zákon
S rostoucím počtem jader obvykle řešíme větší úlohu (víc dat), kde sériová část zůstává malá, takže zisk roste lépe, než předpovídá Amdahl pro pevně velkou úlohu.

paralelizovat **velké, datově nezávislé** úlohy (obraz, signál, neuronové sítě), ne drobné řídicí smyčky.

Teoretický zisk navíc snižuje režie, kterou samotný Amdahlův zákon nezahrnuje: **synchronizace** (zámky, bariéry — čekání jednoho jádra na druhé), **komunikace** (předání dat mezi jádry, tedy drahý pohyb dat z oddílu 8.1), **soupeření o sdílenou paměť** a sběrnici (přednáška 4) a **nevyváženost** (nejpomalejší vlákno určuje konec). Špatně rozdělená úloha může na více jádrech běžet **pomaleji** než na jednom.

Granularita, homogenní a heterogenní paralelismus

Jak velké kusy práce přidělovat jádrům, je otázkou granularity. **Jemnozrnný** paralelismus (malé jednotky práce) se dobře vyvažuje, ale platí **vysokou režii** synchronizace. **Hrubozrnný** (velké nezávislé bloky) má malou režii, ale hůř se vyvažuje. Cíl je trefit kusy dost velké, aby se vyplatila režie, a dost malé, aby žádné jádro nezůstalo nevyužité.

Granularity
Velikost nezávislých kusů práce v paralelním programu: jemnozrnný dobře vyvažuje, ale má velkou režii synchronizace; hrubozrnný má malou režii, ale hůř se vyvažuje.

Konečně lze multicore stavět **homogenně** (stejná jádra, jeden druh kódu, jednoduchý model) nebo **heterogenně** (různé jednotky pro různé úlohy — výkonná a úsporná jádra, případně rovnou CPU + DSP + GPU + NPU). Embedded jde téměř výhradně cestou **heterogenní**, protože nejde o maximální výkon, ale o **nejlepší výkon na watt pro danou úlohu**. To je most do částí 8.4–8.6.

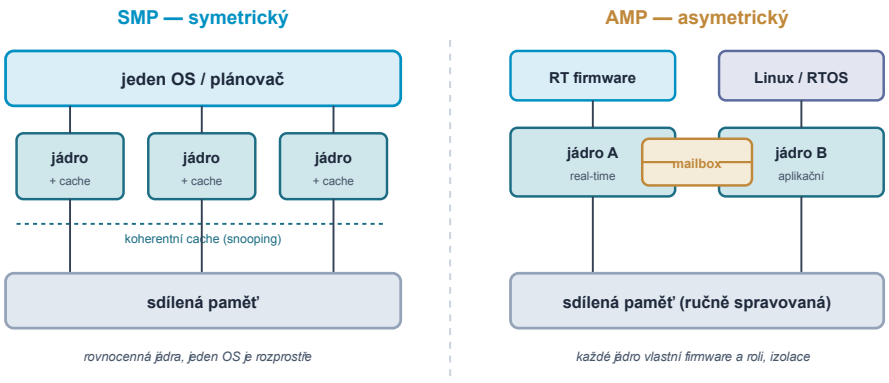
> **Shrnutí oddílu 8.2.** Paralelismus má tři úrovně: **ILP** (zajistí hardware), **DLP** (SIMD, GPU) a **TLP** (multicore); DLP a TLP musí vyjádřit programátor. **Amdahlův zákon** ukazuje, že sériová část zastropuje zisk (90 % paralelní práce nedá víc než desetinásobek); Gustafson to změkčuje pro větší úlohy. Zisk snižuje **synchronizace, komunikace, sdílení paměti a nevyváženost**. Embedded volí **heterogenní** přístup — nejvhodnější jednotka na danou úlohu.

8.3 Multicore v embedded

Přidání jader lze provést několika způsoby. Tato část porovná symetrický a asymetrický multiprocessing, ukáže dual-core mikrokontrolér a projde, co sdílení paměti obnáší — koherenci cache, synchronizaci a komunikaci mezi jádry.

SMP, AMP a big.LITTLE

Víc jader lze provozovat dvěma základními způsoby.



Obrázek 8.2. SMP versus AMP: vlevo jeden OS rozděluje vlákna na rovnocenná jádra se sdílenou koherentní pamětí; vpravo má každé jádro vlastní firmware a roli, jádra komunikují přes sdílenou paměť a mailbox.

SMP (symetrický multiprocessing) znamená **jeden operační systém** (Linux, SMP RTOS), který vidí všechna jádra, považuje je za **rovnocenná** a sám rozhoduje, kde vlákno poběží; jádra **sdílejí paměť** i periferie. Je to pohodlné, ale vyžaduje **koherentní cache** a hardwarovou podporu (snooping, zamykání) — typické pro aplikační procesory (Cortex-A), méně pro malé mikrokontroléry.

AMP (asymetrický multiprocessing) naproti tomu dává každému jádru **vlastní obraz** (firmware nebo RTOS) a **pevně danou roli**; jádra bývají **heterogenní** (výkonné a úsporné, nebo jádro pro real-time a jádro pro aplikaci) a komunikují přes **sdílenou paměť a mailbox**, ne přes společný plánovač. AMP vládne v embedded, protože dává **izolaci** (jedno jádro deterministické real-time, druhé, méně kritické aplikační) a předvídatelnost.

SMP
Symetrický multiprocessing: jeden operační systém vidí všechna rovnocenná jádra, sdílejí paměť i plánovač a OS sám rozhoduje, kde vlákno poběží.

AMP
Asymetrický multiprocessing: každé jádro má vlastní firmware nebo OS a pevnou roli; jádra komunikují přes sdílenou paměť a zprávy, ne přes společný plánovač.

Příklad: STM32H7 dual-core

Konkrétní heterogenní mikrokontrolér provozovaný v režimu AMP. **Cortex-M7** na 480 MHz je výkonné jádro s cache a FPU — počítá signálové zpracování a řídicí smyčky. **Cortex-M4** na 240 MHz je úsporné jádro — obsluhuje periferie, komunikaci a dohled. Jádra sdílejí SRAM a periferie a synchronizují se přes **HSEM** (hardwarové semafory) a **IPCC** (kanál pro mezijádrovou komunikaci). Rozdělení rolí je přirozené: M7 „počítá", M4 „obsluhuje", a navzájem se neruší.

HSEM
Hardware semaphore: blok hardwarových semaforů na STM32H7 pro zamykání sdílených prostředků mezi jádry.

IPCC
Inter-Processor Communication Controller: hardwarový kanál pro signalizaci a krátké zprávy mezi jádry na STM32H7.

big.LITTLE: výkon versus spotřeba

U aplikačních procesorů se používá heterogenní přístup se **stejnou ISA**: **big.LITTLE** kombinuje **velká** jádra (výkonná, energeticky náročná) pro špičkovou zátěž a **malá** jádra (úsporná, pomalejší) pro běžný a klidový provoz. Protože jádra sdílejí instrukční sadu, může úloha **migrovat** mezi nimi podle zátěže a stavu baterie. Princip je „nejlevnější jádro, které stačí" — v telefonu drtivou většinu času běží jen malá jádra. Je to varianta téhož kompromisu výkon/watt.

Big.LITTLE
Uspořádání jader stejné instrukční sady, kde velká výkonná jádra zvládají špičkovou zátěž a malá úsporná jádra běžný provoz; úloha může mezi nimi migrovat.

Sdílená paměť, koherence cache a false sharing

Jakmile víc jader sdílí paměť, vrací se **cache** z přednášky 4. Každé jádro má vlastní cache; když dvě jádra drží **tutéž adresu**, jejich kopie se mohou rozejít. Tomu brání **koherenční protokol** (například **MESI**): jádra se navzájem „odposlouchávají" (bus snooping) nebo používají adresář a zápis jednoho jádra zneplatní kopie ostatních. Je to **hardwarový** mechanismus, drahý na tranzistory i energii, proto ho malé mikrokontroléry často nemají a spoléhají na AMP s ručně spravovanou sdílenou pamětí.

Koherenční protokol
Hardwarový mechanismus, který zajistí, že zápis jednoho jádra zneplatní kopie téže adresy v cache ostatních jader, takže nikdo nečte zastaralá data.

V protokolu MESI je každý řádek cache v jednom ze čtyř stavů: **Modified** (jen já mám změněnou kopii, paměť je zastaralá), **Exclusive** (jen já mám kopii, shodnou s pamětí), **Shared** (kopii má víc jader, shodnou s pamětí) a **Invalid** (kopie je neplatná). Zápis do sdíleného nebo neplatného řádku pošle ostatním „invalidate" a přejde do Modified — tak se nikdy nepřečte zastaralá hodnota.

Koherence ovšem pracuje po **celých řádcích cache** (typicky 32–64 bajtů), což plodí tichou past zvanou **false sharing**: když dvě jádra zapisují do **různých** proměnných ležících náhodou ve **stejném** řádku, řádek se opakovaně přenáší mezi jejich cache, přestože logicky nesdílejí nic.

False sharing
Pokles výkonu, když dvě jádra zapisují do různých proměnných ve stejném řádku cache; řádek se pak zbytečně opakovaně přenáší mezi jejich cache, ač logicky nic nesdílejí.

```
struct { int a; int b; } shared;   // a a b leží ve stejném řádku cache
// Jádro 0 píše shared.a, Jádro 1 píše shared.b → řádek "ping-pong" mezi cache
```

Řešení je horké proměnné různých jader **zarovnat na samostatné řádky** cache.

Synchronizace a komunikace mezi jádry

Sdílená data potřebují, aby sekvence čtení–úprava–zápis proběhla **nedělitelně**. Cortex-M to staví na páru instrukcí **LDREX/STREX** (exclusive access):

```
// atomický inkrement bez zámku
do {
    v = __LDREXW(&counter);     // načti s "rezervací" adresy
    v = v + 1;
} while (__STREXW(v, &counter)); // ulož; selže (vrátí nenulu), když mezi tím někdo zasáhl → opakuj
```

LDREX/STREX
Exclusive load a store: LDREX načte hodnotu s rezervací adresy, STREX uloží jen tehdy, když adresu mezitím nikdo nezměnil, jinak selže a operace se opakuje. Základ atomik bez zámku.

STREX selže, pokud mezi LDREX a STREX adresu změnil jiný kontext, a smyčka operaci zopakuje. Na tomto páru stojí **spinlocky**, **mutexy** i atomika C11. **Spinlock** aktivně čeká (busy-wait) na uvolnění — levný pro krátké kritické sekce, ale pro dlouhé plýtvá takty; **mutex** (z RTOS) čekající vlákno uspí a hodí se pro delší sekce.

K tomu je nezbytná **paměťová bariéra** (DMB, DSB), která vynutí **pořadí** přístupů do paměti. Moderní jádra a sběrnice totiž přístupy kvůli výkonu **přeskupují**; bez bariéry by druhé jádro uvidělo zápisy v jiném pořadí, než v jakém je první jádro provedlo — což je klasická příčina vzácných, nereprodukovatelných chyb v paralelním kódu.

Paměťová bariéra
Instrukce (DMB, DSB), která vynutí pořadí přístupů do paměti; bez ní smí jádro nebo sběrnice přístupy kvůli výkonu přeskupit a druhé jádro by je vidělo v jiném pořadí.

Komunikace mezi jádry a rozdělení práce

Jádra si předávají data a události několika způsoby. **Sdílená paměť** je nejrychlejší — data leží v paměti viditelné oběma (pozor na koherenci a zarovnání). **Mailbox / IPCC** je hardwarový kanál pro krátké zprávy a události („mám pro tebe data v bufferu X"). **Message passing** kopíruje zprávy (žádná sdílená data, méně chyb souběhu, ale režie kopie). Pro proud dat mezi jádry slouží **kruhový buffer ve sdílené paměti** — tatáž logika producent–konzument a ping-pong jako u DMA z přednášky 5. I zde platí, že **vlastnictví bufferu** mezi jádry je stejná otázka jako u DMA: v každém okamžiku smí do bufferu psát právě jeden.

Typický vzor v dual-core embedded (AMP): **real-time jádro** provádí deterministické úlohy (regulační smyčka, motorové řízení, sběr z ADC přes DMA), **aplikační jádro** dělá „měkké" úlohy (síť, displej, logika, případně Linux), a mezi nimi je **sdílený buffer s mailboxem** — real-time jádro plní data, aplikační je zpracuje. Izolace zde znamená robustnost: pád aplikační části (síť, GUI) nesmí způsobit selhání real-time regulace, což AMP umožní, kdežto jeden velký SMP OS hůř.

> **Shrnutí oddílu 8.3.** **SMP** znamená jeden OS nad rovnocennými jádry se sdílenou koherentní pamětí (aplikační procesory); **AMP** dává každému jádru vlastní firmware a roli a vládne v embedded (STM32H7 M7+M4). **big.LITTLE** kombinuje velká a malá jádra téže ISA podle hesla „nejlevnější jádro, které stačí". Sdílení paměti žádá **koherenci cache** (MESI, snooping) v hardwaru; pozor na **false sharing**. Synchronizace stojí na **LDREX/STREX**, spinlocku/mutexu a **bariérách** (pořadí přístupů); komunikace probíhá přes sdílenou paměť a mailbox.

8.4 Specializovaný hardware

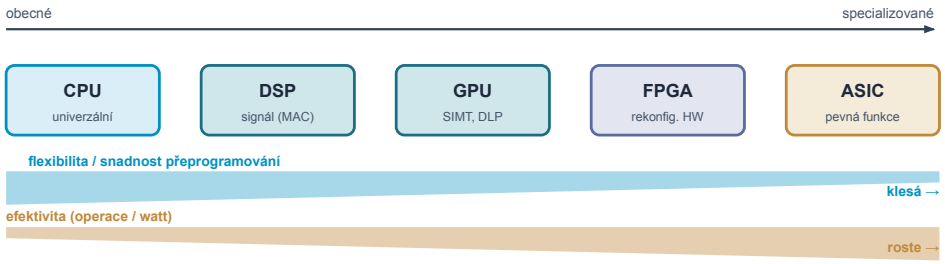
Druhou cestou k výkonu je specializovaný křemík. Tato část vysvětlí, proč specializace šetří energii, seřadí jednotky do spektra od univerzálních po pevné (CPU, DSP, GPU, FPGA, ASIC) a připomene, že skutečnou cenou není výpočet, ale pohyb dat.

Proč specializace funguje: spektrum flexibilita/efektivita

Obecné jádro platí daň za **univerzálnost**. U každé instrukce musí načíst a dekódovat operační kód, vyřešit závislosti, predikovat skoky a přesouvat data přes registry. Tato režie spotřebuje **většinu energie** — samotná aritmetika je jen zlomek. Specializovaná jednotka oproti tomu dělá **jednu věc**, režii odřízne a místo „načti–dekóduj–vykonej" u každé operace má **pevně zapojený datový tok**. Výsledkem jsou desítky až stovky násobků výkonu na watt pro danou úlohu. Princip celé oddílu 8.4 a 5 lze shrnout do věty: **provádět méně druhů operací, ale ty správné — a realizovat je pevně zapojené.**

Spektrum: flexibilita versus efektivita

Jednotky lze seřadit do spektra podle kompromisu mezi univerzálností a efektivitou.



Obrázek 8.3. Spektrum od univerzálních k specializovaným jednotkám: CPU, DSP, GPU, FPGA, ASIC. Vlevo snadná přeprogramovatelnost, vpravo více operací na watt, ale užší rozsah úloh.

Čím více vlevo, tím **univerzálnější** jednotka (snadno se přeprogramuje), čím více vpravo, tím **efektivnější** (víc operací na watt), ale **užší** v tom, co umí. Návrh systému znamená vybrat pro danou úlohu **správný bod** tohoto spektra.

DSP: počítání signálů

DSP (digital signal processor) je jádro optimalizované pro výpočet `suma += a[i]*b[i]` (filtry, FFT, regulace). Umí **MAC** (multiply-accumulate) v jednom taktu, často několik paralelně, pracuje ve **fixed-point** a saturující aritmetice (přednáška 7) a má speciální adresování (kruhové buffery, bit-reverse pro FFT) a SIMD. V jádrech Cortex-M4 a M7 je DSP „rozpuštěné" přímo do jádra: instrukce `SMLAD`, `SSAT`, `QADD` z přednášky 7 dělají DSP práci bez zvláštního čipu. Specializace tedy nemusí být samostatná jednotka — může být sada instrukcí.

DSP
Digital Signal Processor: jádro optimalizované pro operaci násobení s akumulací (MAC) nad proudy vzorků; má pevnou aritmetiku, saturaci a speciální adresování.

MAC
Multiply-accumulate: násobení s akumulací, základní operace signálového a maticového výpočtu; DSP a NPU jí provedou mnoho za takt.

GPU: masivní datový paralelismus

GPU (graphics processing unit) má stovky až tisíce jednoduchých jednotek, které v režimu SIMT provádějí **tutéž instrukci** nad různými daty. Je optimalizováno na **propustnost**, ne na latenci jednoho výsledku, a skvěle se hodí pro grafiku, obrazové operace a husté lineární algebry. Žádá ovšem **hodně paralelních dat**, jinak jednotky zůstanou nevyužité. V embedded jsou GPU u aplikačních SoC (telefon, infotainment) pro grafiku a někdy obecné výpočty; jsou energeticky náročnější a univerzálnější než NPU, a proto pro neuronové sítě často nestačí (oddíl 8.5).

GPU
Graphics Processing Unit: jednotka se stovkami až tisíci jednoduchých výpočetních drah, které v režimu SIMT provádějí tutéž instrukci nad různými daty; optimalizováno na propustnost.

Pevné funkce a FPGA

Na nejefektivnějším konci spektra leží **hardwired** bloky pro přesně danou operaci: video **kodeky** (H.264/H.265, JPEG) enkódují a dekódují zlomkem energie CPU, **kryptografické** bloky (AES, SHA, generátor náhody) šifrují linkovou rychlostí, **ISP** (image signal processor)

zpracovává proud ze senzoru kamery a i drobné akcelerátory (CRC, generátory PWM) patří sem. Nabízejí nulovou flexibilitu a maximální efektivitu — vyplatí se, když je úloha **stabilní a častá** (standard kodeku se nemění každý měsíc).

Mezi softwarem a pevným obvodem stojí **FPGA** — pole logických buněk, které lze zapojit do **libovolného obvodu**. Dosahuje efektivity blízké pevnému hardwaru, ale je **přeprogramovatelné**; hodí se pro malé série, vyvíjející se standardy a extrémně paralelní pevné pipeline. Cenou je nižší takt než ASIC, vyšší spotřeba a cena za kus a náročný návrh (v jazycích HDL).

FPGA
Field-Programmable Gate Array: pole logických buněk, které lze zapojit do libovolného obvodu; efektivita blízká pevnému hardwaru, ale přeprogramovatelná, za cenu nižšího taktu a náročného návrhu.

Skutečná cena: pohyb dat

Klíčové číslo pro celý zbytek přednášky je **relativní energie operace**. Osmibitové sčítání nebo MAC i čtení z registru stojí řádově jednotku; čtení z **on-chip SRAM** je řádově pětkrát až padesátkrát dražší; čtení z **off-chip DRAM** je řádově **dvě stě až tisíckrát** dražší než ta aritmetická operace. Spočítat něco je tedy **levné**; *dostat k tomu data* je drahé. Proto u akcelerátorů je rozhodující hlavně to, **kolik dat musí přečíst z paměti** — ne počet násobiček.

> **Shrnutí oddílu 8.4.** Obecné jádro platí energii za **univerzálnost** (fetch, dekódování, řízení); specializace tuto režii odřízne a získá desítky až stovky násobků operací na watt. Spektrum **CPU** → **DSP** → **GPU** → **FPGA** → **ASIC** jde od flexibilního k efektivnímu a úzkému. **DSP** (MAC, fixed-point) bývá rozpuštěné v jádře; **GPU** je SIMT propustnost; **pevné funkce** (kodeky, krypto) dávají maximum efektivity pro stabilní úlohy; **FPGA** je přeprogramovatelný hardware. Rozhodující veličinou je **pohyb dat**: čtení z DRAM stojí stonásobky proti samotnému výpočtu.

8.5 AI akcelerátory (NPU)

Nejnáročnějším paralelním úkolem dneška je inference neuronové sítě. Tato část ukáže, že se redukuje na maticové násobení, proč na to CPU ani GPU nestačí a jak systolické pole spolu s kvantizací na int8 dělá z NPU řádově úspornější stroj.

Co síť počítá a proč na to CPU/GPU nestačí

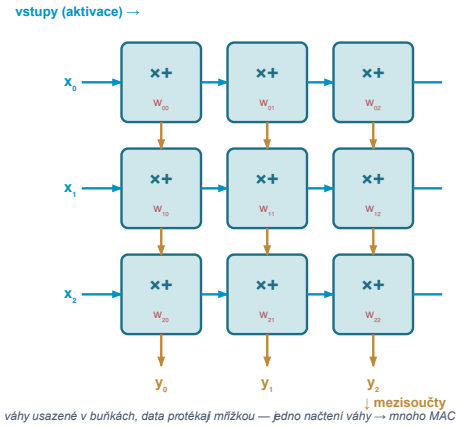
Pod kapotou je inference neuronové sítě z více než 90 % **maticové násobení** a **konvoluce** a obojí se rozpadá na jedinou operaci. Výstup neuronu je $y_j = f\left(\sum_i w_{ji} x_i + b_j\right)$ — tedy ohromné množství operací MAC `suma += váha · vstup`, následované nelineární funkcí f. Datový tok je velmi **pravidelný**, s minimem větvení, a nese velký **datový paralelismus** (DLP): tytéž operace nad spoustou dat. Síť tedy není „chytrá výpočetně"; je to **hora násobení a sčítání** s předvídatelným tokem — přesně to, co se dá specializovat.

Proč na to CPU a GPU nestačí

CPU má málo MAC jednotek a hodně režie na instrukci; toto obrovské množství operací počítá sekvenčně a energeticky draho. **GPU** zvládne paralelismus, ale je **univerzální a energeticky náročná**, takže pro baterii a okraj sítě je předimenzovaná. Obojí navíc **opakovaně načítá váhy a aktivace z paměti**, čímž naráží na cenu pohybu dat z oddílu 8.4. Protože je úloha extrémně pravidelná a paralelní, vyplatí se postavit jednotku ušitou přesně na MAC s minimem pohybu dat — **NPU** (neural processing unit).

NPU
Neural Processing Unit: akcelerátor specializovaný na maticové násobení a konvoluce neuronových sítí, postavený typicky na poli MAC jednotek s minimem přístupů do paměti.

Systolické pole
Mřížka MAC buněk, kterou data protékají v rytmu hodin; váhy zůstávají usazené v buňkách (weight-stationary), takže jedno načtení váhy poslouží mnoha operacím a šetří pohyb dat.



Obrázek 8.4. Systolické pole: váhy usazené v buňkách, vstupy (aktivace) vtékají zleva, mezisoučty stékají dolů. Jedno načtení váhy poslouží mnoha operacím MAC.

Systolické pole: jádro výpočtu NPU

Klíčová myšlenka NPU je **systolické pole** — mřížka MAC buněk, kterou **data protékají** v rytmu hodin, jako krev tepnami (odtud „systolické").

Váhy zůstanou **usazené** v buňkách, vstupy vtékají zleva a mezisoučty stékají dolů. Jedno načtení váhy tak poslouží **mnoha** operacím MAC, čímž se drasticky šetří pohyb dat.

Proč je systolické pole tak efektivní

Efektivita plyne z několika vlastností. Pole je **weight-stationary** — váha se načte jednou a použije se pro celý proud vstupů, takže se málo čte z paměti. Komunikace je **lokální** — buňka si předává data jen se sousedy, žádná dlouhá sběrnice. Paralelismus je **plný** — pole o velikosti $N \times N$ provede N^2 operací MAC každý takt. A pravidelnost znamená **žádnou řídicí režii** na buňku. Je to přesně realizace principu z oddílu 8.4: pevně zapojený datový tok, minimum přístupů do DRAM, maximum MAC na joule. Archetypem je Google **TPU** a „tensor cores" v moderních GPU.

Kvantizace a aritmetická intenzita

Sítě se obvykle učí ve `float32`, ale pro inferenci na okraji sítě se **kvantizují** na **int8** (osmibitová celá čísla). Váhy jsou pak **čtyřikrát menší** (tedy čtyřikrát méně dat z paměti, která je úzkým hrdlem), **int8 MAC** je mnohem levnější na křemík i energii než float a přesnost klesne jen málo — sítě jsou vůči kvantizaci robustní, kvantizuje-li se rozumně.

Int8
Kvantizace vah a aktivací z desetinných čísel float32 na osmibitová celá čísla; zmenší data čtyřikrát, zlevní MAC a sníží spotřebu, přesnost klesne jen málo.

```
// kvantovaný MAC: int8 váhy × int8 aktivace → int32 akumulátor
int32_t acc = 0;
for (i = 0; i < N; i++) acc += (int32_t)w[i] * (int32_t)x[i];
```

Proto mají NPU jednotky MAC pro int8 (a int4 nebo int16); float je v embedded inferenci spíš výjimka.

Compute-bound versus memory-bound

Aritmetická intenzita
Počet operací připadajících na jeden bajt načtený z paměti; nízká intenzita znamená, že úloha čeká na data (memory-bound), vysoká, že limituje počet výpočetních jednotek (compute-bound).

Zda poběží úloha na plný výpočetní výkon, nebo ji brzdí paměť, vystihuje pojem **aritmetická intenzita** — počet operací na jeden bajt načtený z paměti. Nízká intenzita (málo práce na bajt) znamená, že úloha je **memory-bound**: jednotky čekají na data a zůstávají nevyužité. Vysoká intenzita znamená **compute-bound**: limituje samotný počet MAC. Cílem návrhu NPU

i jeho programu je intenzitu **zvednout** — držet váhy a mezivýsledky on-chip, dávkovat (batch) a znovu používat data. Opět je ve výhodě ten, kdo méně přistupuje do DRAM.

Embedded NPU a programovací model

Edge AI

Inference neuronové sítě přímo v koncovém zařízení (na mikrokontroléru), bez odesílání dat do cloudu; přináší nízkou latenci, soukromí, nízkou spotřebu a funkci offline.

Edge AI neboli TinyML znamená inferenci přímo na mikrokontroléru, bez cloudu. **ARM Ethos-U** je malý akcelerátor (microNPU) vedle jádra Cortex-M (typ U55 nebo U65), pracuje v int8 a dosahuje stovek miliard operací za sekundu při jednotkách miliwattů. **STM32N6** integruje jádro Cortex-M55 s akcelerátorem Neural-ART přímo v mikrokontroléru. Klíčové frameworky jsou **TensorFlow Lite Micro** a knihovna **ARM CMSIS-NN**. Důvody inference na okraji jsou **latence** (žádná cesta do cloudu), **soukromí** (data neopustí čip), **energie** (neposílat data rádiem) a **dostupnost** (funguje offline) — typicky klasifikace zvuku, detekce gest, rozpoznání klíčového slova nebo detekce anomálií na senzoru.

Co NPU neumí a programovací model

NPU je úzká: umí pevnou sadu operací (konvoluce, plně propojené vrstvy, pooling, aktivace). Neobvyklou vrstvu musí **dopočítat CPU**, čímž data se opakovaně přenášejí mezi NPU a jádrem tam a zpět. Cesta modelu na čip má proto čtyři kroky: **natrénovat** (v PyTorch nebo TensorFlow, float32, na velkém stroji), **kvantizovat** na int8 a zkonvertovat (TFLite), **zkompilovat** pro daný NPU (nástroj výrobce namapuje vrstvy na hardware) a **nasadit** runtime (TFLite Micro nebo CMSIS-NN) na mikrokontrolér. Vývoj AI úlohy v embedded je tedy z velké části práce s **toolchainem a kvantizací**, ne ruční psaní MAC smyček.

> **Shrnutí oddílu 8.5.** Inference neuronové sítě je z více než 90 % **MAC** (maticové násobení a konvoluce): pravidelný, masivně paralelní datový tok. CPU a GPU jsou pro to neefektivní (režie, žravost, pohyb dat), proto vznikla **NPU** ušitá na MAC. **Systolické pole** (weight-stationary, lokální komunikace, N^2 MAC za takt) je archetypem (TPU, tensor cores) a minimalizuje pohyb dat. **Kvantizace na int8** šetří paměť i energii; **embedded NPU** (Ethos-U, STM32N6, TinyML, TFLite Micro) přináší inferenci na okraj sítě.

8.6 Syntéza: heterogenní SoC

Zbývá spojit víc jader i specializaci do jednoho čipu. Tato závěrečná část ukáže heterogenní SoC jako „město jednotek", mapování úlohy na nejvhodnější z nich a to, že úzkým hrdlem není výpočet, ale pohyb dat mezi nimi.

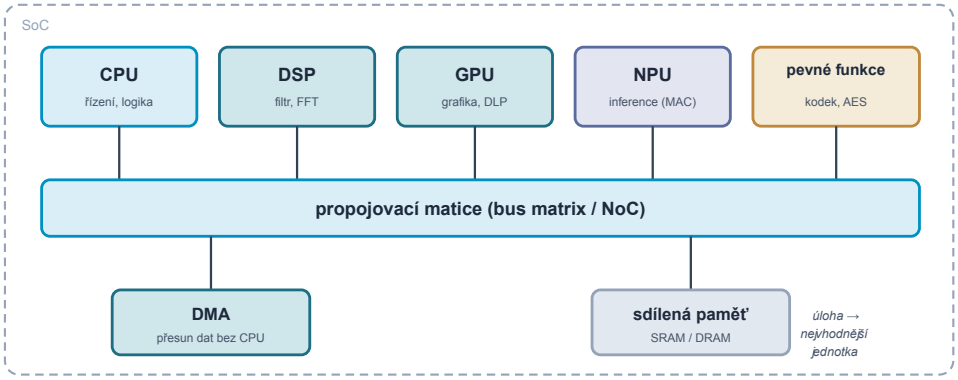
Všechno na jednom čipu a mapování úlohy

Moderní embedded čip není „procesor" — je to **město jednotek**, z nichž každá počítá jiný druh práce.

CPU (řízení), DSP (signál), GPU (grafika), NPU (inference) a pevné funkce (kodek, krypto) jsou propojené **bus matrix** z přednášky 4 a obsluhované **DMA** z přednášky 5.

Mapování úlohy na jednotku

Inženýrské rozhodnutí spočívá v tom, že pro každou část úlohy vybereme **nejvhodnější jednotku**: řídicí logiku a stavové automaty počítá **CPU**, filtr či FFT **DSP** (nebo CPU s DSP instrukcemi), násobení matic a inferenci **NPU**, paralelní obraz a grafiku **GPU** a video kodek či AES **pevná funkce**. Je to tatáž logika jako u big.LITTLE (oddíl 8.3) — „nejlepší jednotka, která úlohu zvládne za nejméně energie" — jen napříč **druhy** jednotek, ne jen mezi velkými a malými jádry.



Obrázek 8.5. Heterogenní SoC: CPU pro řízení, DSP pro signál, GPU pro grafiku, NPU pro inferenci a pevné funkce (kodek, krypto), propojené bus matrix a obsluhované DMA nad sdílenou pamětí.

Úzké hrdlo není výpočet, ale data

Když je výpočetních jednotek dost, limit se přesune jinam — na **pohyb dat mezi nimi** (oddíl 8.4). Data ze senzoru putují přes DMA do bufferu, odtud do NPU, výsledek zpět k CPU; každý přesun přes paměť stojí energii i čas (paměťová zeď, přednáška 4). **Koherence** a **vlastnictví bufferu** mezi CPU a akcelerátorem jsou přitom tatáž otázka jako DMA versus cache z přednášky 5. Návrh heterogenního systému je proto z velké části **návrh datového toku**: kdo kam zapisuje, kdo vlastní buffer a jak se minimalizují kopie. Výpočet je „vyřešený", data ne.

Programovací model a izolace

Celek řídí **CPU jako dirigent**: spouští úlohy na akcelerátorech (přes deskriptory a fronty příkazů), sama dělá řízení a „lepidlo" mezi jednotkami. Akcelerátory běží **asynchronně**; CPU mezitím dělá něco jiného a na dokončení čeká přes **přerušení** (přednáška 5). Ovladače a runtime (CMSIS-NN, SDK výrobce) skrývají detaily před aplikací. Vzor je tentýž jako DMA z přednášky 5, jen „periferií" je celá výpočetní jednotka: nastav, spusť, dělej něco jiného, zpracuj přerušení.

Izolace a bezpečnost napříč jednotkami

Když na čipu sdílí paměť víc jader a akcelerátorů, je nutné je **oddělit**. MPU nebo MMU zpřístupní každému jádru nebo úloze jen jeho část paměti, TrustZone oddělí bezpečný a nezabezpečený svět (klíče, krypto) a firewally sběrnice brání akcelerátoru v přístupu do oblastí, k nimž nemá oprávnění. Heterogenita zvyšuje výkon, ale i počet cest, kudy může chyba nebo útok prosáknout; izolace v hardwaru je proto součást návrhu, ne dodatek.

MPU
Memory Protection Unit: jednotka, která každému jádru nebo úloze zpřístupní jen jeho část paměti; brání tomu, aby chyba v jedné části poškodila jinou.

TrustZone
Hardwarové rozdělení na bezpečný a nezabezpečený svět; klíče a kryptografie běží odděleně od běžné aplikace.

Závěr: proč koexistence

Oblouk z přednášky 7 se uzavírá. Žádná jednotka není „nejlepší" — každá je nejlepší **na něco**. Jedno jádro nezrychlíme (oddíl 8.1), tedy přidáváme **víc jader** (oddíl 8.2 a 3) a **specializaci** (oddíl 8.4 a 5). Univerzálnost (CPU) a efektivita (NPU, pevné funkce) jsou **protiklady**, a proto musí být **vedle sebe**. Moderní embedded SoC je tak **heterogenní z nutnosti**, ne z módy. Inženýrství dneška není „vybrat procesor", ale **poskládat správné jednotky a navrhnout datový tok mezi nimi** — pod tvrdým rozpočtem operací na watt.

Shrnutí oddílu 8.6. Moderní embedded čip je **heterogenní SoC** — CPU, DSP, GPU, NPU a pevné funkce na jednom křemíku, propojené bus matrix a obsluhované DMA. Každou úlohu řešíme na **nejvhodnější jednotce**; úzkým hrdlem není výpočet, ale **pohyb dat**, takže návrh systému je z velké části návrh datového toku. CPU funguje jako **dirigent** asynchronních akcelerátorů; izolaci (MPU, TrustZone) je nutné řešit v hardwaru. Koexistence plyne z toho, že univerzálnost a efektivita jsou protiklady — a moderní čip potřebuje obojí.

8.7 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 8.1 — Konec škálování jednoho jádra

1. Vysvětlete rozdíl mezi Mooreovým zákonem a Dennardovým škálováním; které z nich skončilo a s jakým důsledkem?
2. Odvoďte z $P \approx CV^2 f$, proč nelze libovolně zvyšovat takt.
3. Proč nepomohlo místo taktu dál zvyšovat ILP jednoho jádra?
4. Co znamená, že výkon i energii dnes určuje „pohyb dat", a proč je to v embedded ještě naléhavější?

K oddílu 8.2 — Paralelismus: principy a meze

1. Vysvětlete rozdíl mezi ILP, DLP a TLP a uveďte, kdo každý z nich zajišťuje.
2. Odvoďte z Amdahlova vztahu strop zrychlení pro $p = 0{,}9$.
3. Jak Gustafsonův pohled mění závěr Amdahlova zákona a pro jaké úlohy?
4. Proč skutečné zrychlení zaostává za teoretickým stropem podle Amdahlova zákona?
5. Co je granularita a jaký kompromis představuje?

K oddílu 8.3 — Multicore v embedded

1. Porovnejte SMP a AMP — kdo řídí jádra, jak sdílejí paměť a kam který patří.
2. Jak protokol cache koherence (MESI) zabrání tomu, aby jádro četlo zastaralá data z cache?
3. Co je false sharing a jak mu předejít?
4. Jak pár LDREX/STREX realizuje atomický inkrement bez zámku a proč může selhat?
5. Proč je v paralelním kódu nutná paměťová bariéra?

K oddílu 8.4 — Specializovaný hardware

1. Proč obecné jádro spotřebuje na typickou operaci víc energie než specializovaná jednotka?
2. Seřaďte CPU, DSP, GPU, FPGA a ASIC podle kompromisu flexibilita versus efektivita.
3. Proč je DSP na Cortex-M4 „rozpuštěné do jádra" a co to znamená?
4. Kdy se vyplatí pevná funkce a kdy raději FPGA?
5. Proč je u akcelerátorů rozhodující hlavně počet přístupů do paměti?

K oddílu 8.5 — AI akcelerátory (NPU)

1. Na jakou základní operaci se redukuje inference neuronové sítě a proč je to vhodné pro specializaci?
2. Proč jsou CPU i GPU pro inferenci na okraji sítě neideální?
3. Vysvětlete princip systolického pole a proč je weight-stationary úsporný.
4. Co získá a co ztratí kvantizace na int8?
5. Proč je inference neuronové sítě na NPU řádově úspornější než na CPU a jakou roli v tom hraje kvantizace?

K oddílu 8.6 — Heterogenní SoC

1. Podle čeho mapujete části úlohy na jednotky (CPU, GPU, NPU, DSP)? Uveďte tři příklady.
2. Proč se v heterogenním systému s dostatkem výpočetních jednotek stává úzkým hrdlem pohyb dat mezi nimi, a ne samotný výpočet?
3. Popište programovací model „CPU jako dirigent".
4. Proč je nutná izolace (MPU, TrustZone) a co se stane bez ní?

9

RTOS: souběžnost, plánování a vlastnictví zdrojů

> Tato kapitola řeší, **jak spolehlivě zvládnout víc úloh současně — deterministicky a včas — a kdy na to stačí superloop a kdy už je potřeba operační systém reálného času (RTOS)**. Jedno jádro vykonává v každém okamžiku jen jedinou činnost, takže „současně" znamená rychle se střídat tak, aby žádná činnost nezmeškala svůj termín.

Obsah kapitoly

9.1 Proč RTOS: souběžnost, determinismus, včasnost
9.2 Úlohy a čas
9.3 Plánovatelnost
9.4 Architektury: od superloopu k preemptivnímu multitaskingu
9.5 Synchronizace a komunikace
9.6 Postavme si RTOS na Cortex-M
9.7 Syntéza
9.8 Kontrolní otázky

9.1 Proč RTOS: souběžnost, determinismus, včasnost

Vestavný systém málokdy plní jen jedinou funkci. Tato část ukáže, co znamená zvládnout víc činností současně na jednom jádře, proč „real-time" znamená včasnost, ne rychlost, a jak se souběžnost rozpadá na tři samostatné problémy, které RTOS řeší.

Víc věcí „najednou" a co znamená real-time

Typický regulátor motoru musí současně každou 1 ms přečíst proud a napětí a přepočítat regulační smyčku, každých 10 ms poslat stav po CAN nebo UART, kdykoli zareagovat na povel z nadřazeného systému a průběžně blikat stavovou LED a hlídat teplotu. Každá z těchto činností má **vlastní tempo** a **vlastní termín**. Na jednom jádře je nelze provádět skutečně paralelně; provádějí se tak, že se **rychle střídají** — a celé umění je zařídit, aby se to střídání dělo tak, že žádná činnost nezmeškává svůj termín ani tehdy, když je zbytek systému právě zatížen.

Real-time
Systém reálného času: správnost výsledku závisí nejen na jeho hodnotě, ale i na tom, zda přišel do stanoveného termínu (deadline). Nejde o rychlost, ale o zaručenou včasnost.

Co znamená „real-time"

Systém reálného času není systém *rychlý*, ale **včasný**. Real-time znamená, že výsledek je správný jen tehdy, přijde-li **do daného termínu** (deadline). Podle toho, co se stane při zmeškání termínu, dělíme systémy na dvě třídy:

- **hard real-time** — zmeškaný termín znamená **selhání systému** (airbag, řízení motoru, brzdový systém ABS); zde musí být včasnost matematicky zaručena,
- **soft real-time** — zmeškaný termín **zhorší kvalitu**, ale systém běží dál (přehrávání zvuku, uživatelské rozhraní); občas vynechaný snímek nikoho neohrozí.

Klíčová veličina proto není průměrná rychlost, ale **nejhorší možný případ**: garance, že i za nejnepříznivějších okolností se úloha stihne. Přesně proto nás dále bude zajímat nejhorší doba běhu (WCET) a formální plánovatelnost — průměr nestačí, protože katastrofa se stane právě v tom jednom nejhorším okamžiku.

Klasifikace úloh

Úlohy systému reálného času dělíme podle toho, **kdy** a **jak často** se aktivují. **Periodická** úloha se opakuje s pevnou periodou — typicky regulační smyčka nebo pravidelné čtení senzoru. **Aperiodická** úloha reaguje na vnější událost, jejíž okamžik dopředu neznáme (povel od obsluhy). Sporadická úloha je zvláštní případ aperiodické úlohy s **garantovaným minimálním odstupem** mezi aktivacemi — a právě ten odstup dovolí zahrnout ji do analýzy plánovatelnosti. Sada úloh a jejich priorit navíc může být buď **statická** (pevně dána při návrhu), nebo **dynamická** (mění se za běhu). Většina embedded úloh je periodická — a právě pro ně existuje solidní teorie, kterou rozebereme v oddílu 9.3.

Sporadická úloha
Aperiodická úloha, u níž je zaručen minimální časový odstup mezi dvěma aktivacemi. Díky této dolní mezi periody ji lze zahrnout do analýzy plánovatelnosti jako periodickou s nejhorší periodou.

Tři vrstvy souběhu

Než sáhneme po RTOS, je užitečné rozložit „souběžnost" na tři samostatné problémy. Prvním je **správa systémových zdrojů** — kdo smí kdy na sběrnici, do sdíleného bufferu nebo k periferii; řeší ji vzájemné vyloučení, tedy mutex (oddíl 9.5). Druhým je **plánování v čase** — které úloze právě patří procesor; to je práce plánovače (oddíl 9.4). Třetím je **komunikace** — jak si úlohy předají data, aniž by se navzájem přepsaly; k tomu slouží fronty zpráv (oddíl 9.5). RTOS je software, který tyto tři aspekty souběhu zabalí do znovupoužitelných primitiv (úloha, mutex, semafor, fronta, časovač) — místo aby si je každý projekt psal znovu a pokaždé o něco hůř.

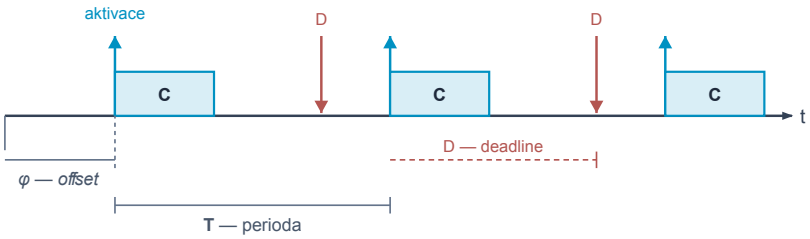
> **Shrnutí oddílu 9.1.** Vestavný systém dělá víc činností s různými periodami a termíny a jedno jádro je zvládá **rychlým střídáním**. Real-time znamená **včasnost**, ne rychlost: správný výsledek přijde **do termínu**; hard real-time = selhání při zmeškání, soft = zhoršení kvality. Rozhoduje **nejhorší případ**, ne průměr. Úlohy jsou hlavně **periodické** (příp. sporadické) a souběh má tři vrstvy — **zdroje, plánování, komunikace** — které RTOS zabalí do primitiv.

9.2 Úlohy a čas

Než začneme plánovat, potřebujeme úlohy popsat čísly. Tato část zavede časový model periodické úlohy, vysvětlí, proč o plánovatelnosti rozhoduje nejhorší doba běhu (WCET), a ukáže, kde přestává stačit prostý superloop.

Model úlohy, časové parametry a WCET

Periodickou úlohu popisují čtyři čísla. **Doba běhu C** je čas, který úloha potřebuje na jednu instanci (v nejhorším případě jde o WCET). **Perioda T** je odstup mezi dvěma po sobě jdoucími aktivacemi. **Deadline D** je okamžik, dokdy musí být instance hotová (velmi často platí `D = T`, tj. instance musí skončit dřív, než přijde další). **Offset φ** je počáteční zpoždění první aktivace, kterým lze úlohy vzájemně fázově rozprostřít. Úloha se pak aktivuje v časech `φ, φ+T, φ+2T, …` a každá instance musí doběhnout do svého termínu.



Obrázek 9.1. Časový model periodické úlohy: aktivace v časech φ, φ+T, …; každá instance běží dobu C a musí skončit do svého deadline D. Offset φ je počáteční fázové posunutí, perioda T je odstup aktivací.

WCET a proč je těžký

WCET je nejdelší možná doba běhu úlohy přes všechny vstupy, cesty kódem a stavy hardwaru — a přesně tato hodnota, nikoli průměr, rozhoduje o plánovatelnosti. Určit ji je překvapivě těžké, protože ji ovlivňuje **větvení** (která větev se vezme), **výpadky cache** (přednáška 4: přístup do pomalé paměti trvá mnohem déle než zásah do cache), **čekací stavy** paměti a **přerušení**, která běh úlohy protáhnou. WCET se měří profilerem nebo pomocí čítače taktů `DWT->CYCCNT` (přednáška 3), nebo se odhaduje staticky analýzou nejdelší cesty. Pro hard real-time musí jít o **horní mez**, ne o naměřený průměr — jinak garance neplatí.

WCET
Worst-case execution time, nejhorší doba běhu: nejdelší možná doba jedné instance úlohy přes všechny vstupy a stavy. Právě WCET, nikoli průměr, rozhoduje o plánovatelnosti hard real-time systému.

Superloop a jeho meze

Nejjednodušší architektura je nekonečná smyčka, která postupně obslouží všechny činnosti:

```
while (1) {
    read_sensors();     // A
    control_step();     // B
    send_status();      // C
    update_display();   // D
}
```

Dokud jsou úlohy krátké a je jich málo, funguje to výborně a nepotřebujete žádný RTOS. Superloop ale naráží na meze ve chvíli, kdy se objeví **dlouhá nebo blokující operace**: pomalý přenos po sběrnici, čekání na odpověď ze sítě nebo překreslení displeje zablokuje **celou smyčku** a regulační krok se zpozdí. Druhým problémem je **jitter** (kolísání okamžiku, kdy se úloha spustí): doba jednoho oběhu smyčky se mění podle toho, jak byly ostatní činnosti právě zatíženy, takže regulační smyčka neběží přesně každou 1 ms, ale „někdy kolem". Třetí slabinou je **absence priorit**: v superloopu mají všechny činnosti stejnou váhu, takže naléhavá úloha musí čekat, až na ni ve smyčce přijde řada. Částečně pomůže vzor **foreground/background** z přednášky 3 — rychlé věci do přerušení (foreground), zbytek do smyčky (background) — ale jakmile roste počet úloh s různými termíny, přestává i tento vzor stačit. To je bod, kde se vyplatí preemptivní plánovač.

Shrnutí oddílu 9.2. Periodickou úlohu popisují čtyři čísla: **C** (doba běhu, resp. WCET), **T** (perioda), **D** (deadline, často `D = T`) a **φ** (offset). O plánovatelnosti rozhoduje **WCET**, který je těžké určit kvůli větvení, cache a přerušením. **Superloop** je jednoduchý, ale selhává na blokujících operacích, jitteru a absenci priorit.

9.3 Plánovatelnost

Máme-li úlohy popsané, můžeme se ptát, zda se všechny stihnou. Tato část zavede utilizaci, mez plánovatelnosti pro rate-monotonic přiřazení priorit a přesnější analýzu doby odezvy, a porovná statické priority s dynamickým EDF.

Utilizace, rate-monotonic a mezní scénáře

Základní mírou zatížení je **utilizace** — podíl času, který procesor stráví danou úlohou. Pro úlohu s dobou běhu C a periodou T je to `C/T`; celková utilizace sady úloh je součet

```
U = Σ (C_i / T_i)
```

Nutná (nikoli však vždy postačující) podmínka plánovatelnosti je $U \leq 1$: nelze naplánovat práci, která by dohromady vyžadovala víc než 100 % procesoru. Otázka zní, kde přesně je mez, za kterou už plánovač nedokáže zaručit dodržení všech termínů — a ta mez závisí na použité strategii přidělování priorit.

Rate-monotonic a Liuova–Laylandova mez

Nejpoužívanější statická strategie je **rate-monotonic** plánování (RMS): **kratší perioda znamená vyšší prioritu**. Liu a Layland v roce 1973 dokázali, že sada `n` periodických úloh je při RMS plánovatelná, pokud

Rate-monotonic
Rate-monotonic scheduling (RMS): statické přiřazení priorit, kde vyšší prioritu dostane úloha s kratší periodou (vyšší frekvencí). Pro periodické úlohy s deadline rovným periodě je to prokazatelně optimální pevné přiřazení priorit.

```
U ≤ n · (2^(1/n) − 1)
```

Pravá strana s rostoucím `n` klesá: pro `n = 1` je to 1,0, pro `n = 2` přibližně 0,828, pro `n = 3` zhruba 0,780, a v limitě klesá k `ln 2`, tedy asi **0,693**. Praktické čtení je střízlivé: se statickými prioritami je bezpečné plánovat zhruba do **69 % utilizace**; nad touto hranicí *může* sada být plánovatelná, ale test to už nezaručí a je třeba přesnější analýza (například analýza doby odezvy). Test je **postačující, nikoli nutný** — sada nad mezí klidně může termíny stíhat, jen o tom tato jednoduchá nerovnost nerozhodne.

Čtyři scénáře

Chování mezní podmínky si ujasníme na čtyřech situacích. **(1)** Nízká utilizace hluboko pod mezí: sada je plánovatelná s velkou rezervou. **(2)** Utilizace těsně pod mezí `n·(2^(1/n)−1)`: sada je zaručeně plánovatelná, ale bez rezervy — každé prodloužení WCET ji ohrozí. **(3)** Utilizace mezi mezí a jedničkou: test *nerozhodne*; sada může, ale nemusí být plánovatelná a je nutná přesnější analýza. **(4)** Utilizace nad jedničkou: sada je **neplánovatelná** za jakékoli strategie, protože souhrnná práce přesahuje kapacitu procesoru — pomůže jen zrychlit kód, prodloužit periody, nebo přidat jádro (přednáška 8).

Rate-monotonic versus EDF

RMS je strategie se **statickými** prioritami (přiřazenými jednou při návrhu). Alternativou jsou **dynamické** priority podle blížícího se termínu — **EDF** (Earliest Deadline First), kde nejvyšší prioritu má vždy úloha s **nejbližším deadline**. EDF je na jednom jádře optimální: zvládne každou sadu až do utilizace **100 %**, tedy víc než RMS. Za to platí složitějším plánovačem

EDF
Earliest Deadline First: dynamické plánování, kde nejvyšší prioritu má vždy úloha s nejbližším termínem. Pro jedno jádro je optimální — dokáže naplánovat každou sadu s utilizací do 100 procent.

(priority se počítají za běhu) a horším chováním při přetížení — když se sada nevejde, EDF může „selhat naráz" a začít zmeškávat termíny domino efektem, zatímco u RMS selhává předvídatelně nejdřív úloha s nejnižší prioritou. Většina praktických RTOS proto volí **statické priority** ve stylu RMS: jsou jednodušší, levnější a jejich chování při přetížení se lépe předvídá.

Přesný test: analýza doby odezvy a deadline-monotonic

Liuova–Laylandova mez je jen **postačující**: pod ní je záruka, nad ní test nerozhodne, i když sada může být plánovatelná. Nutnou **i** postačující odpověď dá **analýza doby odezvy** (RTA). Nejhorší doba odezvy úlohy `i` je součet její vlastní doby běhu a **interference** od úloh s vyšší prioritou:

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

kde `hp(i)` jsou úlohy s vyšší prioritou. Rovnice je implicitní (`R_i` je na obou stranách), a proto se řeší **iterací** od `R_i^{(0)} = C_i`, dokud se hodnota neustálí; strop `⌈R_i/T_j⌉` udává, kolikrát úloha `j` úlohu `i` během okna `R_i` preemptne. Sada je plánovatelná, právě když pro každou úlohu vyjde `R_i ≤ D_i`. Přidá-li se blokování sdíleným zdrojem (priority inversion), přičte se do součtu člen `B_i`.

Analýza doby odezvy
Response Time Analysis (RTA): přesný test plánovatelnosti s pevnými prioritami. Pro každou úlohu spočítá nejhorší dobu odezvy jako součet vlastní doby běhu a interference od všech úloh s vyšší prioritou; sada je plánovatelná, právě když u každé úlohy vyjde doba odezvy nejvýš rovna jejímu deadlinu.

RTA zároveň umožňuje pracovat s úlohami, kde je **deadline kratší než perioda** (`D_i < T_i`) — což je běžné u řídicích smyček, jež musí dokončit dřív, než přijde další vzorek. Pro takový případ je optimálním pevným přiřazením priorit **deadline-monotonic** (kratší deadline = vyšší priorita); rate-monotonic je jeho speciální případ pro `D = T`. Utilizační mez pro `D < T` neplatí, záruku dá právě analýza doby odezvy.

Deadline-monotonic
Deadline Monotonic (DM): přiřazení pevných priorit podle deadlinu — kratší deadline znamená vyšší prioritu. Pro úlohy s deadlinem nejvýš rovným periodě je optimální mezi všemi pevnými přiřazeními; rate-monotonic je jeho speciální případ pro deadline rovný periodě.

> **Shrnutí oddílu 9.3.** Zatížení měříme **utilizací** `U = Σ (C_i/T_i)`; nutně musí být `U ≤ 1`. **Rate-monotonic** (kratší perioda = vyšší priorita) je optimální statické přiřazení a je bezpečné zhruba do `U ≈ 0,69` (Liuova–Laylandova mez `n·(2^(1/n)−1)`). Nad mezí test nerozhodne. **Analýza doby odezvy** `R_i = C_i + Σ ⌈R_i/T_j⌉ C_j` (iterace, `R_i ≤ D_i`) je nutná i postačující a zvládne i `D < T`, kde je optimální **deadline-monotonic**. **EDF** (dynamické priority) zvládne až `U = 1`, ale je složitější a hůř se chová při přetížení. RTOS obvykle volí statické priority.

9.4 Architektury: od superloopu k preemptivnímu multitaskingu

Souběh lze řešit několika architekturami. Tato část je seřadí od nejjednodušší po nejmocnější, vysvětlí zásadní rozdíl mezi kooperativním a preemptivním plánováním a projde životní cyklus úlohy a přepnutí kontextu.

Spektrum architektur: kooperativní vs. preemptivní

Souběh lze řešit několika způsoby, které tvoří spektrum od nejjednoduššího po nejmocnější: **cyklické dotazování** (superloop bez přerušení, jednoduché, ale se špatnou odezvou), **řízení přerušením** (obsluha událostí v ISR plus smyčka), **foreground/background** (rychlé ISR jako foreground a smyčka jako background, vzor z přednášky 3), **kooperativní multitasking** (úlohy se samy vzdávají procesoru) a **preemptivní multitasking** (plánovač procesor odebere podle priority). Čím dál v tomto výčtu, tím lepší garance včasnosti — za cenu složitějšího jádra, které musí umět plánovat a přepínat kontext.

Kooperativní versus preemptivní

Zásadní dělicí čára vede podle toho, **kdo rozhoduje o odebrání procesoru**. V **kooperativním** modelu běží úloha, dokud se procesoru **sama** nevzdá (voláním `yield()`) nebo se nezablokuje. To je jednoduché a bezpečné, protože k přepnutí dojde jen v předvídatelných bodech, ale jediná dlouhá nebo zaseknutá úloha zablokuje celý systém. V **preemptivním** modelu může plánovač úlohu **preemptovat** kdykoli — typicky na tiku SysTicku — a přepnout na připravenou úlohu s vyšší prioritou. Tím zaručí odezvu i proti úloze, která se procesoru sama nevzdá„ ale za cenu nutné synchronizace sdílených dat (oddíl 9.5). Real-time garance dává až preemptivní plánování s prioritami; jemu se věnuje zbytek přednášky.

Preemptovat
Preempce: nedobrovolné odebrání procesoru běžící úloze plánovačem (typicky na tiku časovače) ve prospěch připravené úlohy s vyšší prioritou. Umožňuje garantovat odezvu i proti úloze, která se procesoru sama nevzdá.

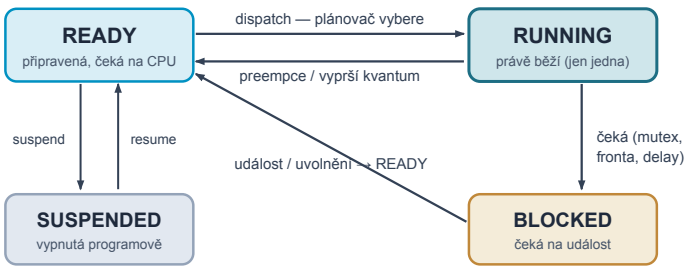
Preemptivní plánování podle priorit

Pravidlo preemptivního plánovače je jednoduché: **vždy běží připravená úloha s nejvyšší prioritou**. Po každé události (tik časovače, odblokování úlohy) plánovač vybere nejvyšší připravenou úlohu. Mezi úlohami **stejné priority** se procesor střídá metodou **Round-Robin**: každá dostane časové kvantum a po jeho vypršení se předá další. Úloha s nižší prioritou běží jen tehdy, když všechny vyšší nemají co dělat (jsou blokované). Priority tak nesou návrhový záměr: regulační smyčka s tvrdým termínem dostane vyšší prioritu než logování. Rate-monotonic z oddílu 9.3 přitom říká, **jak** je přiřadit — kratší perioda znamená vyšší prioritu.

Životní cyklus úlohy a přepnutí kontextu

Úloha se v RTOS pohybuje mezi několika stavy. Ve stavu **READY** je připravená a čeká jen na procesor; ve stavu **RUNNING** právě běží (na jednom jádře může být v tomto stavu vždy jen jedna úloha); ve stavu **BLOCKED** čeká na událost (uvolnění mutexu, data ve frontě, vypršení `delay`); ve stavu **SUSPENDED** je programově vypnutá. Plánovač vybírá vždy jen z úloh ve stavu READY.

Plánovač vybírá vždy z úloh ve stavu READY.

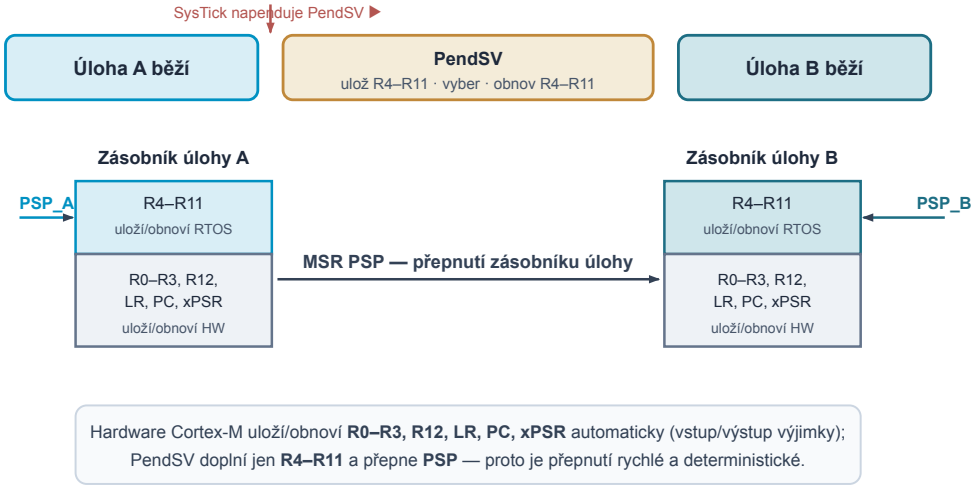


Obrázek 9.2. Životní cyklus úlohy v RTOS: plánovač vybírá z úloh READY, běžící úloha se může vrátit do READY (preempce), zablokovat (BLOCKED) nebo být programově pozastavena (SUSPENDED). Přechod BLOCKED→READY může vyvolat preempci.

Přechod z BLOCKED do READY (například po uvolnění mutexu) je důležitý: pokud právě odblokovaná úloha má vyšší prioritu než ta běžící, vyvolá **preempci** a okamžitě dostane procesor.

Přepnutí kontextu

Každá úloha má **vlastní zásobník**. Přepnutí mezi úlohami znamená uložit kompletní stav běžící úlohy a obnovit stav té nové: uloží se pracovní registry a ukazatel zásobníku PSP běžící úlohy, plánovač vybere novou úlohu z READY listu a obnoví se **její** PSP a registry — jádro pak pokračuje přesně tam, kde ta úloha naposledy skončila.



Obrázek 9.3. Přepnutí kontextu mezi dvěma úlohami: hardware Cortex-M uloží a obnoví část registrů automaticky při vstupu a výstupu z výjimky, RTOS doplní zbytek (R4 až R11) a přepne ukazatel zásobníku PSP mezi zásobníky obou úloh.

Podstatné je, že **část kontextu uloží hardware Cortex-M sám** při vstupu do výjimky (přednáška 1): registry `R0`–`R3`, `R12`, `LR`, `PC` a `xPSR`. RTOS proto musí ručně uložit a obnovit jen zbývající registry `R4`–`R11` a přepnout PSP. Přepnutí musí být rychlé a deterministické — jak přesně se dělá, ukáže oddíl 9.6.

> **Shrnutí oddílu 9.4.** Architektury tvoří spektrum od cyklického dotazování přes foreground/background po **kooperativní** a **preemptivní** multitasking. Kooperativní model se procesoru vzdává sám (jednoduchý, ale sobecká úloha blokuje), preemptivní jej odebírá plánovač podle priority (real-time). Preemptivní plánovač spouští vždy **nejvyšší připravenou úlohu**, stejnou prioritu řeší Round-Robin. Úloha je ve stavu **READY/RUNNING/BLOCKED/SUSPENDED**; přepnutí kontextu znamená uložit a obnovit registry (RTOS řeší R4–R11 a PSP, zbytek hardware).

9.5 Synchronizace a komunikace

Jakmile úlohy sdílejí data, vzniká souběh. Tato část projde nástroje, které jej řeší — mutex a jeho past inverze priority, semafory, fronty zpráv, prevenci uváznutí a bezpečné sdílení dat s přerušeními.

Sdílený zdroj a mutex

Ve víceúlohovém systému je téměř vždy nějaký **sdílený zdroj**: vysílací buffer UARTu, globální stav, periferie. Když k němu přistoupí dvě úlohy současně — jedna zapisuje a druhá ji uprostřed zápisu preemptne a hodnotu přepíše — vznikne poškozený stav, tedy vzácná a těžko laditelná chyba. Klasickým příkladem je nevinně vypadající inkrement:

```
count = count + 1;   // NEatomické: přeloží se na LDR / ADD / STR
```

Preempce mezi načtením a zápisem způsobí ztrátu jednoho inkrementu. Jde o tentýž **read-**

-modify-write hazard jako u sdílení dat s přerušením (přednáška 3), jen teď mezi úlohami. Řešení je **kritická sekce** chráněná zámkem.

Mutex: vzájemné vyloučení

Mutex je token, který drží vždy jen jedna úloha. Kdo chce do kritické sekce, musí ho nejdřív získat:

Mutex
Mutual exclusion, vzájemné vyloučení: synchronizační objekt (token) s vlastníkem, který v daném okamžiku smí držet jen jedna úloha. Chrání kritickou sekci; odemknout jej smí jen ten, kdo jej zamkl, a kvalitní mutex umí dědění priority.

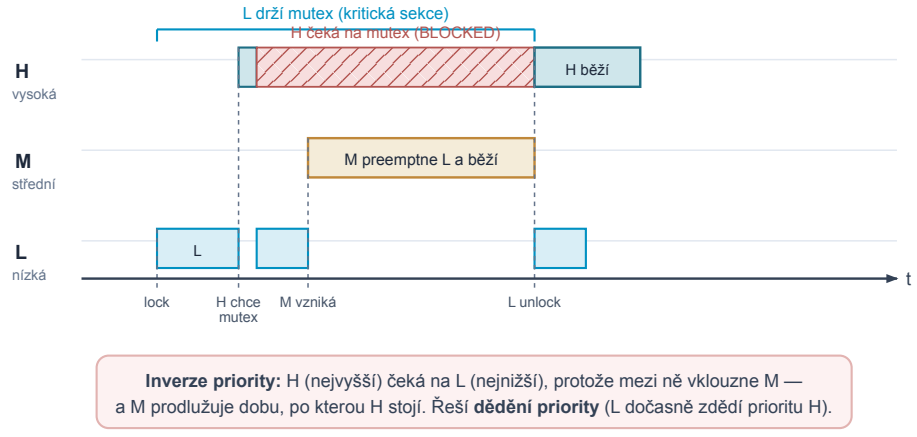
```
rtos_mutex_lock(&uart_mtx);     // vstup do kritické sekce
uart_write(data, len);          // sdílený zdroj – jen jeden vlastník
rtos_mutex_unlock(&uart_mtx);   // výstup
```

Je-li mutex volný, úloha ho zamkne a stane se jeho vlastníkem. Je-li obsazený, RTOS úlohu přepne do stavu BLOCKED, zařadí ji do *waiting listu* mutexu a přepne na jinou. Typický průběh, když o jeden zdroj soupeří dvě úlohy, má pět kroků: (1) úloha A si vezme mutex a stane se vlastníkem; (2) úloha B chce tentýž mutex, ten je obsazený, takže B přejde do BLOCKED a do waiting listu; (3) A dokončí kritickou sekci a zavolá `unlock`; (4) RTOS probudí B (READY) a předá jí vlastnictví; (5) B pokračuje tam, kde byla přerušena. Platí přitom tři pravidla: mutex smí odemknout **jen jeho vlastník**; blokující `lock` **nesmí běžet v obsluze přerušení** (ISR nesmí čekat); a vždy se hodí nabídnout i neblokující variantu `try_lock`. Zápis do waiting listu musí být **atomický** — kryje jej krátké `__disable_irq()`, aby do struktury nezasáhlo přerušení. Mutex je konkrétní podobou **vlastnictví zdroje** z naší ownership-spirály: v přednášce 3 to byl opaque handle v C, v přednášce 5 vlastnictví DMA bufferu a zde mutex.

Priority inversion a její řešení

S mutexy je spojen záludný problém, kdy **vysoká priorita čeká na nízkou**. Nazývá se **inverze priority**. Nízkoprioritní úloha L drží mutex; vysokoprioritní úloha H ho chce, takže se zablokuje a čeká na L. Mezitím se probudí středněprioritní úloha M, která **preemptne L** (M má vyšší prioritu než L), takže L nedokončí kritickou sekci a neuvolní mutex — a H tak čeká „nekonečně", ve skutečnosti po celou dobu běhu M. Nejde o teoretickou kuriozitu: klasickým příkladem selhání je sonda **Mars Pathfinder (1997)**, která se kvůli inverzi priority na Marsu opakovaně resetovala.

Inverze priority
Priority inversion: situace, kdy úloha s vysokou prioritou čeká na zdroj držený úlohou s nízkou prioritou, a mezi ně se dostane úloha se střední prioritou, která nízkoprioritní úlohu preemptuje — a tím dobu čekání vysoké priority prodlužuje.



Obrázek 9.4. Inverze priority na časové ose: nízkoprioritní úloha L drží mutex, vysokoprioritní H na něj čeká (BLOCKED), a středněprioritní M mezitím preemptuje L a čekání H prodlužuje. Řešením je dědění priority.

Řešením je **protokol dědění priority** (priority inheritance): jakmile vysokoprioritní úloha H začne čekat na zdroj držený nízkoprioritní úlohou L, úloha L **dočasně zdědí prioritu H**. Díky

tomu ji úloha M už nepreemptne, L rychle dokončí kritickou sekci a mutex uvolní; po `unlock` se priorité L vrátí původní hodnota. Právě proto se „správný" RTOS mutex liší od prostého binárního semaforu: mutex má **vlastníka** a umí **dědit prioritu**, zatímco semafor vlastníka nemá.

Dědění priority je **reaktivní** — zasáhne, až když ke konfliktu dojde. Přísnější variantou je **protokol stropu priority** (priority ceiling), který konfliktu **předchází**: každému mutexu se předem přiřadí strop rovný nejvyšší prioritě úloh, jež ho kdy použijí, a úloha po jeho zamčení okamžitě získá prioritu stropu. Tím se žádná další úloha usilující o tentýž mutex mezitím nespustí, kritická sekce doběhne bez preempce a protokol navíc **prokazatelně vylučuje uváznutí** a omezuje blokování na jednu kritickou sekci na úlohu (což zpřesní člen `B_i` v analýze doby odezvy). FreeRTOS používá jednodušší dědění priority; priority ceiling je deterministický, ale vyžaduje znát strop předem.

Protokol stropu priority
Priority Ceiling Protocol: každému mutexu se staticky přiřadí strop rovný nejvyšší prioritě úloh, které jej používají. Úloha po zamčení mutexu okamžitě získá prioritu stropu, takže kritická sekce doběhne bez preempce soupeřů. Protokol prokazatelně vylučuje uváznutí a omezuje blokování na jednu kritickou sekci na úlohu.

Semafory a fronty

Semafor je obecnější než mutex — je to čítač povolení. **Binární** semafor nabývá hodnot 0 a 1 a slouží k **signalizaci události**: obsluha přerušení jím dá úloze vědět, že „přišla data" (operaci `give` smí volat i ISR, protože semafor nemá vlastníka). **Čítací** semafor nabývá hodnot 0 až N a spravuje **N stejných zdrojů** — například tři kanály DMA: operace `take` čítač sníží, `give` zvýší, a při nule se žadatel zablokuje, dokud někdo zdroj nevrátí. Orientační pravidlo zní: **mutex** pro vzájemné vyloučení (má vlastníka a dědí prioritu), **binární semafor** pro signalizaci mezi přerušením a úlohou (vlastníka nemá).

Semafor
Synchronizační čítač povolení. Binární semafor (0/1) slouží k signalizaci události (například z přerušení do úlohy); čítací semafor (0 až N) spravuje N stejných zdrojů. Na rozdíl od mutexu nemá vlastníka.

Komunikace mezi úlohami: fronty

Úlohy by neměly sdílet data přes globální proměnné, ale **posílat si zprávy** frontou:

```
// producent (např. ISR nebo úloha čtoucí senzor)
rtos_queue_send(&q, &sample, timeout);
// konzument (zpracování)
rtos_queue_receive(&q, &sample, timeout);   // prázdná fronta → BLOCKED
```

Fronta zpráv zprávu **kopíruje**, takže producent a konzument nesdílejí paměť a je méně příležitostí k chybám souběhu. Prázdná fronta zablokuje konzumenta a plná fronta producenta (s volitelným timeoutem) — což je přirozená forma synchronizace. Především je to ale **přenos vlastnictví dat zprávou**: poslední článek ownership-spirály před vlastnictvím zdrojů v RTOS. Vzor producent–konzument je přitom tentýž jako kruhový buffer u DMA (přednáška 5), jen zabalený do RTOS primitiva s blokováním. Když stačí signalizovat **jedné konkrétní úloze**, je levnější **přímá notifikace**: každá úloha nese ve svém TCB jedno notifikační slovo, do kterého odesílatel přímo zapíše hodnotu nebo nastaví bity, a adresát na ně čeká. Bez samostatného objektu je notifikace rychlejší a menší než binární semafor; nehodí se ale tam, kde na signál čeká víc úloh — tam je potřeba semafor nebo fronta se skutečnou frontou čekatelů.

Fronta zpráv
Message queue: RTOS primitivum pro komunikaci mezi úlohami, které zprávu kopíruje od producenta ke konzumentovi. Prázdná nebo plná fronta čekající úlohu zablokuje. Realizuje přenos vlastnictví dat a vzor producent–konzument.

Uváznutí a jeho prevence

Vzájemné vyloučení řeší jeden problém, ale otevírá druhý. Vezmou-li dvě úlohy dva zámky v **opačném pořadí**, každá drží jeden a čeká na druhý — a systém uvázne natrvalo. To je **uváznutí**. Nastane, právě když současně platí **všechny čtyři Coffmanovy podmínky**: vzájemné vyloučení (zdroj drží nejvýš jedna úloha), drž a čekej (úloha drží jeden zdroj a čeká na další), bez odebrání (zdroj nelze vynutit zpět) a kruhové čekání (řetěz úloh, kde každá čeká na následující). Preemptivní RTOS uváznutí sám neřeší — je to chyba návrhu, ne plánovače.

Protože porušení **jediné** z podmínek uváznutí vyloučí, v embedded se mu **předchází návrhem**:

Uváznutí
Deadlock: stav, kdy skupina úloh čeká navzájem na zdroje tak, že žádná nemůže pokračovat. Nastane, právě když současně platí všechny čtyři Coffmanovy podmínky (vzájemné vyloučení, drž a čekej, bez odebrání, kruhové čekání); porušení kterékoli z nich uváznutí vylučuje.

globálním **pořadím zámků** (všechny úlohy berou mutexy ve stejném pořadí, čímž padne kruhové čekání), **zámkem s timeoutem** (po vypršení úloha ustoupí, čímž padne „drž a čekej"), **protokolem stropu priority** nebo hrubší zrnitostí (jeden zámek nad celou strukturou místo dvou vnořených). Nejlevnější obranou je **nedržet víc než jeden mutex naráz**; když to nejde, vynutí se pořadí a hlídá revizí — porušené pořadí zámků umí odhalit i statické analyzátory (přednáška 3).

Přerušení, kritické sekce a reentrance

Obsluha přerušení běží nad prioritou všech úloh, a musí být proto **krátká**. Vzor **odloženého zpracování** (deferred interrupt processing) ji rozdělí: ISR jen potvrdí příznak a **signalizuje** (notifikací nebo semaforem), zatímco těžké zpracování dělá běžná úloha. Z obsluhy přerušení se přitom smí volat jen varianty API určené pro ISR (`...FromISR`) — ostatní by mohly zablokovat, což v přerušení nelze — a po jejím doběhnutí lze funkcí typu `portYIELD_FROM_ISR` rovnou předat řízení probuzené úloze, aniž se čeká na příští tik. Priorita zpracování se tak řídí prioritou úlohy, ne pořadím v NVIC; je to tentýž handoff jako z ISR do kruhového bufferu u DMA (přednáška 5), jen s blokující úlohou na konci.

Nejkratší vzájemné vyloučení ale není mutex, nýbrž krátké **zamaskování přerušení**. Je-li chráněná sekce jen několik instrukcí (inkrement čítače, zápis do sdílené struktury), zabalí ji RTOS do `taskENTER_CRITICAL` / `taskEXIT_CRITICAL`, které na Cortexu-M maskují přes `PRIMASK` nebo `BASEPRI` (přednáška 2); `BASEPRI` přitom nechá kritická přerušení dál běžet. Taková sekce nesmí blokovat ani volat plánovač a musí trvat jednotky instrukcí, jinak roste latence přerušení. Zatímco zámek řeší souběh mezi úlohami, maskování řeší souběh mezi úlohou a ISR (nebo velmi krátkou sekci).

S přerušeními a sdílením souvisí i **reentrance**: volá-li tutéž funkci víc úloh nebo úloha i ISR, musí být funkce bezpečně přerušitelná uprostřed a znovu spustitelná. Reentrantní funkce nepoužívá sdílený zapisovatelný stav (žádné `static` proměnné ani globály bez zámku), pracuje jen s argumenty a lokálami. Klasickými pastmi jsou `strtok`, sdílené `errno` nebo alokátor `malloc`; v RTOS je proto nutná thread-safe verze knihovny (u newlib-nano ochrana struktury `_reent` nebo zámek haldy). Reentrance je vlastnost kódu, thread-safety se zajistí zámkem — a nejjistější je sdílené funkce psát **bez vnitřního stavu**, pak jsou reentrantní z principu.

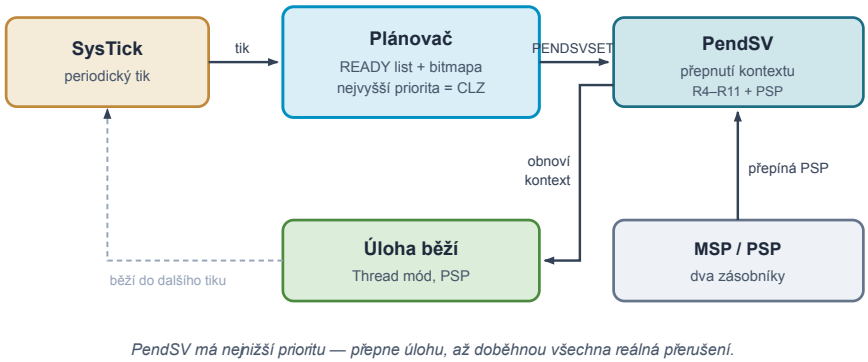
> **Shrnutí oddílu 9.5.** Sdílený zdroj plus preempce vede na **read-modify-write hazard**, který řeší kritická sekce pod zámkem, u krátkých sekcí **maskování přerušení** (`BASEPRI`). **Mutex** je token s **vlastníkem**; **inverze priority** (H čeká na L, M preemptne L) se řeší **děděním priority** (reaktivně) nebo **stropem priority** (proaktivně, brání uváznutí). Klasikou je Mars Pathfinder. **Uváznutí** vzniká při splnění čtyř Coffmanových podmínek a předchází se mu pořadím zámků, timeoutem či stropem priority. **Semafory** signalizují (binární) a spravují N zdrojů (čítací), **fronty** přenášejí vlastnictví dat zprávou, **notifikace** jsou nejlehčí signál 1↔1. **Přerušení** se obsluhují odloženým zpracováním přes `...FromISR`; sdílené funkce musí být **reentrantní**.

9.6 Postavme si RTOS na Cortex-M

Nejlépe RTOS pochopíme tak, že si ho postavíme. Tato část ukáže, na kterých hardwarových blocích Cortex-M minimalistický RTOS stojí, a projde jeho jádro — TCB, výběr úlohy v konstantním čase, přepnutí kontextu v PendSV a tik plánovače.

Co jádro potřebuje a proč PendSV

Minimalistický preemptivní RTOS na Cortex-M stojí na třech blocích, které už známe z přednášek 1 a 2. Prvním jsou **dva zásobníky** — `MSP` pro jádro a obsluhu přerušení a `PSP` pro úlohy — přepínané bitem v registru `CONTROL`. Druhým je **SysTick**, periodické přerušení, které pohání plánovač. Třetím je **PendSV**, vyhrazená výjimka s **nejnižší prioritou**, ve které se provádí samotné přepnutí kontextu.



Obrázek 9.5. Jádro RTOS na Cortex-M: SysTick dává periodický tik plánovači, ten napenduje PendSV, který přepne kontext úlohy pomocí zásobníků MSP a PSP. PendSV má nejnižší prioritu, takže přepne až po doběhnutí reálných přerušení.

Celé přepínání úloh se dá postavit jen z těchto tří bloků — proto je Cortex-M pro RTOS mimořádně vhodný.

Proč právě PendSV

Přepnutí kontextu je „těžká” operace (uložit a obnovit registry) a nesmí přerušit obsluhu skutečné periferie. **PendSV** to řeší elegantně: plánovač jen **naplánuje** PendSV zápisem bitu `PENDSVSET` do registru `SCB->ICSR`, a procesor spustí PendSV, až když nedobíhá žádné přerušení s vyšší prioritou (PendSV má nejnižší). Přepnutí kontextu tak proběhne „v bezpečí”, mimo obsluhu periferií:

```
SCB->ICSR = SCB_ICSR_PENDSVSET_Msk;   // „až bude klid, přepni úlohu"
```

Rychlá obsluha přerušení (například data z UARTu) doběhne první, a drahé přepnutí kontextu se odloží na PendSV. Je to tentýž vzor **rychlá ISR plus odložené zpracování**, který známe z předchozích přednášek, jen teď’ na úrovni jádra RTOS.

PendSV
Pendable Service Call: softwarem vyvolatelná výjimka Cortex-M, kterou RTOS konfiguruje na nejnižší prioritu a používá pro odložené přepnutí kontextu. Naplánuje se zápisem bitu PENDSVSET a spustí se, až doběhnou všechna přerušení s vyšší prioritou.

TCB a výběr úlohy v O(1)

Každou úlohu popisuje struktura, která úlohu pro plánovač jednoznačně identifikuje:

```
typedef enum { TASK_READY, TASK_RUNNING, TASK_BLOCKED,
               TASK_SUSPENDED, TASK_TERMINATED } task_state_t;

typedef struct tcb {
    uint32_t     *stack_ptr;    // aktuální PSP – MUSÍ být první položkou
    uint32_t     *stack_base;   // dno zásobníku (kontrola přetečení)
    uint32_t      stack_size;
    uint8_t       priority;     // 0 = nejvyšší
    task_state_t  state;
```

```
    struct tcb   *next;         // zřetězení v ready listu
} tcb_t;
```

Položka `stack_ptr` je záměrně **první**: obsluha PendSV k ní pak přistoupí jedinou instrukcí `ldr` nebo `str` přes ukazatel na TCB, bez počítání offsetu.

READY list a výběr úlohy v čase O(1)

Plánovač drží pro každou úroveň priority frontu připravených úloh a k tomu **bitmapu**, jejíž jednotlivé bity říkají, které fronty jsou neprázdné:

```
tcb_t   *ready_lists[MAX_PRIORITIES];
uint32_t ready_bitmap;             // bit i = fronta priority i je neprázdná

tcb_t *scheduler_select_next(void) {
    if (ready_bitmap == 0) return idle_tcb;
    int hi = 31 - __CLZ(ready_bitmap);      // nejvyšší priorita v O(1) – CLZ z přednášky 7
    tcb_t *next = ready_lists[hi];
    ready_lists[hi] = next->next;           // Round-Robin: vyjmi z čela fronty
    if (!ready_lists[hi]) ready_bitmap &= ~(1u << hi);
    return next;
}
```

Výběr nejvyšší připravené priority je díky instrukci `CLZ` (count leading zeros, přednáška 7) **jediná operace** bez ohledu na počet úloh — tedy v konstantním čase O(1). Přesně tak to dělá i FreeRTOS.

Přepnutí kontextu a tik plánovače

Vlastní přepnutí uloží registry `R4`–`R11` a PSP staré úlohy a obnoví je u nové (zbytek — `R0`–`R3`, `R12`, `LR`, `PC`, `xPSR` — řeší hardware, viz část 9.4):

```
PendSV_Handler:
    cpsid i                     ; krátce zakázat přerušení
    MRS   r0, PSP               ; PSP běžící úlohy
    STMDB r0!, {r4-r11}         ; ulož R4-R11 na její zásobník
    ; current_tcb->stack_ptr = r0
    ; current_tcb = scheduler_select_next()
    ; r0 = current_tcb->stack_ptr
    LDMIA r0!, {r4-r11}         ; obnov R4-R11 nové úlohy
    MSR   PSP, r0
    cpsie i
    BX    lr                    ; EXC_RETURN: HW obnoví zbytek a skočí do úlohy
```

Blokové přesuny `STMDB` a `LDMIA` uloží a obnoví všech osm registrů dvěma instrukcemi. Návrat `BX lr` s hodnotou EXC_RETURN řekne jádru, aby se vrátilo do režimu Thread na zásobník `PSP` — tedy do vybrané úlohy.

SysTick jako tik plánovače

Periodické přerušení SysTick měří systémový čas a spouští preempci:

```
void SysTick_Handler(void) {
    sys_ticks++;
    odblokuj_uplynule_delaye();             // BLOCKED→READY po vypršení timeoutu
```

```
    if (scheduler_should_preempt())          // je připravená vyšší priorita než běžící?
        SCB->ICSR = SCB_ICSR_PENDSVSET_Msk;  // naplánuj přepnutí (PendSV)
}
```

Podstatná je dělba práce: SysTick jen **rozhodne**, že se má přepnout, a napenduje PendSV — sám nepřepíná, protože přepnutí kontextu by bylo drahou operací uvnitř obsluhy přerušení. SysTick tedy říká „kdy", PendSV dělá „jak".

API minimalistického RTOS

Nad jádrem stojí hrstka funkcí, se kterými pracuje aplikace:

```
// správa úloh
int  rtos_task_create(void (*entry)(void*), void *arg,
                      uint32_t *stack, uint32_t size, uint8_t prio);
void rtos_task_sleep(uint32_t ms);      // úloha → BLOCKED na ms (uvolní procesor)
void rtos_task_yield(void);             // dobrovolně předej procesor
// synchronizace a komunikace
void rtos_mutex_lock(mutex_t *m);   void rtos_mutex_unlock(mutex_t *m);
int  rtos_sem_wait(sem_t *s, uint32_t ms);   void rtos_sem_post(sem_t *s);
int  rtos_queue_send(queue_t *q, void *msg, uint32_t ms);
int  rtos_queue_receive(queue_t *q, void *out, uint32_t ms);
```

Toto zjednodušené API odpovídá téměř jedna k jedné FreeRTOS (`xTaskCreate`, `vTaskDelay`, `xSemaphoreTake`, `xQueueSend`). Kdo pochopí tento základ, se ve zdrojovém kódu FreeRTOS bez potíží orientuje.

Zásobník úlohy a jeho přetečení

Každá úloha má **vlastní zásobník**, přepínaný přes PSP, a jeho velikost je záludná: příliš malý přeteče, příliš velký zbytečně zabírá vzácnou RAM. Přetečení navíc přepíše cizí paměť a projeví se **jinde a později**, takže se hůř hledá. Rozpočet zásobníku je nejhlubší řetěz volání plus **rámec přerušení** (automatické uložení registrů, u FPU i lazy stacking z přednášky 11), který se ukládá na zásobník právě běžící úlohy. Přetečení se hlídá **kontrolním vzorkem** (dno se vyplní značkou `0xA5A5A5A5` a plánovač při přepnutí ověří, že zůstala celá) a měří **high-water markem** — nejnižší dosaženou hladinou, která ukáže skutečnou rezervu. Tvrdou variantou je **MPU** (přednáška 4), která pod každý zásobník umístí *guard* stránku bez přístupu, takže přetečení skončí okamžitým MemManage faultem místo tiché koroze paměti. Zásobníky se proto dimenzují **s měřením, ne odhadem**: spusťme zátěž, přečtěme high-water mark a přidej rezervu; přetečení zásobníku je jedna z nejčastějších a nejhůř dohledatelných chyb v RTOS.

> **Shrnutí oddílu 9.6.** Preemptivní RTOS na Cortex-M stojí na **MSP/PSP** (přednáška 1), **SysTicku** (přednáška 2) a **PendSV** (nejnižší priorita, vyhrazený na přepnutí). PendSV se naplánuje bitem `PENDSVSET` a přepnutí kontextu (registry `R4`–`R11` a PSP přes `STMDB`/`LDMIA`) proběhne, až doběhnou reálná přerušení. **TCB** popisuje úlohu (`stack_ptr` je první položkou), **READY list s bitmapou** vybírá nejvyšší prioritu v **O(1) přes `CLZ`** (přednáška 7). Každá úloha má **vlastní zásobník**; přetečení se hlídá vzorkem a high-water markem, tvrdě **MPU guard** stránkou (přednáška 4). SysTick rozhodne „kdy", PendSV udělá „jak"; API se blíží FreeRTOS.

9.7 Syntéza

Zbývá to, co jsme postavili, zasadit do reality. Tato závěrečná část ukáže, jak naše jádro odpovídá průmyslovému FreeRTOS, jak se v RTOS řeší paměť a ladění a kdy RTOS vůbec nasadit.

FreeRTOS, paměť a runtime ladění

To, co jsme postavili ručně, dělá i **FreeRTOS** — nejrozšířenější embedded RTOS (dnes pod správou AWS), který běží v miliardách zařízení. Naše funkce mají u něj přímé protějšky: `rtos_task_create` odpovídá `xTaskCreate`, `rtos_task_sleep` odpovídá `vTaskDelay`, `rtos_mutex_lock` odpovídá `xSemaphoreTake` nad mutexem a `rtos_queue_send` odpovídá `xQueueSend`. FreeRTOS navíc přidává správu paměti (varianty `heap_1` až `heap_5`), softwarové časovače, event groups, tickless idle pro úsporu energie a portace na desítky architektur — ale samotné jádro (TCB, ready list, PendSV, SysTick) je přesně to naše.

Paměť v RTOS a runtime ladění

RTOS musí odněkud brát paměť pro úlohy, zásobníky a synchronizační objekty. Dynamický `malloc` je v embedded rizikový (fragmentace, nedeterminismus, přednáška 4), proto FreeRTOS nabízí **pět schémat haldy**. `heap_1` jen alokuje a nikdy neuvolňuje — nejjednodušší a plně deterministické, objekty vzniknou při startu a žijí navždy. `heap_2` umí i uvolnit, ale nesceluje sousední bloky, a proto **fragmentuje** (dnes se nedoporučuje). `heap_4` uvolňuje a **scelí sousední volné bloky** (coalescing), což je praktický kompromis, a `heap_5` totéž přes několik nespojitých oblastí RAM (interní i externí SRAM). Alternativou je **statická alokace** (`xTaskCreateStatic`), kdy paměť dodá aplikace a halda se nepoužije vůbec. Bezpečnostně kritické systémy volí právě statickou alokaci nebo `heap_1`: veškerá paměť je známá při překladu, běh je deterministický a nehrozí selhání alokace za běhu — je to tentýž princip „ne-malloc" jako v přednášce 4, jen na úrovni RTOS.

Souběžný systém navíc nelze ladit prostým krokováním — přerušení a přepínání kontextu se krokem rozpadnou. Používají se proto **runtime statistiky a trasování**: využití procesoru na úlohu (odhalí přetíženou nebo zaseknutou úlohu), high-water mark zásobníků (dimenzování, viz část 9.6), počty úloh ve stavech READY a BLOCKED (odhalí uváznutí či hladovění) a **trace hooky**, které zaznamenají události přepnutí, blokování a probuzení a offline je zobrazí na časové ose. Trasování vyvádí události ven přes **ITM/SWO nebo DWT** (přednáška 3) bez zásahu do běhu; nástroje jako Tracealyzer nebo SystemView z nich rekonstruují, kdo kdy běžel a na co čekal — což je nezbytné právě u problémů souběhu, které v debuggeru běžně zůstanou skryté.

Kdy RTOS nasadit a kdy ne

RTOS není zdarma: přidává paměť (jádro plus zásobník na každou úlohu), režii přepínání kontextu a složitost (synchronizace, riziko uváznutí). **Superloop stačí**, když je úloh málo, jsou krátké a nekolidují — blikání LED, jednoduchý senzor. **RTOS se vyplatí**, když máme víc úloh s různými periodami a tvrdými termíny, blokující operace (síť, souborový systém), nebo když potřebujeme **izolaci**, aby pád jedné úlohy neshodil ostatní. Volba je inženýrská, ne módní: RTOS řeší souběžnost a včasnost — kde je nepotřebujeme, jen si přidáme režii a nové příležitosti k chybám souběhu.

Uzavření ownership-spirály

Vlastnictví zdroje jsme potkávali celý kurz, pokaždé o úroveň formálněji. V **přednášce 3** šlo o ruční vlastnictví v jazyce C: opaque handle s funkcemi `create` a `destroy`. V **přednášce 5** o vlastnictví **bufferu** předávaného mezi DMA, přerušením a aplikací. A v **přednášce 9** o vlastnictví zdroje **mutexem** (vzájemné vyloučení) a o **přenosu vlastnictví dat zprávou** ve frontě. Otázka „kdo právě vlastní ten zdroj?" se tak táhne od jediného handle až po plánovač; RTOS jí dává jazyk — mutex, semafor, frontu — a pravidla (dědění priority), aby souběh nebyl náhodný. Formálnější vyústění téhož principu — vlastnictví vynucené typovým systémem

(RAII, přesun, výpůjčka) — rozebírá Příloha A.

> **Shrnutí oddílu 9.7.** **FreeRTOS** je průmyslový standard, jehož jádro odpovídá tomu, co jsme postavili; přidává správu paměti, časovače a portace. **Paměť** dodá halda `heap_1`–`heap_5` nebo **statická alokace** — kritické systémy volí determinismus (`heap_1`/static). Souběh se ladí **runtime statistikami a trasováním** (ITM/SWO/DWT, přednáška 3), ne krokováním. **Superloop** volíme pro málo krátkých nekolidujících úloh, **RTOS** pro víc úloh s různými periodami, tvrdými termíny, blokujícími operacemi a potřebou izolace. RTOS uzavírá **ownership-spirálu** kurzu: vlastnictví zdroje od handle (přednáška 3) přes buffer (přednáška 5) po mutex a frontu (přednáška 9).

9.8 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 9.1 — Proč RTOS

1. Vysvětlete rozdíl mezi „rychlým" a „real-time" systémem.
2. Uveďte po dvou příkladech hard a soft real-time úloh a zdůvodněte zařazení.
3. Proč u plánovatelnosti pracujeme s nejhorším případem, a ne s průměrem?
4. Čím se liší aperiodická a sporadická úloha a proč je ten rozdíl důležitý pro analýzu?
5. Vyjmenujte tři vrstvy souběhu a přiřaďte ke každé odpovídající RTOS primitivum.

K oddílu 9.2 — Úlohy a čas

1. Definujte časové parametry periodické úlohy C, T, D a φ a nakreslete jejich vztah na časové ose.
2. Proč se pro hard real-time používá WCET a ne průměrná doba běhu?
3. Proč je určení WCET (nejhoršího času) obtížné — co ho znepředvídatelňuje?
4. Proč superloop přestává stačit s rostoucím počtem úloh s různými periodami a termíny?
5. Co řeší vzor foreground/background a kde i on naráží na mez?

K oddílu 9.3 — Plánovatelnost

1. Jak spočítáte utilizaci sady periodických úloh a co znamená podmínka $U \leq 1$?
2. Formulujte rate-monotonic pravidlo a Liuovu–Laylandovu mez; k jaké hodnotě mez konverguje?
3. Je Liuův–Laylandův test nutnou, nebo postačující podmínkou; jak analýza doby odezvy tento nedostatek řeší a co v ní znamená člen se stropem?
4. Kdy zvolíte deadline-monotonic místo rate-monotonic a proč pro $D < T$ neplatí utilizační mez?
5. Sada má $U = 0{,}9$ a používá RMS; co o její plánovatelnosti víte a jak byste ji přesně ověřili?

K oddílu 9.4 — Architektury

1. Seřaďte architektury od nejjednodušší po preemptivní multitasking a u každé uveďte cenu a přínos.
2. V čem je zásadní rozdíl mezi kooperativním a preemptivním plánováním?
3. Jak preemptivní plánovač zachází s úlohami stejné priority?
4. Vyjmenujte stavy úlohy a přechody mezi nimi; který přechod může vyvolat preempci?
5. Které registry ukládá při přepnutí kontextu hardware a které RTOS, a proč to tak je?

K oddílu 9.5 — Synchronizace a komunikace

1. Vysvětlete read-modify-write hazard na příkladu `count++` a jak jej řeší kritická sekce.
2. Popište životní cyklus mutexu, když o něj soupeří dvě úlohy.
3. Co je inverze priority a jak vzniká? Nakreslete časovou osu tří úloh.
4. Porovnejte dědění priority a protokol stropu priority (reaktivní × proaktivní, uváznutí, blokování B_i).
5. Za jakých podmínek vzniká uváznutí (deadlock) a jak se dá každé z nich předejít?
6. Kdy použijete mutex, kdy binární semafor, kdy frontu a kdy přímou notifikaci?
7. Popište vzor odloženého zpracování přerušení a pravidla pro volání API z ISR.
8. Co je reentrantní funkce, které knihovní funkce nejsou reentrantní a jak to v RTOS řešíte?

K oddílu 9.6 — Stavba RTOS na Cortex-M

1. Na kterých hardwarových blocích Cortex-M staví minimalistický RTOS a proč jsou právě ony klíčové pro přepínání úloh?
2. Proč se přepnutí kontextu dělá v PendSV a ne přímo v SysTicku?
3. Proč musí být `stack_ptr` první položkou v TCB?
4. Jak funguje výběr nejvyšší připravené priority v čase O(1) a kterou instrukci k tomu využívá?
5. Které registry uloží obsluha PendSV a které hardware, a jakou roli hraje EXC_RETURN?
6. Z čeho se skládá rozpočet zásobníku úlohy a jak přetečení detekujete vzorkem, high-water markem a MPU?
7. Uveďte protějšky funkcí minimalistického RTOS API (vytvoření úlohy, uspání, zamčení mutexu, odeslání do fronty) ve FreeRTOS a co FreeRTOS přidává navíc.

K oddílu 9.7 — FreeRTOS, paměť a nasazení

1. Porovnejte schémata haldy `heap_1`, `heap_2`, `heap_4` a `heap_5` a řekněte, proč kritické systémy volí statickou alokaci nebo `heap_1`.
2. Proč nelze souběžný systém ladit krokováním a jaké runtime statistiky a trasovací mechanismy se místo toho používají?
3. Podle jakých kritérií se rozhodujete mezi superloopem a RTOS a jakou režii a rizika RTOS přináší?

10

Nízká spotřeba a spolehlivost

> Tato kapitola řeší, **jak navrhnout systém, který dělá svou práci s průměrným proudem v jednotkách mikroampér — a když se přesto něco zasekne, sám se zotaví**. Životnost neurčuje výkon v aktivním stavu, ale **jak krátce a jak zřídka** je systém aktivní a **jak hluboko** spí mezi tím; a protože běží roky bez obsluhy, musí navíc **přežít** vybíjející se baterii i zaseknutý program.

Obsah kapitoly

10.1 Odkud se bere spotřeba
10.2 Napájecí architektura STM32
10.3 Úsporné režimy
10.4 Probouzení a energetický kompromis
10.5 Návrhové vzory nízké spotřeby
10.6 Spolehlivost: supervize a watchdog
10.7 RTOS v low-power
10.8 Syntéza
10.9 Kontrolní otázky

10.1 Odkud se bere spotřeba

Než začneme spotřebu snižovat, musíme vědět, odkud se bere. Tato část rozliší dynamickou a statickou složku, ukáže, proč má napájecí napětí kvadratický vliv, a vysvětlí, jak se spotřeba měří jako profil proudu v čase.

Dvě složky spotřeby

Spotřeba mikrokontroléru se dělí na dvě podstatně odlišné složky, které se potírají různými nástroji. **Dynamická spotřeba** vzniká **přepínáním** logických stavů — nabíjením a vybíjením parazitních kapacit hradel — a teče jen, když se něco děje (hodiny tikají, signály se mění). **Statická spotřeba** naproti tomu vzniká **úniky** (leakage): tranzistory CMOS nejsou ve „vypnutém" stavu dokonale nevodivé, takže proud teče **i když se nic neděje**, i ve spánku.

Rozdíl je zásadní pro celý návrh: dynamickou složku snižujeme frekvencí, napětím a vypínáním hodin, kdežto statickou lze potlačit jen **fyzickým odpojením napájení** částí čipu (hluboký spánek, napájecí domény). Návrh nízké spotřeby proto musí řešit obě složky současně.

Dynamická spotřeba
Energie spotřebovaná při přepínání logických stavů, tedy nabíjením a vybíjením parazitních kapacit hradel. Teče jen tehdy, když se v obvodu něco děje, a roste s frekvencí a druhou mocninou napájecího napětí.

Statická spotřeba
Energie spotřebovaná únikovým proudem tranzistorů, které nejsou ve vypnutém stavu dokonale nevodivé. Teče i tehdy, když se nic neděje a systém spí; roste s miniaturizací procesu a exponenciálně s teplotou.

Dynamická spotřeba

Hlavní složkou dynamické spotřeby je **přepínací ztráta**: při každé změně stavu na výstupu hradla se nabije a vybije parazitní kapacita `C`. Střední přepínací výkon je úměrný

```
P_switching ∝ C · V_DD² · f
```

Kvadratická závislost na napětí je klíč celé přednášky: **snížení `V_DD` na polovinu sníží dynamickou spotřebu na čtvrtinu**. Odtud plyne význam škálování napětí (část 10.2). Menší, druhou složkou je **zkratový proud**: během přepínání jsou PMOS i NMOS tranzistor hradla krátce oba částečně vodivé a vytvoří zkratovou cestu z `V_DD` na zem.

Dva nástroje potlačení plynou přímo ze vzorce. **DVFS** snižuje `V_DD` i `f`; **clock gating** uzavře hodiny nepoužívané periferie. Samotné vypínání hodin ušetří 10–90 % dynamické spotřeby.

DVFS
Dynamic Voltage and Frequency Scaling: současné snižování frekvence a napětí jádra podle aktuální potřeby výkonu. Protože spotřeba roste s druhou mocninou napětí, přináší snížení napětí největší úsporu.

Clock gating
Uzavření hodinového hradla nepoužívané periferie nebo bloku. Bez hodin blok nepřepíná stavy, a tedy neodebírá dynamickou spotřebu. Uspoří deset až devadesát procent dynamické spotřeby.

Co rozhoduje o velikosti dynamické spotřeby

O odběru v aktivním stavu rozhoduje především **zátěž signálových vodičů** (například I²C pull-up 4,7 kΩ při 3,3 V odebírá v logické nule 0,7 mA), **kapacita buzených vodičů** (GPIO budící dlouhý kabel) a hlavně **počet aktivních periferií**. Každá zapnutá periferie něco stojí: PLL jednotky miliampér, UART 100–300 μA, ADC 100 μA až 1 mA. Rádio je řádově dražší: BLE v režimu advertising 5–15 mA, LoRa 40–120 mA, Wi-Fi při vysílání 80–200 mA. Právě proto se u IoT zařízení šetří s každým přeneseným bitem dat — vysílání je energeticky nejdražší operace celého cyklu.

Účinnost jádra se normuje na **mikroampéry na megahertz** (μA/MHz), tedy kolik proudu stojí jeden megahertz výpočtu. Rozdíl mezi výkonnou a ultra-low-power rodinou je řádový: STM32F4 při 168 MHz má stovky μA/MHz, STM32L4 desítky, nRF52 kolem 55 a MSP430 jednotky μA/MHz. Volba rodiny je proto první a největší návrhové rozhodnutí — ultra-low-power řady mají nižší maximální takt výměnou za desetinovou spotřebu na megahertz.

Statická spotřeba

Únikový proud protéká tranzistory i ve vypnutém stavu a s miniaturizací procesu roste; statický výkon je `P_static = I_leakage · V_DD`. Zásadní je jeho **exponenciální závislost na teplotě**: standby proud, který je při −20 °C 300 nA, může při +85 °C vyskočit na 2–10 μA. Typické hodnoty klidového proudu: ultra-low-power STM32 (L0/L4/L5/U5) 200–800 nA, nRF52 kolem 300 nA, MSP430 v LPM3 100–500 nA, ESP32 v hlubokém spánku 5–20 μA (nemá extrémně nízký leakage).

Statickou spotřebu **nelze snížit softwarem** — je dána výrobním procesem. Jedinou obranou je odpojit napájení velkých částí čipu (napájecí domény, oddíl 10.2) a v horkém prostředí s ní počítat už při návrhu.

Měření spotřeby

Měřit proud v rozsahu od nanoampér po miliampéry současně je náročné. Běžný multimetr nemá dost rozlišení a jeho **burden voltage** (úbytek na měřicím bočníku) zkreslí výsledek. Potřebujeme nástroj se širokým dynamickým rozsahem a rychlým vzorkováním, aby zachytil i krátké proudové špičky při probuzení — například Nordic PPK2 (200 nA až 1 A), Otii Arc, Joulescope nebo laboratorní Keysight N6705C (od jednotek nanoampér).

Zásadní je, že spotřebu neměříme jako jedno číslo, ale jako **profil proudu v čase** přes celý pracovní cyklus; teprve integrál pod křivkou dá průměrný proud a životnost baterie (oddíl 10.5). Praxe má tři časté pasti: **připojený debugger** (SWD/JTAG a ST-Link přidají 100 μA až 10 mA a mohou blokovat vstup do hlubokého spánku), **floating piny** (nezapojený vstup

plave kolem prahu a jeho digitální buffer teče — pin je třeba nastavit jako analog nebo s pull--up/down) a **nedostatečná doba ustálení** před odečtem. Většina „záhadně vysoké" spotřeby ve spánku má první dvě příčiny; ověřte je jako první.

> **Shrnutí oddílu 10.1.** Spotřeba má **dynamickou** složku ($P \propto C \cdot V_{DD}^2 \cdot f$, přepínání — potírá se frekvencí, napětím a clock gatingem) a **statickou** (leakage — jen odpojením napájení, roste s teplotou). Kvadratická závislost na napětí dělá ze **škálování napětí** nejmocnější páku. Účinnost jádra měříme v **µA/MHz**; volba rodiny je řádové rozhodnutí. Spotřeba se **měří jako profil proudu v čase**; pozor na debugger a floating piny.

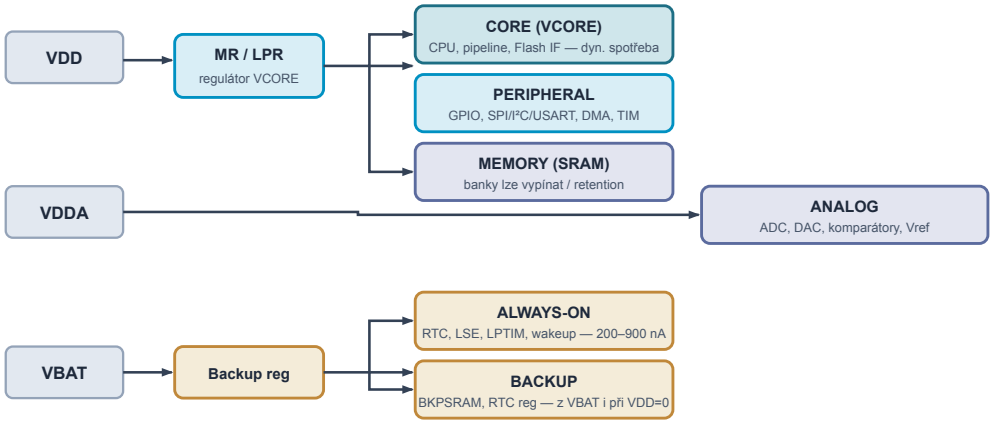
10.2 Napájecí architektura STM32

Aby šlo spotřebu vypínat po částech, není čip napájen jako jeden blok. Tato část projde rozdělení na napájecí domény, roli externích zdrojů (VDD, VBAT, VDDA), regulátory jádra a škálování napětí a frekvence.

Napájecí domény a externí zdroje

Aby bylo možné vypínat spotřebu po částech, moderní mikrokontrolér nemá jedno společné napájecí napětí, ale je rozdělen na oddělené **napájecí domény**. Každou lze nezávisle vypnout, zpomalit, přepnout na jiný takt nebo udržet v retenci (pod napětím zůstane jen paměť či registry, aby přežil stav). Právě toto rozdělení je hlavní hardwarový mechanismus umožňující mikroampérové a nanoampérové režimy — software jen řídí, které domény zůstanou napájené.

Napájecí domény
Části čipu s vlastním regulátorem a hodinami, které lze nezávisle vypnout, zpomalit, přepnout na jiný takt nebo udržet v retenci. Rozdělení na domény je hlavní hardwarový mechanismus, který umožňuje mikroampérové a nanoampérové režimy.



Obrázek 10.1. Zjednodušená napájecí architektura STM32: externí zdroje (VDD, VDDA, VBAT) přes regulátory napájejí oddělené domény (CORE, PERIPHERAL, MEMORY, ANALOG, ALWAYS-ON, BACKUP), které lze vypínat nezávisle. Backup doména je napájena z VBAT i při odpojeném VDD.

Externí zdroje napájení

STM32 má několik napájecích pinů s odlišnou rolí. **VDD** (1,7–3,6 V) je hlavní digitální napájení regulátorů, I/O a supervize. **VBAT** (1,2–3,6 V) je záloha pro RTC, backup registry a oscilátor LSE — napájí backup doménu i při `VDD = 0`, takže hodiny reálného času a zálohovaná paměť zůstanou funkční i při výpadku hlavního napájení. **VDDA** (1,8–3,6 V) napájí analog (ADC, DAC, komparátory, referenci) a **VDDUSB** (3,0–3,6 V) budič USB, jen když se USB používá. Při návrhu desky se VDD a VDDA spojují přes feritovou perličku (filtr šumu do analogu), VBAT se připojuje i bez RTC (jinak floating pin zvyšuje spotřebu) a blokovací kondenzátory (100 nF

a 10 μF) patří co nejblíž pinům.

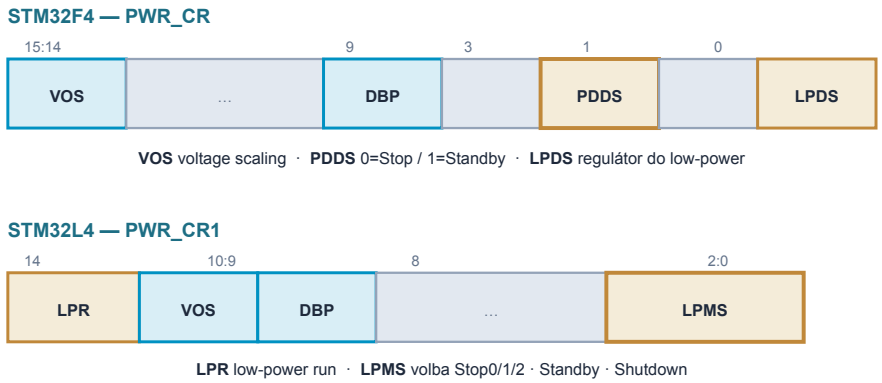
Napájecí domény

Základní domény a jejich chování ve spánku: **CORE (VCORE)** obsahuje procesor, pipeline, rozhraní Flash a případně FPU a cache — je hlavním zdrojem dynamické spotřeby a ve Stop se přepíná na low-power LDO, v Shutdown se vypne. **PERIPHERAL** (GPIO, SPI/I²C/USART, DMA, časovače) ve Stop většinou zastaví hodiny. **MEMORY** jsou banky SRAM, které lze vypínat po částech (u deep-sleep IoT zařízení bývá polovina RAM vypnutá, což ušetří 30–40 % standby proudu). **ANALOG** (ADC, DAC, komparátory) typicky odebírá 150–800 μA a vypíná se mimo měření. **ALWAYS-ON** (RTC, LSE, LPTIM, wakeup logika) běží dál za 200–900 nA a **BACKUP** (BKPSRAM, RTC registry) je napájena z VBAT a odebírá 200–500 nA. Zatímco CORE určuje dynamickou spotřebu, domény ALWAYS-ON a BACKUP určují **statickou** spotřebu ve spánku.

Regulátory a DVFS

Napětí jádra dodává jeden ze tří regulátorů podle režimu: **Main Regulator (MR)** pro Run/Sleep/Stop0 (vysoký výkon, desítky μA), **Low-Power Regulator (LPR)** pro LP-Run/Stop1/Stop2 (jednotky μA, ale pomalejší) a **Backup Regulator** napájející backup doménu z VBAT. Přechod na LPR (STM32L4) vyžaduje nejdřív snížit takt (LP-Run běží na MSI do ~2 MHz), pak nastavit `PWR_CR1.LPR` a počkat na potvrzení `PWR_SR2.REGLPF`.

Cílový režim a škálování napětí se volí v registru PWR, jehož rozložení se **liší mezi rodinami**:



Obrázek 10.2. Registr PWR řízení napájení, STM32F4 (PWR_CR) versus STM32L4 (PWR_CR1). Klíčová pole: VOS (voltage scaling), PDDS respektive LPMS (co znamená deep sleep — Stop, Standby, Shutdown), LPDS respektive LPR (regulátor do low-power). Pojmenování bitů se mezi rodinami mění.

DVFS: škálování napětí a frekvence

Nižší frekvence dovolí nižší napětí jádra, a tím díky kvadratické závislosti výrazně nižší dynamickou spotřebu. Rodiny to řeší různě. **STM32L4/L5/U5** přepínají mezi Range 1 (~1,2 V, 80/120 MHz) a Range 2 (~1,0 V, 26 MHz) **za běhu** — hardware sám upraví napětí za ~5 μs, PLL může zůstat zapnutý, pokud splní frekvenční limit. **STM32F4** používá VOS Scale 1/2/3, ale změna napětí je možná jen při **vypnutém PLL**, takže přechod znamená přepnout hodiny na HSI, vypnout PLL, změnit VOS a znovu nastartovat PLL — řádově stovky mikrosekund.

Hodiny jako páka spotřeby: PLL a MSI

PLL má **největší spotřebu** ze všech hodinových zdrojů a jeho rozběh (lock) trvá 100–500 μs; po tu dobu nelze přejít do Run. To je drahé energeticky i časově. Strategie „taktovat co nejníže" má tři kroky: základ držet na nízkofrekvenčním úsporném zdroji, PLL zapínat jen krátce na výkonovou špičku a hned poté přepnout zpět. Tím zdrojem je u low-power řad **MSI** — rychlý start (~10 μs), nízká spotřeba, běžně hlavní clock po probuzení ze Stop a zdroj, ke kterému se přepíná při DVFS. Alternativou pro nejnižší takt jsou LSI/LSE 32 kHz.

MSI
Multi-Speed Internal: úsporný interní RC oscilátor low-power řad STM32 s rozsahem od stovek kilohertzů po desítky megahertzů a rychlým startem kolem deseti mikrosekund. Používá se jako hlavní hodiny po probuzení ze Stop a jako zdroj při škálování frekvence.

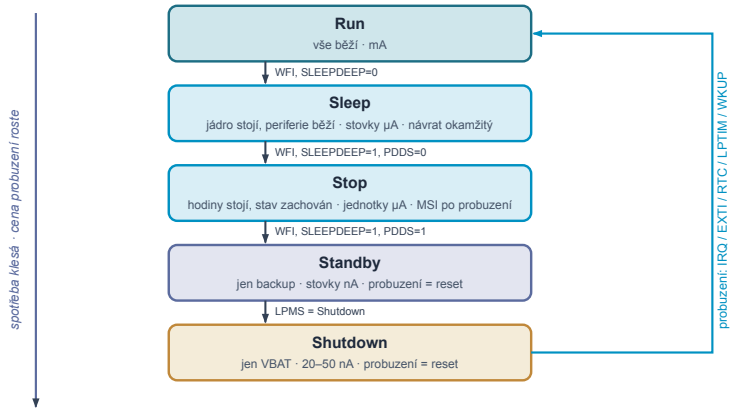
> **Shrnutí oddílu 10.2.** MCU je rozdělen na **napájecí domény** (CORE/PERIPHERAL/SRAM/ANALOG/AON/BACKUP) — hlavní HW mechanismus mikroampérových režimů; software řídí, které přežijí. **VBAT** napájí backup doménu i při `VDD = 0`. Tři regulátory **MR/LPR/Backup**; přechod na LPR vyžaduje nejdřív snížit takt. **DVFS** využívá $P \propto V^2$ — L řady mění napětí za běhu, F řady přes restart PLL. **PLL** je energeticky nejdražší zdroj, proto se základ drží na **MSI/LSI**.

10.3 Úsporné režimy

Největší úsporu dává nepočítat vůbec — spát. Tato část zavede jediné pravidlo (hlubší spánek = nižší proud, ale dražší probuzení), ukáže, jak se do spánku vstupuje, a projde žebřík režimů od Sleep po Shutdown i retenci mezi nimi.

Filozofie a vstup do spánku

MCU přechází mezi režimy podle aktuální aktivity, požadované odezvy a dostupných zdrojů probuzení. Platí jediné pravidlo, které určuje celý návrh: **čím hlubší spánek, tím nižší spotřeba, ale tím vyšší cena probuzení.** Cena má tři složky — latenci (jak dlouho trvá být zase funkční), ztrátu kontextu (co se zapomene — RAM, registry) a energii na samotné probuzení (rozběh oscilátorů, reinicializace).

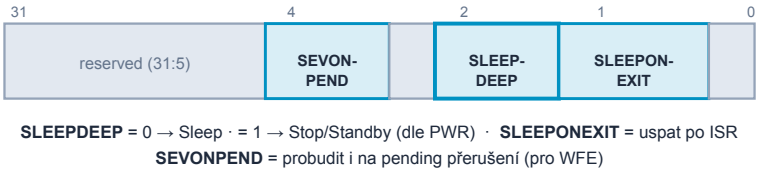


Obrázek 10.3. Režimy procesoru a přechody mezi nimi. Přechodem dolů (Run → Sleep → Stop → Standby → Shutdown) klesá spotřeba, ale roste latence a ztráta stavu při probuzení. Vstup určuje instrukce WFI s bitem SLEEPDEEP a registry PWR; probuzení vrací systém do Run nebo přes reset.

WFI
Wait For Interrupt: instrukce jádra Cortex-M, která uspí procesor do příchodu libovolného přerušení. Podle bitu SLEEPDEEP a registrů PWR znamená mělký Sleep, nebo hluboký Stop či Standby.

WFE
Wait For Event: instrukce jádra Cortex-M, která uspí procesor do příchodu přerušení nebo události. Na rozdíl od WFI ji probudí i událost z periferie, například dokončení přenosu DMA.

SLEEPDEEP
Bit v registru SCB SCR, který rozlišuje mělký Sleep od hlubokých režimů. S nulovým bitem instrukce WFI vstoupí do Sleep, s nastaveným do Stop nebo Standby podle registrů PWR.



Obrázek 10.4. Registr SCB SCR řízení spánku jádra. SLEEPDEEP rozlišuje Sleep (0) od Stop/Standby (1); SLEEPONEXIT uspí jádro automaticky po návratu z obsluhy přerušení; SEVONPEND probudí i na čekající přerušení (pro WFE).

Vstup do spánku: WFI, WFE, SLEEPDEEP

Do spánku se vstupuje instrukcí jádra Cortex-M. **WFI** (Wait For Interrupt) uspí jádro do příchodu **libovolného přerušení**; **WFE** (Wait For Event) ho probudí přerušením **nebo událostí** (i z periferie jako DMA). Zda `WFI` znamená mělký Sleep, nebo hluboký Stop/Standby, rozhoduje bit **SLEEPDEEP** v `SCB->SCR` (spolu s registry PWR).

Bit `SLEEPONEXIT` je mocný vzor pro čistě událostní systém: po návratu z obsluhy přerušení se jádro automaticky uspí, aniž by se vrátilo do hlavní smyčky — mezi událostmi tak neběží žádný kód.

Přehled režimů: Sleep, Stop, Standby, Shutdown

Režimy tvoří žebřík od nejmělčího po nejhlubší. Přechod dolů znamená méně proudu, ale delší probuzení a větší ztrátu stavu.

režim	hodiny/jádro	stav	probuzení	latence	spotřeba
Run	vše běží	—	—	—	mA
Sleep	jádro stojí, periferie běží	zachován	IRQ/event	1–5 μs	stovky μA
Stop	systémové hodiny stojí	zachován	EXTI, RTC, LPTIM, LPUART	50 μs–2 ms	jednotky μA
Standby	vše off krom backup	ztracen	WKUP, RTC, NRST, IWDG	1–5 ms	stovky nA
Shutdown	vše off krom VBAT	ztracen	WKUP, RTC, NRST	10+ ms	20–50 nA

Sleep, Stop, Standby, Shutdown

Sleep je nejmělčí režim: zastaví se jen hodiny jádra, periferie a jejich hodiny běží dál, RAM a registry zůstávají a probuzení je rychlé (1–5 μs) — program pokračuje přesně tam, kde usnul. Hodí se na krátkou nečinnost, kdy periferie musí zůstat aktivní. Sleep je prakticky „bezplatný”: patří do každé čekací smyčky místo aktivního pollingu — kdykoli jádro na něco čeká, mělo by být v Sleep.

```
void enter_sleep(void) {
    SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;   // mělký Sleep
    __WFI();                              // usni; probudí libovolné IRQ
}
```

Stop má výrazně nižší spotřebu: zastaví se systémové hodiny včetně většiny oscilátorů, takže

stojí skoro všechny periferie kromě těch s nezávislým taktem (LSI/LSE pro RTC, LPTIM, LPUART). RAM a registry se zachovají, regulátor běží v low-power režimu, probuzení přes EXTI nebo periferii s vlastním taktem. Po probuzení jádro běží na tom, co bylo hlavním zdrojem (typicky MSI) — PLL a plný takt je nutné znovu nastartovat, což je ona latence 0,5–2 ms.

```
RCC->APB1ENR |= RCC_APB1ENR_PWREN;   // hodiny pro blok PWR
PWR->CR &= ~PWR_CR_PDDS_Msk;         // deep sleep = Stop (ne Standby)
PWR->CR |= PWR_CR_LPDS_Msk;          // regulátor do low-power
SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;   // deep sleep
__WFI();                             // usni; probudí EXTI/RTC/LPTIM
SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk; // po probuzení: rekonfigurace hodin
```

Standby je nejhlubší běžný režim: vypne se hlavní regulátor i všechny oscilátory, žije jen backup doména. **Obsah RAM a registrů se ztratí** (krom backup a retention), takže probuzení je ekvivalentní resetu — firmware se musí znovu inicializovat a stav si vzít z retention RAM. **Shutdown** (jen low-power řady) je absolutní minimum ~20–50 nA: vypne se i BOR, žije jen VBAT. V obou režimech se program po probuzení **nevrací** — probíhá cold boot, proto se stav (co měřit, kolikátý cyklus) ukládá do backup registrů nebo retention RAM.

```
PWR->CR |= PWR_CR_PDDS_Msk;          // deep sleep = Standby
SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
__WFI();                             // vstup do Standby
// KÓD ZDE NEPOKRAČUJE – probuzení = reset, běh se vrací na start main()
```

Retention: mezi Stop a Standby

Retention

Vlastnost hlubokého režimu, kdy zůstane pod napětím jen vybraná paměť nebo registry, zatímco zbytek čipu je vypnutý. Dovolí velmi nízkou spotřebu a přitom návrat bez plné reinicializace, tedy něco mezi režimem Stop a Standby.

Retention není samostatný režim, ale vlastnost hlubokého režimu: určuje, které části zůstanou pod napětím a jaký stav přežije. Leží mezi Stop (stav zachován) a Standby (stav ztracen, reset) a dovolí velmi nízkou spotřebu a přitom návrat bez plné reinicializace. Udržet lze malý blok RAM (4–16 kB), RTC a jeho registry, backup registry, konfiguraci LPTIM/LPUART, konfiguraci GPIO a u podporujících platforem i kontext RTOS (stack a TCB). U STM32 L4/L5/U5 zůstává napájená SRAM2 nebo backup RAM a probuzení proběhne přes reset, po němž si firmware vezme data z retained RAM. Retention je odpověď na otázku „jak spát jako Standby, ale probudit se skoro jako ze Stop" — klíč k periodickému měření, kde nechceme platit plný cold boot každý cyklus.

> **Shrnutí oddílu 10.3.** Jediné pravidlo: **hlubší spánek = nižší proud, vyšší cena probuzení** (latence, ztráta stavu, energie). Vstup: instrukce **WFI/WFE** plus bit **SLEEPDEEP** a registry PWR; `SLEEPONEXIT` uspí po ISR automaticky. **Sleep** (jádro stojí, periferie žijí, návrat okamžitý) → **Stop** (hodiny stojí, stav zachován, MSI po probuzení) → **Standby/Shutdown** (jen backup, probuzení = reset). **Retention** udrží vybrané SRAM banky a registry i v hlubokém režimu.

10.4 Probouzení a energetický kompromis

Spánek má cenu až tehdy, když se z něj systém dokáže probudit. Tato část projde zdroje probuzení a jejich latenci, spočítá, kdy se hlubší režim vyplatí (break-even), a vysvětlí strategii race-to-sleep i časté pasti při ladění.

Zdroje probuzení, latence a break-even

Který zdroj systém probudí, závisí na hloubce režimu — čím hlouběji, tím méně zdrojů zůstává aktivních. **RTC** (z LSE/LSI 32 kHz, 300–600 nA) běží až po Standby a Shutdown a poskytuje datum, čas a alarm. **LPTIM** běží ve Stop za desítky nanoampér a jako one-shot wake-up timer je úspornější než RTC, když nepotřebujeme datum a čas. **EXTI z GPIO** reaguje na hranu (v mělkých režimech) nebo úroveň (v hlubokých); pin musí být v režimu analog nebo pull, aby netekl. **LPUART** umí probudit detekcí start-bitu, **I²C** shodou adresy nebo STOP condition, **komparátor** překročením prahu světla, zvuku či vibrací v ultra-low-power.

Latence probuzení roste s hloubkou hlavně kvůli rozběhu oscilátorů: Sleep 1–5 μs, Stop s MSI 50–200 μs, Stop s PLL/HSE 0,5–2 ms, Standby 1–5 ms (cold boot), Shutdown 10+ ms. Latence není jen prodleva — během ní teče zvýšený proud (rozběh oscilátoru, lock PLL), a právě proto se hluboký režim vyplatí jen při dostatečně dlouhém spánku.

Kdy se hlubší spánek vyplatí: break-even

Celková energie na cyklus má tři složky, `E_total = E_sleep + E_wakeup + E_active`. Hlubší režim sníží `E_sleep`, ale zvýší `E_wakeup`, takže se vyplatí jen tehdy, když je perioda dost dlouhá, aby úspora ve spánku převážila dražší probuzení. Bod zvratu (break-even) je poměr rozdílu energií probuzení a rozdílu spánkových výkonů:

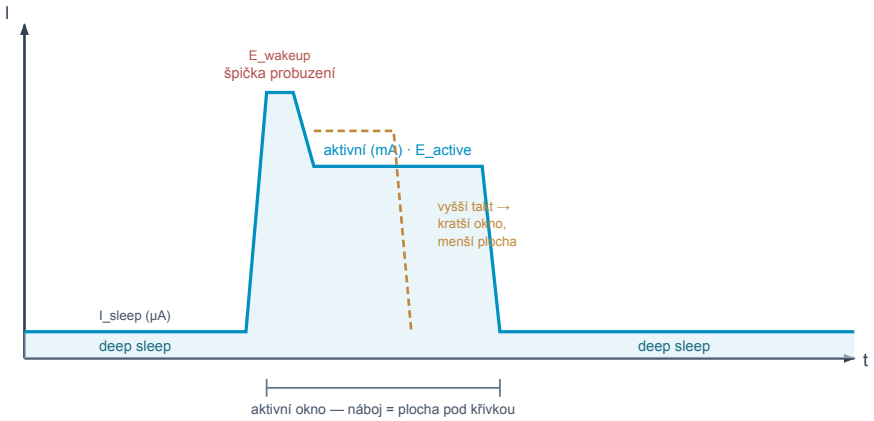
```
T_break-even = (E_wakeup,deep − E_wakeup,shallow) / (P_shallow − P_deep)
```

Pro modelový příklad — Stop (1 μA ve spánku, probuzení 1 μJ) versus Standby (100 nA ve spánku, probuzení 30 μJ) — vychází break-even $\approx$ **33 s**. Pro periody kratší než 33 s je efektivnější **Stop** (levné probuzení), pro delší **Standby** (levný spánek). Hloubka režimu je tedy funkce periody buzení, ne intuice.

Race-to-sleep

Protože aktivní stav je řádově dražší než spánek, nejlepší strategie je být aktivní co nejkratší dobu. **Race-to-sleep** znamená probudit se na nejvyšší rozumné frekvenci (minimalizovat aktivní čas), rychle zpracovat úlohu a okamžitě přejít do nejhlubšího možného režimu.

Race-to-sleep
Strategie, kdy se úloha vykoná co nejrychleji na vysokém taktu a systém se okamžitě vrátí do nejhlubšího možného spánku. Kratší aktivní okno znamená menší plochu pod křivkou proudu, tedy nižší celkovou energii, i když je špičkový proud vyšší.



Obrázek 10.5. Profil proudu během jednoho pracovního cyklu: nízký proud v deep sleep, špička při probuzení, plató v aktivní fázi a návrat do spánku. Náboj (a tedy energie) je plocha pod křivkou; race-to-sleep ji zmenšuje zkrácením aktivního okna, i za cenu vyššího špičkového proudu.

Zní to paradoxně, ale **vyšší takt šetří energii**: doběhnout úlohu na 80 MHz za 1 ms a spát 999 ms je úspornější než ji zpracovávat na 8 MHz po dobu 10 ms, protože integrál pod křivkou

proudu je menší — dřív se vrátíme do spánku. Po probuzení je volba hodin kompromis start-up versus výkon: MSI (~4 MHz, start ~10 μs) pro většinu úloh, HSI (16 MHz, ~2 μs) pro vyšší výkon, PLL (200 μs–2 ms) jen když je nutný.

Pasti a ladění

Časté pasti při probouzení: **floating piny** (I/O leak o několik μA vyšší — nastavit jako analog/pull), **clock gating** (některé wakeup detektory potřebují zapnuté hodiny, například UART RX vyžaduje napájený APB) a **race condition** (po probuzení běží ISR, ale hodiny periferie ještě nejsou připravené). Ladit spánek je zvláštní disciplína, protože **debugger sám mění spotřebu**: SWD/JTAG odebírá 100 μA až 10 mA a blokuje vstup do deep sleep. Proto se měří bez něj a loguje se přes UART/RTT; timing probuzení se měří osciloskopem přes GPIO pin (set na začátku ISR, clear před uspáním). Typické závady: vysoká spotřeba ve spánku (floating GPIO), neprobouzení (špatná hrana/úroveň EXTI), kolísavá spotřeba (běžící periferní clock), špička při probuzení (start-up PLL — budit přes MSI).

> **Shrnutí oddílu 10.4. Zdroje probuzení** klesají s hloubkou (RTC/LPTIM/EXTI/LPUART/komparátor); pro periodu bez data/času je nejúspornější **LPTIM**, pro Standby dedikovaný **WKUP pin**. **Latence** roste s hloubkou (μs → 10+ ms) kvůli rozběhu oscilátorů. Hloubku volí **break-even** perioda. **Race-to-sleep**: vyšší takt a kratší aktivní okno = méně energie. Pozor na floating piny, clock gating a debugger.

10.5 Návrhové vzory nízké spotřeby

Nízká spotřeba není jediná technika, ale skladba návrhových vzorů. Tato část projde událostmi řízený návrh, přesun práce na DMA a fyzické odpojení senzorů, a na případové studii autonomního senzoru ukáže, jak se počítá energetický rozpočet.

Vzory: event-driven, DMA offload, power-gating

Základním vzorem je **událostmi řízený** firmware: procesor je většinu času ve spánku a probouzí ho jen konkrétní událost (EXTI, LPTIM, ADC event, dokončení DMA). Veškerá logika je stavový automat (událost → přechod → akce → zpět do spánku) — žádný polling, žádné busy-wait. Návrh má čtyři zásady: identifikovat události, které skutečně vyžadují CPU; použít EXTI/LPTIM/RTC místo pollovacích smyček; v obsluze přerušení jen signalizovat a práci přesunout do úlohy; a po dokončení race-to-sleep. V RTOS je to idle hook s `WFI` nebo tickless režim (oddíl 10.7).

DMA offload

Druhým vzorem je přesun práce na **DMA místo procesoru** (přednáška 5): data z ADC, UART nebo SPI zpracovává DMA a procesor se probudí jen na významnou událost (naplněný buffer), nebo vůbec. Typicky ADC → DMA v kruhovém či dvojitém bufferu → RAM, s přerušením half-transfer a transfer-complete jen tehdy, když CPU opravdu musí. Efekt je zásadní: procesor může spát **i při vysoké frekvenci vzorkování**. Zmíněná úskalí platí — zarovnání a koherence cache (na MCU s D-cache invalidovat či čistit), bezpečné sdílení bufferu mezi ISR a hlavní smyčkou, a ne každá periferie umí DMA v hlubokém Stop (ověřit v datasheetu).

Power-gating senzorů

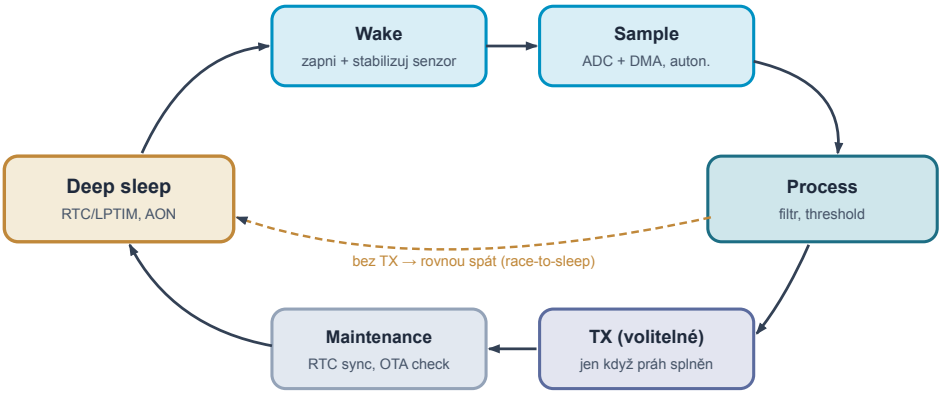
Třetím vzorem je power-gating: senzory a externí moduly se napájejí odděleně a zapínají jen pro měření nebo vysílání. Spínačem je P-FET (load switch) s pinem `EN` řízeným z GPIO. Řeší to, co software nedokáže — **statickou** spotřebu externího obvodu: senzor s klidovým proudem

Power-gating
Fyzické odpojení napájení bloku nebo externího senzoru spínačem (load switch), typicky P-FET řízeným z GPIO. Odstraní i statickou spotřebu externího obvodu, kterou software vypnout nedokáže.

50 μA, zapnutý 1 s z 900 s, přispěje průměrem jen ~55 nA, ale jen když ho fyzicky odpojíme. Pozor na čas stabilizace senzoru (typicky 1–10 ms), který se často podcení, takže se měření zbytečně opakuje.

Případová studie a energetický rozpočet

Uvažme úlohu: každých 15 minut přečíst senzor, lokálně rozhodnout, odeslat data jen když je to nutné, jinak co nejnižší průměrná spotřeba; cílem je životnost ≥ 1 rok na CR2032, nebo autonomní provoz se solárním panelem a superkapacitou. Pracovní cyklus je stavový automat:



Obrázek 10.6. Stavový automat autonomního senzoru: z deep sleep přes probuzení a stabilizaci senzoru k odběru vzorků (ADC a DMA), rychlému zpracování a volitelnému vysílání, načež se systém vrací do deep sleep. Bez vysílání se přechází rovnou do spánku (race-to-sleep).

Klíčové hardwarové vzory: load switch pro power-gating, LDO s nízkým klidovým proudem (pod 1 μA) nebo buck-boost měnič, u harvestingu MPPT front-end a superkapacitor (0,1–1 F) jako buffer na vysílací špičky. Softwarové vzory: event-driven nebo tickless RTOS, race-to-sleep, DMA a autonomní periferie, power-aware ovladače se suspend/resume, DVFS a fail-safe přes IWDG (oddíl 10.6).

Energetický rozpočet

Životnost se nespočítá z odběru v jednom stavu, ale z **integrálu proudu přes celý cyklus** `T = 900 s`. Rozpad jednoho cyklu (bez vysílání): start senzoru 0,05 s při 5,0 mA (0,25 mA·s), vzorkování 1,00 s při 10,0 mA (10,00 mA·s), zpracování 0,10 s při 5,0 mA (0,50 mA·s) a spánek 898,85 s při 1,0 μA (0,90 mA·s). Aktivní náboj 10,75 mA·s plus spánek 0,90 mA·s dá 11,65 mA·s na cyklus, takže průměrný proud je

```
I_avg = 11,65 mA·s / 900 s ≈ 12,9 µA
```

S vysíláním v 10 % cyklů vyjde `I_avg ≈ 13 µA`. Životnost na CR2032 ($\approx$ 220 mAh) je pak `t = 220 / 0,013 ≈ 16 900 h ≈ 1,9 roku`.

Alternativou je **energy harvesting**. Průměrný výkon systému je `P_avg = 13 µA · 3,3 V ≈ 43 µW`. Malý solární panel 4–10 cm² dodá i uvnitř budovy stovky μW až jednotky mW, tedy výrazně víc než 43 μW. Superkapacitor 0,1 F při 3,3 V uchová `E = ½·C·V² ≈ 0,54 J`, což je energie na stovky vysílacích špiček (jedna $\approx$ 1,6 mJ). Průměrný proud v jednotkách μA je hranice, za kterou přestává jít o „výdrž baterie" a začíná energetická soběstačnost; rozhoduje o ní celý řetěz od rodiny MCU přes hloubku spánku a délku aktivního okna po power-gating periferií.

Shrnutí oddílu 10.5. Event-driven = spánek plus stavový automat buzený událostí (v ISR pouze signalizovat). **DMA offload** (přednáška 5) nechá procesor spát i při vysokém vzorkování; pozor na koherenci cache. **Power-gating** (load switch) odřízne statickou spotřebu externích senzorů. Životnost se počítá z **integrálu proudu přes cyklus** (`I_avg`); jednotky μA otevírají **energy harvesting**.

10.6 Spolehlivost: supervize a watchdog

Zařízení, které spí roky bez obsluhy, musí zůstat funkční i při poruchách, které za ty roky nastanou. Tato část ukáže hardwarové pojistky, které běží, i když hlavní program selže — supervizi napětí (BOR, PVD) a watchdog.

Proč spolehlivost patří ke spotřebě

Zařízení, které spí roky bez obsluhy, musí automaticky zajistit dvě věci: detekovat pokles napájení (vybitá baterie, výpadek harvestingu), a zotavit se ze zaseknutí (software uvázl, periferie se zablokovala). Obojí je hardwarová pojistka, která běží i tehdy, když hlavní program selže. Bez nich je „low-power" jen teorie: baterie klesá, MCU se dostane pod funkční napětí a začne zapisovat poškozená data, nebo se zasekne ve spánku a nikdy se neprobudí. Spolehlivost je podmínka, ne bonus.

Supervize napětí: BOR a PVD

BOR (Brown-out Reset) tvrdě **resetuje MCU** při poklesu V_{DD} pod kritický práh (konfigurovatelný, např. 1,7 / 2,0 / 2,2 / 2,4 V). Je aktivní i v nejhlubších režimech, odebírá ~1–5 μA a u většiny MCU jej nelze vypnout — chrání před nepředvídatelným chováním při nízkém napětí. **PVD** (Power Voltage Detector) je oproti tomu softwarově konfigurovatelný detektor s **přerušením**: nastaví se práh vyšší než BOR, takže program dostane čas zareagovat (uložit stav) dřív, než přijde tvrdý BOR reset.

```
PWR->CR |= PWR_CR_PLS_LEV5;   // práh PVD ~2,5 V (nad BOR)
PWR->CR |= PWR_CR_PVDE;       // povolit PVD
EXTI->IMR |= EXTI_IMR_MR16;   // PVD je na EXTI lince 16
EXTI->RTSR |= EXTI_RTSR_TR16; // náběžná hrana = VDD klesá pod práh
NVIC_EnableIRQ(PVD_IRQn);
```

Typický řetěz supervize: PVD přijde první (práh 2,5 V) a dá programu čas uložit kritická data do backup SRAM nebo Flash; teprve pak BOR (2,0 V) provede čistý reset na již konzistentním stavu. Sekvence PVD → uložení → BOR reset je základem odolnosti proti vybíjení baterie — data se zapíšou, dokud je napětí ještě v pořádku.

Watchdog: nezávislý strážce běhu

Watchdog je časovač, který program musí periodicky obsluhovat (refresh); když to přestane dělat, protože se zasekl, watchdog vyprší a **resetuje MCU**. Je to poslední pojistka proti uváznutí bez obsluhy. STM32 má dva druhy. **IWDG** (Independent Watchdog) má vlastní oscilátor LSI, běží i ve Stop a Standby a přežije i výpadek hlavních hodin — je to robustní hlídač „systém žije". **WWDG** (Window Watchdog) je taktovaný systémovými hodinami a vyžaduje refresh jen uvnitř **časového okna**, takže odhalí i program, který běží příliš rychle (příznak rozběhlé smyčky). Pro low-power se používá hlavně IWDG, protože běží i v hlubokém spánku.

BOR
Brown-out Reset: hardwarová pojistka, která resetuje MCU při poklesu napájecího napětí pod konfigurovatelný kritický práh. Je aktivní i v nejhlubších režimech a u většiny MCU ji nelze vypnout.

PVD
Power Voltage Detector: softwarově nastavitelný detektor napájecího napětí s přerušením. Práh se volí nad prahem BOR, aby program dostal čas uložit kritická data dřív, než přijde tvrdý reset.

IWDG
Independent Watchdog: watchdog s vlastním oscilátorem LSI, který resetuje MCU, pokud ho program přestane periodicky obsluhovat. Běží i ve Stop a Standby a přežije i výpadek hlavních hodin.

WWDG
Window Watchdog: watchdog taktovaný systémovými hodinami, který vyžaduje obsluhu jen uvnitř časového okna. Kromě zaseknutí odhalí i příliš rychlý, rozběhlý běh programu, kdy obsluha přijde dříve, než má.

IWDG a fail-safe

IWDG se ovládá přes „klíčový" registr `KR`, kam se zapisují magické hodnoty (odemknutí zápisu, spuštění, refresh) — návrh odolný proti náhodnému přepisu. Watchdog jednou spuštěný **nelze zastavit**.

```
IWDG->KR  = 0x5555;          // odemkni přístup k PR/RLR
IWDG->PR  = IWDG_PR_PR_2;    // prescaler LSI (např. /64)
IWDG->RLR = 0x0FFF;          // reload -> timeout (LSI ~32 kHz)
IWDG->KR  = 0xCCCC;          // spusť watchdog (už ho nelze zastavit)

// v hlavní smyčce / úloze, dokud systém žije:
IWDG->KR  = 0xAAAA;          // refresh — "jsem naživu"
```

Po resetu lze z reset flagů v `RCC_CSR` zjistit **příčinu** — odlišit zapnutí napájení, externí reset, watchdog a probuzení z hlubokého spánku — a podle toho zvolit normální start, nebo safe mode.

```
if (RCC->CSR & RCC_CSR_IWDGRSTF) enter_safe_mode();  // reset způsobil watchdog
RCC->CSR |= RCC_CSR_RMVF;                            // vymaž reset flagy
```

Volba timeoutu je kompromis: příliš krátký způsobí falešné resety v legitimně dlouhé operaci, příliš dlouhý zpomalí zotavení. Watchdog se obsluhuje z jednoho místa, ne roztroušeně — jinak by zakryl právě tu chybu, kterou má odhalit.

> **Shrnutí oddílu 10.6.** Zařízení bez obsluhy potřebuje **hardwarové pojistky**, které běží, i když hlavní program selže. **BOR** tvrdě resetuje pod kritickým napětím; **PVD** (vyšší práh, přerušení) dá čas uložit stav — sekvence PVD → uložení → BOR reset chrání data při vybíjení. **Watchdog** resetuje zaseknutý systém: **IWDG** (vlastní LSI, běží i ve Stop/Standby) hlídá „systém žije", **WWDG** (okno) chytá i předčasný běh. Po resetu se zjistí příčina z `RCC_CSR` a případně přejde do safe mode.

10.7 RTOS v low-power

RTOS a hluboký spánek si nemusí odporovat. Tato část ukáže, že idle task je správné místo pro uspání jádra, jak tickless režim odstraní zbytečná probouzení a jak nejúspornější vzor spojí přerušení, notifikaci a úlohu.

Idle task a tickless idle

V RTOS (přednáška 9) plánovač spustí idle task, když jsou všechny ostatní úlohy blokované — to je přesně okamžik, kdy žádná úloha nevyžaduje procesor, a tedy správné místo pro uspání jádra. Nejjednodušší integrace je idle hook, který jen zavolá `WFI`:

```
void vApplicationIdleHook(void) {
    __WFI();     // když nemá běžet žádná úloha, uspi jádro
}
```

Aby úlohy skutečně uvolnily jádro, nesmějí pollovat — musí se **blokovat** (`vTaskDelay`, čekání na notifikaci nebo frontu). Blokovaná úloha nespotřebovává nic; teprve pak se plánovač dostane do idle a jádro usne.

Tickless idle
Režim RTOS, kdy se v době nečinnosti zastaví periodický systémový tik a jádro se uspí na jeden dlouhý interval měřený úsporným časovačem (RTC nebo LPTIM). Odstraní zbytečná probouzení každou milisekundu a umožní hluboký spánek mezi událostmi.

Tickless idle

Problém obyčejného idle hooku je periodický **SysTick**, který budí jádro každou milisekundu, i když se nic neděje — tisíc zbytečných probuzení za sekundu znemožní hluboký spánek. Řešením je **tickless idle**: na dobu nečinnosti se tik zastaví a jádro se uspí na jeden dlouhý interval měřený úsporným časovačem (RTC/LPTIM). Ve FreeRTOS stačí povolit `configUSE_TICKLESS_IDLE` a nastavit minimální dobu nečinnosti; plánovač pak volá hook `vPortSuppressTicksAndSleep(xExpectedIdleTime)` a sám spočítá, na kolik taktů se má uspat.

```
void vPortSuppressTicksAndSleep(TickType_t xExpectedIdleTime) {
    portNVIC_SYSTICK_CTRL_REG &= ~portNVIC_SYSTICK_ENABLE_BIT;  // stop SysTick
    Setup_RTC_Wakeup(xExpectedIdleTime);   // RTC/LPTIM za (idle × tick)
    PreSleepProcessing();                  // vypnout periferie, ztlumit I/O
    HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON, PWR_STOPENTRY_WFI);
    PostSleepProcessing();                 // obnovit hodiny, PLL, periferie
    portNVIC_SYSTICK_CTRL_REG |= portNVIC_SYSTICK_ENABLE_BIT;   // restart SysTick
}
```

Funkce `PreSleepProcessing` a `PostSleepProcessing` jsou power-aware ovladače z oddílu 10.5: suspend a resume periferií kolem spánku. Bez nich zůstane viset zapnutá periferie a spánek nebude hluboký. Tickless naplňuje to, co bylo předznamenáno v přednášce 9 — RTOS umí spát mezi událostmi stejně hluboko jako holý event-driven kód, jen s pohodlím úloh, priorit a synchronizace.

Event-driven v RTOS: ISR → notify → task

Nejúspornější vzor spojuje vše dohromady: žádný polling, úloha spí blokovaná na notifikaci a obsluha přerušení ji probudí jen když přijdou data; mezi událostmi je jádro v tickless Stopu. **Direct-to-task notification** je nejrychlejší a nejlevnější mechanismus FreeRTOS (rychlejší než semafor či fronta) — budí přímo jednu konkrétní úlohu s minimální režií. Příklad příjmu z UARTu:

```
void USART2_IRQHandler(void) {                      // přišel bajt
    BaseType_t woken = pdFALSE;
    rxBuffer[rxWriteIndex++] = LL_USART_ReceiveData8(USART2);
    vTaskNotifyGiveFromISR(uartTaskHandle, &woken);  // probuď úlohu
    portYIELD_FROM_ISR(woken);                        // hned přepni, je-li vyšší
}

void vUartTask(void *arg) {
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);      // spí, dokud nepřijde notify
        while (rxReadIndex != rxWriteIndex)
            processByte(rxBuffer[rxReadIndex++]);
        // hotovo → blokuje se → plánovač jde do idle → jádro spí
    }
}
```

Tentýž vzor funguje pro periodické buzení (RTC alarm probudí úlohu místo `vTaskDelay`, který udržuje systémový tik v chodu) i pro DMA a ADC pipeline (úloha se budí jen na událost half-transfer nebo transfer-complete, přednáška 5). Efekt je stejný: žádné tick-based probouzení, jádro se zapne jen při přenosu dat a po zpracování bloku se scheduler vrátí do idle a uspí ho.

Shrnutí oddílu 10.7. Idle task (přednáška 9) je správné místo pro uspání jádra — ale jen když se úlohy blokují, ne pollují. **Tickless idle** zastaví periodický SysTick a uspí jádro na jeden dlouhý interval (RTC/LPTIM) přes `vPortSuppressTicksAndSleep`. `PreSleepProcessing`/`PostSleepProcessing` jsou power-aware suspend/resume periferií. Nejúspornější vzor: **ISR** → **direct-to-task notify** → **úloha**, mezi událostmi tickless Stop.

10.8 Syntéza

Zbývá vrstvy poskládat do jednoho receptu. Tato závěrečná část shrne cestu k mikroampérovému systému od volby rodiny MCU po tickless RTOS, připomene, že spolehlivost je podmínka, a uzavře oblouk operací na watt.

Recept na mikroampérový systém

Nízká spotřeba není jediná technika, ale hierarchie rozhodnutí, kde každá vrstva odřezává jiný kus proudu. Prvním a největším je **volba rodiny MCU** (ultra-low-power řada s dobrým μA/MHz, hlubokými režimy a retencí). Následuje **spát co nejhlouběji** podle periody buzení (Sleep → Stop → Standby, podle break-even), **race-to-sleep** (krátké aktivní okno na vysokém taktu), **event-driven** návrh (žádný polling, buzení přes RTC/LPTIM/EXTI/DMA), **DMA a autonomní periferie** (data bez procesoru, přednáška 5), **power-gating** externích senzorů a rádia (statická spotřeba, kterou software nevypne) a nakonec **tickless RTOS** (pohodlí úloh bez zabití spánku periodickým tikem). Žádná vrstva sama nestačí: průměrný proud je integrál přes celý cyklus a dominuje mu ten článek, který se zapomene — floating pin, běžící PLL, nezavřený clock gate nebo tik SysTicku.

Spolehlivost a uzavření oblouku ops/watt

Spolehlivost drží systém pohromadě po celou dobu jeho života bez obsluhy: BOR a PVD ohlídají klesající napájení a dají čas uložit stav, IWDG (vlastní LSI, běží i ve Stop a Standby) resetuje zaseknutý systém a po resetu `RCC_CSR` řekne příčinu, takže lze zvolit normální start nebo safe mode. Nízká spotřeba a spolehlivost jsou dvě strany téže mince: systém, který spí roky, musí zůstat odolný vůči poruchám, které za ty roky nastanou — vybití, výpadek, zaseknutí.

Uzavření oblouku ops/watt

Přednáška 8 zavedla měnu moderního embedded — operace na joule, ne hertze. Tato přednáška ukázala její druhou polovinu: nejen efektivně počítat (multicore, akcelerátory), ale hlavně většinu času nepočítat vůbec a spát. Přednáška 9 mezi ně vložila strukturovanou nečinnost (idle task), a tato přednáška ukázala, jak z ní udělat hluboký spánek. Nejúspornější operace je ta, kterou neuděláš; druhá nejúspornější je ta, po které hned usneš.

Shrnutí oddílu 10.8. Nízká spotřeba je návrh **celého cyklu**, ne jeden registr: rodina MCU → hloubka spánku → race-to-sleep → event-driven → DMA offload → power-gating → tickless RTOS. Průměrný proud je integrál přes cyklus a dominuje mu nejslabší článek. **Spolehlivost** (BOR/PVD, IWDG, reset flagy) je podmínkou, aby systém přežil roky bez obsluhy. Vše dohromady uzavírá oblouk **ops/watt** otevřený v přednášce 8.

10.9 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 10.1 — Odkud se bere spotřeba

1. Vysvětlete rozdíl mezi dynamickou a statickou spotřebou a uveďte, čím se každá potlačuje.
2. Odvoďte ze vzorce $P \propto C \cdot V^2 \cdot f$, proč je škálování napětí účinnější než škálování frekvence.
3. Co je μA/MHz a proč je volba rodiny MCU řádové rozhodnutí?
4. Proč statická spotřeba roste s teplotou a proč ji nelze snížit softwarem?
5. Proč se spotřeba měří jako profil v čase a jaké dvě chyby nejčastěji zkreslí měření ve spánku?

K oddílu 10.2 — Napájecí architektura STM32

1. Proč se moderní MCU dělí na napájecí domény a co to umožňuje?
2. Jakou zvláštní roli hraje VBAT a co díky ní zůstane funkční při výpadku VDD?
3. Proč MCU nabízí několik regulátorů jádra (VCORE) a co musí předcházet přechodu na Low-Power Regulator?
4. Jak se liší DVFS u řad L a F a proč?
5. Proč je PLL energeticky nevýhodný a jakou strategii taktování z toho plyne?

K oddílu 10.3 — Úsporné režimy

1. Formulujte základní pravidlo úsporných režimů a rozepište tři složky ceny probuzení.
2. Jak instrukce WFI/WFE a bity SLEEPDEEP/SLEEPONEXIT řídí, kdy a jak hluboko procesor usne?
3. Čím se liší Sleep a Stop z hlediska toho, co běží a jak vypadá probuzení?
4. Proč kód po vstupu do Standby nepokračuje a kam se ukládá stav mezi cykly?
5. Co je retention a jaký problém řeší oproti Stop a Standby?

K oddílu 10.4 — Probouzení a energetický kompromis

1. Seřaďte zdroje probuzení podle hloubky režimu, ve kterém fungují, a řekněte, kdy je LPTIM lepší než RTC.
2. Proč latence roste s hloubkou a proč není zdarma?
3. Odvoďte break-even periodu a vysvětlete, kdy zvolit Stop a kdy Standby.
4. Vysvětlete, proč vyšší takt může snížit celkovou energii (race-to-sleep).
5. S jakými úskalími je nutné při probouzení počítat a proč se low-power ladí bez připojeného debuggeru?

K oddílu 10.5 — Návrhové vzory nízké spotřeby

1. Popište vzor event-driven a řekněte, co patří do ISR a co do úlohy.
2. Jak DMA offload umožní procesoru spát i při vysoké frekvenci vzorkování a jaká úskalí (koherence cache, sdílení bufferu mezi ISR a hlavní smyčkou, podpora DMA v hlubokém Stop) je přitom nutné ohlídat?
3. Co řeší power-gating, co software nedokáže, a proč je důležitý čas stabilizace?
4. Pracovní cyklus trvá 900 s: start senzoru 0,05 s @ 5,0 mA, vzorkování 1,00 s @ 10,0 mA, zpracování 0,10 s @ 5,0 mA a spánek 898,85 s @ 1,0 μA. Spočítejte průměrný proud a životnost na baterii CR2032 (220 mAh).
5. Kdy má smysl energy harvesting a jakou roli hraje superkapacitor?

K oddílu 10.6 — Spolehlivost: supervize a watchdog

1. Proč spolehlivost patří k tématu nízké spotřeby?
2. Jak se liší BOR a PVD a proč se volí práh PVD nad prahem BOR?
3. Popište sekvenci PVD → uložení → BOR reset a co chrání.
4. Srovnejte IWDG a WWDG — takt, chování ve spánku a co každý odhalí.
5. Jak se z `RCC_CSR` pozná příčina resetu a proč se watchdog obsluhuje z jednoho místa?

K oddílu 10.7 — RTOS v low-power

1. Proč je idle task správné místo pro uspání a co musí úlohy dělat, aby se do něj plánovač dostal?
2. Jaký problém řeší tickless idle a jak?
3. Co dělají `PreSleepProcessing` a `PostSleepProcessing` a proč jsou nutné?
4. Proč je direct-to-task notification vhodnější pro low-power než semafor nebo fronta?
5. Jak vzor ISR → notify → task odstraní tick-based probouzení?

11

Číslicové zpracování signálů na MCU

> Tato kapitola řeší, **jak z proudu vzorků ze senzoru na MCU v reálném čase získat informaci — a jaká aritmetika (plovoucí čárka, pevná čárka, FPU, SIMD) je na to správná**. Mikrokontrolér se stává **edge procesorem**: z dat extrahuje *informaci*, ne aby je jen přeposílal do cloudu (drahé, pomalé, často nemožné a zbytečné).

Obsah kapitoly

11.1 Proč DSP na MCU
11.2 Číselné formáty
11.3 FPU a DSP instrukce
11.4 Číslicová filtrace
11.5 FFT a spektrální analýza
11.6 Matematické akcelerátory (STM32G4)
11.7 Číslicový PID regulátor
11.8 Syntéza
11.9 Kontrolní otázky

11.1 Proč DSP na MCU

Mikrokontrolér se stává edge procesorem, který z dat získá informaci na místě. Tato část ukáže tři důvody počítat lokálně, zavede klíčovou veličinu — rozpočet cyklů na vzorek — a nastíní dilema mezi plovoucí a pevnou řádovou čárkou.

MCU jako edge procesor: determinismus a rozpočet cyklů

Moderní senzorické aplikace generují velké množství dat, ale přenášet je všechna do vyšších vrstev bývá drahé (energie, zvlášť u baterie), pomalé, často nemožné (konektivita, bezpečnost) a hlavně zbytečné, protože většina dat je nezajímavá. Proto se zpracování přesouvá přímo na MCU, který z dat extrahuje informaci místo posílání syrových vzorků. Počítat lokálně má tři důvody: **determinismus** (real-time odezva), **nízkou spotřebu** (proti mobilnímu procesoru) a **autonomii se soukromím** (data neopouštějí zařízení).

Determinismus a rozpočet cyklů

DSP úlohy potřebují konstantní latenci (typicky 10 až 200 mikrosekund), a přesně to Cortex-M dává: žádné vrstvy operačního systému mezi kódem a hardwarem (RTOS je deterministické, přednáška 9), na jádrech M0 a M3 žádná penalta za výpadek cache a předvídatelná latence přerušení (přednáška 2). Vzorek dorazí přerušením nebo přes DMA a musí být zpracován dřív,

než přijde další. Klíčovou veličinou je proto **rozpočet cyklů na vzorek**: při vzorkování 44,1 kHz má procesor mezi dvěma vzorky zhruba 3800 taktů na 168 MHz. Celá přednáška je o tom, jak se do tohoto rozpočtu vejít. Typické real-time úlohy jsou řízení motorů a robotů, fúze dat z inerciální jednotky, aktivní potlačení hluku ve sluchátkách a audio efekty.

Spotřeba a dilema FPU

DSP pipeline na MCU odebírá jednotky až desítky miliampér, proti stovkám miliampér u mobilního procesoru — to otevírá aplikace, které by na aplikačním procesoru nešly (nositelná elektronika, bateriové IoT senzory, prediktivní údržba). Příklad z přednášky 10: analýza vibrací, úloha 2 až 3 ms každých 20 ms, znamená roční provoz na jednom knoflíkovém článku.

Historicky byly projekty na MCU „binární" a s plovoucí řádovou čárkou se šetřilo. Dnes je volba návrhový kompromis. Pro plovoucí čárku s FPU mluví nižší riziko přetečení, lepší přesnost u malých amplitud a to, že filtry a FFT ve `float` bývají rychlejší než v pevné čárce, *má-li MCU FPU*. Pro pevnou čárku mluví, že FPU není vždy (jádra M0, M0+ a M3 ji nemají, low-power režimy ji vypínají), že pevná čárka bývá rychlejší (SIMD MAC), úspornější a determinističtější, a že obsluha přerušení s FPU ukládá velký kontext. Nejefektivnější DSP obvykle obě aritmetiky kombinuje: filtry v pevné čárce, přesnostně citlivé nebo AI části v plovoucí.

> **Shrnutí oddílu 11.1.** MCU je **edge procesor** — extrahuje z dat informaci, místo aby posílal syrová data. Počítat lokálně má tři důvody: **determinismus** (latence 10 až 200 mikrosekund), **nízkou spotřebu** a **autonomii se soukromím**. Klíčová veličina je **rozpočet cyklů na vzorek**. Volba **plovoucí versus pevná čárka** je kompromis (FPU není vždy, obsluha přerušení s FPU je drahá); nejlepší řešení často obě aritmetiky kombinuje.

11.2 Číselné formáty

Volba čísel rozhoduje o ceně DSP víc než algoritmus. Tato část projde plovoucí řádovou čárku i její pasti a cenu bez FPU a pevnou řádovou čárku ve formátu Q se saturací a zaokrouhlením.

Plovoucí řádová čárka: IEEE 754, pasti a cena bez FPU

Reálné číslo se v plovoucí čárce ukládá jako znaménko krát mantisa krát dvě na exponent. **Float32** (znaménko, osmibitový exponent, dvacetitříbitová mantisa) je základním pracovním formátem DSP na MCU. `double` (64 bitů) nemá na většině jader hardwarovou podporu a v softwarové emulaci je velmi pomalé, takže se mu v embedded vyhýbáme. `half` (float16) podporuje až Helium na M55; `bfloat16` obětuje mantisu za rozsah `float32` a míří do AI (přednáška 12). Pozor na příponu literálů a funkcí: `sinf`, `1.0f`, `fabsf` — bez `f` počítá kompilátor v `double` a i na M4F zavolá pomalou emulaci.

Float32
Jednoduchá přesnost IEEE 754: dvaatřicet bitů (znaménko, osmibitový exponent s posunem, dvacetitříbitová mantisa). Hardwarově podporovaná jednotkou FPU jader Cortex-M4F a M7F; pracovní formát DSP na MCU.

Pasti plovoucí řádové čárky

Plovoucí čárka má **konečné rozlišení, které se mění s velikostí čísla**. Strojové epsilon je nejmenší číslo, které přičtené k jedné ještě změní výsledek, přibližně 1,19 krát deset na mínus sedmou. Krok mezi sousedními čísly roste s absolutní hodnotou (kolem jedné je asi deset na mínus sedmou, kolem milionu už asi 0,125). Z toho plynou dva praktické důsledky: čísla se **nikdy neporovnávají operátorem rovnosti**, ale testem `fabsf(a-b) < eps`, a **sčítání čísel s velmi rozdílným exponentem** ztratí mantisu menšího (`1000000.0f + 1.0f` zůstane `1000000.0f`). Speciální hodnoty (nula, nekonečno, NaN) mají vyhrazené kódy exponentu; `NaN` se šíří výpočtem a neplatí pro něj rovnost sama se sebou, což často odhalí chybu (dělení nulou, odmocnina ze záporu).

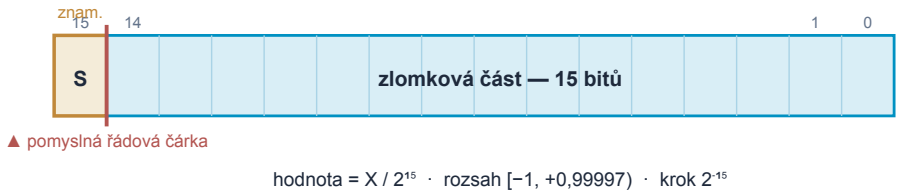
Cena plovoucí čárky bez FPU

Bez FPU emuluje jádro standard IEEE 754 knihovnou (`libgcc`, funkce `__aeabi_*`), a jedna operace tak stojí desítky až stovky taktů: sčítání zhruba 80 až 120, násobení 120 až 200, dělení 300 až 800 a odmocnina 600 až 1200 taktů. Na jádře bez FPU je tedy každá operace v plovoucí čárce voláním funkce; filtr s osmdesáti násobeními na vzorek se tím stane nerealizovatelným v reálném čase — a to je hlavní důvod pro pevnou řádovou čárku.

Pevná řádová čárka: Qm.n, saturace a zaokrouhlení

Fixed-point Qm.n reprezentuje reálné číslo celým číslem s pomyslnou řádovou čárkou na pevné pozici: `m` bitů celé části (se znaménkem) a `n` bitů zlomkové, hodnota je `X / 2^n`.

Fixed-point Qm.n
Reprezentace reálného čísla celým číslem s pevnou pozicí řádové čárky: m bitů celé části (se znaménkem) a n bitů zlomkové. Hodnota je celé číslo dělené dvěma na n-tou a počítá se celočíselnou aritmetikou, tedy i na jádře bez FPU.



Obrázek 11.1. Rozložení formátu Q1.15 v šestnáctibitovém čísle: nejvyšší bit je znaménko, za pomyslnou řádovou čárkou následuje patnáct zlomkových bitů. Hodnota je celé číslo dělené dvěma na patnáctou, rozsah je od minus jedné do téměř plus jedné.

CMSIS definuje standardní typy: `q7_t` (Q0.7 v `int8_t`), `q15_t` (Q1.15 v `int16_t`), `q31_t` (Q1.31 v `int32_t`, standard CMSIS-DSP) a `q63_t` jako akumulační typ. Q15 se tak ukládá v `int16_t` a počítá se celočíselnou ALU na každém Cortex-M, i bez FPU; cenou je pevný rozsah přibližně od minus jedné do plus jedné a nutnost hlídat přetečení. Převod z plovoucí čárky je násobení mocninou dvou (`(q15_t)(x * 32768.0f)`), mezi dvěma Q formáty jde o bitový posun. Zásadní je pravidlo násobení: `Q_n` krát `Q_n` dá `Q_2n`, takže součin má dvojnásobek zlomkových bitů a musí se posunout zpět o `n`. Proto se v DSP akumuluje do širšího typu (`q31_t` nebo `q63_t`) a teprve na konci se výsledek ořízne zpět na Q15.

Saturaci
Řízení přetečení, kdy se výsledek místo otočení omezí na maximální nebo minimální hodnotu formátu. Chyba tak zůstane omezená a předvídatelná; v číslicovém zpracování signálu je saturace pravidlem, ne výjimkou.

Ořezání
Zpracování spodních bitů po posunu pevné čárky pouhým zahozením (aritmetický posun vpravo). Levné, ale zaokrouhluje vždy směrem dolů, a vnáší proto systematickou zápornou chybu (stejnosměrný posun), která se ve zpětnovazebních filtrech hromadí.

Zaokrouhlení
Zpracování spodních bitů po posunu pevné čárky, kdy se před posunem přičte polovina kroku (2 na n minus 1). O jednu instrukci dražší než ořezání, ale chyba je nezkreslená se střední hodnotou nula; nutné ve zpětnovazebních (IIR) filtrech.

Přetečení a saturace

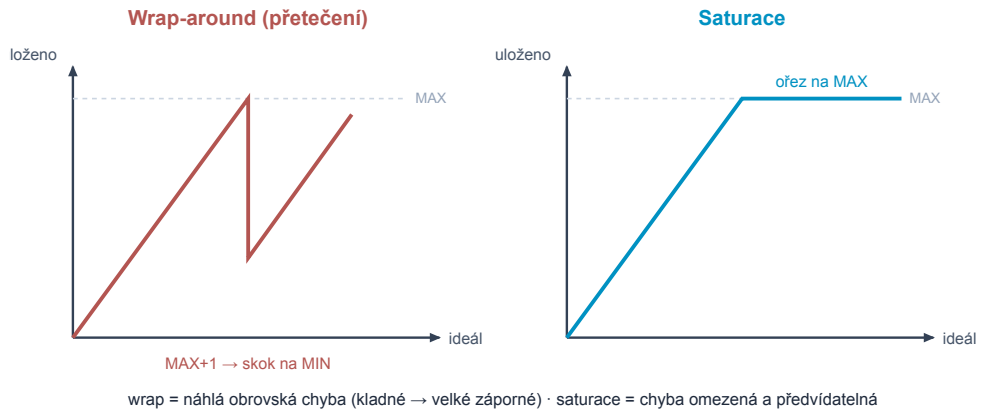
Když výsledek překročí rozsah formátu, standardní celočíselná aritmetika přeteče otočením (wrap-around): maximum plus jedna se stane minimem. V signálu je to katastrofa — kladná špička se skokem překlopí na velkou zápornou hodnotu a filtr se rozkmitá.

DSP proto používá **saturaci**: výsledek se místo otočení omezí na maximum nebo minimum. Cortex-M4 a M7 mají dedikované saturující instrukce (`__SSAT(x, 16)` sytí na šestnáctibitový rozsah v jedné instrukci); saturují i `QADD` a knihovní `arm_add_q15`.

Zaokrouhlení versus ořezání

Po násobení `Q_n` krát `Q_n` vzniká výsledek `Q_2n`, který se **posouvá zpět** o `n` bitů — a odseknuté spodní bity je nutné zpracovat. **Ořezání** (truncation) je zahodí prostým posunem `>> n`; je levné, ale zaokrouhluje vždy dolů, a vnáší tak systematickou zápornou chybu — stejnosměrný posun signálu. **Zaokrouhlení** (rounding) před posunem přičte polovinu kroku (`+ 2^{n-1}`, pak `>> n`); stojí o instrukci víc, ale chyba je nezkreslená se střední hodnotou nula:

```
int16_t trunc = (int16_t)(acc >> 15);                // ořezání: bias dolů
int16_t round = (int16_t)((acc + (1 << 14)) >> 15);  // zaokrouhlení: bez biasu
```



Obrázek 11.2. Wrap-around versus saturace. Při wrap-around výsledek po překročení maxima skokem spadne na minimum (obrovská náhlá chyba); při saturaci se výsledek ořízne na maximální hodnotu, takže chyba zůstane omezená a předvídatelná.

V jednom kroku je rozdíl zanedbatelný, ale ve **zpětnovazebním filtru** (IIR, část 11.4) se ořezávací bias hromadí a projeví se jako stejnosměrný offset nebo mezní cyklus (limit cycle). Proto se v IIR zaokrouhluje; knihovní rutiny CMSIS `q15` a `q31` zaokrouhlují také.

> **Shrnutí oddílu 11.2. Float32** je hlavní pracovní formát (M4F a výš); `double` je bez FPU velmi pomalé, `bfloat16` míří do AI. Plovoucí čárka má **proměnné rozlišení** — neporovnávat rovností, pozor na sčítání různých řádů. **Bez FPU** je každá operace desítky až stovky taktů. **Pevná čárka Qm.n** (`x = X / 2^n`) počítá celočíselnou ALU; `Q_n` krát `Q_n` je `Q_2n` (posunout zpět), akumulovat do širšího typu. Po posunu **zaokrouhlovat** (přičíst `2^{n-1}`), neořezávat — bias se v IIR hromadí. **Saturace** místo wrap-around je v DSP pravidlem.

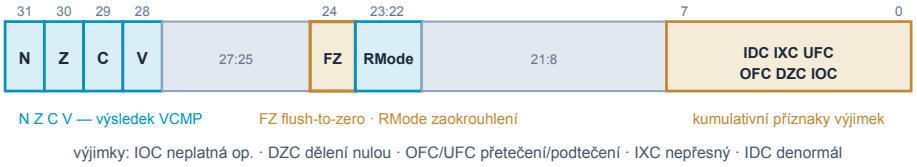
11.3 FPU a DSP instrukce

Kde je hardware, tam je počítání levné. Tato část představí jednotku FPU (a proč ji nestačí zapnout jen jednou), její cenu v obsluze přerušení a instrukce SIMD MAC, na nichž s knihovnou CMSIS-DSP stojí rychlé filtry i FFT.

FPU: jednotka, zapnutí a cena instrukcí

Jádra Cortex-M4F, M7F a M33F mají hardwarovou **FPU**. Je to **koprocesor s vlastním registrovým souborem**, mezi který a jádro se data přesouvají explicitně (`VLDR`, `VSTR`, `VMOV`). Má dvaatřicet registrů `S0` až `S31` (32bitový `float`), z nichž dvojice tvoří jeden 64bitový `D0` až `D15`; M4F je pouze jednoduchá přesnost (double se emuluje softwarově), M7F má i dvojnásobnou. Stav a řízení výpočtu nese registr `FPSCR`.

FPU
Floating-Point Unit: hardwarová jednotka pro operace v plovoucí řádové čárce. Na Cortex-M4F a M7F počítá float v jednotkách taktů, ale musí se povolit při kompilaci i za běhu, jinak se použije pomalá softwarová emulace.



Obrázek 11.3. Registr FPSCR: bity N, Z, C, V nesou výsledek porovnání VCMP; FZ (flush-to-zero) převádí denormály na nulu; RMode volí zaokrouhlení; spodní bity jsou kumulativní příznaky výjimek (neplatná operace, dělení nulou, přetečení a podtečení, nepřesný výsledek, denormální vstup).

Bit `FZ` (flush-to-zero) je praktický kompromis: převede velmi malá (denormální) čísla na nulu, což je rychlejší, ale ztrácí přesnost u nejmenších amplitud. Kumulativní příznaky výjimek (`IOC`, `DZC`, `OFC`, `UFC`, `IXC`, `IDC`) se nastavují a zůstávají nastavené, dokud je software nevynuluje — slouží k diagnostice numerických problémů.

Zapnutí FPU a cena instrukcí

FPU se musí zapnout **dvakrát**. Jednak přepínači překladače (`-mfloat-abi=hard` a `-mfpu=fpv4-sp-d16` pro M4F nebo `fpv5-sp-d16` pro M7F), jednak povolením koprocesoru za běhu v registru `CPACR` (přístup ke `CP10` a `CP11`), což typicky dělá `SystemInit`:

```
SCB->CPACR |= (0xF << 20);   // CP10 a CP11: plný přístup k FPU
```

Bez povolení v `CPACR` skončí první instrukce FPU výjimkou HardFault. Ověření, že se FPU skutečně používá, je v assembleru: má tam být `vadd.f32`, ne skok `bl __aeabi_fadd`. Časování single-precision instrukcí na M4F je řádově lepší než emulace (`VADD`, `VSUB`, `VMUL` kolem jednoho taktu, `VMLA` tři takty), ale **dělení a odmocnina zůstávají drahé** (`VDIV` a `VSQRT` kolem čtrnácti taktů), takže se jim vyhýbáme i s FPU. Zásadní je, že **FPU umí jen sčítání, odčítání, násobení, dělení a odmocninu — trigonometrii ne**: `sinf` a `cosf` jsou knihovní funkce za stovky taktů. To je hlavní motivace pro hardwarový CORDIC (oddíl 11.6).

FPU v přerušení: lazy stacking

Když je FPU aktivní, ukládá hardware při vstupu do výjimky **rozšířený rámec** — kromě obvyklých osmi slov (`R0` až `R3`, `R12`, `LR`, `PC`, `xPSR`) i registry `S0` až `S15` a `FPSCR`. Kontext je velký, a kdyby se ukládal při každém vstupu do obsluhy přerušení, byla by obsluha drahá. Cortex-M to řeší **lazy stackingem** (bit `LSPEN` v `FPCCR`). Ve výchozím stavu se místo na zásobníku rezervuje, ale zápis se odloží, dokud obsluha FPU skutečně nepoužije (první instrukce FPU), s režií zhruba 20 až 40 taktů; při vypnutém lazy stackingu se `S0` až `S15` ukládají při každém vstupu (přes sto taktů navíc). Uložený FP kontext je `S0` až `S15` a `FPSCR`, tedy 17 slov (68 bajtů), zarovnaných na 72 bajtů. Návrhové pravidlo zní: v obsluze přerušení se plovoucí čárce vyhýbáme, pokud to jde — pak se lazy stacking nikdy nespustí; souvisí to s rámcem přerušení a přepnutím kontextu (přednášky 3 a 9).

Lazy stackingem
Odložené ukládání registrů FPU při vstupu do obsluhy přerušení: místo na zásobníku se rezervuje, ale zápis se odloží, dokud obsluha FPU skutečně nepoužije. Šetří takty u obsluh, které s plovoucí čárkou nepracují.

SIMD MAC a CMSIS-DSP

Filtry i FFT stojí na jediné operaci — násobení s akumulací (MAC). **SIMD MAC** z DSP rozšíření Cortex-M4 a M7 (přednáška 7) zpracuje dvě hodnoty Q15 najednou ve třicetidvoubitovém slově: `__SMLAD` udělá dvě šestnáctibitová násobení plus akumulaci v jednom taktu, `__SMUAD` bez akumulace, `__QADD16` dvě saturující sčítání. To je důvod, proč umí i malé jádro počítat filtry v reálném čase — jedna instrukce, dvě operace MAC; Helium na M55 posouvá totéž na čtyři až osm prvků.

Ruční SIMD je ale křehké, proto se v praxi sahá po knihovně **CMSIS-DSP**. Nabízí filtry, FFT, matice a statistiku a volí nejlepší instrukce za nás; funkce existují v každém formátu (`_f32`, `_q31`, `_q15`) a kód se liší jen typem. Pravidlo zní: algoritmus napíšeme jednou ručně, abychom mu rozuměli, a do produkce použijeme CMSIS-DSP.

SIMD MAC
Single Instruction Multiple Data násobení s akumulací: jedna instrukce, například SMLAD, provede dvě šestnáctibitová násobení a jejich součet najednou. Je základem rychlých filtrů a FFT na jádrech Cortex-M4 a M7.

CMSIS-DSP
Optimalizovaná knihovna ARM pro zpracování signálu (filtry, FFT, matice, statistika). Volí nejlepší instrukce, používá tabulky a ruční assembler; funkce jsou dostupné ve formátech float32, Q31 i Q15.

Cena výpočtu na jednom příkladu

Kolik stojí volba aritmetiky, ukáže generátor harmonického signálu z tabulky (LUT): fázový akumulátor indexuje tabulku `sin`, vzorek jde na výstup. Ve `float` bez FPU je šest operací na vzorek, každá volání emulace; assembler odhalí realitu — dělení fáze 42 taktů, násobení 28, převod na celé číslo 24 a tak dále, celkem zhruba **195 taktů na vzorek**, přestože jde jen o čtení z tabulky a inkrement fáze.

Technika pevné čárky: fázový akumulátor jako `q31_t`, jehož horních osm bitů je přímo index tabulky; přetečení akumulátoru zajistí zabalení fáze zdarma (žádná podmínka), tabulka je v `q15_t`.

```
void generate_sample_q15(void) {
    uint32_t index = (phase_q31 >> 24) & 0xFF;   // horních 8 bitů = index
    q15_t sample = sin_table_q15[index];
    GPIOA->ODR = ((int32_t)sample + 32768) >> 4; // Q15 -> 12bit
    phase_q31 += phase_increment_q31;            // 32bit součet, wrap zdarma
}
```

Assembler je několik celočíselných instrukcí bez jediného volání, zhruba **15 taktů na vzorek** — třináctkrát rychleji než float bez FPU, na tomtéž jádře. Kdyby MCU mělo FPU, float verze se zrychlí řádově, ale pevná čárka zůstává úsporná a deterministická. To je celé DSP-inženýrství v kostce: stejný matematický výsledek, řádový rozdíl v ceně podle volby čísel.

> **Shrnutí oddílu 11.3. FPU** je koprocesor s registry `S0` až `S31` a `FPSCR`; musí se zapnout **dvakrát** — přepínači překladače **a** v `CPACR` za běhu (jinak HardFault). FPU umí jen sčítání, odčítání, násobení, dělení a odmocninu; dělení a odmocnina jsou drahé a **trigonometrii neumí** (to je motivace pro CORDIC, oddíl 11.6). V obsluze přerušení pozor na **lazy stacking** (rozšířený rámec, 68 bajtů). **SIMD MAC** (`__SMLAD`) dělá dvě Q15 násobení s akumulací v jednom taktu; **CMSIS-DSP** volí nejlepší instrukce. Příklad: generátor je v Q15 třináctkrát rychlejší než float bez FPU.

11.4 Číslicová filtrace

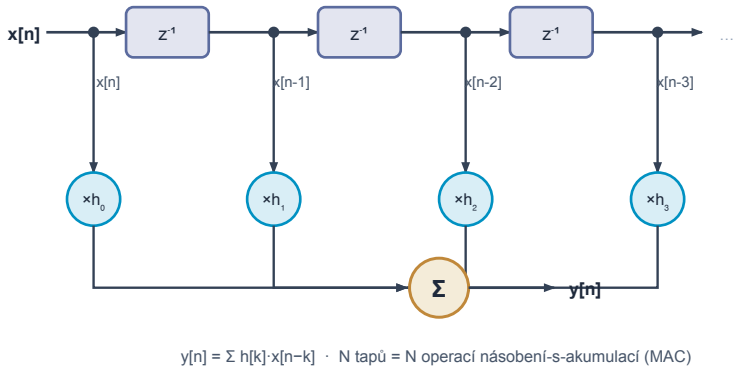
Nejčastější DSP úlohou je potlačit část spektra. Tato část ukáže FIR filtr jako konvoluci a jeho implementaci — kde rozhoduje aritmetika, ne „lepší kód" — a rozšíří ji na rekurzivní filtry IIR a biquad ekvalizér.

FIR filtr: konvoluce a implementace

Mnoho úloh potřebuje potlačit část spektra — typicky **antialiasingovou dolní propust** před vzorkováním nebo decimací (přednáška 5: složky nad Nyquistovou frekvencí se zrcadlí do užitečného pásma). **FIR filtr** počítá každý výstupní vzorek jako vážený součet posledních `N` vstupních vzorků s koeficienty `h[k]`, tedy konvoluci `y[n] = suma přes k od nuly do N minus jedna z h[k] krát x[n-k]`.

FIR filtr
Filtr s konečnou impulsní odezvou: výstupní vzorek je vážený součet posledních N vstupních vzorků s koeficienty. Realizuje konvoluci, je vždy stabilní a má lineární fázi, ale potřebuje mnoho koeficientů.

Struktura je zpožďovací linka (historie vzorků) a `N` operací MAC na každý výstup — proto je FIR modelovým příkladem pro MAC a SIMD z oddílu 11.3. `N` koeficientů znamená `N` operací MAC na vzorek; filtr s osmdesáti jedna tapy při 44 kHz je 3,6 milionu MAC za sekundu, což rozhoduje o tom, zda se vejdeme do rozpočtu cyklů. Návrh filtru (výpočet koeficientů ze specifikací propustného a nepropustného pásma) se dělá offline na PC; na MCU běží jen hotové koeficienty a nás zajímá implementace konvoluce.



Obrázek 11.4. Struktura FIR filtru: zpožďovací linka (řetěz zpoždění z na mínus jednu) drží historii vzorků, každý odbočený vzorek se násobí koeficientem a všechny součiny se sečtou do výstupu. N koeficientů znamená N operací násobení s akumulací na každý výstupní vzorek.

Ruční implementace a CMSIS-DSP

Přímočará `float` verze je čitelná (`for` přes tapy, `acc += x[k] * h[k]`), ale na jádře bez FPU drahá. Pevná čárka se SIMD zpracuje dva tapy najednou přes `__SMLAD`, s akumulací do `int32_t` a závěrečnou saturací zpět na Q15. Výsledky pro týž osmdesátijedentapý filtr ukazují, že rozhoduje aritmetika, ne „lepší kód" (tab. 11.1):

Tabulka 11.1. Výkon FIR filtru podle aritmetiky a platformy

implementace	platforma	takty na vzorek	CPU při 44,1 kHz
ruční float	STM32F103 (bez FPU)	6 020	37 %
ruční float	STM32F407 (FPU)	335	0,88 %
CMSIS `arm_fir_f32`	STM32F407	32	0,084 %
CMSIS `arm_fir_q15`	STM32F407	20	0,05 %

Skok z ručního floatu bez FPU (37 % CPU, filtr jako hlavní zátěž) na CMSIS Q15 (0,05 %, nezměřitelné) je zhruba tři sta násobný. CMSIS-DSP přitom zapouzdří zpožďovací linku i SIMD do instance filtru (`arm_fir_init_f32`, `arm_fir_f32`); kód je identický pro `f32` i `q15` a liší se jen typem. Zpracování po **blocích** amortizuje režii volání a lépe využije pipeline; vzor je stejný jako u DMA z přednášky 5 — ADC plní blok, CMSIS-DSP jej zpracuje, výsledek jde dál.

Rozšíření: filtry IIR a biquad EQ

FIR filtr je vždy stabilní a má lineární fázi, ale strmou charakteristiku vykoupí mnoha koeficienty. **Filtr IIR** zavádí **zpětnou vazbu** a dosáhne stejné strmosti s řádově méně koeficienty. Základní stavební jednotkou je **biquad** — sekce druhého řádu s přenosovou funkcí, kde čitatel i jmenovatel jsou polynomy druhého stupně v `z na mínus jednu`. Koeficienty se počítají z centrální frekvence, jakosti `Q` a zesílení (například podle Bristow-Johnsonových „cookbook" vzorců pro peaking EQ).

Filtr IIR
Filtr s nekonečnou impulsní odezvou; biquad je jeho dvoupólová sekce druhého řádu se zpětnou vazbou. Dosáhne strmé charakteristiky s málo koeficienty, ale je nutné hlídat stabilitu a nelineární fázi.

Zpracování jednoho biquadu stojí zhruba jedenáct taktů na vzorek (pět násobení, čtyři sčítání, dvě uložení). **Desetipásmový ekvalizér** je pak banka deseti biquad sekcí s centrálními frekvencemi po třetinách oktávy (31,25 Hz až 16 kHz); ruční implementace stojí kolem 8300 taktů na blok. CMSIS-DSP nabízí `arm_biquad_cascade_df1_f32`, která celou kaskádu zpracuje optimalizovaně. Cenou za úspornost IIR je nutnost hlídat **stabilitu** (póly uvnitř jednotkové kružnice) a **nelineární fázi**, kvůli níž se u aplikací citlivých na tvar signálu často přesto volí FIR.

> **Shrnutí oddílu 11.4. FIR filtr je konvoluce** `y[n] = suma h[k] krát x[n-k]` — zpožďovací linka a `N` operací MAC na vzorek; `N` tapů určuje zátěž. **Aritmetika rozhoduje**: ruční float bez FPU 37 % CPU, CMSIS Q15 0,05 %, tedy zhruba tři sta násobek. **CMSIS-DSP** (`arm_fir_*`) schová SIMD i zpožďovací linku a pracuje po blocích (navazuje na DMA z přednášky 5). **IIR biquad** dosáhne

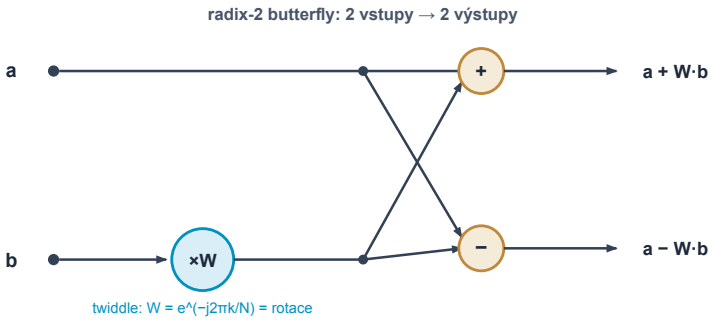
Od času k frekvenci: radix-2 FFT

Filtr pracuje v čase, ale mnoho úloh potřebuje **frekvenční obraz** signálu — kolik které frekvence obsahuje. To dává Fourierova transformace. Přímý výpočet diskrétní transformace (DFT) však pro každý z `N` výstupních binů projde celý vstup, takže má složitost `N na druhou`; pro `N` rovno 1024 je to přes milion komplexních násobení na jedno spektrum, což je pro reálný čas nepoužitelné.

Radix-2 FFT

FFT využije symetrie a sníží složitost na `N krát logaritmus N`; pro `N` rovno 1024 je to rozdíl mezi zhruba milionem a deseti tisíci operací, tedy faktor sto. Radix-2 varianta rozloží `N`-bodovou transformaci rekurzivně na dvě `N` půl-bodové (sudé a liché vzorky) a ty spojí butterfly operací s twiddle faktory.

FFT
Fast Fourier Transform: rychlý algoritmus výpočtu diskrétní Fourierovy transformace se složitostí N krát logaritmus N místo N na druhou. Převádí signál z časové do frekvenční oblasti.



Obrázek 11.5. Radix-2 butterfly: dva vstupy a a b dají dva výstupy. Dolní vstup se násobí twiddle faktorem (rotací) a výsledky se sečtou a odečtou: horní výstup je a plus W krát b, dolní a minus W krát b.

Butterfly je základní jednotka: dva vstupy, dva výstupy, `out1 = a + W krát b`, `out2 = a minus W krát b`. **Twiddle faktor** `W` je jednotkový komplexní činitel — jen rotace v komplexní rovině. Pro `N` rovno 1024 je deset stupňů po 512 butterfly.

Twiddle faktor
Komplexní jednotkový činitel ve tvaru rotace o úhel, kterým FFT spojuje menší transformace. V optimalizovaných knihovnách je předpočítaný v tabulce místo počítání sinu a kosinu za běhu.

Ruční FFT versus CMSIS-DSP

Butterfly sám je levný (dvě sčítání, dvě odčítání), ale naivní implementace má dvě pasti: **twiddle faktor počítaný za běhu** (`sinf` a `cosf` stojí sto až dvě stě taktů, a tisíce twiddle dají statisíce taktů) a **bez FPU** je každé komplexní sčítání butterfly 30 až 50 taktů. Ruční FFT tak potřebuje kolem 410 tisíc taktů na jednu transformaci (2,44 ms, 24 % CPU při aktualizaci 10 Hz) a na jádře bez FPU je nerealizovatelná.

CMSIS-DSP má twiddle faktory v tabulce (žádné `sin` a `cos` za běhu), butterfly v ručním assembleru a SIMD:

```
arm_rfft_fast_instance_f32 fft;
arm_rfft_fast_init_f32(&fft, 1024);
arm_rfft_fast_f32(&fft, time_data, freq_data, 0);   // reálná FFT 1024 bodů
arm_cmplx_mag_f32(freq_data, magnitude, 512);       // amplituda spektra
```

implementace	takty na FFT	CPU při 10 Hz
ruční	410 000	24,4 %
CMSIS `float32`	3 190 (19 mikrosekund)	0,19 %
CMSIS `Q31`	2 350	0,14 %

To je sto dvacet osm násobek proti ruční implementaci; 99,8 % CPU zůstane volných. `Q31` je ještě rychlejší než `float32` (celočíselná ALU bez úskalí FPU, SIMD).

Reálná FFT, okénkování a rozlišení

Signál z ADC je **reálný**, ne komplexní, a spektrum reálného signálu je hermitovsky **symetrické**: horní polovina binů je jen komplexně sdruženým zrcadlem té dolní a nenese žádnou novou informaci. Proto se počítá **reálná FFT** (`arm_rfft_fast_f32`), která je zhruba dvakrát rychlejší a spočítá jen `N/2 + 1` užitečných binů — od stejnosměrné složky (bin 0) po Nyquistovu frekvenci (`Fs/2`). Právě proto všechny FFT řetězce této přednášky používají `rfft`, ne plnou komplexní transformaci.

FFT předpokládá, že blok `N` vzorků je **jedna perioda** nekonečně opakovaného signálu. Nevejde-li se frekvence přesně do celého počtu period, konce bloku na sebe nenavazují a energie jednoho tónu se rozlije do okolních binů — **spektrální únik** (leakage). Potlačí ho **okénkování**: blok se před FFT vynásobí funkcí, která plynule utlumí okraje k nule (`arm_mult_f32(block, hann, out, N)`). Hann a Hamming jsou dobrým kompromisem, Blackman potlačí postranní laloky silněji za cenu širšího hlavního laloku; obdélníkové „okno" (žádné) má nejlepší rozlišení, ale nejhorší únik.

Každý bin pokrývá pásmo šířky `delta f rovno Fs děleno N` a bin `k` odpovídá frekvenci `k krát Fs děleno N`. Jemnější rozlišení tedy žádá víc vzorků `N` nebo nižší `Fs` — vždy **delší okno v čase** (`N děleno Fs` sekund). Je to přímý kompromis čas versus frekvence: nelze mít současně krátké okno i jemné rozlišení. **Nulové doplnění** (zero-padding) na delší FFT spektrum jen hladce interpoluje; skutečné rozlišení dvou blízkých frekvencí určuje pouze délka *reálného* měření, ne počet doplněných nul.

Spektrogram a prediktivní údržba

Jedna FFT dá spektrum celého bloku, ale ztratí informaci o tom, **kdy** která frekvence zněla. **Krátkodobá FFT** (STFT) proto počítá FFT z klouzavých, obvykle překrývajících se oken (typicky s padesátiprocentním překryvem) a skládá je vedle sebe do matice **čas krát frekvence** — spektrogramu. Kompromis čas versus frekvence platí i zde: krátké okno dá dobrou časovou, ale hrubou frekvenční lokalizaci, a naopak. Právě takto vzniká log-mel spektrum pro detekci klíčových slov (přednáška 12): STFT po rámcích, mel škála, logaritmus. Spektrogram je tak most, kterým číslicové zpracování signálu předává příznaky strojovému učení.

Aplikace: prediktivní údržba

Spektrum vibrací točivého stroje nese **podpis závady**: každá porucha kmitá na své frekvenci, takže sledováním amplitud na těchto frekvencích poznáme problém dřív, než dojde k havárii.

Frekvenční rozlišení je `delta f rovno Fs děleno N`; pro 1024-bodovou FFT při 10 kHz je to 9,77 Hz na bin, což stačí na rozlišení blízkých mechanických frekvencí. Celý řetěz je: akcelerometr, ADC a DMA (přednáška 5) plní blok 1024 vzorků, CMSIS-DSP spočítá FFT, jednoduchá detekce špiček najde maxima a „amplituda na 157 Hz roste" odejde jako několik bajtů po sběrnici. Data zůstala na zařízení, odešla jen informace.

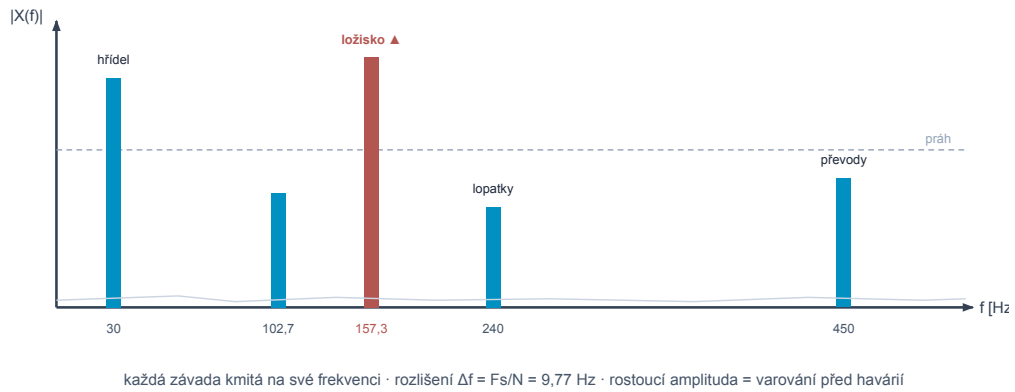
> **Shrnutí oddílu 11.5.** Frekvenční obraz dává **Fourierova transformace**; přímá DFT je `N na druhou`, pro reálný čas nepoužitelná. **FFT** (radix-2, rozděl a panuj) sníží složitost na `N krát logaritmus N` a staví na **butterfly** a **twiddle** rotacích. **Ruční FFT** propadne kvůli `sin` a `cos` za běhu (410 tisíc taktů); **CMSIS-DSP** má twiddle v tabulce, tedy sto dvacet osm krát rychleji. Reálný signál → **reálná FFT** (`N/2+1` binů, dvakrát rychleji). **Okénkování** (Hann) tlumí **spektrální únik**; rozlišení `delta f rovno Fs děleno N` je kompromis čas versus frekvence (nulové doplnění jen interpoluje).

Reálná FFT
Varianta FFT pro reálný vstupní signál. Využívá symetrie spektra reálného signálu, počítá jen dolní polovinu binů (N/2+1 hodnot od stejnosměrné složky po Nyquistovu frekvenci), a je proto zhruba dvakrát rychlejší a paměťově úspornější než plná komplexní FFT.

Spektrální únik
Rozlití energie jedné frekvence do sousedních binů FFT, když se do délky bloku nevejde celý počet period signálu. Potlačuje se okénkováním, které plynule utlumí okraje bloku k nule.

Okénkování
Vynásobení bloku vzorků před FFT funkcí (oknem), která plynule utlumí jeho okraje k nule, čímž se omezí spektrální únik. Hann a Hamming jsou dobrý kompromis, Blackman silněji potlačí postranní laloky za cenu širšího hlavního laloku.

Krátkodobá FFT
Short-Time Fourier Transform (STFT): posloupnost FFT počítaných z klouzavých, obvykle překrývajících se oken signálu. Skládá se do matice čas krát frekvence (spektrogramu), která zachycuje vývoj spektra v čase.



Obrázek 11.6. Spektrum vibrací stroje se spektrálními signaturami závad: hlavní hřídel na 30 Hz, defekty ložiska na 102,7 a 157,3 Hz, záběr lopatek na 240 Hz a záběr převodů na 450 Hz. Rostoucí amplituda na frekvenci ložiska je varováním před havárií.

> **Spektrogram** (STFT) je most k příznakům pro strojové učení (přednáška 12). **Prediktivní údržba**: spektrum vibrací nese podpisy závad.

11.6 Matematické akcelerátory (STM32G4)

Dvě úzká hrdla DSP — trigonometrii a filtraci — lze přesunout do vyhrazeného hardwaru. Tato část představí matematické akcelerátory řady STM32G4: CORDIC pro goniometrii jen posuny a součty a FMAC pro filtry, oba s DMA a bez zatížení jádra.

Dvě drahé operace a hardware na obě

Napříč přednáškou vystupují dvě úzká hrdla: **trigonometrii** (`sinf` a `cosf` v generátoru, twiddle faktory ve FFT, `atan2` ve fázi) a **násobení s akumulací** (konvoluce filtru). FPU umí jen sčítání, odčítání, násobení, dělení a odmocninu — trigonometrii ne; SIMD MAC pomůže, ale jádro pořád počítá. Řešením je vyhrazený hardware vedle jádra. Je to tentýž princip jako specializace z přednášky 8, jen o úroveň níž — akcelerátor jako periferie na sběrnici, ne samostatný čip. Řada **STM32G4** (Cortex-M4F na 170 MHz) přidává dva matematické akcelerátory, **CORDIC** (trigonometrie) a **FMAC** (filtry), a proto je cílená na řízení motorů a digitální napájení, kde obojí běží v každé periodě regulace.

CORDIC: rotace jen posuny a součty

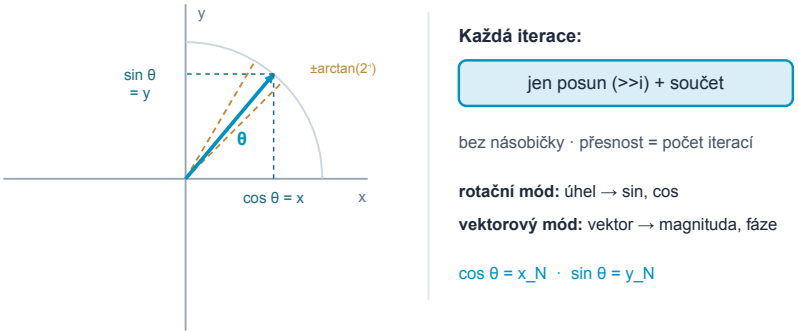
CORDIC počítá `sin`, `cos`, `atan2`, magnitudu, `√` a `ln` **bez násobičky** — jen posuny a součty. Cílový úhel se srazí k nule posloupností rotací o klesající úhly `arctan(dvě na mínus i)`.

Iterace jsou `x na příště rovno x plus minus x posunuté o i`, `y na příště rovno y plus minus x posunuté o i` a `z na příště rovno z plus minus arctan dvě na mínus i`. Po `N` iteracích je kosinus úhlu vodorovná složka a sinus svislá. Existují dva režimy: **rotační** (úhel na vstupu, sinus a kosinus na výstupu) a **vektorový** (vektor na vstupu, magnituda a fáze na výstupu). Protože každá iterace je jen posun a součet na složku, je CORDIC v hardwaru levný a deterministický — přesnost je přímo počet iterací.

CORDIC
COordinate Rotation DIgital Computer: algoritmus a hardwarová periferie, která počítá sinus, kosinus, arkustangens, magnitudu, odmocninu a logaritmus pomocí posloupnosti rotací o klesající úhly, tedy jen posuny a součty bez násobičky. Přesnost je dána počtem iterací.

CORDIC na STM32G4

Na G4 je CORDIC periferie s jednoduchým registrovým rozhraním; vstup i výstup jsou v Q1.31 nebo Q1.15:



Obrázek 11.7. Princip CORDIC: jednotkový vektor se otáčí k cílovému úhlu θ posloupností rotací o klesající úhly, přičemž každá iterace je jen posun a součet. Po N iteracích je vodorovná složka rovna kosinu a svislá sinu úhlu. V rotačním módu jde úhel na vstup a sinus s kosinem na výstup, ve vektorovém módu vektor na vstup a magnituda s fází na výstup.

```
CORDIC->CSR = FUNC_COSINE | PRECISION(6) | SCALE(0);  // konfigurace funkce
CORDIC->WDATA = angle_q31;          // zapiš argument (úhel)
int32_t cos = CORDIC->RDATA;        // přečti cos
int32_t sin = CORDIC->RDATA;        // a sin – současně
```

Funkcemi jsou kosinus, sinus, fáze (`atan2`), modulus (odmocnina ze součtu čtverců), arkustangens, hyperbolické funkce, přirozený logaritmus a odmocnina. Přesnost je konfigurovatelná polem `PRECISION` (počet iterací), latence jsou jednotky taktů a periferie umí **DMA**, takže proud sinů a kosinů teče bez jediné instrukce jádra. Zápis argumentu a čtení výsledku stojí jednotky taktů místo zhruba sta až dvou set za softwarový `sinf`, a sinus i kosinus přijdou naráz.

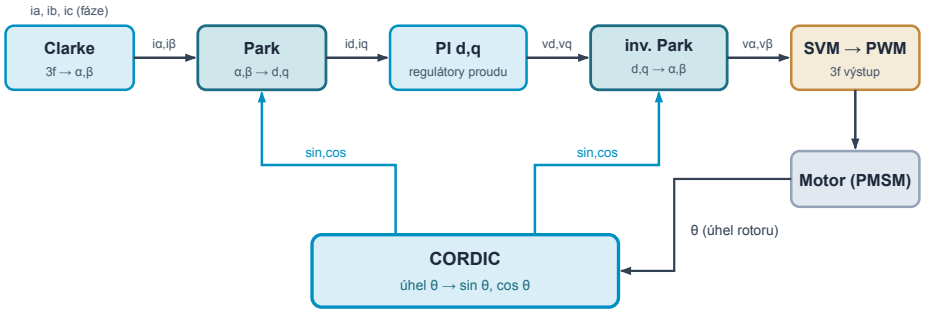
Aplikace: generátor, řízení motoru, magnituda a fáze

Generátor bez tabulky. V oddílu 11.3 se sinus generoval z tabulky (2 až 4 kB paměti, pevná přesnost, interpolační chyba). CORDIC ji nahradí úplně: fázový akumulátor zůstává, jen místo indexace tabulky pošle úhel do CORDICu a dostane sinus i kosinus přesně a naráz. Odpadá paměť i interpolační chyba a jádro zůstává volné.

Řízení motoru (FOC). Vektorové řízení synchronního nebo BLDC motoru (Field-Oriented Control) je vlajková aplikace G4. Každou PWM periodu (desítky kHz) potřebuje sinus a kosinus úhlu rotoru pro Parkovu a inverzní Parkovu transformaci mezi statorovým a rotujícím souřadným systémem.

Regulační smyčka běží v obsluze přerušení časovače, deterministicky, desítky tisíckrát za sekundu. CORDIC dodá sinus a kosinus v pevném počtu taktů, takže se smyčka spolehlivě vejde do PWM periody — přesně proto řada G4 vznikla.

Magnituda a fáze. Vektorový mód převede kartézské souřadnice na polární (magnituda a `atan2`) v jednom průchodu. To se hodí pro amplitudu a fázi binu FFT (bez `arm_cmplx_mag`, oddíl 11.5), pro demodulaci AM a FM, měření impedance, smyčku fázového závěsu i pro normalizaci vektoru a určení směru (fúze dat z inerciální jednotky).



Obrázek 11.8. Smyčka vektorového řízení motoru: fázové proudy projdou Clarkovou transformací do stacionárního rámce, Parkovou transformací do rotujícího rámce, PI regulátory proudů a inverzní Parkovou transformací zpět; z modulace prostorového vektoru jde třífázový výstup na PWM a motor. CORDIC dodává sinus a kosinus úhlu rotoru pro Parkovu a inverzní Parkovu transformaci každou PWM periodu.

FMAC: filtr v hardwaru

Druhým akcelerátorem G4 je **FMAC** (Filter Math Accelerator) — hardwarová jednotka násobení s akumulací s vlastní pamětí (256 krát 16 bitů) pro filtry FIR a IIR v Q1.15. Konvoluci z oddílu 11.4 spočítá hardware; jádro jen dodává a odebírá vzorky, ideálně přes DMA, s deterministickou latencí a volným jádrem pro zbytek aplikace. CORDIC a FMAC jsou tak odpovědí G4 na dvě úzká hrdla této přednášky: trigonometrie i filtrace se přesunou z jádra do periferie — je to specializovaný hardware, teď přímo na čipu mikrokontroléru.

> **Shrnutí oddílu 11.6.** Dvě úzká hrdla DSP — **trigonometrie** a **MAC** — má STM32G4 ve vyhrazeném hardwaru vedle jádra (princip specializace z přednášky 8, na úrovni periferie). **CORDIC** počítá sinus, kosinus, `atan2`, modulus a odmocninu jen posuny a součty (rotace o `arctan(dvě na mínus i)`); přesnost je počet iterací, sinus i kosinus naráz, umí DMA. Aplikace: **generátor bez tabulky**, **FOC řízení motoru** (sinus a kosinus rotoru každou PWM periodu — vlajková úloha G4) a **magnituda s fází** (FFT, demodulace). **FMAC** je hardwarový filtr FIR a IIR; CORDIC a FMAC uvolní jádro od trigonometrie i filtrace.

11.7 Číslicový PID regulátor

Nejrozšířenější řídicí úlohou vestavného systému je **regulační smyčka**: měřenou veličinu (otáčky, teplotu, proud) je třeba udržet na **žádané hodnotě** (setpointu). Smyčka v pevném rytmu změří skutečný stav, spočítá **odchylku** od žádané hodnoty a z ní **akční zásah**, kterým přes akční člen (typicky PWM) vrací proces k žádané hodnotě. Naprostou většinu takových smyček řeší jediný algoritmus — **PID regulátor**.

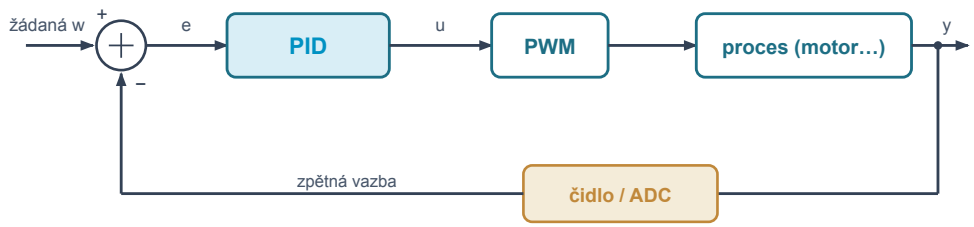
PID regulátor
Proporcionálně-integračně-derivační regulátor: akční zásah skládá ze tří členů — úměrného okamžité odchylce (P), integrálu odchylky v čase (I) a její derivace (D). P reaguje na velikost odchylky, I odstraní trvalou odchylku, D tlumí a předjímá.

Akční zásah `u` skládá PID ze tří členů odchylky `e = w − y` (žádaná mínus měřená veličina):

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\,\mathrm{d}\tau + K_d\, \frac{\mathrm{d}e(t)}{\mathrm{d}t}.$$

Každý člen má jasný úkol. **Proporcionální** (`P`) působí úměrně velikosti odchylky, sám o sobě ale nechá **trvalou regulační odchylku** — aby držel nenulový zásah, potřebuje nenulovou odchylku. **Integrační** (`I`) sčítá odchylku v čase, takže i malou trvalou odchylku postupně **vynuluje**, za cenu sklonu k překmitu. **Derivační** (`D`) reaguje na *rychlost změny* odchylky, čímž

smyčku **tlumí** a reaguje na trend odchylky, ale zesiluje šum. Vyvážení tří zesílení `Kp`, `Ki`, `Kd` rozhoduje o chování smyčky; metody jejich ladění rozebírá **Příloha D**.



Obrázek 11.9. Regulační smyčka s PID: žádaná hodnota se porovná s měřenou, odchylka jde do PID regulátoru, jeho akční zásah přes PWM ovládá proces a čidlo (ADC) uzavírá zpětnou vazbu.

Diskrétní PID v přerušení časovače

Na MCU neběží PID spojitě, ale **v pevných okamžicích** — vzorkovací periodě `Ts`, kterou udává přerušení časovače (deterministicky, bez jitteru). Integrál se stane **akumulátorem** (součtem odchylek násobených `Ts`), derivace **rozdílem** dvou po sobě jdoucích odchylek děleným `Ts`. Vznikne tzv. **poziční** tvar:

```
// PID v ISR časovače, perioda Ts; volá se pevnou frekvencí
float pid_step(float w, float y) {      // w = žádaná, y = měřená
    float e   = w - y;                  // odchylka
    integ    += e * Ts;                 // integrační akumulátor
    float der = (e - e_prev) / Ts;      // derivace odchylky
    e_prev    = e;
    return Kp*e + Ki*integ + Kd*der;    // akční zásah
}
```

Celá smyčka v obsluze přerušení pak je: **přečti vzorek z ADC** → **spočítej** `pid_step` → **zapiš výsledek do PWM**. Protože běží pevnou frekvencí a v krátké ISR, je časování přesné a nezávislé na okamžité zátěži jádra — přesně ten determinismus, který regulace vyžaduje.

Anti-windup a filtrace derivace

Naivní PID má dvě zákeřné vady, které se projeví až v provozu.

První je **integrální windup**: narazí-li akční zásah na **mez akčního členu** (PWM nemůže přes 100 %), integrátor přesto sčítá dál a „přeteče"; po návratu do rozsahu pak trvá dlouho, než se vybije, a smyčka výrazně překmitne. Řešením je **anti-windup** — při nasycení výstupu integraci zastavit, nebo integrátor omezit (clamp).

Druhá je **derivační ráz**: při skokové změně žádané hodnoty prudce naroste derivace odchylky a akční zásah skokově vybočí. Odstraní se tím, že se **derivuje měřená veličina** místo odchylky (skok žádané hodnoty pak derivaci neovlivní), a derivace se navíc **filtruje dolní propustí**, aby nezesilovala šum čidla.

Integrální windup
Nahromadění integračního členu ve chvíli, kdy akční člen narazí na svůj limit (nasycení PWM). Zásah už dál růst nemůže, ale integrátor sčítá dál, takže po návratu do rozsahu způsobí velký překmit. Brání se mu zastavením nebo omezením integrace při nasycení (anti-windup).

Derivační ráz
Prudký impulz v derivačním členu při skokové změně žádané hodnoty, jejíž derivace je teoreticky nekonečná. Odstraní se derivováním měřené veličiny místo odchylky a filtrací derivace dolní propustí, aby nezesilovala šum čidla.

Pevná, nebo plovoucí čárka

Na jádře s FPU je nejpohodlnější počítat PID ve `float` (jako výše). Bez FPU nebo pro maximální determinismus se volí **pevná řádová čárka**: zesílení se uloží ve formátu Q, násobení akumuluje do širokého typu a **integrační akumulátor potřebuje dostatečnou šířku** (`int32`/`int64`), aby dlouhým sčítáním nepřetekl. Je to táž disciplína pevné čárky a saturace

jako u filtrů — PID je koneckonců jednoduchý rekurzivní systém, blízký příbuzný IIR filtru. Podrobnosti implementace i ladění nese Příloha D.

> **Shrnutí oddílu 11.7. PID regulátor** udržuje měřenou veličinu na žádané hodnotě: akční zásah je součet **P** (úměrný odchylce, nechá trvalou odchylku), **I** (integrál, odstraní trvalou odchylku) a **D** (derivace, tlumí, zesiluje šum). Na MCU běží **diskrétně** v přerušení časovače pevnou periodou `Ts` (integrál = akumulátor, derivace = rozdíl) v řetězu ADC → výpočet → PWM. Provoz vyžaduje **anti-windup** (nasycení akčního členu) a **derivaci z měřené veličiny s filtrem** (proti rázu a šumu). Počítá se ve `float` (FPU), nebo v pevné čárce se širokým akumulátorem. Ladění zesílení řeší **Příloha D**.

11.8 Syntéza

Zbývá z probraných voleb udělat rozhodnutí. Tato závěrečná část dá rozhodovací tabulku aritmetik, shrne, že cena výpočtu je otázka aritmetiky (s rozdíly až tří řádů), a naznačí přechod ke strojovému učení.

Rozhodovací tabulka a co si odnést

Volba není „plovoucí versus pevná" dogmaticky, ale podle jádra, rozpočtu a přesnosti. Na jádře bez FPU (M0, M3) je jedinou možností pevná čárka Q15 nebo Q31. Na jádře s FPU a pro pohodlí se volí `float32` s CMSIS-DSP (rychlé a čitelné). Pro minimum energie nebo maximální propustnost je nejlepší Q15 nebo Q31 se SIMD (celočíselná ALU, deterministické). Ve výpočtu uvnitř obsluhy přerušení se volí pevná čárka, aby se předešlo lazy stackingu FPU. Pro velký dynamický rozsah nebo AI část je na místě `float32` nebo `bfloat16` (přednáška 12). Má-li čip vyhrazený akcelerátor (STM32G4), přesuneme trigonometrii na **CORDIC** a filtraci na **FMAC** a jádro necháme volné. A vždy platí: algoritmus napíšeme jednou ručně pro pochopení a nasadíme CMSIS-DSP.

Co si odnést

Napříč všemi příklady se opakoval jeden vzorec — **cena výpočtu je otázka aritmetiky, ne jen algoritmu**. Generátor stál ve `float` bez FPU 195 taktů, v Q15 patnáct (třináctkrát méně). FIR filtr stál ručně bez FPU 6020 taktů (37 % CPU), v CMSIS Q15 dvacet (0,05 %, zhruba tři sta násobek). FFT stála ručně 410 tisíc taktů (24 % CPU), v CMSIS Q31 2350 (0,14 %, zhruba sto sedmdesát pět násobek). Stejný matematický výsledek, tři řády rozdílu ve spotřebě cyklů. V rozpočtu real-time DSP — a pod baterií (přednáška 10) — je to rozdíl mezi „nejde to" a „99 % CPU volných".

Most na strojové učení

Číslicové zpracování signálu je předstupeň strojového učení na embedded. Klasifikátor nepracuje se syrovými vzorky, ale s **příznaky**, které mu spočítá DSP: efektivní hodnota (RMS) v okně, spektrální energie v pásmech, dominantní frekvence z FFT, statistiky. Tytéž úvahy o pevné a plovoucí čárce a tatáž knihovní filozofie (nově CMSIS-NN) platí i pro inferenci; neuronová síť běží na NPU (přednáška 8), klasické DSP na jádře nebo DSP rozšíření. Prediktivní údržba přes prahy na FFT binech je „ruční klasifikátor"; přednáška 12 ho nahradí naučeným modelem, ale příznaky mu pořád počítá DSP z této přednášky.

> **Shrnutí oddílu 11.8.** Volba aritmetiky se řídí **jádrem, rozpočtem a přesností** (bez FPU pevná čárka, s FPU float nebo Q15 se SIMD, v obsluze přerušení pevná čárka, na G4 offload na CORDIC a FMAC). Napříč příklady platí: **cena výpočtu je otázka aritmetiky**, s rozdíly až tří řádů. DSP počítá **příznaky** pro strojové učení (přednáška 12); FFT a filtry z této přednášky jsou jeho vstupem.

11.9 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 11.1 — Proč DSP na MCU

1. Proč je nevýhodné posílat syrová data ze senzoru do cloudu a zpracovávat je tam? Uveďte čtyři důvody.
2. Co je rozpočet cyklů na vzorek a jak jej spočítáte pro danou vzorkovací frekvenci a takt?
3. Proč je Cortex-M vhodný pro real-time DSP?
4. Jaký je kompromis mezi plovoucí a pevnou řádovou čárkou a kdy zvolíte kterou?
5. Proč je obsluha přerušení používající FPU drahá?

K oddílu 11.2 — Číselné formáty

1. Jak je uložen `float32` a proč je `double` v embedded nevhodné?
2. Co je strojové epsilon a jaké dva důsledky má proměnné rozlišení pro práci s `float`?
3. Kolik stojí operace v plovoucí čárce bez FPU a co z toho plyne pro filtry?
4. Definujte formát Qm.n a vysvětlete pravidlo `Q_n` krát `Q_n` je `Q_2n`.
5. Proč se v DSP používá saturace místo wrap-around a jaký je rozdíl mezi zaokrouhlením a ořezáním po posunu?

K oddílu 11.3 — FPU a DSP instrukce

1. Popište registrový soubor FPU a co nese registr FPSCR.
2. Které dvě podmínky musí být splněny, aby se FPU používala, a co se stane, když chybí povolení v CPACR?
3. Které operace FPU umí a které ne, a co z toho plyne pro dělení, odmocninu a trigonometrii?
4. Jak lazy stacking zlevní obsluhu přerušení, která používá FPU (rozšířený rámec)?
5. Co dělá instrukce `__SMLAD` a jaký je vztah ruční SIMD implementace a CMSIS-DSP?
6. Proč je generátor v Q15 řádově rychlejší než ve `float` bez FPU?

K oddílu 11.4 — Číslicová filtrace

1. Napište vztah pro FIR filtr a popište jeho strukturu.
2. Kolik operací MAC na vzorek stojí filtr s N tapy a jak to souvisí s rozpočtem cyklů?
3. O kolik se liší ruční float bez FPU a CMSIS Q15 a čím?
4. Proč se filtry zpracovávají po blocích a jak to navazuje na DMA?
5. V čem se liší FIR a IIR a kdy zvolíte který?

K oddílu 11.5 — FFT a spektrální analýza

1. Proč je přímá DFT nepoužitelná a jakou složitost má FFT?
2. Popište radix-2 princip, butterfly a twiddle faktor.
3. Jaké dvě pasti má ruční FFT a jak je CMSIS-DSP řeší?
4. Proč u reálného signálu stačí `N/2+1` binů a jak okénkování potlačuje spektrální únik?
5. Co určuje frekvenční rozlišení FFT a proč nulové doplnění (zero-padding) skutečné rozlišení nezpřesní?
6. Co je spektrogram (STFT) a jaký kompromis čas versus frekvence v něm platí?

K oddílu 11.6 — Matematické akcelerátory (STM32G4)

1. Jaká dvě úzká hrdla DSP řeší matematické akcelerátory STM32G4 a na jakém obecném principu (specializovaný hardware jako periferie na sběrnici) stojí?
2. Popište princip CORDICu a proč je v hardwaru levný.
3. Jaký je rozdíl mezi rotačním a vektorovým módem?
4. Jak CORDIC nahradí tabulkový generátor a jaké má výhody?
5. Proč je CORDIC klíčový pro FOC řízení motoru?
6. Co dělá FMAC a jak spolu s CORDICem uvolní jádro?

K oddílu 11.7 — Číslicový PID regulátor

1. Co dělá každý ze tří členů PID a proč samotný P-regulátor nechá trvalou regulační odchylku?
2. Jak se spojitý PID převede na diskrétní běžící v přerušení časovače (integrál, derivace, role `Ts`)?
3. Co je integrální windup, kdy nastane a jak mu anti-windup zabrání?
4. Proč se derivuje měřená veličina místo odchylky a proč se derivace filtruje?
5. Co je třeba ohlídat při implementaci PID v pevné řádové čárce?

12

Strojové učení na embedded (TinyML)

> Tato kapitola řeší, **jak natrénovaný model dostat na mikrokontrolér a spustit inferenci v rozpočtu desítek kilobajtů RAM, jednotek miliwattů a milisekund**. To je náplň **TinyML** — ne nový druh AI, ale disciplína **nasazení**: model **zmenšit, převést a spustit**.

Obsah kapitoly

12.1 Proč ML na embedded
12.2 Inference versus trénink
12.3 Workflow TinyML
12.4 Kvantizace
12.5 Inference na MCU
12.6 Případové studie
12.7 Syntéza
12.8 Kontrolní otázky

12.1 Proč ML na embedded

Tam, kde hranici mezi třídami neumíme napsat, ji model naučí z dat. Tato část vysvětlí, co TinyML je a co není, proč se inference vyplatí počítat na zařízení místo v cloudu a na jakých třech pilířích stojí.

Od ručního prahu k naučenému modelu: edge versus cloud

V přednášce 11 jsme klasifikovali ručně: spočítali jsme příznaky (RMS, spektrum z FFT) a porovnali je s prahem. Takový „ruční klasifikátor" funguje pro jednoduché, dobře pochopené hranice, ale u složitých dat (obraz, zvuk, komplexní pohyb) selhává, protože hranici mezi třídami neumíme napsat. Strojové učení ji naučí z dat (Příloha B). Nás dnes nezajímá, *jak* se naučí, ale jak naučený model **spustit na MCU** — a proč to vůbec dělat lokálně místo poslání dat do cloudu. TinyML stojí na třech pilířích: **malé modely** (architektury navržené pro MCU, například MobileNet nebo DS-CNN), **kvantizace** (int8 místo float32) a **efektivní runtime** (TFLite Micro, CMSIS-NN, nástroje výrobce).

Edge versus cloud

Proč počítat inferenci na zařízení, a ne na serveru? **Edge inference** je proti cloudu řádově výhodnější v latenci (jednotky až desítky milisekund proti stovkám), energii (desítky až stovky mikrojoulů proti desítkám až stovkám milijoulů), nulové přenosové náročnosti, stoprocentní dostupnosti (nezávisí na síti) i soukromí (data neopouštějí zařízení). Platí se za to menším

Edge inference
Provádění inference neuronové sítě přímo na zařízení, kde data vznikají, místo jejich odeslání na server. Přináší nízkou latenci, malou spotřebu, nezávislost na síti a soukromí, za cenu menšího modelu než v cloudu.

modelem a o něco nižší přesností. Je to tentýž vzor jako u DSP z přednášky 11: **extrahovat informaci na místě, neposílat syrová data** — kamera, která lokálně rozhodne „je tam osoba", pošle jeden bit, ne video.

Co TinyML je a není

Kolem TinyML koluje řada mýtů. Není to nový druh AI — jsou to **stejné neuronové sítě jako na serveru**, jen optimalizované. Není to jednodušší matematika — konvoluce, plně propojené vrstvy a softmax jsou tytéž. A většinou to neznamená výrazně horší přesnost — kvantizace stojí obvykle jen jedno až tři procenta. TinyML tedy **není o zjednodušení modelu, ale o jeho efektivním nasazení**; mění se jen čísla (int8), runtime a rozpočet, ne matematika. Společným jmenovatelem aplikací (detekce klíčových slov, rozpoznání aktivity, detekce osoby, prediktivní údržba) je, že vstupem je **proud dat ze senzoru** — mikrofon, akcelerometr, kamera — který přes DMA (přednáška 5) plní buffer, a model z něj odvodí rozhodnutí.

> **Shrnutí oddílu 12.1. TinyML** není nový druh AI, ale disciplína **nasazení** hotových sítí na MCU (stejná matematika, jen int8, runtime, rozpočet). **Edge inference** je proti cloudu deset až dvě stě krát rychlejší a úspornější, nezávislá na síti a chrání soukromí — za cenu menšího modelu. Stojí na třech pilířích: **malé modely, kvantizace, efektivní runtime**. Vstupem je vždy proud dat ze senzoru přes DMA (přednáška 5).

12.2 Inference versus trénink

Nasazení modelu na MCU stojí na jednom rozdílu: trénink a inference jsou dvě různé úlohy na dvou místech. Tato část ukáže, že síť je hlavně násobení s akumulací, projde typy vrstev a konvoluci a vymezí, co na MCU vlastně běží.

Síť je hlavně MAC: vrstvy a konvoluce

Neuron spočítá vážený součet vstupů plus posun a prožene ho aktivací; dominantní operací je tedy **násobení s akumulací (MAC)**, přesně ta z přednášky 11. Celá inference jsou **miliony MAC** — konvoluce i plně propojené vrstvy jsou jen strukturované MAC:

```
int32_t acc = bias;
for (int i = 0; i < n; i++)
    acc += weight[i] * input[i];   // MAC, běží milionkrát za inferenci
output = activation(acc);
```

Proto zde platí vše z přednášky 11 (pevná řádová čárka, `__SMLAD`, saturace) a proto existují NPU (přednáška 8): akcelerují právě MAC. Aktivační funkce přidává nelinearitu (Příloha B), ale na MCU se liší cenou o řád: **ReLU** je jedno porovnání s nulou (jeden až dva takty) a proto dominuje, kdežto sigmoida a tanh se počítají z tabulky (dvacet až padesát taktů) a nechávají se jen na výstupní vrstvě. Volba architektury je i volbou levných operací (proto MobileNet používá ReLU6).

Anatomie sítě: vrstvy a konvoluce

Neuronová síť je **posloupnost vrstev**; každá transformuje aktivace předchozí. Na embedded potkáme hlavně čtyři typy. **Plně propojená (dense)** vrstva váží každým neuronem všechny vstupy — má hodně parametrů, ale málo výpočtu na parametr. **Konvoluční (conv)** vrstva klouže malým filtrem po vstupu: řádově méně parametrů, ideální pro obraz a zvuk. **Pooling** (max nebo průměr) podvzorkuje a zmenší rozlišení bez jediného parametru. **Softmax** na výstupu převede skóre na pravděpodobnosti tříd. Volba vrstev určuje rozpočet: dense vrstvy jsou drahé na *parametry* (Flash), conv na *výpočet* (MAC), a proto modely pro MCU (MobileNet,

DS-CNN) staví právě na konvoluci.

Klíč ke konvoluci jsou **sdílené váhy**. Plně propojená vrstva nad obrazem 96×96 by měla desítky milionů vah; konvoluce použije jediný filtr, například 3×3, a **tentýž ho aplikuje na všechny pozice**. Devět vah místo milionů — a navíc s prostorovou invariancí: filtr detekující hranu ji najde kdekoli. Právě sdílení vah dělá vidění na MCU vůbec proveditelným.

Sdílené váhy
Princip konvoluční vrstvy: tentýž malý filtr (jádro) se aplikuje na všechny pozice vstupu, takže jedna sada vah slouží celému obrazu. Oproti plně propojené vrstvě to řádově zmenší počet parametrů a zavádí prostorovou invarianci — hrana se detekuje stejně bez ohledu na polohu.

Dvě různé úlohy na dvou místech

Zásadní rozdíl, na kterém stojí celá přednáška: **trénink a inference jsou dvě různé úlohy na dvou různých místech.** Trénink probíhá na PC nebo v cloudu (na GPU): váhy se mění, počítají se gradienty, běží dopředný i **zpětný** průchod, po dávkách desítek až stovek vzorků, ve float32, a paměti je potřeba zhruba třikrát tolik (váhy, gradienty, stav optimalizátoru). Inference na MCU je opak: váhy jsou **fixní** (read-only ve Flash), gradienty nejsou potřeba, běží **jen dopředný průchod** nad **jedním** vzorkem v reálném čase, v int8, a paměti stačí jen na váhy a aktivace. Na MCU tedy běží deterministický dopředný výpočet s fixními váhami — žádné učení; trénink (backpropagation, Adam) je záležitost PC a patří do Přílohy B. Právě toto rozdělení určuje celý rozpočet.

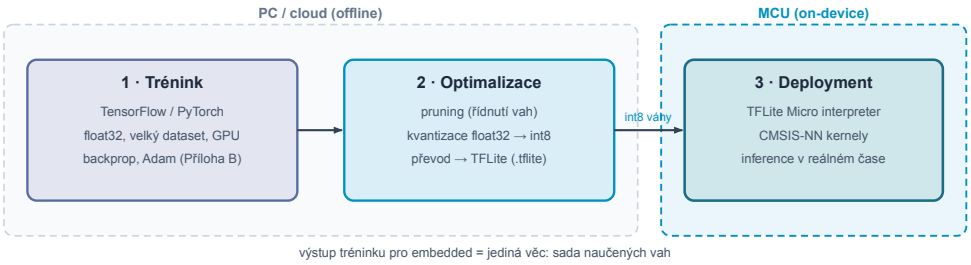
> **Shrnutí oddílu 12.2.** Neuron je **MAC plus aktivace**; celá inference jsou miliony MAC, takže zde platí vše z přednášky 11 a proto existují NPU (přednáška 8). **Aktivace se liší cenou**: ReLU (jeden až dva takty) dominuje, drahé sigmoid a tanh jen na výstupu. Síť je **posloupnost vrstev** (dense, conv, pooling, softmax); **konvoluce sdílí váhy** (jeden filtr na všechny pozice), a proto má řádově méně parametrů — modely pro MCU staví na ní. **Trénink a inference** jsou dvě úlohy: trénink na PC (zpětný průchod, gradienty, float32, Příloha B), na MCU jen **dopředný průchod s fixními int8 váhami**. Toto rozdělení určuje rozpočet.

12.3 Workflow TinyML

Cesta od nápadu k běžícímu modelu má tři fáze a jen poslední je na MCU. Tato část je projde — trénink, optimalizaci a nasazení — a ukáže dělbu práce mezi nástroji TFLite, TFLite Micro a CMSIS-NN.

Tři kroky od dat k firmwaru

Cesta od nápadu k běžícímu modelu má tři fáze, a jen poslední je na MCU:



Obrázek 12.1. Workflow TinyML ve třech krocích. Trénink na PC (TensorFlow nebo PyTorch, float32, GPU) vytvoří model; optimalizace na PC ho prořeže, kvantizuje na int8 a převede do formátu TFLite; deployment na MCU spustí model přes interpreter TFLite Micro a kernely CMSIS-NN. Mezi PC a MCU putuje jediná věc: sada naučených vah.

Nejprve **trénink** (PC nebo cloud) ve standardním TensorFlow nebo PyTorch, plná přesnost float32, velký dataset, GPU. Pak **optimalizace** (PC): pruning (odstranění zbytečných vah), kvantizace (float32 na int8) a převod do formátu TFLite. Nakonec **deployment** (MCU):

interpreter TFLite Micro plus kernely CMSIS-NN provádějí inferenci v reálném čase. Výstupem tréninku je jediná věc, kterou embedded zajímá — **sada naučených vah**; jak vznikly, řeší Příloha B. Kroky optimalizace a deploymentu jsou náplní této přednášky.

Frameworky: kdo co dělá

Na MCU se model nespouští ručně; mezi váhami a hardwarem je vrstva runtime. **TFLite Micro** (TFLM) je minimální interpreter, který přečte model `.tflite` a provede jeho vrstvy bez operačního systému a bez dynamické alokace. Jednotlivé vrstvy počítá knihovna **CMSIS-NN** — optimalizované kernely konvoluce, plně propojených vrstev a poolingu, které využívají SIMD MAC z přednášky 11. Nástroje výrobce (STM32Cube.AI) navíc umějí model převést rovnou na C kód pro danou rodinu. Dělba práce je tedy jasná: TFLite převede a kvantizuje na PC, TFLM řídí běh vrstev na MCU, CMSIS-NN počítá jednotlivé vrstvy rychle na jádře — a na NPU (přednáška 8) volá runtime místo CMSIS-NN akcelerátor.

> **Shrnutí oddílu 12.3.** Workflow má tři fáze: **trénink** (PC, float32), **optimalizace** (PC: pruning, kvantizace, převod TFLite) a **deployment** (MCU). Výstup tréninku pro embedded je **sada vah**. Runtime na MCU: **TFLite Micro** (interpreter modelu) volá **CMSIS-NN** (rychlé kernely přes SIMD MAC), nebo **NPU** (přednáška 8). Nástroje výrobce (STM32Cube.AI) umějí model převést rovnou na C kód.

12.4 Kvantizace

Nejúčinnější jedinou optimalizací je převod čísel z float32 na int8. Tato část vysvětlí, proč síti kvantizace nevadí, jak funguje měřítko a nulový bod, jak vypadá kvantovaný MAC a requantizace a jak se model kalibruje a laděním připraví (PTQ, QAT).

Proč int8: scale a zero-point

Kvantizace je převod vah a aktivací z float32 na int8 — nejúčinnější jediná optimalizace TinyML. Int8 je čtyřikrát menší (jeden bajt místo čtyř), MAC je dvakrát až pětkrát rychlejší a SIMD zpracuje čtyři hodnoty najednou; cenou je nižší přesnost (256 hodnot místo sedmi platných cifer), obvykle jen jedno až tři procenta. Model o velikosti 100 kB ve float32 se v int8 vejde do 25 kB, a často teprve tím do Flash. V jádru je to **přesně pevná řádová čárka z přednášky 11**, jen aplikovaná na váhy sítě: méně bitů, celočíselná ALU, SIMD MAC.

Proč int8 vůbec stačí? Neuronová síť je **redundantní a robustní vůči šumu**: byla trénována se šumem v datech i s regularizací, takže několik desetin procenta chyby na každé váze se v součtu milionů MAC zprůměruje a nevadí. Osm bitů dá 256 úrovní, což na váhy i aktivace bohatě stačí — na rozdíl od klasického DSP filtru, kde by kvantizační chyba koeficientů posunula póly a rozladila charakteristiku. V síti se kvantizační šum zprůměruje, ve filtru ne.

Scale a zero-point

Kvantizace mapuje reálný interval na 256 celých čísel afinním vztahem mezi reálnou hodnotou a kvantovanou: $r = S \cdot (q - Z)$.

Parametr **scale** `S` je šířka jednoho kroku (`S = (r_max − r_min) / (q_max − q_min)`) a **zero-point** `Z` je celé číslo, kterému odpovídá reálná nula. Podle rozložení dat volíme **symetrickou** kvantizaci (`Z = 0`, pro váhy kolem nuly) nebo **asymetrickou** ($Z \neq 0$, pro aktivace po ReLU, které jsou nezáporné). Váhy se navíc kvantují **per-channel** (vlastní scale na kanál) pro vyšší přesnost.

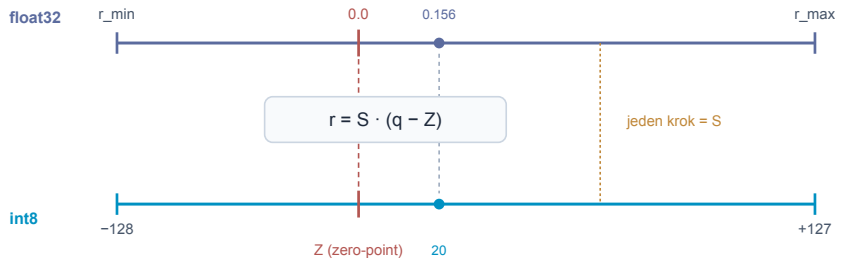
TFLite Micro
Minimální interpreter neuronových sítí pro mikrokontroléry. Přečte model ve formátu TFLite a provede jednotlivé vrstvy bez operačního systému a bez dynamické alokace; jednotlivé vrstvy počítá voláním optimalizovaných kernelů.

CMSIS-NN
Knihovna optimalizovaných kernelů neuronových sítí pro jádra Cortex-M (konvoluce, plně propojené vrstvy, pooling). Využívá celočíselné SIMD MAC instrukce, a je proto rychlá; TFLite Micro ji volá pro provedení vrstev.

Kvantizace
Převod vah a aktivací sítě z plovoucí řádové čárky (float32) na celočíselný formát s nižší přesností (typicky int8). Zmenší model a zrychlí výpočet za malou ztrátu přesnosti; je to nejúčinnější jediná optimalizace TinyML a v jádru je to pevná řádová čárka z přednášky 11.

Scale
Měřítko kvantizace: šířka jednoho kroku, tedy o kolik se změní reálná hodnota při změně kvantované hodnoty o jedničku. Počítá se z rozsahu reálných hodnot dělených počtem kvantovacích úrovní.

Zero-point
Celé číslo, kterému v kvantizaci odpovídá reálná nula. U symetrické kvantizace je nulový, u asymetrické nenulový; umožňuje přesně reprezentovat nulu a využít celý rozsah u nesymetrických dat.



Obrázek 12.2. Kvantizační schéma. Reálný interval float32 se lineárně zobrazí na 256 hodnot int8; scale S je šířka jednoho kroku a zero-point Z je celé číslo, kterému odpovídá reálná nula. Například hodnota 0,156 se zobrazí na celé číslo 20.

Kvantovaný MAC a requantizace

Kvantovaná vrstva počítá MAC v int8, ale **akumuluje do širokého int32** (jinak přeteče), a pak výsledek **requantizuje** zpět na int8:

```
int32_t acc = 0;
for (int i = 0; i < n; i += 2)
    acc = __SMLAD(*(int32_t*)&w[i], *(int32_t*)&x[i], acc);  // 2x int8 MAC
acc += bias;
int32_t out = ((int64_t)acc * multiplier) >> shift;         // requantizace
out += output_zero_point;
output = __SSAT(out, 8);                                     // saturace na int8
```

Je to táž konstrukce jako v přednášce 11: `__SMLAD` (dvě MAC naráz), široký akumulátor, `__SSAT`. **Saturace je přitom kritická**: bez ní by int8 přetekl otočením (100 plus 50 dá místo 150 hodnotu −106) a hodnota aktivace by se znehodnotila; instrukce `__SSAT` ořízne výsledek na platný rozsah v jednom taktu. Kvantizace je tak most mezi „naučeným modelem" a „fixed-point DSP kódem", který na MCU umíme.

Requantizace: zpět na int8 bez dělení

Akumulátor je v int32, ale jeho měřítko je jiné než měřítko výstupu: vstupy měly scale `S_x`, váhy `S_w`, takže součin má scale `S_w · S_x`, kdežto výstup má být ve scale `S_y`. **Requantizace** přepočítá mezi třemi škálami: `q_out = round(S_w·S_x / S_y · acc) + Z_out`.

Poměr měřítek `M = S_w·S_x / S_y` je reálné číslo mezi nulou a jedničkou, ale **známé už při překladu**. Předpočítá se proto na PC na celočíselný násobitel a bitový posun (`M ≈ multiplier / 2^31`), takže se za běhu počítá jen `((int64_t)acc * multiplier) >> shift` — jedno násobení a jeden posun, žádné dělení a žádná plovoucí čárka. Proto je kvantovaná inference tak rychlá: i přechod mezi vrstvami je několik celočíselných instrukcí.

Per-tensor, per-channel a kalibrace

Kvantizovat lze v různé zrnitosti. **Per-tensor** dává celému tenzoru vah jediný scale — jednoduché, ale kanál s malými hodnotami ztrácí na přesnosti, protože měřítko určuje kanál s velkým rozsahem. **Per-channel** dá každému výstupnímu kanálu konvoluce vlastní scale, takže se každý filtr kvantuje na svůj rozsah; stojí to jen pole scale místo jednoho čísla a je to dnes standard pro váhy. Aktivace se kvantují per-tensor (zjednodušuje runtime).

Rozsah vah známe (jsou fixní), ale **rozsah aktivací závisí na vstupu** — a ten při překladu neznáme. Zjistí se **kalibrací**: hotovou float sítí se pustí několik set reprezentativních vzorků,

Requantizace
Převod širokého celočíselného akumulátoru (int32) kvantované vrstvy zpět na int8 ve správném měřítku výstupu. Poměr měřítek vah, vstupu a výstupu je konstanta známá při překladu, takže se předpočítá jako celočíselný násobitel a bitový posun — celý přepočet je pak jedno násobení a jeden posun, bez dělení a bez plovoucí čárky.

Kalibrací
Krok kvantizace, ve kterém se změří rozsahy aktivací: hotovou float sítí se pustí několik set reprezentativních vzorků a zaznamenají se minima a maxima výstupů každé vrstvy. Z nich se určí scale a zero-point aktivací. Kalibrační sada nemusí mít štítky a nepotřebuje trénink.

zaznamenají se minima a maxima na výstupu každé vrstvy a z nich se určí scale a zero-point. Kalibrační data **nepotřebují štítky ani trénink** — proto je PTQ tak levné. Nereprezentativní kalibrační sada je ale častou příčinou propadu přesnosti po kvantizaci.

Pruning a layer fusion

Kvantizace není jediná optimalizace na PC. **Pruning** vynuluje váhy blízké nule (síť je řídká, mnoho vah k výsledku nepřispívá) a doučí zbytek; zmenší model a u strukturovaného pruningu i výpočet. **Layer fusion** slije sousední operace do jedné: klasicky konvoluci, batch normalization a ReLU do jediného kernelu. Batch norm je za inference jen afinní přeškálování, které lze **sloučit přímo do vah a biasu** konvoluce — ušetří se průchod pamětí a jeden mezitenzor. Obě optimalizace probíhají na PC před převodem; na MCU jde už menší a slitý model.

PTQ versus QAT

Kdy kvantizovat — až po tréninku, nebo už během něj? **PTQ** (Post-Training Quantization) kvantuje hotový model; nevyžaduje re-trénink, stačí malý kalibrační dataset k určení rozsahů scale a zero-point, a ztráta přesnosti bývá malá. **QAT** (Quantization-Aware Training) simuluje kvantizaci už při tréninku: dopředný průchod zaokrouhluje jako int8, ale gradienty tečou skrz (Straight-Through Estimator), takže se model naučí být robustní vůči kvantizačnímu šumu a dosáhne vyšší přesnosti — za cenu nového tréninku. Pravidlo zní: začněme PTQ (levné); když je ztráta přesnosti neúnosná (malé modely, agresivní kvantizace), sáhněme po QAT. Obojí probíhá na PC, na MCU jde už hotový int8 model.

PTQ
Post-Training Quantization: kvantizace hotového, již natrénovaného modelu. Nevyžaduje nový trénink, stačí malý kalibrační dataset k určení rozsahů. Rychlé a jednoduché, se zpravidla malou ztrátou přesnosti.

QAT
Quantization-Aware Training: kvantizace simulovaná už během tréninku (fake quantization), kdy dopředný průchod zaokrouhluje jako int8, ale gradienty tečou skrz. Model se naučí být robustní vůči kvantizačnímu šumu, čímž dosáhne vyšší přesnosti za cenu nového tréninku.

> **Shrnutí oddílu 12.4. Kvantizace float32 na int8** je nejúčinnější optimalizace: čtyřikrát menší, dvakrát až pětkrát rychlejší, čtyřnásobná SIMD, za jedno až tři procenta přesnosti (síť je robustní vůči šumu). Schéma `r = S·(q − Z)`: **scale** (krok) a **zero-point** (nula), symetrická pro váhy, asymetrická pro aktivace. Kvantovaný MAC je **táž konstrukce jako přednáška 11**: `__SMLAD`, široký akumulátor, `__SSAT`. **Requantizace** vrací int32 na int8 jedním násobením a posunem (`M = S_w·S_x/S_y` předpočítáno). Váhy se kvantují **per-channel**, aktivace potřebují **kalibraci** (rozsahy z reprezentativních vzorků). Dále **pruning** (řídké váhy) a **layer fusion** (conv+BN+ReLU do jednoho). **PTQ** (po tréninku, rychlé) a **QAT** (simuluje kvantizaci při tréninku, přesnější); vše na PC.

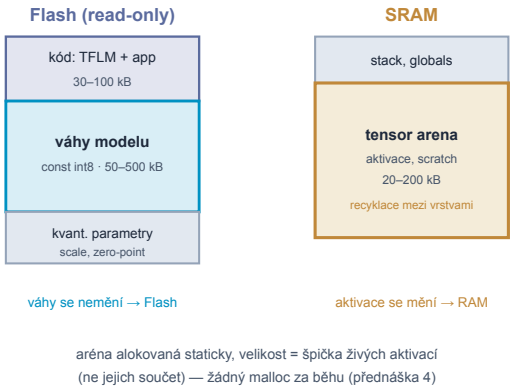
12.5 Inference na MCU

Model musí zapadnout do tvrdého rozpočtu paměti. Tato část ukáže, kam patří váhy a kam aktivace, jak tensor arena recykluje paměť, kde inference běží a jak se profiluje — a proč je deterministická.

Rozpočet paměti a tensor arena

Inference potřebuje v paměti dvě věci: **váhy** (velké, ale neměnné, a tedy do Flash) a **aktivace** (mezivýsledky vrstev, do RAM). Rozpočet je tvrdý:

Ve Flash je kód (TFLite Micro plus aplikace, 30 až 100 kB), **váhy modelu** (const int8, 50 až 500 kB) a kvantizační parametry; v SRAM je zásobník, globální stav a **tensor arena** (20 až 200 kB) na aktivace. Váhy do Flash, aktivace do RAM. Volba rodiny MCU (velikost Flash a RAM) je proto první, řádové omezení — stejně jako u spotřeby v přednášce 10.



Obrázek 12.3. Rozpočet paměti při inferenci. Ve Flash je kód runtime a aplikace (desítky až sto kilobajtů), váhy modelu (const int8, padesát až pět set kilobajtů) a kvantizační parametry; v SRAM je zásobník, globální stav a tensor arena (dvacet až dvě stě kilobajtů) na aktivace. Váhy se nemění, patří do Flash; aktivace se mění, patří do RAM.

Tensor arena

Aktivace by naivně potřebovaly RAM pro všechny vrstvy najednou, což se nevejde. Řešením je **tensor arena** — jeden buffer, ve kterém se aktivace vrstev recyklují. Vrstva čte vstup a zapisuje výstup do arény; jakmile jej spotřebuje následující vrstva, může jej vrstva za ní přepsat. Runtime dopředu spočítá **maximální současně živou** paměť a jen tu alokuje. Aréna se přitom alokuje **staticky, jednou** (žádný `malloc` za běhu, přednáška 4), a její velikost je špička živých aktivací, ne jejich součet — proto se i síť s desítkami vrstev vejde do desítek kilobajtů RAM.

Tensor arena
Jeden staticky alokovaný buffer v RAM, ve kterém se recyklují mezivýsledky (aktivace) jednotlivých vrstev sítě. Jakmile výstup vrstvy spotřebuje následující vrstva, může jej další vrstva přepsat; velikost arény je špička současně živých aktivací, ne jejich součet.

Kde inference běží

Tytéž vrstvy může počítat několik jednotek; volba je stejná jako v přednášce 11 (jaká aritmetika) a 8 (jaký hardware). Na běžném MCU (M4, M7) počítá **CMSIS-NN** na jádře int8 MAC přes SIMD (`__SMLAD`); na jádrech M55 a M85 pomůže vektorové rozšíření **Helium** (přednáška 7); a na čipech s **NPU** (STM32N6, Ethos-U) běží model na systolickém poli (přednáška 8) desítky až stokrát rychleji při zlomku energie. Runtime (TFLite Micro) je pořád stejný; liší se jen kernely, které volá. Volba rodiny MCU (Flash, RAM, přítomnost NPU) je tak první a řádové omezení celého návrhu.

Operátory a profilování

Runtime nepočítá „síť", ale posloupnost **operátorů** — `conv2d`, `depthwise_conv`, `fully_connected`, `pool`, `softmax`, `add`. Interpreter zpracovává model vrstvu po vrstvě a pro každou zavolá odpovídající kernel (na M4/M7 z CMSIS-NN). Praktický důsledek: model se převede jen tehdy, jsou-li **všechny jeho operátory v runtime k dispozici** — exotická vrstva z tréninku znamená chybu při převodu, ne až za běhu.

Operátorů
Elementární operace, ze kterých interpreter skládá běh sítě: konvoluce, depthwise konvoluce, plně propojená vrstva, pooling, softmax, sčítání. Každou počítá jeden optimalizovaný kernel; model se převede jen tehdy, jsou-li všechny jeho operátory v runtime dostupné.

Kolik která vrstva stojí, se změří **profilováním** přes čítač cyklů `DWT->CYCCNT` (přednáška 11): kolem každého operátoru se odečte stav čítače a rozdíl dá takty, tedy i čas při známé frekvenci. Profilování odhalí **úzké hrdlo** — typicky jedna dvě velké conv nebo dense vrstvy nesou většinu MAC — a optimalizace (pruning, menší šířka, přesun na NPU) míří právě tam. Bez měření se optimalizuje naslepo, stejné pravidlo jako u WCET v přednášce 11.

Determinismus a real-time

Na rozdíl od tréninku má inference na MCU **pevnou, předvídatelnou cenu**. Váhy jsou fixní, dávka je jeden vzorek, není žádné větvení podle hodnoty dat a tensor arena je staticky alokovaná — doba inference je proto **konstantní**, nezávisí na vstupu a neběží v ní `malloc` (přednáška 4), takže nehrozí fragmentace ani nedeterminismus. Kvantovaný model se tak dá zařadit do RTOS úlohy s pevným deadlinem (přednáška 9) nebo do duty-cyklu pod baterií (přednáška 10). „Průměrně rychlá" inference nestačí — pro embedded musí být rychlá vždy, a inference tuto vlastnost přirozeně má.

> **Shrnutí oddílu 12.5.** Rozpočet: **váhy do Flash** (read-only, 50 až 500 kB), **aktivace do RAM** (tensor arena, 20 až 200 kB), runtime 30 až 100 kB Flash. **Tensor arena** recykluje paměť mezi vrstvami — alokuje se staticky a jen na špičku živých aktivací, ne jejich součet (žádný `malloc`, přednáška 4). Vrstvy počítá **CMSIS-NN** na jádře, **Helium** na M55, nebo **NPU** (přednáška 8); runtime stejný, mění se kernely. Runtime skládá běh z **operátorů** (conv2d, depthwise, dense, pool, softmax); jejich cenu měří **profilování** přes `DWT->CYCCNT` (přednáška 11) a míří na úzké hrdlo. Inference je **deterministická** (konstantní čas, žádný `malloc`) — zařaditelná do RTOS deadlinu (přednáška 9). Volba rodiny MCU je první, řádové omezení.

12.6 Případové studie

Teorii nejlépe ukáží konkrétní úlohy. Tato část projde tři modality jedním postupem: detekci klíčových slov ze zvuku, prediktivní údržbu z vibrací a detekci osoby z obrazu.

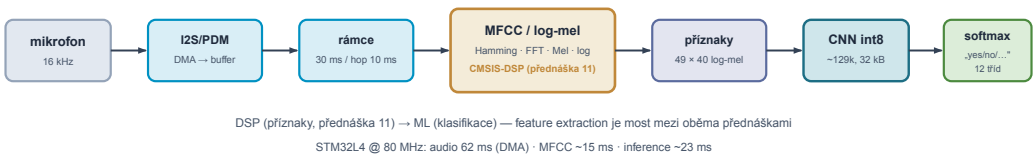
Tři modality, jeden postup

Všechny TinyML aplikace sdílejí tentýž řetěz: **senzor přes DMA (přednáška 5)** → **předzpracování a příznaky (DSP, přednáška 11)** → **kvantovaný model (přednáška 8)** → **rozhodnutí**. Liší se jen modalita a model. Projdeme tři reprezentativní úlohy: **detekci klíčových slov** (zvuk, mikrofon, CNN nad spektrem), **prediktivní údržbu** (vibrace, akcelerometr, autoencoder) a **detekci osoby** (obraz, kamera, MobileNetV2). Ukazují tři modality, ale jediný inženýrský postup.

Detekce klíčových slov (zvuk)

Úloha **detekce klíčových slov** (keyword spotting) rozpozná malou sadu povelů v proudu zvuku. Mikrofon (16 kHz) plní přes I2S nebo PDM a DMA buffer, signál se rozdělí na rámce (30 ms, krok 10 ms) a z každého se spočítají příznaky.

Detekce klíčových slov
Keyword spotting (KWS): rozpoznání malé sady povelů (například ano, ne, stop) v proudu zvuku z mikrofonu. Vstupem je log-mel spektrum, modelem malá CNN v int8; běží trvale s nízkou spotřebou jako hlasový spouštěč.



Obrázek 12.4. Řetěz detekce klíčových slov: mikrofon 16 kHz plní přes I2S/PDM a DMA buffer, signál se dělí na rámce 30 ms s krokem 10 ms, z každého rámce se přes CMSIS-DSP (přednáška 11) spočítá log-mel spektrum, 49 rámců po 40 hodnotách tvoří příznakovou matici, tu klasifikuje CNN v int8 a softmax dá pravděpodobnosti 12 tříd. Feature extraction je most mezi přednáškou 11 (DSP) a 12 (ML).

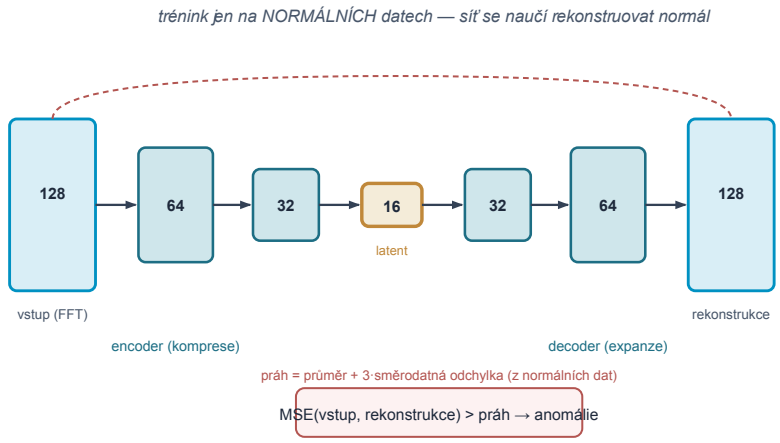
Příznaky jsou **log-mel spektrum**: surové audio je pro síť špatný vstup, proto se předzpracuje na logaritmickou mel škálu, která napodobuje lidský sluch a komprimuje méně důležité vysoké frekvence. Celý řetěz je čistý DSP z přednášky 11 — okno Hammingovým oknem,

`arm_rfft_fast_f32`, mel filtrbanka, logaritmus. Historicky se z mel spektra počítal ještě DCT na koeficienty MFCC (13 hodnot vhodných pro klasické GMM/HMM), ale na vstup moderní **CNN** přivádíme přímo log-mel spektrum (40 hodnot): DCT sice dekoreluje, ale ztrácí lokální frekvenční vztahy, které konvoluce umí využít, a filtry si síť naučí sama. Zde je **feature extraction doslova most mezi přednáškou 11 a 12**: DSP spočítá příznaky, ML nad nimi klasifikuje. Matici 49×40 klasifikuje malá CNN (přibližně 129 tisíc parametrů, v int8 asi 32 kB) a softmax dá pravděpodobnosti dvanácti tříd. Na STM32L4 na 80 MHz stojí audio 62 ms (DMA), MFCC asi 15 ms a inference asi 23 ms — vejde se do reálného času s velkou rezervou.

Prediktivní údržba (autoencoder)

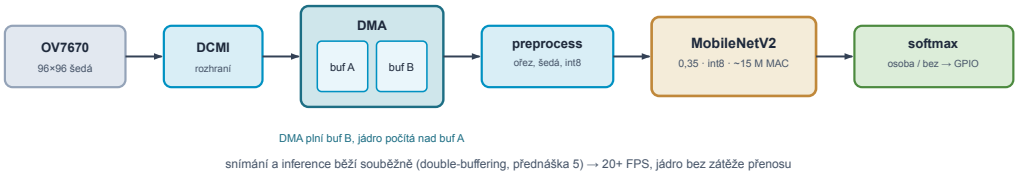
Prediktivní údržba sleduje stav stroje a hlásí anomálii dřív, než dojde k poruše. Vstupem jsou vibrace z akcelerometru, ze kterých se přes FFT (přednáška 11) spočítá spektrum. Problém je, že **poruchy jsou vzácné a různorodé** — nešly by naučit jako běžná klasifikace, protože příklady poruch skoro nemáme. Řeší to **autoencoder**: síť trénovaná **jen na normálních datech** rekonstruovat vlastní vstup.

Autoencoder
Neuronová síť trénovaná rekonstruovat vlastní vstup přes úzké latentní hrdlo. Naučí se komprimovat a obnovit jen data, která viděla při tréninku (normální stav). Na anomálii, která nebyla v trénovacích datech, rekonstrukce selže a chyba (MSE) překročí práh — proto detekuje anomálie bez jejich příkladů.



Obrázek 12.5. Autoencoder pro detekci anomálií: vstupní spektrum (128 hodnot z FFT) se přes encoder komprimuje do úzkého latentního prostoru (128 → 64 → 32 → 16) a decoder ho zase rozbalí zpět na 128 hodnot. Síť se trénuje jen na normálních datech, takže je umí rekonstruovat s malou chybou; na anomálii rekonstrukce selže a chyba MSE překročí práh nastavený jako průměr plus tři směrodatné odchylky z normálních dat.

Síť má tvar motýlka: encoder spektrum komprimuje do malého latentního prostoru (128 → 64 → 32 → 16) a decoder ho zase rozbalí (16 → 32 → 64 → 128). Trénink na normálním provozu naučí síť rekonstruovat **normál** s malou chybou. Na anomálii, která nebyla v trénovacích datech, rekonstrukce selže a chyba `MSE(vstup, rekonstrukce)` prudce vzroste; překročí-li práh, hlásí se anomálie. Práh se nastaví staticky jako **průměr plus tři směrodatné odchylky** rekonstrukční chyby na normálních datech. Podstata je, že **anomálie nepotřebuje trénovací příklady** — cokoli mimo naučený normál se rekonstruuje špatně; model je malý (kolem 22 kB int8) a běží pod baterií na akcelerometru.



Obrázek 12.6. Řetěz detekce osoby: kamera OV7670 streamuje snímky přes rozhraní DCMI a DMA do dvou bufferů v RAM, snímek se předzpracuje (ořez a škála na 96×96, převod na int8), projde modelem MobileNetV2 a softmax dá rozhodnutí osoba nebo bez osoby. Díky dvojitému bufferu plní DMA jeden snímek, zatímco jádro počítá inferenci nad druhým.

Detekce osoby: řetěz, model a rozpočet

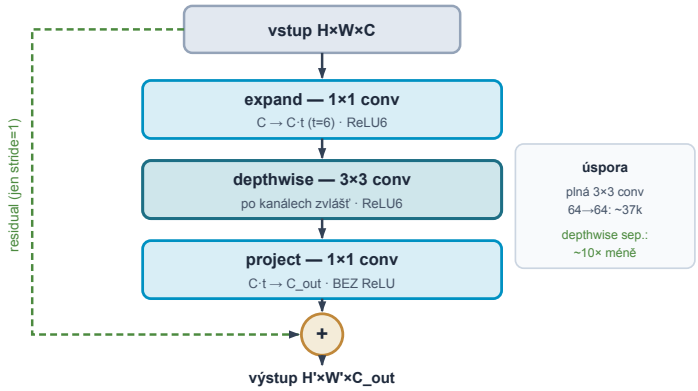
Pokročilá, ale reálná TinyML úloha je kamera, která lokálně rozhodne o přítomnosti osoby. Vstupem je kamera OV7670 (obraz 96×96 ve stupních šedi), modelem MobileNetV2 (width 0,35) v int8, výstupem softmax (osoba, nebo bez osoby) vedoucí na rozhodnutí přes GPIO nebo UART; platformou je STM32H7 (Cortex-M7 na 480 MHz) s přibližně 350 kB Flash, 150 kB SRAM a snímkovou frekvencí přes dvacet za sekundu.

Kamera streamuje snímky přes rozhraní DCMI do RAM pomocí DMA (bez zátěže jádra, přednáška 5), snímek se předzpracuje, projde modelem a softmax dá rozhodnutí. Klíčem k rychlosti je **dvojitý buffer** (přednáška 5): zatímco DMA plní snímek B, jádro počítá inferenci nad snímkem A, takže snímání a výpočet běží souběžně a drží dvacet snímků za sekundu; bez překryvu by se čekání na snímek a inference sčítaly.

Proč se model vejde: inverted residual

Naivní konvoluční síť by měla miliony parametrů. MobileNetV2 je navržená pro MCU a klíčem je **inverted residual blok**:

Inverted residual blok
Stavební blok sítě MobileNetV2, který nahradí drahou plnou konvoluci třemi levnými kroky: rozšíření kanálů konvolucí 1×1, prostorová depthwise konvoluce 3×3 zvlášť po kanálech a projekce zpět konvolucí 1×1. Má řádově méně parametrů než plná konvoluce a přidává reziduální spojení.



Obrázek 12.7. Inverted residual blok: vstup projde konvolucí 1×1 (expand, rozšíří kanály faktorem t s aktivací ReLU6), depthwise konvolucí 3×3 (prostorové zpracování zvlášť po kanálech, ReLU6) a konvolucí 1×1 (project, zmenší kanály zpět, bez aktivace); při kroku jedna se přičte reziduální spojení. Depthwise separovatelná konvoluce má řádově méně parametrů než plná.

Místo jedné plné konvoluce ji blok rozloží na tři levné kroky: **expand** (1×1 zvětší kanály), **depthwise** (3×3 zvlášť po kanálech) a **project** (1×1 zmenší zpět). Plná konvoluce 3×3 z 64 na 64 kanálů má zhruba $9 \cdot 64 \cdot 64$, tedy asi 37 000 parametrů; depthwise separovatelná verze má řádově méně (kolem desetiny). Proto se celý MobileNetV2 vejde do zhruba 250 kB v int8 a přibližně 15 milionů MAC na snímek — zvládnutelné na M7 v reálném čase.

Rozpočet a kompromisy

Model má vrstvenou strukturu, kde rozlišení klesá a počet kanálů roste (od 96×96×1 po 3×3×280), pak následuje global pooling a softmax. Rozpočet: přibližně 250 kB parametrů ve Flash, 150 kB aktivací v aréně, 15 milionů MAC na snímek a přes dvacet snímků za sekundu na M7. Vždy jde o kompromis **přesnost krát velikost krát spotřeba** (přednáška 10): menší width (0,35 místo 1,0) znamená méně parametrů a MAC a o něco nižší přesnost, ale model se vejde a stíhá. Na NPU (STM32N6) by tentýž model běžel při zlomku energie.

> **Shrnutí oddílu 12.6.** Tři modality, jeden postup: **senzor přes DMA** → **příznaky (DSP)** → **model** → **rozhodnutí**. **Detekce klíčových slov** (zvuk): mikrofon, log-mel spektrum přes CMSIS-DSP (přednáška 11), CNN 32 kB int8, softmax; feature extraction je most přednášek 11 a 12. **Prediktivní údržba**: akcelerometr, FFT, **autoencoder** trénovaný jen na normálu — anomálie = velké MSE nad prahem (průměr + 3σ), nepotřebuje příklady poruch. **Detekce osoby**: kamera OV7670 96×96, MobileNetV2 0,35 int8, STM32H7; **DCMI, DMA a dvojitý buffer** (přednáška 5) drží snímkovou frekvenci, **inverted residual** (expand, depthwise, project) dá řádově méně parametrů (∼250 kB, 15 M MAC/snímek). Vždy kompromis **přesnost krát velikost krát spotřeba** (přednáška 10); NPU (přednáška 8) by inferenci zlevnilo.

12.7 Syntéza

Zbývá zasadit tuto přednášku mezi ostatní. Tato závěrečná část ukáže trojí dělbu AI v kurzu — základy, hardware a nasazení — a rozhodne, kdy sáhnout po naučeném modelu a kdy stačí klasický DSP.

Trojí dělba AI v kurzu

Strojové učení jsme v kurzu potkali na třech úrovních a teprve dohromady dávají celý obraz. **Příloha B** je o *základech* — jak model vzniká (perceptron, backpropagation, trénink). **Přednáška 8** je o *hardwaru* — čím se inference počítá (NPU, systolická pole). A **tato přednáška** je o *nasazení* — jak model dostat na MCU (kvantizace, runtime, rozpočet). Tato přednáška je tak mostem mezi naučeným modelem (Příloha B) a hardwarem, který ho počítá (přednáška 8): kvantuje, převádí, vejde se do rozpočtu a spustí.

Kdy TinyML a kdy klasický DSP

Naučený model není vždy správná volba; často stačí a je lepší klasický přístup z přednášky 11. Pro jednoduchou, dobře pochopenou úlohu (detekce překročení, jednoduchá anomálie) je **klasický DSP nebo práh** deterministický, malý a laditelný. **TinyML** má smysl tam, kde je hranice mezi třídami tak složitá, že ji neumíme napsat (obraz, zvuk, komplexní pohyb), je dost dat na trénink a unese se model i runtime. Rozhodnutí je inženýrské, ne módní: nejprve zvolme příznaky a práh (přednáška 11), po naučeném modelu sáhněme, až když ruční hranice nestačí. TinyML řeší *složitost hranice*, ne každý problém.

> **Shrnutí oddílu 12.7.** ML jsme potkali na třech úrovních: **Příloha B** (základy — jak model vzniká), **přednáška 8** (hardware — čím se počítá), **přednáška 12** (nasazení — jak ho dostat na MCU). Tato přednáška je most mezi nimi. Volba mezi **klasickým DSP** (jednoduchá hranice, deterministické) a **TinyML** (složitá hranice, dost dat) je inženýrská: nejdřív příznaky a práh, naučený model až když ruční hranice nestačí.

12.8 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 12.1 — Proč ML na embedded

1. Co je a co není TinyML?
2. Jaké výhody má edge inference proti cloudu a za jakou cenu?
3. Na jakých třech pilířích TinyML stojí a proč je každý z nich nutný?
4. Jaký je společný vstup TinyML aplikací a jakým mechanismem se přenáší ze senzoru do paměti bez zátěže jádra?
5. V čem se liší ruční klasifikátor (příznaky RMS/FFT porovnané s prahem) od naučeného modelu a kdy který selhává?

K oddílu 12.2 — Inference versus trénink

1. Proč je MAC dominantní operací neuronové sítě a jak to zdůvodňuje využití fixed-point/SIMD instrukcí a existenci NPU?
2. Proč ReLU dominuje a kam se dávají drahé aktivace?
3. Jmenujte čtyři typy vrstev a řekněte, na co je která drahá (parametry, výpočet).
4. Co jsou sdílené váhy v konvoluci a proč tolik šetří parametry?
5. Vyjmenujte rozdíly mezi tréninkem a inferencí (místo, váhy, směr, dávka, přesnost, paměť) a proč na MCU běží jen dopředný průchod.

K oddílu 12.3 — Workflow TinyML

1. Jaké jsou tři fáze workflow TinyML (od dat k firmwaru) a která z nich běží na MCU?
2. Co je jediným výstupem tréninku pro embedded?
3. Jaká je dělba práce mezi TFLite, TFLite Micro a CMSIS-NN?
4. Jak se liší běh na jádře a na NPU z pohledu runtime?
5. Co v nasazení modelu automatizují nástroje výrobce (např. STM32Cube.AI)?

K oddílu 12.4 — Kvantizace

1. Jaké výhody přináší int8 proti float32 a za jakou cenu; proč síti kvantizace nevadí, kdežto DSP filtru ano?
2. Co je scale a zero-point a kdy volíte symetrickou a kdy asymetrickou kvantizaci?
3. Proč se v kvantovaném MAC akumuluje do int32 a jak requantizace vrátí výsledek na int8 bez dělení?
4. V čem se liší per-tensor a per-channel a proč se aktivace musí kalibrovat?
5. Co dělá pruning a co layer fusion při optimalizaci modelu na PC?
6. Srovnejte PTQ a QAT a řekněte, kdy sáhnout po QAT.

K oddílu 12.5 — Inference na MCU

1. Proč patří váhy do Flash a aktivace do RAM?
2. Co je tensor arena a proč je její velikost špička, ne součet aktivací?
3. Proč se tensor arena alokuje staticky a jednou, a proč se za běhu inference vyhýbáme dynamické alokaci (malloc)?
4. Co jsou operátory a jak profilováním najdete úzké hrdlo?
5. Proč je inference deterministická a co z toho plyne pro RTOS a real-time (přednášky 9 a 11)?

K oddílu 12.6 — Případové studie

1. Jaký zpracovatelský řetěz (senzor → příznaky → model → rozhodnutí) sdílí detekce klíčových slov, prediktivní údržba a detekce osoby, a co se mezi nimi liší?
2. Jaké příznaky (log-mel spektrum) používá detekce klíčových slov a proč se pro CNN volí log-mel místo MFCC?
3. Proč detekce anomálií používá autoencoder a jak práh (průměr + 3σ) rozhodne o anomálii bez příkladů poruch?
4. Popište řetěz detekce osoby a roli DCMI, DMA a dvojitého bufferu.
5. Z čeho se skládá inverted residual blok, proč má méně parametrů a jaký kompromis by změnilo NPU?
6. Kdy zvolíte klasický DSP (příznaky + práh) a kdy naučený model (TinyML)? Uveďte rozhodovací kritérium.

13

Bezpečnost embedded zařízení

> Tato kapitola řeší, **jak lze vestavné zařízení napadnout a jak jej systematicky ubránit**, když běží bez dohledu, s fyzicky dostupnými rozhraními a čipem v ruce útočníka. Embedded bezpečnost se od IT liší třemi omezeními: útočník má **fyzický přístup**, chybí ochrany OS (žádné ASLR ani izolace procesů) a zařízení běží **roky bez záplat**.

Obsah kapitoly

13.1 Bezpečnostní model a attack surface
13.2 Anatomie útoku
13.3 Paměťová izolace: MPU
13.4 ROP a obrana v kódu
13.5 Chain of trust a izolace
13.6 Hardwarové útoky
13.7 Syntéza
13.8 Kontrolní otázky

13.1 Bezpečnostní model a attack surface

Zabezpečit mikrokontrolér je těžší než server, protože chybí většina obranných vrstev. Tato část vysvětlí, čím je embedded jiné, zavede bezpečnostní cíle a model hrozeb a ukáže, co tvoří attack surface — a proč je jeho minimalizace první obranou.

Proč je embedded jiné: cíle a model hrozeb

Zabezpečit mikrokontrolér je v mnoha ohledech těžší než server, protože chybí většina obranných vrstev, na které je klasická IT bezpečnost zvyklá. Útočník má **fyzický přístup** — zařízení drží v ruce a dosáhne na debug port, sběrnice i napájení. Adresy jsou **deterministické**, protože bez operačního systému nikdo neprovádí randomizaci paměti; paměťový model je **plochý**, bez izolace procesů jednotkou MMU. Zařízení běží **roky bez záplat**, takže nalezená zranitelnost zůstává zneužitelná po celou dobu životnosti. A zdrojů na obranu je málo — kryptografie a monitoring stojí paměť, výkon a energii (přednáška 10).

Bezpečnostní cíle a model hrozeb

Bezpečnost není jeden požadavek, ale tři odlišné cíle (model CIA): **důvěrnost** (utajení firmwaru, klíčů a dat), **integrita** (neměnnost kódu a dat) a **dostupnost** (zařízení plní svůj účel). Konkrétní produkt váží jinak — u platebního terminálu dominuje důvěrnost a integrita, u průmyslového regulátoru integrita a dostupnost. Racionální návrh obrany začíná

modelem hrozeb: explicitním popisem toho, proti komu a čemu se bráníme. Útočníky dělíme podle schopností a přístupu na **vzdálené** (jen přes rozhraní), **lokální** (fyzicky u zařízení, neinvazivně), **fyzické** (manipulace hardwaru) a **invazivní** (dekapsulace čipu); cena i účinnost útoku roste v tomto pořadí. Obrana je vždy ekonomická: cílem není absolutní bezpečnost, která neexistuje, ale **zvýšit cenu útoku nad hodnotu chráněného aktiva**.

Modelem hrozeb
Explicitní popis toho, proti komu a čemu se zařízení brání: jaká aktiva chrání, jaké schopnosti má útočník a jaké cesty útoku připadají v úvahu. Bez modelu hrozeb se buď přepálí náklady na obranu, nebo se přehlédne reálná cesta útoku.

Attack surface

Množina všech vstupních bodů, které lze napadnout, tvoří **attack surface**. U embedded zařízení má tři vrstvy: **softwarovou** (přetečení bufferu, injection příkazů, chyby parserů, bypass bootloaderu, downgrade firmwaru), **hardwarovou** (debug port JTAG/SWD, dump externí Flash a EEPROM, glitching napájení a hodin, side-channel) a **sběrnicovou a síťovou** (odposlech I²C a SPI, útoky na protokoly CAN, Modbus, MQTT, MITM při aktualizaci). Každé rozhraní přidané v kurzu kvůli funkci (UART, SPI, I²C, DMA, debug) je zároveň vstupním bodem attack surface. Nástroje útoku jsou přitom levné a dostupné — logic analyzer, debugger s OpenOCD, Ghidra na zpětný překlad, binwalk, ropper — takže úroveň obrany běžného produktu jim musí odpovídat.

Attack surface
Souhrn všech míst, kterými lze do systému proniknout: rozhraní, sběrnice, debug port, mechanismus aktualizace. Čím menší je attack surface, tím méně cest útoku; vypnutí nepoužívaných rozhraní je proto první a nejlevnější obrana.

> **Shrnutí oddílu 13.1.** Embedded bezpečnost je omezena **fyzickým přístupem**, **absencí ASLR a izolace procesů** a **absencí záplat**. Cíle jsou tři (**důvěrnost, integrita, dostupnost**). **Model hrozeb** (vzdálený → lokální → fyzický → invazivní útočník) určuje, proti čemu se bránit; obrana je **ekonomická**. **Attack surface** má vrstvy software, hardware a sběrnice/síť; jeho minimalizace je první obrana.

13.2 Anatomie útoku

Nejlépe se obrana pochopí na konkrétním útoku. Tato část rozebere přetečení zásobníkového bufferu krok za krokem — od zranitelné funkce přes přepis návratové adresy až ke spuštění vlastního kódu — a ukáže, proč to na embedded funguje.

Přetečení bufferu: od zranitelné funkce ke spuštění kódu

Nejrozšířenější třídou zranitelností je **přetečení bufferu** na zásobníku. Ukažme ji na parseru příkazu volaném z obsluhy UARTu:

Přetečení bufferu
Zápis za konec pole, který přepíše sousední data. U zásobníkového bufferu zápis roste k vyšším adresám a přepíše uloženou návratovou adresu funkce; útočník tím může přesměrovat tok programu na vlastní kód.

```
void process_command(const char *input) {
    char cmd[64];           // lokální buffer — leží na zásobníku
    strcpy(cmd, input);     // kopíruje bez ohledu na velikost cíle
}
```

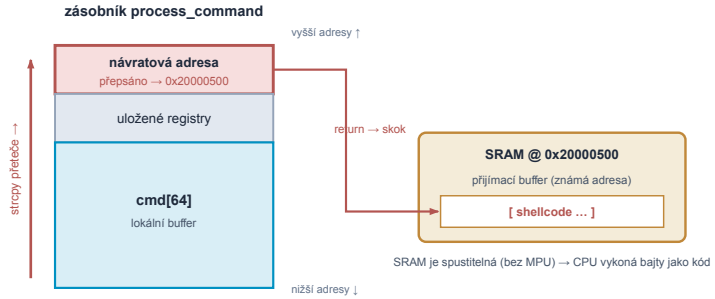
Funkce `strcpy` kopíruje vstup, dokud nenarazí na ukončovací nulu, bez ohledu na velikost pole `cmd`. Když se `process_command` zavolá, její prolog uloží na zásobník návratovou adresu (registr `LR`) a použité registry; lokální buffer leží pod nimi, na nižších adresách (přednáška 3). Zápis do pole roste směrem k **vyšším** adresám — tedy k uložené návratové adrese, jejíž pozice je na pevném offsetu nad bufferem. Toto přesně zakazuje MISRA: žádné `strcpy`, `gets` ani `sprintf` bez omezení délky; každý zápis do pole musí mít kontrolu mezí.

Od přetečení ke spuštění kódu

Legitimní krátký vstup se do bufferu vejde a návratová adresa zůstane nedotčená. Pošle-li útočník vstup delší než 64 bajtů, `strcpy` přepíše nejprve uložené registry a nakonec návratovou adresu. Naivní přetečení (samé znaky „A”, tedy návratová adresa `0x41414141`) vede jen k **HardFaultu**: procesor skočí na neplatnou adresu a jeho běh se zhroutí, což lze zpětně ověřit z registru `CFSR` (přednáška 3). To je narušení dostupnosti, ne převzetí kontroly.

Skutečný útok přepíše návratovou adresu na **platnou** adresu, kde leží **shellcode** — útočníkův strojový kód. Přijatý řádek přitom zůstává uložený v přijímacím bufferu na známé, pevné adrese v SRAM.

Shellcode
Posloupnost bajtů, kterou procesor při vykonávání dekóduje jako strojové instrukce. Procesor nerozlišuje kód a data; skočí-li ukazatel instrukcí do dat útočníka, vykoná je jako program. Název pochází z původního účelu spustit systémový shell.



Obrázek 13.1. Přetečení zásobníku. Zápis do lokálního bufferu roste k vyšším adresám a přepíše uloženou návratovou adresu; přepsaná adresa ukazuje do přijímacího bufferu v SRAM, kde leží shellcode. Na konci funkce procesor načte tuto adresu do ukazatele instrukcí a začne vykonávat shellcode.

Procesor při vykonávání provádí cyklus fetch–decode–execute nad tím, kam ukazuje ukazatel instrukcí, a **nerozlišuje kód a data**. Tytéž bajty jsou proto jednou „přijatá data" a podruhé „program", takže přepis návratové adresy stačí ke spuštění libovolného kódu.

Proč to na embedded funguje

Útok stojí na dvou podmínkách. **Zaprvé** musí být SRAM spustitelná: bez MPU je celý adresní prostor spustitelný, protože datové oblasti nemají atribut execute-never. Na plochém paměťovém modelu jsou data i kód nerozlišitelné. **Zadruhé** musí útočník znát cílovou adresu. Na PC to znesnadňuje **ASLR**, který při každém spuštění náhodně posune umístění paměti. Na embedded ASLR chybí — bez operačního systému nemá kdo randomizaci provést a linker script umístění pevně určí (přednáška 4). Útočník tedy zná adresu SRAM přesně; není-li si jistý na bajt, použije **NOP sled** — pole instrukcí „nedělej nic" před shellcode, kterým skok na libovolné místo pokračuje až k užitečnému kódu. Offset návratové adresy zjistí i naslepo fuzzingem (posílá vstupy rostoucí délky a sleduje pád) nebo Metasploit patternem (neopakující se sekvence, z níž se offset odečte přímo); tyto techniky payload jen zpřesňují, princip zůstává stejný.

ASLR
Address Space Layout Randomization: ochrana, která při každém spuštění náhodně posune umístění zásobníku, haldy a knihoven, takže útočník nezná cílovou adresu skoku. Na embedded bez operačního systému chybí — linker script umístění paměti pevně určí a adresy jsou při každém spuštění totožné.

> **Shrnutí oddílu 13.2. Přetečení zásobníkového bufferu** přepíše uloženou **návratovou adresu**, protože zápis do pole roste k vyšším adresám. Naivní přetečení vede k **HardFaultu**; řízený útok ji přepíše na platnou adresu a přesměruje tok na **shellcode** (data dekódovaná jako instrukce). Funguje to díky **spustitelné SRAM** a **absenci ASLR**; offset lze zjistit fuzzingem či Metasploit patternem.

13.3 Paměťová izolace: MPU

První hardwarovou obranou proti takovému útoku je jednotka ochrany paměti. Tato část ukáže, jak MPU rozdělí paměť na regiony s právy a spustitelností, jak z toho plyne politika W^X a které další ochrany jediná jednotka poskytuje.

MPU: princip, registry a W^X

MPU (Memory Protection Unit, přednáška 4) rozděluje adresní prostor na **regiony** a u každého přístupu kontroluje, zda je povolen; při porušení vyvolá výjimku MemManage fault. Je region-based: velikost je mocnina dvou (32 B až 4 GB) a báze musí být na velikost zarovnaná; regiony se smějí překrývat a při překryvu platí atributy regionu s vyšším číslem; MPU rozlišuje privilegovaný a neprivilegovaný přístup. Konfiguruje se pěti registry (`TYPE`, `CTRL`, `RNR`, `RBAR`, `RASR`); globální zapnutí je nutné dokončit bariérami `DSB` a `ISB` (přednáška 4). Celé chování regionu určuje registr `RASR`:

MPU
Memory Protection Unit: volitelná jednotka jader Cortex-M3/M4/M7, která rozděluje adresní prostor na regiony s vlastními přístupovými právy, spustitelností a atributy paměti. Při porušení pravidel vyvolá výjimku MemManage fault; jádra M0 a M0+ ji nemají.



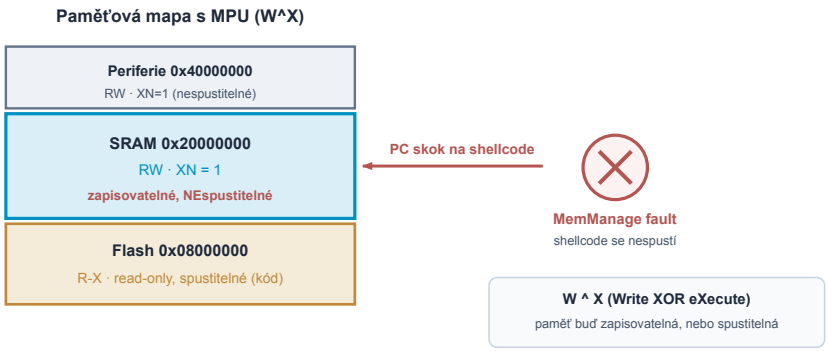
Obrázek 13.2. Registr RASR: pole XN (execute-never) řídí spustitelnost, AP přístupová práva, TEX/C/B/S typ paměti (cacheable, bufferable, shareable), SRD masku osmi podregionů, SIZE velikost jako dvě na SIZE plus jedna a ENABLE aktivaci regionu.

Přístupová práva a execute-never

Pole `AP` definuje práva čtení a zápisu odděleně pro privilegovaný a neprivilegovaný režim: od zakázané oblasti (`000`) přes „jen jádro” (`001`, RW privileged, žádný přístup unprivileged) a „jádro zapisuje, aplikace čte” (`010`) po plný přístup (`011`) a read-only pro všechny (`110`). Kombinace `AP` a bitu `XN` je základ obrany: kód je read-only a spustitelný, data jsou zapisovatelná a nespustitelná, klíče jsou přístupné jen privilegovaně a nespustitelné.

Přímou odpovědí na útok z oddílu 13.2 je označit SRAM a zásobník jako nespustitelné (`XN = 1`). Skok ukazatele instrukcí do datové oblasti pak vyvolá MemManage fault a shellcode se nevykoná. Této politice se říká **Write XOR eXecute** (W ^ X):

Write XOR eXecute
Zkráceně W-X, též DEP (Data Execution Prevention): pravidlo, že žádná paměťová oblast není zároveň zapisovatelná a spustitelná. Kód je read-only a spustitelný, data jsou zapisovatelná a nespustitelná. Znemožňuje spuštění vloženého kódu, například shellcode v datovém bufferu.



Obrázek 13.3. Politika W ^ X. Flash je read-only a spustitelná (obsahuje kód), SRAM je zapisovatelná a nespustitelná (XN rovno jedné). Skok ukazatele instrukcí na shellcode v SRAM vyvolá MemManage fault, takže se shellcode nespustí.

Atributy paměti, velikost, podregiony

Pole `TEX`, `C`, `B` a `S` určují **typ paměti** — jak s ní pracuje cache a sběrnice (přednáška 4 a 5): od strongly-ordered (kritické registry periferií) přes device (periferie) po normal write-back (běžná SRAM). Zásadní pro datový tok je non-cacheable normal pro **DMA buffer**, který řeší problém koherence z přednášky 5. Velikost regionu kóduje pole `SIZE` jako dvě na SIZE plus jedna a báze musí být zarovnaná na tuto velikost — chráněná struktura proto musí být zarovnaná a velká jako mocnina dvou, což ovlivňuje už rozvržení paměti v době překladu. Region lze rozdělit na osm podregionů a jednotlivě je zakázat maskou `SRD`, což dovolí jemnější granularitu bez spotřebování dalšího z omezeného počtu regionů.

Praktické vzory a obsluha faultu

MPU poskytuje několik nezávislých ochran z jediné jednotky. **Read-only spustitelná Flash** brání přepsání kódu a vektorové tabulky. **Nespustitelná SRAM** realizuje W ^ X. **Guard page** — úzký region bez přístupu těsně pod dnem zásobníku — promění přetečení zásobníku na okamžitý, lokalizovaný fault místo tichého přepsání sousedních dat. **Region s klíči** přístupný jen privilegovaně je aplikace v neprivilegovaném režimu vůbec nedostane. Porušení pravidel vyvolá `MemManage_Handler`, kde příčinu i adresu odhalí registry `CFSR` a `MMFAR` (přednáška 3):

```
void MemManage_Handler(void) {
    uint32_t cfsr = SCB->CFSR;
    if (cfsr & SCB_CFSR_MMARVALID_Msk) log("MPU fault @ 0x%08lx", SCB->MMFAR);
    if (cfsr & SCB_CFSR_IACCVIOL_Msk)  log("pokus o spuštění nespustitelné paměti");
    SCB->CFSR = cfsr;
    system_secure_reset();     // v produkci řízený reset, ve vývoji breakpoint
}
```

Ruční skládání `RASR` je náchylné k chybám; CMSIS proto poskytuje makra `ARM_MPU_RBAR` a `ARM_MPU_RASR` a funkci `ARM_MPU_SetRegion`. V RTOS (přednáška 9) plánovač přepíná MPU regiony při každém přepnutí úlohy, takže každá úloha vidí jen svou paměť — MPU tak dává **izolaci úloh** a pád jedné úlohy neohrozí ostatní ani jádro.

> **Shrnutí oddílu 13.3. MPU** dělí paměť na **regiony** (`RBAR`/`RASR`) s právy `AP`, spustitelností `XN` a atributy `TEX/C/B/S`; porušení je **MemManage fault** (`CFSR`/`MMFAR`). **W ^ X** (nespustitelná SRAM) je přímá obrana proti shellcode. MPU dále poskytuje **guard page** proti přetečení zásobníku, **privileged-only** region na klíče a **non-cacheable** DMA buffer (přednáška 5); v RTOS dává **izolaci úloh** (přednáška 9).

13.4 ROP a obrana v kódu

Politiku W ^ X ovšem útočník obejde, aniž by spouštěl nový kód. Tato část ukáže techniku ROP, která řetězí existující úseky kódu, softwarovou obranu stack canary a nejlevnější obranu ze všech — prevenci u zdroje.

ROP
Return-Oriented Programming: technika, která obchází politiku W-X tím, že nespouští nový kód, ale řetězí krátké úseky již existujícího spustitelného kódu (gadgety) končící návratem. Přetečením se na zásobník uloží posloupnost adres gadgetů, z nichž se složí libovolný výpočet.

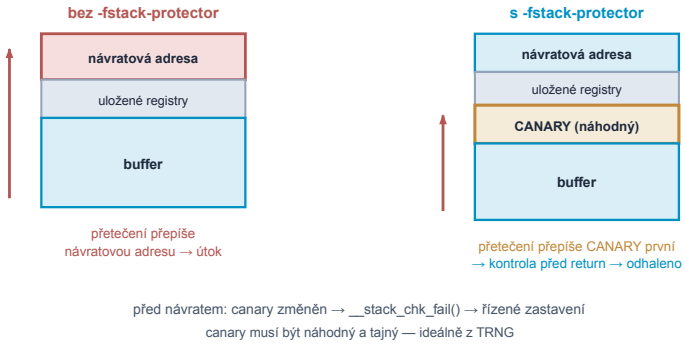
Return-Oriented Programming

Politika W^X zabrání spuštění vlastního kódu ze zásobníku, ale útočník ji obejde technikou **ROP** (Return-Oriented Programming): nespouští nový kód, ale řetězí krátké úseky již existujícího a tedy spustitelného kódu ve Flash. Stavebním kamenem je **gadget** — krátká sekvence instrukcí končící návratem (například `pop {r0}; bx lr`). Přetečením se na zásobník uloží posloupnost adres gadgetů; každý provede malý krok a návratem předá řízení dalšímu, takže z „střípků" složí útočník libovolný výpočet, aniž by potřeboval spustitelná data. Gadgety vyhledá nástroj ropper ve staženém firmwaru. Typickým cílem řetězu je zapsat nulu do `MPU->CTRL` a tím vypnout ochranu z oddílu 13.3, po čemž už W^X neplatí. ROP ukazuje zásadní pravdu bezpečnosti: **jedna vrstva nikdy nestačí** — a citlivé operace jako vypnutí MPU by měly být samy chráněné (privileged, TrustZone).

Stack canary a prevence u zdroje

Druhou, softwarovou vrstvou je **stack canary**. Kompilátor s přepínačem `-fstack-protector` vloží mezi buffer a uloženou návratovou adresu náhodnou hodnotu a před návratem funkce ověří, že se nezměnila:

Stack canary
Náhodná strážní hodnota, kterou kompilátor s přepínačem stack-protector vloží mezi lokální buffer a uloženou návratovou adresu. Před návratem funkce ověří, že se hodnota nezměnila; spojité přetečení bufferu ji přepíše dříve než návratovou adresu, takže se odhalí.



Obrázek 13.4. Stack canary. Bez ochrany přetečení přepíše přímo návratovou adresu. S ochranou leží mezi bufferem a návratovou adresou náhodná strážní hodnota, kterou spojité přetečení přepíše první; kontrola v epilogu změnu odhalí a program řízeně zastaví.

Přetečení směřující k návratové adrese přepíše nejdřív canary a kontrola v epilogu to odhalí dříve, než se přepsaná návratová adresa použije. Ochrana stojí a padá s **nepředvídatelností** canary — konstantní hodnota je bezcenná, protože ji útočník do payloadu dopíše. Proto se strážní hodnota inicializuje při startu z **hardwarového generátoru náhodných čísel (TRNG)**, takže je při každém startu jiná. Přepínač má tři úrovně: jen rizikové funkce, širší výběr (`-strong`) a všechny funkce (`-all`). Canary chrání proti spojitému přetečení bufferu, které je zdaleka nejčastější; nechrání proti zápisu na konkrétní adresu přes přepsaný ukazatel.

Prevence u zdroje

MPU a canary zachycují následek přetečení; nejlevnější obrana je nenechat ho vzniknout, což je disciplína z přednášky 3: kontrola mezí u každého indexu a zápisu do pole, funkce s omezením délky (`strncpy`, `snprintf`, `memcpy` se známou velikostí) místo `strcpy`, `gets` a `sprintf`, validace délky a formátu vstupu ze sběrnice **před** zpracováním, a striktní oddělení dat od řízení (vstup ze sítě se nikdy neinterpretuje jako příkaz bez kontroly). Chyby odhalí nástroje ještě před nasazením — **statická analýza** (kontrola MISRA z přednášky 3) najde chybějící kontroly mezí v době překladu, **fuzzingem** se systematicky posílají zdeformované vstupy a detekují pády, a **sanitizery** při testech odhalí latentní přetečení. Ideál je najít zranitelnost dřív než útočník; statická analýza a fuzzing v kontinuální integraci posouvají nález

z „po incidentu v terénu” na „při každém commitu”.

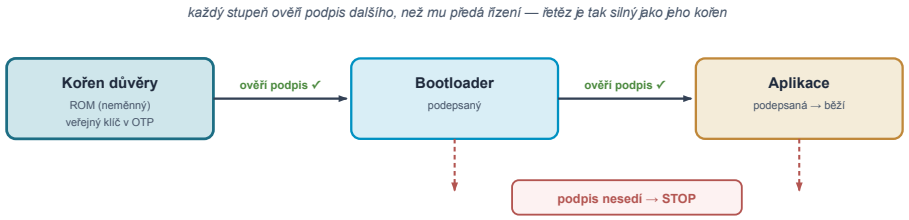
> **Shrnutí oddílu 13.4. ROP** obchází W^X řetězením **gadgetů** (existujících úseků kódu); jedna vrstva nestačí. **Stack canary** vloží před návratovou adresu náhodnou hodnotu a v epilogu ověří její neporušenost; musí být **náhodný z TRNG**. Nejlevnější obrana je **prevence u zdroje** — kontrola mezí, funkce s limitem, validace vstupu (MISRA) — a **statická analýza s fuzzingem** ve vývojovém procesu.

13.5 Chain of trust a izolace

Předchozí vrstvy chránily běžící program; zbývá zaručit integritu firmwaru samotného. Tato část vysvětlí secure boot a řetěz důvěry, hardwarové kryptoakcelerátory a ochranu čtení a nejsilnější izolaci klíčů přes TrustZone.

Secure boot a řetěz důvěry

Předchozí vrstvy chránily běžící program; zbývá otázka integrity firmwaru. Nahradí-li útočník obraz vlastním, všechny softwarové obrany běží v jeho režii. Odpovědí je **secure boot**, který při startu ověří **digitální podpis** firmwaru. Každý stupeň bootu ověří ten následující a vzniká **řetěz důvěry**:



Obrázek 13.5. Řetěz důvěry secure boot. Neměnný kořen v paměti ROM s veřejným klíčem v OTP ověří podpis bootloaderu, ten ověří podpis aplikace, a teprve pak se aplikace spustí. Nesedí-li podpis, zařízení se zastaví.

Kořen důvěry musí být v **read-only ROM** (nelze přepsat) a veřejný klíč v **OTP** (jednorázově programovatelná paměť); kdyby šel kořen změnit, celý řetěz padá. Ověření stojí na dvou kryptografických primitivech: **hašovací funkce** (SHA-256) spočítá z obrazu krátký jedinečný otisk, a **asymetrický podpis** (RSA, ECDSA) tento otisk zaručí. Výrobce podepíše otisk **privátním** klíčem, který nikdy neopustí jeho zázemí; zařízení ověří podpis **veřejným** klíčem v OTP. Zásadní je asymetrie: veřejným klíčem lze podpis jen ověřit, ne vytvořit, takže i když ho útočník přečte, nemůže podepsat vlastní firmware. Bezpečnost celého schématu se redukuje na utajení jednoho privátního klíče.

Kryptoakcelerátory, RDP a izolace klíčů

Kryptografie je výpočetně náročná; moderní MCU proto mají hardwarové akcelerátory (tentýž princip specializace jako v přednášce 8): **AES** pro symetrické šifrování, **PKA** pro asymetrické operace, **HASH** pro SHA-256 a **TRNG** jako zdroj entropie z fyzikálního šumu (klíče, nonce, stack canary z oddílu 13.4). I podepsaný firmware lze stáhnout přes debug port a analyzovat; proti tomu je **RDP** (Readout Protection). Nastavuje se v Option Bytes do tří úrovní: Level 0 (žádná ochrana, Flash čitelná), Level 1 (debug nesmí číst Flash a návrat na Level 0 spustí smazání Flash) a Level 2 (debug trvale a nevratně zakázán). Level 1 je obvyklá volba; Level 2 je nejpřísnější, ale znemožní i servis. RDP zásadně zvyšuje náročnost útoku, ale není absolutní — lze ji na některých čipech obejít glitchingem (oddíl 13.6), takže citlivé klíče se ukládají do

Secure boot
Mechanismus, který při každém startu ověří digitální podpis firmwaru, než mu předá řízení. Každý stupeň bootu ověří podpis následujícího, čímž vzniká řetěz důvěry ukotvený v neměnném kořeni; cizí nebo změněný firmware se nespustí.

Řetěz důvěry
Chain of trust: posloupnost ověření, kdy neměnný kořen v paměti ROM ověří podpis bootloaderu, ten ověří podpis aplikace, a teprve pak se aplikace spustí. Řetěz je tak silný jako jeho kořen, který musí být v read-only paměti s klíčem v jednorázově programovatelné paměti.

TRNG
True Random Number Generator: hardwarový generátor náhodných čísel odvozující entropii z fyzikálního šumu. Poskytuje kryptografické klíče, jednorázové nonce a stack canary; softwarový pseudonáhodný generátor je pro kryptografii nedostatečný.

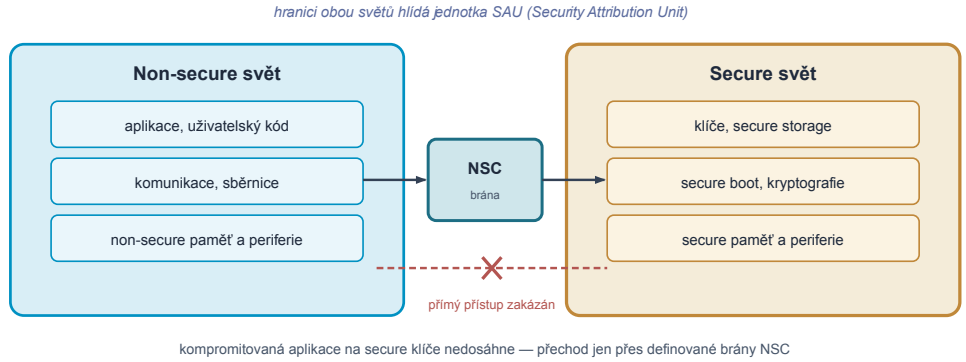
RDP
Readout Protection: ochrana STM32 proti přečtení Flash přes debug port, konfigurovaná v Option Bytes do tří úrovní. Level 1 zakáže čtení a při snížení ochrany spustí smazání Flash; Level 2 debug trvale a nevratně zakáže.

secure elementu nebo secure světa TrustZone.

Podepsané aktualizace a izolace

Zařízení se aktualizuje po síti a aktualizace je útočný vektor. Bezpečný update ověří **podpis** obrazu (proti podvržení), **verzi** (nová musí být vyšší než stávající, proti **downgrade** útoku vnucujícímu starou zranitelnou verzi) a **integritu přenosu** šifrovaným kanálem (proti MITM). Nejsilnější izolaci dává TrustZone-M:

TrustZone-M
TrustZone for Armv8-M: hardwarové rozdělení jádra Cortex-M33/M55 na secure a non-secure svět s oddělenou pamětí a periferiemi. Přechod je možný jen přes definované brány (Non-Secure Callable), hranici vynucuje jednotka SAU; kompromitovaná aplikace na secure klíče nedosáhne.



Obrázek 13.6. TrustZone-M dělí čip na non-secure svět (aplikace, komunikace) a secure svět (klíče, secure boot, kryptografie). Přechod je možný jen přes brány NSC, hranici vynucuje jednotka SAU; přímý přístup non-secure kódu do secure světa je zakázán.

I když je aplikace v non-secure světě kompromitovaná, na klíče a secure kód nedosáhne — TrustZone je silnější než MPU, protože nejde jen o práva k paměti, ale o dva oddělené procesorové stavy. Pro nejvyšší nároky se klíče přesouvají do vyhrazeného **secure elementu** (ATECC608, TPM), který kryptooperaci provede interně a klíč z něj fyzicky nevyjde — na rozdíl od klíčů v běžné Flash, které dump paměti prozradí.

> **Shrnutí oddílu 13.5. Secure boot** = **řetěz důvěry**: kořen v ROM ověří podpis bootloaderu a aplikace. Stojí na **hašování** a **asymetrickém podpisu** (privátní klíč jen u výrobce). **Kryptoakcelerátory** a **TRNG** jsou hardwarové bloky. **RDP** chrání firmware před přečtením (lze obejít glitchingem). **Podepsané OTA** brání podvržení, downgrade i MITM; **TrustZone-M** a **secure element** izolují klíče mimo dosah aplikace.

13.6 Hardwarové útoky

S fyzickým přístupem se otevírá druhá polovina attack surface. Tato část projde nejrychlejší cestu dovnitř — debug port — i odposlech sběrnic a externí paměti a nejsofistikovanější útoky pod digitální vrstvou: glitching a postranní kanály.

Debug port, sběrnice a externí paměť

Softwarové obrany platí jen tehdy, pokud útočník nemůže obejít procesor zdola; s fyzickým přístupem se otevírá druhá polovina attack surface. Nejrychlejší cestou dovnitř je **debug port** JTAG/SWD (přednáška 3): rozhraní SWD stačí dva vodiče a poskytuje plnou kontrolu — čtení a zápis SRAM, čtení Flash (pokud RDP dovolí) a nahrání vlastního kódu (nástrojem OpenOCD příkazy `halt`, `mdw`, `flash read_bank`). Ponechaný otevřený debug port je nejčastější zranitelnost produkčních zařízení a obrana je triviální a povinná: **zamknout debug** pomocí RDP Level 1 nebo 2 (oddíl 13.5). Sběrnice mezi čipy na desce (přednáška 6) lze odposlechnout logic analyzerem, protože data po nich jdou většinou nešifrovaně; externí Flash a EEPROM

lze vyčíst i mimo desku připojením programátoru. Platí pravidlo: co jde po sběrnici v otevřené podobě, je čitelné. Citlivá data mezi čipy musí být šifrovaná a jejich integrita ověřená, klíče patří do secure elementu.

Glitching a side-channel

Nejsofistikovanější útoky míří pod digitální vrstvu. **Fault injection** (glitching) krátkou přesně načasovanou poruchou napájení nebo hodin přinutí procesor přeskočit instrukci — například kontrolu podpisu (obejde secure boot) nebo kontrolu RDP (odemkne debug). Je překvapivě dostupný a účinný, protože nemění kód, ale jeho vykonání; obranou jsou redundantní kontroly, náhodné časování a hardwarové detektory napětí a hodin (BOR a PVD z přednášky 10 jsou první úrovní). **Side-channel** útoky měří fyzikální projevy výpočtu: spotřebu (power analysis), dobu trvání (timing) nebo elektromagnetické vyzařování, a statistickou analýzou mnoha měření rekonstruují klíč. Obranou je konstantní-časová implementace nezávislá na datech, maskování a vyrovnávání spotřeby; proto se kryptografie nepíše ručně, ale používají se ověřené knihovny a certifikované secure elementy.

Fault injection
Glitching: útok, který krátkou přesně načasovanou poruchou napájení, hodin nebo elektromagnetickým pulzem přinutí procesor udělat chybu, typicky přeskočit instrukci. Přeskočená kontrola podpisu nebo RDP obejde i korektní software, protože se nemění kód, ale jeho vykonání.

Side-channel
Únik postranním kanálem: útok, který z fyzikálních projevů výpočtu (spotřeba, doba trvání, elektromagnetické vyzařování) odvodí tajemství, například kryptografický klíč. Bráníme se konstantní-časovou implementací nezávislou na datech a maskováním mezivýpočtů.

Proti fyzickému útočníkovi s vybavením neexistuje absolutní ochrana; cílem je zvýšit cenu útoku nad hodnotu aktiva. U vysokých nároků (platby, identita) se odolnost certifikuje (Common Criteria) nezávislou laboratoří; pro běžný produkt stačí zamčený debug, šifrovaná externí data a podepsaný firmware, což odradí naprostou většinu útočníků.

> **Shrnutí oddílu 13.6. Fyzický přístup** otevírá druhou polovinu attack surface. **Debug port** je nejrychlejší cesta dovnitř — povinně zamknout RDP. **Sběrnice a externí paměť** jsou čitelné → data **šifrovat**, klíče do **secure elementu**. **Glitching** přeskočí instrukci (obejde secure boot); **side-channel** prozradí klíč ze spotřeby či času. Obrana je redundance, detektory a konstantní-časová kryptografie; absolutní ochrana neexistuje.

13.7 Syntéza

Žádná jediná obrana nestačí. Tato závěrečná část ukáže bezpečnost jako obranu do hloubky — vrstvy, které násobí cenu útoku — a jako proces, jímž se uzavírá oblouk celého předmětu.

Obrana do hloubky

Žádná jediná obrana nestačí — bezpečnost je **vrstvení nezávislých ochran (defense in depth)**, kde útočník musí prorazit všechny, zatímco obránci stačí, aby držela jediná. Vrstvy jdou od kódu (MISRA, kontrola mezí — brání vzniku přetečení) přes kompilátor (stack canary — přepisu návratové adresy), procesor (MPU s W^X a izolací — spuštění shellcode a přístupu ke klíčům), boot (secure boot a podpis — cizímu a starému firmwaru), konfiguraci (RDP a zamčený debug — přečtení firmwaru) až po hardware (TrustZone a secure element — krádeži klíčů a fyzickému útoku). Vrstvy se nesčítají, ale **násobí** cenu útoku: přetečení, které projde přes chybějící kontrolu mezí, zastaví canary; co obejde canary, zastaví W^X; co obejde W^X přes ROP, ztíží secure boot a izolace klíčů.

Bezpečnost jako proces a uzavření kurzu

Bezpečnost není hotová položka, ale **návrhový postoj** provázející celý životní cyklus produktu — stejný inženýrský přístup jako u spotřeby (přednáška 10) nebo aritmetiky (přednáška 11). Vždy začíná **modelem hrozeb** (proti komu a čemu se bránit); pamatuje na **nejslabší článek** (otevřený debug znehodnotí i dokonalou kryptografii); váží **cenu proti hodnotě** (každá vrstva stojí zdroje, chráníme úměrně hodnotě aktiva); a počítá s **aktualizovatelností**, protože zranitelnosti se najdou až za provozu. Cílem není nedobytnost, ale posunout cenu útoku nad hodnotu aktiva a umět reagovat.

Přednáškou o bezpečnosti se uzavírá oblouk celého předmětu. Postupovali jsme od tranzistoru k systému a jeho ochraně: od hradla a klopného obvodu přes jádro, paměť a periferie po sběrnice, architektury a akcelerátory (hardware); od překladu C a firmwaru přes datový tok, RTOS a nízkou spotřebu (software); přes číslicové zpracování signálů, strojové učení a jeho nasazení (aplikace); a nakonec bezpečnost, která to vše brání. Vestavný systém není žádná z těchto vrstev samostatně, ale jejich souhra — a rozumět jí od tranzistoru po neuronovou síť a její ochranu byl cíl tohoto předmětu.

> **Shrnutí oddílu 13.7.** Bezpečnost je **obrana do hloubky**: nezávislé vrstvy (kód, canary, MPU, secure boot, RDP, TrustZone) **násobí** cenu útoku. Je to **proces** vycházející z modelu hrozeb, hlídající nejslabší článek, vážící cenu proti hodnotě a počítající s aktualizacemi. Uzavírá kurz od tranzistoru po systém a jeho ochranu.

13.8 Kontrolní otázky

Otázky jsou seřazeny podle oddílů kapitoly. Ověřte si na nich, že rozumíte jak jednotlivým mechanismům, tak jejich souvislostem.

K oddílu 13.1 — Bezpečnostní model a attack surface

1. Proč je bezpečnost vestavných zařízení těžší než serverová?
2. Co je model CIA a jak se váhy cílů liší podle typu zařízení?
3. K čemu slouží model hrozeb a jaké třídy útočníků rozlišujeme?
4. Co je attack surface a proč je jeho minimalizace nejlevnější obranou?
5. Proč je obrana vždy „ekonomická”?

K oddílu 13.2 — Anatomie útoku

1. Proč zápis do lokálního pole ohrozí návratovou adresu?
2. Čím se liší naivní přetečení (HardFault) od řízeného útoku?
3. Co je shellcode a proč funguje, když procesor nerozlišuje kód a data?
4. Aby přetečení zásobníkového bufferu vedlo ke spuštění vloženého shellcode, musí být splněny dvě podmínky. Které to jsou a proč jsou na embedded platformě obvykle obě splněné?
5. Jak útočník zjistí offset návratové adresy bez znalosti firmwaru?

K oddílu 13.3 — Paměťová izolace: MPU

1. Co je region MPU a jaká pravidla platí pro jeho velikost a překryv?
2. Vysvětlete pole XN a AP v konfiguračním registru MPU (RASR) a jak z nich plyne politika W ^ X.
3. Jak MPU detekuje přetečení zásobníku a chrání klíče?
4. Co odhalí registry CFSR a MMFAR při MemManage faultu?
5. Jak MPU zajišťuje izolaci úloh v RTOS?

K oddílu 13.4 — ROP a obrana v kódu

1. Jak ROP obchází politiku W^X a co je gadget?
2. Jak funguje stack canary a proč musí být náhodný?
3. Proti čemu canary nechrání?
4. Jak lze už při psaní kódu předejít vzniku přetečení zásobníku?
5. Jak statická analýza a fuzzing zapadají do vývojového procesu?

K oddílu 13.5 — Chain of trust a izolace

1. Popište řetěz důvěry a proč musí být kořen neměnný.
2. Jak funguje podpis firmwaru a proč stačí utajit jen privátní klíč?
3. Jak úrovně ochrany čtení (RDP) postupně zamykají čip a proti čemu chrání?
4. Co ověřuje bezpečná aktualizace a co je downgrade útok?
5. Čím se liší TrustZone od MPU a co přidává secure element?

K oddílu 13.6 — Hardwarové útoky

1. Co umožňuje otevřený debug port a jaká je obrana?
2. Proč je odposlech sběrnic a dump externí paměti nebezpečný a jak se bránit?
3. Jak funguje glitching a proč obejde i korektní software?
4. Co je side-channel útok a jak se proti němu bránit?
5. Lze proti fyzickému útočníkovi s vybavením dosáhnout absolutní ochrany? Co je tedy realistickým cílem obrany embedded zařízení?

Příloha A — Vlastnictví zdrojů a moderní C++ v embedded

> Tato příloha není samostatnou přednáškou, ale prohloubením jednoho motivu, který se kurzem opakovaně vracel spirálovitě: **vlastnictví zdroje**. V přednášce 3 šlo o ruční vlastnictví v jazyce C (opaque handle s `create`/`destroy`), v přednášce 5 o vlastnictví DMA bufferu mezi hardwarem, přerušením a aplikací a v přednášce 9 o vlastnictví zdroje mutexem a o přenosu vlastnictví dat frontou. Zde spirálu uzavřeme: ukážeme, jak totéž vlastnictví **vynutí typový systém** — v moderním C++ (RAII, přesun, výpůjčka, sdílení) a ještě důsledněji v jazyce Rust.

Otázka zní: **kdo zdroj vlastní, kdo jej smí používat a kdy se uvolní — a lze to nechat ověřovat překladač, místo abychom to kontrolovali sami?** V čistém C spočívá odpověď vždy na programátorovi a jeho kázni: u každého `malloc` musí někde být `free`, u každého `open` někde `close`, u každého zamčeného mutexu někde `unlock`. Jeden zapomenutý protějšek znamená únik paměti, dvojí uvolnění nebo uváznutí. Moderní C++ tuto disciplínu **přenáší z hlavy do typu**: vlastnictví se stane vlastností objektu, kterou kontroluje překladač a vynucuje běhové chování destruktoru. Půjdeme od základního mechanismu (RAII), přes tři režimy vlastnictví (unikátní, výpůjčka, sdílené) a pluggable alokátory (`std::pmr`) k jazyku Rust jako důslednému kontrastu a nakonec ke **skutečné ceně C++ v embedded**.

A.1 Od object-based C k C++

V přednášce 3 jsme viděli, že i v čistém C se dá vlastnictví modelovat: **opaque handle** skryje vnitřní stav a dvojice funkcí `uart_create` / `uart_destroy` (nebo `init` / `deinit`) vymezí život zdroje. Slabina je, že spárování je **konvence, ne pravidlo**: nic nebrání tomu zapomenout na `destroy`, zavolat ho dvakrát, nebo použít handle po jeho zničení. C++ tentýž vzor povýší na **třídu**: vnitřní stav jsou privátní členy, `create`/`destroy` se stanou **konstruktorem** a **destruktorem** a překladač zaručí, že se destruktor zavolá — právě jednou a ve správný okamžik. Tím se z ruční konvence stává jazykem vynucené pravidlo. Zbytek přílohy je o tom, co všechno z toho plyne.

A.2 RAII: životnost objektu je životnost zdroje

Jádrem moderního C++ je idiom **RAII**: zdroj se získá v konstruktoru a uvolní v destruktoru. Protože C++ volá destruktor **automaticky** na konci rozsahu platnosti (a to i tehdy, když se rozsah opouští přes výjimku), je uvolnění zdroje zaručené a **deterministické** — nastane přesně ve chvíli, kdy objekt zaniká, ne „někdy později" jako u garbage collectoru.

RAII
Resource Acquisition Is Initialization: zdroj se získá v konstruktoru objektu a uvolní v jeho destruktoru. Životnost zdroje je tak svázána s životností objektu a uvolnění je automatické a deterministické, včetně cesty přes výjimku.

```
class ScopedLock {
    Mutex& m_;
public:
    explicit ScopedLock(Mutex& m) : m_(m) { m_.lock();   }   // získání zdroje
    ~ScopedLock()                         { m_.unlock(); }   // uvolnění zdroje
};
```

```
void send(Mutex& mtx, const uint8_t* data, size_t len) {
    ScopedLock guard(mtx);      // zamkne
    uart_write(data, len);      // kritická sekce
}                               // guard zaniká → automaticky odemkne, i při výjimce
```

Determinismus destruktoru je pro embedded klíčový: uvolnění mutexu, vypnutí periferie nebo návrat bufferu se stane přesně a včas, bez běhové pauzy garbage collectoru — přesně to, co systém reálného času potřebuje. RAII je zobecný nástroj: stejně jako mutex jím obalíme handle periferie, DMA přenos nebo vlastnictví paměti.

A.3 Unikátní vlastnictví a přesun

Nejjednodušší a v embedded nejžádanější režim je **unikátní vlastnictví**: zdroj má v každém okamžiku **právě jednoho vlastníka**. Vyjadřuje ho `unique_ptr` (nad haldou) nebo jakákoli RAII třída s jediným vlastníkem. Kopírování takového objektu nedává smysl (dva vlastníci téhož zdroje by se ho oba pokusily uvolnit), a proto je zakázané. Předat vlastnictví ale jde — pomocí **přesunu**. Přesun převede zdroj do nového vlastníka a starý nechá prázdný:

Přesunu
Move semantics, přesun: operace, která převede vlastnictví zdroje z jednoho objektu do druhého a původní nechá v prázdném stavu (bez zdroje). Nahrazuje drahou kopii u typů, které kopírovat nelze nebo nechceme; vyvolá se pomocí std::move.

```
std::unique_ptr<Buffer> a = make_buffer(256);   // a vlastní buffer
// std::unique_ptr<Buffer> b = a;               // CHYBA: kopie zakázána
std::unique_ptr<Buffer> b = std::move(a);       // přesun: b vlastní, a je nyní prázdný
```

Přesun je koncepčně tentýž **přenos vlastnictví**, jaký jsme viděli u fronty v RTOS (producent předá zprávu konzumentovi a sám ji už nedrží) i u ping-pong bufferu u DMA (vlastnictví půlky přejde z aplikace na DMA a zpět). Rozdíl je, že zde jej kontroluje překladač: po `std::move(a)` je použití `a` zjevná chyba, kterou nástroje odhalí. Unikátní vlastnictví má navíc **nulovou režii** — `unique_ptr` je jen ukazatel, žádný čítač, žádná zámková režie.

A.4 Výpůjčka bez vlastnictví

Ne každý, kdo se zdrojem pracuje, ho musí vlastnit. Funkce, která data jen přečte nebo zpracuje, si zdroj **vypůjčí** — dostane k němu dočasný přístup, ale zůstane cizí. V C++ výpůjčku vyjadřuje **reference**, **nevlastnící ukazatel** nebo — pro pole a buffery — `std::span`, které nese ukazatel i délku, ale data nevlastní:

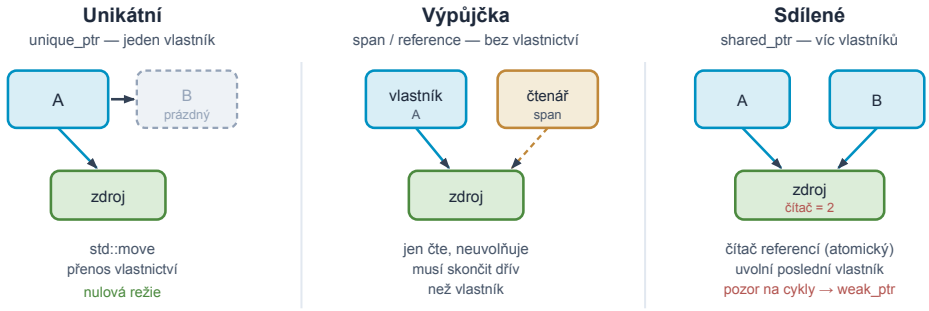
Vypůjčí
Borrow, výpůjčka: dočasný přístup ke zdroji bez převzetí vlastnictví. Vypůjčitel zdroj nesmí uvolnit a musí skončit dřív než vlastník. V C++ ji vyjadřuje reference, nevlastnící ukazatel nebo std::span.

```
uint32_t checksum(std::span<const uint8_t> data) {   // půjčí si, nevlastní
    uint32_t s = 0;
    for (auto b : data) s += b;                       // jen čte
    return s;                                          // nic neuvolňuje
}
```

Výpůjčka má dvě pravidla, na kterých stojí bezpečnost: vypůjčitel zdroj **nesmí uvolnit** a **musí skončit dřív než vlastník** (jinak by četl už uvolněnou paměť — dangling reference). V C++ jsou tato pravidla **konvencí** (kontrolují je jen nástroje a pozornost), zatímco v Rustu je vynucuje překladač (A.7). `std::span` je zároveň přesně ten nástroj, který jsme postrádali u DMA API v přednášce 5: bezpečně předá „ukazatel plus délku" bez toho, aby si někdo myslel, že buffer vlastní.

A.5 Sdílené vlastnictví

Někdy zdroj skutečně vlastní **víc stran najednou** a není předem jasné, kdo skončí poslední. Pro tento případ existuje `shared_ptr` — chytrý ukazatel s **čítačem referencí**: každá kopie čítač zvýší, každý zánik sníží, a při dosažení nuly se zdroj uvolní. Je to pohodlné, ale v embedded se používá **s rozmyslem**, protože nese cenu: čítač musí být **atomický** (kvůli souběhu z více úloh), což stojí výkon; přidává paměťovou režii; a uvolnění nastane v nepředvídatelný okamžik (u toho, kdo zrovna drží poslední referenci). Navíc dvě struktury, které se `shared_ptr` odkazují navzájem, vytvoří **cyklus**, který se nikdy neuvolní — to se řeší `weak_ptr` (slabý odkaz, který vlastnictví nedrží). Praktické pravidlo pro embedded: **default je unikátní vlastnictví a výpůjčka**; `shared_ptr` jen tam, kde je sdílení opravdu nutné a jeho cena únosná.



Obrázek 13.7. Tři režimy vlastnictví: unikátní (jeden vlastník, přesunem předá dál, nulová režie), výpůjčka (dočasný přístup bez vlastnictví, span nebo reference) a sdílené (víc vlastníků, čítač referencí, cena atomického čítače).

A.6 Pluggable alokátory: std::pmr

V přednášce 4 jsme viděli, proč se v embedded vyhýbáme obecnému `malloc`: nedeterministická doba, fragmentace, riziko selhání za běhu. Řešením byly **specializované alokátory** — pool s pevnými bloky, arena s posuvným ukazatelem, statická alokace. Moderní C++ umožňuje tyto strategie **zaměňovat, aniž se změní kód, který paměť používá**, díky knihovně **std::pmr** (polymorphic memory resources). Kontejner zde dostane alokátor jako **běhový objekt**, takže tentýž `pmr::vector` může jednou brát paměť z haldy a podruhé z předalokovaného bufferu na zásobníku:

```
std::array<std::byte, 4096> arena;                          // paměť předem, žádná halda
std::pmr::monotonic_buffer_resource pool{arena.data(), arena.size()};
std::pmr::vector<Sample> samples{&pool};                    // alokuje jen z arény
```

`monotonic_buffer_resource` je posuvná aréna (rychlá, uvolní se naráz), `unsynchronized_pool_resource` je pool pevných bloků. Do `std::pmr` tak přirozeně zapadá i obecná správa paměti z přednášky 4 — jen zabalená do standardizovaného rozhraní, které lze vyměnit bez zásahu do algoritmu.

Std::pmr
Polymorphic Memory Resources: část standardní knihovny C++, kde kontejnery dostávají alokátor jako běhový objekt (memory resource). Umožňuje vyměnit strategii přidělování paměti bez změny typu kontejneru, například nahradit haldu předalokovaným bufferem.

Borrow checkeru
Borrow checker: součást překladače Rustu, která staticky ověří pravidla vlastnictví a výpůjček. Zaručí, že v daném okamžiku existuje buď jedna zapisovatelná výpůjčka, nebo libovolně mnoho jen pro čtení, a že žádná výpůjčka nepřežije vlastníka. Chyby odhalí při překladu.

A.7 Rust jako kontrast: vlastnictví vynucené překladačem

C++ dává nástroje pro vlastnictví, ale jejich správné použití je stále **na disciplíně** programátora: nic v jazyce nezabrání použít přesunutý objekt nebo podržet výpůjčku déle než vlastníka. Jazyk **Rust** jde o krok dál a totéž **vynutí překladač** pomocí **borrow checkeru**. Pravidla jsou tři a jsou statická: každá hodnota má právě jednoho vlastníka; přiřazení hodnotu **přesune** (starý vlastník ji už nesmí použít); a výpůjčka je buď **jedna zapisovatelná** (`&mut`), nebo **libovolně mnoho jen pro čtení** (`&`), nikdy obojí zároveň — a žádná nepřežije vlastníka. Tím Rust při překladu **eliminuje celou třídu chyb** (use-after-free, dvojí uvolnění, datové závody), kterou v C i C++ hlídáme až nástroji a testy. Za to platí strmější křivkou učení a občas „bojem s překladačem". Pro embedded je Rust zajímavý právě proto, že tytéž garance dává **bez garbage collectoru a bez běhové režie** — stejná filozofie jako RAII, jen důsledně vynucená.

A.8 Cena C++ v embedded: co ano a co s rozmyslem

Rozšířený mýtus tvrdí, že C++ je pro malé mikrokontroléry „těžké". Pravda je jemnější: **velká část C++ je zdarma** (nula režie oproti ekvivalentnímu C), ale některé rysy cenu mají a v embedded se buď vypínají, nebo používají uvážlivě. **Zdarma** jsou třídy, RAII, `unique_ptr`, reference, `span`, šablony, `constexpr` (výpočet při překladu) — přeloží se do stejného kódu jako ručně psané C, jen bezpečněji. **Cenu mají** výjimky a RTTI (v hard real-time se běžně vypínají přepínačem `-fno-exceptions` a `-fno-rtti`, protože nesou nedeterminismus a kód navíc), dynamická alokace přes `new`/`shared_ptr` (viz A.5 a přednáška 4) a nekontrolované šablony (mohou nafouknout kód, tzv. code bloat). Praxe míří na profil **„freestanding C++"**: bez závislosti na plné standardní knihovně a OS, s deterministickou alokací a bez výjimek; místo dynamických kontejnerů se sahá po knihovnách jako ETL (Embedded Template Library) s pevnou kapacitou. C++ v embedded tak není „ano/ne", ale **výběr podmnožiny**, která dává bezpečnost RAII a typů bez ceny, které se chceme vyhnout.

A.9 Syntéza: uzavření ownership-spirály

Vlastnictví zdroje jsme v kurzu potkali čtyřikrát, pokaždé o úroveň formálněji. V **přednášce 3** šlo o ruční vlastnictví v C: opaque handle a spárované `create`/`destroy`, kde spárování hlídá programátor. V **přednášce 5** o vlastnictví DMA bufferu, které putuje mezi hardwarem, přerušením a aplikací a řeší se konvencí a časovou osou vlastnictví. V **přednášce 9** o vlastnictví zdroje mutexem a o přenosu vlastnictví dat zprávou ve frontě — s pravidly (jen vlastník odemyká, dědění priority). A v této příloze o vlastnictví **vynuceném typovým systémem**: RAII sváže život zdroje s životem objektu, přesun převádí unikátní vlastnictví, výpůjčka dává přístup bez vlastnictví, `shared_ptr` řeší skutečné sdílení a `std::pmr` mění strategii paměti; Rust totéž vynutí překladačem. Napříč všemi čtyřmi je to jediná otázka — **kdo právě vlastní ten zdroj a kdy se uvolní** — jen se posouvá od „hlídám to v hlavě" přes „hlídá to konvence a nástroje" k „hlídá to překladač". To je celá pointa: čím výš po spirále, tím míň příležitostí udělat chybu.

Příloha B — Základy strojového učení

> Tato příloha není samostatnou přednáškou, ale **přehledem základů strojového učení**, na které navazuje přednáška 12 (nasazení naučených modelů na MCU). Vlastní teorie učení — jak model vzniká — patří spíš do kurzu strojového učení než do kurzu aplikací mikroprocesorů; přednáška 12 proto předpokládá jen intuici a odkazuje sem. Cílem je, aby čtenář rozuměl tomu, **co** za model na MCU nasazuje a **jak vznikl**, než se v přednášce 12 zaměříme na jeho běh v rozpočtu paměti a energie.

Otázka zní: **jak z dat vznikne model, který umí klasifikovat — místo abychom rozhodovací pravidlo napsali ručně?** Začneme **obecnou klasifikační úlohou** a pak ji povedeme na jednom konkrétním příkladu, který se táhne celou přílohou: **detekci pohybu z akcelerometru**. Je dostatečně malá, abychom ji prošli od začátku do konce, a ukáže celou cestu od ručního pravidla (~60 % přesnost) k naučené síti (~95 %) o pouhých 17 parametrech — tedy modelu nasaditelnému na mikrokontrolér.

Půjdeme po vrstvách. Nejprve **klasifikační úloha** (B.1) a **příznaky** (B.2). Pak **perceptron** jako lineární klasifikátor (B.3) a **vícevrstvá síť** pro nelinearitu (B.4). Dále **jak se síť učí** — ztráta a gradient (B.5), **backpropagation** (B.6) a **optimalizátory** od SGD k Adamovi (B.7). Nakonec **trénink, validace a přeučení** (B.8) a **cesta od tréninku k nasazení** (B.9), která ústí do přednášky 12.

B.1 Klasifikační úloha

Začněme obecně. **Klasifikace** je úloha, kdy máme vstup popsaný **vektorem příznaků** `x = (x1, …, xn)` a chceme jej zařadit do jedné z **konečně mnoha tříd**. Tak vypadá překvapivě mnoho praktických úloh: obraz → který objekt, zvuk → které klíčové slovo, proud dat ze senzoru → jaká aktivita, měření → v normě, nebo anomálie. Klasifikátor je tedy funkce z prostoru příznaků do množiny tříd; zajímavé na ní je, že rozděluje prostor příznaků **rozhodovací hranicí** na oblasti jednotlivých tříd.

Klasifikace
Úloha, kdy se vstup popsaný vektorem příznaků zařazuje do jedné z konečně mnoha tříd. Klasifikátor je funkce z prostoru příznaků do množiny tříd; rozděluje prostor rozhodovací hranicí na oblasti jednotlivých tříd.

Jak takovou hranici získat? Klasické programování ji **napíše ručně**: „když je příznak větší než práh, je to třída A". To funguje pro jednoduché, dobře pochopené hranice, ale selhává, jakmile je hranice složitá nebo předem neznámá — což je u reálných dat ze senzorů, obrazu a zvuku pravidlo, ne výjimka. **Strojové učení** postup obrací: pravidlo se **nezapíše, ale naučí z dat**. Dostaneme sadu **označených příkladů** (vstupy se známou třídou) a algoritmus najde hranici, která je nejlépe odděluje. Tomu se říká **učení s učitelem** (supervised learning); existuje i **učení bez učitele** (hledání struktury v neoznačených datech), ale nás zajímá klasifikace se značkami. Klíčové přitom je, že cílem není trénovací příklady zapamatovat nazpaměť, ale **zobecnit** na nová, dosud neviděná data (k tomu se vrátíme v B.8).

Strojové učení
Přístup, kdy se rozhodovací pravidlo nezapíše ručně, ale odvodí (naučí) z dat na základě příkladů se známými odpověďmi. Model se nastavuje tak, aby na trénovacích datech dělal co nejmenší chybu, a přitom zobecnil na nová data.

Celou přílohou nás povede jeden konkrétní **dvoutřídní** příklad — **detekce pohybu z akcelerometru**. Zařízení s tříosým akcelerometrem má v reálném čase, s nízkou spotřebou a na malém jádře Cortex-M0+ rozpoznat dva stavy: *statický* (leží na stole) a *pohyb* (chůze, běh, třes). Je to kanonická klasifikační úloha (nositelná elektronika, sledování aktivity) a dostatečně malá na to, abychom ji prošli od začátku do konce.

B.2 Příznaky: co dáme modelu na vstup

Nejnaivnější klasifikátor pro náš příklad je jediný ruční práh na jedné ose zrychlení — a rovnou ukáže, proč potřebujeme lepší přístup: nakloněné zařízení má trvale nenulové zrychlení (falešný poplach), pomalý pohyb práh nepřekročí (přehlédnutí), rotace bez posunu vede k chybné klasifikaci, a přesnost se drží kolem 60 %. Problém není v modelu, ale ve **vstupu**: hranice mezi třídami není v syrovém zrychlení jednoduchá.

Modelu proto na vstup nedáváme surové vzorky, ale **příznaky** — veličinami, které z dat získají to podstatné. Návrh příznaků (feature engineering) často rozhoduje o úspěchu víc než volba modelu. Pro detekci pohybu je surové zrychlení špatný vstup (obsahuje gravitaci, náklon, šum); dobrým příznakem je **efektivní hodnota (RMS) v časovém okně**, která měří „energii" pohybu:

```
float rms(const float *buf, int n) {   // RMS po odečtení průměru
    float mean = 0, sum_sq = 0;
    for (int i = 0; i < n; i++) mean += buf[i];  mean /= n;
    for (int i = 0; i < n; i++) { float d = buf[i] - mean; sum_sq += d*d; }
    return sqrtf(sum_sq / n);
}
```

Příznaky
Feature: číselná veličina odvozená ze surových dat, která zachycuje podstatnou vlastnost pro danou úlohu. Dobré příznaky oddělí třídy lépe než surová data a jsou základem úspěchu klasifikátoru; jejich návrh se nazývá feature engineering.

Odečtení průměru odstraní stálou složku (gravitaci a náklon), takže zůstane jen **dynamika pohybu**. Ze dvou os dostaneme dvourozměrný **vektor příznaků** `[RMS_x, RMS_y]`, v němž se statické a pohybové příklady seskupí do odlišných oblastí. Už samotné prahování na varianci (příbuzné RMS) zvedne přesnost na ~75 %. Výpočet příznaků je přesně to, co umí číslicové zpracování signálu z přednášky 11 (RMS, spektrální energie, dominantní frekvence z FFT) — DSP tak dodává klasifikátoru vstup.

B.3 Perceptron: lineární klasifikátor

Nejjednodušší naučitelný klasifikátor je **perceptron**. Spočítá **vážený součet vstupů plus posun** a propustí ho nelineární **aktivační funkcí** — zde sigmoidě, která výsledek stlačí do intervalu (0, 1) a lze ho číst jako pravděpodobnost:

```
float sigmoid(float x) { return 1.0f / (1.0f + expf(-x)); }

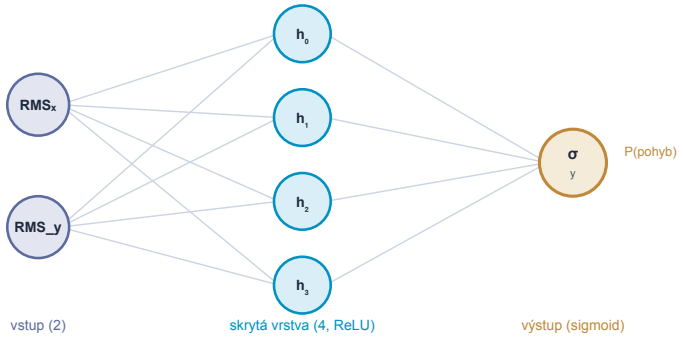
float perceptron(const float w[2], float b, float rms_x, float rms_y) {
    float z = w[0]*rms_x + w[1]*rms_y + b;   // vážený součet
    return sigmoid(z);                        // pravděpodobnost pohybu
}
```

Perceptron
Nejjednodušší neuron: spočítá vážený součet vstupů plus posun a propustí ho aktivační funkcí. Naučí se lineární rozhodovací hranici (přímku, rovinu), jeho parametry jsou váhy a posun.

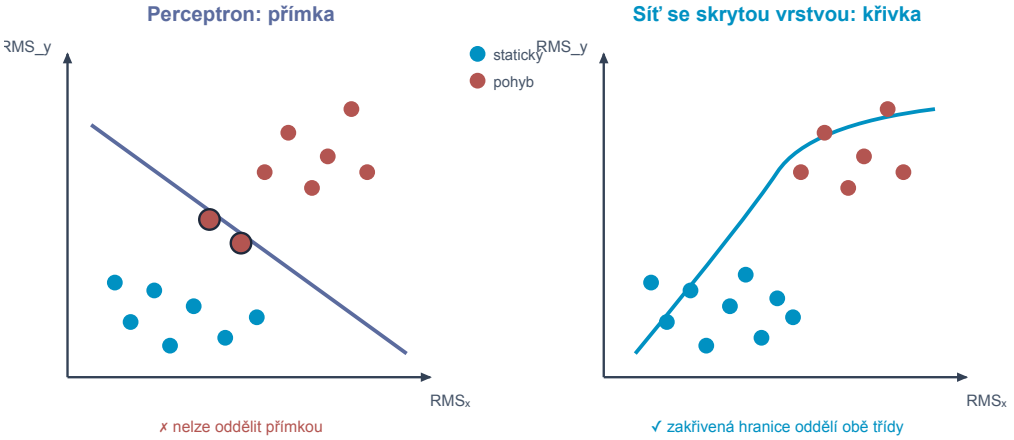
Aktivační funkcí
Nelineární funkce na výstupu neuronu. Sigmoida mapuje libovolné číslo do intervalu nula až jedna (pravděpodobnost), ReLU vrací kladnou část vstupu a nulu pro záporný. Bez aktivační funkce by síť byla jen lineární.

ReLU
Rectified Linear Unit: aktivační funkce, která vrací vstup, je-li kladný, a nulu jinak. Je levná (jen porovnání s nulou) a v hlubokých sítích se učí lépe než sigmoida; vytváří po částech lineární, celkově nelineární zobrazení.

Rozhodnutí padne prahováním výstupu na 0,5. Parametry jsou jen tři (`w0`, `w1`, `b`) a učení znamená najít jejich hodnoty z dat (B.5–B.7). Perceptron má ale zásadní mez: rovnice $w0 \cdot x + w1 \cdot y + b = 0$ je **přímka**, takže umí oddělit třídy jen tehdy, jsou-li **lineárně separovatelné**. Pomalý pohyb se v prostoru příznaků překrývá s klidem, takže se přesnost perceptronu zastaví kolem 82 %.



Obrázek 13.8. Dvouvrstvá neuronová síť pro detekci pohybu: dva vstupní příznaky (RMS ve dvou osách) vedou do skryté vrstvy čtyř neuronů s aktivací ReLU, jejíž výstupy sbíhají do jednoho výstupního neuronu se sigmoidou, který dá pravděpodobnost pohybu.



Obrázek 13.9. Prostor příznaků. Vlevo perceptron odděluje třídy přímkou a na nelineárně rozložených datech chybuje; vpravo dvouvrstvá síť vede zakřivenou rozhodovací hranici, která obě třídy oddělí správně.

B.4 Vícevrstvá síť a nelinearita

Aby hranice mohla být **křivá**, vloží se mezi vstup a výstup **skrytá vrstva** neuronů s nelineární aktivací (typicky ReLU). Skrytá vrstva **přemapuje prostor příznaků** do vyšší dimenze, ve které data už lineárně separovatelná jsou, a výstupní vrstva pak vede přímku v tomto přetvořeném prostoru:

Dopředný průchod (forward pass) je jen řetěz lineárních kombinací a aktivací:

```c
float nn_forward(const NeuralNetwork *nn, float rms_x, float rms_y) {
    float in[2] = {rms_x, rms_y}, h[4];
    for (int i = 0; i < 4; i++) {                // skrytá vrstva + ReLU
        float z = nn->b1[i];
        for (int j = 0; j < 2; j++) z += nn->W1[i][j] * in[j];
        h[i] = z > 0 ? z : 0;
    }
    float z = nn->b2[0];                          // výstupní vrstva + sigmoid
    for (int i = 0; i < 4; i++) z += nn->W2[0][i] * h[i];
    return sigmoid(z);
}
```

Se čtyřmi skrytými neurony (17 parametrů) vzroste přesnost na ~95 %. Zásadní je role aktivace: bez ní by složení dvou lineárních vrstev bylo zase jen lineární — teprve nelinearita dá síti schopnost vést zakřivenou hranici.

B.5 Jak se síť učí: ztráta a gradient

Učení znamená **nastavit parametry tak, aby síť dělala co nejmenší chybu**. Chybu měří ztrátová funkce — pro klasifikaci **křížová entropie**, která silně penalizuje sebejisté omyly. Ztráta je funkcí všech parametrů; učení je **hledání jejího minima**. Nástrojem je gradientní sestup: spočítá se **gradient** ztráty podle parametrů (směr, kterým ztráta nejrychleji roste) a parametry se posunou o malý krok **proti** němu. Velikost kroku je *rychlost učení* (learning rate) — malá učí pomalu, velká přeskočí minimum. Otázkou zůstává, jak gradient spočítat efektivně pro všechny parametry najednou; na to slouží backpropagation.

Ztrátová funkce
Loss: číslo, které měří, jak moc se předpověď modelu liší od správné odpovědi na trénovacích datech. Učení hledá parametry minimalizující průměrnou ztrátu; pro klasifikaci se používá křížová entropie.

Gradientní sestup
Gradient descent: iterační metoda hledání minima. Spočítá se gradient ztráty podle parametrů (směr nejrychlejšího růstu) a parametry se posunou o malý krok proti němu. Velikost kroku určuje rychlost učení (learning rate).

B.6 Backpropagation

Backpropagation je efektivní způsob, jak spočítat gradient ztráty podle **všech** parametrů jedním zpětným průchodem. Využívá **řetězové pravidlo** derivace: chyba se šíří od výstupu zpět vrstvami. Pro naši síť s křížovou entropií a sigmoidou vychází chyba výstupní vrstvy pozoruhodně jednoduše — rozdíl předpovědi a pravdy:

```
delta2 = y_pred - y_true                          // chyba výstupní vrstvy
delta1[j] = W2[j] * delta2 * (z1[j] > 0 ? 1 : 0) // chyba skryté vrstvy (přes ReLU')
```

Z chyb `delta` se pak gradienty parametrů dostanou jako součin chyby vrstvy a jejího vstupu (například gradient váhy `W2[j]` je `delta2 · h[j]`). Podstatné je pochopit princip: **jeden dopředný průchod spočítá výstup, jeden zpětný spočítá gradienty pro celou síť** — a to je celé jádro učení. Derivace ReLU je triviální (jedna pro kladný vstup, nula jinak), což je jeden z důvodů její obliby.

Backpropagation
Zpětné šíření chyby: efektivní výpočet gradientu ztráty podle všech parametrů sítě pomocí řetězového pravidla derivace. Chyba se propaguje od výstupu zpět vrstvami, takže jeden zpětný průchod dá gradienty pro celou síť.

B.7 Optimalizátory: od SGD k Adamovi

Nejjednodušší optimalizátor je **stochastický gradientní sestup (SGD)**: parametr se posune o `learning_rate · gradient`. V praxi ale naráží na dva problémy — gradienty **oscilují** (klikatá trajektorie) a mají **různé škály** (jeden learning rate nesedí všem parametrům). Standardním řešením je **Adam**, který kombinuje dvě myšlenky. **Setrvačnost** (první moment) je klouzavý průměr gradientů — uchovává informaci o směru a vyhlazuje oscilace. **Adaptivní rychlost** (druhý moment) je klouzavý průměr čtverců gradientů — parametry s velkými gradienty dostanou menší krok a naopak, takže každý parametr má vlastní tempo.

Adam
Adaptivní optimalizátor, který kombinuje setrvačnost (klouzavý průměr gradientů) s adaptivní rychlostí učení pro každý parametr (klouzavý průměr čtverců gradientů). Konverguje rychle a robustně a je výchozí volbou pro trénink neuronových sítí.

```c
void adam(float *param, float grad, AdamState *s, int t) {
    s->m = 0.9f*s->m   + 0.1f  *grad;         // 1. moment (setrvačnost)
    s->v = 0.999f*s->v + 0.001f*grad*grad;    // 2. moment (škála)
    float m_hat = s->m / (1 - powf(0.9f,  t));   // korekce vychýlení
    float v_hat = s->v / (1 - powf(0.999f, t));
    *param -= 0.001f * m_hat / (sqrtf(v_hat) + 1e-8f);
}
```

Korekce vychýlení (bias correction) napraví to, že momenty startují z nuly a zpočátku by byly příliš malé. Díky setrvačnosti a adaptivní rychlosti Adam konverguje rychle a spolehlivě, a je proto výchozí volbou; detaily vzorců jsou méně důležité než pochopení, **co** obě složky dělají.

B.8 Trénink, validace a přeučení

Trénink je smyčka: přes celý dataset (jedna **epocha**) se pro každý příklad udělá dopředný průchod, backpropagation a krok optimalizátoru; epoch je mnoho, dokud ztráta neklesne. Zásadní je oddělit **trénovací** a **validační** data. Model totiž může natolik přesně proložit trénovací data včetně šumu a na nových datech pak selhat — tomu se říká **přeučení** (overfitting). Pozná se tak, že chyba na trénovacích datech dál klesá, ale na validačních začne růst. Brání se mu **jednodušším modelem**, **více daty** a **regularizací** (penalizace velkých vah, předčasné zastavení). Kvalita se neměří jen přesností, ale i dalšími metrikami (přesnost a úplnost, matice záměn) — zvlášť u nevyvážených tříd, kde „vždy řekni statický" může mít vysokou přesnost, ale je bezcenné.

Přeučení
Overfitting, přeučení: stav, kdy se model naučí trénovací data i s jejich šumem a ztratí schopnost zobecnit na nová data. Pozná se tak, že chyba na trénovacích datech klesá, ale na validačních roste; brání se mu jednodušším modelem, více daty a regularizací.

B.9 Od tréninku k nasazení

Trénink probíhá **offline na PC** (v Pythonu, TensorFlow nebo PyTorch, s reálnými daty a výpočetním výkonem). Jeho jediným výstupem pro embedded je **sada naučených vah**, kterou vložíme do firmwaru jako konstantní pole — a na MCU pak běží už jen **dopředný průchod (inference)**:

```c
const NeuralNetwork model = {                 // vygenerováno po tréninku
    .W1 = {{1.82f,-0.45f},{-0.73f,2.15f},{1.24f,1.56f},{-1.12f,0.89f}},
    .b1 = {-0.32f,-0.18f,-0.85f,0.22f},
    .W2 = {{1.45f,1.78f,2.12f,-0.95f}}, .b2 = {-1.23f}
};
int detect_motion(float rms_x, float rms_y) {
    return nn_forward(&model, rms_x, rms_y) > 0.5f;   // jen inference
}
```

Tato síť má 17 parametrů, zabere 68 bajtů a inferuje za ~3 mikrosekundy — na MCU zdarma. To je klíčové poselství přílohy: **model se učí na PC, na MCU jen běží**. Reálné modely jsou ovšem větší (konvoluční sítě pro obraz a zvuk) a nevejdou se tak snadno; jak

je zmenšit (**kvantizace na int8**, navazující na pevnou řádovou čárku z přednášky 11), převést (TensorFlow Lite), spustit (TFLite Micro, CMSIS-NN) a vejít se s nimi do rozpočtu paměti a energie — to je téma **přednášky 12**, která na tuto přílohu přímo navazuje.

Příloha C — Návrh sériového protokolu a marshalling

> Tato příloha ukazuje, **jak nad sériovou sběrnicí postavit protokol** — sadu pravidel, která z holého datového proudu udělají spolehlivé, strukturované a bezpečně zpracovatelné zprávy. Projdeme vrstvení, robustní rámcování (COBS), kontrolu integrity (CRC), potvrzovací schémata, serializaci dat (od ručního TLV po Protocol Buffers a CBOR) a bezpečnou deserializaci, a nakonec **osvědčené protokoly z praxe** — s důrazem na **CSP** (CubeSat Space Protocol).

Sériová sběrnice přenese datový proud z jednoho konce na druhý, ale sama o sobě nedává datům žádnou strukturu ani záruku. Požaduje-li aplikace přijetí a potvrzení telemetrie, musí mezi ni a sběrnici vstoupit **protokol**: dohodnutá pravidla, kde zpráva začíná a končí, jak se pozná její poškození, komu patří, jak se potvrdí a jak se strukturovaná data zabalí tak, aby jim rozuměl i jinak postavený druhý konec. Návrh protokolu je inženýrský kompromis stejného druhu jako výběr sběrnice — mezi jednoduchostí, režií, spolehlivostí a rozšiřitelností. Půjdeme po vrstvách odspodu nahoru a uzavřeme návrhovým checklistem.



Obrázek 13.10. Anatomie rámce: hlavička (SOF, LEN, TYPE/ID), užitečná data a kontrolní pole CRC počítané přes hlavičku i data.

C.1 Vrstvy: od sběrnice k aplikaci

Osvědčený postoj k návrhu je **vrstvení**: každá vrstva řeší jeden problém a nabízí čistou službu té nad sebou, nezávisle na tom, jak je realizována vrstva pod ní. Zjednodušený model, který stačí pro vestavné systémy, má čtyři vrstvy:

Vrstva	Odpovědnost	Příklad
fyzická	přenos dat po vodiči	UART, RS-485, SPI
linková	rámce, integrita, adresace na lince	rámcování, CRC, ACK
síťová	doručení mezi uzly, směrování	adresa uzlu a portu, routing
aplikační	význam zpráv (příkazy, telemetrie)	typy zpráv, serializace dat

Vrstvení má konkrétní přínos: **výměna jedné vrstvy nerozbije ostatní**. Tentýž aplikační protokol poběží po UARTu i po rádiu, změní-li se jen fyzická vrstva; totéž rámcování poslouží více typům zpráv. Ne každý protokol potřebuje všechny vrstvy — protokol mezi dvěma čipy na desce vystačí s linkovou vrstvou, kdežto síť senzorů s adresací a směrováním využije i síťovou

vrstvu. Vrstvení je tentýž princip oddělení odpovědností jako členění kódu do modulů, jen aplikovaný na komunikaci.

C.2 Rámcování a synchronizace

Přijímač dostává **nepřetržitý datový proud** a musí v něm najít hranice zpráv. Bez rámcování by nerozeznal, kde jedna zpráva končí a druhá začíná, a jediný ztracený byte by narušil výklad všech následujících dat. Existují dva přístupy a jejich kombinace.

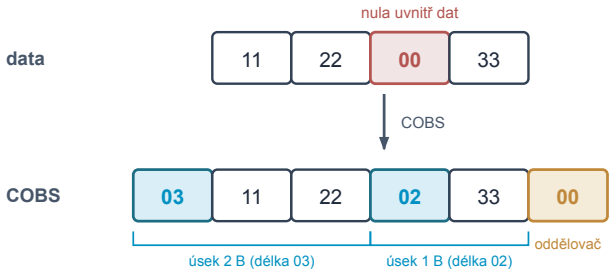
Délkový prefix. Hlavička nese počet bytů datové části; přijímač načte přesně tolik bytů. Je to kompaktní a rychlé, ale **křehké vůči ztrátě synchronizace**: přijme-li se o byte míň nebo víc, posune se výklad délky a všechny následující rámce jsou špatně, dokud se linka nezotaví. Proto se délkový prefix doplňuje úvodní značkou (SOF) a kontrolou CRC, aby přijímač poznal nesmysl a resynchronizoval se.

Oddělovací znak. Zprávu ohraničí vyhrazený byte (typicky `0x7E` nebo `0x00`). Výhodou je snadná **resynchronizace** — přijímač jen počká na další oddělovač a je opět synchronizován. Problémem je, že se oddělovač nesmí vyskytnout uvnitř dat. Řeší se to dvěma způsoby:

- **Byte stuffing (úniková sekvence).** Objeví-li se v datech oddělovač nebo únikový znak, předřadí se před něj únikový byte (jako `\` v řetězcích jazyka C). Jednoduché, ale režie je **proměnná** a v nejhorším případě zdvojnásobí délku (samé oddělovače v datech).
- **COBS (Consistent Overhead Byte Stuffing).** Elegantní kódování, které z rámce **odstraní všechny nulové byty** a nulu vyhradí výhradně jako oddělovač rámců. Data se rozdělí na úseky mezi nulami a před každý úsek se vloží jeden byte s délkou do další nuly. Režie je **pevná a malá** — jeden byte na každých 254 bytů dat, tedy nejhůř 0,4 %.

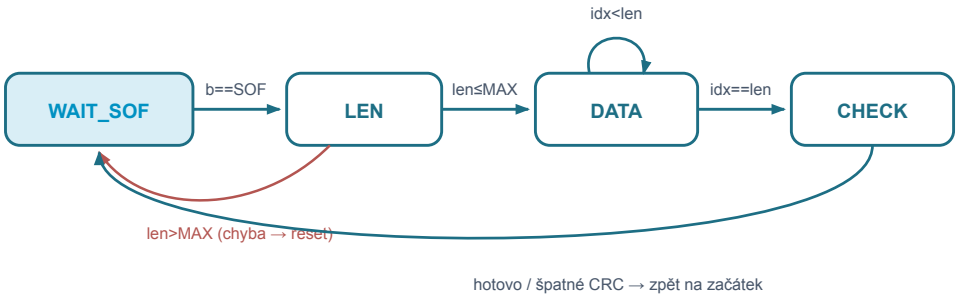
COBS je oblíbený právě proto, že spojuje spolehlivou resynchronizaci (nula je vždy hranice rámce) s **předvídatelnou** režií — což je výhodné pro vestavný systém, který si musí paměť rozpočítat dopředu.

COBS
Consistent Overhead Byte Stuffing: zakóduje rámec tak, že se v něm vůbec nevyskytne nulový byte; nula pak jednoznačně odděluje rámce. Přidá pevnou režii jeden byte na 254 dat, na rozdíl od proměnné (až dvojnásobné) režie klasického byte stuffingu.



Obrázek 13.11. COBS: kódování odstraní z rámce všechny nuly a vyhradí nulu jako oddělovač rámců. Každý úsek začíná bajtem s délkou do další nuly; režie je jeden bajt na 254 dat.

Ať se zvolí kterýkoli přístup, cílem je jediné: aby přijímač po jakékoli chybě dokázal **spolehlivě najít začátek dalšího rámce**, ne zůstat trvale mimo synchronizaci.



Obrázek 13.12. Rámcování konečným automatem: každý přijatý bajt posune automat mezi stavy; kontrola meze (len≤MAX) a návrat do WAIT_SOF po chybě činí parser odolným vůči ztrátě synchronizace.

C.3 Integrita: od checksumu ke CRC

Přenos může byte poškodit — rušení překlopí bit, most mezi sběrnicemi zahodí byte. Přijímač proto musí umět **poznat poškozený rámec** a zahodit ho. Slouží k tomu redundantní kontrolní pole spočítané z obsahu rámce.

Nejjednodušší je **checksum** — prostý součet bytů (nebo jejich XOR) modulo velikost pole. Je levný, ale slabý: dvě opačné chyby se vyruší, přehození bytů neodhalí a proti shluku chyb je téměř neúčinný.

Robustní odpovědí je CRC (cyklický redundantní součet). Data se chápou jako binární polynom a CRC je **zbytek po jejich dělení** pevně zvoleným generujícím polynomem. Tato konstrukce zaručí odhalení všech jednobitových chyb, všech shluků chyb kratších než kontrolní pole a naprosté většiny ostatních. Kvalitu kódu popisuje **Hammingova vzdálenost** — nejmenší počet bitů, které musí selhat, aby chyba prošla neodhalena; dobře zvolený CRC-16 či CRC-32 ji drží dostatečně vysokou pro embedded linky. Volba polynomu není libovolná: standardní protokoly používají prověřené polynomy (CRC-16-CCITT, CRC-32), u nichž je odhalovací schopnost dokázaná. Většina MCU navíc má **hardwarovou jednotku CRC**, takže výpočet nic nestojí.

CRC
Cyclic Redundancy Check: kontrolní pole počítané jako zbytek po polynomiálním dělení dat pevně zvoleným generujícím polynomem. Spolehlivě odhalí jednobitové chyby, jejich shluky do délky pole i většinu vícebitových chyb; charakterizuje ho Hammingova vzdálenost kódu.

CRC ovšem chybu jen **detekuje**, neopraví. Kde opakování není možné (jednosměrný spoj, kosmická linka s dlouhým zpožděním), nastupuje **dopředná korekce chyb** (FEC, forward error correction): přenese se tolik redundance, že přijímač chybu sám opraví. Cenou je vyšší režie, takže na krátké obousměrné linky se FEC obvykle nevyplatí a vystačí se s CRC a opakováním.

C.4 Spolehlivost: potvrzování a opakování

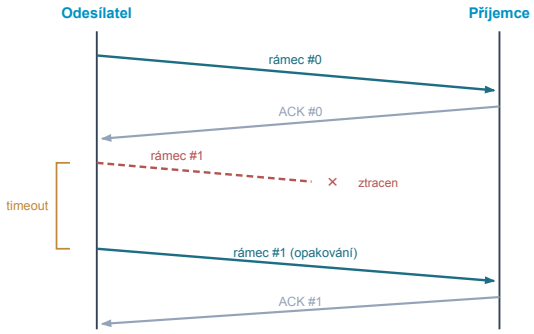
CRC indikuje, že rámec je vadný; **spolehlivé doručení** ale žádá víc — zaručit, že se každá zpráva dostane k cíli právě jednou, i když se cestou ztratí. Mechanismu opakování na základě potvrzení se říká ARQ (Automatic Repeat reQuest) a stojí na třech prvcích:

ARQ
Automatic Repeat reQuest: rodina metod spolehlivého přenosu, kde příjemce potvrzuje správně přijaté rámce a odesílatel nepotvrzené po vypršení časovače opakuje. Sekvenční čísla umožní rozpoznat ztrátu i duplikát.

- **Potvrzení (ACK/NACK).** Příjemce správný rámec potvrdí kladně, vadný odmítne záporně (nebo neodpoví). Odesílatel podle toho pokračuje, nebo opakuje.
- **Sekvenční čísla.** Každý rámec nese pořadové číslo. Odesílatel podle něj pozná **ztrátu** (chybí potvrzení) a příjemce **duplikát** (rámec přišel dvakrát, protože se ztratilo jen potvrzení). Bez sekvenčních čísel by opakování nepozorovaně zdvojovalo data.
- **Timeout a retransmise.** Nepřijde-li potvrzení do vypršení časovače, odesílatel rámec pošle znovu. Volba timeoutu je kompromis: krátký způsobuje zbytečná opakování,

dlouhý zdržuje zotavení.

Nejjednodušší schéma je **stop-and-wait**: odeslat rámec, počkat na potvrzení, poté odeslat další. Je robustní, ale pomalé, protože přenos je pozastaven do příchodu každého potvrzení. Výkonnější je **klouzavé okno** (sliding window): odesílatel smí mít současně nepotvrzeno více rámců a potvrzení přicházejí průběžně; tím se linka zaplní i při nenulovém zpoždění, za cenu složitějšího stavu na obou koncích.



Obrázek 13.13. Stop-and-wait s opakováním: odesílatel čeká na potvrzení; ztratí-li se rámec, po vypršení timeoutu jej pošle znovu. Sekvenční čísla rozliší ztrátu od duplikátu.

Se spolehlivostí souvisí i **idempotence** příkazů: opakovaný příkaz (kvůli ztracenému potvrzení) nesmí způsobit škodu. „Nastav výstup na 40 %" je idempotentní; „přičti 10 %" nikoli, a proto se takové příkazy buď převádějí na absolutní tvar, nebo chrání sekvenčním číslem. Otázky vlastnictví a souběhu, známé ze sdílených zdrojů, se tak vracejí na úrovni zpráv.

C.5 Marshalling: serializace dat

Marshalling
Serializace a deserializace: převod strukturovaných dat na plochou posloupnost bytů pro přenos a zpět. Musí být nezávislý na architektuře, překladači a jeho nastavení zarovnání, jinak si dva různé konce nebudou rozumět.

Datová část nese strukturovaná data, ale linka přenáší jen souvislý datový proud. Převod tam a zpět je marshalling a jeho jediné pevné pravidlo zní: **výsledek nesmí záviset na architektuře, překladači ani jeho nastavení**, protože druhý konec může být úplně jiný procesor.

Endianita. Vícebajtové číslo lze uložit od nejvýznamnějšího bytu (big-endian), nebo od nejméně významného (little-endian). Cortex-M je typicky little-endian, ale síťové protokoly se dohodly na big-endian (**network byte order**). Serializace proto pořadí bytů **explicitně** volí, nikdy nespoléhá na nativní.

Zarovnání a padding. Přímé odeslání C struktury přes `memcpy` je klasická past: překladač mezi členy vkládá **výplň** (padding) kvůli zarovnání a její velikost závisí na architektuře i překladači.

```
struct telemetry { uint8_t id; uint32_t value; };   // sizeof bývá 8, ne 5 (3 B padding)
```

Atribut `__attribute__((packed))` padding zruší, ale zavede **nezarovnané přístupy** k paměti, které jsou na některých jádrech pomalé nebo zakázané. Robustní řešení proto strukturu **neposílá přímo**, ale pole skládá do bufferu ručně, byte po bytu, v dohodnutém pořadí a šířce:

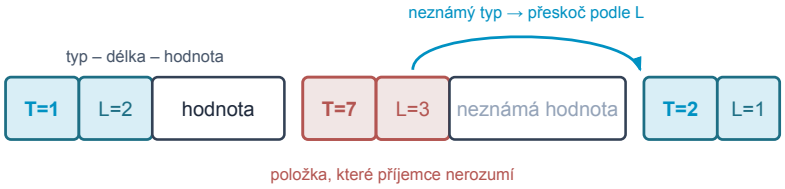
```
size_t pack_telemetry(uint8_t *buf, uint8_t id, uint32_t value) {
    buf[0] = id;
    buf[1] = value >> 24;  buf[2] = value >> 16;    // big-endian, explicitně
    buf[3] = value >> 8;   buf[4] = value;
    return 5;                                        // pevná, známá délka
```

```
}
```

Používají se přitom **typy s pevnou šířkou** (`uint16_t`, `int32_t`), ne `int`, jehož šířka není standardem daná.

TLV a rozšiřitelnost. Aby se protokol dal rozšiřovat, aniž se rozbijí starší uzly, zapisují se volitelné položky ve formátu TLV (Type-Length-Value). Příjemce, který daný *typ* nezná, položku podle *délky* přeskočí — nová verze protokolu tak přidá pole, aniž porušila starší implementaci. Je to tentýž princip dopředné kompatibility jako rezervovaná pole ve formátech instrukcí.

TLV
Type-Length-Value: každá položka nese svůj typ, délku a hodnotu. Příjemce neznámý typ podle délky přeskočí, takže do formátu lze přidávat nové položky zpětně kompatibilně.



Obrázek 13.14. TLV: každá položka nese typ, délku a hodnotu. Položku s neznámým typem příjemce podle délky přeskočí, takže lze formát rozšiřovat bez porušení starších implementací.

Serializační schémata. U složitějších zpráv se ruční skládání nevyplatí a používají se knihovny řízené **schématem** (popisem zpráv), z něhož se vygeneruje kód:

Formát	Charakteristika	Vhodné pro
Protocol Buffers / nanopb	schéma `.proto`, kompaktní, generuje C; nanopb je varianta pro MCU bez haldy	strukturovaná telemetrie, verzované API
CBOR	binární obdoba JSONu, bez schématu, self-describing	flexibilní zprávy, konfigurace
MessagePack	kompaktní self-describing binární formát	rychlá výměna dat

Schémata řeší verzování a validaci za nás; cenou je závislost na knihovně a o něco větší kód. Volba je opět kompromis: pevný ruční rámec pro nejmenší a nejrychlejší cestu, schéma tam, kde se zprávy vyvíjejí a je jich mnoho druhů.

C.6 Bezpečná deserializace

Rozbalení přijatého rámce je nejcitlivější místo celého protokolu, protože pracuje s **nedůvěryhodným vstupem** — data mohou přijít od kohokoli, být poškozená nebo záměrně zkonstruovaná k útoku. Většina zneužití vestavných zařízení začíná právě u parseru, který vstup přijal bez kontroly.

Nejnebezpečnější je **délkové pole**. Zkopíruje-li se podle přijatého `LEN` do bufferu bez kontroly, útočník přetečením přepíše sousední paměť:

```
// ZRANITELNÉ: LEN pochází z linky, buffer má pevných 64 bytů
memcpy(buf, payload, len);              // len > 64 → přetečení

// BEZPEČNÉ: nejdřív ověř mez, teprve pak kopíruj
if (len > sizeof buf) return ERR_LEN;   // nikdy nevěř délce ze vstupu
```

```
memcpy(buf, payload, len);
```

Zásady bezpečné deserializace:

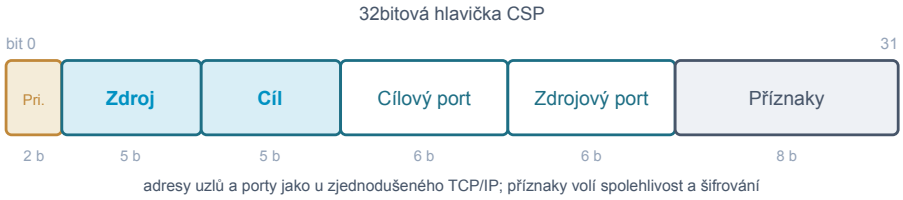
- **nikdy nedůvěřovat žádnému poli ze vstupu** — délku, počet položek i offsety je nutné ověřit proti skutečné velikosti bufferu **před** použitím,
- **ověřit CRC dřív, než se rámec interpretuje** — poškozený rámec se nesmí dostat k parseru,
- **kontrolovat každý index a mez** — chybějící kontrola je nejčastější zranitelnost,
- **odmítat nejednoznačné vstupy** — protokol by měl mít **kanonický tvar**; dovolí-li tentýž obsah zakódovat víc způsoby, otvírá to cestu k obcházení kontrol,
- **fuzzovat parser** — systematické posílání zdeformovaných rámců odhalí pády dřív než útočník.

Robustní protokol validuje strukturu **před** zpracováním a k datům přistupuje, teprve když je ověřeno, že jsou celá, neporušená a v mezích.

C.7 Robustní protokoly v praxi

Návrhové zásady nejlépe ukáží protokoly, které se osvědčily v provozu. Nemá smysl vymýšlet vlastní tam, kde existuje prověřený standard; stojí ale za to rozumět, **proč** jsou postavené právě takto.

CSP (CubeSat Space Protocol). Malý vrstvený protokol navržený pro **piko-a nanosatelity** (CubeSaty), kde je linka pomalá a nespolehlivá, výpočetní rozpočet malý a oprava na oběžné dráze nemožná. Přesto — nebo právě proto — je CSP plnohodnotně **vrstvený**: má 32bitovou hlavičku s adresami a **porty** zdroje a cíle (jako zjednodušené TCP/IP), umí **směrování** mezi uzly družice (rádio, CAN, I2C, UART) a nabízí jak nespolehlivé, tak spolehlivé doručení. Přitom se vejde do jednotek kilobajtů kódu a běží i na osmibitových jádrech. Poučení je dvojí: **vrstvení a malá stopa se nevylučují**, a protokol pro nejnáročnější prostředí (kosmos) staví na týchž principech, které jsme probrali — rámcování, CRC, adresace, porty, volitelná spolehlivost.



Obrázek 13.15. 32bitová hlavička CSP: priorita, adresy zdroje a cíle, cílový a zdrojový port a příznaky — adresace jako u zjednodušeného TCP/IP.

MAVLink. Protokol pro **bezpilotní prostředky** (drony). Klade důraz na kompaktní rámec, silnou kontrolu integrity a na **verzování**: do CRC se přimíchá „razítko" definice zprávy, takže dva konce s neshodnou verzí zprávy se rozpoznají a nezpracují nekonzistentní data. Ukazuje, jak vzít rozšiřitelnost a verzování vážně už v návrhu.

Modbus. Dlouhodobě zavedený průmyslový protokol, postavený na jednoduchém modelu **čtení a zápisu registrů** přes master-slave dotaz-odpověď. Jeho síla je **jednoduchost a rozšířenost**; poučení je, že minimalistický, dobře zdokumentovaný protokol přežije desetiletí, i když postrádá moderní vymoženosti.

SLIP a jeho následovníci. SLIP (Serial Line IP) je krajně jednoduché rámcování IP paketů po sériové lince oddělovačem a byte stuffingem; jeho slabinou je absence kontroly integrity. Moderní minimalistické protokoly (například **MIN**, Microcontroller Interconnect Network)

přidávají to, co SLIPu chybí — CRC, sekvenční čísla a rámcování odolné proti ztrátě synchronizace.

Společný jmenovatel je poučný: každý z těchto protokolů je **jiný bod v prostoru kompromisů** (jednoduchost proti schopnostem, režie proti robustnosti) — přesně jako sběrnice samotné.

C.8 Návrhový checklist

Návrh vlastního protokolu (nebo výběr existujícího) lze shrnout do posloupnosti rozhodnutí, která kopírují vrstvy této přílohy:

- **Rámcování.** Oddělovač s COBS (spolehlivá resynchronizace), nebo délkový prefix (kompaktní)? Je definovaná úvodní značka a cesta zpět do synchronizace po chybě?
- **Integrita.** CRC (jaký standardní polynom?), ne slabý checksum. Je hardwarová jednotka CRC využitá?
- **Adresace.** Bod-bod, nebo víc uzlů (adresa, port)? Je potřeba směrování?
- **Spolehlivost.** Stačí nespolehlivé doručení, nebo ARQ (ACK, sekvenční čísla, timeout)? Jsou příkazy idempotentní?
- **Marshalling.** Pevné pořadí bytů (network byte order), pevná šířka typů, žádné přímé posílání struktur. Ruční TLV, nebo schéma (Protocol Buffers/nanopb, CBOR)?
- **Rozšiřitelnost.** Verzování zprávy, rezervovaná pole, přeskočení neznámých položek (TLV).
- **Bezpečnost.** Validace délky a mezí před zpracováním, CRC před interpretací, kanonický tvar, fuzzing parseru.
- **Rozpočet.** Vejde se stav protokolu (buffery, okno, tabulky) do paměti? Je latence a režie úměrná lince a rozpočtu cyklů?

Návrh protokolu tak spojuje mnoho nití celého kurzu — konečný automat parseru, obsluhu událostí přerušením a DMA, timeouty a souběh, pevnou šířku dat i bezpečné zacházení s nedůvěryhodným vstupem. Spolehlivá komunikace není vlastnost jediného postupu, ale souhra vrstev — stejně jako vestavný systém sám.

Příloha D — Ladění a implementace PID regulátoru

> Tato příloha prohlubuje část 11.7: **jak PID regulátor správně diskretizovat, implementovat a naladit**, aby v provozu spolehlivě držel žádanou hodnotu. Projdeme fyzikální roli členů P, I a D, varianty diskretizace (poziční a přírůstkový tvar), **anti-windup** schémata, ošetření derivace a žádané hodnoty, implementaci v pevné řádové čárce a nakonec **metody ladění** — ruční, Ziegler–Nicholsovy a automatické ladění relé — spolu s praktickými rozšířeními.

Číslicový PID je jednoduchý, ale zrádný: napsat tři řádky s `Kp`, `Ki` a `Kd` je snadné, avšak regulátor, který se v provozu nerozkmitá, nepřekmitne po nasycení a nevyvolá skokový zásah při změně žádané hodnoty, vyžaduje několik ošetření, které základní tvar pomíjí. Stejně tak jeho **naladění** rozhoduje o tom, zda smyčka reaguje svižně a stabilně, nebo pomalu a s překmity. Půjdeme od chování jednotlivých členů přes robustní implementaci k systematickému ladění.

D.1 Co dělají členy P, I a D

Chování smyčky pochopíme nejlépe, když se členy přidávají postupně.

Jen P. Proporcionální regulátor je pružina: čím větší odchylka, tím větší zásah. Reaguje rychle, ale má zásadní vadu — aby udržel nenulový zásah (například proti trvalému zatížení motoru), potřebuje **nenulovou odchylku**. Zůstává proto **trvalá regulační odchylka** (offset). Zvyšování `Kp` ji zmenšuje, ale zároveň smyčku přibližuje k nestabilitě a překmitům.

Trvalá regulační odchylka
Ustálená nenulová odchylka od žádané hodnoty, kterou samotný proporcionální regulátor nedokáže odstranit, protože k udržení nenulového zásahu potřebuje nenulovou odchylku. Odstraní ji až integrační člen.

PI. Integrační člen sčítá odchylku v čase, takže i nepatrná trvalá odchylka postupně naroste v akumulátoru a zásah ji **postupně vynuluje**. Odstraní tak offset, ale vnáší **fázové zpoždění** a sklon k **překmitu** a kmitání — smyčka je pomalejší a náchylnější k nestabilitě.

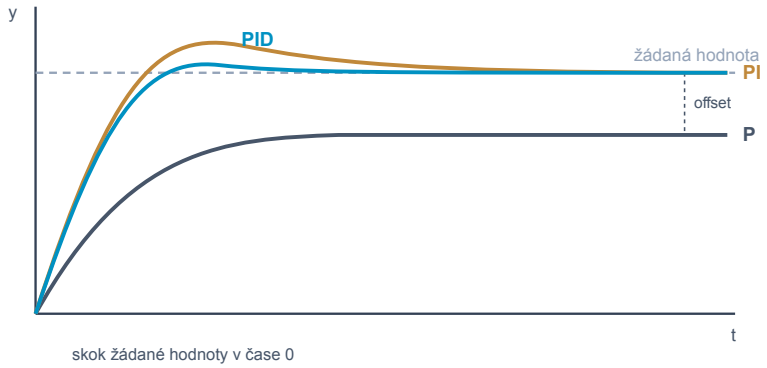
PID. Derivační člen reaguje na *rychlost změny* odchylky, tedy reaguje na její trend a působí **tlumivě** — umožní zvýšit `Kp` a `Ki` bez ztráty stability, čímž zrychlí odezvu a potlačí překmit. Daní je, že derivace **zesiluje šum** čidla, a proto vyžaduje filtraci (oddíl D.4).

D.2 Diskretizace: poziční a přírůstkový tvar

Spojité vztahy je nutné převést na výpočet v pevných okamžicích `Ts`. Integrál se aproximuje **součtem** (obdélníková, Eulerova metoda), derivace **zpětnou diferencí**:

$$\int e\,\mathrm{d}t \;\approx\; \sum e_k\,T_s, \qquad \frac{\mathrm{d}e}{\mathrm{d}t} \;\approx\; \frac{e_k - e_{k-1}}{T_s}.$$

Přesnější (a u pomalých vzorkování vhodnější) je **Tustinova** (bilineární) aproximace, která integrál nahradí lichoběžníkem; pro rychle vzorkované vestavné smyčky ale obdélníková metoda plně postačuje. Volba **vzorkovací periody** `Ts` je přitom zásadní: orientačním pravidlem je vzorkovat zhruba **desetkrát až dvacetkrát rychleji**, než je nejrychlejší dynamika



Obrázek 13.16. Odezva na skok žádané hodnoty: samotné P nechá trvalou odchylku, PI ji odstraní za cenu překmitu, PID překmit ztlumí a odezvu zrychlí.

procesu, a `Ts` držet **konstantní** (kolísání period vnáší do derivace i integrálu chybu — proto PID patří do přerušení časovače, ne do hlavní smyčky).

Existují dva tvary implementace. **Poziční** (polohový) tvar drží integrál jako stav a počítá zásah přímo (jako v části 11.7). **Přírůstkový** (rychlostní) tvar počítá jen *změnu* zásahu `Δu` a přičítá ji k předchozímu výstupu:

$$\Delta u_k = K_p(e_k - e_{k-1}) + K_i\,T_s\,e_k + \frac{K_d}{T_s}\,(e_k - 2e_{k-1} + e_{k-2}).$$

```
// přírůstkový (rychlostní) PID: počítá Δu, drží se jen tři poslední odchylky
float pid_incremental(float w, float y) {
    float e = w - y;
    float du = Kp*(e - e1) + Ki*Ts*e + (Kd/Ts)*(e - 2*e1 + e2);
    e2 = e1;  e1 = e;
    u += du;                        // nový zásah = starý + přírůstek
    return u;
}
```

Přírůstkový tvar má dvě praktické výhody: **nemá výslovný integrační akumulátor** (integrace je schovaná v `u += du`), takže se **přirozeně brání windupu** a hladce přechází mezi ručním a automatickým režimem (oddíl D.7); a hodí se pro akční členy, které samy integrují (krokový motor, ventil). Nevýhodou je citlivější chování při ztrátě kroku a horší přehled o absolutní hodnotě integrálu.

D.3 Anti-windup

Nejčastější příčinou špatného chování PID v praxi je **windup**. Když regulátor žádá víc, než akční člen dokáže dodat (PWM na 100 %, ventil naplno), zásah se **nasytí** a přestane růst — ale integrátor na to nereaguje a odchylku sčítá dál, takže integrační člen naroste do velmi vysokých hodnot. Jakmile se proces konečně přiblíží k cíli a odchylka změní znaménko, musí se nahromaděný integrál nejprve odbourat, což trvá dlouho a způsobí mohutný **překmit**. Řešením je **anti-windup**; používají se tři schémata:

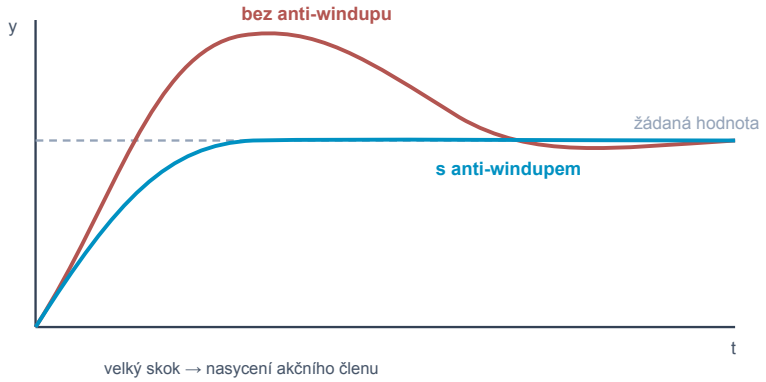
Anti-windup
Souhrn technik, které brání nahromadění integračního členu při nasycení akčního členu. Nejběžnější jsou omezení integrátoru (clamp), podmíněná integrace a zpětný přepočet (back-calculation), který rozdíl mezi žádaným a skutečným zásahem vrací zpět do integrátoru.

- **Omezení integrátoru (clamping).** Integrační akumulátor se tvrdě ořízne na rozsah, který odpovídá nenasycenému výstupu. Jednoduché a účinné.
- **Podmíněná integrace.** Integruje se jen tehdy, když výstup **není nasycený**, nebo když

by integrace nasycení **zmenšovala**. Zabrání dalšímu růstu ve špatném směru.

- **Zpětný přepočet (back-calculation).** Rozdíl mezi *žádaným* a *skutečně dodaným* (ořízntutým) zásahem se se zesílením `Kt` vrací do integrátoru a plynule ho stahuje zpět. Nejhladší, ale žádá další konstantu.

```
float u = Kp*e + Ki*integ + Kd*der;
float u_sat = clampf(u, U_MIN, U_MAX);          // mez akčního členu
integ += Ts * (e + Kt*(u_sat - u));             // back-calculation: stahuj při nasycení
return u_sat;
```



Obrázek 13.17. Bez anti-windupu integrátor po nasycení přeroste a smyčka mohutně překmitne; s anti-windupem se výstup vrátí k cíli hladce.

D.4 Derivace a žádaná hodnota

Derivační člen je nejcitlivější a v naivním tvaru způsobuje dva problémy.

Derivační ráz (derivative kick). Změní-li se žádaná hodnota **skokem**, odchylka `e = w − y` skočí také, a její derivace teoreticky vyletí do nekonečna — zásah vytvoří nesmyslný krátký ráz. Náprava je počítat derivaci z **měřené veličiny** `y` (která se skokem nemění), ne z odchylky:

$$D_k = -K_d \frac{y_k - y_{k-1}}{T_s}.$$

Zesílení šumu. Derivace zvýrazňuje vysoké frekvence, tedy i šum čidla. Proto se derivační člen **filtruje dolní propustí** prvního řádu; míru filtrace určuje koeficient (často značený `N`, typicky 8 až 20), který omezí zesílení derivace na vysokých kmitočtech. Filtrovaná derivace je kompromis mezi tlumivým účinkem a odolností proti šumu.

Obecnějším nástrojem je **váhování žádané hodnoty** (setpoint weighting): P a D člen se počítají s **menší vahou** žádané hodnoty (`b·w − y`, `c·w − y`), zatímco I člen z plné odchylky. Změny žádané hodnoty tak vyvolají hladší odezvu, aniž se zhorší potlačení poruch. Volba `b < 1` a `c = 0` (derivace jen z měření) je běžné výchozí nastavení.

Váhování žádané hodnoty
Setpoint weighting: proporcionální a derivační člen se počítají z odchylky s menší vahou žádané hodnoty (koeficienty b a c), zatímco integrační člen z plné odchylky. Změny žádané hodnoty pak vyvolají hladší odezvu bez rázů, aniž se zhorší potlačení poruch.

D.5 Implementace v pevné řádové čárce

Na jádře s FPU se PID pohodlně počítá ve `float`. Bez FPU nebo pro nejtvrdší determinismus se volí **pevná řádová čárka**, kde platí táž pravidla jako u filtrů:

- **Zesílení ve formátu Q.** `Kp`, `Ki`, `Kd` se uloží jako celá čísla v pevném měřítku (například Q16), aby se vešla malá i velká zesílení.

- **Široký akumulátor integrálu.** Integrační člen sčítá po dlouhou dobu, takže potřebuje `int32` **nebo** `int64`, jinak přeteče. Přetečení integrátoru je zákeřná chyba, která se projeví až po minutách provozu.
- **Saturace na každém kroku.** Výstup i integrátor se ořezávají saturací (ne přetečením otočením), což je zároveň součást anti-windupu.
- **Zaokrouhlování při posunu.** Po vynásobení a posunu zpět do užšího typu se přičítá polovina kroku, aby se nehromadil systematický posun (bias) — přesně jako u IIR filtru.

Vzorkovací frekvence bývá u regulace nižší než u audio filtrů (stovky Hz až jednotky kHz), takže rozpočet cyklů na jeden krok je pohodlný; kritická je spíš **stálost periody** než syrová rychlost.

D.6 Ladění regulátoru

Naladit PID znamená najít `Kp`, `Ki`, `Kd` tak, aby smyčka byla rychlá i stabilní. Existují ruční i systematické postupy.

Ruční ladění. Osvědčený postup: začneme jen s `P` a zvyšujeme `Kp`, dokud se odezva nezačne mírně kmitat; pak přidáme `I`, abychom odstranili trvalou odchylku (menší `Ti` = silnější integrace); nakonec přidáme `D` k potlačení překmitu. Je to rychlé a nevyžaduje model, ale závisí na zkušenosti.

Ziegler–Nicholsova metoda dává výchozí hodnoty z jednoduchého experimentu. V uzavřené smyčce se vypne I i D a `Kp` se zvyšuje, až smyčka **trvale kmitá**; zaznamená se **kritické zesílení** `Ku` a **perioda kmitů** `Tu`. Konstanty se pak odečtou z tabulky (standardní tvar s `Ti`, `Td`, kde `Ki = Kp/Ti` a `Kd = Kp·Td`):

Regulátor	`Kp`	`Ti`	`Td`
P	0,5·`Ku`	—	—
PI	0,45·`Ku`	`Tu`/1,2	—
PID	0,6·`Ku`	`Tu`/2	`Tu`/8

Existuje i varianta z **přechodové charakteristiky** (open-loop): na proces se pustí skok a z odezvy se odečte dopravní zpoždění `L` a časová konstanta `T`, z nichž tabulka opět dá konstanty. Ziegler–Nichols je rychlý odrazový můstek, ale je **agresivní** — vede zhruba na čtvrtinový překmit, který se pak ručně zjemní.

Automatické ladění relé je metoda vhodná přímo pro vestavný firmware. PID se dočasně nahradí **relé**, které přepíná zásah mezi dvěma hodnotami; smyčka se rozkmitá řízeným mezním cyklem a z jeho **amplitudy a periody** se spočítají `Ku` a `Tu` bez rizika, že se přetáhne do nestability. Zařízení lze tak naladit automaticky stiskem tlačítka.

Automatické ladění relé
Relay (Åström–Hägglund) auto-tuning: regulátor se dočasně nahradí relé, které přepíná zásah mezi dvěma hodnotami. Smyčka se rozkmitá s mezním cyklem, z jehož amplitudy a periody se přímo určí kritické zesílení a perioda — a z nich konstanty PID. Bezpečnější a automatizovatelné přímo v firmwaru.

D.7 Praktická rozšíření a checklist

Reálné regulátory doplňují několik dalších prvků:

- **Bumpless transfer.** Při přepnutí z ručního do automatického režimu (nebo po změně konstant) se integrátor přednastaví tak, aby zásah **neskočil**. Přírůstkový tvar to řeší přirozeně.
- **Dopředná vazba (feedforward).** Je-li porucha nebo požadovaná změna známá předem, přičte se přímo k zásahu, takže regulátor nezasahuje až po projevení odchylky, ale s předstihem — smyčka reaguje rychleji.

- **Kaskádní regulace.** Rychlá vnitřní smyčka (například proud motoru) uvnitř pomalé vnější (otáčky) zlepší potlačení poruch; vnější regulátor zadává žádanou hodnotu vnitřnímu.
- **Omezení rychlosti a pásmo necitlivosti.** Omezení rychlosti změny zásahu chrání akční člen; malé pásmo necitlivosti kolem cíle brání zbytečným drobným zákmitům zásahu kolem nuly.

Návrhový checklist číslicového PID:

- **Vzorkování.** Pevná perioda `Ts` z přerušení časovače, dost rychlá vůči procesu; PID nikdy v hlavní smyčce.
- **Diskretizace.** Poziční, nebo přírůstkový tvar (přírůstkový pro snadný anti-windup a bumpless).
- **Anti-windup.** Vždy — clamp, podmíněná integrace, nebo back-calculation.
- **Derivace.** Z měřené veličiny, filtrovaná; zvaž váhování žádané hodnoty.
- **Aritmetika.** `float` s FPU, nebo pevná čárka se širokým akumulátorem a saturací.
- **Ladění.** Ruční, Ziegler–Nichols jako výchozí, nebo relé pro automatiku; vždy ověřit na reálném procesu.
- **Robustnost.** Saturace zásahu, omezení rychlosti, bumpless přepínání, ošetření výpadku čidla.

Číslicový PID je tak učebnicová ukázka toho, že **rozdíl mezi „funguje na papíře" a „funguje v provozu" je v detailech** — v diskretizaci, anti-windupu, ošetření derivace a pečlivém naladění.

Příloha E — AI nástroje pro embedded vývoj

> Tato příloha není samostatnou přednáškou, ale **praktickým přehledem AI nástrojů, které se dnes dotýkají vývoje vestavného softwaru** — ve dvou rovinách. První je AI jako **nástroj vývojáře**: jazykové modely, které pomáhají psát, kontrolovat a ladit firmware. Druhá je AI jako **součást produktu**: řetězec nástrojů, které naučený model dostanou na mikrokontrolér (navazuje na přednášku 12 a Přílohu B). U obou rovin je společné jedno: nástroj urychlí práci, ale **odpovědnost za korektnost a bezpečnost zůstává na inženýrovi**. Tento kritický pohled je hlavní nití celé přílohy.

Vestavný vývoj má rysy, kvůli nimž se AI nástroje chovají jinak než ve webovém nebo aplikačním programování: kód přistupuje přímo k registrům konkrétního čipu, chyba se často projeví až na hardwaru, prostředky jsou omezené a část systémů je **bezpečnostně kritická**. Následující text proto u každé skupiny nástrojů ukazuje nejen *co umí*, ale i *kde je jejich hranice* a *co musí ověřit člověk*.

E.1 Dvě roviny AI v embedded vývoji

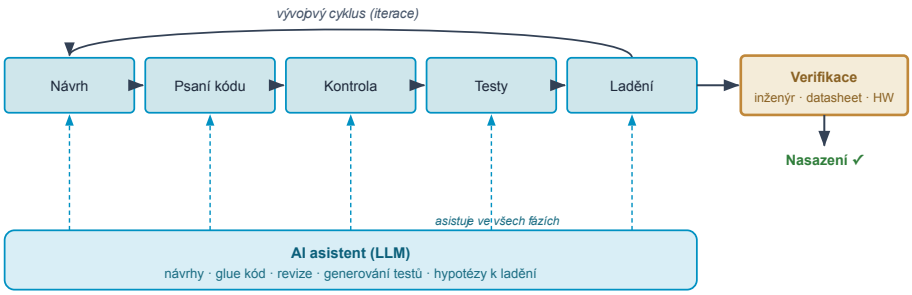
Slovo „AI" v kontextu embedded znamená dvě zcela odlišné věci a je užitečné je od začátku oddělit.

AI jako nástroj vývoje. Sem patří jazykové modely (LLM) integrované do editoru nebo terminálu. Běží na serveru nebo na výkonném počítači vývojáře, **nikdy na cílovém mikrokontroléru**, a jejich výstupem je zdrojový kód, vysvětlení nebo návrh konfigurace, který teprve člověk přeloží a ověří. Tato rovina zrychluje *psaní* firmwaru.

Jazykové modely
Large Language Model (LLM): model natrénovaný na velkém korpusu textu a kódu, který na základě promptu generuje pokračování. V embedded se používá jako asistent pro psaní, kontrolu a ladění kódu, nikoli jako součást běžícího firmwaru.

AI jako součást produktu. Sem patří **nasazení naučených modelů na MCU** — inference přímo na čipu (TinyML). Zde AI *není* nástroj, ale funkce výrobku: detekce klíčových slov, klasifikace vibrací, rozpoznání gesta. K tomu slouží samostatný řetězec nástrojů (oddíly E.7 a E.8), který model zkvantuje, převede a zabalí do knihovny, jež poběží v reálném čase na jádru nebo na NPU.

Obě roviny se v praxi potkávají — vývojář může použít LLM asistenta k napsání obslužného kódu kolem modelu, který na čip dostal přes TinyML toolchain. Přesto je nutné je metodicky rozlišovat: první je otázkou **produktivity**, druhá otázkou **architektury produktu**.



Obrázek 13.18. AI ve vývojovém cyklu: asistent pomáhá v každé fázi (návrh, psaní, kontrola, testy, ladění), ale brána verifikace a nasazení zůstává v rukou inženýra.

E.2 LLM asistenti pro psaní firmwaru

Nejviditelnější skupinou jsou **kódoví asistenti** integrovaní do vývojového prostředí: doplňování v editoru (GitHub Copilot, Codeium/Windsurf), konverzační asistenti v IDE (Cursor, Continue, JetBrains AI, Zed) a agentní nástroje v terminálu (Claude Code a podobné), které umějí číst repozitář, spouštět překlad a navrhovat úpravy. Modely lze provozovat i **lokálně** (přes `ollama` nebo `llama.cpp`), což je v embedded relevantní tam, kde firemní politika zakazuje posílat zdrojový kód na cizí server (oddíl E.6).

V čem embedded asistenti pomáhají spolehlivě:

- **rutinní glue kód** — obsluhy přerušení, inicializace periferií podle vzoru, přepisy mezi podobnými API (např. z jednoho HAL na jiný);
- **bitové manipulace registrů** — sestavení masky a posunů z popisu „nastav pole `PSC` na 15”, které je snadné napsat s překlepem;
- **překlad mezi jazyky a styly** — z Arduino skici na přímý přístup k registrům, z C na C++ s RAII (Příloha A);
- **vysvětlení cizího kódu** a generování komentářů či dokumentace;
- **skeleton testů** a testovací data (oddíl E.4).

Kde naopak selhávají nebo vytvářejí zavádějící výstup:

- **konkrétní čísla z datasheetu** — hodnoty registrů, adresy, čísla kanálů DMA a alternativních funkcí si model často **domýšlí** (halucinuje), protože přesná tabulka nebyla v tréninkových datech nebo ji model ve svých parametrech nezachytil. Tato čísla je nutné **vždy ověřit v datasheetu**;
- **časování a soubytnost** — model nemá informaci o reálném časování přerušení, o závodech nad sdílenou proměnnou ani o prioritní inverzi; navrhne kód, který se přeloží, ale za běhu selže;
- **novější nebo méně rozšířené čipy** — u okrajových rodin je tréninkových dat málo a kvalita prudce klesá.

Praktické pravidlo: **čím blíž je kód k hardwaru konkrétního čipu, tím pečlivěji se ověřuje.** Asistent je výborný na strukturu a syntaxi, nespolehlivý na fakta specifická pro součástku.

RAG
Retrieval-Augmented Generation: model odpovídá na dotaz nad konkrétními dokumenty (např. datasheetem), které se nejprve vyhledají a vloží do promptu. Odpověď se opírá o skutečný text, ne jen o naučené znalosti, takže je přesnější a lze ji dohledat ke zdroji.

E.3 Datasheety, registry a konfigurace

Zvláštní skupinou jsou nástroje, které pracují s **dokumentací součástky**. Datasheet mikrokontroléru mívá stovky až tisíce stran a najít v něm správné bitové pole je zdlouhavé. Zde pomáhá RAG (retrieval-augmented generation): dotaz se nezodpoví z paměti modelu, ale nad **vloženým textem datasheetu**, takže odpověď lze dohledat ke konkrétní stránce.

Typická použití:

- **dotazy nad datasheetem** — „na kterém pinu je alternativní funkce `TIM4_CH2`?" s odkazem na tabulku, odkud odpověď pochází;
- **generování registrové konfigurace** z popisu v přirozeném jazyce — z věty „ADC1, kanál 0, spouštěný TIM5, přenos přes DMA" navrhnout sekvenci zápisů do registrů;
- **grafické konfigurátory** (STM32CubeMX, MCUXpresso Config Tools) — samy o sobě nejsou AI, ale novější verze doplňují asistenty, kteří konfiguraci navrhnou a vysvětlí.

I zde platí ostražitost: RAG **snižuje** míru halucinací, protože odpověď se opírá o skutečný text, ale **neruší** ji — model může špatně přečíst tabulku, zaměnit revizi čipu nebo spojit údaje ze dvou různých periferií. Vygenerovanou konfiguraci je nutné porovnat s datasheetem a ideálně ji ověřit měřením na osciloskopu, přesně jako kdyby ji psal člověk.

E.4 Kontrola kódu, testy a statická analýza

AI asistenti se dobře uplatní i tam, kde vývojář obvykle šetří čas — u **kontroly a testů**.

Revize kódu. Model umí projít změnu a upozornit na typické chyby: chybějící `volatile` u proměnné sdílené s přerušením, přetečení při bitovém posunu, práci s ukazatelem po uvolnění, nekonzistentní návratové hodnoty. Nenahrazuje lidskou revizi ani **statické analyzátory** (cppcheck, Clang-Tidy, Coverity, Polyspace), ale doplňuje je: analyzátor hlásí porušení pravidel podle přesných vzorů, model spíše upozorní na *záměr*, který nedává smysl. Nejlepší je kombinace obou.

Generování testů. Asistent umí vytvořit kostru jednotkových testů (Unity, Ceedling, GoogleTest) i **testovací data**, včetně okrajových případů, na které člověk zapomíná (nula, přetečení, prázdný buffer). To dobře podpoří přístup **test-driven development** ze semináře 9. Pozor ale na to, že model občas napíše test, který jen **potvrzuje současné (možná chybné) chování** — test se musí číst stejně kriticky jako produkční kód.

Soulad s MISRA. V bezpečnostně kritickém kódu platí kodifikovaná pravidla MISRA C. Model umí na hrubá porušení upozornit a navrhnout opravu, ale **certifikovanou kontrolu MISRA provádí výhradně schválený analyzátor** — výstup jazykového modelu není v certifikačním procesu přípustný jako důkaz shody.

MISRA C
Soubor pravidel pro bezpečné psaní kódu v C v kritických aplikacích (automotive, letectví). Zakazuje nejednoznačné a riskantní konstrukce jazyka. Dodržení se ověřuje certifikovanými statickými analyzátory; AI může na porušení upozornit, ale certifikaci nenahrazuje.

E.5 Ladění: analýza logů, pádů a trace

Ladění vestavného systému znamená pracovat s daty, kterých je málo a jsou nízkoúrovňová — obsah registrů, hodnoty čítačů, výpisy po sériové lince. AI zde pomáhá hlavně s **interpretací**:

- **rozbor pádu** — z obsahu registrů `CFSR`, `HFSR` a `MMFAR` po HardFaultu určit pravděpodobnou příčinu (neplatný přístup do paměti, nezarovnaný přístup, dělení nulou) a nasměrovat k místu v kódu;
- **sumarizace logů** — z dlouhého výpisu vytáhnout anomálie, zopakované vzory nebo okamžik, kdy se chování změnilo;
- **výklad chybových stavů periferií** — proč `USART` hlásí overrun, proč `I2C` uvízl v `BUSY`.

Hranice je stejná jako jinde: model **navrhne hypotézu**, nikoli diagnózu. Rozhoduje reprodukce chyby, měření a krokování v debuggeru. AI zkrátí cestu k hypotéze, ale potvrdit ji musí data z reálného hardwaru.

E.6 Meze a rizika v bezpečnostně kritickém kódu

Tato část je pro embedded nejdůležitější, protože zde se AI nástroje střetávají s požadavky, které web ani desktop nemají.

Halucinace faktů. Model generuje *pravděpodobné*, ne *pravdivé*. Adresa registru, číslo kanálu DMA nebo mezní hodnota z datasheetu mohou být vymyšlené a přitom vypadat věrohodně. V kódu, který přímo pracuje s hardwarem, je proto **nutná verifikace proti primárnímu zdroji** (datasheet, referenční manuál, errata).

Bezpečnostně kritické normy. Systémy podle **funkční bezpečnosti** (IEC 61508, ISO 26262 v automotive, DO-178C v letectví) vyžadují **dohledatelnost** od požadavku ke kódu k testu a používání **kvalifikovaných nástrojů**. Výstup jazykového modelu je nedeterministický a neopakovatelný, takže ho v tomto řetězci nelze použít jako důkaz — smí sloužit nanejvýš jako pomůcka, jejíž výsledek projde standardní verifikací.

Funkční bezpečnosti
Požadavek, aby systém při poruše přešel do bezpečného stavu. Kodifikují ji normy IEC 61508 (obecně), ISO 26262 (automotive) a DO-178C (letectví). Vyžadují dohledatelnost od požadavku ke kódu, verifikaci a schválené nástroje — do tohoto řetězce nelze bez kvalifikace zařadit výstup jazykového modelu.

Determinismus a reprodukovatelnost. Tentýž prompt může dát podruhé jiný kód. Pro sestavení, které má být reprodukovatelné (a auditovatelné), to znamená, že se do repozitáře commituje **výsledný ověřený kód**, ne prompt — a že se generovaný kód posuzuje stejně přísně jako kód psaný ručně.

Duševní vlastnictví a důvěrnost. Posílání zdrojového kódu na cizí server může být v rozporu s NDA nebo firemní politikou; navíc hrozí, že asistent navrhne kód pod nekompatibilní licencí. Řešením je **lokální model** (oddíl E.2) nebo poskytovatel se smluvní zárukou, že data neopouštějí organizaci a nepoužijí se k tréninku.

Dodavatelský řetězec. U modelů nasazovaných na čip (další oddíly) přibývá otázka **původu modelu i dat**: podvržený model nebo otrávená trénovací data jsou bezpečnostní riziko (souvisí s přednáškou 13). Model je nutné brát jako **nedůvěryhodný vstup**, dokud není ověřen.

> **Shrnutí meze.** AI nástroj v embedded práci **zrychluje**, ale **nezaručuje správnost**. Každý jeho výstup — kód, konfigurace, diagnóza, model — je hypotéza, kterou potvrzuje až verifikace proti primárnímu zdroji a měření na hardwaru. V bezpečnostně kritickém řetězci nesmí AI výstup nahradit kvalifikovaný nástroj ani lidskou revizi.

E.7 Nasazení modelů na MCU: toolchainy

Druhá rovina — AI jako součást produktu — má vlastní ekosystém nástrojů, které naučený model dostanou na mikrokontrolér. Sdílejí tentýž řetěz (trénink → kvantizace → převod → integrace → profilace, oddíl E.8), liší se mírou automatizace a cílovým hardwarem.

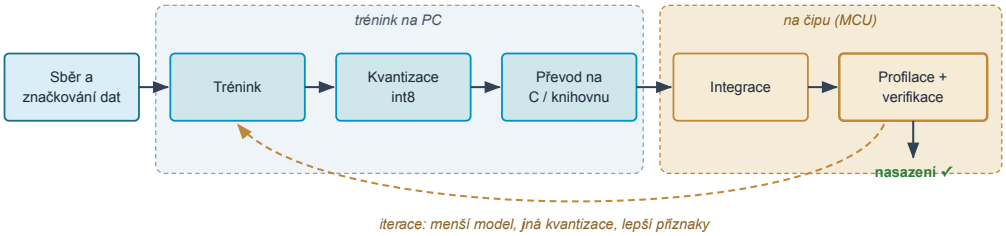
Nástroj	Vrstva	Poznámka
Edge Impulse	end-to-end platforma	sběr dat, trénink, kvantizace i export knihovny v jednom; nízký práh vstupu
NanoEdge AI Studio (ST)	end-to-end	hledá malé modely pro anomálie a klasifikaci přímo pro STM32
STM32Cube.AI (X-CUBE-AI)	převodník	z natrénovaného modelu (Keras, TFLite, ONNX) vygeneruje optimalizovaný C kód pro STM32
TensorFlow Lite for Microcontrollers	runtime	přenositelný interpret modelů `.tflite` bez OS a bez dynamické paměti
CMSIS-NN	knihovna jader	ručně optimalizovaná jádra vrstev pro Cortex-M (využívá SIMD); pod runtime
Ethos-U Vela	překladač pro NPU	přeloží model pro akcelerátor Arm Ethos-U (přednáška 8)
microTVM / Glow	překladače	z grafu modelu generují nízkoúrovňový kód napříč cíli
výrobcovské SDK (NXP eIQ, Renesas)	integrované	totéž pro rodiny mimo STM32

Volba se řídí **cílem a týmem**: pro rychlý prototyp nebo tým bez zkušeností s ML je vhodná end-to-end platforma (Edge Impulse, NanoEdge). Pro plnou kontrolu nad modelem a integrací se sáhne po kombinaci **TFLite Micro + CMSIS-NN** (nebo Cube.AI), kterou zmiňuje i přednáška 12. Na čipech s NPU je klíčový překladač pro daný akcelerátor (Ethos-U Vela).

E.8 Od dat k čipu: typický pipeline

Bez ohledu na zvolený nástroj má nasazení modelu na MCU stejnou kostru. Je to tentýž workflow jako v přednášce 12, zde ale z pohledu **nástrojů**, které jednotlivé kroky provádějí.

1. **Sběr a značkování dat** — nasbírat reprezentativní data ze *stejného* senzoru a v *stejných* podmínkách jako v cílové aplikaci; kvalita dat rozhoduje víc než volba modelu (Příloha B).
2. **Trénink na PC** — v obvyklém prostředí (TensorFlow/PyTorch); cílem je **malý** model, který se vejde do paměti a rozpočtu výpočtu.
3. **Kvantizace** — převod vah a aktivací na `int8` (přednáška 12); nástroj provede kalibraci na vzorku dat a změří ztrátu přesnosti.
4. **Převod na C / knihovnu** — Cube.AI, TFLite Micro nebo Vela vygenerují kód či data modelu a rozhraní pro volání inference.
5. **Integrace do firmwaru** — napojení na senzor přes DMA, alokace bufferů předem (bez `malloc` za běhu), zajištění determinismu (přednáška 9). Zde často pomůže LLM asistent z první roviny.
6. **Profilace a verifikace na cíli** — změřit skutečnou dobu inference (čítačem `DWT-`



Obrázek 13.19. Řetěz nasazení modelu na MCU: sběr a značkování dat, trénink na PC, kvantizace do int8, převod na C/knihovnu, integrace do firmwaru a profilace na cíli; zpětné šipky značí iteraci, brána vpravo je verifikace na reálném hardwaru.

`->CYCCNT`), spotřebu a přesnost **na reálném hardwaru**, ne jen na PC. Teprve to je důkaz, že model splňuje rozpočet.

Kroky se **iterují**: pokud model na čip nezapadne nebo je pomalý, vrací se k menší architektuře, agresivnější kvantizaci nebo lepším příznakům. Poslední krok — měření na cíli — je nepřekročitelná brána, stejně jako verifikace u AI asistentů v první rovině.

E.9 Doporučení a checklist

Jak AI nástroje v embedded používat s rozmyslem:

- **Asistenta ano na strukturu, ne na fakta.** Nechte model psát kostru, glue kód a testy; každé číslo z datasheetu, každou adresu a časování ověřte proti primárnímu zdroji.
- **Generovaný kód posuzujte jako cizí.** Prošel by vaší revizí a statickou analýzou? Pokud ne, nepatří do repozitáře.
- **V kritickém kódu jen jako pomůcka.** Certifikaci, MISRA a funkční bezpečnost zajišťují kvalifikované nástroje a lidská verifikace, ne jazykový model.
- **Chraňte zdroj a data.** Pro citlivý kód lokální model nebo poskytovatel se smluvní zárukou.
- **U modelů na čipu ověřte původ.** Model i trénovací data berte jako nedůvěryhodný vstup, dokud neprojdou kontrolou (přednáška 13).
- **Vždy měřte na cíli.** Ať jde o vygenerovaný ovladač, nebo o nasazený model, posledním slovem je chování na reálném hardwaru — osciloskop, čítač cyklů, měření spotřeby.

Checklist před nasazením kódu nebo modelu vzniklého s pomocí AI:

1. Jsou všechny hodnoty specifické pro čip ověřené v datasheetu / referenčním manuálu?
2. Prošel kód statickou analýzou a lidskou revizí stejně jako ručně psaný?
3. Existují testy včetně okrajových případů — a testují správné (ne jen současné) chování?
4. Je sestavení reprodukovatelné z commitnutého kódu, nezávisle na promptu?
5. Neopustil citlivý zdroj nebo data organizaci v rozporu s politikou?
6. U modelu na čipu: je ověřený jeho původ i původ dat, a změřená doba inference i přesnost na cíli?